```
// ==WindhawkMod==
// @id             windows-11-taskbar-styler-fork
// @name           Windows 11 Taskbar Styler - Fork
// @description    Customize the taskbar with themes contributed by others or create your own
// @version        1.10
// @author         m417z
// @github         https://github.com/m417z
// @twitter        https://twitter.com/m417z
// @homepage       https://m417z.com/
// @include        explorer.exe
// @architecture   x86-64
// @compilerOptions -lcomctl32 -lgdi32 -lole32 -loleaut32 -lruntimeobject -lshlwapi
// ==/WindhawkMod==

// Source code is published under The GNU General Public License v3.0.
//
// For bug reports and feature requests, please open an issue here:
// https://github.com/ramensoftware/windhawk-mods/issues
//
// For pull requests, development takes place here:
// https://github.com/m417z/my-windhawk-mods

// ==WindhawkModReadme==
/*
# Windows 11 Taskbar Styler

Customize the taskbar with themes contributed by others or create your own.

Also check out the **Windows 11 Start Menu Styler**, **Windows 11 Notification
Center Styler** mods.

## Themes

Themes are collections of styles. The following themes are integrated into the
mod and can be selected in the settings:

[![TranslucentTaskbar](https://raw.githubusercontent.com/ramensoftware/windows-11-
taskbar-styling-guide/main/Themes/TranslucentTaskbar/screenshot.png)
\
TranslucentTaskbar](https://github.com/ramensoftware/windows-11-taskbar-styling-
guide/blob/main/Themes/TranslucentTaskbar/README.md)

[![DockLike](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/main/Themes/DockLike/screenshot.png)
\
DockLike](https://github.com/ramensoftware/windows-11-taskbar-styling-
guide/blob/main/Themes/DockLike/README.md)

[![SimplyTransparent](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-
styling-guide/main/Themes/SimplyTransparent/screenshot.png)
\
SimplyTransparent](https://github.com/ramensoftware/windows-11-taskbar-styling-
guide/blob/main/Themes/SimplyTransparent/README.md)

[![Squircle](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/main/Themes/Squircle/screenshot.png)
\
```

Squircle](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/Squircle/README.md)

[![Matter](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/Matter/screenshot.png)
\
Matter](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/Matter/README.md)

[![WinXP](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/WinXP/screenshot-small.png)
\
WinXP](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/WinXP/README.md)

[![Bubbles](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/Bubbles/screenshot.png)
\
Bubbles](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/Bubbles/README.md)

[![RosePine](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/RosePine/screenshot.png)
\
RosePine](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/RosePine/README.md)

[![WinVista](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/WinVista/screenshot.png)
\
WinVista](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/WinVista/README.md)

[![CleanSlate](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/CleanSlate/screenshot.png)
\
CleanSlate](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/CleanSlate/README.md)

[![Lucent](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/Lucent/screenshot.png)
\
Lucent](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/Lucent/README.md)

[![SunValley](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/SunValley/screenshot.png)
\
SunValley](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/SunValley/README.md)

[![SunValley
(Legacy)](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/21996Taskbar/screenshot.png)
\
SunValley
(Legacy)](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/21996Taskbar/README.md)

[![BottomDensy](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/BottomDensy/screenshot.png)
\
BottomDensy](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/BottomDensy/README.md)

[![TaskbarXII](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/TaskbarXII/screenshot.png)
\
TaskbarXII](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/TaskbarXII/README.md)

[![xdark](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/xdark/screenshot.png)
\
xdark](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/xdark/README.md)

[![Windows7](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/Windows7/screenshot.png)
\
Windows7](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/Windows7/README.md)

[![Aeris](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/Aeris/screenshot.png)
\
Aeris](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/Aeris/README.md)

[![Plasma](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/Plasma/screenshot.png)
\
Plasma](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/Plasma/README.md)

[![WindowGlass](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/WindowGlass/screenshot.png)
\
WindowGlass](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/WindowGlass/README.md)

[![Surface](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/Surface/screenshot.png)
\
Surface](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/Surface/README.md)

[![Oversimplified&Accentuated](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/Oversimplified&Accentuated/screenshot.png)
\
Oversimplified&Accentuated](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/Oversimplified&Accentuated/README.md)

[![Luminosity](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/Luminosity/screenshot.png)
\
Luminosity](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/Luminosity/README.md)

[![LayerMicaUI](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/LayerMicaUI/screenshot.png)
\
LayerMicaUI](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/LayerMicaUI/README.md)

[![Fluid](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/Fluid/screenshot.png)
\

Fluid](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/Fluid/README.md)

[![TintedGlass](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/TintedGlass/screenshot.png)
\
TintedGlass](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/TintedGlass/README.md)

[![TaskbarToStatusbar](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/TaskbarToStatusbar/screenshot.png)
\
TaskbarToStatusbar](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/TaskbarToStatusbar/README.md)

[![UltraWideFriendly](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/UltraWideFriendly/screenshot.png)
\
UltraWideFriendly](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/UltraWideFriendly/README.md)

[![LiquidGlass2](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/LiquidGlass2/screenshot.png)
\
LiquidGlass2](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/LiquidGlass2/README.md)

[![LiquidGlass
(Legacy)](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/LiquidGlass/screenshot.png)
\
LiquidGlass
(Legacy)](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/LiquidGlass/README.md)

[![Borderless](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/Borderless/screenshot.png)
\
Borderless](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/Borderless/README.md)

[![Command
Center](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/Command%20Center/screenshot.png)
\
Command
Center](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/Command%20Center/README.md)

[![FluentGlass](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/FluentGlass/screenshot.png)
\
FluentGlass](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/FluentGlass/README.md)

[![OS26 Liquid
Glass](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/OS26%20Liquid%20Glass/screenshot.png)
\
OS26 Liquid
Glass](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/OS26%20Liquid%20Glass/README.md)

[![FrostyGlass](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/FrostyGlass/screenshot.png)
\
FrostyGlass](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/FrostyGlass/README.md)

[![FrostedAcrylic](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/FrostedAcrylic/screenshot.png)
\
FrostedAcrylic](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/FrostedAcrylic/README.md)

[![Pills](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/Pills/screenshot.png)
\
Pills](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/Pills/README.md)

[![Blob](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/Blob/screenshot.png)
\
Blob](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/Blob/README.md)

[![Minecraft Hotbar](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/Minecraft%20Hotbar/screenshot.png)
\
Minecraft Hotbar](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/Minecraft%20Hotbar/README.md)

[![One UI 8.5](https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/main/Themes/One%20UI%208.5/screenshot.png)
\
One UI 8.5](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/Themes/One%20UI%208.5/README.md)

More themes can be found in the **Themes** section of [The Windows 11 taskbar styling guide](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/README.md#themes).
Contributions of new themes are welcome!

Advanced styling

Aside from themes, the settings have two sections: control styles and resource variables. Control styles allow to override styles, such as size and color, for the target elements. Resource variables allow to override predefined variables. For a more detailed explanation and examples, refer to the sections below.

The taskbar's XAML resources can help find out which elements and resource variables can be customized. To the best of my knowledge, there are no public tools that are able to decode the resource files of recent Windows versions, but here are XAML resources which were obtained via other means for your convenience: [TaskbarResources.xbf and SystemTrayResources.xbf](https://gist.github.com/m417z/ad0ab39351aca905f1d186b1f1c3d8c7).

The [UWPSpy](https://ramensoftware.com/uwpspy) tool can be used to inspect the taskbar's control elements in real time, and experiment with various styles.

For a collection of commonly requested taskbar styling customizations, check out [The Windows 11 taskbar styling guide](https://github.com/ramensoftware/windows-11-taskbar-styling-guide/blob/main/README.md).

Control styles

Each entry has a target control and a list of styles.

The target control is written as `Class` or `Class#Name`, i.e. the target control class name (the tag name in XAML resource files), such as `Taskbar.TaskListButton` or `Rectangle`, optionally followed by `#` and the target control's name (`x:Name` attribute in XAML resource files). The target control can also include:
* Child control index, for example: `Class#Name[2]` will only match the relevant control that's also the second child among all of its parent's child controls.
* Control properties, for example: `Class#Name[Property1=Value1][Property2=Value2]`.
* Parent controls, separated by `>`, for example: `ParentClass#ParentName > Class#Name`.
* `*` between two `>` separators to match any number of intermediate parent controls, for example: `ParentClass > * > Class#Name` matches `Class#Name` when `ParentClass` is any of its ancestors (with zero or more controls in between). `*` must be in the middle of the target (not the leftmost or rightmost part), and consecutive `*` are not allowed.
* `:root` as the leftmost target part to require that the next part has no parent, i.e., it's a root element. For example: `:root > Class#Name` matches `Class#Name` only when it has no parent control. `:root` must be followed by a non-`*` target part.
* Visual state group name, for example: `Class#Name@VisualStateGroupName`. It can be specified for the target control or for a parent control, but can be specified only once per target. The visual state group can be used in styles as specified below.

Several target controls can be specified for the same styles by separating them with commas, for example: `ParentClass > Class#Name1, ParentClass > Class#Name2`. Commas inside `[...]` are part of the property value and don't separate targets, for example: `Class[Margin=0,0,0,1]` is a single target.

Note: The target is evaluated only once. If, for example, the index or the properties of a control change, the target conditions aren't evaluated again.

Each style is written as `Style=Value`, for example: `Height=5`. The `:=` syntax can be used to use XAML syntax, for example: `Fill:=<SolidColorBrush Color="Red"/>`. Specifying an empty value with the XAML syntax will clear the property value, for example: `Fill:=`. In addition, a visual state can be specified as following: `Style@VisualState=Value`, in which case the style will only apply when the visual state group specified in the target matches the specified visual state.

For the XAML syntax, in addition to the built-in taskbar objects, the mod provides a built-in blur brush via the `WindhawkBlur` object, which supports the

`BlurAmount`, `TintColor`, `TintOpacity`, `TintLuminosityOpacity`, `TintSaturation`, `NoiseOpacity`, `NoiseDensity`, and `FallbackColor` properties. For example: `Fill:=<WindhawkBlur BlurAmount="10" TintColor="#80FF00FF"/>`. Theme resources are also supported for `TintColor` and `FallbackColor`, for example: `Fill:=<WindhawkBlur BlurAmount="18" TintColor="{ThemeResource SystemAccentColorDark1}" TintOpacity="0.5"/>`. The `FallbackColor` is used in place of the blur effect when battery saver is on or when transparency effects are disabled in the system settings.

Targets and styles starting with two slashes (`//`) are ignored. This can be useful for temporarily disabling a target or style.

A couple of practical examples:

Task list button corner radius

* Target: `Taskbar.TaskListButton`
* Style: `CornerRadius=0`

Running indicator size and color

* Target: `Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Rectangle#RunningIndicator`
* Styles:
  * `Fill=#FFED7014`
  * `Height=2`
  * `Width=12`
  * `Fill@ActiveRunningIndicator=Red`
  * `Width@ActiveRunningIndicator=20`

Task list button background gradient

* Targets:
  * `Taskbar.TaskListButtonPanel > Border#BackgroundElement`
  * `Taskbar.TaskListLabeledButtonPanel > Border#BackgroundElement`
* Style: `Background:=<LinearGradientBrush StartPoint="0.5,0" EndPoint="0.5,1"><GradientStop Offset="0" Color="DodgerBlue"/><GradientStop Offset="1" Color="Yellow"/></LinearGradientBrush>`

Hide the start button

* Target: `Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton]`
* Style: `Visibility=Collapsed`

Hide the network notification icon

* Target: `SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter >

ItemsPresenter > StackPanel > ContentPresenter[1] > SystemTray.IconView > Grid > Grid`
* Style: `Visibility=Collapsed`

Note: To hide the volume notification icon instead, use `[2]` instead of `[1]`.

Style variables

Beyond literal values, XAML values, and style constants, styles can reference live property values via *style variables*. A capture rule of the form `Property=>VarName` observes a control's property and publishes its value as `VarName`; other styles then substitute it with `{{VarName}}`. Whenever the captured value changes or the variable becomes undefined, every dependent style is recomputed and reapplied.

Capture rules cannot be combined with `:=` or with the per-rule `@VisualState` qualifier.

For example, these two styles on the same target keep it square, with the height tracking the width:

```
ActualWidth=>width1
Height={{width1}}
```

A capture rule may match several controls at once. A style reading `{{VarName}}` gets the value from whichever capturing control is closest to it in the control tree, i.e. the one it shares the deepest common parent with; between equally close controls, the one that appeared last wins. The variable only becomes undefined once the last capturing control is gone.

A substitution can appear anywhere in a value, including alongside literal text:

```
Margin=0,{{x1}},0,{{x2 + 10}}
```

Inside `{{ ... }}`, the supported expression syntax is:

* Numbers (e.g. `42`, `3.14`).
* Backtick-delimited string literals (e.g. `` `Auto` ``, `` `*` ``), where a doubled backtick encodes one literal backtick. Backtick is used rather than a quote so literals don't clash with YAML or XAML quoting.
* Variable references (a previously captured `VarName`).
* Arithmetic `+`, `-`, `*`, `/` with standard precedence, and unary `+`, `-`.
* Comparisons `<`, `<=`, `==`, `>=`, `>`, `!=`, evaluating to `1` or `0`. The relational operators require numbers; `==` and `!=` compare two numbers or two strings by value and treat a number-versus-string mismatch as unequal.
* The conditional `cond ? a : b`: `a` when `cond` is non-zero, otherwise `b`. The condition must be numeric, but each branch may be a number or a string, e.g. `` {{width > 0 ? `*` : `Auto`}} `` selects a `GridLength` keyword.
* `min(a, b)` and `max(a, b)`.
* `skip()`: leaves the style unapplied, so the property keeps (or returns to)

its original value, e.g. `{{width > 0 ? width : skip()}}` applies only once `width` is positive.
* Parentheses for grouping, and nesting such as `{{min(a, b + 1) * 2}}`.

Arithmetic, the unary sign, the relational comparisons, and `min` / `max` require numeric operands. A string can only be produced by a literal or a string-typed variable, compared with `==` / `!=`, and selected by the conditional.

Brace pairs match innermost-first, so `{{{x}}}` is parsed as a literal `{`, the substitution `{{x}}`, and a literal `}`, producing `{<value-of-x>}`.

A bare substitution `{{VarName}}` (with no operators) inserts the variable's captured value verbatim. This works only for primitive captured types (numeric, boolean, string); other types (brushes, thicknesses, etc.) are unsupported, and substituting one skips the style, as does a bare reference to an undefined variable.

Inside an expression, an undefined variable instead evaluates to the empty string, letting a style supply its own default via the conditional, e.g. ``` {{width == `` ? 80 : width}} ``` yields `80` until `width` is captured. The numeric operators above then fail on such a variable, skipping the style rather than treating it as `0`.

Resource variables

Some variables, such as size and padding for various controls, colors, and brushes, are defined as resource variables. You can override existing resources or define new theme-aware resources.

Overriding existing resources

Use `key=value` to override an existing resource, for example:

* `TaskbarContextMenuMargin=4`: The margin between the taskbar and the start button context menu.

* `ContextMenuMargin=8`: The margin between the taskbar and the tray area context menu.

* `MediumTaskbarButtonExtent=48`: The width of the taskbar buttons.

Defining theme-aware resources

Use `Key@Dark=value` and `Key@Light=value` to define new resources with different values for dark and light themes. These can then be referenced in styles using `{ThemeResource key}`.

For example, to define a custom accent color that automatically adjusts based on the system theme:

```
AutoAccent@Dark={ThemeResource SystemAccentColorDark1}
AutoAccent@Light={ThemeResource SystemAccentColorLight2}
```

Then use it in a style:

```
Background:=<SolidColorBrush Color="{ThemeResource AutoAccent}" />
```

The value will automatically update when the system accent color changes.

Using XAML syntax

The `:=` syntax can be used to set a XAML value as a resource, for example: `MyBrush:=<SolidColorBrush Color="Red"/>`. This can be combined with theme variants: `MyBrush@Dark:=<SolidColorBrush Color="#FF202020"/>`. Specifying an empty value with the XAML syntax will clear the resource value, for example: `MyBrush:=`.

Style constants

Style constants allow defining a value once and referencing it in multiple styles. Each entry contains a name and value, separated by `=`, for example:

```
mainColor=#fafad2
```

The constant can then be used in style definitions by prepending `$`, for example:

```
Fill=$mainColor
Background:=<AcrylicBrush TintColor="$mainColor" TintOpacity="0.3" />
```

Some themes use style constants to allow easy customization. Refer to the theme page for details on which constants are available.

XAML diagnostics consumer handling

This mod uses XAML diagnostics to inspect and customize the taskbar. However, there can only be one XAML diagnostics consumer at a time. If another program (such as ExplorerBlurMica or TranslucentTB) tries to use XAML diagnostics while this mod is running, there will be a conflict.

The **XAML diagnostics consumer handling** setting controls how this mod handles such conflicts:

* **Alert**: When another program tries to use XAML diagnostics, a message box will appear asking whether to block it. This allows you to decide on a case-by-case basis.
* **Block**: Automatically block other programs from using XAML diagnostics. This ensures this mod works correctly, but might break the other program.
* **Allow**: Allow other programs to use XAML diagnostics. This might break this mod's styling, but allows the other program to work.

Implementation notes

The VisualTreeWatcher implementation is based on the [ExplorerTAP](https://github.com/TranslucentTB/TranslucentTB/tree/develop/ExplorerTAP) code from the **TranslucentTB** project.

The `WindhawkBlur` brush object implementation is based on [XamlBlurBrush](https://github.com/TranslucentTB/TranslucentTB/blob/release/ExplorerTAP/XamlBlurBrush.cpp) from the **TranslucentTB** project.
*/
// ==/WindhawkModReadme==

// ==WindhawkModSettings==
/*
- theme: ""
  $name: Theme
  $description: >-
    Themes are collections of styles. For details about the themes below, or for
    information about submitting your own theme, refer to the relevant section
    in the mod details.
  $options:
  - "": None
  - TranslucentTaskbar: TranslucentTaskbar
  - DockLike: DockLike
  - SimplyTransparent: SimplyTransparent
  - Squircle: Squircle
  - Matter: Matter
  - WinXP: WinXP
  - WinXP_variant_Zune: WinXP (Zune)
  - Bubbles: Bubbles
  - RosePine: RosePine
  - WinVista: WinVista
  - CleanSlate: CleanSlate
  - Lucent: Lucent (Accented Bar)
  - Lucent_variant_Light: Lucent (Light Bar)
  - SunValley: SunValley
  - 21996Taskbar: SunValley (Legacy)
  - BottomDensy: BottomDensy
  - BottomDensy_variant_NoInd: BottomDensy (NoInd)
  - TaskbarXII: TaskbarXII
  - xdark: xdark
  - Windows7: Windows7
  - Aeris: Aeris
  - Plasma: Plasma
  - WindowGlass: WindowGlass
  - WindowGlass_variant_Split: WindowGlass (Split)
  - WindowGlass_variant_FullLength: WindowGlass (Full-Length)
  - Surface: Surface
  - Oversimplified&Accentuated: Oversimplified&Accentuated
  - Luminosity_variant_Dock: Luminosity (Dock)
  - Luminosity_variant_Classic: Luminosity (Classic)
  - Luminosity_variant_Compact: Luminosity (Compact)
  - LayerMicaUI: LayerMicaUI
  - Fluid: Fluid

```
  - TintedGlass: TintedGlass
  - TaskbarToStatusbar: TaskbarToStatusbar
  - UltraWideFriendly: UltraWideFriendly
  - LiquidGlass2: LiquidGlass
  - LiquidGlass: LiquidGlass (Legacy)
  - LiquidGlass_variant_Alternate: LiquidGlass (Legacy - Alternate)
  - Borderless: Borderless
  - Command_Center: Command Center
  - FluentGlass: FluentGlass
  - OS26_Liquid_Glass_variant_ClearMacDock: OS26 Liquid Glass (Clear MacDock)
  - OS26_Liquid_Glass_variant_ClearMacDockCompact: >-
      OS26 Liquid Glass (Clear MacDock Compact)
  - OS26_Liquid_Glass_variant_ClearTaskbar: OS26 Liquid Glass (Clear Taskbar)
  - OS26_Liquid_Glass_variant_DarkMacDock: OS26 Liquid Glass (Dark MacDock)
  - OS26_Liquid_Glass_variant_DarkMacDockCompact: >-
      OS26 Liquid Glass (Dark MacDock Compact)
  - OS26_Liquid_Glass_variant_DarkTaskbar: OS26 Liquid Glass (Dark Taskbar)
  - FrostyGlass: FrostyGlass
  - FrostedAcrylic: FrostedAcrylic
  - Pills: Pills
  - Blob: Blob
  - Minecraft_Hotbar: Minecraft Hotbar
  - One_UI_8_5_variant_Dock: One UI 8.5 (Dock)
  - One_UI_8_5_variant_Taskbar: One UI 8.5 (Taskbar)
- styleConstants: [""]
  $name: Style constants
  $description: >-
    Some themes support style constants for customization, such as colors. Refer
    to the theme page for available constants. For technical details, refer to
    the mod description.
- controlStyles:
  - - target: ""
      $name: Target
    - styles: [""]
      $name: Styles
  $name: Control styles
- themeResourceVariables: [""]
  $name: Resource variables
  $description: >-
    Use "Key=Value" to override an existing resource with a new value.

    Use "Key@Dark=Value" or "Key@Light=Value" to define theme-aware resources
    that can be referenced with {ThemeResource Key} in styles.

    The ":=" syntax can be used to set a XAML value. For details, refer to the
    mod description.
- clickThroughTaskbar: false
  $name: Click-through taskbar
  $description: >-
    Make the empty parts of the taskbar click-through. Only the area with the
    taskbar buttons and the system tray stays visible and clickable. Mainly
    useful for dock themes such as DockLike.
- xamlDiagnosticsHandling: alert
  $name: XAML diagnostics consumer handling
  $description: >-
```

```
    How to handle other programs that try to use XAML diagnostics. There can
    only be one consumer at a time. Block will prevent other programs from using
    it, which might break them. Allow will let them use it, which might break
    this mod.
  $options:
  - alert: Alert (prompt before blocking)
  - block: Block other consumers
  - allow: Allow other consumers
*/
// ==/WindhawkModSettings==

#include <commctrl.h>
#include <xamlom.h>

#include <atomic>
#include <vector>

#undef GetCurrentTime

#include <winrt/Windows.UI.Xaml.h>

struct ThemeTargetStyles {
    PCWSTR target;
    std::vector<PCWSTR> styles;
};

struct Theme {
    std::vector<ThemeTargetStyles> targetStyles;
    std::vector<PCWSTR> styleConstants;
    std::vector<PCWSTR> themeResourceVariables;
};

// clang-format off

const Theme g_themeTranslucentTaskbar = {{
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground > Grid > Rectangle#BackgroundFill", {
        L"Fill:=$CommonBgBrush"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid > Rectangle#BackgroundFill", {
        L"Fill:=$CommonBgBrush"}},
    ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"MenuFlyoutPresenter > Border", {
        L"Background:=$CommonBgBrush",
        L"BorderThickness=0,0,0,0",
        L"CornerRadius=14",
        L"Padding=3,4,3,4"}},
    ThemeTargetStyles{L"Border#OverflowFlyoutBackgroundBorder", {
        L"Background:=$CommonBgBrush",
        L"BorderThickness=0,0,0,0",
        L"CornerRadius=15",
        L"Margin=-2,-2,-2,-2"}},
    ThemeTargetStyles{L"Grid#ConfirmatorMainGrid", {
        L"Background:=$CommonBgBrush",
        L"BorderThickness=0"}},
```

```
ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputSwitcher > ContentControl > ContentPresenter > Grid", {
    L"Background:=<WindhawkBlur BlurAmount=\"25\" TintColor=\"#25323232\"/>",
    L"BorderThickness=0"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputSwitcher > ContentControl > ContentPresenter > Grid > Grid", {
    L"Background:=<SolidColorBrush Color=\"Transparent\"/>"}},
}, {
  L"CommonBgBrush=<WindhawkBlur BlurAmount=\"18\" TintColor=\"#25323232\"/>",
}};

const Theme g_themeDockLike = {{
  ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid@DockingStates", {
    L"Tag=horizontal",
    L"Tag@DockedLeft=vertical",
    L"Tag@DockedRight=vertical",
    L"Tag=>taskbarDock"}},
  ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
    L"Width={{taskbarDock==`vertical`?skip():`Auto`}}",
    L"HorizontalAlignment=Center",
    L"Margin=250,0,250,0"}},
  ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
    L"Background:=<AcrylicBrush TintColor=\"{ThemeResource SystemChromeAltHighColor}\" TintOpacity=\"0.8\" FallbackColor=\"{ThemeResource SystemChromeLowColor}\" />",
    L"Padding=6,0,6,0",
    L"CornerRadius=8,8,0,0",
    L"BorderBrush:=<SolidColorBrush Color=\"{ThemeResource SurfaceStrokeColorDefault}\" />"}},
  ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground > Grid > Rectangle#BackgroundFill", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
    L"Margin=0"}},
  ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
    L"Background:=<AcrylicBrush TintColor=\"{ThemeResource SystemChromeAltHighColor}\" TintOpacity=\"0.8\" FallbackColor=\"{ThemeResource SystemChromeLowColor}\" />",
    L"Margin=-4,-8,-4,-8",
    L"CornerRadius=10",
    L"BorderThickness=12,12,12,12",
    L"BackgroundSizing=InnerBorderEdge"}},
  ThemeTargetStyles{L"SystemTray.ChevronIconView", {
    L"Padding=0"}},
  ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon", {
    L"Padding=0"}},
  ThemeTargetStyles{L"SystemTray.OmniButton", {
    L"Padding=0"}},
  ThemeTargetStyles{L"SystemTray.CopilotIcon", {
    L"Padding=0"}},
  ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid", {
    L"Padding=4,0,4,0"}},
  ThemeTargetStyles{L"SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid", {
```

```
        L"Padding=0"}},
    ThemeTargetStyles{L"SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel >
ContentPresenter > SystemTray.IconView#SystemTrayIcon", {
        L"Padding=0"}},
    ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack", {
        L"Margin=0,-4,-12,-4"}},
    ThemeTargetStyles{L"Taskbar.Gripper#GripperControl", {
        L"Width=Auto",
        L"MinWidth=24"}},
}};

const Theme g_themeSimplyTransparent = {{
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground >
Grid > Rectangle#BackgroundFill", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
        L"Fill=Transparent"}},
}};

const Theme g_themeSquircle = {{
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground >
Grid > Rectangle#BackgroundFill", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid >
Rectangle#BackgroundFill", {
        L"Fill=#CC222222"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel@CommonStates > Grid >
Border#BackgroundElement, Taskbar.TaskListButtonPanel@CommonStates >
Border#BackgroundElement,
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel@CommonStates >
Grid > Border#BackgroundElement,
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel@CommonStates >
Border#BackgroundElement", {
        L"CornerRadius=5",
        L"Background:=<AcrylicBrush TintColor=\"Black\" TintOpacity=\"0.8\"
FallbackColor=\"#BB222222\" />",
        L"Background@InactivePointerOver:=<AcrylicBrush TintColor=\"Black\"
TintOpacity=\"0.8\" FallbackColor=\"#CC222222\" />",
        L"Background@ActivePointerOver:=<AcrylicBrush TintColor=\"Black\" TintOpacity=\"0.9\"
FallbackColor=\"#CC222222\" />",
        L"Background@ActiveNormal:=<AcrylicBrush TintColor=\"Black\" TintOpacity=\"0.8\"
FallbackColor=\"#CC222222\" />",
        L"Background@InactiveNormal:=<AcrylicBrush TintColor=\"Black\" TintOpacity=\"0.7\"
FallbackColor=\"#BB222222\" />",
        L"Background@InactivePressed:=<AcrylicBrush TintColor=\"Black\" TintOpacity=\"0.8\"
FallbackColor=\"#CC222222\" />",
        L"Background@ActivePressed:=<AcrylicBrush TintColor=\"Black\" TintOpacity=\"0.8\"
FallbackColor=\"#CC222222\" />"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"Background:=<AcrylicBrush TintColor=\"Black\" TintOpacity=\"0.8\"
FallbackColor=\"#BB222222\" />",
        L"CornerRadius=5",
        L"Margin=0,5,14,5",
        L"Padding=10,0,0,0"}},
    ThemeTargetStyles{L"Rectangle#RunningIndicator", {
        L"Fill=Transparent",
        L"RadiusX=5",
        L"RadiusY=5",
        L"Height=38",
        L"Width=40"}},
    ThemeTargetStyles{L"Grid#IconPanel > TextBlock#LabelControl,
Taskbar.TaskListLabeledButtonPanel > TextBlock#LabelControl", {
        L"Margin=4,0,0,0",
```

```
        L"Foreground=White"}},
    ThemeTargetStyles{L"Taskbar.SearchBoxButton", {
        L"Foreground=White",
        L"Margin=-11,0,0,0"}},
    ThemeTargetStyles{L"TextBlock#SearchBoxTextBlock", {
        L"FontSize=12",
        L"Foreground=White"}},
    ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"Grid", {
        L"RequestedTheme=2"}},

ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton[AutomationProperties.Name=Copilot] > Grid#IconPanel > Border#BackgroundElement,
Taskbar.TaskListButton#TaskListButton[AutomationProperties.Name=Copilot] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border#BackgroundElement", {
        L"Background:=<AcrylicBrush TintColor=\"Red\" TintOpacity=\"0.8\" />"}},
    ThemeTargetStyles{L"Border#BackgroundBorder", {
        L"Margin=0,3,0,3",
        L"CornerRadius=5"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid > Border#BackgroundElement@CommonStates,
Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Border#BackgroundElement@CommonStates", {
        L"Background@InactivePointerOver:=<AcrylicBrush TintColor=\"Black\" TintOpacity=\"0\" />",
        L"Background:=<AcrylicBrush TintColor=\"Black\" TintOpacity=\"0.8\" FallbackColor=\"#BB222222\" />"}},
    ThemeTargetStyles{L"Border#MultiWindowElement", {
        L"Background:=<AcrylicBrush TintColor=\"Black\" TintOpacity=\"0.8\" FallbackColor=\"#CC222222\" />"}},
    ThemeTargetStyles{L"TextBlock#TimeInnerTextBlock", {
        L"Foreground=White"}},
    ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
        L"Foreground=White"}},
    ThemeTargetStyles{L"SystemTray.TextIconContent > Grid > SystemTray.AdaptiveTextBlock#Base > TextBlock", {
        L"Foreground=White"}},
    ThemeTargetStyles{L"Border#BackgroundElement", {
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton", {
        L"Margin=-11,0,0,0"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.Name=Task View]", {
        L"Margin=-12,0,0,0"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Border,
Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Border", {
        L"Background@ActiveRunningIndicator:=<AcrylicBrush TintOpacity=\"0.8\" TintColor=\"Black\" />",
        L"Background@InactiveRunningIndicator:=<AcrylicBrush TintOpacity=\"0.8\" TintColor=\"Black\" />",
        L"Background@InactiveRunningIndicatorPointerOver:=<AcrylicBrush TintOpacity=\"0.8\" TintColor=\"Black\" />"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border#BackgroundElement,
Taskbar.TaskListLabeledButtonPanel@CommonStates > Border#BackgroundElement", {
        L"Background@InactivePointerOver:=<AcrylicBrush TintOpacity=\"0.8\" TintColor=\"Black\" FallbackColor=\"#DD222222\"/>",
        L"Background@ActivePointerOver:=<AcrylicBrush TintOpacity=\"0.8\" TintColor=\"Black\" FallbackColor=\"#EE222222\"/>",
        L"Background@InactiveNormal:=<AcrylicBrush TintOpacity=\"0.2\" TintColor=\"Black\" FallbackColor=\"#BB222222\"/>",
```

```
        L"Background@ActiveNormal:=<AcrylicBrush TintOpacity=\"0.8\" TintColor=\"Black\"
FallbackColor=\"#CC222222\" />",
        L"Background@ActivePressed:=<AcrylicBrush TintOpacity=\"0.8\" TintColor=\"#333333\"
FallbackColor=\"#BB333333\" />",
        L"Background@InactivePressed:=<AcrylicBrush TintOpacity=\"0.8\" TintColor=\"#333333\"
FallbackColor=\"#BB333333\" />",
        L"CornerRadius=5",
        L"Margin=1"}},
};

const Theme g_themeSquircle_variant_WeatherOnTheRight = {{
    ThemeTargetStyles{L"Rectangle#BackgroundFill", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Border#BackgroundElement,
Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Border#BackgroundElement", {
        L"Background@NoRunningIndicator:=<AcrylicBrush TintOpacity=\"0.2\"
TintColor=\"#222222\" FallbackColor=\"#DD222222\" />",
        L"Background:=<AcrylicBrush TintOpacity=\"0.8\" TintColor=\"Black\"
FallbackColor=\"#DD222222\" />",
        L"Margin=0",
        L"BorderThickness@NoRunningIndicator=1",
        L"Background@InactiveRunningIndicator:=<AcrylicBrush TintOpacity=\"0.7\"
TintColor=\"Black\" FallbackColor=\"#DD222222\" />",
        L"Background@ActiveRunningIndicator:=<AcrylicBrush TintOpacity=\"1\"
TintColor=\"Black\" FallbackColor=\"#DD222222\" />"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"Background:=<AcrylicBrush TintOpacity=\"0.8\" TintColor=\"Black\"
FallbackColor=\"#BB222222\" />",
        L"Margin=0,5,18,5",
        L"Padding=10,0,0,0",
        L"CornerRadius=5"}},
    ThemeTargetStyles{L"Taskbar.SearchBoxButton", {
        L"Margin=0,-2,0,0"}},
    ThemeTargetStyles{L"Border#MultiWindowElement", {
        L"Height=0"}},
    ThemeTargetStyles{L"Grid#OverflowRootGrid > Border", {
        L"Background:=<AcrylicBrush TintOpacity=\"0.8\" TintColor=\"Black\"
FallbackColor=\"#BB222222\" />"}},
    ThemeTargetStyles{L"Grid", {
        L"RequestedTheme=2"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid >
Border#BackgroundElement@CommonStates,
Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Border#BackgroundElement@CommonStates", {
        L"Background:=<AcrylicBrush TintOpacity=\"0.8\" TintColor=\"Black\"
FallbackColor=\"#BB222222\" />",
        L"Width=125",
        L"BorderBrush=Transparent"}},
    ThemeTargetStyles{L"Grid#OverflowRootGrid", {
        L"Background:=<AcrylicBrush TintOpacity=\"0.8\" TintColor=\"Black\"
FallbackColor=\"#BB222222\" />",
        L"Padding=0",
        L"Margin=0,0,0,12",
        L"CornerRadius=5"}},
    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton >
Taskbar.TaskListButtonPanel > Grid > Border,
Taskbar.ExperienceToggleButton#LaunchListButton > Taskbar.TaskListButtonPanel > Border", {
        L"Background:=<AcrylicBrush TintOpacity=\"0.8\" TintColor=\"Black\"
FallbackColor=\"#BB222222\" />"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton", {
        L"Margin=20,1,-20,1"}},
    ThemeTargetStyles{L"Grid#AugmentedEntryPointContentGrid", {
```

```
        L"Margin=10,0,-5,0"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.Name=Task View] > Taskbar.TaskListButtonPanel > Grid > Border, Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.Name=Task View] > Taskbar.TaskListButtonPanel > Border", {
        L"CornerRadius=0,5,5,0"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchBoxButton", {
        L"Background:=<AcrylicBrush TintOpacity=\"0.8\" TintColor=\"Black\" FallbackColor=\"#BB333333\" />"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid > Grid > Border, SearchUx.SearchUI.SearchButtonRootGrid > Border", {
        L"Opacity=0"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid", {
        L"Background:=<AcrylicBrush TintOpacity=\"0.8\" TintColor=\"Black\" FallbackColor=\"#BB222222\" />",
        L"MinWidth=0",
        L"CornerRadius=0,5,5,0",
        L"Margin=0,4,2,4"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchBoxButton#SearchBox > SearchUx.SearchUI.SearchButtonRootGrid@CommonStates > Grid > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon, SearchUx.SearchUI.SearchBoxButton#SearchBox > SearchUx.SearchUI.SearchButtonRootGrid@CommonStates > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon", {
        L"Margin=14,0,0,0"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.Name=Start] > Taskbar.TaskListButtonPanel > Grid > Border, Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.Name=Start] > Taskbar.TaskListButtonPanel > Border", {
        L"Margin=1,0,0,0"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.Name=Start] > Taskbar.TaskListButtonPanel", {
        L"RenderTransform:=<TranslateTransform X=\"5\" />"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.Name=Task View] > Taskbar.TaskListButtonPanel", {
        L"RenderTransform:=<TranslateTransform X=\"-9\" />"}},
    ThemeTargetStyles{L"Border#BackgroundBorder", {
        L"Margin=0,3,0,3"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid > Rectangle#BackgroundFill", {
        L"Fill:=<WindhawkBlur BlurAmount=\"12\" TintColor=\"#00000000\" />"}},
    ThemeTargetStyles{L"Border#BackgroundElement", {
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border#BackgroundElement, Taskbar.TaskListLabeledButtonPanel@CommonStates > Border#BackgroundElement", {
        L"BorderBrush=Transparent",
        L"BorderBrush@ActivePointerOver:=<AcrylicBrush TintOpacity=\"0.2\" TintColor=\"Black\" FallbackColor=\"#DD222222\" />",
        L"BorderBrush@InactivePointerOver:=<AcrylicBrush TintOpacity=\"0.2\" TintColor=\"Black\" FallbackColor=\"#DD222222\" />",
        L"CornerRadius=5"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Rectangle#RunningIndicator, Taskbar.TaskListLabeledButtonPanel@CommonStates > Rectangle#RunningIndicator", {
        L"Fill=Transparent",
        L"Fill@ActivePointerOver:=<AcrylicBrush TintOpacity=\"0.4\" TintColor=\"#88444444\" FallbackColor=\"#BB222222\" />",
        L"Fill@InactivePointerOver:=<AcrylicBrush TintOpacity=\"0.2\" TintColor=\"#88444444\" FallbackColor=\"#BB222222\" />",
        L"Fill@MultiWindowNormal=#22AAAABB",
        L"Fill@MultiWindowPointerOver=#88AAAABB",
        L"Fill@MultiWindowActive=#88AAAABB",
```

```
      L"Fill@MultiWindowPressed=#88AAAABB",
      L"Fill@RequestingAttention:=<SolidColorBrush Opacity=\"0.4\" Color=\"DarkOrange\" />",
      L"Fill@RequestingAttentionPointerOver:=<SolidColorBrush Opacity=\"0.4\"
Color=\"Orange\" />",
      L"RadiusX=5",
      L"RadiusY=5",
      L"Height=38",
      L"Width=39",
      L"MinWidth=Auto"}},
}};

const Theme g_themeMatter = {{
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground >
Grid > Rectangle#BackgroundFill", {
      L"Fill := $transparent"}},
   ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
      L"Fill := $transparent"}},
   ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl", {
      L"Fill:=$base",
      L"CornerRadius = $mainRadius"}},
   ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton", {
      L"Margin=-1,1,1,1"}},
   ThemeTargetStyles{L"Taskbar.TaskListButtonPanel@CommonStates > Grid >
Border#BackgroundElement, Taskbar.TaskListButtonPanel@CommonStates >
Border#BackgroundElement", {
      L"CornerRadius = $mainRadius",
      L"Background :=$base",
      L"Background@InactivePointerOver :=$overlay2",
      L"Background@ActivePointerOver:=$overlay",
      L"Background@ActiveNormal :=$active"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.
Name=Task View]", {
      L"Margin=0,0,2,0"}},

ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton[AutomationProperties.Name=Copil
ot] > Grid#IconPanel > Border#BackgroundElement,
Taskbar.TaskListButton#TaskListButton[AutomationProperties.Name=Copilot] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border#BackgroundElement", {
      L"Visibility = 1"}},
   ThemeTargetStyles{L"Taskbar.SearchBoxButton", {
      L"Margin=0,0,2,0"}},
   ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl", {
      L"Margin=0,0,2,0"}},
   ThemeTargetStyles{L"Border#BackgroundElement", {
      L"BorderThickness=0"}},
   ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border#BackgroundElement,
Taskbar.TaskListLabeledButtonPanel@CommonStates > Border#BackgroundElement", {
      L"Background@InactiveNormal :=$base",
      L"Background@ActiveNormal :=$active",
      L"Background@InactivePointerOver :=$overlay2",
      L"Background@ActivePointerOver:=$overlay",
      L"CornerRadius = $mainRadius",
      L"Margin = 1,0,1,0",
      L"Background@MultiWindowNormal:=$base",
      L"Background@MultiWindowPointerOver:=$overlay2",
      L"Background@MultiWindowActive:=$active",
      L"Background@MultiWindowPressed:=$overlay"}},
   ThemeTargetStyles{L"Border#MultiWindowElement", {
      L"CornerRadius = $mainRadius",
```

```
    L"Padding = 7,0,8,0",
    L"Background :=$accentColor"}},
  ThemeTargetStyles{L"Grid#IconPanel > TextBlock#LabelControl, Taskbar.TaskListLabeledButtonPanel > TextBlock#LabelControl", {
    L"Margin=0,0,2,0"}},
  ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Rectangle#RunningIndicator, Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Rectangle#RunningIndicator", {
    L"Fill := $inverseBW",
    L"RadiusX=1.5",
    L"RadiusY=1.5",
    L"Height=4",
    L"Width=12",
    L"Fill@ActiveRunningIndicator :=$accentColor",
    L"Width@ActiveRunningIndicator=21"}},
  ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
    L"Background:=$base",
    L"CornerRadius = $mainRadius",
    L"Margin=0,5,12,5",
    L"Padding=5,0,0,0"}},
  ThemeTargetStyles{L"Border#BackgroundBorder", {
    L"Margin=2,5,2,5",
    L"CornerRadius=8",
    L"BorderThickness = 0"}},
  ThemeTargetStyles{L"Grid#OverflowRootGrid > Border", {
    L"Background:=$base",
    L"Shadow :=",
    L"CornerRadius = 14"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalTrackRect", {
    L"Height = 8",
    L"Margin = 0",
    L"Fill := $overlay"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalDecreaseRect", {
    L"Height = 8"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.TextBlock#volumeLevelText", {
    L"FontFamily = Tektur",
    L"Margin = 0,-2,0,0"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#VolumeConfirmator", {
    L"Padding = 8,0,3,0",
    L"CornerRadius = 20"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#ConfirmatorMainGrid", {
    L"Background :=$base",
    L"CornerRadius = 14",
    L"BorderThickness = 0",
    L"Margin = 0,0,0,10",
    L"Shadow :="}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#BrightnessConfirmator", {
    L"Padding = 15,0,17,0",
    L"CornerRadius = 20"}},
  ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.AnimatedIcon#BrightnessIcon", {
    L"Margin = 0,-1,12,0"}},
  ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ProgressBar#ProgressIndicator", {
    L"Margin = 0,0,0,1"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#ProgressBarTrack", {
    L"Fill := $inverseBW",
    L"RadiusX = 1.5",
    L"RadiusY = 1.5"}},
```

```
ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#DeterminateProgressBarIndicator", {
      L"Fill :=$accentColor"}},
   ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton > Grid#IconPanel > Microsoft.UI.Xaml.Controls.ProgressBar#ProgressIndicator, Taskbar.TaskListButton#TaskListButton > Taskbar.TaskListLabeledButtonPanel#IconPanel > Microsoft.UI.Xaml.Controls.ProgressBar#ProgressIndicator", {
      L"MinHeight = 4",
      L"Width = 26"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.ContentPresenter#ContentPresenter", {
      L"BorderThickness = 0"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.Name=Start]", {
      L"Margin = 0,0,2,0"}},
   ThemeTargetStyles{L"Taskbar.Badge#BadgeControl", {
      L"Height = 14",
      L"MinWidth = 14",
      L"Margin = 0,0,0,0",
      L"CornerRadius = 20"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#BackgroundRect", {
      L"RadiusX = 4",
      L"RadiusY = 4"}},
   ThemeTargetStyles{L"MenuFlyoutPresenter", {
      L"Background := $base",
      L"Shadow :=",
      L"CornerRadius = 8"}},
   ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid@CommonStates > Grid > Border#BackgroundElement, SearchUx.SearchUI.SearchButtonRootGrid@CommonStates > Border#BackgroundElement", {
      L"Background@InactiveNormal :=$base",
      L"CornerRadius = $mainRadius",
      L"Background :=$base",
      L"Background@InactivePointerOver :=$overlay2",
      L"Background@ActivePointerOver:=$overlay",
      L"Background@ActiveNormal :=$active"}},
}, {
   L"mainRadius = 8",
   L"transparent = <SolidColorBrush Color=\"Transparent\"/>",
   L"base = <AcrylicBrush TintColor=\"{ThemeResource SystemAltLowColor}\" TintOpacity=\"1\" TintLuminosityOpacity=\"0.7\" FallbackColor=\"{ThemeResource SystemChromeLowColor}\" />",
   L"overlay = <AcrylicBrush TintColor=\"{ThemeResource SystemAltLowColor}\" TintOpacity=\"1\" TintLuminosityOpacity=\"0.8\" FallbackColor=\"{ThemeResource CardStrokeColorDefaultSolid}\" />",
   L"overlay2 = <AcrylicBrush TintColor=\"{ThemeResource SystemAltLowColor}\" TintOpacity=\"1\" TintLuminosityOpacity=\"0.5\" FallbackColor=\"{ThemeResource CardStrokeColorDefaultSolid}\" />",
   L"accentColor = <SolidColorBrush Color=\"{ThemeResource SystemAccentColor}\" Opacity = \"1\" />",
   L"inverseBW = <SolidColorBrush Color=\"{ThemeResource SystemBaseHighColor}\" Opacity = \"1\" />",
   L"active = <AcrylicBrush TintColor=\"{ThemeResource SystemAltLowColor}\" TintOpacity=\"1\" TintLuminosityOpacity=\" 1\" FallbackColor=\"{ThemeResource CardStrokeColorDefaultSolid}\" />",
}};

const Theme g_themeWinXP = {{
   ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
      L"Fill:=<LinearGradientBrush StartPoint=\"0.5,0\" EndPoint=\"0.5,1\"> <GradientStop Color=\"#3168d5\" Offset=\"0.0\" /> <GradientStop Color=\"#4993e6\" Offset=\"0.08\" /> <GradientStop Color=\"#245dd7\" Offset=\"0.18\" /> <GradientStop Color=\"#2561de\" Offset=\"0.9\" /> <GradientStop Color=\"#1941a5\" Offset=\"1.0\" /></LinearGradientBrush>",
      L"VerticalAlignment=Stretch",
```

```
        L"Height=Auto"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid > Rectangle#BackgroundFill", {
        L"Fill:=<LinearGradientBrush StartPoint=\"0.5,0\" EndPoint=\"0.5,1\"> <GradientStop Color=\"#3168d5\" Offset=\"0.0\" /> <GradientStop Color=\"#4993E6\" Offset=\"0.1\" /> <GradientStop Color=\"#2157D7\" Offset=\"0.35\" /> <GradientStop Color=\"#2663E0\" Offset=\"0.8\" /> <GradientStop Color=\"#1941A5\" Offset=\"1.0\" /></LinearGradientBrush>"}},

    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton]", {
        L"Margin=-9,0,10,0"}},

    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton] > Taskbar.TaskListButtonPanel", {
        L"Padding=0",
        L"Width=60",
        L"CornerRadius=9",
        L"Background:=<LinearGradientBrush StartPoint=\"0.5,0\" EndPoint=\"0.5,1\"> <GradientStop Color=\"#2D6B2D\" Offset=\"0.0\" /> <GradientStop Color=\"#7ED57E\" Offset=\"0.08\" /> <GradientStop Color=\"#3DB43D\" Offset=\"0.35\" /> <GradientStop Color=\"#2A752E\" Offset=\"0.85\" /> <GradientStop Color=\"#144818\" Offset=\"1.0\" /></LinearGradientBrush>",
        L"BorderThickness=0,0,2,0",
        L"BorderBrush:=<LinearGradientBrush StartPoint=\"0.5,0\" EndPoint=\"0.5,1\"> <GradientStop Color=\"#400D330D\" Offset=\"0.0\" /> <GradientStop Color=\"#800D330D\" Offset=\"0.4\" /> <GradientStop Color=\"#FF0D330D\" Offset=\"1.0\" /></LinearGradientBrush>"}},

    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton] > Taskbar.TaskListButtonPanel > Grid > Border#BackgroundElement, Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton] > Taskbar.TaskListButtonPanel > Border#BackgroundElement", {
        L"Background:=<ImageBrush Stretch=\"None\" ImageSource=\"https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/WinXP/Assets/orb.png\" />"}},

    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton] > Taskbar.TaskListButtonPanel > Grid > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon, Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton] > Taskbar.TaskListButtonPanel > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"TextBlock#LabelControl", {
        L"Foreground=White"}},
    ThemeTargetStyles{L"Rectangle#RunningIndicator", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"TextBlock#TimeInnerTextBlock", {
        L"Foreground=White"}},
    ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
        L"Foreground=White"}},
    ThemeTargetStyles{L"SystemTray.TextIconContent > Grid > SystemTray.AdaptiveTextBlock#Base > TextBlock", {
        L"Foreground=White"}},
    ThemeTargetStyles{L"SystemTray.BatteryIconContent > Grid#ContainerGrid > StackPanel > TextBlock", {
        L"Foreground=White"}},
    ThemeTargetStyles{L"SystemTray.BatteryIconContent > Grid#ContainerGrid > StackPanel > Grid > TextBlock[1]", {
        L"Foreground=White"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Border#BackgroundElement, Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Border#BackgroundElement", {
        L"Background@NoRunningIndicator=Transparent",
        L"Background@ActiveRunningIndicator:=<LinearGradientBrush StartPoint=\"0.5,0\" EndPoint=\"0.5,1\"> <GradientStop Color=\"#1542A8\" Offset=\"0.0\" /> <GradientStop Color=\"#245DD4\" Offset=\"0.2\" /> <GradientStop Color=\"#1542A8\" Offset=\"1.0\" /></LinearGradientBrush>",
        L"Background:=<LinearGradientBrush StartPoint=\"0.5,0\" EndPoint=\"0.5,1\"> <GradientStop Color=\"#6599E6\" Offset=\"0.0\" /> <GradientStop Color=\"#4A88E0\" Offset=\"0.1\" /> <GradientStop Color=\"#4282D9\" Offset=\"0.9\" /> <GradientStop Color=\"#2A62B5\" Offset=\"1.0\" /></LinearGradientBrush>",
```

```
        L"BorderBrush@NoRunningIndicator=Transparent",
        L"BorderBrush@ActiveRunningIndicator=#083192",
        L"BorderBrush=#1A4DBF"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton", {
        L"Margin=-2"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"Background:=<LinearGradientBrush StartPoint=\"0.5,0\" EndPoint=\"0.5,1\"><GradientStop Color=\"#16acf0\" Offset=\"0.0\" /> <GradientStop Color=\"#19b6f2\" Offset=\"0.08\" /> <GradientStop Color=\"#0c8eea\" Offset=\"0.18\" /> <GradientStop Color=\"#0c8eea\" Offset=\"0.9\" /> <GradientStop Color=\"#095bc9\" Offset=\"1.0\" /></LinearGradientBrush>",
        L"BorderThickness=2,1,0,1",
        L"BorderBrush=#095BC9",
        L"Padding={{-ChevronIconWidth/2}},0,0,0"}},
    ThemeTargetStyles{L"SystemTray.Stack#NotifyIconStack", {
        L"ActualWidth=>ChevronIconWidth"}},
    ThemeTargetStyles{L"Grid#OverflowRootGrid > Border", {
        L"Background:=<LinearGradientBrush StartPoint=\"0.5,0\" EndPoint=\"0.5,1\"><GradientStop Color=\"#16acf0\" Offset=\"0.0\" /> <GradientStop Color=\"#19b6f2\" Offset=\"0.08\" /> <GradientStop Color=\"#0c8eea\" Offset=\"0.18\" /> <GradientStop Color=\"#0c8eea\" Offset=\"0.9\" /> <GradientStop Color=\"#095bc9\" Offset=\"1.0\" /></LinearGradientBrush>"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView > Grid#ContainerGrid > Border#BackgroundBorder", {
        L"Background:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"1,1\"> <GradientStop Color=\"#86cbef\" Offset=\"0.0\" /><GradientStop Color=\"#71ccf8\" Offset=\"0.25\" /><GradientStop Color=\"#0983ee\" Offset=\"0.75\" /><GradientStop Color=\"#0872eb\" Offset=\"1\" /></LinearGradientBrush>",
        L"Width=20",
        L"Height=20",
        L"CornerRadius=10",
        L"BorderThickness=1",
        L"BorderBrush:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"1,1\"> <GradientStop Color=\"#000000\" Offset=\"0.0\" /><GradientStop Color=\"#FFFFFF\" Offset=\"1\" /></LinearGradientBrush>"}},
}};

const Theme g_themeWinXP_variant_Zune = {{
    ThemeTargetStyles{L"Rectangle#BackgroundFill", {
        L"Fill:=<LinearGradientBrush StartPoint=\"0.5,0.5\" EndPoint=\"0.5,1\"> <GradientStop Color=\"#656565\" Offset=\"0.0\" /> <GradientStop Color=\"#363636\" Offset=\"0.1\" /> <GradientStop Color=\"#363636\" Offset=\"0.35\" /> <GradientStop Color=\"#363636\" Offset=\"0.8\" /> <GradientStop Color=\"#363636\" Offset=\"1.0\" /></LinearGradientBrush>",
        L"VerticalAlignment=Stretch",
        L"Height=Auto"}},

    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton]", {
        L"CornerRadius=0",
        L"Margin=-4,0,4,0",
        L"MaxWidth=48"}},

    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton] > Taskbar.TaskListButtonPanel", {
        L"Padding=0",
        L"Background:=<LinearGradientBrush StartPoint=\"0.5,0.5\" EndPoint=\"0.5,1\"><GradientStop Color=\"#D76A27\" Offset=\"0.05\" /> <GradientStop Color=\"#B44704\" Offset=\"0.1\" /> <GradientStop Color=\"#772E01\" Offset=\"0.5\" /> <GradientStop Color=\"#772E01\" Offset=\"1\" /> <GradientStop Color=\"#AA4201\" Offset=\"1\" /></LinearGradientBrush>"}},

    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton] > Taskbar.TaskListButtonPanel > Grid > Border#BackgroundElement, Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton] > Taskbar.TaskListButtonPanel > Border#BackgroundElement", {
        L"Background:=<ImageBrush Stretch=\"Uniform\" ImageSource=\"https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/WinXP/Assets/orb.png\" />",
        L"Height=32"}},
```

```
ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton] > Taskbar.TaskListButtonPanel > Grid > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon, Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton] > Taskbar.TaskListButtonPanel > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"TextBlock#LabelControl", {
    L"Foreground=White"}},
  ThemeTargetStyles{L"Rectangle#RunningIndicator", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"TextBlock#TimeInnerTextBlock", {
    L"Foreground=White"}},
  ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
    L"Foreground=White"}},
  ThemeTargetStyles{L"SystemTray.TextIconContent > Grid > SystemTray.AdaptiveTextBlock#Base > TextBlock", {
    L"Foreground=White"}},
  ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Border, Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Border", {
    L"BorderThickness=1",
    L"CornerRadius=2",
    L"BorderBrush@NoRunningIndicator=Transparent",
    L"Margin=-2,-1,-2,-1"}},
  ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Border#BackgroundElement, Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Border#BackgroundElement", {
    L"BorderBrush=#BB4B4B4B",
    L"Margin=1",
    L"BorderThickness=1",
    L"Background:=<LinearGradientBrush StartPoint=\"0.5,0.42\" EndPoint=\"0.5,0.75\"><GradientStop Color=\"#6B6B6B\" Offset=\"0.0\" /><GradientStop Color=\"#363636\" Offset=\"0.5\" /><GradientStop Color=\"#363636\" Offset=\"0.35\" /><GradientStop Color=\"#363636\" Offset=\"0.8\" /><GradientStop Color=\"#363636\" Offset=\"1.0\" /></LinearGradientBrush>",
    L"Background@ActiveRunningIndicator:=<LinearGradientBrush StartPoint=\"0.5,0.5\" EndPoint=\"0.5,1\"><GradientStop Color=\"#6B6B6B\" Offset=\"0.0\" /><GradientStop Color=\"#434343\" Offset=\"0.1\" /><GradientStop Color=\"#434343\" Offset=\"0.35\" /><GradientStop Color=\"#434343\" Offset=\"0.8\" /><GradientStop Color=\"#434343\" Offset=\"1.0\" /></LinearGradientBrush>",
    L"BorderBrush@NoRunningIndicator=Transparent",
    L"Background@NoRunningIndicator=Transparent"}},
  ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
    L"Fill=#858585"}},
  ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
    L"Background:=<LinearGradientBrush StartPoint=\"0.5,0.42\" EndPoint=\"0.5,0.75\"><GradientStop Color=\"#454545\" Offset=\"0.0\" /><GradientStop Color=\"#313131\" Offset=\"0.5\" /><GradientStop Color=\"#363636\" Offset=\"0.35\" /><GradientStop Color=\"#1D1D1D\" Offset=\"0.8\" /><GradientStop Color=\"#1D1D1D\" Offset=\"1.0\" /></LinearGradientBrush>",
    L"BorderThickness=1,0,0,0",
    L"BorderBrush=#222222",
    L"Padding=4,0,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton > Grid#IconPanel@RunningIndicatorStates > Windows.UI.Xaml.Controls.Image#Icon, Taskbar.TaskListButton#TaskListButton > Taskbar.TaskListLabeledButtonPanel#IconPanel@RunningIndicatorStates > Windows.UI.Xaml.Controls.Image#Icon", {
    L"Height@NoRunningIndicator=16"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton > Grid#IconPanel@RunningIndicatorStates, Taskbar.TaskListButton#TaskListButton > Taskbar.TaskListLabeledButtonPanel#IconPanel@RunningIndicatorStates", {
    L"Margin@NoRunningIndicator=-7,0,-7,0",
    L"Padding@NoRunningIndicator=0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton", {
    L"Margin=-1.5"}},
  ThemeTargetStyles{L"Grid#OverflowRootGrid > Border", {
```

```
      L"Background:=<LinearGradientBrush StartPoint=\"0.5,0.5\" EndPoint=\"0.5,1\"><GradientStop Color=\"#656565\" Offset=\"0.0\" /><GradientStop Color=\"#363636\" Offset=\"0.1\" /><GradientStop Color=\"#363636\" Offset=\"0.35\" /><GradientStop Color=\"#363636\" Offset=\"0.8\" /><GradientStop Color=\"#363636\" Offset=\"1.0\" /></LinearGradientBrush>"}},
};

const Theme g_themeBubbles = {{
   ThemeTargetStyles{L"Rectangle#BackgroundFill", {
      L"Fill:=<SolidColorBrush x:Name=\"SystemChromeLow\" Color=\"{ThemeResource SystemChromeLowColor}\" />"}},
   ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Border#BackgroundElement, Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Border#BackgroundElement", {
      L"CornerRadius=20",
      L"Background@NoRunningIndicator:=<SolidColorBrush x:Name=\"SystemChromeHigh\" Opacity=\"0.18\" Color=\"{ThemeResource SystemChromeHighColor}\" />",
      L"Background:=<SolidColorBrush x:Name=\"SearchBoxTextBlock\" Opacity=\"0.15\" Color=\"{ThemeResource SearchPillButtonForeground}\" />",
      L"BorderThickness=1.5",
      L"BorderBrush:=<SolidColorBrush x:Name=\"SearchBoxTextBlock\" Opacity=\"0.25\" Color=\"{ThemeResource SearchPillButtonForeground}\" />",
      L"BorderThickness@NoRunningIndicator=1",
      L"BorderBrush@NoRunningIndicator:=<SolidColorBrush x:Name=\"SearchBoxTextBlock\" Opacity=\"0.15\" Color=\"{ThemeResource SearchPillButtonForeground}\" />",
      L"Margin=1"}},
   ThemeTargetStyles{L"Taskbar.TaskListButtonPanel@CommonStates > Grid > Border#BackgroundElement, Taskbar.TaskListButtonPanel@CommonStates > Border#BackgroundElement, SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel@CommonStates > Grid > Border#BackgroundElement, SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel@CommonStates > Border#BackgroundElement", {
      L"Background:=<SolidColorBrush x:Name=\"SystemChromeHigh\" Opacity=\"0.3\" Color=\"{ThemeResource SystemChromeHighColor}\" />",
      L"BorderBrush:=<SolidColorBrush x:Name=\"SystemChromeHigh\" Opacity=\"0.6\" Color=\"{ThemeResource SystemChromeHighColor}\" />",
      L"Background@ActivePointerOver:=<SolidColorBrush x:Name=\"SystemChromeHigh\" Opacity=\"0.8\" Color=\"{ThemeResource SystemChromeHighColor}\" />",
      L"Background@InactivePointerOver:=<SolidColorBrush x:Name=\"SystemChromeHigh\" Opacity=\"0.8\" Color=\"{ThemeResource SystemChromeHighColor}\" />",
      L"Background@ActivePressed:=<SolidColorBrush x:Name=\"SystemChromeHigh\" Opacity=\"1\" Color=\"{ThemeResource SystemChromeHighColor}\" />",
      L"Background@InactivePressed:=<SolidColorBrush x:Name=\"SystemChromeHigh\" Opacity=\"1\" Color=\"{ThemeResource SystemChromeHighColor}\" />",
      L"BorderBrush@InactivePressed:=<SolidColorBrush x:Name=\"SystemChromeHigh\" Opacity=\"0.8\" Color=\"{ThemeResource SystemAccentColor}\" />",
      L"CornerRadius=20",
      L"BorderThickness@InactivePressed=3",
      L"BorderThickness=2"}},
   ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
      L"Background:=<SolidColorBrush x:Name=\"SystemChromeHigh\" Opacity=\"0.6\" Color=\"{ThemeResource SystemChromeHighColor}\" />",
      L"CornerRadius=20",
      L"Margin=-5,5,8,5",
      L"Padding=10,0,-10,0",
      L"BorderBrush:=<SolidColorBrush x:Name=\"SystemChromeHigh\" Opacity=\"0.9\" Color=\"{ThemeResource SystemChromeHighColor}\" />",
      L"BorderThickness=1.5"}},
   ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Rectangle#RunningIndicator, Taskbar.TaskListLabeledButtonPanel@CommonStates > Rectangle#RunningIndicator", {
      L"Stroke@InactivePointerOver=#75A8E6",
      L"Stroke@InactivePressed=#7CB1F2",
      L"Stroke@ActiveNormal=#5F87B9",
      L"Stroke@ActivePointerOver=#75A8E6",
      L"Stroke@ActivePressed=#7CB1F2",
      L"Fill=Transparent",
```

```
        L"RadiusX=20",
        L"RadiusY=20",
        L"StrokeThickness=3",
        L"Stroke@MultiWindowPointerOver=#CCCCDD",
        L"Stroke@MultiWindowPressed=White",
        L"Stroke@MultiWindowActive=#BBBBCC",
        L"Fill@MultiWindowNormal=#88AAAABB",
        L"Fill@MultiWindowPointerOver=#88AAAABB",
        L"Fill@MultiWindowActive=#88AAAABB",
        L"Fill@MultiWindowPressed=#88AAAABB",
        L"Stroke@RequestingAttention=Crimson",
        L"Stroke@RequestingAttentionPointerOver=Red",
        L"Fill@RequestingAttention:=<SolidColorBrush Opacity=\"0.4\" Color=\"DarkOrange\" />",
        L"Fill@RequestingAttentionPointerOver:=<SolidColorBrush Opacity=\"0.4\"
Color=\"Orange\" />",
        L"StrokeThickness@RequestingAttention=2.5",
        L"StrokeThickness@RequestingAttentionPointerOver=2.5",
        L"Height=38",
        L"Width=38",
        L"MinWidth=Auto"}},
    ThemeTargetStyles{L"Grid#IconPanel > TextBlock#LabelControl,
Taskbar.TaskListLabeledButtonPanel > TextBlock#LabelControl", {
        L"Margin=4,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.SearchBoxButton", {
        L"Height=48",
        L"Margin=0,-2,0,0"}},

ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Grid > Border#BackgroundElement,
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Border#BackgroundElement", {
        L"CornerRadius=20",
        L"Background:=<SolidColorBrush x:Name=\"SearchBoxTextBlock\" Opacity=\"0.15\"
Color=\"{ThemeResource SearchPillButtonForeground}\" />",
        L"BorderBrush:=<SolidColorBrush x:Name=\"SearchBoxTextBlock\" Opacity=\"0.25\"
Color=\"{ThemeResource SearchPillButtonForeground}\" />"}},
    ThemeTargetStyles{L"Border#MultiWindowElement", {
        L"Height=0"}},
    ThemeTargetStyles{L"Grid#OverflowRootGrid > Border", {
        L"Background:=<SolidColorBrush x:Name=\"SystemChromeHigh\" Opacity=\"0.8\"
Color=\"{ThemeResource SystemChromeHighColor}\" />",
        L"BorderBrush:=<SolidColorBrush x:Name=\"SearchBoxTextBlock\" Opacity=\"0.1\"
Color=\"{ThemeResource SearchPillButtonForeground}\" />"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.
AutomationId=StartButton] > Taskbar.TaskListButtonPanel > Grid >
Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon,
Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartB
utton] > Taskbar.TaskListButtonPanel > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon",
{
        L"Margin=1,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack", {
        L"Padding=5,0,5,0",
        L"Margin=2,0,10,0"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#ShowDesktopPipe", {
        L"Grid.Row=0",
        L"Grid.RowSpan=3",
        L"Grid.Column=0",
        L"Grid.ColumnSpan=3",
        L"Height=16",
        L"Width=4",
        L"RadiusX=2",
```

```
        L"RadiusY=2",
        L"Margin=-5,0,5,0"}},
    ThemeTargetStyles{L"SystemTray.Stack#NotifyIconStack > Windows.UI.Xaml.Controls.Grid > SystemTray.StackListView > Windows.UI.Xaml.Controls.ItemsPresenter > Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.ContentPresenter > SystemTray.ChevronIconView > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"CornerRadius=16,5,5,16",
        L"Margin=-3,4,0,4"}},
    ThemeTargetStyles{L"TextBlock#InnerTextBlock", {
        L"Foreground:=<SolidColorBrush x:Name=\"SearchBoxTextBlock\" Color=\"{ThemeResource SearchPillButtonForeground}\" />"}},
    ThemeTargetStyles{L"TextBlock#LabelControl", {
        L"Foreground:=<SolidColorBrush x:Name=\"SearchBoxTextBlock\" Color=\"{ThemeResource SearchPillButtonForeground}\" />"}},
    ThemeTargetStyles{L"TextBlock#TimeInnerTextBlock", {
        L"Foreground:=<SolidColorBrush x:Name=\"SearchBoxTextBlock\" Color=\"{ThemeResource SearchPillButtonForeground}\" />",
        L"Margin=0,2.5,0,-2.5"}},
    ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
        L"Foreground:=<SolidColorBrush x:Name=\"SearchBoxTextBlock\" Color=\"{ThemeResource SearchPillButtonForeground}\" />",
        L"Margin=0,-1,0,2"}},
    ThemeTargetStyles{L"Grid#ContainerGrid@ > Border#BackgroundBorder", {
        L"Background@PointerOver:=<SolidColorBrush x:Name=\"SearchBoxTextBlock\" Opacity=\"0.2\" Color=\"{ThemeResource SearchPillButtonForeground}\" />",
        L"CornerRadius=20",
        L"Margin=-1",
        L"Height=28",
        L"Width=28",
        L"BorderBrush=Transparent"}},
    ThemeTargetStyles{L"SystemTray.IconView > Grid#ContainerGrid@ > Border#BackgroundBorder", {
        L"CornerRadius=20",
        L"Background@PointerOver:=<SolidColorBrush x:Name=\"SearchBoxTextBlock\" Opacity=\"0.15\" Color=\"{ThemeResource SearchPillButtonForeground}\" />",
        L"Height=34",
        L"Width=34"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Grid@CommonStates > Border#BackgroundBorder", {
        L"CornerRadius=20",
        L"Width=75",
        L"Margin=-2,1,-2,1",
        L"Background@PointerOver:=<SolidColorBrush x:Name=\"SearchBoxTextBlock\" Opacity=\"0.15\" Color=\"{ThemeResource SearchPillButtonForeground}\" />"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid@CommonStates > Border#BackgroundBorder", {
        L"CornerRadius=20",
        L"Background@PointerOver:=<SolidColorBrush x:Name=\"SearchBoxTextBlock\" Opacity=\"0.15\" Color=\"{ThemeResource SearchPillButtonForeground}\" />",
        L"Margin=1"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton > Taskbar.TaskListButtonPanel@CommonStates", {
        L"Background:=<SolidColorBrush x:Name=\"SystemChromeHigh\" Opacity=\"0.6\" Color=\"{ThemeResource SystemChromeHighColor}\" />",
        L"BorderBrush:=<SolidColorBrush x:Name=\"SystemChromeHigh\" Opacity=\"0.9\" Color=\"{ThemeResource SystemChromeHighColor}\" />",
        L"BorderThickness=1.5",
        L"Margin=5",
        L"Background@InactivePointerOver:=<SolidColorBrush x:Name=\"SystemChromeHigh\" Opacity=\"1\" Color=\"{ThemeResource SystemChromeHighColor}\" />",
        L"Padding=-1.5,-1,-1.5,-1",
        L"CornerRadius=20"}},
```

```
    ThemeTargetStyles{L"Grid#OverflowRootGrid", {
        L"Background:=<SolidColorBrush x:Name=\"SystemChromeHigh\" Opacity=\"0.5\"
Color=\"{ThemeResource SystemChromeHighColor}\" />",
        L"Padding=0",
        L"Margin=0,0,0,12",
        L"CornerRadius=8"}},
    ThemeTargetStyles{L"Rectangle#LeftDropInsertionMarker", {
        L"Fill:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight1}\" />"}},
    ThemeTargetStyles{L"Rectangle#RightDropInsertionMarker", {
        L"Fill:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight1}\" />"}},
    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton >
Taskbar.TaskListButtonPanel > Grid > Border,
Taskbar.ExperienceToggleButton#LaunchListButton > Taskbar.TaskListButtonPanel > Border", {
        L"CornerRadius=20",
        L"Background:=<SolidColorBrush x:Name=\"SearchBoxTextBlock\" Opacity=\"0.15\"
Color=\"{ThemeResource SearchPillButtonForeground}\" />",
        L"BorderBrush:=<SolidColorBrush x:Name=\"SearchBoxTextBlock\" Opacity=\"0.25\"
Color=\"{ThemeResource SearchPillButtonForeground}\" />"}},
}};

const Theme g_themeRosePine = {{
    ThemeTargetStyles{L"Taskbar.TaskListButton", {
        L"CornerRadius=3"}},
    ThemeTargetStyles{L"SystemTray.TextIconContent > Grid#ContainerGrid >
SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock", {
        L"FontSize=16"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon", {
        L"MinWidth=25"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid >
ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[1] >
SystemTray.IconView > Grid > Grid", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"SystemTray.TextIconContent > Grid#ContainerGrid", {
        L"Padding=2"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView", {
        L"MinWidth=27"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Grid >
ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.IconView#SystemTrayIcon > Grid > Grid > SystemTray.TextIconContent", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Grid#IconPanel > Border#BackgroundElement,
Taskbar.TaskListLabeledButtonPanel > Border#BackgroundElement", {
        L"Background:=#302d47",
        L"CornerRadius=6"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"Background:=#302d47",
        L"CornerRadius=6",
        L"Margin=0,5,4,4",
        L"Padding=3,0,-8,0"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Rectangle#RunningIndicator,
Taskbar.TaskListLabeledButtonPanel@CommonStates > Rectangle#RunningIndicator", {
        L"Height=Auto",
        L"RadiusX=5",
        L"RadiusY=5",
        L"StrokeThickness=2",
        L"Stroke@InactivePointerOver=#ebbcba",
        L"Stroke@InactivePressed=#ebbcba",
        L"Stroke@ActiveNormal=#ebbcba",
        L"Stroke@ActivePointerOver=#ebbcba",
        L"Stroke@ActivePressed=#ebbcba",
```

```
        L"Fill=Transparent",
        L"Width=37",
        L"VerticalAlignment=Stretch",
        L"Canvas.ZIndex=1"}},
    ThemeTargetStyles{L"SystemTray.ImageIconContent > Grid#ContainerGrid > Image", {
        L"Width=13"}},
    ThemeTargetStyles{L"SystemTray.TextIconContent > Grid#ContainerGrid >
SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock", {
        L"FontSize=14"}},
    ThemeTargetStyles{L"TextBlock#LabelControl", {
        L"FontFamily=JetBrainsMono NF",
        L"Foreground=#e0def4",
        L"Padding=2,0,8,0"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.
AutomationId=StartButton]", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.TextBlock#InnerTextBlock[Text=\uE971]", {
        L"Text=\uE712"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground >
Grid > Rectangle#BackgroundFill", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid >
Rectangle#BackgroundFill", {
        L"Fill=#302d47"}},
    ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid >
Border#BackgroundElement, Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Border#BackgroundElement,
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Grid >
Border#BackgroundElement,
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Border#BackgroundElement", {
        L"Background=#302d47"}},
    ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
        L"Margin=0,0,0,-2"}},
    ThemeTargetStyles{L"Grid#OverflowRootGrid > Border", {
        L"Background=#302d47"}},
}};

const Theme g_themeWinVista = {{
    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton", {
        L"CornerRadius=2"}},
    ThemeTargetStyles{L"Taskbar.SearchBoxButton", {
        L"CornerRadius=2"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid > Grid > Border,
SearchUx.SearchUI.SearchButtonRootGrid > Border", {
        L"CornerRadius=2"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton", {
        L"CornerRadius=2"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Rectangle#RunningIndicator,
Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Rectangle#RunningIndicator", {
        L"Height=2",
        L"Width@ActiveRunningIndicator=30",
        L"Width@InactiveRunningIndicator=8",
        L"Fill@ActiveRunningIndicator=#00BEE0",
        L"Fill@InactiveRunningIndicator=#DDDDDD"}},
    ThemeTargetStyles{L"Rectangle#BackgroundFill", {
        L"Fill:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"
Opacity=\"0.7\"><GradientStop Color=\"#B5B9BC\" Offset=\"0.0\" /><GradientStop
```

```
Color=\"#B5B9BC\" Offset=\"0.03125\" /><GradientStop Color=\"#909296\"
Offset=\"0.03125\" /><GradientStop Color=\"#464B51\" Offset=\"0.5\" /><GradientStop
Color=\"#060F15\" Offset=\"0.5\" /><GradientStop Color=\"#040C11\" Offset=\"0.96875\"
/><GradientStop Color=\"#000000\" Offset=\"0.96875\" /><GradientStop Color=\"#000000\"
Offset=\"1.0\" /></LinearGradientBrush>"}},
  ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Border,
Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Border", {
    L"Background@ActiveRunningIndicator:=<LinearGradientBrush StartPoint=\"0,0\"
EndPoint=\"0,1\" Opacity=\"0.2\"><GradientStop Color=\"#111111\" Offset=\"0.0\"
/><GradientStop Color=\"#111111\" Offset=\"1.0\" /></LinearGradientBrush>",
    L"CornerRadius=2",
    L"Background@RequestingAttentionRunningIndicator:=<LinearGradientBrush
StartPoint=\"0,0\" EndPoint=\"0,1\" Opacity=\"0.2\"><GradientStop Color=\"#D53300\"
Offset=\"0.0\" /><GradientStop Color=\"#111111\" Offset=\"1.0\" /></LinearGradientBrush>",
    L"BorderBrush=#33101010",
    L"BorderThickness=1",
    L"BorderBrush@NoRunningIndicator=Transparent",
    L"Background@NoRunningIndicator=Transparent",
    L"Background@ActiveRunningIndicator=#55BBBBBB",
    L"BorderBrush@ActiveRunningIndicator=#55212121"}},
  ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border#BackgroundElement,
Taskbar.TaskListLabeledButtonPanel@CommonStates > Border#BackgroundElement", {
    L"Margin=0,0,0,2",
    L"BorderThickness=1",
    L"Background@ActivePointerOver=#88DDDDDD",
    L"Background@ActiveNormal=#33BBBBBB",
    L"Background@InactivePointerOver=#33BBBBBB",
    L"BorderBrush@ActiveNormal=#44AAAAAA",
    L"BorderBrush@ActivePointerOver=#FF888888",
    L"BorderBrush@InactiveNormal=Transparent"}},
  ThemeTargetStyles{L"Grid#IconPanel > TextBlock, Taskbar.TaskListLabeledButtonPanel >
TextBlock", {
    L"FontFamily=Segoe UI"}},
  ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock#LanguageInnerTextBlock >
TextBlock#InnerTextBlock", {
    L"FontFamily=Segoe UI"}},
  ThemeTargetStyles{L"TextBlock#TimeInnerTextBlock", {
    L"FontFamily=Segoe UI"}},
  ThemeTargetStyles{L"Grid", {
    L"RequestedTheme=2"}},
  ThemeTargetStyles{L"Taskbar.TaskbarBackground#BackgroundControl > Grid", {
    L"Background:=<AcrylicBrush TintColor=\"Transparent\" TintOpacity=\"0\"
TintLuminosityOpacity=\"0.1\" Opacity=\"1\" />"}},
  ThemeTargetStyles{L"Border#MultiWindowElement", {
    L"Background=#BB212121",
    L"BorderThickness=0",
    L"Margin=0,2,1,4"}},
  ThemeTargetStyles{L"Grid#OverflowRootGrid > Border", {
    L"Background:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"
Opacity=\"0.7\"><GradientStop Color=\"#B5B9BC\" Offset=\"0.0\" /><GradientStop
Color=\"#B5B9BC\" Offset=\"0.03125\" /><GradientStop Color=\"#909296\"
Offset=\"0.03125\" /><GradientStop Color=\"#464B51\" Offset=\"0.5\" /><GradientStop
Color=\"#060F15\" Offset=\"0.5\" /><GradientStop Color=\"#040C11\" Offset=\"0.96875\"
/><GradientStop Color=\"#000000\" Offset=\"0.96875\" /><GradientStop Color=\"#000000\"
Offset=\"1.0\" /></LinearGradientBrush>"}},
  ThemeTargetStyles{L"Grid#OverflowRootGrid", {
    L"Background:=<AcrylicBrush TintColor=\"Transparent\" TintOpacity=\"0\"
TintLuminosityOpacity=\"0.1\" Opacity=\"1\" />",
    L"Padding=-1",
    L"Margin=0,6,0,6",
    L"CornerRadius=8"}},
}};

const Theme g_themeCleanSlate = {{
```

```
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground > Grid > Rectangle#BackgroundFill", {
        L"Fill:=<AcrylicBrush TintColor=\"{ThemeResource SystemAccentColorDark2}\" TintOpacity=\"0.4\" FallbackColor=\"{ThemeResource SystemAccentColorDark1}\" />"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel@CommonStates > Grid > Border#BackgroundElement, Taskbar.TaskListButtonPanel@CommonStates > Border#BackgroundElement", {
        L"CornerRadius=100",
        L"Background:=<AcrylicBrush TintColor=\"{ThemeResource SystemAccentColorDark2}\" TintOpacity=\"0.4\" FallbackColor=\"{ThemeResource SystemAccentColorDark2}\" />",
        L"Background@InactivePointerOver:=<AcrylicBrush TintColor=\"{ThemeResource SystemAccentColorDark1}\" TintOpacity=\"0.4\" FallbackColor=\"{ThemeResource SystemAccentColorDark2}\"/>",
        L"Background@ActivePointerOver:=<AcrylicBrush TintColor=\"{ThemeResource SystemAccentColorDark1}\" TintOpacity=\"0.6\" FallbackColor=\"{ThemeResource SystemAccentColorDark2}\" />",
        L"Background@ActiveNormal:=<AcrylicBrush TintColor=\"{ThemeResource SystemAccentColorDark1}\" TintOpacity=\"0.6\" FallbackColor=\"{ThemeResource SystemAccentColorDark2}\" />",
        L"Background@InactivePressed:=<AcrylicBrush TintColor=\"{ThemeResource SystemAccentColorDark1}\" TintOpacity=\"0.6\" FallbackColor=\"{ThemeResource SystemAccentColorDark2}\" />",
        L"Background@ActivePressed:=<AcrylicBrush TintColor=\"Black\" TintOpacity=\"0.8\" />"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"Background:=<AcrylicBrush TintColor=\"{ThemeResource SystemAccentColorDark2}\" TintOpacity=\"0.5\" FallbackColor=\"{ThemeResource SystemAccentColorDark2}\" />",
        L"CornerRadius=5",
        L"Margin=0,5,5,5",
        L"Padding=1,0,-10,0"}},
    ThemeTargetStyles{L"Rectangle#RunningIndicator", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"Grid#IconPanel > TextBlock#LabelControl, Taskbar.TaskListLabeledButtonPanel > TextBlock#LabelControl", {
        L"Margin=8,0,0,0",
        L"Foreground=White"}},
    ThemeTargetStyles{L"Taskbar.SearchBoxButton", {
        L"Padding=8",
        L"Margin=4,0,4,0"}},
    ThemeTargetStyles{L"TextBlock#SearchBoxTextBlock", {
        L"FontSize=12"}},
    ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock", {
        L"Foreground=White"}},

ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton[AutomationProperties.Name=Copilot] > Grid#IconPanel > Border#BackgroundElement, Taskbar.TaskListButton#TaskListButton[AutomationProperties.Name=Copilot] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border#BackgroundElement", {
        L"Background:=<AcrylicBrush TintColor=\"Black\" TintOpacity=\"0.8\" />"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView > Grid > Border#BackgroundBorder", {
        L"Margin=0,3,0,3"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid > Border#BackgroundElement@CommonStates, Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Border#BackgroundElement@CommonStates", {
        L"Background:=<AcrylicBrush TintColor=\"{ThemeResource SystemAccentColorDark2}\" TintOpacity=\"0.4\" FallbackColor=\"{ThemeResource SystemAccentColorDark2}\" />",
        L"CornerRadius=20",
        L"Margin=0,1,0,1"}},
    ThemeTargetStyles{L"TextBlock#TimeInnerTextBlock", {
        L"Foreground=White"}},
    ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
```

```
        L"Foreground=White"}},
    ThemeTargetStyles{L"SystemTray.TextIconContent > Grid > SystemTray.AdaptiveTextBlock#Base > TextBlock", {
        L"Foreground=White"}},
    ThemeTargetStyles{L"Border#BackgroundElement", {
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Border#BackgroundElement, Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Border#BackgroundElement", {
        L"Background@InactiveRunningIndicator:=<SolidColorBrush Color=\"Black\" Opacity=\"0.4\" />",
        L"Background@InactiveRunningIndicator:=<SolidColorBrush Color=\"Black\" Opacity=\"0.4\" />",
        L"Background@ActiveRunningIndicator:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorDark2}\" Opacity=\"0.4\" />",
        L"Background@RequestingAttentionRunningIndicator:=<SolidColorBrush Color=\"#ffdf5e\" Opacity=\"0.4\" />"}},
    ThemeTargetStyles{L"Rectangle#ShowDesktopPipe", {
        L"Grid.Row=0",
        L"Grid.RowSpan=3",
        L"Grid.Column=0",
        L"Grid.ColumnSpan=3",
        L"Width=12",
        L"Height=38",
        L"Margin=-6,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack", {
        L"Width=12"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel", {
        L"Margin=-3,0,0,0"}},
    ThemeTargetStyles{L"Grid#OverflowRootGrid > Border", {
        L"Background:=<AcrylicBrush TintColor=\"{ThemeResource SystemAccentColorDark2}\" TintOpacity=\"0.4\" FallbackColor=\"{ThemeResource SystemAccentColorDark1}\" />",
        L"BorderBrush:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColor}\" />"}},
    ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView > Grid > Border", {
        L"Background:=<AcrylicBrush TintColor=\"{ThemeResource SystemAccentColorDark3}\" TintOpacity=\"0.4\" FallbackColor=\"{ThemeResource SystemAccentColorDark1}\" />",
        L"BorderBrush:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColor}\" />",
        L"BorderThickness=1",
        L"CornerRadius=5"}},
    ThemeTargetStyles{L"Border#ProgressBarRoot > Border > Grid > Rectangle", {
        L"Fill:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight1}\" />"}},
    ThemeTargetStyles{L"Border#ProgressBarRoot > Border > Grid > Rectangle#ProgressBarTrack", {
        L"Fill:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorDark3}\" />"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border#BackgroundElement, Taskbar.TaskListLabeledButtonPanel@CommonStates > Border#BackgroundElement", {
        L"BorderBrush@InactivePointerOver:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight1}\" />",
        L"BorderBrush@ActiveNormal:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight2}\" />",
        L"BorderBrush@ActivePointerOver:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight1}\" />",
        L"BorderBrush@MultiWindowPointerOver:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight1}\" />",
        L"BorderThickness@ActiveNormal=1",
        L"BorderThickness@ActivePointerOver=1",
        L"BorderThickness@MultiWindowActive=2",
        L"BorderThickness@MultiWindowNormal=2",
        L"BorderThickness@MultiWindowPointerOver=2.5",
        L"BorderBrush@MultiWindowActive:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight1}\" />",
        L"BorderBrush@MultiWindowNormal:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight1}\" />",
```

```
        L"Margin@MultiWindowNormal=0",
        L"Margin@MultiWindowPointerOver=0",
        L"Margin@MultiWindowPressed=0"}},
    ThemeTargetStyles{L"ToolTip", {
        L"Background:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorDark2}\"
/>",
        L"Foreground=White"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border#MultiWindowElement,
Taskbar.TaskListLabeledButtonPanel@CommonStates > Border#MultiWindowElement", {
        L"Background=Transparent",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"Taskbar.SearchBoxButton >
Taskbar.TaskListButtonPanel@CommonStates > Grid > Border#BackgroundElement,
Taskbar.SearchBoxButton > Taskbar.TaskListButtonPanel@CommonStates >
Border#BackgroundElement", {
        L"CornerRadius=5",
        L"CornerRadius@InactiveNormal_SearchIcon=100",
        L"CornerRadius@InactivePointerOver_SearchIcon=100",
        L"CornerRadius@InactivePressed_SearchIcon=100",
        L"CornerRadius@ActiveNormal_SearchIcon=100",
        L"CornerRadius@ActivePointerOver_SearchIcon=100",
        L"CornerRadius@ActivePressed_SearchIcon=100",
        L"Background@InactiveNormal_SearchIcon:=<AcrylicBrush TintColor=\"{ThemeResource
SystemAccentColorDark2}\" TintOpacity=\"0.4\" FallbackColor=\"{ThemeResource
SystemAccentColorDark2}\" />",
        L"Background@InactivePointerOver_SearchIcon:=<AcrylicBrush
TintColor=\"{ThemeResource SystemAccentColorDark1}\" TintOpacity=\"0.4\"
FallbackColor=\"{ThemeResource SystemAccentColorDark1}\" />",
        L"Background@InactivePressed_SearchIcon:=<AcrylicBrush TintColor=\"{ThemeResource
SystemAccentColorDark1}\" TintOpacity=\"0.4\" FallbackColor=\"{ThemeResource
SystemAccentColorDark1}\" />",
        L"Background@ActiveNormal_SearchIcon:=<AcrylicBrush TintColor=\"{ThemeResource
SystemAccentColorDark2}\" TintOpacity=\"0.4\" FallbackColor=\"{ThemeResource
SystemAccentColorDark2}\" />",
        L"Background@ActivePointerOver_SearchIcon:=<AcrylicBrush
TintColor=\"{ThemeResource SystemAccentColorDark1}\" TintOpacity=\"0.4\"
FallbackColor=\"{ThemeResource SystemAccentColorDark1}\" />",
        L"Background@ActivePressed_SearchIcon:=<AcrylicBrush TintColor=\"{ThemeResource
SystemAccentColorDark1}\" TintOpacity=\"0.4\" FallbackColor=\"{ThemeResource
SystemAccentColorDark1}\" />"}},
    ThemeTargetStyles{L"Taskbar.OverflowToggleButton", {
        L"Margin=8,0,8,0",
        L"Background:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorDark2}\"
/>"}},
    ThemeTargetStyles{L"Rectangle#RightOverflowButtonDivider", {
        L"Fill:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight2}\"  />",
        L"Margin=8,4,-8,4"}},
    ThemeTargetStyles{L"Taskbar.OverflowToggleButton >
Taskbar.TaskListButtonPanel@CommonStates > Grid > Border, Taskbar.OverflowToggleButton >
Taskbar.TaskListButtonPanel@CommonStates > Border", {
        L"Background:=<AcrylicBrush TintColor=\"{ThemeResource SystemAccentColorDark2}\"
TintOpacity=\"0.4\" FallbackColor=\"{ThemeResource SystemAccentColorDark2}\" />",
        L"Background@InactivePointerOver:=<AcrylicBrush TintColor=\"{ThemeResource
SystemAccentColorDark1}\" TintOpacity=\"0.4\" FallbackColor=\"{ThemeResource
SystemAccentColorDark1}\" />"}},
    ThemeTargetStyles{L"Grid#IconPanel, Taskbar.TaskListLabeledButtonPanel", {
        L"Margin=0"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid@CommonStates > Grid >
Border#BackgroundElement, SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid@CommonStates > Border#BackgroundElement", {
        L"CornerRadius=100",
        L"Background:=<AcrylicBrush TintColor=\"{ThemeResource SystemAccentColorDark2}\"
TintOpacity=\"0.4\" FallbackColor=\"{ThemeResource SystemAccentColorDark2}\" />"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid", {
        L"Margin=0,0,3,0"}},
```

```
   ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid > Grid > Border#BackgroundElement, SearchUx.SearchUI.SearchButtonRootGrid > Border#BackgroundElement", {
      L"Background:=<AcrylicBrush TintColor=\"{ThemeResource SystemAccentColorDark2}\" TintOpacity=\"0.4\" FallbackColor=\"{ThemeResource SystemAccentColorDark2}\" />"}},
   ThemeTargetStyles{L"Border#SearchPillBackgroundElement", {
      L"BorderBrush:=<AcrylicBrush TintColor=\"{ThemeResource SystemAccentColorDark2}\" TintOpacity=\"0.4\" FallbackColor=\"{ThemeResource SystemAccentColorDark2}\" />",
      L"BorderThickness=16",
      L"CornerRadius=8"}},
}};

const Theme g_themeLucent = {{
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground > Grid > Rectangle#BackgroundFill", {
      L"Fill:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#00000000\" Offset=\"0.3\" /><GradientStop Color=\"#AA000000\" Offset=\"0.9\" /></LinearGradientBrush>"}},
   ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid > Rectangle#BackgroundFill", {
      L"Fill:=<LinearGradientBrush StartPoint=\"0,0.5\" EndPoint=\"0,1\"><GradientStop Color=\"#ee000000\" Offset=\"0.1\" /><GradientStop Color=\"{ThemeResource SystemAccentColorDark2}\" Offset=\"0.9\" /><GradientStop Color=\"#AAFFFFFF\" Offset=\"1.0\" /></LinearGradientBrush>"}},
   ThemeTargetStyles{L"Grid#IconPanel > Rectangle#RunningIndicator, Taskbar.TaskListLabeledButtonPanel > Rectangle#RunningIndicator", {
      L"Fill=Transparent"}},
   ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
      L"Visibility=Collapsed"}},
   ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Border#BackgroundElement, Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Border#BackgroundElement", {
      L"Background@InactiveRunningIndicator:=<LinearGradientBrush StartPoint=\"0,0.5\" EndPoint=\"0,1\"><GradientStop Color=\"#33002290c\" Offset=\"0.1\" /><GradientStop Color=\"{ThemeResource SystemAccentColorDark2}\" Offset=\"0.9\" /><GradientStop Color=\"#AAFFFFFF\" Offset=\"1.0\" /></LinearGradientBrush>",
      L"Margin=0,-1,0,-1"}},
   ThemeTargetStyles{L"Grid#IconPanel@CommonStates > TextBlock#LabelControl, Taskbar.TaskListLabeledButtonPanel@CommonStates > TextBlock#LabelControl", {
      L"Foreground@ActiveNormal=Black",
      L"Foreground@ActivePointerOver=Black",
      L"Margin=0,0,3,0",
      L"Foreground@ActivePressed=#BFBFBF"}},
   ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid", {
      L"Background:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#50000000\" Offset=\"0.3\" /><GradientStop Color=\"#EE000000\" Offset=\"0.9\" /></LinearGradientBrush>",
      L"Margin=0",
      L"CornerRadius=0"}},
   ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon", {
      L"Padding=2"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton] > Taskbar.TaskListButtonPanel", {
      L"CornerRadius=0",
      L"BorderThickness=0",
      L"MaxWidth=48",
      L"Margin=0",
      L"Padding=0",
      L"Background:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#80000000\" Offset=\"0.0\" /><GradientStop Color=\"#FF000000\" Offset=\"1.0\" /></LinearGradientBrush>"}},
   ThemeTargetStyles{L"Grid", {
      L"RequestedTheme=2"}},
   ThemeTargetStyles{L"Grid#OverflowRootGrid > Border", {
```

```
      L"Background:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop
Color=\"#50000000\" Offset=\"0.3\" /><GradientStop Color=\"#EE000000\" Offset=\"0.9\"
/></LinearGradientBrush>",
      L"CornerRadius=0"}},
   ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton", {
      L"Margin=10,0,-10,0"}},
   ThemeTargetStyles{L"Grid#IconPanel@CommonStates,
Taskbar.TaskListLabeledButtonPanel@CommonStates", {
      L"Background@ActiveNormal:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight3}\"/>",
      L"Background@ActivePointerOver:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight2}\"/>",
      L"Background@InactivePointerOver=#EBEBEB",
      L"Background@InactivePressed=#BBBBBB",
      L"Background@ActivePressed:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight1}\"/>",
      L"Background@RequestingAttention=#FFE9AFAA",
      L"Background@RequestingAttentionPointerOver=#FFF8E7E5",
      L"Background@RequestingAttentionPressed=#FFFEEEF0",
      L"Background@MultiWindowPointerOver:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight2}\"/>",
      L"Background@MultiWindowActive:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight3}\"/>",
      L"Background@MultiWindowPressed:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight1}\"/>"}},
   ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border,
Taskbar.TaskListLabeledButtonPanel@CommonStates > Border", {
      L"BorderThickness=0",
      L"Margin=-2,-4,-2,-4",
      L"CornerRadius=0"}},
   ThemeTargetStyles{L"Taskbar.TaskListButtonPanel@CommonStates > Grid > Border,
Taskbar.TaskListButtonPanel@CommonStates > Border", {
      L"CornerRadius=0",
      L"Background@InactivePointerOver:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight3}\"/>",
      L"Background@InactivePressed:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight2}\"/>",
      L"Background@ActiveNormal:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight3}\"/>",
      L"Background@ActivePointerOver:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight3}\"/>",
      L"Background@ActivePressed:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight1}\"/>",
      L"Margin=-3,-10,-3,-10",
      L"BorderThickness=0"}},
   ThemeTargetStyles{L"Grid#ContainerGrid@", {
      L"Background@PointerOver:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight3}\"/>",
      L"CornerRadius=0"}},
   ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid@CommonStates >
Border#BackgroundBorder", {
      L"Margin=0",
      L"Padding=0",
      L"CornerRadius=0",
      L"Background=Transparent",
      L"BorderThickness=0"}},
   ThemeTargetStyles{L"SystemTray.OmniButton > Grid@CommonStates", {
      L"Background@PointerOver:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight3}\"/>",
      L"Background@Pressed:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight1}\"/>",
      L"Background@Checked:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight2}\"/>",
      L"Background@CheckedPointerOver:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight3}\"/>",
```

```
      L"Background@CheckedPressed:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight1}\"/>"}},
   ThemeTargetStyles{L"Rectangle#ShowDesktopPipe", {
      L"Opacity=0"}},
   ThemeTargetStyles{L"SystemTray.ChevronIconView > Grid@", {
      L"Background@Pressed:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight1}\"/>",
      L"Background@CheckedPressed:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight1}\"/>",
      L"Background@PointerOver:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight3}\"/>",
      L"Background@CheckedNormal:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight2}\"/>",
      L"Background@CheckedPointerOver:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight3}\"/>"}},
   ThemeTargetStyles{L"SystemTray.Stack#NonActivatableStack > Grid >
SystemTray.StackListView > Windows.UI.Xaml.Controls.ItemsPresenter >
Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.ContentPresenter >
SystemTray.IconView > Grid@", {
      L"Background@Pressed:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight1}\"/>",
      L"Background@CheckedPressed:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight1}\"/>",
      L"Background@PointerOver:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight2}\"/>",
      L"Background@CheckedNormal:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight2}\"/>",
      L"Background@CheckedPointerOver:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight3}\"/>"}},
   ThemeTargetStyles{L"Grid#OverflowRootGrid > Windows.UI.Xaml.Controls.ItemsControl >
Windows.UI.Xaml.Controls.ItemsPresenter > Windows.UI.Xaml.Controls.WrapGrid >
Windows.UI.Xaml.Controls.ContentPresenter > SystemTray.NotifyIconView > Grid@", {
      L"CornerRadius=0",
      L"Margin=2,-5,2,-5"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.
AutomationId=StartButton] > Taskbar.TaskListButtonPanel > Grid > Border,
Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartB
utton] > Taskbar.TaskListButtonPanel > Border", {
      L"Width=48"}},
   ThemeTargetStyles{L"Grid#IconPanel > Border#MultiWindowElement,
Taskbar.TaskListLabeledButtonPanel > Border#MultiWindowElement", {
      L"Height=0"}},
   ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Rectangle,
Taskbar.TaskListLabeledButtonPanel@CommonStates > Rectangle", {
      L"StrokeThickness=3",
      L"Stroke@MultiWindowNormal:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight3}\"/>",
      L"Stroke@MultiWindowPointerOver:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight2}\"/>",
      L"Stroke@MultiWindowActive:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight1}\"/>",
      L"Stroke@MultiWindowPressed:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight1}\"/>",
      L"Width=Auto",
      L"RadiusX=0",
      L"Margin=-2,0,-2,-3"}},
   ThemeTargetStyles{L"SystemTray.ChevronIconView > Grid > Border", {
      L"BorderThickness=0",
      L"Background=Transparent"}},
   ThemeTargetStyles{L"SystemTray.OmniButton > Grid > Border", {
      L"BorderThickness=0",
      L"Background=Transparent"}},
   ThemeTargetStyles{L"Taskbar.SearchBoxButton >
Taskbar.TaskListButtonPanel@CommonStates", {
      L"Background@InactivePointerOver_SearchIcon=#BEBEBE",
      L"Background@InactivePressed_SearchIcon=#DDDDDD",
```

```
        L"Background@ActiveNormal_SearchIcon=#BEBEBE",
        L"Background@ActivePointerOver_SearchIcon=#DDDDDD",
        L"Background@ActivePressed_SearchIcon=#EEEEEE"}},

ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel@CommonStates", {
        L"Background@InactivePointerOver=#BEBEBE",
        L"Background@InactivePressed=#DDDDDD",
        L"Background@ActiveNormal=#BEBEBE",
        L"Background@ActivePointerOver=#DDDDDD",
        L"Background@ActivePressed=#EEEEEE"}},

ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Grid > Border, SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Border", {
        L"CornerRadius=0",
        L"Margin=-3,-10,-3,-10",
        L"BorderThickness=0"}},
}};

const Theme g_themeLucent_variant_Light = {{
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground > Grid > Rectangle#BackgroundFill", {
        L"Fill:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#00000000\" Offset=\"0.3\" /><GradientStop Color=\"#AA000000\" Offset=\"0.9\" /></LinearGradientBrush>"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid > Rectangle#BackgroundFill", {
        L"Fill:=<LinearGradientBrush StartPoint=\"0,0.5\" EndPoint=\"0,1\"><GradientStop Color=\"#ee000000\" Offset=\"0.1\" /><GradientStop Color=\"#EBEBEB\" Offset=\"0.9\" /><GradientStop Color=\"#AAFFFFFF\" Offset=\"1.0\" /></LinearGradientBrush>"}},
    ThemeTargetStyles{L"Grid#IconPanel > Rectangle#RunningIndicator, Taskbar.TaskListLabeledButtonPanel > Rectangle#RunningIndicator", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Border#BackgroundElement, Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Border#BackgroundElement", {
        L"Background@InactiveRunningIndicator:=<LinearGradientBrush StartPoint=\"0,0.5\" EndPoint=\"0,1\"><GradientStop Color=\"#33000000\" Offset=\"0.1\" /><GradientStop Color=\"#33EBEBEB\" Offset=\"0.9\" /><GradientStop Color=\"#AAFFFFFF\" Offset=\"1.0\" /></LinearGradientBrush>",
        L"Margin=0,-1,0,-1"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > TextBlock#LabelControl, Taskbar.TaskListLabeledButtonPanel@CommonStates > TextBlock#LabelControl", {
        L"Foreground@ActiveNormal=Black",
        L"Foreground@ActivePointerOver=Black",
        L"Margin=0,0,3,0",
        L"Foreground@ActivePressed=#424242"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid", {
        L"Background:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#50000000\" Offset=\"0.3\" /><GradientStop Color=\"#EE000000\" Offset=\"0.9\" /></LinearGradientBrush>",
        L"Margin=0",
        L"CornerRadius=0"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon", {
        L"Padding=2"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton] > Taskbar.TaskListButtonPanel", {
        L"CornerRadius=0",
        L"BorderThickness=0",
        L"MaxWidth=48",
        L"Margin=0",
```

```
      L"Padding=0",
      L"Background:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#80000000\" Offset=\"0.0\" /><GradientStop Color=\"#FF000000\" Offset=\"1.0\" /></LinearGradientBrush>"}},
   ThemeTargetStyles{L"Grid", {
      L"RequestedTheme=2"}},
   ThemeTargetStyles{L"Grid#OverflowRootGrid > Border", {
      L"Background:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#50000000\" Offset=\"0.3\" /><GradientStop Color=\"#EE000000\" Offset=\"0.9\" /></LinearGradientBrush>",
      L"CornerRadius=0"}},
   ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton", {
      L"Margin=10,0,-10,0"}},
   ThemeTargetStyles{L"Grid#IconPanel@CommonStates, Taskbar.TaskListLabeledButtonPanel@CommonStates", {
      L"Background@ActiveNormal=#FCFCFC",
      L"Background@ActivePointerOver=#BBBBBB",
      L"Background@InactivePointerOver=#BBBBBB",
      L"Background@InactivePressed=#EBEBEB",
      L"Background@ActivePressed=#EBEBEB",
      L"Background@RequestingAttention=#FFE9AFAA",
      L"Background@RequestingAttentionPointerOver=#FFF8E7E5",
      L"Background@RequestingAttentionPressed=#FFFEEEF0",
      L"Background@MultiWindowPointerOver=#BBBBBB",
      L"Background@MultiWindowActive=#BBBBBB",
      L"Background@MultiWindowPressed=#EBEBEB"}},
   ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border, Taskbar.TaskListLabeledButtonPanel@CommonStates > Border", {
      L"BorderThickness=0",
      L"Margin=-2,-4,-2,-4",
      L"CornerRadius=0"}},
   ThemeTargetStyles{L"Taskbar.TaskListButtonPanel@CommonStates > Grid > Border, Taskbar.TaskListButtonPanel@CommonStates > Border", {
      L"CornerRadius=0",
      L"Background@InactivePointerOver=#BBBBBB",
      L"Background@InactivePressed=#EBEBEB",
      L"Background@ActiveNormal=#BBBBBB",
      L"Background@ActivePointerOver=#BBBBBB",
      L"Background@ActivePressed=#EBEBEB",
      L"Margin=-3,-10,-3,-10",
      L"BorderThickness=0"}},
   ThemeTargetStyles{L"Grid#ContainerGrid@", {
      L"Background@PointerOver=#BBBBBB",
      L"CornerRadius=0"}},
   ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid@CommonStates > Border#BackgroundBorder", {
      L"Background@PointerOver=#BBBBBB",
      L"Margin=0",
      L"Padding=0",
      L"CornerRadius=0"}},
   ThemeTargetStyles{L"SystemTray.OmniButton > Grid@CommonStates", {
      L"Background@PointerOver=#BBBBBB",
      L"Background@Pressed=#EBEBEB",
      L"Background@Checked=#BBBBBB",
      L"Background@CheckedPointerOver=#BBBBBB",
      L"Background@CheckedPressed=#EBEBEB"}},
   ThemeTargetStyles{L"Rectangle#ShowDesktopPipe", {
      L"Opacity=0"}},
```

```
   ThemeTargetStyles{L"SystemTray.ChevronIconView > Grid@", {
      L"Background@Pressed=#EBEBEB",
      L"Background@CheckedPressed=#EBEBEB",
      L"Background@PointerOver=#BBBBBB",
      L"Background@CheckedNormal=#BBBBBB",
      L"Background@CheckedPointerOver=#EBEBEB"}},
   ThemeTargetStyles{L"SystemTray.Stack#NonActivatableStack > Grid >
SystemTray.StackListView > Windows.UI.Xaml.Controls.ItemsPresenter >
Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.ContentPresenter >
SystemTray.IconView > Grid@", {
      L"Background@Pressed=#EBEBEB",
      L"Background@CheckedPressed=#EBEBEB",
      L"Background@PointerOver=#BBBBBB",
      L"Background@CheckedNormal=#BBBBBB",
      L" Background@CheckedPointerOver=#EBEBEB"}},
   ThemeTargetStyles{L"Grid#OverflowRootGrid > Windows.UI.Xaml.Controls.ItemsControl >
Windows.UI.Xaml.Controls.ItemsPresenter > Windows.UI.Xaml.Controls.WrapGrid >
Windows.UI.Xaml.Controls.ContentPresenter > SystemTray.NotifyIconView > Grid@", {
      L"CornerRadius=0",
      L"Margin=2,-5,2,-5"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.
AutomationId=StartButton] > Taskbar.TaskListButtonPanel > Grid > Border,
Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartB
utton] > Taskbar.TaskListButtonPanel > Border", {
      L"Width=48"}},
   ThemeTargetStyles{L"Grid#IconPanel > Border#MultiWindowElement,
Taskbar.TaskListLabeledButtonPanel > Border#MultiWindowElement", {
      L"Height=0"}},
   ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Rectangle,
Taskbar.TaskListLabeledButtonPanel@CommonStates > Rectangle", {
      L"StrokeThickness=3",
      L"Stroke@MultiWindowNormal=#BBBBBB",
      L"Stroke@MultiWindowPointerOver=#EBEBEB",
      L"Stroke@MultiWindowPressed=#EBEBEB",
      L"Stroke@MultiWindowActive=#EBEBEB",
      L"Width=Auto",
      L"RadiusX=0",
      L"Margin=-2,0,-2,-3"}},
   ThemeTargetStyles{L"SystemTray.ChevronIconView > Grid > Border", {
      L"BorderThickness=0",
      L"Background=Transparent"}},
   ThemeTargetStyles{L"SystemTray.OmniButton > Grid > Border", {
      L"BorderThickness=0",
      L"Background=Transparent"}},
   ThemeTargetStyles{L"Taskbar.SearchBoxButton >
Taskbar.TaskListButtonPanel@CommonStates", {
      L"Background@InactivePointerOver_SearchIcon=#BEBEBE",
      L"Background@ActiveNormal_SearchIcon=#BEBEBE",
      L"Background@ActivePointerOver_SearchIcon=#EBEBEB"}},

ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel@C
ommonStates", {
      L"Background@InactivePointerOver=#BEBEBE",
      L"Background@ActiveNormal=#BEBEBE",
      L"Background@ActivePointerOver=#EBEBEB"}},

ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Grid > Border, SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Border",
{
      L"CornerRadius=0",
      L"Margin=-3,-10,-3,-10",
```

```
        L"BorderThickness=0"}},
};

const Theme g_themeSunValley = {{
    ThemeTargetStyles{L"Taskbar.SearchBoxButton#SearchBoxButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundElement, Taskbar.SearchBoxButton#SearchBoxButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"CornerRadius=4",
        L"BorderThickness=0,1,0,0"}},
    ThemeTargetStyles{L"Taskbar.SearchBoxButton", {
        L"Margin=0"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid > Grid > SystemTray.TextIconContent > Windows.UI.Xaml.Controls.Grid > SystemTray.AdaptiveTextBlock#Base > Windows.UI.Xaml.Controls.TextBlock", {
        L"Text=\u200E \u200E\u200E\u200E\uE7E7",
        L"FontSize=17.3",
        L"Width=30",
        L"FontWeight=ExtraLight",
        L"Foreground:=<SolidColorBrush Color=\"{ThemeResource SystemBaseHighColor}\" />"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.FontIcon#SearchBoxFontIcon", {
        L"FontFamily=Segoe Fluent Icons"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchBoxButton#SearchBox > SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.TextBlock#SearchBoxTextBlock, SearchUx.SearchUI.SearchBoxButton#SearchBox > SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Windows.UI.Xaml.Controls.TextBlock#SearchBoxTextBlock", {
        L"Text=Type here to search",
        L"FontSize=15",
        L"FontFamily=Segoe UI Variable Text",
        L"Margin=35,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon", {
        L"CornerRadius=3",
        L"Height=Auto",
        L"Margin=0,0,0,0",
        L"Padding=0,4,0,2",
        L"BorderThickness=0,1,0,0"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView", {
        L"CornerRadius=3",
        L"Height=Auto",
        L"Width=24",
        L"BorderThickness=0,1,0,0",
        L"Padding=0,2,0,2"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Button#GleamEntryPointButton > Windows.UI.Xaml.Controls.Border", {
        L"CornerRadius=4"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#DynamicSearchBoxGleamContainer", {
        L"CornerRadius=3"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton", {
        L"CornerRadius=3",
        L"Padding=0,2,0,2",
        L"Margin=0,0,5,0",
        L"BorderThickness=0,1,0,0"}},
    ThemeTargetStyles{L"Grid#SystemTrayFrameGrid > SystemTray.Stack#NonActivatableStack, StackPanel#SystemTrayFrameGrid > SystemTray.Stack#NonActivatableStack", {
        L"Height=Auto",
```

```
        L"CornerRadius=3",
        L"Margin=0,0,0,0",
        L"Padding=0,2,0,2",
        L"BorderThickness=0,1,0,0"}},
    ThemeTargetStyles{L"Grid#SystemTrayFrameGrid > SystemTray.Stack#NonActivatableStack", {
        L"Grid.Column=4"}},
    ThemeTargetStyles{L"Rectangle#ShowDesktopPipe@CommonStates", {
        L"Grid.Row=0",
        L"Grid.RowSpan=3",
        L"Grid.Column=0",
        L"Grid.ColumnSpan=3",
        L"Width=9",
        L"Margin=0,0,-10,0",
        L"Height=500",
        L"Fill@Active:=<AcrylicBrush TintColor=\"{ThemeResource SystemBaseLowColor}\"
TintOpacity=\"0.5\" Opacity=\"0\"/>",
        L"Stroke:=<SolidColorBrush Color=\"{ThemeResource SystemBaseHighColor}\"
Opacity=\"0.3\"/>"}},
    ThemeTargetStyles{L"Grid#SystemTrayFrameGrid >
SystemTray.OmniButton#ControlCenterButton, StackPanel#SystemTrayFrameGrid >
SystemTray.OmniButton#ControlCenterButton", {
        L"Padding=0,2,0,2",
        L"CornerRadius=3",
        L"Margin=0,0,0,0",
        L"BorderThickness=0,1,0,0"}},
    ThemeTargetStyles{L"Grid#SystemTrayFrameGrid >
SystemTray.OmniButton#ControlCenterButton", {
        L"Grid.Column=3"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock#LanguageInnerTextBlock >
TextBlock#InnerTextBlock", {
        L"FontFamily=Segoe UI Variable Text",
        L"Margin=-8,0,0,0",
        L"FontSize=12"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame >
Windows.UI.Xaml.Controls.StackPanel#SystemTrayFrameGrid >
SystemTray.Stack#NotifyIconStack > Windows.UI.Xaml.Controls.Grid#Content >
SystemTray.StackListView#IconStack > Windows.UI.Xaml.Controls.ItemsPresenter >
Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.ContentPresenter >
SystemTray.ChevronIconView > Windows.UI.Xaml.Controls.Grid#ContainerGrid >
Windows.UI.Xaml.Controls.ContentPresenter#ContentPresenter >
Windows.UI.Xaml.Controls.Grid#ContentGrid > SystemTray.TextIconContent >
Windows.UI.Xaml.Controls.Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base >
Windows.UI.Xaml.Controls.TextBlock#InnerTextBlock, SystemTray.SystemTrayFrame >
Windows.UI.Xaml.Controls.Grid#SystemTrayFrameGrid > SystemTray.Stack#NotifyIconStack >
Windows.UI.Xaml.Controls.Grid#Content > SystemTray.StackListView#IconStack >
Windows.UI.Xaml.Controls.ItemsPresenter > Windows.UI.Xaml.Controls.StackPanel >
Windows.UI.Xaml.Controls.ContentPresenter > SystemTray.ChevronIconView >
Windows.UI.Xaml.Controls.Grid#ContainerGrid >
Windows.UI.Xaml.Controls.ContentPresenter#ContentPresenter >
Windows.UI.Xaml.Controls.Grid#ContentGrid > SystemTray.TextIconContent >
Windows.UI.Xaml.Controls.Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base >
Windows.UI.Xaml.Controls.TextBlock#InnerTextBlock", {
        L"FontFamily=Segoe Fluent Icons",
        L"FontSize=12.4",
        L"Foreground:=<SolidColorBrush Color=\"{ThemeResource SystemBaseHighColor}\" />"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame >
Windows.UI.Xaml.Controls.StackPanel#SystemTrayFrameGrid >
SystemTray.Stack#NotifyIconStack > Windows.UI.Xaml.Controls.Grid#Content >
SystemTray.StackListView#IconStack > Windows.UI.Xaml.Controls.ItemsPresenter >
Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.ContentPresenter,
SystemTray.SystemTrayFrame > Windows.UI.Xaml.Controls.Grid#SystemTrayFrameGrid >
SystemTray.Stack#NotifyIconStack > Windows.UI.Xaml.Controls.Grid#Content >
SystemTray.StackListView#IconStack > Windows.UI.Xaml.Controls.ItemsPresenter >
Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.ContentPresenter", {
        L"Width=30"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock#Base >
Windows.UI.Xaml.Controls.TextBlock#InnerTextBlock", {
        L"FontFamily=Segoe Fluent Icons",
        L"Foreground:=<SolidColorBrush Color=\"{ThemeResource SystemBaseHighColor}\" />"}},
```

```
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock#AccentOverlay > Windows.UI.Xaml.Controls.TextBlock#InnerTextBlock", {
        L"FontFamily=Segoe Fluent Icons"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock#Underlay > Windows.UI.Xaml.Controls.TextBlock#InnerTextBlock", {
        L"FontFamily=Segoe Fluent Icons"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[1] > SystemTray.IconView > Grid > Grid", {
        L"Margin=-3,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.Stack#MainStack > Windows.UI.Xaml.Controls.Grid#Content", {
        L"CornerRadius=3",
        L"Height=Auto",
        L"Margin=0,0,0,0",
        L"Padding=0,4,0,4",
        L"BorderThickness=0,1,0,0"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.TextBlock#TimeInnerTextBlock", {
        L"FontFamily=Segoe UI Variable Display",
        L"TextAlignment=0",
        L"FontSize=12",
        L"Margin=0,1,0,0"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.TextBlock#DateInnerTextBlock", {
        L"FontFamily=Segoe UI Variable",
        L"TextAlignment=0",
        L"FontSize=12",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.NotificationAreaIcons#NotificationAreaIcons > Windows.UI.Xaml.Controls.ItemsPresenter > Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.ContentPresenter", {
        L"Width=23",
        L"Margin=0,-2,0,0"}},
    ThemeTargetStyles{L"SystemTray.NotificationAreaIcons#NotificationAreaIcons > Windows.UI.Xaml.Controls.ItemsPresenter > Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.ContentPresenter > SystemTray.NotifyIconView#NotifyItemIcon > Windows.UI.Xaml.Controls.Grid#ContainerGrid", {
        L"Transform3D:=<CompositeTransform3D TranslateY=\"0\" TranslateX=\"0\" />",
        L"HorizontalAlignment=0"}},
    ThemeTargetStyles{L"SystemTray.Stack#NotifyIconStack", {
        L"Width=24"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid > Grid > SystemTray.TextIconContent > Windows.UI.Xaml.Controls.Grid > SystemTray.AdaptiveTextBlock > Windows.UI.Xaml.Controls.TextBlock", {
        L"FontSize=16",
        L"Margin=0,-1,-0,0",
        L"FontWeight=0"}},
    ThemeTargetStyles{L"SystemTray.CopilotIcon#CopilotIcon", {
        L"Visibility=Visible",
        L"Padding=2",
        L"Height=61"}},
    ThemeTargetStyles{L"SystemTray.NotificationAreaOverflow > Windows.UI.Xaml.Controls.Grid#OverflowRootGrid > Windows.UI.Xaml.Controls.Border#OverflowFlyoutBackgroundBorder", {
        L"CornerRadius=7",
        L"Background:=<AcrylicBrush TintColor=\"{ThemeResource SystemChromeMediumHighColor}\" TintOpacity=\"0\" TintLuminosityOpacity=\"0.8\" FallbackColor=\"{ThemeResource SystemChromeMediumColor}\" />",
        L"Margin=0,0,0,0",
        L"BorderBrush:=<SolidColorBrush Color=\"{ThemeResource SystemChromeHighColor}\" Opacity=\"0.5\" />",
```

```
        L"BorderThickness=1"}},
    ThemeTargetStyles{L"SystemTray.NotificationAreaOverflow > Windows.UI.Xaml.Controls.Grid#OverflowRootGrid > Windows.UI.Xaml.Controls.ItemsControl > Windows.UI.Xaml.Controls.ItemsPresenter > Windows.UI.Xaml.Controls.WrapGrid", {
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView", {
        L"CornerRadius=3"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.ScrollViewer > Windows.UI.Xaml.Controls.ScrollContentPresenter > Windows.UI.Xaml.Controls.Border > SystemTray.NotificationAreaOverflow", {
        L"Transform3D:=<CompositeTransform3D TranslateY=\"0\" />"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton", {
        L"Visibility=Visible"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#BackgroundControl > Grid", {
        L"Background:=<AcrylicBrush TintColor=\"{ThemeResource SystemChromeAltHighColor}\" TintOpacity=\"0.4\" TintLuminosityOpacity=\"0.4\" FallbackColor=\"{ThemeResource SystemChromeMediumColor}\" />"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#BackgroundControl > Grid > Windows.UI.Xaml.Shapes.Rectangle#BackgroundFill", {
        L"Opacity=0.5"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#BackgroundStroke", {
        L"Opacity=0.5"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[3] > SystemTray.IconView > Grid > Grid", {
        L"Margin=2,0,-4,0",
        L"RenderTransform:=<ScaleTransform ScaleX=\"1\" />"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.ContentPresenter#HoverFlyoutContent", {
        L"CornerRadius=7",
        L"Margin=0,0,0,0",
        L"Background:=<AcrylicBrush TintColor=\"{ThemeResource SystemChromeMediumHighColor}\" TintOpacity=\"0\" TintLuminosityOpacity=\"0.8\" FallbackColor=\"{ThemeResource SystemChromeMediumColor}\" />",
        L"BorderBrush:=<SolidColorBrush Color=\"{ThemeResource SystemChromeHighColor}\" Opacity=\"0.5\" />",
        L"BorderThickness=1"}},
    ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView > Grid > TextBlock", {
        L"FontFamily=Segoe UI",
        L"FontSize=12",
        L"Margin=3,0,8,8"}},
    ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView > Windows.UI.Xaml.Controls.Grid > Microsoft.UI.Xaml.Controls.ItemsRepeater > Windows.UI.Xaml.Controls.Image", {
        L"Margin=0,-7,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView > Grid > Button > ContentPresenter > TextBlock", {
        L"FontFamily=Segoe Fluent Icons"}},
    ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView > Grid > Button", {
        L"CornerRadius=4",
        L"Height=31",
        L"Margin=0,0,0,8",
        L"Width=31"}},
    ThemeTargetStyles{L"Grid#DetailedViewGrid", {
        L"Margin=0,-7,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView > Grid > Border", {
        L"BorderBrush:=<SolidColorBrush Color=\"{ThemeResource SystemBaseHighColor}\" Opacity=\"0.5\" />",
        L"CornerRadius=0"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.ContentPresenter > Windows.UI.Xaml.Controls.ItemsPresenter > Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.ContentPresenter > SystemTray.IconView", {
        L"CornerRadius=0",
        L"Padding=0,0,0,0"}},
```

```
    ThemeTargetStyles{L"SystemTray.DateTimeIconContent > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.TextBlock", {
        L"FontFamily=Segoe UI",
        L"TextAlignment=Center"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[2] > SystemTray.IconView > Grid > Grid", {
        L"Margin=0,0,-3,0"}},
    ThemeTargetStyles{L"Taskbar.ThumbBarButton > ContentPresenter", {
        L"CornerRadius=4",
        L"Background:=<SolidColorBrush Color=\"{ThemeResource SystemChromeMediumHighColor}\" Opacity=\"0.3\" />",
        L"BorderBrush:=<SolidColorBrush Color=\"{ThemeResource SystemChromeHighColor}\" Opacity=\"0.3\" />",
        L"BorderThickness=1"}},
    ThemeTargetStyles{L"Taskbar.ThumbBarButton > ContentPresenter > Image", {
        L"Height=15",
        L"Width=15"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid > Grid > Border, SearchUx.SearchUI.SearchButtonRootGrid > Border", {
        L"CornerRadius=4"}},
    ThemeTargetStyles{L"TextBlock#BatteryTextBlock", {
        L"FontFamily=Segoe UI Variable Text",
        L"Margin=2,0,-2,0"}},
    ThemeTargetStyles{L"SystemTray.BatteryIconContent > Grid > Windows.UI.Xaml.Controls.StackPanel > Grid > TextBlock", {
        L"RenderTransform:=<ScaleTransform ScaleX=\"1\" />"}},
    ThemeTargetStyles{L"SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Border#BackgroundBorder", {
        L"CornerRadius=0",
        L"Opacity=0"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#GleamEntryPointButton > Windows.UI.Xaml.Controls.Border > Windows.UI.Xaml.Controls.ContentPresenter", {
        L"CornerRadius=3"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid", {
        L"Margin=2,0,2,0"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView > Grid > ContentPresenter", {
        L"Transform3D:=<CompositeTransform3D TranslateY=\"-1\" TranslateX=\"2\" />",
        L"HorizontalAlignment=0"}},
    ThemeTargetStyles{L"Border#SearchPillBackgroundElement", {
        L"CornerRadius=4"}},
    ThemeTargetStyles{L"Grid#ConfirmatorMainGrid", {
        L"Background:=<AcrylicBrush TintColor=\"{ThemeResource SystemChromeMediumHighColor}\" TintOpacity=\"0\" TintLuminosityOpacity=\"0.8\" FallbackColor=\"{ThemeResource SystemChromeMediumColor}\" />",
        L"BorderBrush:=<SolidColorBrush Color=\"{ThemeResource SystemChromeHighColor}\" Opacity=\"0.5\" />",
        L"CornerRadius=6",
        L"BorderThickness=1"}},
    ThemeTargetStyles{L"Slider > Grid > Grid > Grid > Windows.UI.Xaml.Controls.Primitives.Thumb#HorizontalThumb", {
        L"Visibility=Visible",
        L"Height=18",
        L"Width=18"}},
    ThemeTargetStyles{L"Slider > Grid > Grid > Grid > Windows.UI.Xaml.Controls.Primitives.Thumb#HorizontalThumb > Border", {
        L"BorderBrush:=<AcrylicBrush TintColor=\"{ThemeResource SystemChromeLowColor}\" TintOpacity=\"0.7\" TintLuminosityOpacity=\"0.7\" FallbackColor=\"{ThemeResource SystemChromeMediumColor}\" />",
        L"BorderThickness=4",
        L"Background:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight1}\" />",
```

```
        L"CornerRadius=10"}},
    ThemeTargetStyles{L"Slider > Grid > Grid > Grid > Rectangle#HorizontalDecreaseRect", {
        L"Fill:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight1}\" />"}},
    ThemeTargetStyles{L"Slider > Grid > Grid > Grid > Windows.UI.Xaml.Controls.Primitives.Thumb#HorizontalThumb", {
        L"BorderBrush:=<AcrylicBrush TintColor=\"{ThemeResource SystemChromeHighColor}\" TintOpacity=\"0.7\" TintLuminosityOpacity=\"0.7\" FallbackColor=\"{ThemeResource SystemChromeMediumColor}\" />",
        L"BorderThickness=0,1,0,0"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchBoxButton", {
        L"Width=340"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchBoxButton > SearchUx.SearchUI.SearchButtonRootGrid", {
        L"Width=340",
        L"Margin=-5,-6,-4,-6"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchBoxButton > SearchUx.SearchUI.SearchButtonRootGrid > Grid", {
        L"Margin=0,-1,-1,-1",
        L"Width=80",
        L"Transitions:=<TransitionCollection>            <ContentThemeTransition/> </TransitionCollection>"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchBoxButton > SearchUx.SearchUI.SearchButtonRootGrid > Grid > Grid > Image", {
        L"Width=80",
        L"Height=40"}},

    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton] > Taskbar.TaskListButtonPanel", {
        L"Width=48",
        L"Background:=<ImageBrush Stretch=\"Uniform\" ImageSource=\"https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/SunValley/Assets/start.png\" />"}},

    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton] > Taskbar.TaskListButtonPanel > Grid > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon, Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton] > Taskbar.TaskListButtonPanel > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"ToolTip > ContentPresenter > TextBlock", {
        L"FontFamily=Segoe UI Variable Text",
        L"FontSize=13",
        L"Margin=0,1,0,-2"}},
    ThemeTargetStyles{L"ToolTip > ContentPresenter > StackPanel > TextBlock", {
        L"FontFamily=Segoe UI Variable Text",
        L"FontSize=13",
        L"Margin=0,1,0,-2"}},
    ThemeTargetStyles{L"ToolTip", {
        L"Background:=<AcrylicBrush TintColor=\"{ThemeResource SystemChromeMediumHighColor}\" TintOpacity=\"0\" TintLuminosityOpacity=\"0.8\" FallbackColor=\"{ThemeResource SystemChromeMediumColor}\" />",
        L"BorderBrush:=<SolidColorBrush Color=\"{ThemeResource SystemChromeHighColor}\" Opacity=\"0.5\" />",
        L"BorderThickness=1"}},
    ThemeTargetStyles{L"ToolTip > ContentPresenter", {
        L"Transitions:=<TransitionCollection>            <ContentThemeTransition VerticalOffset=\"60\" />        </TransitionCollection>"}},

    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton[AutomationProperties.AutomationId=WidgetsButton] > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
        L"Padding=0,2,0,2"}},
    ThemeTargetStyles{L"TextBlock#VirtualDesktopNameBlock", {
        L"FontFamily=Segoe UI Variable Text",
```

```
        L"FontSize=13"}},
    ThemeTargetStyles{L"Grid#RootGrid > Grid#TitleGrid > TextBlock#DisplayName", {
        L"FontFamily=Segoe UI Variable Text",
        L"FontSize=13"}},
    ThemeTargetStyles{L"Grid#MainGrid > TextBlock", {
        L"FontFamily=Segoe UI Variable Text",
        L"FontSize=13"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#BackgroundDimmingLayer", {
        L"Background:=<AcrylicBrush TintColor=\"{ThemeResource
SystemChromeMediumHighColor}\" TintOpacity=\"0\" TintLuminosityOpacity=\"0.7\"
FallbackColor=\"{ThemeResource SystemChromeMediumColor}\" />"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.
AutomationId=TaskViewButton] > Taskbar.TaskListButtonPanel > Grid >
Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon,
Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=TaskVi
ewButton] > Taskbar.TaskListButtonPanel >
Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon", {
        L"RenderTransform:=<ScaleTransform ScaleX=\"1.1\" ScaleY=\"0.9\" />",
        L"Transform3D:=<CompositeTransform3D TranslateY=\"2\" TranslateX=\"-2\" />",
        L"FlowDirection=1"}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab >
Windows.UI.Xaml.Controls.Grid#ModalRootGrid >
Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"Background=Transparent",
        L"Transitions:=<TransitionCollection>            <ContentThemeTransition VerticalOffset=\"-
1000\" />        </TransitionCollection>"}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab >
Windows.UI.Xaml.Controls.Grid#ModalRootGrid >
Windows.UI.Xaml.Controls.Border#BackgroundElement >
WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemList", {
        L"Background:=<AcrylicBrush TintColor=\"{ThemeResource
SystemChromeMediumHighColor}\" TintOpacity=\"0.2\" TintLuminosityOpacity=\"0.9\"
FallbackColor=\"{ThemeResource SystemChromeMediumColor}\" />"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchBoxButton >
SearchUx.SearchUI.SearchButtonRootGrid > Grid >
Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer, SearchUx.SearchUI.SearchBoxButton >
SearchUx.SearchUI.SearchButtonRootGrid > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer",
{
        L"Height=18",
        L"Width=18",
        L"Transform3D:=<CompositeTransform3D TranslateX=\"0\" />"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchPillButton >
SearchUx.SearchUI.SearchButtonRootGrid", {
        L"Margin=-3,-6,-3,-6",
        L"Width=346"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchPillButton >
SearchUx.SearchUI.SearchButtonRootGrid > Grid > Grid#SearchBoxContentGrid,
SearchUx.SearchUI.SearchPillButton > SearchUx.SearchUI.SearchButtonRootGrid >
Grid#SearchBoxContentGrid", {
        L"HorizontalAlignment=0"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchPillButton >
SearchUx.SearchUI.SearchButtonRootGrid > Grid > Grid#SearchBoxContentGrid > FontIcon,
SearchUx.SearchUI.SearchPillButton > SearchUx.SearchUI.SearchButtonRootGrid >
Grid#SearchBoxContentGrid > FontIcon", {
        L"Transform3D:=<CompositeTransform3D TranslateY=\"-1\" TranslateX=\"-10.5\" />",
        L"FontSize=19.4",
        L"FontFamily=Segoe Fluent Icons",
        L"FontWeight=SemiLight",
        L"Opacity=0.7",
        L"Grid.Column=0",
        L"HorizontalAlignment=0"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchPillButton >
SearchUx.SearchUI.SearchButtonRootGrid > Grid > Grid#SearchBoxContentGrid > TextBlock,
SearchUx.SearchUI.SearchPillButton > SearchUx.SearchUI.SearchButtonRootGrid >
Grid#SearchBoxContentGrid > TextBlock", {
        L"Transform3D:=<CompositeTransform3D TranslateX=\"-5\" />",
        L"FontFamily=Segoe UI",
```

```
        L"Opacity=0.7",
        L"Text=Type here to search",
        L"Grid.Column=1",
        L"HorizontalAlignment=0",
        L"FontSize=15"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchPillButton > SearchUx.SearchUI.SearchButtonRootGrid > Grid > Border#SearchPillBackgroundElement, SearchUx.SearchUI.SearchPillButton > SearchUx.SearchUI.SearchButtonRootGrid > Border#SearchPillBackgroundElement", {
        L"CornerRadius=4",
        L"BorderThickness=1",
        L"BorderBrush:=<SolidColorBrush Color=\"{ThemeResource SystemChromeHighColor}\" Opacity=\"0.7\" />"}},
    ThemeTargetStyles{L"Grid#SearchBoxContentGrid", {
        L"Visibility=Visible"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton > Grid#IconPanel, Taskbar.TaskListButton#TaskListButton > Taskbar.TaskListLabeledButtonPanel#IconPanel", {
        L"Padding=2",
        L"Margin=2,0,2,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton > Grid#IconPanel > Rectangle, Taskbar.TaskListButton > Taskbar.TaskListLabeledButtonPanel > Rectangle", {
        L"RadiusX=2",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton > Taskbar.TaskListButtonPanel", {
        L"Padding=2",
        L"Margin=1,0,2,0"}},
    ThemeTargetStyles{L"MenuFlyoutPresenter", {
        L"CornerRadius=7"}},
    ThemeTargetStyles{L"MenuFlyoutPresenter > Border", {
        L"Background:=<AcrylicBrush TintColor=\"{ThemeResource SystemChromeMediumHighColor}\" TintOpacity=\"0.3\" TintLuminosityOpacity=\"0.8\" FallbackColor=\"{ThemeResource SystemChromeMediumColor}\" />",
        L"Transitions:=<TransitionCollection>            <ContentThemeTransition VerticalOffset=\"100\" />        </TransitionCollection>",
        L"BorderThickness=1",
        L"BorderBrush:=<SolidColorBrush Color=\"{ThemeResource SystemChromeHighColor}\" Opacity=\"0.5\" />"}},
    ThemeTargetStyles{L"Grid#IconPanel > Image#OverlayIcon, Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#OverlayIcon", {
        L"Transform3D:=<CompositeTransform3D TranslateY=\"15\" TranslateX=\"-2\" />"}},
    ThemeTargetStyles{L"Grid#IconPanel > Taskbar.Badge, Taskbar.TaskListLabeledButtonPanel#IconPanel > Taskbar.Badge", {
        L"Transform3D:=<CompositeTransform3D TranslateY=\"15\" TranslateX=\"-2\" />"}},
    ThemeTargetStyles{L"Grid#IconPanel > Taskbar.Badge > Grid > Rectangle, Taskbar.TaskListLabeledButtonPanel#IconPanel > Taskbar.Badge > Grid > Rectangle", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"Grid#IconPanel > Taskbar.Badge > Grid, Taskbar.TaskListLabeledButtonPanel#IconPanel > Taskbar.Badge > Grid", {
        L"Background:=<AcrylicBrush TintColor=\"{ThemeResource SystemChromeMediumHighColor}\" TintOpacity=\"0\" TintLuminosityOpacity=\"0,7\" FallbackColor=\"{ThemeResource SystemChromeMediumColor}\" />",
        L"BorderBrush:=<AcrylicBrush TintColor=\"{ThemeResource SystemChromeHighColor}\" TintOpacity=\"0\" TintLuminosityOpacity=\"0.5\" FallbackColor=\"{ThemeResource SystemChromeMediumColor}\" />",
        L"BorderThickness=0,1,0,0",
        L"CornerRadius=15"}},
    ThemeTargetStyles{L"Grid#IconPanel > Taskbar.Badge > Grid > TextBlock, Taskbar.TaskListLabeledButtonPanel#IconPanel > Taskbar.Badge > Grid > TextBlock", {
        L"Foreground:=<SolidColorBrush Color=\"{ThemeResource SystemBaseHighColor}\" />",
        L"Transform3D:=<CompositeTransform3D TranslateY=\"-1\" />"}},
    ThemeTargetStyles{L"Grid#IconPanel > Taskbar.Badge, Taskbar.TaskListLabeledButtonPanel#IconPanel > Taskbar.Badge", {
        L"Height=17",
        L"Width=17"}},
```

```
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchIconButton > SearchUx.SearchUI.SearchButtonRootGrid", {
        L"Padding=0,2,2,2",
        L"Width=48"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchBoxButton > SearchUx.SearchUI.SearchButtonRootGrid > Grid > TextBlock, SearchUx.SearchUI.SearchBoxButton > SearchUx.SearchUI.SearchButtonRootGrid > TextBlock", {
        L"Transform3D:=<CompositeTransform3D TranslateX=\"5\" TranslateY=\"1\" />"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid#SearchV2ButtonRootPanel", {
        L"Padding=0,2,0,2"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchV2Button", {
        L"CornerRadius=4",
        L"Width=346"}},
    ThemeTargetStyles{L"Grid#SearchV2ButtonInactiveUIGrid", {
        L"MaxWidth=346",
        L"MinWidth=346"}},
    ThemeTargetStyles{L"Grid#SearchV2ButtonActiveUIGridWithAnimations", {
        L"Width=346"}},
    ThemeTargetStyles{L"Grid#SearchV2ButtonActiveUIGridWithAnimations > StackPanel", {
        L"HorizontalAlignment=0",
        L"Margin=13,0,0,0"}},
    ThemeTargetStyles{L"Grid#SearchV2ButtonInactiveUIGrid > Button", {
        L"Height=40",
        L"Width=30",
        L"CornerRadius=4"}},
    ThemeTargetStyles{L"TextBlock#SearchV2OnTaskbarButtonText", {
        L"FontFamily=Segoe UI",
        L"FontSize=15",
        L"Transform3D:=<CompositeTransform3D TranslateY=\"-1\" />",
        L"Text=Ask me anything"}},

    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=TaskViewButton] > Taskbar.TaskListButtonPanel > Grid > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon, Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=TaskViewButton] > Taskbar.TaskListButtonPanel > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon", {
        L"Visibility=Collapsed"}},

    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=TaskViewButton] > Taskbar.TaskListButtonPanel@CommonStates", {
        L"Background:=<ImageBrush Stretch=\"Uniform\" ImageSource=\"https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/SunValley/Assets/taskview.png\" />",
        L"Width=48"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchIconButton > SearchUx.SearchUI.SearchButtonRootGrid > Grid > Grid#SearchBoxContentGrid > FontIcon > Grid > TextBlock, SearchUx.SearchUI.SearchIconButton > SearchUx.SearchUI.SearchButtonRootGrid > Grid#SearchBoxContentGrid > FontIcon > Grid > TextBlock", {
        L"FontWeight=Light"}},
    ThemeTargetStyles{L"SystemTray.NotificationAreaOverflow > Windows.UI.Xaml.Controls.Grid#OverflowRootGrid", {
        L"Margin=7,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton > Grid#IconPanel > Border#BackgroundElement, Taskbar.TaskListButton#TaskListButton > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border#BackgroundElement", {
        L"Margin=-3,0,-3,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton > Grid#IconPanel@RunningIndicatorStates > Rectangle, Taskbar.TaskListButton#TaskListButton > Taskbar.TaskListLabeledButtonPanel#IconPanel@RunningIndicatorStates > Rectangle", {
        L"Width@ActiveRunningIndicator=40",
        L"RadiusX=2",
        L"RadiusY=2",
```

```
        L"Width@InactiveRunningIndicator=35"}},
    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton > Taskbar.TaskListButtonPanel > Grid >
Border, Taskbar.ExperienceToggleButton > Taskbar.TaskListButtonPanel > Border", {
        L"Margin=-3,0,-3,0"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid > Grid > Border,
SearchUx.SearchUI.SearchIconButton > SearchUx.SearchUI.SearchButtonRootGrid > Border", {
        L"Margin=-1,0,-1,0",
        L"Opacity=0.5"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton >
Grid#IconPanel@CommonStates > Border#MultiWindowElement,
Taskbar.TaskListButton#TaskListButton >
Taskbar.TaskListLabeledButtonPanel#IconPanel@CommonStates >
Border#MultiWindowElement", {
        L"Margin=-4,0,-4,0",
        L"Opacity=0.5"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid >
Border, Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Border", {
        L"Opacity=0.5"}},
    ThemeTargetStyles{L"Grid#IconPanel > Border#BackgroundElement,
Taskbar.TaskListLabeledButtonPanel > Border#BackgroundElement", {
        L"Opacity=0.5"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchPillButton >
SearchUx.SearchUI.SearchButtonRootGrid > Grid > Grid#SearchBoxContentGrid,
SearchUx.SearchUI.SearchPillButton > SearchUx.SearchUI.SearchButtonRootGrid >
Grid#SearchBoxContentGrid", {
        L"Width=344"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchBoxButton >
SearchUx.SearchUI.SearchButtonRootGrid@CommonStates > Grid > Border,
SearchUx.SearchUI.SearchBoxButton >
SearchUx.SearchUI.SearchButtonRootGrid@CommonStates > Border", {
        L"BorderBrush@InactiveNormal_SearchBox_Wave3:=<SolidColorBrush
Color=\"{ThemeResource SystemChromeHighColor}\" Opacity=\"0.7\" />",
        L"Background@InactiveNormal_SearchBox_Wave3:=<SolidColorBrush
Color=\"{ThemeResource SystemBaseHighColor}\" Opacity=\"0.06\" />",
        L"Background@InactivePointerOver_SearchBox_Wave3:=<SolidColorBrush
Color=\"{ThemeResource SystemChromeHighColor}\" Opacity=\"0.2\" />",
        L"Background@InactivePressed_SearchBox_Wave3:=<SolidColorBrush
Color=\"{ThemeResource SystemChromeHighColor}\" Opacity=\"0.05\" />",
        L"BorderThickness@InactivePointerOver_SearchBox_Wave3=2",
        L"BorderThickness@InactivePressed_SearchBox_Wave3=2",
        L"BorderThickness@InactiveNormal_SearchBox_Wave3=1",
        L"Background@ActivePointerOver_SearchBox_Wave3:=<SolidColorBrush
Color=\"{ThemeResource SystemChromeHighColor}\" Opacity=\"0\" />",
        L"Background@ActivePressed_SearchBox_Wave3:=<SolidColorBrush
Color=\"{ThemeResource SystemChromeHighColor}\" Opacity=\"0\" />",
        L"Background@ActiveNormal_SearchBox_Wave3:=<SolidColorBrush
Color=\"{ThemeResource SystemChromeHighColor}\" Opacity=\"0\" />",
        L"BorderBrush@InactivePointerOver_SearchBox_Wave3:=<SolidColorBrush
Color=\"{ThemeResource SystemChromeHighColor}\" Opacity=\"0.7\" />",
        L"BorderBrush@InactivePressed_SearchBox_Wave3:=<SolidColorBrush
Color=\"{ThemeResource SystemChromeHighColor}\" Opacity=\"0.7\" />",
        L"BorderThickness@InactivePointerOver_SearchBoxCustomTheme=2",
        L"BorderThickness@InactivePressed_SearchBoxCustomTheme=2",
        L"BorderThickness@InactiveNormal_SearchBoxCustomTheme=1",
        L"BorderBrush@InactiveNormal_SearchBoxCustomTheme:=<SolidColorBrush
Color=\"{ThemeResource SystemChromeHighColor}\" Opacity=\"0.7\" />",
        L"BorderBrush@InactivePointerOver_SearchBoxCustomTheme:=<SolidColorBrush
Color=\"{ThemeResource SystemChromeHighColor}\" Opacity=\"0.7\" />",
        L"BorderBrush@InactivePressed_SearchBoxCustomTheme:=<SolidColorBrush
Color=\"{ThemeResource SystemChromeHighColor}\" Opacity=\"0.7\" />",
        L"Background@InactivePointerOver_SearchBoxCustomTheme:=<SolidColorBrush
Color=\"White\" Opacity=\"1\" />",
        L"Background@InactiveNormal_SearchBoxCustomTheme:=<SolidColorBrush
Color=\"White\" Opacity=\"0.9\" />",
        L"Background@InactivePressed_SearchBoxCustomTheme:=<SolidColorBrush
Color=\"White\" Opacity=\"0.7\" />"}},
}};
```

```
const Theme g_theme21996Taskbar = {{
  ThemeTargetStyles{L"Taskbar.SearchBoxButton#SearchBoxButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundElement, Taskbar.SearchBoxButton#SearchBoxButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Windows.UI.Xaml.Controls.Border#BackgroundElement", {
    L"CornerRadius=4",
    L"BorderThickness=0"}},
  ThemeTargetStyles{L"Taskbar.SearchBoxButton", {
    L"Margin=0"}},
  ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid > Grid > SystemTray.TextIconContent > Windows.UI.Xaml.Controls.Grid > SystemTray.AdaptiveTextBlock > Windows.UI.Xaml.Controls.TextBlock", {
    L"Visibility=Visible",
    L"Text=\u200E \u200E\u200E\u200E\uE91C",
    L"FontSize=16.4",
    L"FontFamily=Segoe MDL2 Assets",
    L"Width=30",
    L"FontWeight=ExtraLight",
    L"Foreground:=<SolidColorBrush Color=\"{ThemeResource SystemBaseHighColor}\" />"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.FontIcon#SearchBoxFontIcon", {
    L"FontFamily=Segoe Fluent Icons"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.TextBlock#SearchBoxTextBlock", {
    L"Text=Type here to search",
    L"FontSize=14"}},
  ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon", {
    L"CornerRadius=0",
    L"Height=Auto",
    L"Margin=0,0,0,0",
    L"Padding=0",
    L"BorderThickness=0"}},
  ThemeTargetStyles{L"SystemTray.ChevronIconView", {
    L"CornerRadius=0",
    L"Height=Auto",
    L"Width=24",
    L"BorderThickness=0",
    L"Padding=0"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Button#GleamEntryPointButton > Windows.UI.Xaml.Controls.Border", {
    L"CornerRadius=4"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#DynamicSearchBoxGleamContainer", {
    L"CornerRadius=4"}},
  ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton", {
    L"CornerRadius=0",
    L"Padding=0,0,0,0",
    L"Margin=0,0,0,0",
    L"BorderThickness=0"}},
  ThemeTargetStyles{L"SystemTray.Stack#NonActivatableStack", {
    L"Height=Auto",
    L"CornerRadius=0",
    L"Margin=0,0,0,0",
    L"Padding=0",
    L"BorderThickness=0"}},
  ThemeTargetStyles{L"Rectangle#ShowDesktopPipe@CommonStates", {
    L"Grid.Row=0",
```

```
        L"Grid.RowSpan=3",
        L"Grid.Column=0",
        L"Grid.ColumnSpan=3",
        L"Width=9",
        L"Margin=0,0,-10,0",
        L"Height=500",
        L"Fill@Active:=<AcrylicBrush TintColor=\"{ThemeResource SystemBaseLowColor}\"
TintOpacity=\"0.5\" Opacity=\"0\"/>",
        L"Stroke:=<SolidColorBrush Color=\"{ThemeResource SystemBaseHighColor}\"
Opacity=\"0.5\"/>"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton", {
        L"Padding=0",
        L"CornerRadius=0",
        L"Margin=0,0,0,0",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock#LanguageInnerTextBlock >
TextBlock#InnerTextBlock", {
        L"FontFamily=Segoe UI",
        L"Margin=-8,0,0,0",
        L"FontSize=12"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame >
Windows.UI.Xaml.Controls.StackPanel#SystemTrayFrameGrid >
SystemTray.Stack#NotifyIconStack > Windows.UI.Xaml.Controls.Grid#Content >
SystemTray.StackListView#IconStack > Windows.UI.Xaml.Controls.ItemsPresenter >
Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.ContentPresenter >
SystemTray.ChevronIconView > Windows.UI.Xaml.Controls.Grid#ContainerGrid >
Windows.UI.Xaml.Controls.ContentPresenter#ContentPresenter >
Windows.UI.Xaml.Controls.Grid#ContentGrid > SystemTray.TextIconContent >
Windows.UI.Xaml.Controls.Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base >
Windows.UI.Xaml.Controls.TextBlock#InnerTextBlock, SystemTray.SystemTrayFrame >
Windows.UI.Xaml.Controls.Grid#SystemTrayFrameGrid > SystemTray.Stack#NotifyIconStack >
Windows.UI.Xaml.Controls.Grid#Content > SystemTray.StackListView#IconStack >
Windows.UI.Xaml.Controls.ItemsPresenter > Windows.UI.Xaml.Controls.StackPanel >
Windows.UI.Xaml.Controls.ContentPresenter > SystemTray.ChevronIconView >
Windows.UI.Xaml.Controls.Grid#ContainerGrid >
Windows.UI.Xaml.Controls.ContentPresenter#ContentPresenter >
Windows.UI.Xaml.Controls.Grid#ContentGrid > SystemTray.TextIconContent >
Windows.UI.Xaml.Controls.Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base >
Windows.UI.Xaml.Controls.TextBlock#InnerTextBlock", {
        L"FontFamily=Segoe MDL2 Assets",
        L"FontSize=12.4",
        L"Foreground:=<SolidColorBrush Color=\"{ThemeResource SystemBaseHighColor}\" />"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame >
Windows.UI.Xaml.Controls.StackPanel#SystemTrayFrameGrid >
SystemTray.Stack#NotifyIconStack > Windows.UI.Xaml.Controls.Grid#Content >
SystemTray.StackListView#IconStack > Windows.UI.Xaml.Controls.ItemsPresenter >
Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.ContentPresenter,
SystemTray.SystemTrayFrame > Windows.UI.Xaml.Controls.Grid#SystemTrayFrameGrid >
SystemTray.Stack#NotifyIconStack > Windows.UI.Xaml.Controls.Grid#Content >
SystemTray.StackListView#IconStack > Windows.UI.Xaml.Controls.ItemsPresenter >
Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.ContentPresenter", {
        L"Width=30"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock#Base >
Windows.UI.Xaml.Controls.TextBlock#InnerTextBlock", {
        L"FontFamily=Segoe MDL2 Assets",
        L"Foreground:=<SolidColorBrush Color=\"{ThemeResource SystemBaseHighColor}\" />"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock#AccentOverlay >
Windows.UI.Xaml.Controls.TextBlock#InnerTextBlock", {
        L"FontFamily=Segoe MDL2 Assets"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock#Underlay >
Windows.UI.Xaml.Controls.TextBlock#InnerTextBlock", {
        L"FontFamily=Segoe MDL2 Assets"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid >
ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[1] >
SystemTray.IconView > Grid > Grid", {
        L"Margin=-6,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.Stack#MainStack >
Windows.UI.Xaml.Controls.Grid#Content", {
        L"CornerRadius=0",
        L"Height=Auto",
```

```
        L"Margin=0,0,0,0",
        L"Padding=0",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.TextBlock#TimeInnerTextBlock", {
        L"FontFamily=Segoe UI",
        L"TextAlignment=0",
        L"FontSize=12",
        L"Margin=0,-1,0,0"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.TextBlock#DateInnerTextBlock", {
        L"FontFamily=Segoe UI",
        L"TextAlignment=0",
        L"FontSize=12",
        L"Margin=0,2,0,0"}},
    ThemeTargetStyles{L"SystemTray.NotificationAreaIcons#NotificationAreaIcons > Windows.UI.Xaml.Controls.ItemsPresenter > Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.ContentPresenter", {
        L"Width=23",
        L"Margin=0,-2,0,0"}},
    ThemeTargetStyles{L"SystemTray.NotificationAreaIcons#NotificationAreaIcons > Windows.UI.Xaml.Controls.ItemsPresenter > Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.ContentPresenter > SystemTray.NotifyIconView#NotifyItemIcon > Windows.UI.Xaml.Controls.Grid#ContainerGrid", {
        L"Transform3D:=<CompositeTransform3D TranslateY=\"0\" TranslateX=\"0\" />",
        L"HorizontalAlignment=0"}},
    ThemeTargetStyles{L"SystemTray.Stack#NotifyIconStack", {
        L"Width=24"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid > Grid > SystemTray.TextIconContent > Windows.UI.Xaml.Controls.Grid > SystemTray.AdaptiveTextBlock > Windows.UI.Xaml.Controls.TextBlock", {
        L"FontSize=16",
        L"Margin=0,-1,-0,0",
        L"FontWeight=0"}},
    ThemeTargetStyles{L"SystemTray.CopilotIcon#CopilotIcon", {
        L"Visibility=Visible",
        L"Margin=0,-7,0,0",
        L"Height=61"}},
    ThemeTargetStyles{L"SystemTray.NotificationAreaOverflow > Windows.UI.Xaml.Controls.Grid#OverflowRootGrid > Windows.UI.Xaml.Controls.Border#OverflowFlyoutBackgroundBorder", {
        L"CornerRadius=0",
        L"Background:=<AcrylicBrush TintColor=\"{ThemeResource SystemAltHighColor}\" TintOpacity=\"0.7\" FallbackColor=\"{ThemeResource SystemChromeMediumColor}\" />"}},
    ThemeTargetStyles{L"SystemTray.NotificationAreaOverflow > Windows.UI.Xaml.Controls.Grid#OverflowRootGrid > Windows.UI.Xaml.Controls.ItemsControl > Windows.UI.Xaml.Controls.ItemsPresenter > Windows.UI.Xaml.Controls.WrapGrid", {
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView", {
        L"CornerRadius=0"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.ScrollViewer > Windows.UI.Xaml.Controls.ScrollContentPresenter > Windows.UI.Xaml.Controls.Border > SystemTray.NotificationAreaOverflow", {
        L"Transform3D:=<CompositeTransform3D TranslateY=\"13\" />"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton", {
        L"Visibility=Visible"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#BackgroundControl > Grid", {
        L"Background:=<AcrylicBrush TintColor=\"{ThemeResource SystemChromeMediumHighColor}\" TintOpacity=\"0\" TintLuminosityOpacity=\"0.8\" FallbackColor=\"{ThemeResource SystemChromeMediumColor}\" />"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#BackgroundControl > Grid > Windows.UI.Xaml.Shapes.Rectangle#BackgroundFill", {
```

```
    L"Opacity=0.5"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#BackgroundStroke", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid >
ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[3] >
SystemTray.IconView > Grid > Grid", {
    L"Margin=2,0,-8,0",
    L"RenderTransform:=<ScaleTransform ScaleX=\"0.86\" />"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.ContentPresenter#HoverFlyoutContent", {
    L"CornerRadius=0",
    L"Margin=0,0,0,-15",
    L"Background:=<AcrylicBrush TintColor=\"{ThemeResource SystemAltHighColor}\"
TintOpacity=\"0.7\" FallbackColor=\"{ThemeResource SystemChromeMediumColor}\" />"}},
  ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView > Grid > TextBlock", {
    L"FontFamily=Segoe UI",
    L"FontSize=12",
    L"Margin=3,0,8,8"}},
  ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView > Windows.UI.Xaml.Controls.Grid >
Microsoft.UI.Xaml.Controls.ItemsRepeater > Windows.UI.Xaml.Controls.Image", {
    L"Margin=0,-7,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView > Grid > Button > ContentPresenter >
TextBlock", {
    L"FontFamily=Segoe MDL2 Assets"}},
  ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView > Grid > Button", {
    L"CornerRadius=0",
    L"Height=32",
    L"Margin=0,0,0,9",
    L"Width=32"}},
  ThemeTargetStyles{L"Grid#DetailedViewGrid", {
    L"Margin=0,-7,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView > Grid > Border", {
    L"BorderBrush:=<SolidColorBrush Color=\"{ThemeResource SystemBaseHighColor}\"
Opacity=\"0.5\" />",
    L"CornerRadius=0"}},
  ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton >
Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.ContentPresenter >
Windows.UI.Xaml.Controls.ItemsPresenter > Windows.UI.Xaml.Controls.StackPanel >
Windows.UI.Xaml.Controls.ContentPresenter > SystemTray.IconView", {
    L"CornerRadius=0",
    L"Padding=0,0,0,0"}},
  ThemeTargetStyles{L"SystemTray.DateTimeIconContent > Windows.UI.Xaml.Controls.Grid >
Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.TextBlock", {
    L"FontFamily=Segoe UI",
    L"TextAlignment=Center"}},
  ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid >
ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[2] >
SystemTray.IconView > Grid > Grid", {
    L"Margin=0,0,-3,0"}},
  ThemeTargetStyles{L"Taskbar.ThumbBarButton > ContentPresenter", {
    L"CornerRadius=0",
    L"Background:=<SolidColorBrush Opacity=\"0.5\" />",
    L"BorderBrush:=<AcrylicBrush TintColor=\"{ThemeResource SystemChromeLowColor}\"
FallbackColor=\"{ThemeResource SystemChromeLowColor}\" />"}},
  ThemeTargetStyles{L"Taskbar.ThumbBarButton > ContentPresenter > Image", {
    L"Height=15",
    L"Width=15"}},
  ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid > Grid > Border,
SearchUx.SearchUI.SearchButtonRootGrid > Border", {
    L"CornerRadius=4"}},
  ThemeTargetStyles{L"TextBlock#BatteryTextBlock", {
    L"FontFamily=Segoe UI",
```

```
        L"Margin=2,0,-2,0"}},
    ThemeTargetStyles{L"SystemTray.BatteryIconContent > Grid > Windows.UI.Xaml.Controls.StackPanel > Grid > TextBlock", {
        L"RenderTransform:=<ScaleTransform ScaleX=\"1\" />"}},
    ThemeTargetStyles{L"SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Border#BackgroundBorder", {
        L"CornerRadius=0",
        L"Opacity=0"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#GleamEntryPointButton > Windows.UI.Xaml.Controls.Border > Windows.UI.Xaml.Controls.ContentPresenter", {
        L"CornerRadius=3"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid", {
        L"Margin=2,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView > Grid > ContentPresenter", {
        L"Transform3D:=<CompositeTransform3D TranslateY=\"-1\" TranslateX=\"2\" />",
        L"HorizontalAlignment=0"}},
    ThemeTargetStyles{L"Border#SearchPillBackgroundElement", {
        L"CornerRadius=4"}},
    ThemeTargetStyles{L"Grid#ConfirmatorMainGrid", {
        L"Background:=<AcrylicBrush TintColor=\"{ThemeResource SystemChromeMediumHighColor}\" TintOpacity=\"0\" TintLuminosityOpacity=\"0.8\" FallbackColor=\"{ThemeResource SystemChromeMediumColor}\" />",
        L"BorderBrush:=<AcrylicBrush TintColor=\"{ThemeResource SystemChromeHighColor}\" TintOpacity=\"0\" TintLuminosityOpacity=\"0.3\" FallbackColor=\"{ThemeResource SystemChromeMediumColor}\" />",
        L"CornerRadius=6",
        L"BorderThickness=0,1,0,0"}},
    ThemeTargetStyles{L"Slider > Grid > Grid > Grid > Windows.UI.Xaml.Controls.Primitives.Thumb#HorizontalThumb", {
        L"Visibility=Visible",
        L"Height=18",
        L"Width=18"}},
    ThemeTargetStyles{L"Slider > Grid > Grid > Grid > Windows.UI.Xaml.Controls.Primitives.Thumb#HorizontalThumb > Border", {
        L"BorderBrush:=<AcrylicBrush TintColor=\"{ThemeResource SystemChromeLowColor}\" TintOpacity=\"0.7\" TintLuminosityOpacity=\"0.7\" FallbackColor=\"{ThemeResource SystemChromeMediumColor}\" />",
        L"BorderThickness=4",
        L"Background:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight1}\" />",
        L"CornerRadius=10"}},
    ThemeTargetStyles{L"Slider > Grid > Grid > Grid > Rectangle#HorizontalDecreaseRect", {
        L"Fill:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight1}\" />"}},
    ThemeTargetStyles{L"Slider > Grid > Grid > Grid > Windows.UI.Xaml.Controls.Primitives.Thumb#HorizontalThumb", {
        L"BorderBrush:=<AcrylicBrush TintColor=\"{ThemeResource SystemChromeHighColor}\" TintOpacity=\"0.7\" TintLuminosityOpacity=\"0.7\" FallbackColor=\"{ThemeResource SystemChromeMediumColor}\" />",
        L"BorderThickness=0,1,0,0"}},
}};

const Theme g_themeBottomDensy = {{
    ThemeTargetStyles{L"TaskbarTaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground > Grid > Rectangle#BackgroundFill", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Rectangle#RunningIndicator, Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Rectangle#RunningIndicator", {
        L"Fill=#8f8f8f",
        L"Fill@ActiveRunningIndicator=#fef9f0",
        L"Width=2",
```

```
        L"Height=2",
        L"Margin=0,-2,0,0",
        L"Width@ActiveRunningIndicator=32"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Microsoft.UI.Xaml.Controls.ProgressBar#ProgressIndicator, Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Microsoft.UI.Xaml.Controls.ProgressBar#ProgressIndicator", {
        L"VerticalAlignment=0"}},
    ThemeTargetStyles{L"Rectangle#RunningIndicator", {
        L"VerticalAlignment=0"}},
    ThemeTargetStyles{L"Border#ProgressBarRoot", {
        L"VerticalAlignment=0"}},
    ThemeTargetStyles{L"Rectangle#IndeterminateProgressBarIndicator", {
        L"VerticalAlignment=0"}},
    ThemeTargetStyles{L"Rectangle#IndeterminateProgressBarIndicator2", {
        L"VerticalAlignment=0"}},
    ThemeTargetStyles{L"Grid#IconPanel, Taskbar.TaskListLabeledButtonPanel", {
        L"Padding=2,0,2,0",
        L"VerticalAlignment=2"}},

    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton]", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack", {
        L"Width=1"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#ShowDesktopPipe", {
        L"HorizontalAlignment=0"}},
    ThemeTargetStyles{L"SystemTray.NotificationAreaIcons#NotificationAreaIcons > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.NotifyIconView#NotifyItemIcon > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.ImageIconContent > Grid#ContainerGrid > Image", {
        L"Width=20",
        L"Height=20"}},
    ThemeTargetStyles{L"WrapGrid > ContentPresenter > SystemTray.NotifyIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.ImageIconContent > Grid#ContainerGrid > Image", {
        L"Width=20",
        L"Height=20"}},
    ThemeTargetStyles{L"SystemTray.NotificationAreaOverflow > Grid#OverflowRootGrid", {
        L"Padding=0,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.NotificationAreaOverflow > Grid#OverflowRootGrid > Border#OverflowFlyoutBackgroundBorder", {
        L"BorderThickness=0,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.NotificationAreaOverflow > Grid#OverflowRootGrid > ItemsControl", {
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"SystemSettings.View.RootPage > Grid#RootPageGrid > Grid#RootCommandSearchGrid > SystemSettings.View.SettingsAutoSuggestCommandSearchBox#CommandSearchBox", {
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > SystemTray.NotificationAreaOverflow > Grid#OverflowRootGrid > Border#OverflowFlyoutBackgroundBorder", {
        L"CornerRadius=0"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas > Grid > Grid#ConfirmatorMainGrid", {
        L"BorderThickness=0",
        L"CornerRadius=0",
        L"MinHeight=32"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas > Grid", {
        L"Padding=0,0,0,0"}},
```

```
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas > Grid > Grid#ConfirmatorMainGrid > Grid#VolumeConfirmator", {
        L"Padding=0,0,0,0"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas > Grid > Grid#ConfirmatorMainGrid > Grid#VolumeConfirmator > Button", {
        L"Margin=0,0,2,0",
        L"Height=16",
        L"Width=16"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas > Grid > Grid#ConfirmatorMainGrid", {
        L"MinHeight=16"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas > Grid > Grid#ConfirmatorMainGrid > Grid#VolumeConfirmator > Button > ContentPresenter#ContentPresenter > Microsoft.UI.Xaml.Controls.AnimatedIcon#VolumeIcon", {
        L"Padding=0,0,0,0"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas > Grid > Grid#ConfirmatorMainGrid > Grid#VolumeConfirmator > Slider#volumeSlider > Grid > Grid#SliderContainer > Grid#HorizontalTemplate", {
        L"RowDefinitions:=<RowDefinitionCollection><RowDefinition Height=\"2\"/><RowDefinition Height=\"Auto\"/><RowDefinition Height=\"2\"/></RowDefinitionCollection>"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas > Grid > Grid#ConfirmatorMainGrid > Grid#VolumeConfirmator > TextBlock#volumeLevelText", {
        L"MinWidth=24",
        L"Margin=0,0,0,0",
        L"VerticalAlignment=1"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas", {
        L"VerticalAlignment=1"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas > Grid > Grid#ConfirmatorMainGrid > Grid#VolumeConfirmator", {
        L"Width:=$volume_bar_width_outer_152"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas > Grid > Grid#ConfirmatorMainGrid > Grid#VolumeConfirmator > Slider#volumeSlider", {
        L"Width:=$volume_bar_width_inner_110"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas > Grid > Grid#ConfirmatorMainGrid", {
        L"MaxWidth=4096"}},
}, {
    L"volume_bar_width_outer_152=132",
    L"volume_bar_width_inner_110=90",
}};

const Theme g_themeBottomDensy_variant_NoInd = {{
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground > Grid > Rectangle#BackgroundFill", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Rectangle#RunningIndicator, Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Rectangle#RunningIndicator", {
        L"Fill=#8f8f8f",
        L"Fill@ActiveRunningIndicator=#fef9f0",
        L"Width=0",
        L"Height=0",
        L"Margin=0,0,0,0",
```

```
        L"Width@ActiveRunningIndicator=32",
        L"Height@ActiveRunningIndicator=2",
        L"Margin@ActiveRunningIndicator=0,-2,0,0"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Microsoft.UI.Xaml.Controls.ProgressBar#ProgressIndicator, Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Microsoft.UI.Xaml.Controls.ProgressBar#ProgressIndicator", {
        L"VerticalAlignment=0"}},
    ThemeTargetStyles{L"Rectangle#RunningIndicator", {
        L"VerticalAlignment=0"}},
    ThemeTargetStyles{L"Border#ProgressBarRoot", {
        L"VerticalAlignment=0"}},
    ThemeTargetStyles{L"Rectangle#IndeterminateProgressBarIndicator", {
        L"VerticalAlignment=0"}},
    ThemeTargetStyles{L"Rectangle#IndeterminateProgressBarIndicator2", {
        L"VerticalAlignment=0"}},
    ThemeTargetStyles{L"Grid#IconPanel, Taskbar.TaskListLabeledButtonPanel", {
        L"Padding=2,0,2,0",
        L"VerticalAlignment=2"}},

    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton]", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack", {
        L"Width=1"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#ShowDesktopPipe", {
        L"HorizontalAlignment=0"}},
    ThemeTargetStyles{L"SystemTray.NotificationAreaIcons#NotificationAreaIcons > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.NotifyIconView#NotifyItemIcon > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.ImageIconContent > Grid#ContainerGrid > Image", {
        L"Width=20",
        L"Height=20"}},
    ThemeTargetStyles{L"WrapGrid > ContentPresenter > SystemTray.NotifyIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.ImageIconContent > Grid#ContainerGrid > Image", {
        L"Width=20",
        L"Height=20"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Image#Icon, Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Image#Icon", {
        L"Width@ActiveRunningIndicator=30",
        L"Height@ActiveRunningIndicator=30",
        L"Width@NoRunningIndicator=26",
        L"Height@NoRunningIndicator=26",
        L"Margin@NoRunningIndicator=0,6,0,0"}},
    ThemeTargetStyles{L"SystemTray.NotificationAreaOverflow > Grid#OverflowRootGrid", {
        L"Padding=0,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.NotificationAreaOverflow > Grid#OverflowRootGrid > Border#OverflowFlyoutBackgroundBorder", {
        L"BorderThickness=0,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.NotificationAreaOverflow > Grid#OverflowRootGrid > ItemsControl", {
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"SystemSettings.View.RootPage > Grid#RootPageGrid > Grid#RootCommandSearchGrid > SystemSettings.View.SettingsAutoSuggestCommandSearchBox#CommandSearchBox", {
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > SystemTray.NotificationAreaOverflow > Grid#OverflowRootGrid > Border#OverflowFlyoutBackgroundBorder", {
        L"CornerRadius=0"}},
```

```
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas > Grid > Grid#ConfirmatorMainGrid", {
        L"BorderThickness=0",
        L"CornerRadius=0",
        L"MinHeight=32"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas > Grid", {
        L"Padding=0,0,0,0"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas > Grid > Grid#ConfirmatorMainGrid > Grid#VolumeConfirmator", {
        L"Padding=0,0,0,0"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas > Grid > Grid#ConfirmatorMainGrid > Grid#VolumeConfirmator > Button", {
        L"Margin=0,0,2,0",
        L"Height=16",
        L"Width=16"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas > Grid > Grid#ConfirmatorMainGrid", {
        L"MinHeight=16"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas > Grid > Grid#ConfirmatorMainGrid > Grid#VolumeConfirmator > Button > ContentPresenter#ContentPresenter > Microsoft.UI.Xaml.Controls.AnimatedIcon#VolumeIcon", {
        L"Padding=0,0,0,0"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas > Grid > Grid#ConfirmatorMainGrid > Grid#VolumeConfirmator > Slider#volumeSlider > Grid > Grid#SliderContainer > Grid#HorizontalTemplate", {
        L"RowDefinitions:=<RowDefinitionCollection><RowDefinition Height=\"2\"/><RowDefinition Height=\"Auto\"/><RowDefinition Height=\"2\"/></RowDefinitionCollection>"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas > Grid > Grid#ConfirmatorMainGrid > Grid#VolumeConfirmator > TextBlock#volumeLevelText", {
        L"MinWidth=24",
        L"Margin=0,0,0,0",
        L"VerticalAlignment=1"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas", {
        L"VerticalAlignment=1"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas > Grid > Grid#ConfirmatorMainGrid > Grid#VolumeConfirmator", {
        L"Width:=$volume_bar_width_outer_152"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas > Grid > Grid#ConfirmatorMainGrid > Grid#VolumeConfirmator > Slider#volumeSlider", {
        L"Width:=$volume_bar_width_inner_110"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > HWConfirmatorUI.ConfirmatorHostControl > Canvas#ConfirmatorFrameCanvas > Grid > Grid#ConfirmatorMainGrid", {
        L"MaxWidth=4096"}},
}, {
    L"volume_bar_width_outer_152=132",
    L"volume_bar_width_inner_110=90",
}};

const Theme g_themeTaskbarXII = {{
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > Grid", {
        L"Background:=<AcrylicBrush TintColor=\"{ThemeResource SystemListLowColor}\" TintOpacity=\"0.1\" FallbackColor=\"{ThemeResource SystemChromeHighColor}\" />"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid@DockingStates", {
```

```
        L"Tag=horizontal",
        L"Tag@DockedLeft=vertical",
        L"Tag@DockedRight=vertical",
        L"Tag=>taskbarDock"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
        L"HorizontalAlignment=Right",
        L"Width={{taskbarDock==`vertical`?skip():`Auto`}}",
        L"Height={{taskbarDock==`vertical`?skip():56}}",
        L"Grid.Column=0",
        L"Margin=0,0,2,0"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid", {
        L"Height=48",
        L"CornerRadius=4"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#BackgroundControl", {
        L"Height=48",
        L"Opacity=0.7"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground > Grid", {
        L"CornerRadius=4",
        L"Opacity=1"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater", {
        L"Margin=0,0,3,0"}},
    ThemeTargetStyles{L"Taskbar.SearchBoxButton > Taskbar.TaskListButtonPanel,
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel", {
        L"Margin=2,0,6,0"}},
    ThemeTargetStyles{L"TextBlock#SearchBoxTextBlock", {
        L"Text=\u2726 Meow"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#BackgroundStroke", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Border#LargeTicker1", {
        L"Margin=0,2,4,0"}},
    ThemeTargetStyles{L"Border#LargeTicker1 >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Image", {
        L"MaxHeight=27",
        L"MaxWidth=27"}},
    ThemeTargetStyles{L"Border#LargeTicker1 >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel >
Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer", {
        L"MaxHeight=27",
        L"MaxWidth=27"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
        L"HorizontalAlignment=Left",
        L"VerticalAlignment=Center",
        L"Grid.Column=1",
        L"Margin=2,0,0,0"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"Background:=<SolidColorBrush Color=\"{ThemeResource SystemChromeAltHighColor}\"
Opacity=\"0.6\" />",
        L"CornerRadius=4",
        L"Padding=8,3,0,3"}},
    ThemeTargetStyles{L"Grid#SystemTrayFrameGrid > SystemTray.Stack#SecondaryClockStack", {
        L"Grid.Column=8"}},
    ThemeTargetStyles{L"Grid#SystemTrayFrameGrid >
SystemTray.OmniButton#ControlCenterButton", {
        L"Grid.Column=4"}},
    ThemeTargetStyles{L"Grid#SystemTrayFrameGrid >
SystemTray.OmniButton#NotificationCenterButton", {
        L"Grid.Column=5"}},
    ThemeTargetStyles{L"Grid#SystemTrayFrameGrid > SystemTray.Stack#MainStack", {
```

```
    L"Grid.Column=6"}},
  ThemeTargetStyles{L"Grid#SystemTrayFrameGrid > SystemTray.Stack#ShowDesktopStack", {
    L"Grid.Column=7"}},
  ThemeTargetStyles{L"SystemTray.DateTimeIconContent > Grid > StackPanel", {
    L"Orientation=Horizontal",
    L"Spacing=12"}},
  ThemeTargetStyles{L"TextBlock#InnerTextBlock[Text=\uE971]", {
    L"Text=\uED14"}},
  ThemeTargetStyles{L"TextBlock#TimeInnerTextBlock", {
    L"FontSize=15",
    L"FontWeight=Bold"}},
  ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
    L"FontSize=15",
    L"FontWeight=SemiBold"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.ScrollViewer >
Windows.UI.Xaml.Controls.ScrollContentPresenter > Windows.UI.Xaml.Controls.Border >
Windows.UI.Xaml.Controls.Grid", {
    L"ColumnDefinitions:=<ColumnDefinitionCollection><ColumnDefinition
Width=\"*\"/><ColumnDefinition Width=\"*\"/></ColumnDefinitionCollection>"}},
  ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton > Taskbar.TaskListButtonPanel", {
    L"Margin=0"}},
}};

const Theme g_themexdark = {{
  ThemeTargetStyles{L"Taskbar.TaskListButton", {
    L"CornerRadius=13",
    L"Padding=6,0,6,0",
    L"HorizontalContentAlignment=Left"}},
  ThemeTargetStyles{L"SystemTray.TextIconContent > Grid#ContainerGrid >
SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock", {
    L"FontSize=16",
    L"Foreground=#facc15"}},
  ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon", {
    L"MinWidth=25"}},
  ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid >
ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[1] >
SystemTray.IconView > Grid > Grid", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"SystemTray.TextIconContent > Grid#ContainerGrid", {
    L"Padding=2"}},
  ThemeTargetStyles{L"SystemTray.ChevronIconView", {
    L"MinWidth=27"}},
  ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Grid >
ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.IconView#SystemTrayIcon > Grid > Grid > SystemTray.TextIconContent", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Grid#IconPanel > Border#BackgroundElement,
Taskbar.TaskListLabeledButtonPanel > Border#BackgroundElement", {
    L"Background=#000000"}},
  ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
    L"Background=#000000",
    L"CornerRadius=13",
    L"Margin=0,5,4,5",
    L"Padding=2,0,-18,0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton > Grid > Rectangle#RunningIndicator", {
    L"Height=3",
    L"RadiusX=1.5",
    L"RadiusY=1.5",
    L"Fill@ActiveNormal=#facc15",
```

```
        L"VerticalAlignment=Bottom",
        L"Margin=16,0,16,4",
        L"StrokeThickness=0"}},
    ThemeTargetStyles{L"SystemTray.ImageIconContent > Grid#ContainerGrid > Image", {
        L"Width=13"}},
    ThemeTargetStyles{L"SystemTray.TextIconContent > Grid#ContainerGrid >
SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock", {
        L"FontSize=13",
        L"Foreground=#facc15"}},
    ThemeTargetStyles{L"TextBlock#LabelControl", {
        L"FontFamily=Segoe UI Medium",
        L"Foreground=#facc15",
        L"Margin=1,0,0,0",
        L"VerticalAlignment=Center",
        L"TextWrapping=NoWrap"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.
AutomationId=StartButton]", {
        L"Visibility=Visible"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.TextBlock#InnerTextBlock[Text=\uE971]", {
        L"Text=\uE712",
        L"Foreground=#facc15"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground >
Grid > Rectangle#BackgroundFill", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"SystemTray.TextIconContent > Grid#ContainerGrid >
SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock", {
        L"Foreground=#facc15"}},
}};

const Theme g_themeWindows7 = {{
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates >
Windows.UI.Xaml.Controls.Border#BackgroundElement,
Taskbar.TaskListLabeledButtonPanel@CommonStates >
Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"Background@InactiveNormal:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$taskbandInactiveNormal\" />",
        L"Background@InactivePointerOver:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$taskbandInactivePointerOver\" />",
        L"Background@ActiveNormal:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$taskbandActiveNormal\" />",
        L"Background@ActivePressed:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$taskbandActivePressed\" />",
        L"Background@ActivePointerOver:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$taskbandActivePointerOver\" />",
        L"Background@InactivePressed:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$taskbandInactivePressed\" />",
        L"BorderThickness=0",
        L"Background@MultiWindowNormal:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$taskbandInactiveNormal\" />",
        L"Background@MultiWindowActive:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$taskbandActiveNormal\" />",
        L"Background@MultiWindowPressed:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$taskbandActivePressed\" />",
        L"Background@MultiWindowPointerOver:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$taskbandActivePointerOver\" />",
        L"Background@RequestingAttention:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$taskbandRequestingAttention\" />",
        L"Background@RequestingAttentionPointerOver:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$taskbandRequestingAttention\" />",
        L"Background@RequestingAttentionPressed:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$taskbandRequestingAttention\" />",
```

```
        L"Background@RequestingAttentionMulti:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$taskbandRequestingAttention\" />",
        L"Background@RequestingAttentionMultiPointerOver:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$taskbandRequestingAttention\" />",
        L"Background@RequestingAttentionMultiPressed:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$taskbandRequestingAttention\" />"}},
    ThemeTargetStyles{L"Grid#IconPanel, Taskbar.TaskListLabeledButtonPanel", {
        L"Padding=0,0,0,0"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#RunningIndicator", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Grid#IconPanel > Image, Taskbar.TaskListLabeledButtonPanel > Image", {
        L"RenderTransform:=<TranslateTransform X=\"2\" />"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates >
Windows.UI.Xaml.Controls.Border#BackgroundElement,
Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates >
Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"Opacity@NoRunningIndicator=0"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.
AutomationId=StartButton] > Taskbar.TaskListButtonPanel@CommonStates > Grid >
Border#BackgroundElement,
Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartB
utton] > Taskbar.TaskListButtonPanel@CommonStates > Border#BackgroundElement", {
        L"Background@InactiveNormal:=<ImageBrush Stretch=\"None\"
ImageSource=\"$orbNormal\" />",
        L"Background@InactivePointerOver:=<ImageBrush Stretch=\"None\"
ImageSource=\"$orbPointerOver\" />",
        L"Background@InactivePressed:=<ImageBrush Stretch=\"None\"
ImageSource=\"$orbPressed\" />",
        L"Background@ActiveNormal:=<ImageBrush Stretch=\"None\"
ImageSource=\"$orbPressed\" />",
        L"Background@ActivePointerOver:=<ImageBrush Stretch=\"None\"
ImageSource=\"$orbPointerOver\" />",
        L"Background@ActivePressed:=<ImageBrush Stretch=\"None\"
ImageSource=\"$orbPressed\" />",
        L"BorderThickness=0",
        L"Width=54"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.
AutomationId=StartButton] > Taskbar.TaskListButtonPanel > Grid >
Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon,
Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartB
utton] > Taskbar.TaskListButtonPanel > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon",
{
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
        L"Padding=0,0,0,0",
        L"MinWidth=55",
        L"Margin=0,0,5,0"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground >
Grid", {
        L"Background:=<WindhawkBlur BlurAmount=\"3\" TintOpacity=\"$aeroOpacity\"
TintColor=\"$aeroColor\" />"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#BackgroundStroke", {
        L"Fill:=<ImageBrush Stretch=\"UniformToFill\" ImageSource=\"$reflection\" />",
        L"Height=100"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground >
Grid > Rectangle#BackgroundFill", {
        L"Fill:=<ImageBrush Stretch=\"Fill\" ImageSource=\"$taskbarBackground\" />"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton", {
        L"Margin=1,0,1,0"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates >
Windows.UI.Xaml.Controls.Border#MultiWindowElement,
Taskbar.TaskListLabeledButtonPanel@CommonStates >
Windows.UI.Xaml.Controls.Border#MultiWindowElement", {
        L"Background@MultiWindowNormal:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$taskbandInactiveNormal\" />",
        L"Background@MultiWindowActive:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$taskbandActiveNormal\" />",
```

```
        L"Background@MultiWindowPointerOver:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$taskbandActiveNormal\" />",
        L"BorderThickness=0",
        L"RenderTransform:=<TranslateTransform X=\"2\" />",
        L"Background@MultiWindowPressed:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$taskbandActiveNormal\" />",
        L"Background@RequestingAttentionMulti:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$taskbandRequestingAttention\" />",
        L"Background@RequestingAttentionMultiPointerOver:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$taskbandRequestingAttention\" />",
        L"Background@RequestingAttentionMultiPressed:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$taskbandRequestingAttention\" />"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView >
Windows.UI.Xaml.Controls.Grid#ContainerGrid > Windows.UI.Xaml.Controls.ContentPresenter",
{
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView >
Windows.UI.Xaml.Controls.Grid#ContainerGrid@ >
Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"Background@Normal:=<ImageBrush Stretch=\"None\"
ImageSource=\"$overflowNormal\" />",
        L"Background@PointerOver:=<ImageBrush Stretch=\"None\"
ImageSource=\"$overflowPointerOver\" />",
        L"Background@Pressed:=<ImageBrush Stretch=\"None\"
ImageSource=\"$overflowPressed\" />",
        L"Background@CheckedNormal:=<ImageBrush Stretch=\"None\"
ImageSource=\"$overflowPressed\" />",
        L"Background@CheckedPointerOver:=<ImageBrush Stretch=\"None\"
ImageSource=\"$overflowPressed\" />",
        L"Background@CheckedPressed:=<ImageBrush Stretch=\"None\"
ImageSource=\"$overflowPressed\" />",
        L"BorderThickness=0",
        L"Width=21",
        L"Height=20"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton >
Windows.UI.Xaml.Controls.Grid@CommonStates >
Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"Background@Normal=Transparent",
        L"Background@PointerOver:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$clockPointerOver\" />",
        L"Background@Pressed:=<ImageBrush Stretch=\"Fill\" ImageSource=\"$clockPressed\" />",
        L"BorderThickness=0",
        L"Margin=0",
        L"MinWidth=68",
        L"Background@Checked:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$clockPointerOver\" />",
        L"Background@CheckedPointerOver:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$clockPointerOver\" />",
        L"Background@CheckedPressed:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$clockPressed\" />"}},
    ThemeTargetStyles{L"SystemTray.DateTimeIconContent > Windows.UI.Xaml.Controls.Grid >
Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.TextBlock", {
        L"TextAlignment=0",
        L"Foreground=White",
        L"FontFamily=Segoe UI",
        L"FlowDirection=0",
        L"Typography.StylisticSet1=true"}},
    ThemeTargetStyles{L"SystemTray.BatteryIconContent > Grid#ContainerGrid > StackPanel >
TextBlock", {
        L"Foreground=White",
        L"FontFamily=Segoe UI",
        L"Typography.StylisticSet1=true"}},
    ThemeTargetStyles{L"SystemTray.BatteryIconContent > Grid#ContainerGrid > StackPanel >
Grid > TextBlock[1]", {
        L"Foreground=White"}},
```

```
   ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Windows.UI.Xaml.Controls.Grid@CommonStates > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
      L"Background@Normal=Transparent",
      L"Background@PointerOver:=<ImageBrush Stretch=\"Fill\" ImageSource=\"$clockPointerOver\" />",
      L"Background@Pressed:=<ImageBrush Stretch=\"Fill\" ImageSource=\"$clockPressed\" />",
      L"BorderThickness=0",
      L"Margin=0",
      L"Background@Checked:=<ImageBrush Stretch=\"Fill\" ImageSource=\"$clockPointerOver\" />",
      L"Background@CheckedPointerOver:=<ImageBrush Stretch=\"Fill\" ImageSource=\"$clockPointerOver\" />",
      L"Background@CheckedPressed:=<ImageBrush Stretch=\"Fill\" ImageSource=\"$clockPressed\" />"}},
   ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock > Windows.UI.Xaml.Controls.TextBlock", {
      L"Foreground=White"}},
   ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon > Windows.UI.Xaml.Controls.Grid#ContainerGrid@ > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
      L"Background@Normal=Transparent",
      L"Background@PointerOver:=<ImageBrush Stretch=\"Fill\" ImageSource=\"$trayPointerOver\" />",
      L"BorderThickness=0",
      L"Margin=0",
      L"Width=24"}},
   ThemeTargetStyles{L"SystemTray.NotificationAreaIcons > Windows.UI.Xaml.Controls.ItemsPresenter > Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.ContentPresenter", {
      L"Width=24",
      L"Padding=-2,0,-2,0"}},
   ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.ContentPresenter > Windows.UI.Xaml.Controls.ItemsPresenter > Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.ContentPresenter > SystemTray.IconView > Windows.UI.Xaml.Controls.Grid#ContainerGrid > Windows.UI.Xaml.Controls.Grid#ContentGrid > SystemTray.TextIconContent > Windows.UI.Xaml.Controls.Grid#ContainerGrid > SystemTray.AdaptiveTextBlock[2] > Windows.UI.Xaml.Controls.TextBlock", {
      L"FontFamily=Segoe MDL2 Assets",
      L"Text=\uE81B"}},
   ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.ContentPresenter > Windows.UI.Xaml.Controls.ItemsPresenter > Windows.UI.Xaml.Controls.StackPanel", {
      L"FlowDirection=1"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#ContainerGrid@ > Windows.UI.Xaml.Shapes.Rectangle#ShowDesktopPipe", {
      L"Grid.Row=0",
      L"Grid.RowSpan=3",
      L"Grid.Column=0",
      L"Grid.ColumnSpan=3",
      L"Fill@Normal:=<ImageBrush Stretch=\"Fill\" ImageSource=\"$desktopNormal\" />",
      L"Height=39",
      L"Width=$desktopWidth",
      L"RadiusX=0",
      L"RadiusY=0",
      L"Fill@PointerOver:=<ImageBrush Stretch=\"Fill\" ImageSource=\"$desktopPointerOver\" />",
      L"Fill@Pressed:=<ImageBrush Stretch=\"Fill\" ImageSource=\"$desktopPressed\" />"}},
   ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack", {
      L"Width=$desktopWidth"}},
   ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack > Windows.UI.Xaml.Controls.Grid > SystemTray.StackListView > Windows.UI.Xaml.Controls.ItemsPresenter > Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.ContentPresenter", {
      L"Width=$desktopWidth"}},
```

```
    ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack > Windows.UI.Xaml.Controls.Grid > SystemTray.StackListView > Windows.UI.Xaml.Controls.ItemsPresenter > Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.ContentPresenter > SystemTray.IconView > Windows.UI.Xaml.Controls.Grid#ContainerGrid, SystemTray.Stack#ShowDesktopStack > Windows.UI.Xaml.Controls.Grid > SystemTray.StackListView > Windows.UI.Xaml.Controls.ItemsPresenter > Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.ContentPresenter > SystemTray.IconView", {
        L"Width=$desktopWidth"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.ContentPresenter > Windows.UI.Xaml.Controls.ItemsPresenter > Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.ContentPresenter > SystemTray.IconView#SystemTrayIcon > Windows.UI.Xaml.Controls.Grid#ContainerGrid", {
        L"Padding=0"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton", {
        L"Margin=3,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.Stack#MainStack > Windows.UI.Xaml.Controls.Grid > SystemTray.StackListView > Windows.UI.Xaml.Controls.ItemsPresenter > Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.ContentPresenter > SystemTray.IconView > Windows.UI.Xaml.Controls.Grid#ContainerGrid@ > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"Background@Normal=Transparent",
        L"Background@PointerOver:=<ImageBrush Stretch=\"Fill\" ImageSource=\"$trayPointerOver\" />",
        L"Background@Pressed:=<ImageBrush Stretch=\"Fill\" ImageSource=\"$trayPressed\" />",
        L"BorderThickness=0",
        L"Margin=0",
        L"Background@Checked:=<ImageBrush Stretch=\"Fill\" ImageSource=\"$trayPointerOver\" />",
        L"Background@CheckedPointerOver:=<ImageBrush Stretch=\"Fill\" ImageSource=\"$trayPointerOver\" />",
        L"Background@CheckedPressed:=<ImageBrush Stretch=\"Fill\" ImageSource=\"$trayPressed\" />",
        L"Width=24"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Windows.UI.Xaml.Shapes.Rectangle#DefaultIcon, Taskbar.TaskListLabeledButtonPanel#IconPanel@RunningIndicatorStates > Windows.UI.Xaml.Shapes.Rectangle#DefaultIcon", {
        L"Visibility=Collapsed",
        L"Visibility@NoRunningIndicator=Visible"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Windows.UI.Xaml.Shapes.Rectangle#DefaultIcon, Taskbar.TaskListLabeledButtonPanel#IconPanel@CommonStates > Windows.UI.Xaml.Shapes.Rectangle#DefaultIcon", {
        L"Fill=Transparent",
        L"Width=54",
        L"Height=54",
        L"Fill@InactivePointerOver:=<ImageBrush Stretch=\"Uniform\" ImageSource=\"$taskbandNotRunningPointerOver\" />",
        L"Fill@InactivePressed:=<ImageBrush Stretch=\"Uniform\" ImageSource=\"$taskbandNotRunningPressed\" />",
        L"Transform3D:=<CompositeTransform3D ScaleY=\"1.1\" ScaleX=\"1.04\" TranslateY=\"1\" CenterY=\"27\" />"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock#LanguageInnerTextBlock > TextBlock#InnerTextBlock", {
        L"FontFamily=Segoe UI",
        L"Typography.StylisticSet1=true"}},
    ThemeTargetStyles{L"Border#SearchPillBackgroundElement", {
        L"BorderBrush=#4F222222",
        L"BorderThickness=1"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=TaskViewButton] > Taskbar.TaskListButtonPanel@CommonStates > Grid > Border#BackgroundElement, Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=TaskViewButton] > Taskbar.TaskListButtonPanel@CommonStates > Border#BackgroundElement", {
        L"BorderBrush@InactivePointerOver:=<ImageBrush Stretch=\"Uniform\" ImageSource=\"$taskbandNotRunningPointerOver\" />",
        L"BorderThickness=2",
```

```
        L"Background:=<ImageBrush Stretch=\"None\" ImageSource=\"$taskviewIcon\" />",
        L"BorderBrush@InactivePressed:=<ImageBrush Stretch=\"Uniform\"
ImageSource=\"$taskbandNotRunningPressed\" />",
        L"BorderBrush@ActivePressed:=<ImageBrush Stretch=\"Uniform\"
ImageSource=\"$taskbandNotRunningPressed\" />",
        L"BorderBrush@ActivePointerOver:=<ImageBrush Stretch=\"Uniform\"
ImageSource=\"$taskbandNotRunningPointerOver\" />",
        L"BorderBrush@ActiveNormal=Transparent"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.
AutomationId=TaskViewButton] > Taskbar.TaskListButtonPanel > Grid >
Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer,
Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=TaskVi
ewButton] > Taskbar.TaskListButtonPanel > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer", {
        L"Visibility=Collapsed"}},

ThemeTargetStyles{L"Taskbar.SearchBoxButton#SearchBoxButton[AutomationProperties.Autom
ationId=SearchButton] > Taskbar.TaskListButtonPanel@CommonStates >
Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundElement,
Taskbar.SearchBoxButton#SearchBoxButton[AutomationProperties.AutomationId=SearchButton
] > Taskbar.TaskListButtonPanel@CommonStates >
Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"BorderBrush@InactivePointerOver_SearchIcon:=<ImageBrush Stretch=\"Uniform\"
ImageSource=\"$taskbandNotRunningPointerOver\" />",
        L"BorderBrush@InactivePressed_SearchIcon:=<ImageBrush Stretch=\"Uniform\"
ImageSource=\"$taskbandNotRunningPressed\" />",
        L"BorderBrush@ActivePressed_SearchIcon:=<ImageBrush Stretch=\"Uniform\"
ImageSource=\"$taskbandNotRunningPressed\" />",
        L"BorderBrush@ActivePointerOver_SearchIcon:=<ImageBrush Stretch=\"Uniform\"
ImageSource=\"$taskbandNotRunningPointerOver\" />",
        L"BorderBrush@ActiveNormal_SearchIcon=Transparent",
        L"BorderThickness@InactivePointerOver_SearchIcon=2",
        L"BorderThickness@InactivePressed_SearchIcon=2",
        L"BorderThickness@ActivePressed_SearchIcon=2",
        L"BorderThickness@ActivePointerOver_SearchIcon=2",
        L"Background@ActiveNormal_SearchIcon:=<ImageBrush Stretch=\"None\"
ImageSource=\"$ssearchIcon\" />",
        L"Background@InactivePointerOver_SearchIcon:=<ImageBrush Stretch=\"None\"
ImageSource=\"$ssearchIcon\" />",
        L"Background@InactivePressed_SearchIcon:=<ImageBrush Stretch=\"None\"
ImageSource=\"$ssearchIcon\" />",
        L"Background@ActivePressed_SearchIcon:=<ImageBrush Stretch=\"None\"
ImageSource=\"$ssearchIcon\" />",
        L"Background@ActivePointerOver_SearchIcon:=<ImageBrush Stretch=\"None\"
ImageSource=\"$ssearchIcon\" />",
        L"Background@InactiveNormal_SearchIcon:=<ImageBrush Stretch=\"None\"
ImageSource=\"$ssearchIcon\" />",
        L"Height=30",
        L"Height@ActiveNormal_SearchIcon=Auto",
        L"Height@InactivePointerOver_SearchIcon=Auto",
        L"Height@InactivePressed_SearchIcon=Auto",
        L"Height@ActivePressed_SearchIcon=Auto",
        L"Height@ActivePointerOver_SearchIcon=Auto",
        L"Height@InactiveNormal_SearchIcon=Auto"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel@CommonStates >
Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundElement,
SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel@CommonStates >
Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"BorderBrush@InactivePointerOver:=<ImageBrush Stretch=\"Uniform\"
ImageSource=\"$taskbandNotRunningPointerOver\" />",
        L"BorderBrush@InactivePressed:=<ImageBrush Stretch=\"Uniform\"
ImageSource=\"$taskbandNotRunningPressed\" />",
        L"BorderBrush@ActivePressed:=<ImageBrush Stretch=\"Uniform\"
ImageSource=\"$taskbandNotRunningPressed\" />",
        L"BorderBrush@ActivePointerOver:=<ImageBrush Stretch=\"Uniform\"
ImageSource=\"$taskbandNotRunningPointerOver\" />",
        L"BorderBrush@ActiveNormal=Transparent",
```

```
        L"BorderThickness@InactivePointerOver=2",
        L"BorderThickness@InactivePressed=2",
        L"BorderThickness@ActivePressed=2",
        L"BorderThickness@ActivePointerOver=2",
        L"Background@ActiveNormal:=<ImageBrush Stretch=\"None\"
ImageSource=\"$searchIcon\" />",
        L"Background@InactivePointerOver:=<ImageBrush Stretch=\"None\"
ImageSource=\"$searchIcon\" />",
        L"Background@InactivePressed:=<ImageBrush Stretch=\"None\"
ImageSource=\"$searchIcon\" />",
        L"Background@ActivePressed:=<ImageBrush Stretch=\"None\"
ImageSource=\"$searchIcon\" />",
        L"Background@ActivePointerOver:=<ImageBrush Stretch=\"None\"
ImageSource=\"$searchIcon\" />",
        L"Background@InactiveNormal:=<ImageBrush Stretch=\"None\"
ImageSource=\"$searchIcon\" />",
        L"Height=30",
        L"Height@ActiveNormal=Auto",
        L"Height@InactivePointerOver=Auto",
        L"Height@InactivePressed=Auto",
        L"Height@ActivePressed=Auto",
        L"Height@ActivePointerOver=Auto",
        L"Height@InactiveNormal=Auto"}},

ThemeTargetStyles{L"Taskbar.SearchBoxButton#SearchBoxButton[AutomationProperties.Autom
ationId=SearchButton] > Taskbar.TaskListButtonPanel@CommonStates > Grid >
Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer,
Taskbar.SearchBoxButton#SearchBoxButton[AutomationProperties.AutomationId=SearchButton
] > Taskbar.TaskListButtonPanel@CommonStates >
Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer", {
        L"Visibility@ActiveNormal_SearchIcon=Collapsed",
        L"Visibility@InactivePointerOver_SearchIcon=Collapsed",
        L"Visibility@InactivePressed_SearchIcon=Collapsed",
        L"Visibility@ActivePressed_SearchIcon=Collapsed",
        L"Visibility@ActivePointerOver_SearchIcon=Collapsed",
        L"Visibility@InactiveNormal_SearchIcon=Collapsed"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel@CommonStates >
Grid > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer, SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel@CommonStates >
Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer", {
        L"Visibility@ActiveNormal=Collapsed",
        L"Visibility@InactivePointerOver=Collapsed",
        L"Visibility@InactivePressed=Collapsed",
        L"Visibility@ActivePressed=Collapsed",
        L"Visibility@ActivePointerOver=Collapsed",
        L"Visibility@InactiveNormal=Collapsed"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.StackPanel >
Windows.UI.Xaml.Controls.ContentPresenter > SystemTray.IconView >
Windows.UI.Xaml.Controls.Grid@ > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"Background@CheckedPressed:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$clockPressed\" />",
        L"Background@CheckedPointerOver:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$clockPointerOver\" />",
        L"Background@CheckedNormal:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$clockPointerOver\" />",
        L"BorderThickness=0",
        L"Background@Pressed:=<ImageBrush Stretch=\"Fill\" ImageSource=\"$clockPressed\" />",
        L"Background@PointerOver:=<ImageBrush Stretch=\"Fill\"
ImageSource=\"$clockPointerOver\" />",
        L"Background@Normal=Transparent",
        L"Margin=0"}},
    ThemeTargetStyles{L"SystemTray.Stack#NonActivatableStack", {
        L"Margin=4,0,0,0"}},
```

```
ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton[AutomationProperties.AutomationId=WidgetsButton] > Taskbar.TaskListButtonPanel@CommonStates > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundElement, Taskbar.AugmentedEntryPointButton[AutomationProperties.AutomationId=WidgetsButton] > Taskbar.TaskListButtonPanel@CommonStates > Windows.UI.Xaml.Controls.Border#BackgroundElement", {
    L"Background@InactivePointerOver:=<ImageBrush Stretch=\"Fill\" ImageSource=\"$widgetsPointerOver\" />",
    L"Background@InactivePressed:=<ImageBrush Stretch=\"Fill\" ImageSource=\"$widgetsPressed\" />",
    L"Background@ActivePointerOver:=<ImageBrush Stretch=\"Fill\" ImageSource=\"$widgetsPointerOver\" />",
    L"Background@ActivePressed:=<ImageBrush Stretch=\"Fill\" ImageSource=\"$widgetsPressed\" />",
    L"Background@ActiveNormal:=<ImageBrush Stretch=\"Fill\" ImageSource=\"$widgetsPointerOver\" />",
    L"BorderThickness=0",
    L"Margin=0"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton] > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
    L"Width=54"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.ToolTip > Windows.UI.Xaml.Controls.ContentPresenter > Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.TextBlock", {
    L"FontFamily=Segoe UI",
    L"Typography.StylisticSet1=true"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.TextBlock#LabelControl", {
    L"FontFamily=Segoe UI",
    L"Typography.StylisticSet1=true",
    L"Foreground=White"}},
  ThemeTargetStyles{L"Border#OverflowFlyoutBackgroundBorder", {
    L"Background:=<ImageBrush Stretch=\"None\" ImageSource=\"$reflectionBlue\" />"}},
}, {
  L"orbNormal=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/orbNormal.png",
  L"orbPointerOver=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/orbHover.png",
  L"orbPressed=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/orbPressed.png",
  L"aeroColor={ThemeResource SystemAccentColor}",
  L"aeroOpacity=0.3",
  L"reflection=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/AeroPeek.png",
  L"reflectionBlue=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/AeroPeekBlue.png",
  L"taskbandInactiveNormal=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/InactiveNormal.png",
  L"taskbandInactivePointerOver=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/InactivePointerOver.png",
  L"taskbandInactivePressed=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/ActiveNormal.png",
  L"taskbandActiveNormal=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/ActiveNormal.png",
  L"taskbandActivePointerOver=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/ActiveNormal.png",
  L"taskbandActivePressed=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/ActiveNormal.png",
  L"overflowNormal=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/overflowNormal.png",
  L"overflowPointerOver=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/overflowPointerOver.png",
```

```
    L"overflowPressed=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/overflowPressed.png",
    L"clockPointerOver=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/clockPointerOver.png",
    L"clockPressed=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/clockPressed.png",
    L"trayPointerOver=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/trayPointerOver.png",
    L"trayPressed=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/trayPressed.png",
    L"desktopNormal=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/desktopNormal.png",
    L"desktopPointerOver=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/desktopPointerOver.png",
    L"desktopPressed=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/desktopPressed.png",
    L"desktopWidth=15",

L"taskbandRequestingAttention=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/requestingAttention.png",

L"taskbandNotRunningPointerOver=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/notRunningPointerOver.png",

L"taskbandNotRunningPressed=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/notRunningPressed.png",
    L"taskbarBackground=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/taskbarBackground.png",
    L"taskviewIcon=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/taskview.png",
    L"searchIcon=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/search.png",
    L"widgetsPointerOver=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/widgetsPointerOver.png",
    L"widgetsPressed=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Windows7/ThemeResources/widgetsPressed.png",
}};

const Theme g_themeAeris = {{
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
        L"Background:=$taskbarBackground"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground > Grid", {
        L"Background:=<WindhawkBlur BlurAmount=\"$taskbarBlurIncreace\" TintColor=\"#00000000\" />"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground > Grid > Rectangle#BackgroundFill", {
        L"Fill:=<SolidColorBrush Color=\"$themeColor\" Opacity=\"$themeColorOpacity\"/>"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground > Grid > Rectangle#BackgroundStroke", {
        L"Visibility=Collapsed",
        L"Fill:=<SolidColorBrush Color=\"$primaryColor\" Opacity=\"0.05\"/>"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel@CommonStates", {
        L"Padding=0",
        L"Margin=$taskListMargin,0,$taskListMargin,0",
        L"Background@ActiveNormal:=<SolidColorBrush Color=\"$activeColor\" Opacity=\"0.5\"/>",
        L"Background@ActivePointerOver:=<SolidColorBrush Color=\"$activeColor\" Opacity=\"0.5\"/>",
        L"Background@ActivePressed:=<SolidColorBrush Color=\"$activeColor\" Opacity=\"0.5\"/>"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel@CommonStates > Grid > Border#BackgroundElement, Taskbar.TaskListButtonPanel@CommonStates > Border#BackgroundElement", {
```

```
        L"Background:=$transparent",
        L"Background@InactivePointerOver:=$pointerOver",
        L"Background@InactivePressed:=$pressed",
        L"Background@ActivePointerOver:=$pointerOver",
        L"Background@ActivePressed:=$pressed",
        L"BorderThickness=0",
        L"CornerRadius=0",
        L"Margin=0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton > Grid#IconPanel@RunningIndicatorStates,
Taskbar.TaskListButton > Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates", {
        L"Padding=0",
        L"Margin=$taskListMargin,0,$taskListMargin,0",
        L"Background@NoRunningIndicator:=$transparent",
        L"Background@InactiveRunningIndicator:=<SolidColorBrush Color=\"$primaryColor\"
Opacity=\"0.1\"/>",
        L"Background@ActiveRunningIndicator:=<SolidColorBrush Color=\"$activeColor\"
Opacity=\"0.5\"/>",
        L"Background:=<SolidColorBrush Color=\"$requestAttentionColor\" Opacity=\"0.5\"/>"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton > Grid#IconPanel@CommonStates >
Border#BackgroundElement, Taskbar.TaskListButton >
Taskbar.TaskListLabeledButtonPanel@CommonStates > Border#BackgroundElement", {
        L"Background:=$transparent",
        L"Background@InactivePointerOver:=$pointerOver",
        L"Background@InactivePressed:=$pressed",
        L"Background@ActivePointerOver:=$pointerOver",
        L"Background@ActivePressed:=$pressed",
        L"Background@MultiWindowPointerOver:=$pointerOver",
        L"Background@MultiWindowPressed:=$pressed",
        L"Background@RequestingAttentionPointerOver:=$pointerOver",
        L"Background@RequestingAttentionPressed:=$pressed",
        L"Background@RequestingAttentionMultiPointerOver:=$pointerOver",
        L"Background@RequestingAttentionMultiPressed:=$pressed",
        L"BorderThickness=0",
        L"CornerRadius=0",
        L"Margin=0"}},
    ThemeTargetStyles{L"Grid#IconPanel > Border#MultiWindowElement,
Taskbar.TaskListLabeledButtonPanel > Border#MultiWindowElement", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Rectangle#RunningIndicator,
Taskbar.TaskListLabeledButtonPanel@CommonStates > Rectangle#RunningIndicator", {
        L"Visibility=Collapsed",
        L"Fill:=<SolidColorBrush Color=\"$primaryColor\" Opacity=\"0.2\"/>",
        L"VerticalAlignment=0",
        L"HorizontalAlignment=Stretch",
        L"Margin=0,0,-4,0",
        L"Width=Auto",
        L"Height=3",
        L"RadiusX=0",
        L"RadiusY=0",
        L"Visibility@MultiWindowNormal=Visible",
        L"Visibility@MultiWindowActive=Visible",
        L"Visibility@MultiWindowPointerOver=Visible",
        L"Visibility@MultiWindowPressed=Visible",
        L"Visibility@RequestingAttentionMulti=Visible",
        L"Visibility@RequestingAttentionMultiPointerOver=Visible",
        L"Visibility@RequestingAttentionMultiPressed=Visible"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ProgressBar#ProgressIndicator", {
        L"VerticalAlignment=Stretch",
```

```
        L"HorizontalAlignment=Stretch",
        L"Margin=0,0,-4,0",
        L"Width=Auto",
        L"Height=Auto",
        L"CornerRadius=0"}},
    ThemeTargetStyles{L"Border#ProgressBarRoot > Border > Grid", {
        L"Height=Auto"}},
    ThemeTargetStyles{L"Grid#LayoutRoot@CommonStates > Border#ProgressBarRoot >
Border > Grid > Rectangle#ProgressBarTrack", {
        L"Margin=0",
        L"RadiusX=0",
        L"RadiusY=0",
        L"Fill:=<SolidColorBrush Color=\"$progressColor\" Opacity=\"0.15\"/>",
        L"Fill@Paused:=<SolidColorBrush Color=\"$progressPausedColor\" Opacity=\"0.15\"/>"}},
    ThemeTargetStyles{L"Grid#LayoutRoot@CommonStates > Border#ProgressBarRoot >
Border > Grid > Rectangle#DeterminateProgressBarIndicator", {
        L"RadiusX=0",
        L"RadiusY=0",
        L"Fill:=<SolidColorBrush Color=\"$progressColor\" Opacity=\"0.4\"/>",
        L"Fill@Paused:=<SolidColorBrush Color=\"$progressPausedColor\" Opacity=\"0.4\"/>"}},
    ThemeTargetStyles{L"Grid#IconPanel > Image, Taskbar.TaskListLabeledButtonPanel > Image", {
        L"Transform3D:=<CompositeTransform3D TranslateX=\"2\" TranslateY=\"1\" />"}},
    ThemeTargetStyles{L"Grid#IconPanel > Rectangle#DefaultIcon,
Taskbar.TaskListLabeledButtonPanel > Rectangle#DefaultIcon", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel > Grid > AnimatedVisualPlayer,
Taskbar.TaskListButtonPanel > AnimatedVisualPlayer,
SearchUx.SearchUI.SearchButtonRootGrid > Grid > AnimatedVisualPlayer,
SearchUx.SearchUI.SearchButtonRootGrid > AnimatedVisualPlayer", {
        L"Transform3D:=<CompositeTransform3D TranslateX=\"0\" TranslateY=\"1\" />"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.
AutomationId=StartButton] > Taskbar.TaskListButtonPanel", {
        L"MinWidth=60",
        L"Margin=0,0,$taskListMargin,0"}},
    ThemeTargetStyles{L"Taskbar.SearchBoxButton >
Taskbar.TaskListButtonPanel@CommonStates", {
        L"Padding=0",
        L"Margin=$taskListMargin,0,$taskListMargin,0",
        L"Background@ActiveNormal_SearchIcon:=<SolidColorBrush Color=\"$activeColor\"
Opacity=\"0.5\"/>",
        L"Background@ActivePointerOver_SearchIcon:=<SolidColorBrush Color=\"$activeColor\"
Opacity=\"0.5\"/>",
        L"Background@ActivePressed_SearchIcon:=<SolidColorBrush Color=\"$activeColor\"
Opacity=\"0.5\"/>"}},
    ThemeTargetStyles{L"Taskbar.SearchBoxLaunchListButton > Taskbar.TaskListButtonPanel", {
        L"Padding=0",
        L"Margin=$taskListMargin,0,$taskListMargin,0"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid@CommonStates", {
        L"Padding=0",
        L"Margin=$taskListMargin,0,$taskListMargin,0",
        L"Background@ActiveNormal:=<SolidColorBrush Color=\"$activeColor\"
Opacity=\"0.5\"/>",
        L"Background@ActivePointerOver:=<SolidColorBrush Color=\"$activeColor\"
Opacity=\"0.5\"/>",
        L"Background@ActivePressed:=<SolidColorBrush Color=\"$activeColor\"
Opacity=\"0.5\"/>"}},
    ThemeTargetStyles{L"Taskbar.SearchBoxButton >
Taskbar.TaskListButtonPanel@CommonStates > Grid > Border#BackgroundElement,
Taskbar.SearchBoxButton > Taskbar.TaskListButtonPanel@CommonStates >
Border#BackgroundElement", {
        L"BorderThickness=0",
        L"CornerRadius=0",
```

```
        L"Margin=0",
        L"Background@InactiveNormal_SearchIcon:=$transparent",
        L"Background@InactivePointerOver_SearchIcon:=$pointerOver",
        L"Background@InactivePressed_SearchIcon:=$pressed",
        L"Background@ActiveNormal_SearchIcon:=$transparent",
        L"Background@ActivePointerOver_SearchIcon:=$pointerOver",
        L"Background@ActivePressed_SearchIcon:=$pressed"}},
    ThemeTargetStyles{L"Taskbar.SearchBoxLaunchListButton > Taskbar.TaskListButtonPanel >
Grid > Border#BackgroundElement, Taskbar.SearchBoxLaunchListButton >
Taskbar.TaskListButtonPanel > Border#BackgroundElement", {
        L"BorderThickness=0",
        L"CornerRadius=0",
        L"Margin=0"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid@CommonStates > Grid >
Border#BackgroundElement, SearchUx.SearchUI.SearchButtonRootGrid@CommonStates >
Border#BackgroundElement", {
        L"BorderThickness=0",
        L"CornerRadius=0",
        L"Margin=0",
        L"Background@InactiveNormal:=$transparent",
        L"Background@InactivePointerOver:=$pointerOver",
        L"Background@InactivePressed:=$pressed",
        L"Background@ActiveNormal:=$transparent",
        L"Background@ActivePointerOver:=$pointerOver",
        L"Background@ActivePressed:=$pressed"}},
    ThemeTargetStyles{L"Border#SearchPillBackgroundElement", {
        L"BorderThickness=0",
        L"CornerRadius=0",
        L"Margin=0"}},
    ThemeTargetStyles{L"Grid#DynamicSearchBoxGleamContainer", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Grid@ > Border#BackgroundBorder", {
        L"Padding=0",
        L"CornerRadius=0",
        L"Margin=0",
        L"BorderThickness=0",
        L"Background@PointerOver:=$pointerOver",
        L"Background@Pressed:=$pressed"}},
    ThemeTargetStyles{L"SystemTray.OmniButton > Grid@CommonStates >
Border#BackgroundBorder", {
        L"Background@PointerOver:=$pointerOver",
        L"Background@Pressed:=$pressed"}},
    ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack", {
        L"MinWidth=$showDesktopWidth",
        L"MaxWidth=$showDesktopWidth"}},
    ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack > Grid >
SystemTray.StackListView > Windows.UI.Xaml.Controls.ItemsPresenter >
Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.ContentPresenter >
SystemTray.IconView > Grid#ContainerGrid, SystemTray.Stack#ShowDesktopStack > Grid >
SystemTray.StackListView > Windows.UI.Xaml.Controls.ItemsPresenter >
Windows.UI.Xaml.Controls.StackPanel > Windows.UI.Xaml.Controls.ContentPresenter >
SystemTray.IconView", {
        L"MinWidth=$showDesktopWidth",
        L"MaxWidth=$showDesktopWidth"}},
    ThemeTargetStyles{L"SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid@ >
Rectangle#ShowDesktopPipe", {
        L"Grid.Row=0",
        L"Grid.RowSpan=3",
        L"Grid.Column=0",
        L"Grid.ColumnSpan=3",
```

```
        L"VerticalAlignment=Stretch",
        L"HorizontalAlignment=Stretch",
        L"Height=Auto",
        L"Width=Auto",
        L"Fill@PointerOver:=$pointerOver",
        L"Fill@Pressed:=$pressed"}},
}, {
    L"themeColor={ThemeResource SystemAccentColorDark1}",
    L"themeColorOpacity=0",
    L"primaryColor={ThemeResource TextFillColorPrimary}",
    L"activeColor=#33AAFF",
    L"requestAttentionColor=#FF7788",
    L"progressColor=#44CC66",
    L"progressPausedColor=#EECC44",
    L"taskbarBackground=<AcrylicBrush TintColor=\"{ThemeResource CardStrokeColorDefaultSolid}\" FallbackColor=\"{ThemeResource CardStrokeColorDefaultSolid}\" TintOpacity=\"0\" TintLuminosityOpacity=\"0.8\" Opacity=\"1\"/>",
    L"taskbarBlurIncreace=0",
    L"taskListMargin=2",
    L"showDesktopWidth=10",
    L"transparent=<SolidColorBrush Color=\"{ThemeResource TextFillColorPrimary}\" Opacity=\"0\"/>",
    L"pointerOver=<SolidColorBrush Color=\"{ThemeResource TextFillColorPrimary}\" Opacity=\"0.075\"/>",
    L"pressed=<SolidColorBrush Color=\"{ThemeResource TextFillColorPrimary}\" Opacity=\"0.05\"/>",
}};

const Theme g_themePlasma = {{
    ThemeTargetStyles{L"Taskbar.TaskListButton > Grid#IconPanel, Taskbar.TaskListButton > Taskbar.TaskListLabeledButtonPanel", {
        L"Padding=0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton > Grid#IconPanel@CommonStates > Windows.UI.Xaml.Controls.Border#BackgroundElement, Taskbar.TaskListButton > Taskbar.TaskListLabeledButtonPanel@CommonStates > Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"CornerRadius=0",
        L"BorderThickness=1,0,1,0",
        L"BorderBrush=Transparent",
        L"Background@ActiveNormal:=$taskbandActive",
        L"Background@InactiveNormal:=$taskbandInactiveNormal",
        L"Background@ActivePointerOver:=$taskbandPointerOver",
        L"Background@MultiWindowNormal:=$taskbandInactiveNormal",
        L"Background@MultiWindowPointerOver:=$taskbandPointerOver",
        L"Background@InactivePointerOver:=$taskbandPointerOver",
        L"Background@ActivePressed:=$taskbandPointerOver",
        L"Background@InactivePressed:=$taskbandPointerOver",
        L"Margin=0",
        L"Background@MultiWindowPressed:=$taskbandPointerOver",
        L"Background@MultiWindowActive:=$taskbandActive",
        L"Background@RequestingAttention:=$taskbandAttention",
        L"Background@RequestingAttentionPointerOver:=$taskbandPointerOver",
        L"Background@RequestingAttentionPressed:=$taskbandPointerOver",
        L"Background@RequestingAttentionMulti:=$taskbandAttention",
        L"Background@RequestingAttentionMultiPointerOver:=$taskbandPointerOver",
        L"Background@RequestingAttentionMultiPressed:=$taskbandPointerOver"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton > Grid#IconPanel@RunningIndicatorStates > Windows.UI.Xaml.Controls.Border#BackgroundElement, Taskbar.TaskListButton >
```

```
Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates >
Windows.UI.Xaml.Controls.Border#BackgroundElement", {
      L"Opacity@NoRunningIndicator=0"}},
   ThemeTargetStyles{L"Grid#IconPanel > Windows.UI.Xaml.Controls.Image#Icon,
Taskbar.TaskListLabeledButtonPanel#IconPanel > Windows.UI.Xaml.Controls.Image#Icon", {
      L"RenderTransform:=<TranslateTransform X=\"2\" />"}},
   ThemeTargetStyles{L"Grid#IconPanel@CommonStates >
Windows.UI.Xaml.Shapes.Rectangle#RunningIndicator,
Taskbar.TaskListLabeledButtonPanel@CommonStates >
Windows.UI.Xaml.Shapes.Rectangle#RunningIndicator", {
      L"Width=50",
      L"RadiusX=0",
      L"RadiusY=0",
      L"Height=3",
      L"VerticalAlignment=Top",
      L"RenderTransform:=<TranslateTransform X=\"2\" />",
      L"Margin=-1,0,-1,0",
      L"Fill@ActiveNormal:=$indicatorActive",
      L"Fill@ActivePointerOver:=$indicatorPointerOver",
      L"Fill:=$indicatorInactive",
      L"Fill@InactivePointerOver:=$indicatorPointerOver",
      L"Fill@ActivePressed:=$indicatorPointerOver",
      L"Fill@InactivePressed:=$indicatorPointerOver",
      L"Fill@MultiWindowNormal:=$indicatorInactive",
      L"Fill@MultiWindowPointerOver:=$indicatorPointerOver",
      L"Fill@MultiWindowPressed:=$indicatorPointerOver",
      L"Fill@MultiWindowActive:=$indicatorActive",
      L"Fill@RequestingAttention:=$indicatorAttention",
      L"Fill@RequestingAttentionPointerOver:=$indicatorPointerOver",
      L"Fill@RequestingAttentionPressed:=$indicatorPointerOver",
      L"Fill@RequestingAttentionMulti:=$indicatorAttention",
      L"Fill@RequestingAttentionMultiPointerOver:=$indicatorPointerOver",
      L"Fill@RequestingAttentionMultiPressed:=$indicatorPointerOver"}},
   ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
      L"Visibility=Collapsed"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground >
Grid > Rectangle#BackgroundFill", {
      L"Fill:=$WindhawkBlur"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#MultiWindowElement", {
      L"Visibility=Collapsed"}},
   ThemeTargetStyles{L"Grid#IconPanel@CommonStates >
Windows.UI.Xaml.Shapes.Rectangle#DefaultIcon,
Taskbar.TaskListLabeledButtonPanel@CommonStates >
Windows.UI.Xaml.Shapes.Rectangle#DefaultIcon", {
      L"Fill:=<ImageBrush Stretch=\"Uniform\" ImageSource=\"$plusIndicator\" />",
      L"Width=11",
      L"Height=11",
      L"RadiusX=0",
      L"RadiusY=0",
      L"VerticalAlignment=Bottom",
      L"RenderTransform:=<TranslateTransform X=\"1\" />",
      L"Visibility@MultiWindowNormal=Visible",
      L"Visibility@MultiWindowActive=Visible",
      L"Visibility@MultiWindowPointerOver=Visible",
      L"HorizontalAlignment=Center",
      L"Visibility@MultiWindowPressed=Visible",
      L"Visibility=Collapsed"}},
   ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel,
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel", {
```

```
        L"Padding=0",
        L"Width=50"}},

ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel@CommonStates > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundElement,
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel@CommonStates > Windows.UI.Xaml.Controls.Border#BackgroundElement,
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel@CommonStates > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundElement,
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel@CommonStates > Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"CornerRadius=0",
        L"BorderThickness=0",
        L"Width=32",
        L"Background=Transparent"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton] > Taskbar.TaskListButtonPanel > Grid > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon,
Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton] > Taskbar.TaskListButtonPanel > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon", {
        L"Visibility=Collapsed"}},

ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel@CommonStates,
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel@CommonStates", {
        L"BorderThickness@ActiveNormal=0,3,0,0",
        L"Width=50",
        L"BorderBrush:=$selectionBorder",
        L"BorderThickness@ActivePointerOver=0,3,0,0",
        L"BorderThickness@ActivePressed=0,3,0,0"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton] > Taskbar.TaskListButtonPanel > Grid > Border#BackgroundElement,
Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartButton] > Taskbar.TaskListButtonPanel > Border#BackgroundElement", {
        L"Background:=<ImageBrush Stretch=\"Uniform\" ImageSource=\"https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Plasma/ThemeResources/$StartButton.png\" />"}},

ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton[AutomationProperties.AutomationId=WidgetsButton] > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
        L"Width=Auto"}},
    ThemeTargetStyles{L"Grid#SystemTrayFrameGrid > SystemTray.Stack#NotifyIconStack", {
        L"Grid.Column=5"}},
    ThemeTargetStyles{L"Grid#SystemTrayFrameGrid > SystemTray.NotificationAreaIcons#NotificationAreaIcons", {
        L"Grid.Column=0"}},
    ThemeTargetStyles{L"Grid#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton", {
        L"Grid.Column=1"}},
    ThemeTargetStyles{L"SystemTray.OmniButton > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"Background=Transparent",
        L"Margin=1,0,1,0",
        L"CornerRadius=0"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#ContainerGrid@ > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"BorderBrush@CheckedNormal:=$selectionBorderExtended",
        L"BorderThickness@CheckedNormal:=0,3,0,0",
        L"BorderBrush@CheckedPointerOver:=$selectionBorderExtended",
        L"BorderThickness@CheckedPointerOver:=0,3,0,0",
        L"BorderBrush@CheckedPressed:=$selectionBorderExtended",
        L"BorderThickness@CheckedPressed:=0,3,0,0",
        L"BorderThickness=0"}},
```

```
    ThemeTargetStyles{L"SystemTray.OmniButton >
Windows.UI.Xaml.Controls.Grid@CommonStates >
Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"BorderBrush@Checked:=$selectionBorderExtended",
        L"BorderThickness@Checked:=0,3,0,0",
        L"BorderBrush@CheckedPointerOver:=$selectionBorderExtended",
        L"BorderThickness@CheckedPointerOver:=0,3,0,0",
        L"BorderBrush@CheckedPressed:=$selectionBorderExtended",
        L"BorderThickness@CheckedPressed:=0,3,0,0",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.TextBlock#TimeInnerTextBlock", {
        L"FontSize=17.33",
        L"TextAlignment=Center",
        L"FontFamily=Noto Sans, Segoe UI",
        L"Foreground=White"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.TextBlock#DateInnerTextBlock", {
        L"FontSize=13.33",
        L"TextAlignment=Center",
        L"FontFamily=Noto Sans, Segoe UI",
        L"Margin=0,-5,0,0",
        L"Foreground=White"}},
    ThemeTargetStyles{L"SystemTray.DateTimeIconContent >
Windows.UI.Xaml.Controls.Grid#ContainerGrid", {
        L"Padding=0"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView >
Windows.UI.Xaml.Controls.Grid#ContainerGrid >
Windows.UI.Xaml.Controls.ContentPresenter#ContentPresenter >
Windows.UI.Xaml.Controls.Grid#ContentGrid > SystemTray.TextIconContent >
Windows.UI.Xaml.Controls.Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base >
Windows.UI.Xaml.Controls.TextBlock#InnerTextBlock", {
        L"FontSize=17.33",
        L"Foreground=White"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView", {
        L"Margin=-5,0"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon", {
        L"MinWidth=28"}},
    ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack", {
        L"Width=48",
        L"Height=Auto"}},
    ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack > Grid#Content >
SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid,
SystemTray.Stack#ShowDesktopStack > Grid#Content > SystemTray.StackListView#IconStack >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon", {
        L"Width=48"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#ShowDesktopPipe", {
        L"Grid.Row=0",
        L"Grid.RowSpan=3",
        L"Grid.Column=0",
        L"Grid.ColumnSpan=3",
        L"Width=48",
        L"Height=50",
        L"Fill:=<ImageBrush Stretch=\"None\" ImageSource=\"$desktopButton\" />"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView > Windows.UI.Xaml.Controls.Grid >
Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"Background=Transparent",
        L"Margin=1,0,1,0",
        L"CornerRadius=0"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView > Windows.UI.Xaml.Controls.Grid >
Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"Background=Transparent",
```

```
      L"Margin=1,0,1,0",
      L"CornerRadius=0"}},
   ThemeTargetStyles{L"Grid#SystemTrayFrameGrid > SystemTray.Stack#NonActivatableStack", {
      L"Grid.Column=3"}},
   ThemeTargetStyles{L"SystemTray.IconView#SystemTrayIcon >
Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
      L"Background=Transparent",
      L"Margin=1,0,1,0",
      L"CornerRadius=0"}},
   ThemeTargetStyles{L"SystemTray.StackListView[AutomationProperties.AutomationId=Main]",
{
      L"Margin=-8,0,0,0"}},
   ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock#LanguageInnerTextBlock >
Windows.UI.Xaml.Controls.TextBlock", {
      L"FontFamily=Noto Sans, Segoe UI"}},
}, {
   L"taskbandInactiveNormal=<SolidColorBrush Color=\"Gray\" Opacity=\"0.25\" />",
   L"taskbandPointerOver=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight1}\" Opacity=\"0.4\" />",
   L"taskbandActive=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight1}\"
Opacity=\"0.5\" />",
   L"indicatorActive=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight2}\"
Opacity=\"0.7\" />",
   L"indicatorInactive=<SolidColorBrush Color=\"Gray\" Opacity=\"0.7\" />",
   L"indicatorPointerOver=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight2}\" Opacity=\"0.8\" />",
   L"taskbandAttention=<SolidColorBrush Color=\"#ce640c\" Opacity=\"0.5\" />",
   L"indicatorAttention=<SolidColorBrush Color=\"#ce640c\" Opacity=\"0.9\" />",
   L"selectionBorder=<LinearGradientBrush StartPoint='0,0' EndPoint='1,0'><GradientStop
Color='Transparent' Offset='0.0' /><GradientStop Color='Transparent' Offset='0.2'
/><GradientStop Color='{ThemeResource SystemAccentColorLight2}' Offset='0.2'
/><GradientStop Color='{ThemeResource SystemAccentColorLight2}' Offset='0.8'
/><GradientStop Color='Transparent' Offset='0.8' /><GradientStop Color='Transparent'
Offset='1.0' /></LinearGradientBrush>",
   L"selectionBorderExtended=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorLight2}\" />",
   L"desktopButton=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-
styling-guide/refs/heads/main/Themes/Plasma/ThemeResources/desktop.png",
   L"plusIndicator=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-
styling-guide/refs/heads/main/Themes/Plasma/ThemeResources/plus.png",
   L"StartButton=default",
   L"WindhawkBlur=<WindhawkBlur BlurAmount=\"30\" TintColor=\"#cc2a2e32\" />",
   L"Acrylic=<AcrylicBrush TintColor=\"#2a2e32\" TintOpacity=\"0.8\"
FallbackColor=\"#2a2e32\" />",
}};

const Theme g_themeWindowGlass = {{
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid@DockingStates", {
      L"Tag=horizontal",
      L"Tag@DockedLeft=vertical",
      L"Tag@DockedRight=vertical",
      L"Tag=>taskbarDock"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame", {

L"MaxWidth={{containerGridWidth>0?min($TaskbarFrameMaxWidth,containerGridWidth):$Tas
kbarFrameMaxWidth}}",
      L"Width={{taskbarDock==`vertical`?skip():`Auto`}}",
      L"MinWidth:=100",
      L"Grid.Column=1"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
      L"Margin=10,2,0,2",
      L"BorderThickness=0.5,1,0,1",
      L"CornerRadius=$CornerRadius,0,0,$CornerRadius",
```

```
        L"Background:=$Background",
        L"Padding=10,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#BackgroundControl > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Shapes.Rectangle#BackgroundFill", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#BackgroundControl > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Shapes.Rectangle#BackgroundStroke", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
        L"Margin=0,4,0,4",
        L"Background:=Transparent",
        L"CornerRadius=0",
        L"BorderThickness=0,0,1,0",
        L"BorderBrush:=#20808080",
        L"Padding=2,0,5,0",
        L"MaxWidth:=200"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
        L"Grid.Column=2",
        L"Width=Auto",
        L"HorizontalAlignment=Left",
        L"Margin=0"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"Margin=0,2,0,2",
        L"Padding=0,0,8,0",
        L"Background:=$Background",
        L"BorderThickness=0,1,0.5,1",
        L"CornerRadius=0,$CornerRadius,$CornerRadius,0"}},
    ThemeTargetStyles{L":root > ScrollViewer > ScrollContentPresenter > Border > Grid", {
        L"ColumnDefinitions:=<ColumnDefinitionCollection><ColumnDefinition Width=\"*\"/><ColumnDefinition Width=\"Auto\"/><ColumnDefinition Width=\"Auto\"/><ColumnDefinition Width=\"*\"/></ColumnDefinitionCollection>",
        L"ActualWidth=>containerGridWidth"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView", {
        L"Padding=6,0,6,0",
        L"CornerRadius=8",
        L"Margin=2,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon", {
        L"Padding=6,0,6,0",
        L"CornerRadius=8",
        L"Margin=2,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.OmniButton", {
        L"Padding=6,0,6,0",
        L"CornerRadius=8",
        L"Margin=2,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.CopilotIcon", {
        L"Padding=6,0,6,0",
        L"Margin=2,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid", {
        L"Padding=$TrayPadding"}},
    ThemeTargetStyles{L"SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid", {
        L"Padding=10",
        L"CornerRadius=10"}},
    ThemeTargetStyles{L"SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon", {
```

```
    L"Padding=0"}},
  ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack", {
    L"Visibility=Visible"}},
  ThemeTargetStyles{L"Taskbar.Gripper#GripperControl", {
    L"Width=Auto",
    L"MinWidth=24"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#AugmentedEntryPointContentGrid", {
    L"Margin=4,0,0,0"}},
  ThemeTargetStyles{L"TextBlock#TimeInnerTextBlock", {
    L"FontSize=13",
    L"FontFamily=vivo Sans EN VF",
    L"Margin=0",
    L"Padding=0",
    L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"0\" />"}},
  ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
    L"Visibility=Collapsed",
    L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"-9\" />",
    L"FontSize=11",
    L"FontFamily=vivo Sans EN VF"}},
  ThemeTargetStyles{L"TextBlock#InnerTextBlock[Text=\uE971]", {
    L"Text=\uED14"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#ConfirmatorMainGrid", {
    L"CornerRadius=22",
    L"BorderThickness=$BorderThickness",
    L"BorderBrush:=$BorderBrush",
    L"Background:=$Background"}},
  ThemeTargetStyles{L"TextBlock#SearchBoxTextBlock", {
    L"Text=Search",
    L"FontSize=10",
    L"FontFamily=vivo Sans EN VF"}},
  ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Grid >
ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.IconView#SystemTrayIcon > Grid > Grid > SystemTray.TextIconContent", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button", {
    L"BorderThickness=$BorderThickness"}},
  ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab >
Windows.UI.Xaml.Controls.Grid#ModalRootGrid >
Windows.UI.Xaml.Controls.Border#BackgroundElement", {
    L"BorderThickness=0",
    L"Background=Transparent",
    L"CornerRadius=$CornerRadius"}},
  ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab >
Windows.UI.Xaml.Controls.Grid#ModalRootGrid >
Windows.UI.Xaml.Controls.Border#BackgroundElement >
WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemList", {
    L"Background:=$Background",
    L"BorderThickness=$BorderThickness",
    L"BorderBrush:=$BorderBrush",
    L"CornerRadius=$CornerRadius"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopB
arElement#VirtualDesktopBar > Grid > Border", {
    L"BorderThickness=$BorderThickness",
    L"BorderBrush:=$BorderBrush",
    L"Background:=$Background",
    L"CornerRadius=$CornerRadius"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopB
arElement#VirtualDesktopBar", {
```

```
      L"Width=Auto",
      L"Visibility=Visible",
      L"HorizontalAlignment=1"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#BackgroundDimmingLayer", {
      L"Background:=Transparent"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton", {
      L"CornerRadius=10"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapBarBorder", {
      L"Background:=$Background",
      L"BorderBrush:=$BorderBrush",
      L"CornerRadius=$CornerRadius",
      L"BorderThickness=$BorderThickness",
      L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"10\" />",
      L"Margin=0,0,0,-10"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapPickerBorder", {
      L"Background:=$Background",
      L"BorderBrush:=$BorderBrush",
      L"CornerRadius=$CornerRadius",
      L"BorderThickness=$BorderThickness"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SearchPillBackgroundElement", {
      L"BorderBrush:=$ElementBorderBrush",
      L"CornerRadius=$ElementCornerRadius",
      L"BorderThickness=$ElementBorderThickness",
      L"MaxWidth:=100",
      L"Width=Auto"}},
    ThemeTargetStyles{L"Taskbar.TaskbarExtensionElement", {
      L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"0\" />"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
      L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"0\" />"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.ToolTip >
Windows.UI.Xaml.Controls.ContentPresenter#LayoutRoot", {
      L"Background:=$Background",
      L"BorderBrush:=$BorderBrush",
      L"BorderThickness:=$BorderThickness",
      L"CornerRadius=12"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl", {
      L"MaxWidth:=300",
      L"MinWidth:=10",
      L"Width=Auto",
      L"Margin=0"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopB
arElement > Windows.UI.Xaml.Controls.Grid#GridElement >
Windows.UI.Xaml.Controls.Border#VirtualDesktopSwitcherBackground", {
      L"Background:=$Background",
      L"BorderBrush:=$BorderBrush",
      L"BorderThickness=$BorderThickness",
      L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#BackgroundFill", {
      L"Fill:=$Background"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#OverflowFlyoutBackgroundBorder",
{
      L"Background:=$Background",
      L"BorderBrush:=$BorderBrush",
      L"BorderThickness=$BorderThickness",
      L"CornerRadius=$CornerRadius"}},
```

```
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.MenuFlyoutPresenter > Windows.UI.Xaml.Controls.Border", {
      L"Background:=$Background",
      L"BorderBrush:=$BorderBrush",
      L"BorderThickness=$BorderThickness",
      L"CornerRadius=$CornerRadius"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid > Windows.UI.Xaml.Controls.Border#HoverFlyoutBackground", {
      L"Background:=Transparent",
      L"BorderBrush:=$BorderBrush",
      L"BorderThickness=$BorderThickness",
      L"CornerRadius=$CornerRadius"}},
}, {
   L"Translucent=<WindhawkBlur BlurAmount=\"15\" TintColor=\"#10808080\"/>",
   L"Glass=<WindhawkBlur BlurAmount=\"5\" TintColor=\"{ThemeResource SystemChromeMediumColor}\" TintOpacity=\"0.7\" />",
   L"Frosted=<WindhawkBlur BlurAmount=\"20\" TintColor=\"{ThemeResource SystemChromeMediumColor}\" TintOpacity=\"0.7\" />",
   L"Acrylic=<WindhawkBlur BlurAmount=\"30\" TintColor=\"{ThemeResource SystemChromeMediumColor}\" TintOpacity=\"0.8\" />",
   L"Background=$Glass",
   L"BorderBrush=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#50808080\" Offset=\"0.0\" /><GradientStop Color=\"#30404040\" Offset=\"0.25\" /><GradientStop Color=\"#40808080\" Offset=\"1\" /></LinearGradientBrush>",
   L"BorderBrush2=<WindhawkBlur BlurAmount=\"10\" TintColor=\"#909090\" TintOpacity=\"0.2\"/>",
   L"ElementBG=<SolidColorBrush Color=\"{ThemeResource SystemChromeLowColor}\" Opacity=\"0.3\" />",
   L"ElementBorderThickness=0.3,0.3,0.3,1",
   L"ElementBorderBrush=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#50808080\" Offset=\"1\" /><GradientStop Color=\"#50606060\" Offset=\"0.15\" /></LinearGradientBrush>",
   L"ElementCornerRadius=12",
   L"ElementSysColor=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight1}\" Opacity=\"1\" />",
   L"ElementSysColor2=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight2}\" Opacity=\"1\" />",
   L"ElementSysColor3=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight3}\" Opacity=\"1\" />",
   L"ElementSysColor4=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorDark1}\" Opacity=\"1\" />",
   L"BorderThickness=0.5,1,0.5,1",
   L"CornerRadius=25",
   L"TrayPadding=2,4,2,4",
   L"Height=70",
   L"TaskbarFrameMaxWidth=1895",
}};

const Theme g_themeWindowGlass_variant_Split = {{
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid@DockingStates", {
      L"Tag=horizontal",
      L"Tag@DockedLeft=vertical",
      L"Tag@DockedRight=vertical",
      L"Tag=>taskbarDock"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
      L"Grid.Column=1",
      L"MaxWidth={{containerGridWidth>0?min($TaskbarFrameMaxWidth,containerGridWidth):$TaskbarFrameMaxWidth}}",
      L"Width={{taskbarDock==`vertical`?skip():`Auto`}}",
      L"MinWidth:=100"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
```

```
        L"Margin=10,2,3,2",
        L"BorderThickness=$BorderThickness",
        L"CornerRadius=25,5,5,25",
        L"Background:=$Background",
        L"Padding=10,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#BackgroundControl > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Shapes.Rectangle#BackgroundFill", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#BackgroundControl > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Shapes.Rectangle#BackgroundStroke", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
        L"Margin=0,8,0,8",
        L"Background:=Transparent",
        L"CornerRadius=0",
        L"BorderThickness=0,0,1,0",
        L"BorderBrush:=#20808080",
        L"Padding=2,0,5,0",
        L"MaxWidth:=200"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"Margin=0,2,3,2",
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness=$BorderThickness",
        L"CornerRadius=5,25,25,5",
        L"Padding=0,0,10,0"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView", {
        L"Padding=$TrayPadding",
        L"CornerRadius=10",
        L"Margin=5,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon", {
        L"Padding=$TrayPadding",
        L"CornerRadius=10",
        L"Margin=5,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.OmniButton", {
        L"Padding=$TrayPadding",
        L"CornerRadius=10"}},
    ThemeTargetStyles{L"SystemTray.CopilotIcon", {
        L"Padding=$TrayPadding"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid", {
        L"Padding=$TrayPadding"}},
    ThemeTargetStyles{L"SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid", {
        L"Padding=10",
        L"CornerRadius=10"}},
    ThemeTargetStyles{L"SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon", {
        L"Padding=0"}},
    ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack", {
        L"Visibility=Visible"}},
    ThemeTargetStyles{L"Taskbar.Gripper#GripperControl", {
        L"Width=Auto",
        L"MinWidth=24"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#AugmentedEntryPointContentGrid", {
```

```
        L"Margin=4,0,0,0"}},
    ThemeTargetStyles{L"TextBlock#TimeInnerTextBlock", {
        L"FontSize=13",
        L"FontFamily=vivo Sans EN VF",
        L"Margin=0",
        L"Padding=0",
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"0\" />"}},
    ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
        L"Visibility=Collapsed",
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"-9\" />",
        L"FontSize=11",
        L"FontFamily=vivo Sans EN VF"}},
    ThemeTargetStyles{L"TextBlock#InnerTextBlock[Text=\uE971]", {
        L"Text=\uED14"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#ConfirmatorMainGrid", {
        L"CornerRadius=22",
        L"BorderThickness=$BorderThickness",
        L"BorderBrush:=$BorderBrush",
        L"Background:=$Background"}},
    ThemeTargetStyles{L"TextBlock#SearchBoxTextBlock", {
        L"Text=Search",
        L"FontSize=10",
        L"FontFamily=vivo Sans EN VF"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Grid >
ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.IconView#SystemTrayIcon > Grid > Grid > SystemTray.TextIconContent", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button", {
        L"BorderThickness=$BorderThickness"}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab >
Windows.UI.Xaml.Controls.Grid#ModalRootGrid >
Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"BorderThickness=0",
        L"Background=Transparent",
        L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab >
Windows.UI.Xaml.Controls.Grid#ModalRootGrid >
Windows.UI.Xaml.Controls.Border#BackgroundElement >
WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemList", {
        L"Background:=$Background",
        L"BorderThickness=$BorderThickness",
        L"BorderBrush:=$BorderBrush",
        L"CornerRadius=$CornerRadius"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopB
arElement#VirtualDesktopBar > Grid > Border", {
        L"BorderThickness=$BorderThickness",
        L"BorderBrush:=$BorderBrush",
        L"Background:=$Background",
        L"CornerRadius=$CornerRadius"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopB
arElement#VirtualDesktopBar", {
        L"Width=Auto",
        L"Visibility=Visible",
        L"HorizontalAlignment=1"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#BackgroundDimmingLayer", {
        L"Background:=Transparent"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundElement,
```

```
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Windows.UI.Xaml.Controls.Border#BackgroundElement,
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundElement,
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Windows.UI.Xaml.Controls.Border#BackgroundElement", {
      L"CornerRadius=10"}},
   ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton", {
      L"CornerRadius=10"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapBarBorder", {
      L"Background:=$Background",
      L"BorderBrush:=$BorderBrush",
      L"CornerRadius=$CornerRadius",
      L"BorderThickness=$BorderThickness",
      L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"10\" />",
      L"Margin=0,0,0,-10"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapPickerBorder", {
      L"Background:=$Background",
      L"BorderBrush:=$BorderBrush",
      L"CornerRadius=$CornerRadius",
      L"BorderThickness=$BorderThickness"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SearchPillBackgroundElement", {
      L"BorderBrush:=$ElementBorderBrush",
      L"CornerRadius=$ElementCornerRadius",
      L"BorderThickness=$ElementBorderThickness",
      L"MaxWidth:=100",
      L"Width=Auto"}},
   ThemeTargetStyles{L"Taskbar.TaskbarExtensionElement", {
      L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"0\" />"}},
   ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
      L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"0\" />"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.ToolTip >
Windows.UI.Xaml.Controls.ContentPresenter#LayoutRoot", {
      L"Background:=$Background",
      L"BorderBrush:=$BorderBrush",
      L"BorderThickness:=$BorderThickness",
      L"CornerRadius=12"}},
   ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl", {
      L"MaxWidth:=300",
      L"MinWidth:=10",
      L"Width=Auto",
      L"Margin=0"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopB
arElement" > Windows.UI.Xaml.Controls.Grid#GridElement >
Windows.UI.Xaml.Controls.Border#VirtualDesktopSwitcherBackground", {
      L"Background:=$Background",
      L"BorderBrush:=$BorderBrush",
      L"BorderThickness=$BorderThickness",
      L"CornerRadius=$CornerRadius"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#BackgroundFill", {
      L"Fill:=$Background"}},
   ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
      L"Grid.Column=2"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#OverflowFlyoutBackgroundBorder",
{
      L"Background:=$Background",
      L"BorderBrush:=$BorderBrush",
      L"BorderThickness=$BorderThickness",
```

```
      L"CornerRadius=$CornerRadius"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.MenuFlyoutPresenter > Windows.UI.Xaml.Controls.Border", {
      L"Background:=$Background",
      L"BorderBrush:=$BorderBrush",
      L"BorderThickness=$BorderThickness",
      L"CornerRadius=$CornerRadius"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid > Windows.UI.Xaml.Controls.Border#HoverFlyoutBackground", {
      L"Background:=Transparent",
      L"BorderBrush:=$BorderBrush",
      L"BorderThickness=$BorderThickness",
      L"CornerRadius=$CornerRadius"}},
   ThemeTargetStyles{L":root > ScrollViewer > ScrollContentPresenter > Border > Grid", {
      L"ColumnDefinitions:=<ColumnDefinitionCollection><ColumnDefinition Width=\"*\"/><ColumnDefinition Width=\"Auto\"/><ColumnDefinition Width=\"Auto\"/><ColumnDefinition Width=\"*\"/></ColumnDefinitionCollection>",
      L"ActualWidth=>containerGridWidth"}},
}, {
   L"Translucent=<WindhawkBlur BlurAmount=\"15\" TintColor=\"#10808080\"/>",
   L"Glass=<WindhawkBlur BlurAmount=\"5\" TintColor=\"{ThemeResource SystemChromeMediumColor}\" TintOpacity=\"0.7\" />",
   L"Frosted=<WindhawkBlur BlurAmount=\"20\" TintColor=\"{ThemeResource SystemChromeMediumColor}\" TintOpacity=\"0.7\" />",
   L"Acrylic=<WindhawkBlur BlurAmount=\"30\" TintColor=\"{ThemeResource SystemChromeMediumColor}\" TintOpacity=\"0.8\" />",
   L"Background=$Glass",
   L"BorderBrush=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#50808080\" Offset=\"0.0\" /><GradientStop Color=\"#30404040\" Offset=\"0.25\" /><GradientStop Color=\"#40808080\" Offset=\"1\" /></LinearGradientBrush>",
   L"BorderBrush2=<WindhawkBlur BlurAmount=\"10\" TintColor=\"#909090\" TintOpacity=\"0.2\"/>",
   L"ElementBG=<SolidColorBrush Color=\"{ThemeResource SystemChromeLowColor}\" Opacity=\"0.3\" />",
   L"ElementBorderThickness=0.3,0.3,0.3,1",
   L"ElementBorderBrush=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#50808080\" Offset=\"1\" /><GradientStop Color=\"#50606060\" Offset=\"0.15\" /></LinearGradientBrush>",
   L"ElementCornerRadius=12",
   L"ElementSysColor=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight1}\" Opacity=\"1\" />",
   L"ElementSysColor2=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight2}\" Opacity=\"1\" />",
   L"ElementSysColor3=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight3}\" Opacity=\"1\" />",
   L"ElementSysColor4=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorDark1}\" Opacity=\"1\" />",
   L"BorderThickness=0.5,1,0.5,1",
   L"CornerRadius=25",
   L"TrayPadding=2,4,2,4",
   L"Height=70",
   L"TaskbarFrameMaxWidth=1895",
}};

const Theme g_themeWindowGlass_variant_FullLength = {{
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid@DockingStates", {
      L"Tag=horizontal",
      L"Tag@DockedLeft=vertical",
      L"Tag@DockedRight=vertical",
      L"Tag=>taskbarDock"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
      L"Width={{taskbarDock==`vertical`?skip():`Auto`}}",
```

```
        L"MinWidth:=100"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
        L"Margin=10,2,10,2",
        L"BorderThickness=$BorderThickness",
        L"CornerRadius=$CornerRadius",
        L"Background:=$Background",
        L"Padding=0"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#BackgroundControl > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Shapes.Rectangle#BackgroundFill", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#BackgroundControl > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Shapes.Rectangle#BackgroundStroke", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
        L"Margin=0,10,0,10",
        L"Background:=Transparent",
        L"CornerRadius=0",
        L"BorderThickness=1,0,1,0",
        L"BorderBrush:=#20808080",
        L"Padding=2,0,5,0",
        L"MaxWidth:=200"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"Margin=0,8,8,7",
        L"Background:=Transparent",
        L"BorderBrush:=Transparent",
        L"BorderThickness=$ElementBorderThickness",
        L"CornerRadius=$ElementCornerRadius"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView", {
        L"Padding=$TrayPadding",
        L"CornerRadius=10",
        L"Margin=5,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon", {
        L"Padding=$TrayPadding",
        L"CornerRadius=10",
        L"Margin=5,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.OmniButton", {
        L"Padding=$TrayPadding",
        L"CornerRadius=10"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > StackPanel", {
        L"Padding=0,-4,0,-4"}},
    ThemeTargetStyles{L"SystemTray.CopilotIcon", {
        L"Padding=$TrayPadding"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid", {
        L"Padding=$TrayPadding"}},
    ThemeTargetStyles{L"SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid", {
        L"Padding=10",
        L"CornerRadius=10"}},
    ThemeTargetStyles{L"SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon", {
        L"Padding=0"}},
    ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack", {
        L"Visibility=Visible"}},
    ThemeTargetStyles{L"Taskbar.Gripper#GripperControl", {
```

```
        L"Width=Auto",
        L"MinWidth=24"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#AugmentedEntryPointContentGrid", {
        L"Margin=4,0,0,0",
        L"HorizontalAlignment=Left"}},
    ThemeTargetStyles{L"TextBlock#TimeInnerTextBlock", {
        L"FontSize=13",
        L"FontFamily=vivo Sans EN VF",
        L"Margin=0",
        L"Padding=0",
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"0\" />"}},
    ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
        L"Visibility=Collapsed",
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"-9\" />",
        L"FontSize=11",
        L"FontFamily=vivo Sans EN VF"}},
    ThemeTargetStyles{L"TextBlock#InnerTextBlock[Text=\uE971]", {
        L"Text=\uED14"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#ConfirmatorMainGrid", {
        L"CornerRadius=22",
        L"BorderThickness=$BorderThickness",
        L"BorderBrush:=$BorderBrush",
        L"Background:=$Background"}},
    ThemeTargetStyles{L"TextBlock#SearchBoxTextBlock", {
        L"Text=Search",
        L"FontSize=10",
        L"FontFamily=vivo Sans EN VF"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Grid >
ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.IconView#SystemTrayIcon > Grid > Grid > SystemTray.TextIconContent", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button", {
        L"BorderThickness=$BorderThickness"}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab >
Windows.UI.Xaml.Controls.Grid#ModalRootGrid >
Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"BorderThickness=0",
        L"Background=Transparent",
        L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab >
Windows.UI.Xaml.Controls.Grid#ModalRootGrid >
Windows.UI.Xaml.Controls.Border#BackgroundElement >
WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemList", {
        L"Background:=$Background",
        L"BorderThickness=$BorderThickness",
        L"BorderBrush:=$BorderBrush",
        L"CornerRadius=$CornerRadius"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopB
arElement#VirtualDesktopBar > Grid > Border", {
        L"BorderThickness=$BorderThickness",
        L"BorderBrush:=$BorderBrush",
        L"Background:=$Background",
        L"CornerRadius=$CornerRadius"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopB
arElement#VirtualDesktopBar", {
        L"Width=Auto",
        L"Visibility=Visible",
        L"HorizontalAlignment=1"}},
```

```
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#BackgroundDimmingLayer", {
        L"Background:=Transparent"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundElement,
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Windows.UI.Xaml.Controls.Border#BackgroundElement,
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundElement,
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"CornerRadius=10"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton", {
        L"CornerRadius=10"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapBarBorder", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"CornerRadius=$CornerRadius",
        L"BorderThickness=$BorderThickness",
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"10\" />",
        L"Margin=0,0,0,-10"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapPickerBorder", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"CornerRadius=$CornerRadius",
        L"BorderThickness=$BorderThickness"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SearchPillBackgroundElement", {
        L"BorderBrush:=$ElementBorderBrush",
        L"CornerRadius=$ElementCornerRadius",
        L"BorderThickness=$ElementBorderThickness",
        L"MaxWidth:=100",
        L"Width=Auto"}},
    ThemeTargetStyles{L"Taskbar.TaskbarExtensionElement", {
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"0\" />"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"0\" />"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.ToolTip >
Windows.UI.Xaml.Controls.ContentPresenter#LayoutRoot", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness:=$BorderThickness",
        L"CornerRadius=12"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl", {
        L"MaxWidth:=300",
        L"MinWidth:=10",
        L"Width=Auto",
        L"Margin=0"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopB
arElement"> Windows.UI.Xaml.Controls.Grid#GridElement >
Windows.UI.Xaml.Controls.Border#VirtualDesktopSwitcherBackground", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness=$BorderThickness",
        L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#BackgroundFill", {
        L"Fill:=$Background"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#OverflowFlyoutBackgroundBorder",
{
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
```

```
      L"BorderThickness=$BorderThickness",
      L"CornerRadius=$CornerRadius"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.MenuFlyoutPresenter > Windows.UI.Xaml.Controls.Border", {
      L"Background:=$Background",
      L"BorderBrush:=$BorderBrush",
      L"BorderThickness=$BorderThickness",
      L"CornerRadius=$CornerRadius"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid > Windows.UI.Xaml.Controls.Border#HoverFlyoutBackground", {
      L"Background:=Transparent",
      L"BorderBrush:=$BorderBrush",
      L"BorderThickness=$BorderThickness",
      L"CornerRadius=$CornerRadius"}},
}, {
   L"Translucent=<WindhawkBlur BlurAmount=\"15\" TintColor=\"#10808080\"/>",
   L"Glass=<WindhawkBlur BlurAmount=\"5\" TintColor=\"{ThemeResource SystemChromeMediumColor}\" TintOpacity=\"0.7\" />",
   L"Frosted=<WindhawkBlur BlurAmount=\"20\" TintColor=\"{ThemeResource SystemChromeMediumColor}\" TintOpacity=\"0.7\" />",
   L"Acrylic=<WindhawkBlur BlurAmount=\"30\" TintColor=\"{ThemeResource SystemChromeMediumColor}\" TintOpacity=\"0.8\" />",
   L"Background=$Glass",
   L"BorderBrush=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#50808080\" Offset=\"0.0\" /><GradientStop Color=\"#30404040\" Offset=\"0.25\" /><GradientStop Color=\"#40808080\" Offset=\"1\" /></LinearGradientBrush>",
   L"BorderBrush2=<WindhawkBlur BlurAmount=\"10\" TintColor=\"#909090\" TintOpacity=\"0.2\"/>",
   L"ElementBG=<SolidColorBrush Color=\"{ThemeResource SystemChromeLowColor}\" Opacity=\"0.3\" />",
   L"ElementBorderThickness=0.3,0.3,0.3,1",
   L"ElementBorderBrush=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#50808080\" Offset=\"1\" /><GradientStop Color=\"#50606060\" Offset=\"0.15\" /></LinearGradientBrush>",
   L"ElementCornerRadius=12",
   L"ElementSysColor=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight1}\" Opacity=\"1\" />",
   L"ElementSysColor2=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight2}\" Opacity=\"1\" />",
   L"ElementSysColor3=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight3}\" Opacity=\"1\" />",
   L"ElementSysColor4=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorDark1}\" Opacity=\"1\" />",
   L"BorderThickness=0.5,1,0.5,1",
   L"CornerRadius=20",
   L"TrayPadding=2,4,2,4",
   L"Height=70",
}};

const Theme g_themeSurface = {{
   ThemeTargetStyles{L"Grid#RootGrid > Taskbar.TaskbarBackground", {
      L"Margin=-20,0,-20,0"}},
   ThemeTargetStyles{L"Grid#RootGrid > Taskbar.TaskbarBackground > Grid", {
      L"CornerRadius=20",
      L"BorderThickness=1",
      L"BorderBrush=#40FFFFFF",
      L"Padding=-1"}},
   ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
      L"Fill=Transparent"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid@DockingStates", {
      L"Tag=horizontal",
```

```
        L"Tag@DockedLeft=vertical",
        L"Tag@DockedRight=vertical",
        L"Tag=>taskbarDock"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
        L"Width={{taskbarDock==`vertical`?skip():`Auto`}}",
        L"HorizontalAlignment=Center"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
        L"Visibility=Visible",
        L"Margin=0,0,0,10",
        L"Padding=20,0,20,0"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"Margin=0,0,0,10",
        L"CornerRadius=20,0,0,20",
        L"BorderThickness=1,1,0,1",
        L"BorderBrush=#66FFFFFF",
        L"Padding=10,5,0,5",
        L"Background:=<WindhawkBlur BlurAmount=\"5\" TintColor=\"{ThemeResource
SystemChromeAltHighColor}\" TintOpacity=\"0.5\" />",
        L"Visibility=Visible"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground >
Grid > Rectangle#BackgroundFill", {
        L"Fill:=<WindhawkBlur BlurAmount=\"5\" TintColor=\"#12FFFFFF\"/>"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Border#BackgroundElement,
Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Border#BackgroundElement", {
        L"Background:=$TaskItemBackground",
        L"Margin=-1,5.5,1,4",
        L"CornerRadius=12",
        L"BorderThickness=2,1,0.5,2",
        L"BorderBrush:=$TaskItemBorder"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid >
Border#BackgroundElement, Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Border#BackgroundElement,
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Grid >
Border#BackgroundElement,
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Border#BackgroundElement", {
        L"Background:=$SystemItemBackground",
        L"CornerRadius=12",
        L"Margin=-1,5.5,2.5,4",
        L"BorderThickness=2,1,0.5,2",
        L"BorderBrush:=$SystemItemBorder"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Rectangle#RunningIndicator,
Taskbar.TaskListLabeledButtonPanel@CommonStates > Rectangle#RunningIndicator", {
        L"Margin=0,0,0,8"}},
    ThemeTargetStyles{L"Border#MultiWindowElement", {
        L"Height=0"}},
}, {
    L"TaskItemBackground=<AcrylicBrush TintColor=\"{ThemeResource
SystemChromeAltHighColor}\" TintOpacity=\"0.9\" FallbackColor=\"{ThemeResource
SystemChromeMediumColor}\" />",
    L"TaskItemBorder=<LinearGradientBrush StartPoint=\"0,0\"
EndPoint=\"0,1\"><GradientStop Color=\"#00000000\" Offset=\"0\" /><GradientStop
Color=\"#33000000\" Offset=\"1.5\" /></LinearGradientBrush>",
    L"SystemItemBackground=<AcrylicBrush TintColor=\"{ThemeResource
SystemChromeAltHighColor}\" TintOpacity=\"0.8\" FallbackColor=\"{ThemeResource
SystemChromeLowColor}\" />",
    L"SystemItemBorder=<LinearGradientBrush StartPoint=\"0,0\"
EndPoint=\"0,1\"><GradientStop Color=\"#00000000\" Offset=\"0\" /><GradientStop
Color=\"#33000000\" Offset=\"1.5\" /></LinearGradientBrush>",
}};

const Theme g_themeOversimplified_Accentuated = {{
    ThemeTargetStyles{L"MenuFlyoutPresenter", {
```

```
        L"Background:=$DarkAccent",
        L"BorderBrush=Transparent",
        L"Shadow:="}},
    ThemeTargetStyles{L"ToolTip > ContentPresenter#LayoutRoot", {
        L"Background:=$DarkAccent",
        L"BorderBrush=Transparent",
        L"Shadow:="}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground >
Grid > Rectangle#BackgroundFill", {
        L"Fill:=$Alt"}},
    ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid >
ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid >
SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base >
TextBlock#InnerTextBlock", {
        L"FontSize=22"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Rectangle#RunningIndicator,
Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Rectangle#RunningIndicator", {
        L"Fill@ActiveRunningIndicator:=$SolidAccent",
        L"Height=4",
        L"Width@ActiveRunningIndicator=25"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton > Grid#IconPanel >
Microsoft.UI.Xaml.Controls.ProgressBar#ProgressIndicator, Taskbar.TaskListButton >
Taskbar.TaskListLabeledButtonPanel >
Microsoft.UI.Xaml.Controls.ProgressBar#ProgressIndicator", {
        L"MinHeight=4",
        L"Width=25"}},
    ThemeTargetStyles{L"Grid#LayoutRoot@CommonStates > Border#ProgressBarRoot >
Border > Grid > Rectangle#DeterminateProgressBarIndicator", {
        L"Fill@Updating:= <SolidColorBrush Color=\"Green\" Opacity=\"1\" />",
        L"Fill@Determinate:= <SolidColorBrush Color=\"Green\" Opacity=\"1\" />",
        L"Fill@Paused:= <SolidColorBrush Color=\"Orange\" Opacity=\"1\" />",
        L"Fill@Error:= <SolidColorBrush Color=\"Red\" Opacity=\"1\" />",
        L"Fill@UpdatingError:= <SolidColorBrush Color=\"Red\" Opacity=\"1\" />"}},
    ThemeTargetStyles{L"Rectangle#ProgressBarTrack", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"Canvas#HoverFlyoutCanvas > Grid#HoverFlyoutGrid >
Border#HoverFlyoutBackground", {
        L"BorderBrush=Transparent",
        L"Shadow:="}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid >
Windows.UI.Xaml.Shapes.Rectangle#BackgroundFill", {
        L"Fill:=$DarkAccent"}},
    ThemeTargetStyles{L"Grid#OverflowRootGrid", {
        L"Padding:="}},
    ThemeTargetStyles{L"Border#OverflowFlyoutBackgroundBorder", {
        L"Background:=$Alt",
        L"Shadow:=",
        L"BorderThickness:="}},
    ThemeTargetStyles{L"SystemTray.ImageIconContent >
Windows.UI.Xaml.Controls.Grid#ContainerGrid > Windows.UI.Xaml.Controls.Image", {
        L"Height=20",
        L"Width=20"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputS
witcher > ContentControl > ContentPresenter > Grid", {
        L"Background:=$DarkAccent",
        L"BorderBrush=Transparent",
        L"Shadow:="}},
```

```
ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputSwitcher > ContentControl > ContentPresenter > Grid > Grid#OverlayPanel", {
      L"Background=Transparent",
      L"BorderBrush=Transparent"}},
   ThemeTargetStyles{L"Grid > HyperlinkButton#Footer", {
      L"HorizontalContentAlignment = 1"}},
   ThemeTargetStyles{L"Grid#ConfirmatorMainGrid", {
      L"Background:=$DarkAccent",
      L"BorderBrush=Transparent",
      L"CornerRadius=15",
      L"Margin=0,0,0,5",
      L"Padding=4,0,0,0",
      L"Shadow:="}},
   ThemeTargetStyles{L"Grid#BrightnessConfirmator", {
      L"Padding=6,0,16,0"}},
   ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.AnimatedIcon#BrightnessIcon", {
      L"Height=30",
      L"Width=30",
      L"Margin=0,-1,12,0"}},
   ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.AnimatedIcon#VolumeIcon", {
      L"Height=30",
      L"Width=30"}},
   ThemeTargetStyles{L"TextBlock#volumeLevelText", {
      L"FontSize=15"}},
   ThemeTargetStyles{L"Rectangle#HorizontalDecreaseRect", {
      L"Height=6",
      L"Margin= 0,2,0,0"}},
   ThemeTargetStyles{L"Rectangle#HorizontalTrackRect", {
      L"Fill=Transparent",
      L"Height=6"}},
   ThemeTargetStyles{L"Grid#HorizontalTemplate > Rectangle#HorizontalDecreaseRect", {
      L"Fill:= <AcrylicBrush TintColor=\"{ThemeResource SystemAccentColor}\" TintOpacity=\"1\" TintLuminosityOpacity=\"1\" FallbackColor=\"{ThemeResource SystemAccentColorDark2}\" />"}},
   ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab > Grid#ModalRootGrid > Border#BackgroundElement", {
      L"Background=Transparent",
      L"BorderThickness=0",
      L"CornerRadius=20",
      L"Shadow:="}},
   ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab > Grid#ModalRootGrid > Border#BackgroundElement > WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemList", {
      L"Background:=$DarkAccent"}},
   ThemeTargetStyles{L"Border#BackgroundDimmingLayer", {
      L"Background:= <WindhawkBlur BlurAmount=\"30\" TintColor=\"#00000080\" />"}},
   ThemeTargetStyles{L"Border#VirtualDesktopBarBackground", {
      L"Background:= <SolidColorBrush Color=\"{ThemeResource SystemAccentColorDark1}\" Opacity=\"0.4\" />",
      L"BorderBrush=Transparent"}},
}, {
   L"Alt= <AcrylicBrush TintColor=\"{ThemeResource SystemAltHighColor}\" TintOpacity=\"0.6\" TintLuminosityOpacity=\"0.6\" FallbackColor=\"{ThemeResource SystemAltHighColor}\" />",
   L"Accent = <AcrylicBrush TintColor=\"{ThemeResource SystemAccentColor}\" TintOpacity=\"0.6\" TintLuminosityOpacity=\"0.6\" FallbackColor=\"{ThemeResource SystemAccentColor}\" />",
   L"DarkAccent = <AcrylicBrush TintColor=\"{ThemeResource SystemAccentColorDark1}\" TintOpacity=\"0.6\" TintLuminosityOpacity=\"0.3\" FallbackColor=\"{ThemeResource SystemAccentColorDark1}\" />",
```

```
    L"SolidAccent = <SolidColorBrush Color=\"{ThemeResource SystemAccentColor}\" Opacity=\"1\" />",
    L"Reveal= <RevealBorderBrush Color=\"Transparent\" TargetTheme=\"1\" Opacity=\"1\" />",
}};

const Theme g_themeLuminosity_variant_Dock = {{
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground > Grid > Rectangle#BackgroundFill", {
        L"Fill:=$mbg"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundElement, Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"CornerRadius=$bcr"}},
    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton", {
        L"CornerRadius=$bcr"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton", {
        L"CornerRadius=$bcr"}},

    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundElement, SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"CornerRadius=$bcr"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SearchPillBackgroundElement", {
        L"CornerRadius=$bcr"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#MultiWindowElement", {
        L"CornerRadius=$bcr"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView > Windows.UI.Xaml.Controls.Grid#ContainerGrid > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"CornerRadius=$bcr",
        L"Margin=0,4,2,4"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView > Windows.UI.Xaml.Controls.Grid#ContainerGrid > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"CornerRadius=$bcr",
        L"Margin=2,4,2,4"}},
    ThemeTargetStyles{L"SystemTray.IconView#SystemTrayIcon > Windows.UI.Xaml.Controls.Grid#ContainerGrid > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"CornerRadius=$bcr",
        L"Margin=2,4,2,4"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"CornerRadius=$bcr",
        L"Margin=2,4,2,4"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"CornerRadius=$bcr",
        L"Margin=2,4,2,4"}},
    ThemeTargetStyles{L"Border#OverflowFlyoutBackgroundBorder", {
        L"Background:=$mbg",
        L"CornerRadius:=$wcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb",
        L"Shadow:="}},
    ThemeTargetStyles{L"Taskbar.OverflowToggleButton#OverflowButton > Taskbar.TaskListButtonPanel#OverflowToggleButtonRootPanel > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundElement, Taskbar.OverflowToggleButton#OverflowButton > Taskbar.TaskListButtonPanel#OverflowToggleButtonRootPanel > Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"CornerRadius=$bcr"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#ConfirmatorMainGrid", {
```

```
        L"Background:=$mbg",
        L"CornerRadius=$wcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb",
        L"Shadow:="}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalTrackRect", {
        L"Fill:=#10FFFFFF"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputSwitcher > ContentControl > ContentPresenter > Grid", {
        L"Background:=$mbg",
        L"CornerRadius=$wcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb",
        L"Shadow:="}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputSwitcher > ContentControl > ContentPresenter > Grid > Grid", {
        L"Background:=$t"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid > Rectangle#BackgroundFill", {
        L"Fill:=$t"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid > Windows.UI.Xaml.Controls.Border#HoverFlyoutBackground", {
        L"Background:=$mbg",
        L"CornerRadius=$mcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb",
        L"Shadow:="}},
    ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView > Grid@CommonStates > Border#BackgroundBorder", {
        L"Background=$t",
        L"CornerRadius=$mcr",
        L"BorderThickness@Normal=0",
        L"BorderThickness@PointerOver=0.05,0,0.05,1",
        L"BorderBrush@Normal=$t",
        L"BorderBrush@PointerOver:=$AccentColor"}},
    ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView > Grid > Button#CloseButton", {
        L"CornerRadius=$mcr"}},
    ThemeTargetStyles{L"Taskbar.ThumbBarButton#ThumbBarButton > Windows.UI.Xaml.Controls.ContentPresenter#BorderElement@CommonStates", {
        L"CornerRadius=16",
        L"Margin=-1.5",
        L"Background@Disabled:=$t",
        L"Background@Normal:=$t",
        L"Background@PointerOver:=$nbth",
        L"Background@Pressed:=$nbtp",
        L"BorderThickness=2",
        L"BorderBrush@Disabled:=$t",
        L"BorderBrush@Normal:=$t",
        L"BorderBrush@PointerOver:=$nbb",
        L"BorderBrush@Pressed:=$nbb",
        L"BackgroundSizing=InnerBorderEdge",
        L"BackgroundTransition:=<BrushTransition Duration=\"0:0:0.200\" />"}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab > Windows.UI.Xaml.Controls.Grid#ModalRootGrid > Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"Background=$t",
        L"CornerRadius=$wcr",
```

```
        L"BorderThickness=0",
        L"Shadow:="}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab > Windows.UI.Xaml.Controls.Grid#ModalRootGrid > Windows.UI.Xaml.Controls.Border#BackgroundElement > WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemList", {
        L"Background:=$mbg",
        L"CornerRadius=$wcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.DynamicFlowPanel > WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemListViewItem > Windows.UI.Xaml.Controls.Grid#Root@CommonStates > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"Background:=#09FFFFFF",
        L"CornerRadius=$mcr",
        L"BorderThickness=0.05,1,0.05,0",
        L"BorderBrush@Normal=$t",
        L"BorderBrush@PointerOver:=$AccentColor"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemControl > Grid#Root > WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemThumbnailButton#ThumbnailHost > Windows.UI.Xaml.Controls.Grid#RootGrid", {
        L"CornerRadius=$bcr",
        L"Margin=5"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#BackgroundDimmingLayer", {
        L"Background:=$t"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopBarElement > Windows.UI.Xaml.Controls.Grid#GridElement > Windows.UI.Xaml.Controls.Border#VirtualDesktopSwitcherBackground", {
        L"CornerRadius=$wcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb",
        L"Margin=-2,1,-1,2"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.DynamicFlowPanel#DFCPanel > WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemListViewItem > Windows.UI.Xaml.Controls.Grid#Root@CommonStates > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"Background:=$mbg",
        L"CornerRadius=$wcr,$wcr,$bcr,$bcr",
        L"Margin:=-5,0,-5,-5",
        L"BorderThickness=0.05,1,0.05,0",
        L"BorderBrush@Normal=$t",
        L"BorderBrush@PointerOver:=$AccentColor"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemElement > Windows.UI.Xaml.Controls.Grid#RootGrid > Windows.UI.Xaml.Controls.Grid#TitleGrid > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"Background:=$t"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemElement > Windows.UI.Xaml.Controls.Grid#RootGrid > Windows.UI.Xaml.Controls.Grid#TitleGrid > Image#IconImage", {
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"1\" />"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemElement > Windows.UI.Xaml.Controls.Grid#RootGrid > Windows.UI.Xaml.Controls.Grid#TitleGrid > TextBlock#DisplayName", {
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"1\" />"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#SwitchItemElementCloseButton > ContentPresenter#ContentPresenter", {
        L"CornerRadius=$mcr",
        L"Margin=5"}},
```

```
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#SwitchItemElementCloseButton > ContentPresenter#ContentPresenter > TextBlock", {
        L"RenderTransform:=<TranslateTransform X=\"-0.8\" Y=\"-0.6\" />"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemElement > Grid#RootGrid > WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemThumbnailButton#ThumbnailHost > Grid#RootGrid", {
        L"CornerRadius=0"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopBarElement#VirtualDesktopBar > Grid > Border", {
        L"Background:=$mbg",
        L"CornerRadius=$wcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb",
        L"Margin=-1,2,0,0"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopBarElement#VirtualDesktopBar", {
        L"Width=Auto",
        L"HorizontalAlignment=1"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopBarElement > Grid > Border", {
        L"Background:=$mbg",
        L"Shadow:="}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopElementThemed > Windows.UI.Xaml.Controls.Grid#MainGrid > Windows.UI.Xaml.Controls.Border#MainBorder", {
        L"CornerRadius=$mcr"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopElementThemed > Windows.UI.Xaml.Controls.Grid#MainGrid > Windows.UI.Xaml.Controls.Border#BorderHighlight", {
        L"CornerRadius=$mcr"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.NewVirtualDesktopElementThemed#NewVirtualDesktopButtonThemed > Windows.UI.Xaml.Controls.Grid#MainGrid", {
        L"CornerRadius=$mcr",
        L"BorderThickness=$bt",
        L"Margin=2"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.NewVirtualDesktopElementThemed#NewVirtualDesktopButtonThemed > Windows.UI.Xaml.Controls.Grid#MainGrid > Windows.UI.Xaml.Controls.Border#MainBorder", {
        L"CornerRadius=$mcr"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.NewVirtualDesktopElementThemed#NewVirtualDesktopButtonThemed > Windows.UI.Xaml.Controls.Grid#MainGrid > Windows.UI.Xaml.Controls.Border#BorderHighlight", {
        L"CornerRadius=$mcr"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopThumbnailButton#ThumbnailButtonElement", {
        L"CornerRadius=$bcr"}},

ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#VirtualDesktopElementCloseButton", {
        L"CornerRadius=$bcr"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapBarBorder", {
        L"Background:=$mbg",
        L"CornerRadius=$mcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb",
        L"Shadow:="}},
```

```
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapPickerBorder", {
      L"Background:=$mbg",
      L"CornerRadius=$mcr",
      L"BorderThickness=$bt",
      L"BorderBrush=$bb"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.FlyoutPresenter > Border", {
      L"Shadow:="}},
   ThemeTargetStyles{L"MenuFlyoutPresenter", {
      L"CornerRadius=$mcr",
      L"Shadow:="}},
   ThemeTargetStyles{L"MenuFlyoutPresenter > Border", {
      L"Background:=$mbg",
      L"CornerRadius=$mcr",
      L"BorderThickness=$bt",
      L"BorderBrush=$bb"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.MenuFlyoutItem", {
      L"CornerRadius=$bcr"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.MenuFlyoutSubItem", {
      L"CornerRadius=$bcr"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.ToolTip >
Windows.UI.Xaml.Controls.ContentPresenter#LayoutRoot", {
      L"Background:=$mbg",
      L"CornerRadius=$mcr",
      L"BorderThickness=$bt",
      L"BorderBrush=$bb",
      L"Shadow:="}},
   ThemeTargetStyles{L"ScrollViewer#MenuFlyoutPresenterScrollViewer > Border > Grid >
ScrollContentPresenter > ItemsPresenter > StackPanel", {
      L"ChildrenTransitions:=<TransitionCollection><EntranceThemeTransition
$AnimationSettings /></TransitionCollection>"}},
   ThemeTargetStyles{L"Grid#LayoutRoot", {
      L"BackgroundTransition:=<BrushTransition Duration=\"0:0:0.100\" />"}},
   ThemeTargetStyles{L"Border#BackgroundBorder", {
      L"BackgroundTransition:=<BrushTransition Duration=\"0:0:0.100\" />"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground >
Grid > Rectangle#BackgroundFill", {
      L"Visibility=Collapsed"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid@DockingStates", {
      L"Tag=horizontal",
      L"Tag@DockedLeft=vertical",
      L"Tag@DockedRight=vertical",
      L"Tag=>taskbarDock"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
      L"Width={{taskbarDock==`vertical`?skip():`Auto`}}",
      L"HorizontalAlignment=Stretch",
      L"Margin=$DockMargin,0,$DockMargin,0"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
      L"Background:=$mbg",
      L"BorderThickness=$bt",
      L"BorderBrush=$bb",
      L"CornerRadius=$mcr",
      L"Margin=0,$DockTopGap,0,$DockBottomGap"}},
   ThemeTargetStyles{L"Taskbar.TaskbarBackground#BackgroundControl >
Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Shapes.Rectangle#BackgroundStroke", {
      L"Visibility=Collapsed"}},
   ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Windows.UI.Xaml.Controls.Grid >
Windows.UI.Xaml.Controls.Grid#AugmentedEntryPointContentGrid,
```

```
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Windows.UI.Xaml.Controls.Grid#AugmentedEntryPointContentGrid", {
      L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"-1\" />"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#LargeTicker1", {
      L"Margin=0,0,0,-2"}},
   ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton", {
      L"Margin=0,0,$WidgetGap57,0"}},
   ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
      L"VerticalAlignment=Center",
      L"Margin=0,$DockTopGap,$DockMargin,$DockBottomGap"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
      L"Height={{taskbarDock==`vertical`?skip():$DockHeight}}"}},
}, {
   L"DockMargin=250",
   L"DockHeight=58",
   L"DockTopGap=5",
   L"DockBottomGap=5",
   L"WidgetGap=-",
   L"AccentColor=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight2}\"
Opacity=\"1.0\" />",
   L"AnimationSettings=IsStaggeringEnabled=\"True\" FromHorizontalOffset=\"-50\"
FromVerticalOffset=\"50\"",
   L"mbg=<WindhawkBlur BlurAmount=\"30\" TintColor=\"{ThemeResource
CardStrokeColorDefaultSolid}\" TintOpacity=\"0.0\" TintLuminosityOpacity=\"1.0\"
TintSaturation=\"1.0\" NoiseDensity=\"1.0\" NoiseOpacity=\"0.1\" />",
   L"bcr=10",
   L"wcr=20",
   L"mcr=15",
   L"t=Transparent",
   L"bb=#20FFFFFF",
   L"bt=1",
   L"nbb=<LinearGradientBrush x:Key=\"ShellTaskbarItemGradientStrokeColorSecondaryBrush\"
MappingMode=\"Absolute\" StartPoint=\"0,0\"
EndPoint=\"0,3\"><LinearGradientBrush.GradientStops><GradientStop Offset=\"0.33\"
Color=\"#1AFFFFFF\" /><GradientStop Offset=\"1\" Color=\"#0FFFFFFF\"
/></LinearGradientBrush.GradientStops></LinearGradientBrush>",
   L"nbt=<SolidColorBrush Color=\"{ThemeResource ControlFillColorDefault}\" />",
   L"nbth=<SolidColorBrush Color=\"{ThemeResource ControlFillColorSecondary}\" />",
   L"nbtp=<SolidColorBrush Color=\"{ThemeResource ControlFillColorTertiary}\" />",
}};

const Theme g_themeLuminosity_variant_Classic = {{
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground >
Grid > Rectangle#BackgroundFill", {
      L"Fill:=$mbg"}},
   ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundElement,
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Windows.UI.Xaml.Controls.Border#BackgroundElement", {
      L"CornerRadius=$bcr"}},
   ThemeTargetStyles{L"Taskbar.ExperienceToggleButton", {
      L"CornerRadius=$bcr"}},
   ThemeTargetStyles{L"Taskbar.TaskListButton", {
      L"CornerRadius=$bcr"}},

ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundElement,
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Windows.UI.Xaml.Controls.Border#BackgroundElement", {
      L"CornerRadius=$bcr"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SearchPillBackgroundElement", {
      L"CornerRadius=$bcr"}},
```

```
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#MultiWindowElement", {
      L"CornerRadius=$bcr"}},
   ThemeTargetStyles{L"SystemTray.ChevronIconView >
Windows.UI.Xaml.Controls.Grid#ContainerGrid >
Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
      L"CornerRadius=$bcr",
      L"Margin=0,4,2,4"}},
   ThemeTargetStyles{L"SystemTray.NotifyIconView >
Windows.UI.Xaml.Controls.Grid#ContainerGrid >
Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
      L"CornerRadius=$bcr",
      L"Margin=2,4,2,4"}},
   ThemeTargetStyles{L"SystemTray.IconView#SystemTrayIcon >
Windows.UI.Xaml.Controls.Grid#ContainerGrid >
Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
      L"CornerRadius=$bcr",
      L"Margin=2,4,2,4"}},
   ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton >
Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
      L"CornerRadius=$bcr",
      L"Margin=2,4,2,4"}},
   ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton >
Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
      L"CornerRadius=$bcr",
      L"Margin=2,4,2,4"}},
   ThemeTargetStyles{L"Border#OverflowFlyoutBackgroundBorder", {
      L"Background:=$mbg",
      L"CornerRadius:=$wcr",
      L"BorderThickness=$bt",
      L"BorderBrush=$bb",
      L"Shadow:="}},
   ThemeTargetStyles{L"Taskbar.OverflowToggleButton#OverflowButton >
Taskbar.TaskListButtonPanel#OverflowToggleButtonRootPanel >
Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundElement,
Taskbar.OverflowToggleButton#OverflowButton >
Taskbar.TaskListButtonPanel#OverflowToggleButtonRootPanel >
Windows.UI.Xaml.Controls.Border#BackgroundElement", {
      L"CornerRadius=$bcr"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#ConfirmatorMainGrid", {
      L"Background:=$mbg",
      L"CornerRadius=$wcr",
      L"BorderThickness=$bt",
      L"BorderBrush=$bb",
      L"Shadow:="}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalTrackRect", {
      L"Fill:=#10FFFFFF"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputS
witcher > ContentControl > ContentPresenter > Grid", {
      L"Background:=$mbg",
      L"CornerRadius=$wcr",
      L"BorderThickness=$bt",
      L"BorderBrush=$bb",
      L"Shadow:="}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputS
witcher > ContentControl > ContentPresenter > Grid > Grid", {
      L"Background:=$t"}},
   ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid >
Rectangle#BackgroundFill", {
      L"Fill:=$t"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid >
Windows.UI.Xaml.Controls.Border#HoverFlyoutBackground", {
```

```
        L"Background:=$mbg",
        L"CornerRadius=$mcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb",
        L"Shadow:="}},
    ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView > Grid@CommonStates >
Border#BackgroundBorder", {
        L"Background=$t",
        L"CornerRadius=$mcr",
        L"BorderThickness@Normal=0",
        L"BorderThickness@PointerOver=0.05,0,0.05,1",
        L"BorderBrush@Normal=$t",
        L"BorderBrush@PointerOver:=$AccentColor"}},
    ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView > Grid > Button#CloseButton", {
        L"CornerRadius=$mcr"}},
    ThemeTargetStyles{L"Taskbar.ThumbBarButton#ThumbBarButton >
Windows.UI.Xaml.Controls.ContentPresenter#BorderElement@CommonStates", {
        L"CornerRadius=16",
        L"Margin=-1.5",
        L"Background@Disabled:=$t",
        L"Background@Normal:=$t",
        L"Background@PointerOver:=$nbth",
        L"Background@Pressed:=$nbtp",
        L"BorderThickness=2",
        L"BorderBrush@Disabled:=$t",
        L"BorderBrush@Normal:=$t",
        L"BorderBrush@PointerOver:=$nbb",
        L"BorderBrush@Pressed:=$nbb",
        L"BackgroundSizing=InnerBorderEdge",
        L"BackgroundTransition:=<BrushTransition Duration=\"0:0:0.200\" />"}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab >
Windows.UI.Xaml.Controls.Grid#ModalRootGrid >
Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"Background=$t",
        L"CornerRadius=$wcr",
        L"BorderThickness=0",
        L"Shadow:="}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab >
Windows.UI.Xaml.Controls.Grid#ModalRootGrid >
Windows.UI.Xaml.Controls.Border#BackgroundElement >
WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemList", {
        L"Background:=$mbg",
        L"CornerRadius=$wcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.DynamicFlowPan
el > WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemListViewItem >
Windows.UI.Xaml.Controls.Grid#Root@CommonStates >
Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"Background:=#09FFFFFF",
        L"CornerRadius=$mcr",
        L"BorderThickness=0.05,1,0.05,0",
        L"BorderBrush@Normal=$t",
        L"BorderBrush@PointerOver:=$AccentColor"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemContr
ol > Grid#Root >
WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemThumbnailButton#Thumbn
ailHost > Windows.UI.Xaml.Controls.Grid#RootGrid", {
        L"CornerRadius=$bcr",
```

```
        L"Margin=5"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#BackgroundDimmingLayer", {
        L"Background:=$t"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopBarElement > Windows.UI.Xaml.Controls.Grid#GridElement > Windows.UI.Xaml.Controls.Border#VirtualDesktopSwitcherBackground", {
        L"CornerRadius=$wcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb",
        L"Margin=-2,1,-1,2"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.DynamicFlowPanel#DFCPanel > WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemListViewItem > Windows.UI.Xaml.Controls.Grid#Root@CommonStates > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"Background:=$mbg",
        L"CornerRadius=$wcr,$wcr,$bcr,$bcr",
        L"Margin:=-5,0,-5,-5",
        L"BorderThickness=0.05,1,0.05,0",
        L"BorderBrush@Normal=$t",
        L"BorderBrush@PointerOver:=$AccentColor"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemElement > Windows.UI.Xaml.Controls.Grid#RootGrid > Windows.UI.Xaml.Controls.Grid#TitleGrid > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"Background:=$t"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemElement > Windows.UI.Xaml.Controls.Grid#RootGrid > Windows.UI.Xaml.Controls.Grid#TitleGrid > Image#IconImage", {
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"1\" />"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemElement > Windows.UI.Xaml.Controls.Grid#RootGrid > Windows.UI.Xaml.Controls.Grid#TitleGrid > TextBlock#DisplayName", {
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"1\" />"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#SwitchItemElementCloseButton > ContentPresenter#ContentPresenter", {
        L"CornerRadius=$mcr",
        L"Margin=5"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#SwitchItemElementCloseButton > ContentPresenter#ContentPresenter > TextBlock", {
        L"RenderTransform:=<TranslateTransform X=\"-0.8\" Y=\"-0.6\" />"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemElement > Grid#RootGrid > WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemThumbnailButton#ThumbnailHost > Grid#RootGrid", {
        L"CornerRadius=0"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopBarElement#VirtualDesktopBar > Grid > Border", {
        L"Background:=$mbg",
        L"CornerRadius=$wcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb",
        L"Margin=-1,2,0,0"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopBarElement#VirtualDesktopBar", {
        L"Width=Auto",
        L"HorizontalAlignment=1"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopBarElement > Grid > Border", {
        L"Background:=$mbg",
```

```
        L"Shadow:="}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopElementThemed > Windows.UI.Xaml.Controls.Grid#MainGrid > Windows.UI.Xaml.Controls.Border#MainBorder", {
        L"CornerRadius=$mcr"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopElementThemed > Windows.UI.Xaml.Controls.Grid#MainGrid > Windows.UI.Xaml.Controls.Border#BorderHighlight", {
        L"CornerRadius=$mcr"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.NewVirtualDesktopElementThemed#NewVirtualDesktopButtonThemed > Windows.UI.Xaml.Controls.Grid#MainGrid", {
        L"CornerRadius=$mcr",
        L"BorderThickness=$bt",
        L"Margin=2"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.NewVirtualDesktopElementThemed#NewVirtualDesktopButtonThemed > Windows.UI.Xaml.Controls.Grid#MainGrid > Windows.UI.Xaml.Controls.Border#MainBorder", {
        L"CornerRadius=$mcr"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.NewVirtualDesktopElementThemed#NewVirtualDesktopButtonThemed > Windows.UI.Xaml.Controls.Grid#MainGrid > Windows.UI.Xaml.Controls.Border#BorderHighlight", {
        L"CornerRadius=$mcr"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopThumbnailButton#ThumbnailButtonElement", {
        L"CornerRadius=$bcr"}},

ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#VirtualDesktopElementCloseButton", {
        L"CornerRadius=$bcr"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapBarBorder", {
        L"Background:=$mbg",
        L"CornerRadius=$mcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb",
        L"Shadow:="}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapPickerBorder", {
        L"Background:=$mbg",
        L"CornerRadius=$mcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.FlyoutPresenter > Border", {
        L"Shadow:="}},
    ThemeTargetStyles{L"MenuFlyoutPresenter", {
        L"CornerRadius=$mcr",
        L"Shadow:="}},
    ThemeTargetStyles{L"MenuFlyoutPresenter > Border", {
        L"Background:=$mbg",
        L"CornerRadius=$mcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.MenuFlyoutItem", {
        L"CornerRadius=$bcr"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.MenuFlyoutSubItem", {
        L"CornerRadius=$bcr"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.ToolTip > Windows.UI.Xaml.Controls.ContentPresenter#LayoutRoot", {
        L"Background:=$mbg",
```

```
        L"CornerRadius=$mcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb",
        L"Shadow:="}},
    ThemeTargetStyles{L"ScrollViewer#MenuFlyoutPresenterScrollViewer > Border > Grid > ScrollContentPresenter > ItemsPresenter > StackPanel", {
        L"ChildrenTransitions:=<TransitionCollection><EntranceThemeTransition $AnimationSettings /></TransitionCollection>"}},
    ThemeTargetStyles{L"Grid#LayoutRoot", {
        L"BackgroundTransition:=<BrushTransition Duration=\"0:0:0.100\" />"}},
    ThemeTargetStyles{L"Border#BackgroundBorder", {
        L"BackgroundTransition:=<BrushTransition Duration=\"0:0:0.100\" />"}},
}, {
    L"AccentColor=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight2}\" Opacity=\"1.0\" />",
    L"AnimationSettings=IsStaggeringEnabled=\"True\" FromHorizontalOffset=\"-50\" FromVerticalOffset=\"50\"",
    L"mbg=<WindhawkBlur BlurAmount=\"30\" TintColor=\"{ThemeResource CardStrokeColorDefaultSolid}\" TintOpacity=\"0.0\" TintLuminosityOpacity=\"1.0\" TintSaturation=\"1.0\" NoiseDensity=\"1.0\" NoiseOpacity=\"0.1\" />",
    L"bcr=10",
    L"wcr=20",
    L"mcr=15",
    L"t=Transparent",
    L"bb=#20FFFFFF",
    L"bt=1",
    L"nbb=<LinearGradientBrush x:Key=\"ShellTaskbarItemGradientStrokeColorSecondaryBrush\" MappingMode=\"Absolute\" StartPoint=\"0,0\" EndPoint=\"0,3\"><LinearGradientBrush.GradientStops><GradientStop Offset=\"0.33\" Color=\"#1AFFFFFF\" /><GradientStop Offset=\"1\" Color=\"#0FFFFFFF\" /></LinearGradientBrush.GradientStops></LinearGradientBrush>",
    L"nbt=<SolidColorBrush Color=\"{ThemeResource ControlFillColorDefault}\" />",
    L"nbth=<SolidColorBrush Color=\"{ThemeResource ControlFillColorSecondary}\" />",
    L"nbtp=<SolidColorBrush Color=\"{ThemeResource ControlFillColorTertiary}\" />",
}};

const Theme g_themeLuminosity_variant_Compact = {{
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground > Grid > Rectangle#BackgroundFill", {
        L"Fill:=$mbg"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton", {
        L"Margin=0,0,$WidgetGap57,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundElement, Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"CornerRadius=$bcr"}},
    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton", {
        L"CornerRadius=$bcr"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton", {
        L"CornerRadius=$bcr"}},

    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundElement, SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"CornerRadius=$bcr"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SearchPillBackgroundElement", {
        L"CornerRadius=$bcr"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#MultiWindowElement", {
        L"CornerRadius=$bcr"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView > Windows.UI.Xaml.Controls.Grid#ContainerGrid > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
```

```
        L"CornerRadius=$bcr",
        L"Margin=0,4,2,4"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView > Windows.UI.Xaml.Controls.Grid#ContainerGrid > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"CornerRadius=$bcr",
        L"Margin=2,4,2,4"}},
    ThemeTargetStyles{L"SystemTray.IconView#SystemTrayIcon > Windows.UI.Xaml.Controls.Grid#ContainerGrid > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"CornerRadius=$bcr",
        L"Margin=2,4,2,4"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"CornerRadius=$bcr",
        L"Margin=2,4,2,4"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"CornerRadius=$bcr",
        L"Margin=2,4,2,4"}},
    ThemeTargetStyles{L"Border#OverflowFlyoutBackgroundBorder", {
        L"Background:=$mbg",
        L"CornerRadius:=$wcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb",
        L"Shadow:="}},
    ThemeTargetStyles{L"Taskbar.OverflowToggleButton#OverflowButton > Taskbar.TaskListButtonPanel#OverflowToggleButtonRootPanel > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundElement, Taskbar.OverflowToggleButton#OverflowButton > Taskbar.TaskListButtonPanel#OverflowToggleButtonRootPanel > Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"CornerRadius=$bcr"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#ConfirmatorMainGrid", {
        L"Background:=$mbg",
        L"CornerRadius=$wcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb",
        L"Shadow:="}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalTrackRect", {
        L"Fill:=#10FFFFFF"}},

    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputSwitcher > ContentControl > ContentPresenter > Grid", {
        L"Background:=$mbg",
        L"CornerRadius=$wcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb",
        L"Shadow:="}},

    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputSwitcher > ContentControl > ContentPresenter > Grid > Grid", {
        L"Background:=$t"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid > Rectangle#BackgroundFill", {
        L"Fill:=$t"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid > Windows.UI.Xaml.Controls.Border#HoverFlyoutBackground", {
        L"Background:=$mbg",
        L"CornerRadius=$mcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb",
```

```
        L"Shadow:="}},
    ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView > Grid@CommonStates > Border#BackgroundBorder", {
        L"Background=$t",
        L"CornerRadius=$mcr",
        L"BorderThickness@Normal=0",
        L"BorderThickness@PointerOver=0.05,0,0.05,1",
        L"BorderBrush@Normal=$t",
        L"BorderBrush@PointerOver:=$AccentColor"}},
    ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView > Grid > Button#CloseButton", {
        L"CornerRadius=$mcr"}},
    ThemeTargetStyles{L"Taskbar.ThumbBarButton#ThumbBarButton > Windows.UI.Xaml.Controls.ContentPresenter#BorderElement@CommonStates", {
        L"CornerRadius=16",
        L"Margin=-1.5",
        L"Background@Disabled:=$t",
        L"Background@Normal:=$t",
        L"Background@PointerOver:=$nbth",
        L"Background@Pressed:=$nbtp",
        L"BorderThickness=2",
        L"BorderBrush@Disabled:=$t",
        L"BorderBrush@Normal:=$t",
        L"BorderBrush@PointerOver:=$nbb",
        L"BorderBrush@Pressed:=$nbb",
        L"BackgroundSizing=InnerBorderEdge",
        L"BackgroundTransition:=<BrushTransition Duration=\"0:0:0.200\" />"}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab > Windows.UI.Xaml.Controls.Grid#ModalRootGrid > Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"Background=$t",
        L"CornerRadius=$wcr",
        L"BorderThickness=0",
        L"Shadow:="}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab > Windows.UI.Xaml.Controls.Grid#ModalRootGrid > Windows.UI.Xaml.Controls.Border#BackgroundElement > WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemList", {
        L"Background:=$mbg",
        L"CornerRadius=$wcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.DynamicFlowPanel > WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemListViewItem > Windows.UI.Xaml.Controls.Grid#Root@CommonStates > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"Background:=#09FFFFFF",
        L"CornerRadius=$mcr",
        L"BorderThickness=0.05,1,0.05,0",
        L"BorderBrush@Normal=$t",
        L"BorderBrush@PointerOver:=$AccentColor"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemControl > Grid#Root > WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemThumbnailButton#ThumbnailHost > Windows.UI.Xaml.Controls.Grid#RootGrid", {
        L"CornerRadius=$bcr",
        L"Margin=5"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#BackgroundDimmingLayer", {
        L"Background:=$t"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopB
```

```
arElement > Windows.UI.Xaml.Controls.Grid#GridElement >
Windows.UI.Xaml.Controls.Border#VirtualDesktopSwitcherBackground", {
    L"CornerRadius=$wcr",
    L"BorderThickness=$bt",
    L"BorderBrush=$bb",
    L"Margin=-2,1,-1,2"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.DynamicFlowPan
el#DFCPanel >
WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemListViewItem >
Windows.UI.Xaml.Controls.Grid#Root@CommonStates >
Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
    L"Background:=$mbg",
    L"CornerRadius=$wcr,$wcr,$bcr,$bcr",
    L"Margin:=-5,0,-5,-5",
    L"BorderThickness=0.05,1,0.05,0",
    L"BorderBrush@Normal=$t",
    L"BorderBrush@PointerOver:=$AccentColor"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemElem
ent > Windows.UI.Xaml.Controls.Grid#RootGrid > Windows.UI.Xaml.Controls.Grid#TitleGrid >
Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
    L"Background:=$t"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemElem
ent > Windows.UI.Xaml.Controls.Grid#RootGrid > Windows.UI.Xaml.Controls.Grid#TitleGrid >
Image#IconImage", {
    L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"1\" />"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemElem
ent > Windows.UI.Xaml.Controls.Grid#RootGrid > Windows.UI.Xaml.Controls.Grid#TitleGrid >
TextBlock#DisplayName", {
    L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"1\" />"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#SwitchItemElementCloseButton >
ContentPresenter#ContentPresenter", {
    L"CornerRadius=$mcr",
    L"Margin=5"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#SwitchItemElementCloseButton >
ContentPresenter#ContentPresenter > TextBlock", {
    L"RenderTransform:=<TranslateTransform X=\"-0.8\" Y=\"-0.6\" />"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemElem
ent > Grid#RootGrid >
WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemThumbnailButton#Thumbn
ailHost > Grid#RootGrid", {
    L"CornerRadius=0"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopB
arElement#VirtualDesktopBar > Grid > Border", {
    L"Background:=$mbg",
    L"CornerRadius=$wcr",
    L"BorderThickness=$bt",
    L"BorderBrush=$bb",
    L"Margin=-1,2,0,0"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopB
arElement#VirtualDesktopBar", {
    L"Width=Auto",
    L"HorizontalAlignment=1"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopB
arElement > Grid > Border", {
    L"Background:=$mbg",
    L"Shadow:="}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopEl
ementThemed > Windows.UI.Xaml.Controls.Grid#MainGrid >
Windows.UI.Xaml.Controls.Border#MainBorder", {
```

```
        L"CornerRadius=$mcr"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopElementThemed > Windows.UI.Xaml.Controls.Grid#MainGrid > Windows.UI.Xaml.Controls.Border#BorderHighlight", {
        L"CornerRadius=$mcr"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.NewVirtualDesktopElementThemed#NewVirtualDesktopButtonThemed > Windows.UI.Xaml.Controls.Grid#MainGrid", {
        L"CornerRadius=$mcr",
        L"BorderThickness=$bt",
        L"Margin=2"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.NewVirtualDesktopElementThemed#NewVirtualDesktopButtonThemed > Windows.UI.Xaml.Controls.Grid#MainGrid > Windows.UI.Xaml.Controls.Border#MainBorder", {
        L"CornerRadius=$mcr"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.NewVirtualDesktopElementThemed#NewVirtualDesktopButtonThemed > Windows.UI.Xaml.Controls.Grid#MainGrid > Windows.UI.Xaml.Controls.Border#BorderHighlight", {
        L"CornerRadius=$mcr"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopThumbnailButton#ThumbnailButtonElement", {
        L"CornerRadius=$bcr"}},

ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#VirtualDesktopElementCloseButton", {
        L"CornerRadius=$bcr"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapBarBorder", {
        L"Background:=$mbg",
        L"CornerRadius=$mcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb",
        L"Shadow:="}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapPickerBorder", {
        L"Background:=$mbg",
        L"CornerRadius=$mcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.FlyoutPresenter > Border", {
        L"Shadow:="}},
    ThemeTargetStyles{L"MenuFlyoutPresenter", {
        L"CornerRadius=$mcr",
        L"Shadow:="}},
    ThemeTargetStyles{L"MenuFlyoutPresenter > Border", {
        L"Background:=$mbg",
        L"CornerRadius=$mcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.MenuFlyoutItem", {
        L"CornerRadius=$bcr"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.MenuFlyoutSubItem", {
        L"CornerRadius=$bcr"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.ToolTip > Windows.UI.Xaml.Controls.ContentPresenter#LayoutRoot", {
        L"Background:=$mbg",
        L"CornerRadius=$mcr",
        L"BorderThickness=$bt",
        L"BorderBrush=$bb",
```

```
        L"Shadow:="}},
    ThemeTargetStyles{L"ScrollViewer#MenuFlyoutPresenterScrollViewer > Border > Grid > ScrollContentPresenter > ItemsPresenter > StackPanel", {
        L"ChildrenTransitions:=<TransitionCollection><EntranceThemeTransition $AnimationSettings /></TransitionCollection>"}},
    ThemeTargetStyles{L"Grid#LayoutRoot", {
        L"BackgroundTransition:=<BrushTransition Duration=\"0:0:0.100\" />"}},
    ThemeTargetStyles{L"Border#BackgroundBorder", {
        L"BackgroundTransition:=<BrushTransition Duration=\"0:0:0.100\" />"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
        L"Height=30"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Grid#AugmentedEntryPointContentGrid, Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Windows.UI.Xaml.Controls.Grid#AugmentedEntryPointContentGrid", {
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"-1\" />"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#LargeTicker1", {
        L"Margin=1,-5,0,0",
        L"RenderTransform:=<ScaleTransform ScaleX=\"0.75\" ScaleY=\"0.75\" />"}},

    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundElement, SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"Margin=0,4,0,4"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl", {
        L"Margin=0,-4,0,-4"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon", {
        L"Width=16",
        L"Height=16"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Image#Icon", {
        L"Width=16",
        L"Height=16"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton > Grid#IconPanel > Windows.UI.Xaml.Controls.TextBlock#LabelControl, Taskbar.TaskListButton#TaskListButton > Taskbar.TaskListLabeledButtonPanel#IconPanel > Windows.UI.Xaml.Controls.TextBlock#LabelControl", {
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"-1\" />"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton", {
        L"Margin=0,0,$WidgetGap57,0"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
        L"Height=30"}},
    ThemeTargetStyles{L"SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock", {
        L"FontSize=14"}},
    ThemeTargetStyles{L"SystemTray.ImageIconContent > Grid#ContainerGrid > Image", {
        L"Width=14",
        L"Height=14"}},
}, {
    L"WidgetGap=-",
    L"AccentColor=<SolidColorBrush Color=\"{ThemeResource SystemAccentColorLight2}\" Opacity=\"1.0\" />",
    L"AnimationSettings=IsStaggeringEnabled=\"True\" FromHorizontalOffset=\"-50\" FromVerticalOffset=\"50\"",
    L"mbg=<WindhawkBlur BlurAmount=\"30\" TintColor=\"{ThemeResource CardStrokeColorDefaultSolid}\" TintOpacity=\"0.0\" TintLuminosityOpacity=\"1.0\" TintSaturation=\"1.0\" NoiseDensity=\"1.0\" NoiseOpacity=\"0.1\" />",
    L"bcr=10",
    L"wcr=20",
    L"mcr=15",
    L"t=Transparent",
    L"bb=#20FFFFFF",
```

```
    L"bt=1",
    L"nbb=<LinearGradientBrush x:Key=\"ShellTaskbarItemGradientStrokeColorSecondaryBrush\"
MappingMode=\"Absolute\" StartPoint=\"0,0\"
EndPoint=\"0,3\"><LinearGradientBrush.GradientStops><GradientStop Offset=\"0.33\"
Color=\"#1AFFFFFF\" /><GradientStop Offset=\"1\" Color=\"#0FFFFFFF\"
/></LinearGradientBrush.GradientStops></LinearGradientBrush>",
    L"nbt=<SolidColorBrush Color=\"{ThemeResource ControlFillColorDefault}\" />",
    L"nbth=<SolidColorBrush Color=\"{ThemeResource ControlFillColorSecondary}\" />",
    L"nbtp=<SolidColorBrush Color=\"{ThemeResource ControlFillColorTertiary}\" />",
}};

const Theme g_themeLayerMicaUI = {{
    ThemeTargetStyles{L"Border#BackgroundDimmingLayer", {
        L"Background:=$ThemeBlur"}},
    ThemeTargetStyles{L"Border#OverflowFlyoutBackgroundBorder", {
        L"Background:=$ThemeLayer",
        L"CornerRadius=$OuterRadius",
        L"BorderBrush:=$ThemeOutBorder",
        L"BorderThickness=1"}},
    ThemeTargetStyles{L"Border#VirtualDesktopBarBackground", {
        L"Background:=$ThemeLayer",
        L"CornerRadius=$OuterRadius",
        L"BorderBrush:=$ThemeOutBorder"}},
    ThemeTargetStyles{L"Border#SnapPickerBorder", {
        L"Background:=$ThemeBlur"}},
    ThemeTargetStyles{L"Border#SnapBarBorder", {
        L"Background:=$ThemeBlur"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"Background:=$ThemeLayer",
        L"CornerRadius=$OuterRadius",
        L"Margin=0,3,8,3",
        L"Padding=4,0,0,0",
        L"BorderThickness=1"}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab >
Grid#ModalRootGrid > Border#BackgroundElement", {
        L"Background=Transparent",
        L"CornerRadius=$OuterRadius",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab >
Grid#ModalRootGrid > Border#BackgroundElement >
WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemList", {
        L"Background:=$ThemeBlur",
        L"CornerRadius=$OuterRadius",
        L"BorderThickness=1",
        L"BorderBrush:=$ThemeOutBorder"}},
    ThemeTargetStyles{L"Grid#ConfirmatorMainGrid", {
        L"Background:=$ThemeLayer",
        L"CornerRadius=$OuterRadius",
        L"BorderThickness=1",
        L"BorderBrush:=$ThemeOutBorder",
        L"Height=46"}},
    ThemeTargetStyles{L"Grid#HorizontalTemplate > Rectangle#HorizontalDecreaseRect", {
        L"Height=7.5",
        L"RadiusX=3",
        L"RadiusY=3"}},
    ThemeTargetStyles{L"Grid#HorizontalTemplate > Rectangle#HorizontalTrackRect", {
        L"Height=7.5",
        L"RadiusX=3",
```

```
        L"RadiusY=3",
        L"Opacity=0.5"}},
    ThemeTargetStyles{L"Grid#VolumeConfirmator > Slider#volumeSlider", {
        L"Margin=0,-3.5,0,0",
        L"Width=120"}},
    ThemeTargetStyles{L"Grid#BrightnessConfirmator > Slider#brightnessSlider", {
        L"Width=135",
        L"Margin=0,-2.5,0,0"}},
    ThemeTargetStyles{L"Grid#DynamicSearchBoxGleamContainer", {
        L"Height=30"}},
    ThemeTargetStyles{L"Grid > HyperlinkButton#Footer", {
        L"HorizontalContentAlignment=1"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.AnimatedIcon#BrightnessIcon", {
        L"Height=22",
        L"Width=22"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.AnimatedIcon#VolumeIcon", {
        L"Height=18",
        L"Width=18"}},

ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon[AutomationProperties.Name=Bluetooth Devices] > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.ImageIconContent > Grid#ContainerGrid > Image", {
        L"Visibility=1"}},

ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon[AutomationProperties.Name=Safely Remove Hardware and Eject Media] > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.ImageIconContent > Grid#ContainerGrid > Image", {
        L"Visibility=1"}},

ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon[AutomationProperties.Name=Bluetooth Devices] > Grid#ContainerGrid > ContentPresenter#ContentPresenter", {
        L"Content:=<FontIcon FontFamily=\"Segoe Fluent Icons\" Glyph=\"&#xE702;\" FontSize=\"16\"/>",
        L"Foreground:=<SolidColorBrush Color=\"{ThemeResource Accent1}\" />",
        L"Canvas.ZIndex=-1"}},

ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon[AutomationProperties.Name=Safely Remove Hardware and Eject Media] > Grid#ContainerGrid > ContentPresenter#ContentPresenter", {
        L"Content:=<FontIcon FontFamily=\"Segoe Fluent Icons\" Glyph=\"&#xE88E;\" FontSize=\"16\"/>",
        L"Foreground:=<SolidColorBrush Color=\"{ThemeResource Accent1}\" />",
        L"Canvas.ZIndex=-1"}},
    ThemeTargetStyles{L"Grid#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton, StackPanel#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton", {
        L"CornerRadius=$InnerRadius"}},
    ThemeTargetStyles{L"Grid#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton", {
        L"Grid.Column=4"}},
    ThemeTargetStyles{L"Grid#SystemTrayFrameGrid > SystemTray.OmniButton#NotificationCenterButton, StackPanel#SystemTrayFrameGrid > SystemTray.OmniButton#NotificationCenterButton", {
        L"CornerRadius=$InnerRadius"}},
    ThemeTargetStyles{L"Grid#SystemTrayFrameGrid > SystemTray.OmniButton#NotificationCenterButton", {
        L"Grid.Column=5"}},
    ThemeTargetStyles{L"Grid#SystemTrayFrameGrid > SystemTray.Stack#MainStack, StackPanel#SystemTrayFrameGrid > SystemTray.Stack#MainStack", {
        L"CornerRadius=$InnerRadius"}},
    ThemeTargetStyles{L"Grid#SystemTrayFrameGrid > SystemTray.Stack#MainStack", {
        L"Grid.Column=6"}},
    ThemeTargetStyles{L"Grid#SystemTrayFrameGrid > SystemTray.Stack#ShowDesktopStack, StackPanel#SystemTrayFrameGrid > SystemTray.Stack#ShowDesktopStack", {
```

```
      L"CornerRadius=$InnerRadius",
      L"Width=5"}},
    ThemeTargetStyles{L"Grid#SystemTrayFrameGrid > SystemTray.Stack#ShowDesktopStack", {
      L"Grid.Column=7"}},
    ThemeTargetStyles{L"Grid#SystemTrayFrameGrid > SystemTray.Stack#NonActivatableStack", {
      L"Grid.Column=2"}},
ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.
AutomationId=TaskViewButton] >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
      L"CornerRadius=$InnerRadius",
      L"Width=44"}},
    ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView > Grid > Border", {
      L"CornerRadius=$OuterRadius",
      L"Background:=$ThemeLayer",
      L"BorderThickness=1",
      L"BorderBrush:=$ThemeOutBorder"}},
    ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView > Grid >
TextBlock#DisplayNameTextBlock", {
      L"FontFamily=$ThFnt",
      L"FontWeight=Bold"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton", {
      L"CornerRadius=$InnerRadius",
      L"Margin=0,-1,0,-1",
      L"BorderBrush:=$ThemeControlBorder",
      L"BorderThickness=1"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#BackgroundControl", {
      L"Opacity=0"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid >
Rectangle#BackgroundFill", {
      L"CornerRadius=$OuterRadius"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl", {
      L"FontFamily=$ThFnt"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid@DockingStates", {
      L"Tag=horizontal",
      L"Tag@DockedLeft=vertical",
      L"Tag@DockedRight=vertical",
      L"Tag=>taskbarDock"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
      L"Width={{taskbarDock==`vertical`?skip():`Auto`}}"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
      L"Margin=8,3,0,3",
      L"CornerRadius=$OuterRadius",
      L"Padding=-8,0,0,0",
      L"Background:=$ThemeLayer",
      L"BorderThickness=1",
      L"HorizontalAlignment=Left"}},
    ThemeTargetStyles{L"TextBlock#InnerTextBlock[Text=\uE971]", {
      L"Text=\uF0E2"}},
    ThemeTargetStyles{L"TextBlock#volumeLevelText", {
      L"FontSize=15",
      L"Margin=0,-1,0,0",
      L"FontFamily=$ThFnt",
      L"FontWeight=$ThFntWt"}},
    ThemeTargetStyles{L"TextBlock#BatteryTextBlock", {
      L"FontFamily=$ThFnt",
      L"FontSize=12",
```

```
        L"FontWeight=Bold"}},
    ThemeTargetStyles{L"ToolTip > ContentPresenter#LayoutRoot", {
        L"Background:=$ThemeFlyout",
        L"CornerRadius=$OuterRadius",
        L"BorderBrush:=$ThemeOutBorder",
        L"BorderThickness=1"}},
    ThemeTargetStyles{L"ToolTip > ContentPresenter#LayoutRoot > TextBlock", {
        L"FontFamily=$ThFnt",
        L"FontWeight=$ThFntWt",
        L"FontSize=13"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.NewVirtualDesktopElementThemed#NewVirtualDesktopButtonThemed > Grid#MainGrid > Border#MainBorder", {
        L"CornerRadius=$OuterRadius",
        L"Background:=$ThemeOverlay"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.NewVirtualDesktopElementThemed#NewVirtualDesktopButtonThemed > Grid#MainGrid > Border#BorderHighlight", {
        L"CornerRadius=$OuterRadius"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemElement > Grid#RootGrid > Grid#TitleGrid > Border#BackgroundBorder", {
        L"CornerRadius=$OuterRadius,$OuterRadius,0,0"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemElement > Grid#RootGrid > Grid#TitleGrid", {
        L"FontFamily=$ThFnt",
        L"FontWeight=$ThFntWt"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemListViewItem > Grid#Root > Border#BackgroundBorder", {
        L"CornerRadius=$OuterRadius",
        L"Background:=$ThemeLayer",
        L"BorderBrush:=$ThemeControlBorder",
        L"BorderThickness=1"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopElementThemed > Grid#MainGrid > Border#MainBorder", {
        L"CornerRadius=$OuterRadius",
        L"Background:=$ThemeOverlay"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopElementThemed > Grid#MainGrid > Border#BorderHighlight", {
        L"CornerRadius=$OuterRadius"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopElementThemed > Grid#MainGrid > Border#ActiveDesktopPill", {
        L"CornerRadius=$OuterRadius"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputSwitcher > ContentControl > ContentPresenter > Grid", {
        L"Background:=$ThemeLayer",
        L"BorderBrush:=$ThemeOutBorder",
        L"Shadow:=",
        L"CornerRadius=$OuterRadius"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputSwitcher > ContentControl > ContentPresenter > Grid > Grid#OverlayPanel", {
        L"Background=Transparent",
        L"BorderBrush=Transparent"}},
    ThemeTargetStyles{L"Button#GleamEntryPointButton > Border", {
        L"Height=30"}},
```

```
    ThemeTargetStyles{L"ContentPresenter > SearchUx.SearchUI.SearchButtonControl > Grid > SearchUx.SearchUI.SearchBoxButton#SearchBox > SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Grid > TextBlock#SearchBoxTextBlock, ContentPresenter > SearchUx.SearchUI.SearchButtonControl > Grid > SearchUx.SearchUI.SearchBoxButton#SearchBox > SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > TextBlock#SearchBoxTextBlock", {
        L"FontFamily=$ThFnt",
        L"FontWeight=$ThFntWt",
        L"FontSize=14",
        L"RenderTransform:=<TranslateTransform Y=\"1\" />"}},
    ThemeTargetStyles{L"MenuFlyoutPresenter", {
        L"Background:=$ThemeFlyout",
        L"CornerRadius=$OuterRadius",
        L"FontFamily=$ThFnt",
        L"FontWeight=$ThFntWt",
        L"BorderThickness=1",
        L"BorderBrush:=$ThemeOutBorder"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater", {
        L"Margin=0,0,2,0"}},
    ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
        L"Visibility=1"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > Grid", {
        L"Background:=<WindhawkBlur BlurAmount=\"5\" TintColor=\"{ThemeResource SystemChromeMediumColor}\" TintSaturation=\"2\" NoiseOpacity=\"0.19\" NoiseDensity=\"0.8\" TintOpacity=\"0.2\" TintLuminosityOpacity=\"0\" />"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchBoxButton#SearchBox", {
        L"Margin=0,-1,0,0"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid@CommonStates > Grid > Border#BackgroundElement, SearchUx.SearchUI.SearchButtonRootGrid@CommonStates > Border#BackgroundElement", {
        L"Background:=$ThemeIsland",
        L"BorderBrush:=$ThemeOutBorder",
        L"BorderThickness=1",
        L"CornerRadius=$InnerRadius",
        L"Margin=0,-2.5,0,-2.5"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchPillButton#SearchPill > SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Grid > Border#SearchPillBackgroundElement, SearchUx.SearchUI.SearchPillButton#SearchPill > SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Border#SearchPillBackgroundElement", {
        L"CornerRadius=$InnerRadius",
        L"Height=26",
        L"BorderBrush:=$ThemeBorder",
        L"Background:=$ThemeOverlay"}},
    ThemeTargetStyles{L"Grid#SystemTrayFrameGrid > SystemTray.NotificationAreaIcons#NotificationAreaIcons, StackPanel#SystemTrayFrameGrid > SystemTray.NotificationAreaIcons#NotificationAreaIcons", {
        L"CornerRadius=$InnerRadius"}},
    ThemeTargetStyles{L"Grid#SystemTrayFrameGrid > SystemTray.NotificationAreaIcons#NotificationAreaIcons", {
        L"Grid.Column=1"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
        L"HorizontalAlignment=Right"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Hosting.DesktopWindowXamlSource", {
        L"Background:=$ThemeBlur",
        L"CornerRadius=$OuterRadius",
        L"BorderBrush:=$ThemeOutBorder"}},

    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopBarElement > Grid#GridElement > Border#VirtualDesktopSwitcherBackground", {
        L"Background:=$ThemeLayer"}},
}, {
```

```
   L"ThemeBlur=<WindhawkBlur BlurAmount=\"12\" TintColor=\"{ThemeResource
SystemChromeMediumColor}\" TintSaturation=\"1.25\" NoiseOpacity=\"0.1\"
NoiseDensity=\"0.8\" TintOpacity=\"0.2\" TintLuminosityOpacity=\"0.3\" />",
   L"ThemeLayer=<AcrylicBrush BackgroundSource=\"Backdrop\" TintColor=\"{ThemeResource
SystemChromeMediumColor}\" TintOpacity=\"{ThemeResource bgOpacity}\"
TintLuminosityOpacity=\"{ThemeResource bgLuminosity}\" FallbackColor=\"{ThemeResource
SystemChromeMediumColor}\" />",
   L"OuterRadius=10",
   L"InnerRadius=8",
   L"ThemeBorder=<SolidColorBrush Color=\"{ThemeResource Border}\"/>",
   L"ThFnt=Nunito",
   L"ThemeOutBorder=<SolidColorBrush Color=\"#56757575\"/>",
   L"ThemeOverlay=<SolidColorBrush Color=\"{ThemeResource Overlay}\" />",
   L"ThemeIsland=<SolidColorBrush Color=\"{ThemeResource Island}\" />",
   L"ThemeControlBorder=<SolidColorBrush Color=\"{ThemeResource ControlBorder}\" />",
   L"ThFntWt=Normal",
   L"ThemeFlyout=<AcrylicBrush BackgroundSource=\"Backdrop\" TintColor=\"{ThemeResource
SystemChromeMediumColor}\" TintOpacity=\"0.1\" TintLuminosityOpacity=\"0.8\"
FallbackColor=\"{ThemeResource SystemChromeMediumColor}\" />",
}, {
   L"Accent1@Dark={ThemeResource SystemAccentColorLight3}",
   L"Accent1@Light={ThemeResource SystemAccentColorDark2}",
   L"bgOpacity@Dark=0.3",
   L"bgOpacity@Light=0.2",
   L"bgLuminosity@Light=1",
   L"bgLuminosity@Dark=0.9",
   L"Overlay@Light=#40FFFFFF",
   L"Overlay@Dark=#09FFFFFF",
   L"Border@Light=#0F000000",
   L"Border@Dark=#19000000",
   L"ControlBorder@Light=#0F000000",
   L"ControlBorder@Dark=#15FFFFFF",
   L"Island@Light=#5FFFFFFF",
   L"Island@Dark=#10FFFFFF",
}};

const Theme g_themeFluid = {{
   ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
      L"Visibility=1"}},
   ThemeTargetStyles{L"Grid#OverflowRootGrid > Border", {
      L"CornerRadius=$CornerRadius"}},
   ThemeTargetStyles{L"Rectangle#RunningIndicator", {
      L"Fill:=<AcrylicBrush TintColor=\"{ThemeResource SystemChromeAltMedColor}\"
TintOpacity=\"0.5\" />"}},
   ThemeTargetStyles{L"TextBlock#InnerTextBlock[Text=\uE971]", {
      L"Text=\uED14"}},
   ThemeTargetStyles{L"Border#BackgroundElement", {
      L"CornerRadius=$CornerRadius"}},
   ThemeTargetStyles{L"Border#BackgroundDimmingLayer", {
      L"CornerRadius=$CornerRadius"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopB
arElement#VirtualDesktopBar", {
      L"CornerRadius=$CornerRadius"}},
   ThemeTargetStyles{L"MenuFlyoutPresenter", {
      L"CornerRadius=$CornerRadius"}},
   ThemeTargetStyles{L"Grid#LayoutRoot", {
      L"BackgroundTransition:=<BrushTransition Duration=\"0:0:0.083\" />"}},
   ThemeTargetStyles{L"Border#BackgroundBorder", {
```

```
        L"BackgroundTransition:=<BrushTransition Duration=\"0:0:0.083\" />"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid > Border#BackgroundElement@CommonStates,
Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Border#BackgroundElement@CommonStates", {
        L"Background@ActiveNormal:=$NormalBG",
        L"Background@ActivePointerOver:=$Hover",
        L"Background@ActivePressed:=$PressedBG",
        L"Background@InactivePointerOver:=$Hover",
        L"Background@InactivePressed:=$PressedBG",
        L"BorderBrush@ActiveNormal:=$BorderBrush",
        L"BorderBrush@ActivePointerOver:=$BorderBrush",
        L"BorderBrush@ActivePressed:=$BorderBrush",
        L"BorderBrush@InactivePointerOver:=$BorderBrush",
        L"BorderBrush@InactivePressed:=$BorderBrush",
        L"BackgroundTransition:=<BrushTransition Duration=\"0:0:0.083\" />",
        L"BackgroundSizing=InnerBorderEdge",
        L"Margin=1",
        L"CornerRadius=$CornerRadius",
        L"BorderThickness=$BorderThickness"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border#BackgroundElement,
Taskbar.TaskListLabeledButtonPanel@CommonStates > Border#BackgroundElement", {
        L"Background@ActiveNormal:=$NormalBG",
        L"Background@ActivePointerOver:=$Hover",
        L"Background@ActivePressed:=$PressedBG",
        L"Background@InactivePointerOver:=$Hover",
        L"Background@InactivePressed:=$PressedBG",
        L"Background@MultiWindowNormal:=$NormalBG",
        L"Background@MultiWindowActive:=$NormalBG",
        L"Background@MultiWindowPointerOver:=$Hover",
        L"Background@MultiWindowPressed:=$PressedBG",
        L"BorderBrush@ActiveNormal:=$BorderBrush",
        L"BorderBrush@ActivePointerOver:=$BorderBrush",
        L"BorderBrush@ActivePressed:=$BorderBrush",
        L"BorderBrush@InactivePointerOver:=$BorderBrush",
        L"BorderBrush@InactivePressed:=$BorderBrush",
        L"BorderBrush@MultiWindowNormal:=$BorderBrush",
        L"BorderBrush@MultiWindowActive:=$BorderBrush",
        L"BorderBrush@MultiWindowPointerOver:=$BorderBrush",
        L"BorderBrush@MultiWindowPressed:=$BorderBrush",
        L"BackgroundTransition:=<BrushTransition Duration=\"0:0:0.083\" />",
        L"BackgroundSizing=InnerBorderEdge",
        L"Margin=2",
        L"BorderThickness=$BorderThickness"}},
    ThemeTargetStyles{L"Border#MultiWindowElement", {
        L"BorderBrush:=$BorderBrush",
        L"Background:=$NormalBG",
        L"BackgroundTransition:=<BrushTransition Duration=\"0:0:0.083\" />",
        L"BackgroundSizing=InnerBorderEdge",
        L"BorderThickness=$BorderThickness",
        L"CornerRadius=$CornerRadius",
        L"Margin=2"}},
    ThemeTargetStyles{L"ContentPresenter#ContentPresenter@CommonStates", {
        L"Background@ActiveNormal:=$NormalBG",
        L"Background@ActivePointerOver:=$Hover",
```

```
      L"Background@ActivePressed:=$PressedBG",
      L"Background@InactivePointerOver:=$Hover",
      L"Background@InactivePressed:=$PressedBG",
      L"BorderBrush@ActiveNormal:=$BorderBrush",
      L"BorderBrush@ActivePointerOver:=$BorderBrush",
      L"BorderBrush@ActivePressed:=$BorderBrush",
      L"BorderBrush@InactivePointerOver:=$BorderBrush",
      L"BorderBrush@InactivePressed:=$BorderBrush",
      L"BackgroundTransition:=<BrushTransition Duration=\"0:0:0.083\" />",
      L"BackgroundSizing=InnerBorderEdge",
      L"CornerRadius=$CornerRadius",
      L"BorderThickness=$BorderThickness",
      L"Margin=2"}},
   ThemeTargetStyles{L"ContentPresenter#ContentPresenter > Grid#ContentGrid >
Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#LottieIcon", {
      L"Visibility=1"}},
   ThemeTargetStyles{L"SystemTray.CopilotIcon#CopilotIcon > Grid#ContainerGrid >
Border#BackgroundBorder", {
      L"Visibility=1"}},
}, {
   L"BorderBrush=<LinearGradientBrush
x:Key=\"ShellTaskbarItemGradientStrokeColorSecondaryBrush\" MappingMode=\"Absolute\"
StartPoint=\"0,0\" EndPoint=\"0,3\"><LinearGradientBrush.GradientStops><GradientStop
Offset=\"0.33\" Color=\"{ThemeResource ControlFillColorSecondary}\" /><GradientStop
Offset=\"1\" Color=\"{ThemeResource ControlFillColorTertiary}\"
/></LinearGradientBrush.GradientStops></LinearGradientBrush>",
   L"NormalBG=<SolidColorBrush Color=\"{ThemeResource ControlFillColorDefault}\" />",
   L"HoverBG=<SolidColorBrush Color=\"{ThemeResource ControlFillColorSecondary}\" />",
   L"PressedBG=<SolidColorBrush Color=\"{ThemeResource ControlFillColorTertiary}\" />",
   L"BorderThickness=1",
   L"CornerRadius=4",
}};

const Theme g_themeTintedGlass = {{
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground >
Grid > Rectangle#BackgroundFill", {
      L"Fill:=$CommonBgBrush"}},
   ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid >
Rectangle#BackgroundFill", {
      L"Fill:=$CommonBgBrush"}},
   ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab >
Windows.UI.Xaml.Controls.Grid#ModalRootGrid >
Windows.UI.Xaml.Controls.Border#BackgroundElement", {
      L"Background=Transparent"}},
   ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab >
Windows.UI.Xaml.Controls.Grid#ModalRootGrid >
Windows.UI.Xaml.Controls.Border#BackgroundElement >
WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemList", {
      L"Background:=$CommonBgBrush"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#BackgroundDimmingLayer", {
      L"Background:=$CommonBgBrush"}},
   ThemeTargetStyles{L"MenuFlyoutPresenter > Border", {
      L"Fill:=$CommonBgBrush",
      L"BorderThickness=0,0,0,0",
      L"CornerRadius=14",
      L"Padding=2,2,2,2"}},
   ThemeTargetStyles{L"Border#OverflowFlyoutBackgroundBorder", {
      L"Fill:=$CommonBgBrush",
      L"BorderThickness=0,0,0,0",
      L"CornerRadius=14",
      L"Margin=-2,-2,-2,-2"}},
```

```
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock", {
        L"FontSize=18"}},
    ThemeTargetStyles{L"SystemTray.ImageIconContent > Grid#ContainerGrid > Image", {
        L"Width=18",
        L"Height=18"}},
    ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack", {
        L"Visibility=Collapsed"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.
AutomationId=StartButton] > Taskbar.TaskListButtonPanel > Grid >
Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon,
Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartB
utton] > Taskbar.TaskListButtonPanel > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon",
{
        L"Height=32",
        L"Width=32"}},
    ThemeTargetStyles{L"Grid#IconPanel, Taskbar.TaskListLabeledButtonPanel#IconPanel", {
        L"Padding=2,2,2,2"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel,
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel", {
        L"Padding=2,2,2,2"}},
    ThemeTargetStyles{L"Grid#ContainerGrid", {
        L"Padding=2,2,2,2"}},
    ThemeTargetStyles{L"Taskbar.FlyoutFrame > Canvas#HoverFlyoutCanvas >
Grid#HoverFlyoutGrid", {
        L"Padding=2,2,2,2"}},
    ThemeTargetStyles{L"Image#Icon", {
        L"Margin=2,2,2,2"}},
    ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
        L"Fill:=<WindhawkBlur BlurAmount=\"18\" TintColor=\"#1AFFFFFF\"/>"}},
    ThemeTargetStyles{L"Grid#OverflowRootGrid > Border", {
        L"Background:=$CommonBgBrush"}},
    ThemeTargetStyles{L"Grid#ConfirmatorMainGrid", {
        L"Background:=$CommonBgBrush",
        L"BorderThickness=0"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputS
witcher > ContentControl > ContentPresenter > Grid", {
        L"Background:=$CommonBgBrush",
        L"BorderThickness=0"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputS
witcher > ContentControl > ContentPresenter > Grid > Grid", {
        L"Fill:=<WindhawkBlur BlurAmount=\"18\" TintColor=\"#1AFFFFFF\"/>"}},
}, {
    L"CommonBgBrush=<WindhawkBlur BlurAmount=\"18\" TintColor=\"#80000000\"/>",
}};


const Theme g_themeTaskbarToStatusbar = {{
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid >
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater", {
        L"Width=Auto",
        L"HorizontalAlignment=Left"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.
AutomationId=TaskViewButton] >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
        L"Width=3840",
        L"Margin=0,0,-3840,0"}},
    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton >
Taskbar.TaskListButtonPanel > Grid > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon,
Taskbar.ExperienceToggleButton#LaunchListButton > Taskbar.TaskListButtonPanel >
Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon", {
```

```
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel@CommonStates > Grid > Border#BackgroundElement, Taskbar.TaskListButtonPanel@CommonStates > Border#BackgroundElement", {
        L"Visibility=Collapsed"}},

ThemeTargetStyles{L"Taskbar.SearchBoxButton#SearchBoxButton[AutomationProperties.AutomationId=SearchButton] > Taskbar.TaskListButtonPanel", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskbarExtensionElement", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Grid#IconPanel, Taskbar.TaskListLabeledButtonPanel#IconPanel", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#OverflowToggleButtonRootPanel", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
        L"HorizontalAlignment=Center"}},
    ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack", {
        L"Width=1920",
        L"Margin=0,0,-1920,0"}},

ThemeTargetStyles{L"SystemTray.StackListView#IconStack[AutomationProperties.AutomationId=ShowDesktop] > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid, SystemTray.StackListView#IconStack[AutomationProperties.AutomationId=ShowDesktop] > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon", {
        L"Width=1920"}},
    ThemeTargetStyles{L"SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid@ > Rectangle#ShowDesktopPipe", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid", {
        L"Padding=4,0,4,0"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid", {
        L"Padding=4,0,4,0"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock#LanguageInnerTextBlock > Windows.UI.Xaml.Controls.TextBlock#InnerTextBlock", {
        L"TextTrimming=1",
        L"Height=20",
        L"FontSize=14"}},
    ThemeTargetStyles{L"TextBlock#BatteryTextBlock", {
        L"FontSize=14"}},
    ThemeTargetStyles{L"TextBlock#TimeInnerTextBlock", {
        L"FontSize=14"}},
    ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"Margin=0,-4,0,-4",
        L"Opacity=0.75"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground > Grid > Rectangle#BackgroundStroke", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground > Grid > Rectangle#BackgroundFill", {
        L"Fill:=<SolidColorBrush Color=\"{ThemeResource TextFillColorInverse}\"  Opacity=\"1.0\" />"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground > Grid", {
```

```
        L"Background=Black"}},
};

const Theme g_themeUltraWideFriendly = {{
    ThemeTargetStyles{L":root > ScrollViewer > ScrollContentPresenter > Border > Grid", {
        L"ColumnDefinitions:=<ColumnDefinitionCollection><ColumnDefinition
Width=\"*\"/><ColumnDefinition Width=\"Auto\"/><ColumnDefinition
Width=\"Auto\"/><ColumnDefinition Width=\"*\"/></ColumnDefinitionCollection>",
        L"HorizontalAlignment=Stretch",
        L"Background:=<SolidColorBrush Color=\"$GhostBarBackgroundColor\"/>",
        L"ActualWidth=>containerGridWidth"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid@DockingStates", {
        L"Tag=horizontal",
        L"Tag@DockedLeft=vertical",
        L"Tag@DockedRight=vertical",
        L"Tag=>taskbarDock"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
        L"Grid.Column=1",
        L"Width={{taskbarDock==`vertical`?skip():`Auto`}}",
        L"HorizontalAlignment=Right",
        L"Margin=0,0,$IslandHorizontalMargin,0",
        L"MaxWidth={{containerGridWidth>0?min($TaskbarFrameMaxWidth,containerGridWidth):$Tas
kbarFrameMaxWidth}}"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid", {
        L"Background:=<SolidColorBrush Color=\"$IslandBackgroundColor\"/>",
        L"CornerRadius=10",
        L"Padding=25,0,25,0",
        L"Margin=0,$IslandVerticalMargin,0,$IslandVerticalMargin"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
        L"Grid.Column=2",
        L"Width=Auto",
        L"HorizontalAlignment=Left",
        L"Margin=$IslandHorizontalMargin,0,0,0"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"Background:=<SolidColorBrush Color=\"$IslandBackgroundColor\"/>",
        L"CornerRadius=10",
        L"Padding=5,0,5,0",
        L"Margin=0,$IslandVerticalMargin,0,$IslandVerticalMargin"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#BackgroundControl", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground >
Grid > Rectangle#BackgroundFill", {
        L"Fill=Transparent"}},
}, {
    L"GhostBarBackgroundColor=#66000000",
    L"IslandBackgroundColor={ThemeResource ControlFillColorDefault}",
    L"IslandVerticalMargin=3",
    L"IslandHorizontalMargin=5",
    L"TaskbarFrameMaxWidth=8000",
}};

const Theme g_themeLiquidGlass2 = {{
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid >
Border#BackgroundElement,
Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Border#BackgroundElement",
{
```

```
      L"Margin=0",
      L"BorderThickness=0"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#AugmentedEntryPointContentGrid", {
      L"Margin=12,0,0,0",
      L"HorizontalAlignment=Left"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton", {
      L"Margin=0",
      L"Visibility={{clickThroughTaskbar}}"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
      L"Margin=-400,0,0,0",
      L"Height={{TaskHeight - 8}}",
      L"MinWidth=80",
      L"MaxWidth=200",
      L"Width=Auto",
      L"CornerRadius=30",
      L"Padding=0",
      L"Background:=<WindhawkBlur BlurAmount=\"5\" TintColor=\"#39101010\" TintSaturation=\"1.5\"/>",
      L"CornerRadius={{(TaskHeight/4)*1.8}}",
      L"BorderThickness=1.2,1,1.2,1",
      L"BorderBrush:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#70D3D3D3\" Offset=\"0.0\" /><GradientStop Color=\"#50404040\" Offset=\"0.1\" /><GradientStop Color=\"#60404040\" Offset=\"0.25\" /><GradientStop Color=\"#70202020\" Offset=\"0.5\" /><GradientStop Color=\"#90404040\" Offset=\"0.75\" /><GradientStop Color=\"#90404040\" Offset=\"0.9\" /><GradientStop Color=\"#70C1C1C1\" Offset=\"1\" /></LinearGradientBrush>"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Rectangle, Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Rectangle", {
      L"Width=4.5",
      L"Height=4.2",
      L"Fill@Inactive=#50ffffff"}},
    ThemeTargetStyles{L":root > ScrollViewer > ScrollContentPresenter > Border > Grid", {
      L"ColumnDefinitions:=<ColumnDefinitionCollection><ColumnDefinition Width=\"*\"/><ColumnDefinition Width=\"Auto\"/><ColumnDefinition Width=\"*\"/><ColumnDefinition Width=\"Auto\"/></ColumnDefinitionCollection>",
      L"ActualWidth=>containerGridWidth",
      L"ActualHeight=>TaskHeight",
      L"HorizontalAlignment=Stretch"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid@DockingStates", {
      L"Tag=horizontal",
      L"Tag@DockedLeft=vertical",
      L"Tag@DockedRight=vertical",
      L"Tag=>taskbarDock"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
      L"Width={{taskbarDock==`vertical`?skip():`Auto`}}",
      L"HorizontalAlignment=Center",
      L"MinWidth=100",
      L"MaxWidth={{containerGridWidth>0?max(containerGridWidth-250,100):`Infinity`}}",
      L"Grid.Column=1"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
      L"Padding=41,0,41,0",
      L"Margin=0,0,0,4",
      L"Background:=Transparent",
      L"HorizontalAlignment=Center",
      L"Width=Auto"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground", {
      L"Margin=-40,0,-40,0"}},
```

```
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground > Grid > Rectangle#BackgroundFill", {
        L"Fill:=<WindhawkBlur BlurAmount=\"3\" TintColor=\"#14090909\" TintSaturation=\"1.2\"/>",
        L"RadiusX={{(TaskHeight/4.1)*2}}",
        L"RadiusY={{(TaskHeight/4.1)*2}}",
        L"StrokeThickness=1",
        L"Canvas.ZIndex=1",
        L"Stroke:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#69E0E0E0\" Offset=\"0.0\" /><GradientStop Color=\"#70707070\" Offset=\"0.1\" /><GradientStop Color=\"#70505050\" Offset=\"0.25\" /><GradientStop Color=\"#70292929\" Offset=\"0.5\" /><GradientStop Color=\"#70505050\" Offset=\"0.75\" /><GradientStop Color=\"#70707070\" Offset=\"0.9\" /><GradientStop Color=\"#65D0D0D0\" Offset=\"1\" /></LinearGradientBrush>"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground > Grid > Rectangle#BackgroundStroke", {
        L"Visibility=Visible",
        L"Stroke:=<WindhawkBlur BlurAmount=\"25\" TintColor=\"#00000000\"/>",
        L"StrokeThickness=6",
        L"RadiusX={{(TaskHeight/4.1)*2}}",
        L"RadiusY={{(TaskHeight/4.1)*2}}",
        L"Canvas.ZIndex=-1",
        L"VerticalAlignment=Stretch",
        L"HorizontalAlignment=Stretch",
        L"Height=NaN",
        L"Fill:=<WindhawkBlur BlurAmount=\"0\" TintColor=\"#00101010\"/>"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"Background:=<WindhawkBlur BlurAmount=\"5\" TintColor=\"#39101010\" TintSaturation=\"1.5\"/>",
        L"CornerRadius={{(TaskHeight/4)*1.8}}",
        L"BorderThickness=1.2,1,1.2,1",
        L"BorderBrush:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#70D3D3D3\" Offset=\"0.0\" /><GradientStop Color=\"#50404040\" Offset=\"0.1\" /><GradientStop Color=\"#60404040\" Offset=\"0.25\" /><GradientStop Color=\"#70202020\" Offset=\"0.5\" /><GradientStop Color=\"#90404040\" Offset=\"0.75\" /><GradientStop Color=\"#90404040\" Offset=\"0.9\" /><GradientStop Color=\"#70C1C1C1\" Offset=\"1\" /></LinearGradientBrush>",
        L"Padding=11,0,10,0",
        L"Margin=0,1,0,1",
        L"VerticalAlignment=Center",
        L"Height={{TaskHeight - 8}}"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
        L"HorizontalAlignment=Left",
        L"Margin=50,0,0,0",
        L"Grid.Column=2"}},
    ThemeTargetStyles{L"Taskbar.Gripper#GripperControl", {
        L"MinWidth=24"}},
    ThemeTargetStyles{L"MenuFlyoutPresenter", {
        L"CornerRadius=30"}},
    ThemeTargetStyles{L"MenuFlyoutPresenter > Border", {
        L"Background:=<WindhawkBlur BlurAmount=\"5\" TintColor=\"#2F131313\"/>",
        L"BorderThickness=1,1,1,1",
        L"CornerRadius=32,32,30,30",
        L"Padding=5,9,6,9",
        L"BorderBrush:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#59D3D3D3\" Offset=\"0.0\" /><GradientStop Color=\"#45494949\" Offset=\"0.1\" /><GradientStop Color=\"#50505050\" Offset=\"0.5\" /><GradientStop Color=\"#45494949\" Offset=\"0.9\" /><GradientStop Color=\"#50D3D3D3\" Offset=\"1\" /></LinearGradientBrush>"}},
    ThemeTargetStyles{L"Grid#ConfirmatorMainGrid", {
        L"Background:=<WindhawkBlur BlurAmount=\"5\" TintColor=\"#1C101010\" TintSaturation=\"1.2\"/>",
        L"CornerRadius=34",
```

```
        L"ActualWidth=>FlyWid",
        L"ActualHeight=>FlyHyt",
        L"BorderThickness=1,1,1,0.5",
        L"BorderBrush:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#70D3D3D3\" Offset=\"0.0\" /><GradientStop Color=\"#50454545\" Offset=\"0.16\" /><GradientStop Color=\"#50404040\" Offset=\"0.28\" /><GradientStop Color=\"#70101010\" Offset=\"0.5\" /><GradientStop Color=\"#50404040\" Offset=\"0.72\" /><GradientStop Color=\"#50404040\" Offset=\"0.84\" /><GradientStop Color=\"#70D3D3D3\" Offset=\"1\" /></LinearGradientBrush>",
        L"RenderTransform:=<ScaleTransform ScaleX=\"1.12\" ScaleY=\"1.15\" />",
        L"RenderTransformOrigin=0.5,0",
        L"Margin=7,0,13,155",
        L"MinHeight=53",
        L"MinWidth=180",
        L"Padding={{ max(8, min(10, FlyWid * 0.035)) }},{{ max(0, min(2, FlyHyt * 0.02)) }},{{ max(8, min(10, FlyWid * 0.035)) }},{{ max(0, min(2, FlyHyt * 0.02)) }}"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalTrackRect", {
        L"Height=21",
        L"RadiusX=10.5",
        L"RadiusY=10.5",
        L"Fill:=<WindhawkBlur BlurAmount=\"18\" TintColor=\"#35252525\"/>",
        L"Stroke:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#809F9F9F\" Offset=\"0.0\" /><GradientStop Color=\"#10696969\" Offset=\"0.5\" /><GradientStop Color=\"#809F9F9F\" Offset=\"1\" /></LinearGradientBrush>",
        L"StrokeThickness=1",
        L"Margin=0.5"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalDecreaseRect", {
        L"Height=20",
        L"RadiusX=10",
        L"RadiusY=10"}},
    ThemeTargetStyles{L"Grid#TextConfirmator", {
        L"MinHeight=63",
        L"VerticalAlignment=Center",
        L"HorizontalAlignment=Center"}},
    ThemeTargetStyles{L"TextBlock#confirmatorText", {
        L"FontSize=16",
        L"FontWeight=Medium",
        L"VerticalAlignment=Center",
        L"HorizontalAlignment=Center"}},
    ThemeTargetStyles{L"MenuFlyoutItem", {
        L"FontSize=14",
        L"FontWeight=Medium"}},
    ThemeTargetStyles{L"MenuFlyoutSubItem", {
        L"FontSize=14",
        L"FontWeight=Medium"}},
    ThemeTargetStyles{L"Border#OverflowFlyoutBackgroundBorder", {
        L"Background:=<WindhawkBlur BlurAmount=\"5\" TintColor=\"#15101010\"/>",
        L"BorderThickness=1",
        L"CornerRadius={{ max(27, min(48, (OverflowHeight / 2) * 1)) }}",
        L"ActualHeight=>OverflowHeight",
        L"Margin={{ max(-4, min(-15, OverflowHeight * 0.35)) }},{{ max(-4, min(-8.5, OverflowHeight * 0.2)) }},{{ max(-4, min(-15, OverflowHeight * 0.35)) }},{{ max(-4, min(-9, OverflowHeight * 0.2)) }}",
        L"BorderBrush:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#70D3D3D3\" Offset=\"0.0\" /><GradientStop Color=\"#50404040\" Offset=\"0.15\" /><GradientStop Color=\"#45404040\" Offset=\"0.28\" /><GradientStop Color=\"#55252525\" Offset=\"0.5\" /><GradientStop Color=\"#45404040\" Offset=\"0.72\" /><GradientStop Color=\"#50404040\" Offset=\"0.85\" /><GradientStop Color=\"#70C1C1C1\" Offset=\"1\" /></LinearGradientBrush>"}},

    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputSwitcher > ContentControl > ContentPresenter > Grid", {
```

```
        L"Background:=<WindhawkBlur BlurAmount=\"5\" TintColor=\"#1C101010\"/>"}},

    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputSwitcher > ContentControl > ContentPresenter > Grid > Grid", {
        L"Background:=Red"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.ToolTip > Windows.UI.Xaml.Controls.ContentPresenter#LayoutRoot", {
        L"Background:=<WindhawkBlur BlurAmount=\"4\" TintColor=\"#20000000\"/>",
        L"BorderThickness=1",
        L"CornerRadius={{ max(19, min(40, (TooltipHeight / 2) * 1)) }}",
        L"ActualHeight=>TooltipHeight",
        L"Padding={{ max(16, min(23, TooltipHeight * 0.35)) }},{{ max(9, min(11.5, TooltipHeight * 0.2)) }},{{ max(16, min(22, TooltipHeight * 0.35)) }},{{ max(9.5, min(12, TooltipHeight * 0.22)) }}",
        L"BorderBrush:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#70D3D3D3\" Offset=\"0.0\" /><GradientStop Color=\"#59505050\" Offset=\"0.15\" /><GradientStop Color=\"#60404040\" Offset=\"0.28\" /><GradientStop Color=\"#60202020\" Offset=\"0.5\" /><GradientStop Color=\"#60404040\" Offset=\"0.72\" /><GradientStop Color=\"#50595959\" Offset=\"0.85\" /><GradientStop Color=\"#70C1C1C1\" Offset=\"1\" /></LinearGradientBrush>",
        L"FontSize=14",
        L"FontWeight=Medium"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid > Rectangle#BackgroundFill", {
        L"Fill:=<WindhawkBlur BlurAmount=\"3.5\" TintColor=\"#1D101010\"/>",
        L"RadiusX=34",
        L"RadiusY=34",
        L"StrokeThickness:=1",
        L"Stroke:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#70C0C0C0\" Offset=\"0.0\" /><GradientStop Color=\"#60696969\" Offset=\"0.5\" /><GradientStop Color=\"#50505050\" Offset=\"1\" /></LinearGradientBrush>"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid > Rectangle#BackgroundStroke", {
        L"Visibility=Visible",
        L"Stroke:=<WindhawkBlur BlurAmount=\"30\" TintColor=\"#10303030\"/>",
        L"StrokeThickness=6",
        L"RadiusX=30",
        L"RadiusY=30",
        L"Canvas.ZIndex=-1",
        L"VerticalAlignment=Stretch",
        L"HorizontalAlignment=Stretch",
        L"Height=NaN",
        L"Margin=0,1,0,0",
        L"Fill:=<WindhawkBlur BlurAmount=\"0\" TintColor=\"#00101010\"/>"}},
    ThemeTargetStyles{L"Border#HoverFlyoutBackground", {
        L"Background:=Transparent",
        L"BorderThickness=0",
        L"CornerRadius=34"}},
    ThemeTargetStyles{L"ContentPresenter#HoverFlyoutContent", {
        L"CornerRadius=30",
        L"BorderThickness=0",
        L"Padding=5,2,5,7",
        L"Margin:=1",
        L"Background:=Transparent"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.TextBlock#DisplayName", {
        L""}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#Root > Windows.UI.Xaml.Controls.Image#IconImage", {
        L""}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab > Grid#ModalRootGrid > Border#BackgroundElement", {
        L"CornerRadius={{ max(68, min(90, (AltTabHeight / 5) * 1.75)) }}",
```

```
        L"ActualHeight=>AltTabHeight",
        L"BorderThickness=0",
        L"Background=Transparent"}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab > Grid#ModalRootGrid > Border#BackgroundElement > WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemList", {
        L"Background:=<WindhawkBlur BlurAmount=\"6\" TintColor=\"#20000000\" TintSaturation=\"1.2\"/>",
        L"CornerRadius={{ max(68, min(90, (AltTabHeight / 5) * 1.75)) }}",
        L"BorderThickness=1",
        L"BorderBrush:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#69D3D3D3\" Offset=\"0.0\" /><GradientStop Color=\"#5F303030\" Offset=\"0.1\" /><GradientStop Color=\"#70303030\" Offset=\"0.5\" /><GradientStop Color=\"#5F303030\" Offset=\"0.9\" /><GradientStop Color=\"#69D3D3D3\" Offset=\"1\" /></LinearGradientBrush>"}},

    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemListViewItem > Grid > Border", {
        L"CornerRadius=25,25,12,12",
        L"Background:=<WindhawkBlur BlurAmount=\"18\" TintColor=\"#701F1F1F\"/>",
        L"BorderThickness=1",
        L"BorderBrush:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#50C0C0C0\" Offset=\"0.0\" /><GradientStop Color=\"#509F9F9F\" Offset=\"0.1\" /><GradientStop Color=\"#50707070\" Offset=\"0.5\" /><GradientStop Color=\"#55505050\" Offset=\"0.9\" /><GradientStop Color=\"#69404040\" Offset=\"1\" /></LinearGradientBrush>"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.AnimatedIcon#VolumeIcon", {
        L"Width=20",
        L"Height=20",
        L"VerticalAlignment=Center",
        L"Margin=3,0,-1.2,0"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.AnimatedIcon#BrightnessIcon", {
        L"Width=21",
        L"Height=21"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapBarBorder", {
        L"Background:=<WindhawkBlur BlurAmount=\"5\" TintColor=\"#1B242424\"/>",
        L"BorderThickness=1.2,1,1.2,1",
        L"BorderBrush:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#60D3D3D3\" Offset=\"0.0\" /><GradientStop Color=\"#4F494949\" Offset=\"0.1\" /><GradientStop Color=\"#60505050\" Offset=\"0.5\" /><GradientStop Color=\"#4F494949\" Offset=\"0.9\" /><GradientStop Color=\"#60D3D3D3\" Offset=\"1\" /></LinearGradientBrush>",
        L"CornerRadius=27",
        L"Margin=0",
        L"Padding=-30,0,-30,0"}},

    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.SnapAssistList > Grid > Border#ListBackground", {
        L"ActualHeight=>SnapAssistHyt",
        L"Background:=<WindhawkBlur BlurAmount=\"15\" TintColor=\"#20181818\"/>",
        L"BorderThickness=1.2,1,1.2,1",
        L"BorderBrush:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#70D3D3D3\" Offset=\"0.0\" /><GradientStop Color=\"#4F494949\" Offset=\"0.15\" /><GradientStop Color=\"#50505050\" Offset=\"0.5\" /><GradientStop Color=\"#4F494949\" Offset=\"0.85\" /><GradientStop Color=\"#70C1C1C1\" Offset=\"1\" /></LinearGradientBrush>",
        L"CornerRadius={{ max(20, min(30, SnapAssistHyt * 0.05)) }}",
        L"Margin=8",
        L"Padding=8"}},
    ThemeTargetStyles{L"Border#BackgroundDimmingLayer", {
        L"Background:=<WindhawkBlur BlurAmount=\"12\" TintColor=\"#101F1F1F\"/>"}},
    ThemeTargetStyles{L"Border#VirtualDesktopBarBackground", {
        L"Background:=<WindhawkBlur BlurAmount=\"15\" TintColor=\"#6B242424\"/>",
        L"BorderThickness=1",
```

```
        L"BorderBrush:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#50DDDDDD\" Offset=\"0.0\" /><GradientStop Color=\"#0C696969\" Offset=\"0.28\" /><GradientStop Color=\"#50C1C1C1\" Offset=\"1\" /></LinearGradientBrush>",
        L"Margin=45,-5,45,-2",
        L"CornerRadius=45"}},
    ThemeTargetStyles{L"TextBlock#VirtualDesktopNameBlock", {
        L"Margin=12,8,0,5",
        L"FontSize=14"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.NewVirtualDesktopElementThemed > Grid#MainGrid > TextBlock", {
        L"Margin=12,8,0,0",
        L"FontSize=14"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopElementThemed > Grid#MainGrid > Border#MainBorder", {
        L"CornerRadius=19"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.NewVirtualDesktopElementThemed > Grid#MainGrid > Border#MainBorder", {
        L"CornerRadius=19"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopElementThemed > Grid#MainGrid > Border#BorderHighlight", {
        L"CornerRadius=19",
        L"Background:=<WindhawkBlur BlurAmount=\"0\" TintColor=\"#35252525\"/>",
        L"BorderThickness=1",
        L"BorderBrush:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#50C1C1C1\" Offset=\"0.0\" /><GradientStop Color=\"#20696969\" Offset=\"0.5\" /><GradientStop Color=\"#50AFAFAF\" Offset=\"1\" /></LinearGradientBrush>"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.NewVirtualDesktopElementThemed > Grid#MainGrid > Border#BorderHighlight", {
        L"CornerRadius=19",
        L"Background:=<WindhawkBlur BlurAmount=\"0\" TintColor=\"#35252525\"/>",
        L"BorderThickness=1",
        L"BorderBrush:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#50C1C1C1\" Offset=\"0.0\" /><GradientStop Color=\"#20696969\" Offset=\"0.5\" /><GradientStop Color=\"#50AFAFAF\" Offset=\"1\" /></LinearGradientBrush>"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopElementThemed > Grid#MainGrid > Border#ActiveDesktopPill", {
        L"Margin=0,0,0,2.5",
        L"Width=60"}},
    ThemeTargetStyles{L"Taskbar.OverflowToggleButton", {
        L"MinWidth=60",
        L"Margin=5,0,5,0"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border#BackgroundElement, Taskbar.TaskListLabeledButtonPanel@CommonStates > Border#BackgroundElement", {
        L"Background=Transparent",
        L"BorderBrush=Transparent"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel@CommonStates > Grid > Border#BackgroundElement, Taskbar.TaskListButtonPanel@CommonStates > Border#BackgroundElement", {
        L"BorderBrush=Transparent",
        L"Background=Transparent"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView", {
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"0\" />"}},
    ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack", {
        L"Visibility=1"}},
    ThemeTargetStyles{L"ContentPresenter#ContentPresenter > Grid#ContentGrid > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#LottieIcon", {
        L"Visibility=1"}},
```

```
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid >
ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[2] >
SystemTray.IconView > Grid > Grid", {
        L"Visibility=1"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Grid >
ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.IconView#SystemTrayIcon > Grid > Grid > SystemTray.TextIconContent", {
        L"Visibility=1"}},
    ThemeTargetStyles{L"SystemTray.NotificationAreaIcons#NotificationAreaIcons >
ItemsPresenter", {
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"0\" />"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton", {
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"0\" />"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton", {
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"0\" />"}},
    ThemeTargetStyles{L"SystemTray.DateTimeIconContent > Grid#ContainerGrid > StackPanel >
TextBlock#TimeInnerTextBlock", {
        L""}},
    ThemeTargetStyles{L"SystemTray.DateTimeIconContent > Grid#ContainerGrid > StackPanel >
TextBlock#DateInnerTextBlock", {
        L""}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton@CommonStates > Grid >
Border#BackgroundBorder", {
        L"Background:=Transparent",
        L"BorderBrush:=Transparent"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView > Grid@CommonStates", {
        L"RenderTransformOrigin=0.5,0.5",
        L"RenderTransform:=<ScaleTransform ScaleX=\"1.2\" ScaleY=\"1.2\" />",
        L"RenderTransform@PointerOver:=<ScaleTransform ScaleX=\"1.3\" ScaleY=\"1.3\" />",
        L"RenderTransform@Pressed:=<ScaleTransform ScaleX=\"1.0\" ScaleY=\"1.0\" />"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid@CommonStates",
{
        L"RenderTransformOrigin=0.5,0.5",
        L"RenderTransform:=<ScaleTransform ScaleX=\"1.2\" ScaleY=\"1.2\" />",
        L"RenderTransform@PointerOver:=<ScaleTransform ScaleX=\"1.3\" ScaleY=\"1.3\" />",
        L"RenderTransform@Pressed:=<ScaleTransform ScaleX=\"1.0\" ScaleY=\"1.0\" />"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton >
Grid@CommonStates", {
        L"RenderTransformOrigin=0.5,0.5",
        L"RenderTransform:=<ScaleTransform ScaleX=\"1.05\" ScaleY=\"1.05\" />",
        L"RenderTransform@PointerOver:=<ScaleTransform ScaleX=\"1.15\" ScaleY=\"1.15\" />",
        L"RenderTransform@Pressed:=<ScaleTransform ScaleX=\"0.9\" ScaleY=\"0.9\" />"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView > Grid#ContainerGrid@", {
        L"RenderTransformOrigin=0.5,0.5",
        L"RenderTransform:=<ScaleTransform ScaleX=\"1.28\" ScaleY=\"1.28\" />",
        L"RenderTransform@Normal:=<ScaleTransform ScaleX=\"1.28\" ScaleY=\"1.28\" />",
        L"RenderTransform@PointerOver:=<ScaleTransform ScaleX=\"1.37\" ScaleY=\"1.37\" />",
        L"RenderTransform@Pressed:=<ScaleTransform ScaleX=\"1.1\" ScaleY=\"1.1\" />",
        L"RenderTransform@CheckedNormal:=<ScaleTransform ScaleX=\"1.37\" ScaleY=\"1.37\"
/>",
        L"RenderTransform@CheckedPointerOver:=<ScaleTransform ScaleX=\"1.37\"
ScaleY=\"1.37\" />",
        L"RenderTransform@CheckedPressed:=<ScaleTransform ScaleX=\"1.1\" ScaleY=\"1.1\"
/>"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > Grid >
SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid >
SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView >
Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid >
SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base >
TextBlock#InnerTextBlock, ScrollViewer > ScrollContentPresenter > Border > Grid >
SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid >
SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView >
Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid >
```

```
SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock", {
        L"Margin=2,0,3,0"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton@CommonStates > Grid > Border#BackgroundBorder", {
        L"Background:=Transparent",
        L"BorderBrush:=Transparent"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView > Grid#ContainerGrid > Border#BackgroundBorder", {
        L"Background:=Transparent",
        L"BorderBrush:=Transparent"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView > * > TextBlock#InnerTextBlock[Text=\uE971]", {
        L"Text:=&#xED14;"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView@CommonStates > Grid#ContainerGrid > Border#BackgroundBorder", {
        L"Background:=<SolidColorBrush Color=\"Transparent\"/>",
        L"BorderBrush:=<SolidColorBrush Color=\"Transparent\"/>"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView@CommonStates > Grid > Border#BackgroundBorder", {
        L"Background:=<SolidColorBrush Color=\"Transparent\"/>",
        L"BorderBrush:=<SolidColorBrush Color=\"Transparent\"/>"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#CloseButton", {
        L"CornerRadius=15",
        L"Background:=<WindhawkBlur BlurAmount=\"15\" TintColor=\"#2D101010\"/>",
        L"BorderThickness=1",
        L"BorderBrush:=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#70D3D3D3\" Offset=\"0.0\" /><GradientStop Color=\"#60404040\" Offset=\"0.15\" /><GradientStop Color=\"#55404040\" Offset=\"0.28\" /><GradientStop Color=\"#65252525\" Offset=\"0.5\" /><GradientStop Color=\"#55404040\" Offset=\"0.72\" /><GradientStop Color=\"#60404040\" Offset=\"0.85\" /><GradientStop Color=\"#70C1C1C1\" Offset=\"1\" /></LinearGradientBrush>"}},
    ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView@CommonStates > Grid > Border#BackgroundBorder", {
        L"Background:=Transparent",
        L"BorderBrush:=Transparent"}},
    ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView@CommonStates > Border#BackgroundBorder", {
        L"Background:=Transparent",
        L"BorderBrush:=Transparent"}},
    ThemeTargetStyles{L"Border#ThumbnailVisualHostWrapper", {
        L"HorizontalAlignment=Center",
        L"VerticalAlignment=Center"}},
    ThemeTargetStyles{L"Border#ThumbnailVisualHost", {
        L"CornerRadius=14",
        L"Background:=<SolidColorBrush Color=\"Transparent\"/>",
        L"BorderThickness=1",
        L"BorderBrush:=<SolidColorBrush Color=\"Transparent\"/>"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#SwitchItemElementCloseButton", {
        L"CornerRadius=16",
        L"Padding=2.5",
        L"Margin=0,1,10,0",
        L"Background:=<WindhawkBlur BlurAmount=\"0\" TintColor=\"#5D101010\"/>"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.FlyoutPresenter", {
        L"CornerRadius=33"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapPickerBorder", {
        L"Background:=<WindhawkBlur BlurAmount=\"5\" TintColor=\"#2D101010\"/>",
        L"BorderBrush:=<WindhawkBlur BlurAmount=\"40\" TintColor=\"#3D404040\"/>",
        L"BorderThickness=1",
        L"Padding=2,3,2,3",
```

```
        L"CornerRadius=33",
        L"Canvas.ZIndex=-5"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#LayoutBorder", {
        L"Background:=<WindhawkBlur BlurAmount=\"15\" TintColor=\"#30505050\"/>",
        L"BorderThickness=1",
        L"BorderBrush:=<SolidColorBrush Color=\"#50FFFFFF\"/>",
        L"CornerRadius=12"}},
    ThemeTargetStyles{L"SnapLayout.SnapLayoutControl#SuggestionSnapLayout
Windows.UI.Xaml.Controls.Border#LayoutBorder", {
        L"Background:=Red"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#LayoutGrid >
Windows.UI.Xaml.Controls.Button", {
        L"BorderBrush:=<SolidColorBrush Color=\"#70BBBBBB\"/>",
        L"BorderThickness=1",
        L"CornerRadius=9.2",
        L"Margin=1.5"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#LayoutGrid >
Windows.UI.Xaml.Controls.Button@CommonStates >
Windows.UI.Xaml.Controls.Grid#RootGrid", {
        L"Background@PointerOver:=<SolidColorBrush Color=\"#40FFFFFF\"/>",
        L"Background@Pressed:=<SolidColorBrush Color=\"#20FFFFFF\"/>"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#WindowGroupSuggestionButton >
Windows.UI.Xaml.Controls.ContentPresenter#ContentPresenter", {
        L"Background:=Transparent"}},

ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#WindowGroupSuggestionButton@Co
mmonStates > Windows.UI.Xaml.Controls.Grid#RootGrid", {
        L"Background@PointerOver:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorDark2}\" Opacity=\"1\" />",
        L"Background@Pressed:=<SolidColorBrush Color=\"{ThemeResource
SystemAccentColorDark2}\" Opacity=\"1\" />"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel", {
        L"Visibility=1"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Grid >
Border#BackgroundElement, SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Border#BackgroundElement", {
        L"Visibility=1"}},

ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel", {
        L"Visibility=1"}},

ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Grid > Border#BackgroundElement,
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Border#BackgroundElement", {
        L"Visibility=1"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchPillButton#SearchPill >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel", {
        L"Visibility=0",
        L"Background:=<WindhawkBlur BlurAmount=\"15\" TintColor=\"#30505050\"/>",
        L"CornerRadius=20",
        L"Margin=0,10,0,10",
        L"BorderBrush:=<SolidColorBrush Color=\"#40BBBBBB\"/>",
        L"BorderThickness=1"}},

ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Grid > Border#SearchPillBackgroundElement,
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Border#SearchPillBackgroundElement", {
        L"Visibility=1"}},
}};
```

```
const Theme g_themeLiquidGlass = {{
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
      L"BorderThickness=$BorderThickness",
      L"BorderBrush:=$BorderBrush",
      L"Background:=$Background"}},
   ThemeTargetStyles{L"Rectangle#BackgroundFill", {
      L"Visibility=1"}},
   ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
      L"Visibility=1"}},
   ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
      L"Margin=-3,0"}},
   ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
      L"Background:=$ElementBackground",
      L"BorderBrush:=$ElementBorderBrush",
      L"BorderThickness=$ElementBorderThickness",
      L"CornerRadius=$ElementCornerRadius",
      L"Margin=6"}},
   ThemeTargetStyles{L"SystemTray.ChevronIconView", {
      L"CornerRadius=$ElementCornerRadius"}},
   ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon", {
      L"CornerRadius=$ElementCornerRadius"}},
   ThemeTargetStyles{L"SystemTray.OmniButton", {
      L"CornerRadius=$ElementCornerRadius"}},
   ThemeTargetStyles{L"SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid", {
      L"CornerRadius=$ElementCornerRadius"}},
   ThemeTargetStyles{L"Taskbar.Gripper#GripperControl", {
      L"Width=Auto",
      L"MinWidth=24",
      L"HorizontalAlignment=Left"}},
   ThemeTargetStyles{L"TextBlock#TimeInnerTextBlock", {
      L"FontSize=13",
      L"Margin=0",
      L"Padding=0",
      L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"0\" />"}},
   ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
      L"Visibility=1"}},
   ThemeTargetStyles{L"TextBlock#InnerTextBlock[Text=\uE971]", {
      L"Text=\uED14"}},
   ThemeTargetStyles{L"TextBlock#SearchBoxTextBlock", {
      L"Text=Search This PC",
      L"FontSize=10"}},
   ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid > Grid > SystemTray.TextIconContent", {
      L"Visibility=1"}},
   ThemeTargetStyles{L"Border#OverflowFlyoutBackgroundBorder", {
      L"BorderThickness=$BorderThickness",
      L"BorderBrush:=$BorderBrush",
      L"Background:=$Background",
      L"CornerRadius=$CornerRadius"}},
   ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab > Windows.UI.Xaml.Controls.Grid#ModalRootGrid > Windows.UI.Xaml.Controls.Border#BackgroundElement", {
      L"BorderThickness=0",
      L"Background=Transparent",
```

```
        L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab > Windows.UI.Xaml.Controls.Grid#ModalRootGrid > Windows.UI.Xaml.Controls.Border#BackgroundElement > WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemList", {
        L"Background:=$Background",
        L"BorderThickness=$BorderThickness",
        L"BorderBrush:=$BorderBrush",
        L"CornerRadius=$CornerRadius"}},

    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopBarElement#VirtualDesktopBar", {
        L"CornerRadius=$CornerRadius",
        L"Background:=$Background"}},
    ThemeTargetStyles{L"Border#BackgroundDimmingLayer", {
        L"Background:=$Background",
        L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid > Border#BackgroundElement, Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Border#BackgroundElement", {
        L"CornerRadius=$CornerRadius",
        L"BorderThickness=$ElementBorderThickness"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton", {
        L"CornerRadius=$ElementCornerRadius",
        L"BorderThickness=$ElementBorderThickness"}},
    ThemeTargetStyles{L"Border#SnapBarBorder", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"CornerRadius=$CornerRadius",
        L"BorderThickness=$BorderThickness",
        L"Margin=2"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border#BackgroundElement, Taskbar.TaskListLabeledButtonPanel@CommonStates > Border#BackgroundElement", {
        L"CornerRadius=$ElementCornerRadius",
        L"BorderThickness=$ElementBorderThickness",
        L"Background@ActiveNormal:=$ElementBackground",
        L"Background@ActivePointerOver:=$AccentBackground",
        L"Background@ActivePressed:=$ElementBackground2",
        L"Background@InactivePointerOver:=$AccentBackground",
        L"Background@InactivePressed:=$ElementBackground2",
        L"BorderBrush@ActiveNormal:=$ElementBorderBrush",
        L"BorderBrush@ActivePointerOver:=$ElementBorderBrush",
        L"BorderBrush@ActivePressed:=$ElementBorderBrush",
        L"BorderBrush@InactivePointerOver:=$ElementBorderBrush",
        L"BorderBrush@InactivePressed:=$ElementBorderBrush",
        L"Margin=1"}},
    ThemeTargetStyles{L"ContentPresenter#ContentPresenter@CommonStates", {
        L"CornerRadius=$ElementCornerRadius",
        L"BorderThickness=$ElementBorderThickness",
        L"Background@ActiveNormal:=$ElementBackground",
        L"Background@ActivePointerOver:=$AccentBackground",
        L"Background@ActivePressed:=$ElementBackground2",
        L"Background@InactivePointerOver:=$AccentBackground",
        L"Background@InactivePressed:=$ElementBackground2",
        L"BorderBrush@ActiveNormal:=$ElementBorderBrush",
        L"BorderBrush@ActivePointerOver:=$ElementBorderBrush",
        L"BorderBrush@ActivePressed:=$ElementBorderBrush",
        L"BorderBrush@InactivePointerOver:=$ElementBorderBrush",
```

```
        L"BorderBrush@InactivePressed:=$ElementBorderBrush",
        L"Margin=1"}},
    ThemeTargetStyles{L"Border#SnapPickerBorder", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"CornerRadius=$CornerRadius",
        L"BorderThickness=$BorderThickness",
        L"Margin=2"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
        L"Background:=Transparent"}},
    ThemeTargetStyles{L"ToolTip > ContentPresenter#LayoutRoot", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness:=$BorderThickness",
        L"CornerRadius=$CornerRadius"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopB
arElement > Grid#GridElement > Border#VirtualDesktopSwitcherBackground", {
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness=$BorderThickness",
        L"CornerRadius=$CornerRadius",
        L"Background:=$Background"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemListVi
ewItem > Grid > Border", {
        L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"Border#VirtualDesktopBarBackground", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness=$BorderThickness",
        L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"Rectangle#RunningIndicator", {
        L"Fill:=$AccentBackground",
        L"Width=14"}},
    ThemeTargetStyles{L"Rectangle#ShowDesktopPipe", {
        L"Visibility=1"}},
    ThemeTargetStyles{L"Rectangle#RightOverflowButtonDivider", {
        L"Visibility=1"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid@CommonStates > Grid >
Border#BackgroundElement, SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid@CommonStates > Border#BackgroundElement", {
        L"Background:=Transparent",
        L"BorderBrush:=Transparent"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid", {
        L"Background:=Transparent",
        L"BorderBrush:=Transparent"}},
    ThemeTargetStyles{L"Border#SearchPillBackgroundElement", {
        L"BorderBrush:=$ElementBorderBrush",
        L"BorderThickness=$ElementBorderThickness",
        L"CornerRadius=$ElementCornerRadius",
        L"Margin=0,1"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchBoxButton >
SearchUx.SearchUI.SearchButtonRootGrid@CommonStates > Grid >
Border#BackgroundElement, SearchUx.SearchUI.SearchBoxButton >
SearchUx.SearchUI.SearchButtonRootGrid@CommonStates > Border#BackgroundElement", {
        L"CornerRadius=$ElementCornerRadius",
        L"BorderThickness=$ElementBorderThickness",
        L"BorderBrush:=$ElementBorderBrush",
```

```
      L"Background:=$ElementBackground",
      L"Margin=0,-4"}},
   ThemeTargetStyles{L"Canvas#HoverFlyoutCanvas > Grid#HoverFlyoutGrid > Border#HoverFlyoutBackground", {
      L"Shadow:=",
      L"Background:=$Background",
      L"BorderBrush:=$BorderBrush",
      L"BorderThickness=$BorderThickness",
      L"CornerRadius=$CornerRadius"}},
   ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
      L"HorizontalAlignment=Right"}},
   ThemeTargetStyles{L"Grid#AugmentedEntryPointContentGrid", {
      L"HorizontalAlignment=Left"}},
}, {
   L"Background=<WindhawkBlur BlurAmount=\"15\" TintColor=\"{ThemeResource SystemAltLowColor}\" TintOpacity=\"0.2\" />",
   L"ElementBackground=<WindhawkBlur BlurAmount=\"20\" TintColor=\"{ThemeResource SystemAltLowColor}\" TintOpacity=\"0.4\" />",
   L"ElementBackground2=<WindhawkBlur BlurAmount=\"20\" TintColor=\"{ThemeResource SystemAltLowColor}\" TintOpacity=\"0.2\" />",
   L"AccentBackground=<WindhawkBlur BlurAmount=\"15\" TintColor=\"{ThemeResource SystemAccentColorLight1}\" TintOpacity=\"0.2\" />",
   L"BorderBrush=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#50808080\" Offset=\"0.0\" /><GradientStop Color=\"#50404040\" Offset=\"0.25\" /><GradientStop Color=\"#50808080\" Offset=\"1\" /></LinearGradientBrush>",
   L"ElementBorderBrush=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#50808080\" Offset=\"1\" /><GradientStop Color=\"#50606060\" Offset=\"0.15\" /></LinearGradientBrush>",
   L"BorderThickness=0.3,1,0.3,0.3",
   L"ElementBorderThickness=0.3,0.3,0.3,1",
   L"CornerRadius=12",
   L"ElementCornerRadius=8",
}};

const Theme g_themeLiquidGlass_variant_Alternate = {{
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid@DockingStates", {
      L"Tag=horizontal",
      L"Tag@DockedLeft=vertical",
      L"Tag@DockedRight=vertical",
      L"Tag=>taskbarDock"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
      L"Width={{taskbarDock==`vertical`?skip():`Auto`}}",
      L"MinWidth:=100",
      L"MaxWidth:=1200",
      L"HorizontalAlignment=Center"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
      L"Margin=0,3,12,3",
      L"BorderThickness=$BorderThickness",
      L"BorderBrush:=$BorderBrush",
      L"CornerRadius=$CornerRadius",
      L"Background:=$Background",
      L"Padding=1"}},
   ThemeTargetStyles{L"Rectangle#BackgroundFill", {
      L"Visibility=1"}},
   ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
      L"Visibility=1"}},
   ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
      L"Margin=-3,0"}},
```

```
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"Background:=$ElementBackground",
        L"BorderBrush:=$ElementBorderBrush",
        L"BorderThickness=$ElementBorderThickness",
        L"CornerRadius=$ElementCornerRadius",
        L"Margin=6"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView", {
        L"CornerRadius=$ElementCornerRadius"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon", {
        L"CornerRadius=$ElementCornerRadius"}},
    ThemeTargetStyles{L"SystemTray.OmniButton", {
        L"CornerRadius=$ElementCornerRadius"}},
    ThemeTargetStyles{L"SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid >
ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent >
Grid#ContainerGrid", {
        L"CornerRadius=$ElementCornerRadius"}},
    ThemeTargetStyles{L"Taskbar.Gripper#GripperControl", {
        L"Width=Auto",
        L"MinWidth=24",
        L"HorizontalAlignment=Left"}},
    ThemeTargetStyles{L"TextBlock#TimeInnerTextBlock", {
        L"FontSize=13",
        L"Margin=0",
        L"Padding=0",
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"0\" />"}},
    ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
        L"Visibility=1"}},
    ThemeTargetStyles{L"TextBlock#InnerTextBlock[Text=\uE971]", {
        L"Text=\uED14"}},
    ThemeTargetStyles{L"TextBlock#SearchBoxTextBlock", {
        L"Text=Search This PC",
        L"FontSize=10"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Grid >
ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.IconView#SystemTrayIcon > Grid > Grid > SystemTray.TextIconContent", {
        L"Visibility=1"}},
    ThemeTargetStyles{L"Border#OverflowFlyoutBackgroundBorder", {
        L"BorderThickness=$BorderThickness",
        L"BorderBrush:=$BorderBrush",
        L"Background:=$Background",
        L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab >
Windows.UI.Xaml.Controls.Grid#ModalRootGrid >
Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"BorderThickness=0",
        L"Background=Transparent",
        L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab >
Windows.UI.Xaml.Controls.Grid#ModalRootGrid >
Windows.UI.Xaml.Controls.Border#BackgroundElement >
WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemList", {
        L"Background:=$Background",
        L"BorderThickness=$BorderThickness",
        L"BorderBrush:=$BorderBrush",
        L"CornerRadius=$CornerRadius"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopB
arElement#VirtualDesktopBar", {
        L"CornerRadius=$CornerRadius",
        L"Background:=$Background"}},
```

```
    ThemeTargetStyles{L"Border#BackgroundDimmingLayer", {
        L"Background:=$Background",
        L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
        L"Margin=2",
        L"Background:=$ElementBackground",
        L"CornerRadius=$ElementCornerRadius",
        L"BorderThickness=$ElementBorderThickness",
        L"BorderBrush:=$ElementBorderBrush",
        L"Padding=0,-6",
        L"MaxWidth:=200",
        L"MaxHeight=46"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid > Border#BackgroundElement, Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Border#BackgroundElement", {
        L"CornerRadius=$ElementCornerRadius",
        L"BorderThickness=$ElementBorderThickness"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton", {
        L"CornerRadius=$ElementCornerRadius",
        L"BorderThickness=$ElementBorderThickness"}},
    ThemeTargetStyles{L"Border#SnapBarBorder", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"CornerRadius=$CornerRadius",
        L"BorderThickness=$BorderThickness",
        L"Margin=2"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border#BackgroundElement, Taskbar.TaskListLabeledButtonPanel@CommonStates > Border#BackgroundElement", {
        L"CornerRadius=$ElementCornerRadius",
        L"BorderThickness=$ElementBorderThickness",
        L"Background@ActiveNormal:=$ElementBackground",
        L"Background@ActivePointerOver:=$AccentBackground",
        L"Background@ActivePressed:=$ElementBackground2",
        L"Background@InactivePointerOver:=$AccentBackground",
        L"Background@InactivePressed:=$ElementBackground2",
        L"BorderBrush@ActiveNormal:=$ElementBorderBrush",
        L"BorderBrush@ActivePointerOver:=$ElementBorderBrush",
        L"BorderBrush@ActivePressed:=$ElementBorderBrush",
        L"BorderBrush@InactivePointerOver:=$ElementBorderBrush",
        L"BorderBrush@InactivePressed:=$ElementBorderBrush",
        L"Margin=1"}},
    ThemeTargetStyles{L"ContentPresenter#ContentPresenter@CommonStates", {
        L"CornerRadius=$ElementCornerRadius",
        L"BorderThickness=$ElementBorderThickness",
        L"Background@ActiveNormal:=$ElementBackground",
        L"Background@ActivePointerOver:=$AccentBackground",
        L"Background@ActivePressed:=$ElementBackground2",
        L"Background@InactivePointerOver:=$AccentBackground",
        L"Background@InactivePressed:=$ElementBackground2",
        L"BorderBrush@ActiveNormal:=$ElementBorderBrush",
        L"BorderBrush@ActivePointerOver:=$ElementBorderBrush",
        L"BorderBrush@ActivePressed:=$ElementBorderBrush",
        L"BorderBrush@InactivePointerOver:=$ElementBorderBrush",
        L"BorderBrush@InactivePressed:=$ElementBorderBrush",
        L"Margin=1"}},
```

```
    ThemeTargetStyles{L"Border#SnapPickerBorder", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"CornerRadius=$CornerRadius",
        L"BorderThickness=$BorderThickness",
        L"Margin=2"}},
    ThemeTargetStyles{L"ToolTip > ContentPresenter#LayoutRoot", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness:=$BorderThickness",
        L"CornerRadius=$CornerRadius"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopB
arElement > Grid#GridElement > Border#VirtualDesktopSwitcherBackground", {
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness=$BorderThickness",
        L"CornerRadius=$CornerRadius",
        L"Background:=$Background"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemListVi
ewItem > Grid > Border", {
        L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"Border#VirtualDesktopBarBackground", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness=$BorderThickness",
        L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"Rectangle#RunningIndicator", {
        L"Fill:=$AccentBackground",
        L"Width=14"}},
    ThemeTargetStyles{L"Rectangle#ShowDesktopPipe", {
        L"Visibility=1"}},
    ThemeTargetStyles{L"Rectangle#RightOverflowButtonDivider", {
        L"Visibility=1"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid@CommonStates > Grid >
Border#BackgroundElement, SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid@CommonStates > Border#BackgroundElement", {
        L"Background:=Transparent",
        L"BorderBrush:=Transparent"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid", {
        L"Background:=Transparent",
        L"BorderBrush:=Transparent"}},
    ThemeTargetStyles{L"Border#SearchPillBackgroundElement", {
        L"BorderBrush:=$ElementBorderBrush",
        L"BorderThickness=$ElementBorderThickness",
        L"CornerRadius=$ElementCornerRadius",
        L"Margin=0,1"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchBoxButton >
SearchUx.SearchUI.SearchButtonRootGrid@CommonStates > Grid >
Border#BackgroundElement, SearchUx.SearchUI.SearchBoxButton >
SearchUx.SearchUI.SearchButtonRootGrid@CommonStates > Border#BackgroundElement", {
        L"CornerRadius=$ElementCornerRadius",
        L"BorderThickness=$ElementBorderThickness",
        L"BorderBrush:=$ElementBorderBrush",
        L"Background:=$ElementBackground",
        L"Margin=0,-4"}},
    ThemeTargetStyles{L"Canvas#HoverFlyoutCanvas > Grid#HoverFlyoutGrid >
Border#HoverFlyoutBackground", {
        L"Shadow:=",
```

```
      L"Background:=$Background",
      L"BorderBrush:=$BorderBrush",
      L"BorderThickness=$BorderThickness",
      L"CornerRadius=$CornerRadius"}},
   ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
      L"HorizontalAlignment=Right"}},
   ThemeTargetStyles{L"Grid#AugmentedEntryPointContentGrid", {
      L"HorizontalAlignment=Left"}},
}, {
   L"Background=<WindhawkBlur BlurAmount=\"15\" TintColor=\"{ThemeResource SystemAltLowColor}\" TintOpacity=\"0.2\" />",
   L"ElementBackground=<WindhawkBlur BlurAmount=\"20\" TintColor=\"{ThemeResource SystemAltLowColor}\" TintOpacity=\"0.4\" />",
   L"ElementBackground2=<WindhawkBlur BlurAmount=\"20\" TintColor=\"{ThemeResource SystemAltLowColor}\" TintOpacity=\"0.2\" />",
   L"AccentBackground=<WindhawkBlur BlurAmount=\"15\" TintColor=\"{ThemeResource SystemAccentColorLight1}\" TintOpacity=\"0.2\" />",
   L"BorderBrush=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#50808080\" Offset=\"0.0\" /><GradientStop Color=\"#50404040\" Offset=\"0.25\" /><GradientStop Color=\"#50808080\" Offset=\"1\" /></LinearGradientBrush>",
   L"ElementBorderBrush=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#50808080\" Offset=\"1\" /><GradientStop Color=\"#50606060\" Offset=\"0.15\" /></LinearGradientBrush>",
   L"BorderThickness=0.3,1,0.3,0.3",
   L"ElementBorderThickness=0.3,0.3,0.3,1",
   L"CornerRadius=12",
   L"ElementCornerRadius=8",
}};

const Theme g_themeBorderless = {{
   ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > Grid", {
      L"ColumnDefinitions:=<ColumnDefinitionCollection><ColumnDefinition Width=\"*\" /><ColumnDefinition Width=\"Auto\" /><ColumnDefinition Width=\"Auto\" /><ColumnDefinition Width=\"*\" /></ColumnDefinitionCollection>",
      L"HorizontalAlignment=Stretch"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid@DockingStates", {
      L"Tag=horizontal",
      L"Tag@DockedLeft=vertical",
      L"Tag@DockedRight=vertical",
      L"Tag=>taskbarDock"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
      L"Grid.Column=1",
      L"Width={{taskbarDock==`vertical`?skip():$TaskbarFrameWidth}}",
      L"Margin=0",
      L"MaxWidth=$TaskbarFrameWidth"}},
   ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
      L"Grid.Column=1",
      L"Width=Auto",
      L"HorizontalAlignment=Right",
      L"Height=40",
      L"VerticalAlignment=Center"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Windows.UI.Xaml.Controls.Grid#RootGrid > Taskbar.TaskbarBackground#BackgroundControl", {
      L"CornerRadius=6",
      L"Margin=0,2,0,2"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#BackgroundStroke", {
      L"Visibility=Collapsed"}},
   ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater", {
      L"Margin=0,4,0,4"}},
```

```
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#OverflowFlyoutBackgroundBorder",
{
    L"Shadow:=",
    L"BorderThickness:="}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#ConfirmatorMainGrid", {
    L"Shadow:=",
    L"BorderThickness:="}},
  ThemeTargetStyles{L"Taskbar.OverflowToggleButton#OverflowButton >
Taskbar.TaskListButtonPanel#OverflowToggleButtonRootPanel >
Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.FontIcon#FontIcon >
Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.TextBlock,
Taskbar.OverflowToggleButton#OverflowButton >
Taskbar.TaskListButtonPanel#OverflowToggleButtonRootPanel >
Windows.UI.Xaml.Controls.FontIcon > Windows.UI.Xaml.Controls.Grid >
Windows.UI.Xaml.Controls.TextBlock", {
    L"Text=\uE8F9"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#MostRecentlyUsedDivider", {
    L"Height=32",
    L"Width=2"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#LeftOverflowButtonDivider", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#RightOverflowButtonDivider", {
    L"Visibility=Collapsed"}},

ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#OverflowFlyoutListRepeater", {
    L"Height=48"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputS
witcher > ContentControl > ContentPresenter > Grid > Grid", {
    L"Background:="}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputS
witcher > ContentControl > ContentPresenter > Grid", {
    L"Margin=390,0,574,12",
    L"Shadow:=",
    L"BorderThickness:="}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#HoverFlyoutBackground", {
    L"Shadow:="}},
}, {
  L"TaskbarFrameWidth=800",
}};

const Theme g_themeCommand_Center = {{
  ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground >
Grid > Rectangle#BackgroundFill", {
    L"Visibility=1"}},
  ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
    L"Visibility=1"}},
  ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl", {
    L"Background:=$Background",
    L"BorderBrush:=$BorderBrush",
    L"BorderThickness=$BorderThickness",
    L"CornerRadius=$R1"}},
  ThemeTargetStyles{L"Taskbar.TaskListButtonPanel@CommonStates > Grid >
Border#BackgroundElement, Taskbar.TaskListButtonPanel@CommonStates >
Border#BackgroundElement", {
    L"Padding=6",
    L"Margin=2",
    L"Background@ActiveNormal:=$ActiveColor",
    L"Background@ActivePointerOver:=$OverlayColor",
    L"Background@ActivePressed:=$OverlayColor",
    L"Background@InactiveNormal:=$Background",
```

```
        L"Background@InactivePointerOver:=$OverlayColor2",
        L"Background@InactivePressed:=$OverlayColor",
        L"Background@MultiWindowNormal:=$Background",
        L"Background@MultiWindowPointerOver:=$OverlayColor2",
        L"Background@MultiWindowActive:=$ActiveColor",
        L"Background@MultiWindowPressed:=$OverlayColor",
        L"CornerRadius=$R1"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness=$BorderThickness",
        L"CornerRadius=$R1",
        L"Width=38",
        L"Height=38"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Grid >
Border#BackgroundElement, SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Border#BackgroundElement", {
        L"Visibility=1"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border#BackgroundElement,
Taskbar.TaskListLabeledButtonPanel@CommonStates > Border#BackgroundElement", {
        L"Width=38",
        L"Height=38",
        L"Background@ActiveNormal:=$ActiveColor",
        L"Background@ActivePointerOver:=$OverlayColor",
        L"Background@ActivePressed:=$OverlayColor",
        L"Background@InactiveNormal:=$Background",
        L"Background@InactivePointerOver:=$OverlayColor2",
        L"Background@InactivePressed:=$OverlayColor",
        L"Background@MultiWindowNormal:=$Background",
        L"Background@MultiWindowPointerOver:=$OverlayColor2",
        L"Background@MultiWindowActive:=$ActiveColor",
        L"Background@MultiWindowPressed:=$OverlayColor",
        L"CornerRadius=$R1"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Rectangle#RunningIndicator,
Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Rectangle#RunningIndicator", {
        L"RadiusX=1.5",
        L"RadiusY=1.5",
        L"Height=4",
        L"Width=12",
        L"Fill:=$ActiveColor",
        L"Fill@ActiveRunningIndicator:=$AccentColor",
        L"Width@ActiveRunningIndicator=21"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"Height=38",
        L"Background:=Transparent",
        L"BorderBrush:=Transparent",
        L"BorderThickness=0",
        L"Shadow:="}},
    ThemeTargetStyles{L"Grid#OverflowRootGrid > Border", {
        L"Padding=6",
        L"Background:=Transparent",
        L"BorderBrush:=Transparent",
        L"BorderThickness=0",
        L"Shadow:="}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalTrackRect", {
```

```
        L"Height=8",
        L"Margin=0",
        L"Fill:=$OverlayColor"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalDecreaseRect", {
        L"Height=8"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.TextBlock#volumeLevelText", {
        L"Margin=0,-2,0,0"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#VolumeConfirmator", {
        L"Padding=8,0,3,0",
        L"CornerRadius=$R2"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#ConfirmatorMainGrid", {
        L"Background:=Transparent",
        L"BorderBrush:=Transparent",
        L"BorderThickness=0",
        L"Shadow:="}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#BrightnessConfirmator", {
        L"Padding=25,0,17,0",
        L"CornerRadius=$R2"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.AnimatedIcon#BrightnessIcon", {
        L"Margin=0,-1,12,0"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ProgressBar#ProgressIndicator", {
        L"Margin=0,0,0,1"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#ProgressBarTrack", {
        L"Fill:=$OverlayColor",
        L"RadiusX=1.5",
        L"RadiusY=1.5"}},

ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#DeterminateProgressBarIndicator", {
        L"Fill:=$AccentColor"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton > Grid#IconPanel >
Microsoft.UI.Xaml.Controls.ProgressBar#ProgressIndicator,
Taskbar.TaskListButton#TaskListButton > Taskbar.TaskListLabeledButtonPanel#IconPanel >
Microsoft.UI.Xaml.Controls.ProgressBar#ProgressIndicator", {
        L"MinHeight=4",
        L"Width=26"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.ContentPresenter#ContentPresenter", {
        L"Padding=6",
        L"Margin=2",
        L"BorderThickness=$BorderThickness",
        L"BorderBrush:=$BorderBrush",
        L"Background:=$Background",
        L"CornerRadius=$R1",
        L"Shadow:="}},
    ThemeTargetStyles{L"Taskbar.Badge#BadgeControl", {
        L"Height=14",
        L"MinWidth=14",
        L"Margin=0,0,0,0",
        L"CornerRadius=$R2"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#BackgroundRect", {
        L"RadiusX=4",
        L"RadiusY=4"}},
    ThemeTargetStyles{L"MenuFlyoutPresenter > Border", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness=$BorderThickness",
        L"CornerRadius=$R1"}},
```

```
  ThemeTargetStyles{L"MenuFlyoutSubItem", {
    L"Background:=$Background",
    L"BorderBrush:=$BorderBrush",
    L"BorderThickness=$BorderThickness",
    L"CornerRadius=$R1"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalTrackRect", {
    L"Fill:=$Background"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalDecreaseRect", {
    L"Fill:=$AccentColor"}},
  ThemeTargetStyles{L"Grid#HoverFlyoutGrid > Border#HoverFlyoutBackground", {
    L"Background:=$Background",
    L"BorderBrush:=$BorderBrush",
    L"BorderThickness=$BorderThickness",
    L"CornerRadius=$R1"}},
  ThemeTargetStyles{L"Taskbar.TaskbarExtensionElement", {
    L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"0\" />"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapBarBorder", {
    L"Background:=$Background",
    L"BorderBrush:=$BorderBrush",
    L"CornerRadius=$R1",
    L"BorderThickness=$BorderThickness"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapPickerBorder", {
    L"Background:=$Background",
    L"BorderBrush:=$BorderBrush",
    L"CornerRadius=$R1",
    L"BorderThickness=$BorderThickness"}},
  ThemeTargetStyles{L"SystemTray.DateTimeIconContent", {
    L"FontSize=9"}},
  ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
    L"Visibility=1"}},
  ThemeTargetStyles{L"Border#VirtualDesktopBarBackground", {
    L"Background:=$Background",
    L"BorderThickness=$BorderThickness",
    L"BorderBrush:=$BorderBrush",
    L"CornerRadius=$R1"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopElementThemed", {
    L"CornerRadius=$R1"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.NewVirtualDesktopElementThemed", {
    L"CornerRadius=$R1"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapBarBorder", {
    L"Background:=$Background",
    L"BorderThickness=$BorderThickness",
    L"BorderBrush:=$BorderBrush",
    L"CornerRadius=$R1"}},
  ThemeTargetStyles{L"SnapLayout.SnapLayoutPickerControl", {
    L"Background:=$Background",
    L"BorderThickness=$BorderThickness",
    L"BorderBrush:=$BorderBrush",
    L"CornerRadius=$R1"}},
  ThemeTargetStyles{L"SnapLayout.SnapLayoutControl", {
    L"Background:=$Background",
    L"BorderThickness=$BorderThickness",
```

```
        L"BorderBrush:=$BorderBrush",
        L"CornerRadius=$R1"}},
    ThemeTargetStyles{L"SnapLayout.SnapLayoutControl@CommonStates > Windows.UI.Xaml.Controls.Border#LayoutBorder", {
        L"Background@PointerOver:=$AccentColor",
        L"Background@Pressed:=$Background"}},
    ThemeTargetStyles{L"Grid#ConfirmatorMainGrid", {
        L"Background:=$Background",
        L"BorderThickness=$BorderThickness",
        L"BorderBrush:=$BorderBrush",
        L"CornerRadius=$R1",
        L"Height=20"}},

ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel", {
        L"Background:=$Background",
        L"BorderThickness=$BorderThickness",
        L"BorderBrush:=$BorderBrush",
        L"CornerRadius=$R1",
        L"Margin=0,6"}},

ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Grid > Border#BackgroundElement, SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Border#BackgroundElement", {
        L"Background:=Transparent",
        L"BorderBrush:=Transparent",
        L"BorderThickness=0",
        L"CornerRadius=$R1",
        L"Margin=4,-5,6,-5"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"0\" />"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchPillButton#SearchPill > SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel", {
        L"BorderBrush:=Transparent",
        L"BorderThickness=0",
        L"CornerRadius=$R1",
        L"MaxWidth=150",
        L"MaxHeight=38",
        L"Width=Auto",
        L"Height=Auto"}},

ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Grid > Border#SearchPillBackgroundElement, SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Border#SearchPillBackgroundElement", {
        L"BorderBrush:=$BorderBrush",
        L"CornerRadius=$R1",
        L"BorderThickness=$BorderThickness",
        L"MaxWidth=150",
        L"MaxHeight=38",
        L"Width=Auto",
        L"Height=Auto",
        L"Margin=-4,-5,-6,-5"}},
    ThemeTargetStyles{L"Button#GleamEntryPointButton > Border > ContentPresenter#ContentPresenter", {
        L"Background:=Transparent",
        L"BorderBrush:=Transparent",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"Grid#DynamicSearchBoxGleamContainer", {
        L"Visibility=1"}},
```

```
    ThemeTargetStyles{L"TextBlock#InnerTextBlock[Text=\uE971]", {
        L"Text=\uED14"}},
}, {
    L"Translucent=<WindhawkBlur BlurAmount=\"15\" TintColor=\"#10808080\"/>",
    L"Glass=<WindhawkBlur BlurAmount=\"5\" TintColor=\"{ThemeResource
SystemChromeMediumColor}\" TintOpacity=\"0.7\" />",
    L"Frosted=<WindhawkBlur BlurAmount=\"20\" TintColor=\"{ThemeResource
SystemChromeMediumColor}\" TintOpacity=\"0.7\" />",
    L"Acrylic=<WindhawkBlur BlurAmount=\"30\" TintColor=\"{ThemeResource
SystemChromeMediumColor}\" TintOpacity=\"0.8\" />",
    L"Background=$Frosted",
    L"OverlayColor=<AcrylicBrush TintColor=\"{ThemeResource SystemAltLowColor}\"
TintOpacity=\"1\" TintLuminosityOpacity=\"0.8\" FallbackColor=\"{ThemeResource
CardStrokeColorDefaultSolid}\" />",
    L"OverlayColor2=<AcrylicBrush TintColor=\"{ThemeResource SystemAltLowColor}\"
TintOpacity=\"1\" TintLuminosityOpacity=\"0.5\" FallbackColor=\"{ThemeResource
CardStrokeColorDefaultSolid}\" />",
    L"AccentColor=<AcrylicBrush TintColor=\"{ThemeResource SystemAccentColor}\"
TintOpacity=\"1\" TintLuminosityOpacity=\"0.5\" FallbackColor=\"{ThemeResource
CardStrokeColorDefaultSolid}\" />",
    L"ActiveColor=<AcrylicBrush TintColor=\"{ThemeResource SystemAltLowColor}\"
TintOpacity=\"1\" TintLuminosityOpacity=\"1\" FallbackColor=\"{ThemeResource
CardStrokeColorDefaultSolid}\" />",
    L"BorderBrush=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop
Color=\"#60808080\" Offset=\"0.0\" /><GradientStop Color=\"#50404040\" Offset=\"0.25\"
/><GradientStop Color=\"#40808080\" Offset=\"1\" /></LinearGradientBrush>",
    L"BorderThickness=0.3,1,0.3,1",
    L"R1=6",
    L"R2=20",
}};

const Theme g_themeFluentGlass = {{
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > Grid", {
        L"ColumnDefinitions:=<ColumnDefinitionCollection><ColumnDefinition
Width=\"*\"/><ColumnDefinition Width=\"Auto\"/><ColumnDefinition
Width=\"Auto\"/><ColumnDefinition Width=\"*\"/></ColumnDefinitionCollection>",
        L"HorizontalAlignment=Stretch"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid@DockingStates", {
        L"Tag=horizontal",
        L"Tag@DockedLeft=vertical",
        L"Tag@DockedRight=vertical",
        L"Tag=>taskbarDock"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
        L"Grid.Column=1",
        L"Width={{taskbarDock==`vertical`?skip():$TaskbarFrameWidth}}",
        L"Height={{taskbarDock==`vertical`?skip():$TaskbarHeight}}",
        L"MinHeight=62",
        L"HorizontalAlignment=Center",
        L"VerticalAlignment=Center",
        L"Margin=0,0,0,12",
        L"Padding=0"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#BackgroundControl", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Grid#RootGrid", {
        L"Background:=$LiquidBackground",
        L"BorderBrush:=$LiquidBorder",
        L"BorderThickness=0.5,1,0.5,1",
        L"CornerRadius=$GlobalRadius",
        L"Padding=0"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
        L"Grid.Column=1",
```

```
        L"HorizontalAlignment=Right",
        L"VerticalAlignment=Center",
        L"Height=$TaskbarHeight",
        L"MinWidth=250",
        L"Margin=0,0,0,12"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"Background=Transparent",
        L"BorderThickness=0",
        L"VerticalAlignment=Center",
        L"CornerRadius=$GlobalRadius",
        L"Height=50",
        L"Margin=0"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayStack", {
        L"VerticalAlignment=Center"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton", {
        L"Margin=$ItemMargin",
        L"Padding=$ItemPadding",
        L"VerticalAlignment=Center",
        L"Height=$IconHeight"}},
    ThemeTargetStyles{L"Taskbar.StartButton", {
        L"Margin=$ItemMargin",
        L"Padding=$ItemPadding",
        L"VerticalAlignment=Center",
        L"Height=$IconHeight"}},
    ThemeTargetStyles{L"Taskbar.SearchBoxButton", {
        L"Margin=$ItemMargin",
        L"Padding=$ItemPadding",
        L"VerticalAlignment=Center",
        L"Height=$IconHeight"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl", {
        L"Margin=$ItemMargin",
        L"Padding=$ItemPadding",
        L"VerticalAlignment=Center",
        L"Height=$IconHeight"}},
    ThemeTargetStyles{L"Taskbar.TaskViewButton", {
        L"Margin=$ItemMargin",
        L"Padding=$ItemPadding",
        L"VerticalAlignment=Center",
        L"Height=$IconHeight"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel,
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel", {
        L"VerticalAlignment=Center"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border#BackgroundElement,
Taskbar.TaskListLabeledButtonPanel@CommonStates > Border#BackgroundElement", {
        L"CornerRadius=$GlobalRadius",
        L"BorderThickness=0",
        L"VerticalAlignment=Stretch",
        L"HorizontalAlignment=Stretch",
        L"Background@PointerOver:=<WindhawkBlur BlurAmount=\"10\"
TintColor=\"{ThemeResource SystemAccentColor}\" TintOpacity=\"0.18\" />",
        L"Background@Pressed:=<WindhawkBlur BlurAmount=\"15\"
TintColor=\"{ThemeResource SystemAccentColor}\" TintOpacity=\"0.25\" />",
        L"Background@ActiveNormal:=<WindhawkBlur BlurAmount=\"4\"
TintColor=\"{ThemeResource SystemAccentColor}\" TintOpacity=\"0.32\" />",
        L"Background@ActivePointerOver:=<WindhawkBlur BlurAmount=\"4\"
TintColor=\"{ThemeResource SystemAccentColor}\" TintOpacity=\"0.4\" />"}},
    ThemeTargetStyles{L"Rectangle#RunningIndicator", {
```

```
      L"Visibility=Visible",
      L"Fill={ThemeResource SystemAccentColor}",
      L"Height=4",
      L"Width=30",
      L"RadiusX=2",
      L"RadiusY=2",
      L"VerticalAlignment=Bottom"}},
   ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater", {
      L"VerticalAlignment=Center"}},
   ThemeTargetStyles{L"Grid#AugmentedEntryPointContentGrid", {
      L"Margin=5,0,0,0",
      L"VerticalAlignment=Center"}},
}, {
   L"TaskbarFrameWidth=1280",
   L"TaskbarHeight=62",
   L"IconHeight=54",
   L"GlobalRadius=8",
   L"ItemMargin=4,2,4,2",
   L"ItemPadding=8,6,8,6",
   L"LiquidBackground=<WindhawkBlur BlurAmount=\"22\" TintColor=\"{ThemeResource
SystemAltLowColor}\" TintOpacity=\"0.15\" />",
   L"LiquidBorder=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop
Color=\"#50808080\" Offset=\"0.0\" /><GradientStop Color=\"#50404040\" Offset=\"1\"
/></LinearGradientBrush>",
}};

const Theme g_themeOS26_Liquid_Glass_variant_ClearMacDock = {{
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid@DockingStates", {
      L"Tag=horizontal",
      L"Tag@DockedLeft=vertical",
      L"Tag@DockedRight=vertical",
      L"Tag=>taskbarDock"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
      L"Width={{taskbarDock==`vertical`?skip():`Auto`}}",
      L"MinWidth:=100",
      L"MaxWidth={{containerGridWidth>0?containerGridWidth:`Infinity`}}",
      L"Grid.Column=1",
      L"Transitions:=<TransitionCollection><RepositionThemeTransition
IsStaggeringEnabled=\"False\"/></TransitionCollection>",
      L"Height={{taskbarDock==`vertical`?skip():80}}",
      L"MaxHeight=80",
      L"HorizontalAlignment=Center"}},
   ThemeTargetStyles{L"Grid#IconPanel > Image#Icon,
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
      L"Height=24",
      L"Width=24",
      L"Margin=5,0,-5,0"}},
   ThemeTargetStyles{L"Taskbar.TaskListButtonPanel", {
      L"Width=55",
      L"Height=70"}},
   ThemeTargetStyles{L"Grid#IconPanel, Taskbar.TaskListLabeledButtonPanel", {
      L"Width=55",
      L"Height=70"}},
   ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl > Grid >
SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel", {
      L"Width=55",
      L"Height=70"}},
```

```
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
      L"Margin=0,8,0,2",
      L"Padding=20,0,20,0",
      L"BorderBrush=#40FFFFFF"}},
    ThemeTargetStyles{L"Grid#RootGrid > Taskbar.TaskbarBackground", {
      L"Margin=-20,0,-20,0"}},
    ThemeTargetStyles{L"Grid#RootGrid > Taskbar.TaskbarBackground > Grid", {
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"CornerRadius=20,0,0,20",
      L"BorderThickness=1,1,0,1",
      L"Width=Auto",
      L"BorderBrush=#40FFFFFF",
      L"Padding=-1"}},
    ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
      L"Fill=Transparent"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.FlyoutPresenter", {
      L"RequestedTheme=Dark",
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"BorderThickness=2",
      L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
      L"CornerRadius=33",
      L"Padding=2,3,2,3"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapPickerBorder", {
      L"RequestedTheme=Dark",
      L"Background:=Transparent",
      L"BorderBrush:=Transparent",
      L"BorderThickness=2",
      L"Margin=0"}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab >
Grid#ModalRootGrid > Border#BackgroundElement", {
      L"Background=Transparent",
      L"BorderThickness=0",
      L"CornerRadius=50"}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab >
Grid#ModalRootGrid > Border#BackgroundElement >
WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemList", {
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"BorderThickness=2",
      L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
      L"CornerRadius=50"}},
    ThemeTargetStyles{L"MenuFlyoutPresenter", {
      L"CornerRadius=20"}},
    ThemeTargetStyles{L"MenuFlyoutPresenter > Border", {
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"BorderThickness=2",
      L"CornerRadius=25",
      L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid >
SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView >
Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid,
SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid >
SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView >
Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid", {
      L"Height=48",
      L"Width=48",
      L"Margin=25,2,12,0",
      L"CornerRadius=15",
```

```
        L"Background=transparent",
        L"BorderBrush:=lightgray",
        L"BorderThickness=3"}},
    ThemeTargetStyles{L"SystemTray.LanguageTextIconContent > Windows.UI.Xaml.Controls.Grid#ContainerGrid", {
        L"Width=48",
        L"Height=48",
        L"Margin=-6,4,-3,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2",
        L"CornerRadius=15"}},
    ThemeTargetStyles{L"SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid,
SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid", {
        L"Width=48",
        L"Height=48",
        L"Margin=-3,4,8,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2",
        L"CornerRadius=15"}},
    ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack", {
        L"Visibility=1",
        L"Width=48",
        L"Height=48",
        L"Margin=2,4,2,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2",
        L"CornerRadius=15"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[1] > SystemTray.IconView > Grid > Grid", {
        L"Visibility=Visible",
        L"Width=48",
        L"Height=48",
        L"Margin=2,4,2,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2",
        L"CornerRadius=15"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[2] > SystemTray.IconView > Grid > Grid", {
        L"Visibility=Visible",
        L"Width=48",
        L"Height=48",
        L"Margin=2,4,2,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2",
        L"CornerRadius=15"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[3] > SystemTray.IconView > Grid > Grid", {
```

```
        L"Visibility=Visible",
        L"Width=48",
        L"Height=48",
        L"Margin=2,4,2,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2",
        L"CornerRadius=15"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.BatteryIconContent > Grid#ContainerGrid > StackPanel, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.BatteryIconContent > Grid#ContainerGrid > StackPanel", {
        L"HorizontalAlignment=Center"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock > TextBlock", {
        L"FontSize=30"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock", {
        L"FontSize=32"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock", {
        L"VerticalAlignment=Center",
        L"HorizontalAlignment=Center"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.OmniButton#NotificationCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.OmniButton#NotificationCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock#LanguageInnerTextBlock > TextBlock#InnerTextBlock", {
        L"FontSize=16",
        L"FontWeight=Bold"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent", {
        L"FontWeight=bold",
        L"FontSize=32"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
        L"Height=80",
        L"Grid.Column=2",
        L"Width=Auto",
        L"HorizontalAlignment=Left",
        L"Margin=0,-2,0.5,2"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"Margin=0,0,0.5,0",
        L"Height=70",
```

```
        L"VerticalAlignment=Bottom",
        L"Padding=0",
        L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
        L"BorderBrush=#40FFFFFF",
        L"BorderThickness=0,1,1,1",
        L"CornerRadius=0,20,20,0",
        L"Visibility=Visible"}},
    ThemeTargetStyles{L"SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView > Grid#ContainerGrid > Border#BackgroundBorder", {
        L"Background=transparent",
        L"BorderBrush=#40FFFFFF",
        L"BorderThickness=2,0,0,0",
        L"Padding=0",
        L"CornerRadius=0",
        L"Height=40"}},
    ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
        L"FontWeight=Bold",
        L"Margin=-2,9,2,-9",
        L"Foreground=White",
        L"Visibility=visible",
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"-9\"/>",
        L"FontSize=13",
        L"FontFamily=vivo Sans EN VF"}},
    ThemeTargetStyles{L"TextBlock#TimeInnerTextBlock", {
        L"Foreground=white",
        L"Width=Auto",
        L"FontWeight=Bold",
        L"FontSize=15",
        L"Margin=0,-4,5,4"}},
    ThemeTargetStyles{L"SystemTray.DateTimeIconContent > Grid#ContainerGrid", {
        L"CornerRadius=12",
        L"BorderThickness=1.2",
        L"Background:=<LinearGradientBrush StartPoint=\"0.50,-0.77\" EndPoint=\"0.49,1.77\"><GradientStop Offset=\"0.46\" Color=\"#52A3A3A3\"/><GradientStop Offset=\"0.5\" Color=\"#33000000\"/></LinearGradientBrush>",
        L"BorderBrush:=$IconBorder",
        L"Height=45",
        L"Margin=0,3,15,-1",
        L"Width=auto"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock", {
        L"FontSize=30"}},
    ThemeTargetStyles{L":root > ScrollViewer > ScrollContentPresenter > Border > Grid", {
        L"ColumnDefinitions:=<ColumnDefinitionCollection><ColumnDefinition Width=\"*\"/><ColumnDefinition Width=\"Auto\"/><ColumnDefinition Width=\"Auto\"/><ColumnDefinition Width=\"*\"/></ColumnDefinitionCollection>",
        L"ActualWidth=>containerGridWidth"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.Stack#NonActivatableStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Border#BackgroundBorder, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.Stack#NonActivatableStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Border#BackgroundBorder", {
        L"Background:=Transparent",
```

```
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid >
Border#BackgroundBorder", {
        L"Background:=Transparent",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"SystemTray.Stack#MainStack", {
        L"Visibility=1"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid >
SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon >
Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid >
SystemTray.AdaptiveTextBlock#Underlay > TextBlock#InnerTextBlock,
SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid >
SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon >
Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid >
SystemTray.AdaptiveTextBlock#Underlay > TextBlock#InnerTextBlock", {
        L"Foreground=white"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel@CommonStates > Grid >
Border#BackgroundElement, Taskbar.TaskListButtonPanel@CommonStates >
Border#BackgroundElement", {
        L"CornerRadius=15",
        L"Margin=0,5.5,0,5.5",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid >
Rectangle#BackgroundStroke", {
        L"Fill:=<WindhawkBlur BlurAmount=\"3.5\" TintColor=\"#2D101010\"/>",
        L"Stroke:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
        L"StrokeThickness=5",
        L"RadiusX=14",
        L"RadiusY=14",
        L"Fill:=<<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid >
Rectangle#BackgroundFill", {
        L"Canvas.ZIndex=0",
        L"Fill:=",
        L"Stroke:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
        L"StrokeThickness=5",
        L"RadiusX=14",
        L"RadiusY=14"}},
    ThemeTargetStyles{L"Taskbar.FlyoutFrame >
Windows.UI.Xaml.Controls.Canvas#HoverFlyoutCanvas >
Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid >
Windows.UI.Xaml.Controls.ContentPresenter#HoverFlyoutContent >
Taskbar.TaskItemThumbnailList >
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskItemThumbnailListRepeater >
Taskbar.TaskItemThumbnailView > Windows.UI.Xaml.Controls.Grid >
Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"VerticalAlignment=Bottom",
        L"Canvas.ZIndex=1",
        L"Background:=<WindhawkBlur BlurAmount=\"5\" TintColor=\"#761E1E1E\"/>",
        L"Height=25",
        L"CornerRadius=0,0,15,15",
        L"Margin=5,0,5,0"}},
    ThemeTargetStyles{L"Border#HoverFlyoutBackground", {
        L"Margin=4,36,4,0",
        L"Canvas.ZIndex=1",
        L"Width=Auto",
        L"Background:=Transparent",
        L"BorderThickness=0",
        L"CornerRadius=15"}},
```

```
  ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#ThumbBarRepeater > Taskbar.ThumbBarButton#ThumbBarButton > Windows.UI.Xaml.Controls.ContentPresenter#BorderElement", {
    L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#761E1E1E\"/>",
    L"Margin=0,-20,0,20"}},
  ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#IconsRepeater > Windows.UI.Xaml.Controls.Image", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#CloseButton", {
    L"HorizontalAlignment=left",
    L"Grid.ColumnSpan=1",
    L"Grid.RowSpan=1",
    L"Canvas.ZIndex=1",
    L"CornerRadius=20",
    L"Width=28",
    L"Height=28",
    L"Margin=-18,40,15,-40",
    L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#80ffffff\"/>",
    L"Foreground=black",
    L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#761E1E1E\"/>"}},
  ThemeTargetStyles{L"Taskbar.FlyoutFrame > Windows.UI.Xaml.Controls.Canvas#HoverFlyoutCanvas > Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid > Windows.UI.Xaml.Controls.ContentPresenter#HoverFlyoutContent > Taskbar.TaskItemThumbnailList > Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskItemThumbnailListRepeater > Taskbar.TaskItemThumbnailView > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.TextBlock#DisplayNameTextBlock", {
    L"Grid.ColumnSpan=2",
    L"Grid.RowSpan=2",
    L"VerticalAlignment=bottom",
    L"HorizontalAlignment=Center",
    L"Margin=0,-5,0,5",
    L"Canvas.ZIndex=1"}},
  ThemeTargetStyles{L"SystemTray.NotifyIconView@CommonStates > Grid#ContainerGrid > Border#BackgroundBorder", {
    L"CornerRadius=12",
    L"Background:=$IconBackground",
    L"BorderBrush:=$IconBorder",
    L"Margin=2",
    L"BorderThickness=1.2"}},
  ThemeTargetStyles{L"Border#OverflowFlyoutBackgroundBorder", {
    L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
    L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#60ffffff\"/>",
    L"BorderThickness=2",
    L"CornerRadius=32,32,30,30",
    L"Margin=-10"}},
  ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Rectangle#RunningIndicator, Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Rectangle#RunningIndicator", {
    L"Fill:=#90ffffff",
    L"RadiusX=3",
    L"RadiusY=3",
    L"Margin=-2",
    L"Height=6",
    L"Width=6",
    L"Margin=10,0,0,-2",
    L"Width@ActiveRunningIndicator=12",
    L"Fill@ActiveRunningIndicator=#60CDFF"}},
  ThemeTargetStyles{L"Grid#IconPanel > TextBlock#LabelControl, Taskbar.TaskListLabeledButtonPanel > TextBlock#LabelControl", {
```

```
        L"Margin=4,0,0,0",
        L"Foreground=White"}},
    ThemeTargetStyles{L"Taskbar.SearchBoxButton", {
        L"Background:=<WindhawkBlur BlurAmount=\"60\" TintColor=\"#35ffffff\"/>",
        L"CornerRadius=20",
        L"Margin=2,6,2,6",
        L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,1\" StartPoint=\"0,0\"><GradientStop
Color=\"#E0ffffff\" Offset=\"0.0\"/><GradientStop Color=\"#20ffffff\"
Offset=\"0.5\"/><GradientStop Color=\"#A0ffffff\" Offset=\"1.0\"/></LinearGradientBrush>",
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"TextBlock#SearchBoxTextBlock", {
        L"FontSize=12",
        L"Foreground=White"}},
    ThemeTargetStyles{L"Grid", {
        L"RequestedTheme=2"}},
    ThemeTargetStyles{L"Taskbar.StartButton#StartButton", {
        L"Background:=<WindhawkBlur BlurAmount=\"60\" TintColor=\"#35ffffff\"/>",
        L"CornerRadius=20",
        L"Margin=2,6,2,6",
        L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,1\" StartPoint=\"0,0\"><GradientStop
Color=\"#E0ffffff\" Offset=\"0.0\"/><GradientStop Color=\"#20ffffff\"
Offset=\"0.5\"/><GradientStop Color=\"#A0ffffff\" Offset=\"1.0\"/></LinearGradientBrush>",
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"Border#MultiWindowElement", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"SystemTray.TextIconContent > Grid >
SystemTray.AdaptiveTextBlock#Base > TextBlock", {
        L"Foreground=White"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton", {
        L"Margin=-12,0,0,0"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl > Grid >
SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Grid >
Border#BackgroundElement, SearchUx.SearchUI.SearchButtonControl > Grid >
SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Border#BackgroundElement", {
        L"Margin=0,5.5,0,5.5",
        L"CornerRadius=15",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.
Name=Task View]", {
        L"Background:=<WindhawkBlur BlurAmount=\"60\" TintColor=\"#35ffffff\"/>",
        L"CornerRadius=20",
        L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,1\" StartPoint=\"0,0\"><GradientStop
Color=\"#E0ffffff\" Offset=\"0.0\"/><GradientStop Color=\"#20ffffff\"
Offset=\"0.5\"/><GradientStop Color=\"#A0ffffff\" Offset=\"1.0\"/></LinearGradientBrush>",
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Border,
Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Border", {
        L"Background@InactiveRunningIndicatorPointerOver:=<WindhawkBlur BlurAmount=\"40\"
TintColor=\"#10ffffff\"/>",
        L"CornerRadius=12",
        L"BorderBrush@InactiveRunningIndicatorPointerOver:=<LinearGradientBrush
EndPoint=\"1,0\" StartPoint=\"0,0\"><GradientStop Color=\"#80ffffff\"
Offset=\"0.0\"/><GradientStop Color=\"{ThemeResource SurfaceStrokeColorDefault}\"
Offset=\"0.55\"/><GradientStop Color=\"#80ffffff\" Offset=\"1\"/></LinearGradientBrush>",
        L"BorderThickness@InactiveRunningIndicatorPointerOver=1"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border#BackgroundElement,
Taskbar.TaskListLabeledButtonPanel@CommonStates > Border#BackgroundElement", {
        L"CornerRadius=15",
        L"Margin=0,5.5,0,5.5",
```

```
      L"Background:=$IconBackground",
      L"BorderBrush:=$IconBorder",
      L"BorderThickness=1.2"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground > Grid > Rectangle#BackgroundFill", {
      L"Fill=Transparent"}},
   ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon", {
      L"Background:=<WindhawkBlur BlurAmount=\"10\" TintColor=\"#40ffffff\"/>",
      L"CornerRadius=12",
      L"Margin=2",
      L"Padding=2",
      L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,0\" StartPoint=\"0,0\"><GradientStop Color=\"#80ffffff\" Offset=\"0.0\"/><GradientStop Color=\"{ThemeResource SurfaceStrokeColorDefault}\" Offset=\"0.55\"/><GradientStop Color=\"#80ffffff\" Offset=\"1\"/></LinearGradientBrush>",
      L"BorderThickness=2"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#ConfirmatorMainGrid", {
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"CornerRadius=24",
      L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
      L"BorderThickness=2",
      L"Margin=0,0,0,10"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid.Border#ConfirmatorMainGrid", {
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalTrackRect", {
      L"Fill=#20ffffff",
      L"RadiusX=12",
      L"RadiusY=12",
      L"Height=18",
      L"Margin=0"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalDecreaseRect", {
      L"Fill:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColor}\" />",
      L"RadiusX=12",
      L"RadiusY=12",
      L"Height=18"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#VolumeConfirmator", {
      L"Padding=8,0,8,0"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#BrightnessConfirmator", {
      L"Padding=8,0,8,0"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.TextBlock#volumeLevelText", {
      L"Foreground=White"}},
}, {
   L"IconBackground=<ImageBrush ImageSource=\"https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/OS26%20Liquid%20Glass/Assets/tahoeappbg.png\" Stretch=\"UniformtoFill\"/>",
   L"IconBorder=<LinearGradientBrush EndPoint=\"1,1\" StartPoint=\"0,0\"><GradientStop Color=\"#F5ffffff\" Offset=\"0.0\"/><GradientStop Color=\"#40ffffff\" Offset=\"0.4\"/><GradientStop Color=\"#20ffffff\" Offset=\"0.6\"/><GradientStop Color=\"#90ffffff\" Offset=\"1.0\"/></LinearGradientBrush>",
}};

const Theme g_themeOS26_Liquid_Glass_variant_ClearMacDockCompact = {{
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid@DockingStates", {
      L"Tag=horizontal",
      L"Tag@DockedLeft=vertical",
      L"Tag@DockedRight=vertical",
      L"Tag=>taskbarDock"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
```

```
      L"Width={{taskbarDock==`vertical`?skip():`Auto`}}",
      L"MinWidth:=100",
      L"MaxWidth={{containerGridWidth>0?containerGridWidth:`Infinity`}}",
      L"Grid.Column=1",
      L"Transitions:=<TransitionCollection><RepositionThemeTransition
IsStaggeringEnabled=\"False\"/></TransitionCollection>",
      L"Height={{taskbarDock==`vertical`?skip():70}}",
      L"MaxHeight=80",
      L"HorizontalAlignment=Center"}},
    ThemeTargetStyles{L"Grid#IconPanel > Image#Icon,
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
      L"Height=24",
      L"Width=24",
      L"Margin=3,0,-3,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel", {
      L"Width=50",
      L"Height=65",
      L"Margin=0,-2,0,2"}},
    ThemeTargetStyles{L"Grid#IconPanel, Taskbar.TaskListLabeledButtonPanel", {
      L"Width=50",
      L"Height=65",
      L"Margin=0,-2,0,2"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl > Grid >
SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel", {
      L"Width=50",
      L"Height=65",
      L"Margin=0,-2,0,2"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
      L"Margin=0,8,0,2",
      L"Padding=20,0,20,0",
      L"BorderBrush=#40FFFFFF"}},
    ThemeTargetStyles{L"Grid#RootGrid > Taskbar.TaskbarBackground", {
      L"Margin=-20,0,-20,0"}},
    ThemeTargetStyles{L"Grid#RootGrid > Taskbar.TaskbarBackground > Grid", {
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"CornerRadius=20,0,0,20",
      L"BorderThickness=1,1,0,1",
      L"Width=Auto",
      L"BorderBrush=#40FFFFFF",
      L"Padding=-1"}},
    ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
      L"Fill=Transparent"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.FlyoutPresenter", {
      L"RequestedTheme=Dark",
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"BorderThickness=2",
      L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
      L"CornerRadius=33",
      L"Padding=2,3,2,3"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapPickerBorder", {
      L"RequestedTheme=Dark",
      L"Background:=Transparent",
      L"BorderBrush:=Transparent",
      L"BorderThickness=2",
      L"Margin=0"}},
```

```
   ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab > Grid#ModalRootGrid > Border#BackgroundElement", {
      L"Background=Transparent",
      L"BorderThickness=0",
      L"CornerRadius=50"}},
   ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab > Grid#ModalRootGrid > Border#BackgroundElement > WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemList", {
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"BorderThickness=2",
      L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
      L"CornerRadius=50"}},
   ThemeTargetStyles{L"MenuFlyoutPresenter", {
      L"CornerRadius=20"}},
   ThemeTargetStyles{L"MenuFlyoutPresenter > Border", {
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"BorderThickness=2",
      L"CornerRadius=25",
      L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>"}},
   ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid", {
      L"Height=44",
      L"Width=44",
      L"Margin=25,2,6,0",
      L"CornerRadius=13",
      L"Background=transparent",
      L"BorderBrush:=lightgray",
      L"BorderThickness=3"}},
   ThemeTargetStyles{L"SystemTray.LanguageTextIconContent > Windows.UI.Xaml.Controls.Grid#ContainerGrid", {
      L"Height=44",
      L"Width=44",
      L"Margin=-6,4,-3,0",
      L"Background:=$IconBackground",
      L"BorderBrush:=$IconBorder",
      L"BorderThickness=1.2",
      L"CornerRadius=13"}},
   ThemeTargetStyles{L"SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid, SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid", {
      L"Height=44",
      L"Width=44",
      L"Margin=-3,4,8,0",
      L"Background:=$IconBackground",
      L"BorderBrush:=$IconBorder",
      L"BorderThickness=1.2",
      L"CornerRadius=13"}},
   ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack", {
      L"Visibility=1",
      L"Height=44",
      L"Width=44",
      L"Margin=2,4,2,0",
```

```
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2",
        L"CornerRadius=13"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid >
ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[1] >
SystemTray.IconView > Grid > Grid", {
        L"Visibility=Visible",
        L"Height=44",
        L"Width=44",
        L"Margin=2,4,2,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2",
        L"CornerRadius=13"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid >
ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[2] >
SystemTray.IconView > Grid > Grid", {
        L"Visibility=Collapsed",
        L"Height=44",
        L"Width=44",
        L"Margin=2,4,2,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2",
        L"CornerRadius=13"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid >
ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[3] >
SystemTray.IconView > Grid > Grid", {
        L"Visibility=Visible",
        L"Height=44",
        L"Width=44",
        L"Margin=2,4,2,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2",
        L"CornerRadius=13"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid >
SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon >
Grid#ContainerGrid > Grid#ContentGrid > SystemTray.BatteryIconContent >
Grid#ContainerGrid > StackPanel, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid >
SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon >
Grid#ContainerGrid > Grid#ContentGrid > SystemTray.BatteryIconContent >
Grid#ContainerGrid > StackPanel", {
        L"HorizontalAlignment=Center"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock > TextBlock", {
        L"FontSize=30"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid >
SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView >
Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid >
SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base >
TextBlock#InnerTextBlock, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid >
SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView >
Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid >
SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base >
TextBlock#InnerTextBlock", {
        L"FontSize=32",
        L"Margin=-1,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid >
SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon >
Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid >
SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock, SystemTray.SystemTrayFrame >
Grid#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid >
```

```
ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock", {
      L"VerticalAlignment=Center",
      L"HorizontalAlignment=Center"}},
   ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.OmniButton#NotificationCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.OmniButton#NotificationCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid", {
      L"Visibility=Collapsed"}},
   ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock#LanguageInnerTextBlock > TextBlock#InnerTextBlock", {
      L"FontSize=16",
      L"FontWeight=Bold"}},
   ThemeTargetStyles{L"SystemTray.ChevronIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent", {
      L"FontWeight=bold",
      L"FontSize=32"}},
   ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
      L"Height=70",
      L"Grid.Column=2",
      L"Width=Auto",
      L"HorizontalAlignment=Left",
      L"Margin=0,-2,0.5,2"}},
   ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
      L"Margin=0,0,0.5,0",
      L"Height=60",
      L"VerticalAlignment=Bottom",
      L"Padding=0",
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"BorderBrush=#40FFFFFF",
      L"BorderThickness=0,1,1,1",
      L"CornerRadius=0,20,20,0",
      L"Visibility=Visible"}},
   ThemeTargetStyles{L"SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView > Grid#ContainerGrid > Border#BackgroundBorder", {
      L"Background=transparent",
      L"BorderBrush=#40FFFFFF",
      L"BorderThickness=2,0,0,0",
      L"Padding=0",
      L"CornerRadius=0",
      L"Height=40"}},
   ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
      L"FontWeight=Bold",
      L"Margin=-2,9,2,-9",
      L"Foreground=White",
      L"Visibility=visible",
      L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"-9\"/>",
      L"FontSize=13",
      L"FontFamily=vivo Sans EN VF"}},
   ThemeTargetStyles{L"TextBlock#TimeInnerTextBlock", {
      L"Foreground=white",
      L"Width=Auto",
      L"FontWeight=Bold",
```

```
        L"FontSize=15",
        L"Margin=0,-4,5,4"}},
    ThemeTargetStyles{L"SystemTray.DateTimeIconContent > Grid#ContainerGrid", {
        L"CornerRadius=12",
        L"BorderThickness=1.2",
        L"Background:=<LinearGradientBrush StartPoint=\"0.50,-0.77\"
EndPoint=\"0.49,1.77\"><GradientStop Offset=\"0.46\" Color=\"#52A3A3A3\"/><GradientStop
Offset=\"0.5\" Color=\"#33000000\"/></LinearGradientBrush>",
        L"BorderBrush:=$IconBorder",
        L"Height=45",
        L"Margin=0,3,15,-1",
        L"Width=auto"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid >
SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon >
Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid >
SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock, SystemTray.SystemTrayFrame >
Grid#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid >
ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid >
SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base >
TextBlock#InnerTextBlock", {
        L"FontSize=30"}},
    ThemeTargetStyles{L":root > ScrollViewer > ScrollContentPresenter > Border > Grid", {
        L"ColumnDefinitions:=<ColumnDefinitionCollection><ColumnDefinition
Width=\"*\"/><ColumnDefinition Width=\"Auto\"/><ColumnDefinition
Width=\"Auto\"/><ColumnDefinition Width=\"*\"/></ColumnDefinitionCollection>",
        L"ActualWidth=>containerGridWidth"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid >
SystemTray.Stack#NonActivatableStack > Grid#Content > SystemTray.StackListView#IconStack >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon >
Grid#ContainerGrid > Border#BackgroundBorder, SystemTray.SystemTrayFrame >
Grid#SystemTrayFrameGrid > SystemTray.Stack#NonActivatableStack > Grid#Content >
SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Border#BackgroundBorder", {
        L"Background:=Transparent",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid >
Border#BackgroundBorder", {
        L"Background:=Transparent",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"SystemTray.Stack#MainStack", {
        L"Visibility=1"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid >
SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon >
Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid >
SystemTray.AdaptiveTextBlock#Underlay > TextBlock#InnerTextBlock,
SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid >
SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon >
Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid >
SystemTray.AdaptiveTextBlock#Underlay > TextBlock#InnerTextBlock", {
        L"Foreground=white"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel@CommonStates > Grid >
Border#BackgroundElement, Taskbar.TaskListButtonPanel@CommonStates >
Border#BackgroundElement", {
        L"CornerRadius=13",
        L"Margin=0,5.5,0,5.5",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid >
Rectangle#BackgroundStroke", {
        L"Fill:=<WindhawkBlur BlurAmount=\"3.5\" TintColor=\"#2D101010\"/>",
        L"Stroke:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
        L"StrokeThickness=5",
        L"RadiusX=14",
        L"RadiusY=14",
```

```
        L"Fill:=<<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid > Rectangle#BackgroundFill", {
        L"Canvas.ZIndex=0",
        L"Fill:=",
        L"Stroke:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
        L"StrokeThickness=5",
        L"RadiusX=14",
        L"RadiusY=14"}},
    ThemeTargetStyles{L"Taskbar.FlyoutFrame > Windows.UI.Xaml.Controls.Canvas#HoverFlyoutCanvas > Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid > Windows.UI.Xaml.Controls.ContentPresenter#HoverFlyoutContent > Taskbar.TaskItemThumbnailList > Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskItemThumbnailListRepeater > Taskbar.TaskItemThumbnailView > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"VerticalAlignment=Bottom",
        L"Canvas.ZIndex=1",
        L"Background:=<WindhawkBlur BlurAmount=\"5\" TintColor=\"#761E1E1E\"/>",
        L"Height=25",
        L"CornerRadius=0,0,15,15",
        L"Margin=5,0,5,0"}},
    ThemeTargetStyles{L"Border#HoverFlyoutBackground", {
        L"Margin=4,36,4,0",
        L"Canvas.ZIndex=1",
        L"Width=Auto",
        L"Background:=Transparent",
        L"BorderThickness=0",
        L"CornerRadius=13"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#ThumbBarRepeater > Taskbar.ThumbBarButton#ThumbBarButton > Windows.UI.Xaml.Controls.ContentPresenter#BorderElement", {
        L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#761E1E1E\"/>",
        L"Margin=0,-20,0,20"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#IconsRepeater > Windows.UI.Xaml.Controls.Image", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#CloseButton", {
        L"HorizontalAlignment=left",
        L"Grid.ColumnSpan=1",
        L"Grid.RowSpan=1",
        L"Canvas.ZIndex=1",
        L"CornerRadius=20",
        L"Width=28",
        L"Height=28",
        L"Margin=-18,40,15,-40",
        L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#80ffffff\"/>",
        L"Foreground=black",
        L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#761E1E1E\"/>"}},
    ThemeTargetStyles{L"Taskbar.FlyoutFrame > Windows.UI.Xaml.Controls.Canvas#HoverFlyoutCanvas > Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid > Windows.UI.Xaml.Controls.ContentPresenter#HoverFlyoutContent > Taskbar.TaskItemThumbnailList > Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskItemThumbnailListRepeater > Taskbar.TaskItemThumbnailView > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.TextBlock#DisplayNameTextBlock", {
        L"Grid.ColumnSpan=2",
        L"Grid.RowSpan=2",
        L"VerticalAlignment=bottom",
        L"HorizontalAlignment=Center",
        L"Margin=0,-5,0,5",
```

```
    L"Canvas.ZIndex=1"}},
  ThemeTargetStyles{L"SystemTray.NotifyIconView@CommonStates > Grid#ContainerGrid > Border#BackgroundBorder", {
    L"CornerRadius=12",
    L"Background:=$IconBackground",
    L"BorderBrush:=$IconBorder",
    L"Margin=2",
    L"BorderThickness=1.2"}},
  ThemeTargetStyles{L"Border#OverflowFlyoutBackgroundBorder", {
    L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
    L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#60ffffff\"/>",
    L"BorderThickness=2",
    L"CornerRadius=32,32,30,30",
    L"Margin=-10"}},
  ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Rectangle#RunningIndicator, Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Rectangle#RunningIndicator", {
    L"Fill:=#90ffffff",
    L"RadiusX=2",
    L"RadiusY=2",
    L"Margin=-2",
    L"Height=4",
    L"Width=4",
    L"Margin=8,0,0,1",
    L"Width@ActiveRunningIndicator=8",
    L"Fill@ActiveRunningIndicator=#60CDFF"}},
  ThemeTargetStyles{L"Grid#IconPanel > TextBlock#LabelControl, Taskbar.TaskListLabeledButtonPanel > TextBlock#LabelControl", {
    L"Margin=4,0,0,0",
    L"Foreground=White"}},
  ThemeTargetStyles{L"Taskbar.SearchBoxButton", {
    L"Background:=<WindhawkBlur BlurAmount=\"60\" TintColor=\"#35ffffff\"/>",
    L"CornerRadius=20",
    L"Margin=2,6,2,6",
    L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,1\" StartPoint=\"0,0\"><GradientStop Color=\"#E0ffffff\" Offset=\"0.0\"/><GradientStop Color=\"#20ffffff\" Offset=\"0.5\"/><GradientStop Color=\"#A0ffffff\" Offset=\"1.0\"/></LinearGradientBrush>",
    L"BorderThickness=1.2"}},
  ThemeTargetStyles{L"TextBlock#SearchBoxTextBlock", {
    L"FontSize=12",
    L"Foreground=White"}},
  ThemeTargetStyles{L"Grid", {
    L"RequestedTheme=2"}},
  ThemeTargetStyles{L"Taskbar.StartButton#StartButton", {
    L"Background:=<WindhawkBlur BlurAmount=\"60\" TintColor=\"#35ffffff\"/>",
    L"CornerRadius=20",
    L"Margin=2,6,2,6",
    L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,1\" StartPoint=\"0,0\"><GradientStop Color=\"#E0ffffff\" Offset=\"0.0\"/><GradientStop Color=\"#20ffffff\" Offset=\"0.5\"/><GradientStop Color=\"#A0ffffff\" Offset=\"1.0\"/></LinearGradientBrush>",
    L"BorderThickness=1.2"}},
  ThemeTargetStyles{L"Border#MultiWindowElement", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"SystemTray.TextIconContent > Grid > SystemTray.AdaptiveTextBlock#Base > TextBlock", {
    L"Foreground=White"}},
  ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton", {
    L"Margin=-12,0,0,0"}},
  ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl > Grid > SearchUx.SearchUI.SearchIconButton >
```

```
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Grid >
Border#BackgroundElement, SearchUx.SearchUI.SearchButtonControl > Grid >
SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Border#BackgroundElement", {
    L"Margin=0,5.5,0,5.5",
    L"CornerRadius=13",
    L"Background:=$IconBackground",
    L"BorderBrush:=$IconBorder"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.
Name=Task View]", {
    L"Background:=<WindhawkBlur BlurAmount=\"60\" TintColor=\"#35ffffff\"/>",
    L"CornerRadius=20",
    L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,1\" StartPoint=\"0,0\"><GradientStop
Color=\"#E0ffffff\" Offset=\"0.0\"/><GradientStop Color=\"#20ffffff\"
Offset=\"0.5\"/><GradientStop Color=\"#A0ffffff\" Offset=\"1.0\"/></LinearGradientBrush>",
    L"BorderThickness=1.2"}},
  ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Border,
Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Border", {
    L"Background@InactiveRunningIndicatorPointerOver:=<WindhawkBlur BlurAmount=\"40\"
TintColor=\"#10ffffff\"/>",
    L"CornerRadius=12",
    L"BorderBrush@InactiveRunningIndicatorPointerOver:=<LinearGradientBrush
EndPoint=\"1,0\" StartPoint=\"0,0\"><GradientStop Color=\"#80ffffff\"
Offset=\"0.0\"/><GradientStop Color=\"{ThemeResource SurfaceStrokeColorDefault}\"
Offset=\"0.55\"/><GradientStop Color=\"#80ffffff\" Offset=\"1\"/></LinearGradientBrush>",
    L"BorderThickness@InactiveRunningIndicatorPointerOver=1"}},
  ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border#BackgroundElement,
Taskbar.TaskListLabeledButtonPanel@CommonStates > Border#BackgroundElement", {
    L"CornerRadius=13",
    L"Margin=0,5.5,0,5.5",
    L"Background:=$IconBackground",
    L"BorderBrush:=$IconBorder",
    L"BorderThickness=1.2"}},
  ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground >
Grid > Rectangle#BackgroundFill", {
    L"Fill=Transparent"}},
  ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon", {
    L"Background:=<WindhawkBlur BlurAmount=\"10\" TintColor=\"#40ffffff\"/>",
    L"CornerRadius=12",
    L"Margin=2",
    L"Padding=2",
    L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,0\" StartPoint=\"0,0\"><GradientStop
Color=\"#80ffffff\" Offset=\"0.0\"/><GradientStop Color=\"{ThemeResource
SurfaceStrokeColorDefault}\" Offset=\"0.55\"/><GradientStop Color=\"#80ffffff\"
Offset=\"1\"/></LinearGradientBrush>",
    L"BorderThickness=2"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#ConfirmatorMainGrid", {
    L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
    L"CornerRadius=24",
    L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
    L"BorderThickness=2",
    L"Margin=0,0,0,10"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid.Border#ConfirmatorMainGrid", {
    L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalTrackRect", {
    L"Fill=#20ffffff",
    L"RadiusX=12",
    L"RadiusY=12",
    L"Height=18",
    L"Margin=0"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalDecreaseRect", {
```

```
      L"Fill:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColor}\" />",
      L"RadiusX=12",
      L"RadiusY=12",
      L"Height=18"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#VolumeConfirmator", {
      L"Padding=8,0,8,0"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#BrightnessConfirmator", {
      L"Padding=8,0,8,0"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.TextBlock#volumeLevelText", {
      L"Foreground=White"}},
}, {
   L"IconBackground=<ImageBrush
ImageSource=\"https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-
styling-guide/refs/heads/main/Themes/OS26%20Liquid%20Glass/Assets/tahoeappbg.png\"
Stretch=\"UniformtoFill\" />",
   L"IconBorder=<LinearGradientBrush EndPoint=\"1,1\" StartPoint=\"0,0\"><GradientStop
Color=\"#F5ffffff\" Offset=\"0.0\"/><GradientStop Color=\"#40ffffff\"
Offset=\"0.4\"/><GradientStop Color=\"#20ffffff\" Offset=\"0.6\"/><GradientStop
Color=\"#90ffffff\" Offset=\"1.0\"/></LinearGradientBrush>",
}};

const Theme g_themeOS26_Liquid_Glass_variant_ClearTaskbar = {{
   ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
      L"Height=80",
      L"MaxHeight=80",
      L"HorizontalAlignment=Stretch"}},
   ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
      L"VerticalAlignment=Center"}},
   ThemeTargetStyles{L"Grid#IconPanel > Image#Icon,
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
      L"Height=24",
      L"Width=24",
      L"Margin=5,0,-5,0"}},
   ThemeTargetStyles{L"Taskbar.TaskListButtonPanel", {
      L"Width=55",
      L"Height=70"}},
   ThemeTargetStyles{L"Grid#IconPanel, Taskbar.TaskListLabeledButtonPanel", {
      L"Width=55",
      L"Height=70"}},
   ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl > Grid >
SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel", {
      L"Width=55",
      L"Height=70"}},
   ThemeTargetStyles{L"Grid#RootGrid > Taskbar.TaskbarBackground", {
      L"Margin=-15,0,-15,0"}},
   ThemeTargetStyles{L"Grid#RootGrid > Taskbar.TaskbarBackground > Grid", {
      L"CornerRadius=20",
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"BorderThickness=1",
      L"BorderBrush=#40FFFFFF",
      L"Padding=-1"}},
   ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
      L"Fill=Transparent"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
      L"Visibility=Visible",
      L"Margin=0,8,0,2",
      L"Padding=20,0,20,0"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.FlyoutPresenter", {
```

```
    L"RequestedTheme=Dark",
    L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
    L"BorderThickness=2",
    L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
    L"CornerRadius=33",
    L"Padding=2,3,2,3"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapPickerBorder", {
    L"RequestedTheme=Dark",
    L"Background:=Transparent",
    L"BorderBrush:=Transparent",
    L"BorderThickness=2",
    L"Margin=0"}},
  ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab > Grid#ModalRootGrid > Border#BackgroundElement", {
    L"Background=Transparent",
    L"BorderThickness=0",
    L"CornerRadius=50"}},
  ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab > Grid#ModalRootGrid > Border#BackgroundElement > WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemList", {
    L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
    L"BorderThickness=2",
    L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
    L"CornerRadius=50"}},
  ThemeTargetStyles{L"MenuFlyoutPresenter", {
    L"CornerRadius=20"}},
  ThemeTargetStyles{L"MenuFlyoutPresenter > Border", {
    L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
    L"BorderThickness=2",
    L"CornerRadius=25",
    L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>"}},
  ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > Grid > SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid, ScrollViewer > ScrollContentPresenter > Border > Grid > SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid", {
    L"Height=35",
    L"CornerRadius=12",
    L"Background:=$IconBackground",
    L"BorderBrush:=$IconBorder",
    L"BorderThickness=1.2"}},
  ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
    L"Width=Auto",
    L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
    L"CornerRadius=15",
    L"Margin=200,5,-420,-5",
    L"RenderTransform:=<TranslateTransform X=\"-435\" Y=\"-2\"/>",
    L"Padding=10,2",
    L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,1\" StartPoint=\"0,0\"><GradientStop Color=\"#50ffffff\" Offset=\"0.0\"/><GradientStop Color=\"#10ffffff\" Offset=\"0.5\"/><GradientStop Color=\"#30ffffff\" Offset=\"1.0\"/></LinearGradientBrush>",
    L"BorderThickness=2",
    L"Visibility=Visible"}},
  ThemeTargetStyles{L"Taskbar.TaskListButtonPanel@CommonStates > Grid > Border#BackgroundElement, Taskbar.TaskListButtonPanel@CommonStates > Border#BackgroundElement", {
    L"CornerRadius=15",
```

```
        L"Margin=0,5.5,0,5.5",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid > Rectangle#BackgroundStroke", {
        L"Fill:=<WindhawkBlur BlurAmount=\"3.5\" TintColor=\"#2D101010\"/>",
        L"Stroke:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
        L"StrokeThickness=5",
        L"RadiusX=14",
        L"RadiusY=14",
        L"Fill:=<<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid > Rectangle#BackgroundFill", {
        L"Canvas.ZIndex=0",
        L"Fill:=",
        L"Stroke:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
        L"StrokeThickness=5",
        L"RadiusX=14",
        L"RadiusY=14"}},
    ThemeTargetStyles{L"Taskbar.FlyoutFrame > Windows.UI.Xaml.Controls.Canvas#HoverFlyoutCanvas > Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid > Windows.UI.Xaml.Controls.ContentPresenter#HoverFlyoutContent > Taskbar.TaskItemThumbnailList > Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskItemThumbnailListRepeater > Taskbar.TaskItemThumbnailView > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"VerticalAlignment=Bottom",
        L"Canvas.ZIndex=1",
        L"Background:=<WindhawkBlur BlurAmount=\"5\" TintColor=\"#761E1E1E\"/>",
        L"Height=25",
        L"CornerRadius=0,0,15,15",
        L"Margin=5,0,5,0"}},
    ThemeTargetStyles{L"Border#HoverFlyoutBackground", {
        L"Margin=4,36,4,0",
        L"Canvas.ZIndex=1",
        L"Width=Auto",
        L"Background:=Transparent",
        L"BorderThickness=0",
        L"CornerRadius=15"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#IconsRepeater > Windows.UI.Xaml.Controls.Image", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#ThumbBarRepeater > Taskbar.ThumbBarButton#ThumbBarButton > Windows.UI.Xaml.Controls.ContentPresenter#BorderElement", {
        L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#761E1E1E\"/>",
        L"Margin=0,-20,0,20"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#CloseButton", {
        L"HorizontalAlignment=left",
        L"Grid.ColumnSpan=1",
        L"Grid.RowSpan=1",
        L"Canvas.ZIndex=1",
        L"CornerRadius=20",
        L"Width=28",
        L"Height=28",
        L"Margin=-18,40,15,-40",
        L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#80ffffff\"/>",
        L"Foreground=black",
```

```
        L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#761E1E1E\"/>"}},
    ThemeTargetStyles{L"Taskbar.FlyoutFrame > Windows.UI.Xaml.Controls.Canvas#HoverFlyoutCanvas > Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid > Windows.UI.Xaml.Controls.ContentPresenter#HoverFlyoutContent > Taskbar.TaskItemThumbnailList > Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskItemThumbnailListRepeater > Taskbar.TaskItemThumbnailView > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.TextBlock#DisplayNameTextBlock", {
        L"Grid.ColumnSpan=2",
        L"Grid.RowSpan=2",
        L"VerticalAlignment=bottom",
        L"HorizontalAlignment=Center",
        L"Margin=0,-5,0,5",
        L"Canvas.ZIndex=1"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView@CommonStates > Grid#ContainerGrid > Border#BackgroundBorder", {
        L"CornerRadius=12",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"Margin=2",
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView@CommonStates > Grid#ContainerGrid > Border#BackgroundBorder", {
        L"CornerRadius=12",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"Margin=2",
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"Border#OverflowFlyoutBackgroundBorder", {
        L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
        L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#60ffffff\"/>",
        L"BorderThickness=2",
        L"CornerRadius=32,32,30,30",
        L"Margin=-10"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid > Grid > SystemTray.TextIconContent", {
        L"CornerRadius=15"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Rectangle#RunningIndicator, Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Rectangle#RunningIndicator", {
        L"Fill:=#90ffffff",
        L"RadiusX=3",
        L"RadiusY=3",
        L"Margin=-2",
        L"Height=6",
        L"Width=6",
        L"Margin=10,0,0,-2",
        L"Width@ActiveRunningIndicator=12",
        L"Fill@ActiveRunningIndicator=#60CDFF"}},
    ThemeTargetStyles{L"Grid#IconPanel > TextBlock#LabelControl, Taskbar.TaskListLabeledButtonPanel > TextBlock#LabelControl", {
        L"Margin=4,0,0,0",
        L"Foreground=White"}},
    ThemeTargetStyles{L"Taskbar.SearchBoxButton", {
        L"Background:=<WindhawkBlur BlurAmount=\"60\" TintColor=\"#35ffffff\"/>",
        L"CornerRadius=20",
        L"Margin=2,6,2,6",
        L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,1\" StartPoint=\"0,0\"><GradientStop Color=\"#E0ffffff\" Offset=\"0.0\"/><GradientStop Color=\"#20ffffff\" Offset=\"0.5\"/><GradientStop Color=\"#A0ffffff\" Offset=\"1.0\"/></LinearGradientBrush>",
```

```
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"TextBlock#SearchBoxTextBlock", {
        L"FontSize=12",
        L"Foreground=White"}},
    ThemeTargetStyles{L"Grid", {
        L"RequestedTheme=2"}},

ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton[AutomationProperties.Name=Copilot] > Grid#IconPanel > Border#BackgroundElement, Taskbar.TaskListButton#TaskListButton[AutomationProperties.Name=Copilot] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border#BackgroundElement", {
        L"Background:=$IconBackground"}},
    ThemeTargetStyles{L"Taskbar.StartButton#StartButton", {
        L"Background:=<WindhawkBlur BlurAmount=\"60\" TintColor=\"#35ffffff\"/>",
        L"CornerRadius=20",
        L"Margin=2,6,2,6",
        L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,1\" StartPoint=\"0,0\"><GradientStop Color=\"#E0ffffff\" Offset=\"0.0\"/><GradientStop Color=\"#20ffffff\" Offset=\"0.5\"/><GradientStop Color=\"#A0ffffff\" Offset=\"1.0\"/></LinearGradientBrush>",
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"Border#MultiWindowElement", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"TextBlock#TimeInnerTextBlock", {
        L"Foreground=White",
        L"FontSize=18",
        L"FontFamily=Quantico",
        L"Margin=0",
        L"Padding=0",
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"1\"/>"}},
    ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
        L"Foreground=White",
        L"Visibility=Collapsed",
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"-9\"/>",
        L"FontSize=11",
        L"FontFamily=vivo Sans EN VF"}},
    ThemeTargetStyles{L"SystemTray.TextIconContent > Grid > SystemTray.AdaptiveTextBlock#Base > TextBlock", {
        L"Foreground=White"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton", {
        L"Margin=-12,0,0,0"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl > Grid > SearchUx.SearchUI.SearchIconButton > SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Grid > Border#BackgroundElement, SearchUx.SearchUI.SearchButtonControl > Grid > SearchUx.SearchUI.SearchIconButton > SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Border#BackgroundElement", {
        L"Margin=0,5.5,0,5.5",
        L"CornerRadius=15",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.Name=Task View]", {
        L"Background:=<WindhawkBlur BlurAmount=\"60\" TintColor=\"#35ffffff\"/>",
        L"CornerRadius=20",
        L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,1\" StartPoint=\"0,0\"><GradientStop Color=\"#E0ffffff\" Offset=\"0.0\"/><GradientStop Color=\"#20ffffff\" Offset=\"0.5\"/><GradientStop Color=\"#A0ffffff\" Offset=\"1.0\"/></LinearGradientBrush>",
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Border, Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Border", {
```

```
      L"Background@InactiveRunningIndicatorPointerOver:=<WindhawkBlur BlurAmount=\"40\" TintColor=\"#10ffffff\"/>",
      L"CornerRadius=12",
      L"BorderBrush@InactiveRunningIndicatorPointerOver:=<LinearGradientBrush EndPoint=\"1,0\" StartPoint=\"0,0\"><GradientStop Color=\"#80ffffff\" Offset=\"0.0\"/><GradientStop Color=\"{ThemeResource SurfaceStrokeColorDefault}\" Offset=\"0.55\"/><GradientStop Color=\"#80ffffff\" Offset=\"1\"/></LinearGradientBrush>",
      L"BorderThickness@InactiveRunningIndicatorPointerOver=1"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border#BackgroundElement, Taskbar.TaskListLabeledButtonPanel@CommonStates > Border#BackgroundElement", {
      L"CornerRadius=15",
      L"Margin=0,5.5,0,5.5",
      L"Background:=$IconBackground",
      L"BorderBrush:=$IconBorder",
      L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground > Grid > Rectangle#BackgroundStroke", {
      L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground > Grid > Rectangle#BackgroundFill", {
      L"Fill=Transparent"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon", {
      L"Background:=<WindhawkBlur BlurAmount=\"10\" TintColor=\"#40ffffff\"/>",
      L"CornerRadius=12",
      L"Margin=2",
      L"Padding=2",
      L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,0\" StartPoint=\"0,0\"><GradientStop Color=\"#80ffffff\" Offset=\"0.0\"/><GradientStop Color=\"{ThemeResource SurfaceStrokeColorDefault}\" Offset=\"0.55\"/><GradientStop Color=\"#80ffffff\" Offset=\"1\"/></LinearGradientBrush>",
      L"BorderThickness=2"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#ConfirmatorMainGrid", {
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"CornerRadius=24",
      L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
      L"BorderThickness=2",
      L"Margin=0,0,0,10"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid.Border#ConfirmatorMainGrid", {
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalTrackRect", {
      L"Fill=#20ffffff",
      L"RadiusX=12",
      L"RadiusY=12",
      L"Height=18",
      L"Margin=0"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalDecreaseRect", {
      L"Fill=#ff7060",
      L"RadiusX=12",
      L"RadiusY=12",
      L"Height=18"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#VolumeConfirmator", {
      L"Padding=8,0,8,0"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#BrightnessConfirmator", {
      L"Padding=8,0,8,0"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.TextBlock#volumeLevelText", {
      L"Foreground=White"}},
}, {
    L"IconBackground=<ImageBrush ImageSource=\"https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-
```

```
styling-guide/refs/heads/main/Themes/OS26%20Liquid%20Glass/Assets/tahoeappbg.png\"
Stretch=\"UniformtoFill\"/>",
    L"IconBorder=<LinearGradientBrush EndPoint=\"1,1\" StartPoint=\"0,0\"><GradientStop
Color=\"#F5ffffff\" Offset=\"0.0\"/><GradientStop Color=\"#40ffffff\"
Offset=\"0.4\"/><GradientStop Color=\"#20ffffff\" Offset=\"0.6\"/><GradientStop
Color=\"#90ffffff\" Offset=\"1.0\"/></LinearGradientBrush>",
}};

const Theme g_themeOS26_Liquid_Glass_variant_DarkMacDock = {{
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid@DockingStates", {
        L"Tag=horizontal",
        L"Tag@DockedLeft=vertical",
        L"Tag@DockedRight=vertical",
        L"Tag=>taskbarDock"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
        L"Width={{taskbarDock==`vertical`?skip():`Auto`}}",
        L"MinWidth:=100",
        L"MaxWidth={{containerGridWidth>0?containerGridWidth:`Infinity`}}",
        L"Grid.Column=1",
        L"Transitions:=<TransitionCollection><RepositionThemeTransition
IsStaggeringEnabled=\"False\"/></TransitionCollection>",
        L"Height={{taskbarDock==`vertical`?skip():80}}",
        L"MaxHeight=80",
        L"HorizontalAlignment=Center"}},
    ThemeTargetStyles{L"Grid#IconPanel > Image#Icon,
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Height=24",
        L"Width=24",
        L"Margin=5,0,-5,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel", {
        L"Width=55",
        L"Height=70"}},
    ThemeTargetStyles{L"Grid#IconPanel, Taskbar.TaskListLabeledButtonPanel", {
        L"Width=55",
        L"Height=70"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl > Grid >
SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel", {
        L"Width=55",
        L"Height=70"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
        L"Margin=0,8,0,2",
        L"Padding=20,0,20,0",
        L"BorderBrush=#40FFFFFF"}},
    ThemeTargetStyles{L"Grid#RootGrid > Taskbar.TaskbarBackground", {
        L"Margin=-20,0,-20,0"}},
    ThemeTargetStyles{L"Grid#RootGrid > Taskbar.TaskbarBackground > Grid", {
        L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
        L"CornerRadius=20,0,0,20",
        L"BorderThickness=1,1,0,1",
        L"Width=Auto",
        L"BorderBrush=#40FFFFFF",
        L"Padding=-1"}},
    ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.FlyoutPresenter", {
        L"RequestedTheme=Dark",
        L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
```

```
      L"BorderThickness=2",
      L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
      L"CornerRadius=33",
      L"Padding=2,3,2,3"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapPickerBorder", {
      L"RequestedTheme=Dark",
      L"Background:=Transparent",
      L"BorderBrush:=Transparent",
      L"BorderThickness=2",
      L"Margin=0"}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab > Grid#ModalRootGrid > Border#BackgroundElement", {
      L"Background=Transparent",
      L"BorderThickness=0",
      L"CornerRadius=50"}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab > Grid#ModalRootGrid > Border#BackgroundElement > WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemList", {
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"BorderThickness=2",
      L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
      L"CornerRadius=50"}},
    ThemeTargetStyles{L"MenuFlyoutPresenter", {
      L"CornerRadius=20"}},
    ThemeTargetStyles{L"MenuFlyoutPresenter > Border", {
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"BorderThickness=2",
      L"CornerRadius=25",
      L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid", {
      L"Height=48",
      L"Width=48",
      L"Margin=25,2,12,0",
      L"CornerRadius=15",
      L"Background=transparent",
      L"BorderBrush:=lightgray",
      L"BorderThickness=3"}},
    ThemeTargetStyles{L"SystemTray.LanguageTextIconContent > Windows.UI.Xaml.Controls.Grid#ContainerGrid", {
      L"Width=48",
      L"Height=48",
      L"Margin=-6,4,-3,0",
      L"Background:=$IconBackground",
      L"BorderBrush:=$IconBorder",
      L"BorderThickness=1.2",
      L"CornerRadius=15"}},
    ThemeTargetStyles{L"SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid, SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid", {
      L"Width=48",
      L"Height=48",
```

```
        L"Margin=-3,4,8,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2",
        L"CornerRadius=15"}},
    ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack", {
        L"Visibility=1",
        L"Width=48",
        L"Height=48",
        L"Margin=2,4,2,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2",
        L"CornerRadius=15"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid >
ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[1] >
SystemTray.IconView > Grid > Grid", {
        L"Visibility=Visible",
        L"Width=48",
        L"Height=48",
        L"Margin=2,4,2,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2",
        L"CornerRadius=15"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid >
ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[2] >
SystemTray.IconView > Grid > Grid", {
        L"Visibility=Visible",
        L"Width=48",
        L"Height=48",
        L"Margin=2,4,2,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2",
        L"CornerRadius=15"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid >
ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[3] >
SystemTray.IconView > Grid > Grid", {
        L"Visibility=Visible",
        L"Width=48",
        L"Height=48",
        L"Margin=2,4,2,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2",
        L"CornerRadius=15"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid >
SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon >
Grid#ContainerGrid > Grid#ContentGrid > SystemTray.BatteryIconContent >
Grid#ContainerGrid > StackPanel, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid >
SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon >
Grid#ContainerGrid > Grid#ContentGrid > SystemTray.BatteryIconContent >
Grid#ContainerGrid > StackPanel", {
        L"HorizontalAlignment=Center"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock > TextBlock", {
        L"FontSize=30"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid >
SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView >
```

```
Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid >
SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base >
TextBlock#InnerTextBlock, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid >
SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView >
Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid >
SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base >
TextBlock#InnerTextBlock", {
        L"FontSize=32"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid >
SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon >
Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid >
SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock, SystemTray.SystemTrayFrame >
Grid#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid >
ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid >
SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base >
TextBlock#InnerTextBlock", {
        L"VerticalAlignment=Center",
        L"HorizontalAlignment=Center"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid >
SystemTray.OmniButton#NotificationCenterButton > Grid >
ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid >
SystemTray.TextIconContent > Grid#ContainerGrid, SystemTray.SystemTrayFrame >
Grid#SystemTrayFrameGrid > SystemTray.OmniButton#NotificationCenterButton > Grid >
ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid >
SystemTray.TextIconContent > Grid#ContainerGrid", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock#LanguageInnerTextBlock >
TextBlock#InnerTextBlock", {
        L"FontSize=16",
        L"FontWeight=Bold"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView > Grid#ContainerGrid >
ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent", {
        L"FontWeight=bold",
        L"FontSize=32"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
        L"Height=80",
        L"Grid.Column=2",
        L"Width=Auto",
        L"HorizontalAlignment=Left",
        L"Margin=0,-2,0.5,2"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"Margin=0,0,0.5,0",
        L"Height=70",
        L"VerticalAlignment=Bottom",
        L"Padding=0",
        L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
        L"BorderBrush=#40FFFFFF",
        L"BorderThickness=0,1,1,1",
        L"CornerRadius=0,20,20,0",
        L"Visibility=Visible"}},
    ThemeTargetStyles{L"SystemTray.Stack#NotifyIconStack > Grid#Content >
SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.ChevronIconView > Grid#ContainerGrid > Border#BackgroundBorder", {
        L"Background=transparent",
        L"BorderBrush=#40FFFFFF",
        L"BorderThickness=2,0,0,0",
        L"Padding=0",
        L"CornerRadius=0",
        L"Height=40"}},
    ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
        L"FontWeight=Bold",
        L"Margin=-2,9,2,-9",
```

```
        L"Foreground=White",
        L"Visibility=visible",
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"-9\"/>",
        L"FontSize=13",
        L"FontFamily=vivo Sans EN VF"}},
    ThemeTargetStyles{L"TextBlock#TimeInnerTextBlock", {
        L"Foreground=white",
        L"Width=Auto",
        L"FontWeight=Bold",
        L"FontSize=15",
        L"Margin=0,-4,5,4"}},
    ThemeTargetStyles{L"SystemTray.DateTimeIconContent > Grid#ContainerGrid", {
        L"CornerRadius=12",
        L"BorderThickness=1.2",
        L"Background:=<LinearGradientBrush StartPoint=\"0.50,-1.50\"
EndPoint=\"0.50,2.50\"><GradientStop Offset=\"0.48\" Color=\"#FF3A40\"/><GradientStop
Offset=\"0.49\" Color=\"#141414\"/></LinearGradientBrush>",
        L"BorderBrush:=$IconBorder",
        L"Height=45",
        L"Margin=0,3,15,-1",
        L"Width=auto"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid >
SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon >
Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid >
SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock, SystemTray.SystemTrayFrame >
Grid#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid >
ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid >
SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base >
TextBlock#InnerTextBlock", {
        L"FontSize=30"}},
    ThemeTargetStyles{L":root > ScrollViewer > ScrollContentPresenter > Border > Grid", {
        L"ColumnDefinitions:=<ColumnDefinitionCollection><ColumnDefinition
Width=\"*\"/><ColumnDefinition Width=\"Auto\"/><ColumnDefinition
Width=\"Auto\"/><ColumnDefinition Width=\"*\"/></ColumnDefinitionCollection>",
        L"ActualWidth=>containerGridWidth"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid >
SystemTray.Stack#NonActivatableStack > Grid#Content > SystemTray.StackListView#IconStack >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon >
Grid#ContainerGrid > Border#BackgroundBorder, SystemTray.SystemTrayFrame >
Grid#SystemTrayFrameGrid > SystemTray.Stack#NonActivatableStack > Grid#Content >
SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Border#BackgroundBorder", {
        L"Background:=Transparent",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid >
Border#BackgroundBorder", {
        L"Background:=Transparent",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"SystemTray.Stack#MainStack", {
        L"Visibility=1"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid >
SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon >
Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid >
SystemTray.AdaptiveTextBlock#Underlay > TextBlock#InnerTextBlock,
SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid >
SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon >
Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid >
SystemTray.AdaptiveTextBlock#Underlay > TextBlock#InnerTextBlock", {
        L"Foreground=white"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel@CommonStates > Grid >
Border#BackgroundElement, Taskbar.TaskListButtonPanel@CommonStates >
Border#BackgroundElement", {
        L"CornerRadius=15",
        L"Margin=0,5.5,0,5.5",
        L"Background:=$IconBackground",
```

```
    L"BorderBrush:=$IconBorder",
    L"BorderThickness=1.2"}},
  ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid > Rectangle#BackgroundStroke", {
    L"Fill:=<WindhawkBlur BlurAmount=\"3.5\" TintColor=\"#2D101010\"/>",
    L"Stroke:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
    L"StrokeThickness=5",
    L"RadiusX=14",
    L"RadiusY=14",
    L"Fill:=<<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>"}},
  ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid > Rectangle#BackgroundFill", {
    L"Canvas.ZIndex=0",
    L"Fill:=",
    L"Stroke:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
    L"StrokeThickness=5",
    L"RadiusX=14",
    L"RadiusY=14"}},
  ThemeTargetStyles{L"Taskbar.FlyoutFrame > Windows.UI.Xaml.Controls.Canvas#HoverFlyoutCanvas > Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid > Windows.UI.Xaml.Controls.ContentPresenter#HoverFlyoutContent > Taskbar.TaskItemThumbnailList > Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskItemThumbnailListRepeater > Taskbar.TaskItemThumbnailView > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
    L"VerticalAlignment=Bottom",
    L"Canvas.ZIndex=1",
    L"Background:=<WindhawkBlur BlurAmount=\"5\" TintColor=\"#761E1E1E\"/>",
    L"Height=25",
    L"CornerRadius=0,0,15,15",
    L"Margin=5,0,5,0"}},
  ThemeTargetStyles{L"Border#HoverFlyoutBackground", {
    L"Margin=4,36,4,0",
    L"Canvas.ZIndex=1",
    L"Width=Auto",
    L"Background:=Transparent",
    L"BorderThickness=0",
    L"CornerRadius=15"}},
  ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#ThumbBarRepeater > Taskbar.ThumbBarButton#ThumbBarButton > Windows.UI.Xaml.Controls.ContentPresenter#BorderElement", {
    L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#761E1E1E\"/>",
    L"Margin=0,-20,0,20"}},
  ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#IconsRepeater > Windows.UI.Xaml.Controls.Image", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#CloseButton", {
    L"HorizontalAlignment=left",
    L"Grid.ColumnSpan=1",
    L"Grid.RowSpan=1",
    L"Canvas.ZIndex=1",
    L"CornerRadius=20",
    L"Width=28",
    L"Height=28",
    L"Margin=-18,40,15,-40",
    L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#80ffffff\"/>",
    L"Foreground=black",
    L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#761E1E1E\"/>"}},
```

```
  ThemeTargetStyles{L"Taskbar.FlyoutFrame > Windows.UI.Xaml.Controls.Canvas#HoverFlyoutCanvas > Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid > Windows.UI.Xaml.Controls.ContentPresenter#HoverFlyoutContent > Taskbar.TaskItemThumbnailList > Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskItemThumbnailListRepeater > Taskbar.TaskItemThumbnailView > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.TextBlock#DisplayNameTextBlock", {
    L"Grid.ColumnSpan=2",
    L"Grid.RowSpan=2",
    L"VerticalAlignment=bottom",
    L"HorizontalAlignment=Center",
    L"Margin=0,-5,0,5",
    L"Canvas.ZIndex=1"}},
  ThemeTargetStyles{L"SystemTray.NotifyIconView@CommonStates > Grid#ContainerGrid > Border#BackgroundBorder", {
    L"CornerRadius=12",
    L"Background:=$IconBackground",
    L"BorderBrush:=$IconBorder",
    L"Margin=2",
    L"BorderThickness=1.2"}},
  ThemeTargetStyles{L"Border#OverflowFlyoutBackgroundBorder", {
    L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
    L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#60ffffff\"/>",
    L"BorderThickness=2",
    L"CornerRadius=32,32,30,30",
    L"Margin=-10"}},
  ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Rectangle#RunningIndicator, Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Rectangle#RunningIndicator", {
    L"Fill:=#90ffffff",
    L"RadiusX=3",
    L"RadiusY=3",
    L"Margin=-2",
    L"Height=6",
    L"Width=6",
    L"Margin=10,0,0,-2",
    L"Width@ActiveRunningIndicator=12",
    L"Fill@ActiveRunningIndicator=#60CDFF"}},
  ThemeTargetStyles{L"Grid#IconPanel > TextBlock#LabelControl, Taskbar.TaskListLabeledButtonPanel > TextBlock#LabelControl", {
    L"Margin=4,0,0,0",
    L"Foreground=White"}},
  ThemeTargetStyles{L"Taskbar.SearchBoxButton", {
    L"Background:=<WindhawkBlur BlurAmount=\"60\" TintColor=\"#35ffffff\"/>",
    L"CornerRadius=20",
    L"Margin=2,6,2,6",
    L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,1\" StartPoint=\"0,0\"><GradientStop Color=\"#E0ffffff\" Offset=\"0.0\"/><GradientStop Color=\"#20ffffff\" Offset=\"0.5\"/><GradientStop Color=\"#A0ffffff\" Offset=\"1.0\"/></LinearGradientBrush>",
    L"BorderThickness=1.2"}},
  ThemeTargetStyles{L"TextBlock#SearchBoxTextBlock", {
    L"FontSize=12",
    L"Foreground=White"}},
  ThemeTargetStyles{L"Grid", {
    L"RequestedTheme=2"}},
  ThemeTargetStyles{L"Taskbar.StartButton#StartButton", {
    L"Background:=<WindhawkBlur BlurAmount=\"60\" TintColor=\"#35ffffff\"/>",
    L"CornerRadius=20",
    L"Margin=2,6,2,6",
```

```
        L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,1\" StartPoint=\"0,0\"><GradientStop Color=\"#E0ffffff\" Offset=\"0.0\"/><GradientStop Color=\"#20ffffff\" Offset=\"0.5\"/><GradientStop Color=\"#A0ffffff\" Offset=\"1.0\"/></LinearGradientBrush>",
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"Border#MultiWindowElement", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"SystemTray.TextIconContent > Grid > SystemTray.AdaptiveTextBlock#Base > TextBlock", {
        L"Foreground=White"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton", {
        L"Margin=-12,0,0,0"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl > Grid > SearchUx.SearchUI.SearchIconButton > SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Grid > Border#BackgroundElement, SearchUx.SearchUI.SearchButtonControl > Grid > SearchUx.SearchUI.SearchIconButton > SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Border#BackgroundElement", {
        L"Margin=0,5.5,0,5.5",
        L"CornerRadius=15",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder"}},

    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.Name=Task View]", {
        L"Background:=<WindhawkBlur BlurAmount=\"60\" TintColor=\"#35ffffff\"/>",
        L"CornerRadius=20",
        L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,1\" StartPoint=\"0,0\"><GradientStop Color=\"#E0ffffff\" Offset=\"0.0\"/><GradientStop Color=\"#20ffffff\" Offset=\"0.5\"/><GradientStop Color=\"#A0ffffff\" Offset=\"1.0\"/></LinearGradientBrush>",
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Border, Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Border", {
        L"Background@InactiveRunningIndicatorPointerOver:=<WindhawkBlur BlurAmount=\"40\" TintColor=\"#10ffffff\"/>",
        L"CornerRadius=12",
        L"BorderBrush@InactiveRunningIndicatorPointerOver:=<LinearGradientBrush EndPoint=\"1,0\" StartPoint=\"0,0\"><GradientStop Color=\"#80ffffff\" Offset=\"0.0\"/><GradientStop Color=\"{ThemeResource SurfaceStrokeColorDefault}\" Offset=\"0.55\"/><GradientStop Color=\"#80ffffff\" Offset=\"1\"/></LinearGradientBrush>",
        L"BorderThickness@InactiveRunningIndicatorPointerOver=1"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border#BackgroundElement, Taskbar.TaskListLabeledButtonPanel@CommonStates > Border#BackgroundElement", {
        L"CornerRadius=15",
        L"Margin=0,5.5,0,5.5",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground > Grid > Rectangle#BackgroundFill", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon", {
        L"Background:=<WindhawkBlur BlurAmount=\"10\" TintColor=\"#40ffffff\"/>",
        L"CornerRadius=12",
        L"Margin=2",
        L"Padding=2",
        L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,0\" StartPoint=\"0,0\"><GradientStop Color=\"#80ffffff\" Offset=\"0.0\"/><GradientStop Color=\"{ThemeResource SurfaceStrokeColorDefault}\" Offset=\"0.55\"/><GradientStop Color=\"#80ffffff\" Offset=\"1\"/></LinearGradientBrush>",
        L"BorderThickness=2"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#ConfirmatorMainGrid", {
        L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
        L"CornerRadius=24",
        L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
```

```
      L"BorderThickness=2",
      L"Margin=0,0,0,10"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid.Border#ConfirmatorMainGrid", {
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalTrackRect", {
      L"Fill=#20ffffff",
      L"RadiusX=12",
      L"RadiusY=12",
      L"Height=18",
      L"Margin=0"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalDecreaseRect", {
      L"Fill:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColor}\" />",
      L"RadiusX=12",
      L"RadiusY=12",
      L"Height=18"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#VolumeConfirmator", {
      L"Padding=8,0,8,0"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#BrightnessConfirmator", {
      L"Padding=8,0,8,0"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.TextBlock#volumeLevelText", {
      L"Foreground=White"}},
}, {
   L"IconBackground=<LinearGradientBrush StartPoint=\"0.47,-0.29\"
EndPoint=\"0.50,1.29\"><GradientStop Offset=\"0.18\" Color=\"#2F2F2F\"/><GradientStop
Offset=\"0.3\" Color=\"#292929\"/><GradientStop Offset=\"0.5\"
Color=\"#141414\"/><GradientStop Offset=\"0.68\" Color=\"#080808\"/><GradientStop
Offset=\"0.81\" Color=\"#000000\"/></LinearGradientBrush>",
   L"IconBorder=<LinearGradientBrush StartPoint=\"0.04,-0.14\"
EndPoint=\"1.22,1.10\"><GradientStop Offset=\"0.18\" Color=\"#4FFFFFFF\"/><GradientStop
Offset=\"0.34\" Color=\"#661D1D1D\"/><GradientStop Offset=\"0.63\"
Color=\"#00000000\"/><GradientStop Offset=\"0.72\" Color=\"#662D2D2D\"/><GradientStop
Offset=\"0.84\" Color=\"#4FFFFFFF\"/></LinearGradientBrush>",
}};

const Theme g_themeOS26_Liquid_Glass_variant_DarkMacDockCompact = {{
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid@DockingStates", {
      L"Tag=horizontal",
      L"Tag@DockedLeft=vertical",
      L"Tag@DockedRight=vertical",
      L"Tag=>taskbarDock"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
      L"Width={{taskbarDock==`vertical`?skip():`Auto`}}",
      L"MinWidth:=100",
      L"MaxWidth={{containerGridWidth>0?containerGridWidth:`Infinity`}}",
      L"Grid.Column=1",
      L"Transitions:=<TransitionCollection><RepositionThemeTransition
IsStaggeringEnabled=\"False\"/></TransitionCollection>",
      L"Height={{taskbarDock==`vertical`?skip():70}}",
      L"MaxHeight=80",
      L"HorizontalAlignment=Center"}},
   ThemeTargetStyles{L"Grid#IconPanel > Image#Icon,
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
      L"Height=24",
      L"Width=24",
      L"Margin=3,0,-3,0"}},
   ThemeTargetStyles{L"Taskbar.TaskListButtonPanel", {
      L"Width=50",
      L"Height=65",
      L"Margin=0,-2,0,2"}},
```

```
   ThemeTargetStyles{L"Grid#IconPanel, Taskbar.TaskListLabeledButtonPanel", {
      L"Width=50",
      L"Height=65",
      L"Margin=0,-2,0,2"}},
   ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl > Grid >
SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel", {
      L"Width=50",
      L"Height=65",
      L"Margin=0,-2,0,2"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
      L"Margin=0,8,0,2",
      L"Padding=20,0,20,0",
      L"BorderBrush=#40FFFFFF"}},
   ThemeTargetStyles{L"Grid#RootGrid > Taskbar.TaskbarBackground", {
      L"Margin=-20,0,-20,0"}},
   ThemeTargetStyles{L"Grid#RootGrid > Taskbar.TaskbarBackground > Grid", {
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"CornerRadius=20,0,0,20",
      L"BorderThickness=1,1,0,1",
      L"Width=Auto",
      L"BorderBrush=#40FFFFFF",
      L"Padding=-1"}},
   ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
      L"Fill=Transparent"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.FlyoutPresenter", {
      L"RequestedTheme=Dark",
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"BorderThickness=2",
      L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
      L"CornerRadius=33",
      L"Padding=2,3,2,3"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapPickerBorder", {
      L"RequestedTheme=Dark",
      L"Background:=Transparent",
      L"BorderBrush:=Transparent",
      L"BorderThickness=2",
      L"Margin=0"}},
   ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab >
Grid#ModalRootGrid > Border#BackgroundElement", {
      L"Background=Transparent",
      L"BorderThickness=0",
      L"CornerRadius=50"}},
   ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab >
Grid#ModalRootGrid > Border#BackgroundElement >
WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemList", {
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"BorderThickness=2",
      L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
      L"CornerRadius=50"}},
   ThemeTargetStyles{L"MenuFlyoutPresenter", {
      L"CornerRadius=20"}},
   ThemeTargetStyles{L"MenuFlyoutPresenter > Border", {
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"BorderThickness=2",
      L"CornerRadius=25",
      L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>"}},
```

```
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid >
SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView >
Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid,
SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid >
SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView >
Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid", {
        L"Height=44",
        L"Width=44",
        L"Margin=25,2,6,0",
        L"CornerRadius=13",
        L"Background=transparent",
        L"BorderBrush:=lightgray",
        L"BorderThickness=3"}},
    ThemeTargetStyles{L"SystemTray.LanguageTextIconContent >
Windows.UI.Xaml.Controls.Grid#ContainerGrid", {
        L"Height=44",
        L"Width=44",
        L"Margin=-6,4,-3,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2",
        L"CornerRadius=13"}},
    ThemeTargetStyles{L"SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel >
ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid >
Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid,
SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid >
ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent >
Grid#ContainerGrid", {
        L"Height=44",
        L"Width=44",
        L"Margin=-3,4,8,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2",
        L"CornerRadius=13"}},
    ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack", {
        L"Visibility=1",
        L"Height=44",
        L"Width=44",
        L"Margin=2,4,2,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2",
        L"CornerRadius=13"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid >
ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[1] >
SystemTray.IconView > Grid > Grid", {
        L"Visibility=Visible",
        L"Height=44",
        L"Width=44",
        L"Margin=2,4,2,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2",
        L"CornerRadius=13"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid >
ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[2] >
SystemTray.IconView > Grid > Grid", {
        L"Visibility=Collapsed",
        L"Height=44",
```

```
        L"Width=44",
        L"Margin=2,4,2,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2",
        L"CornerRadius=13"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[3] > SystemTray.IconView > Grid > Grid", {
        L"Visibility=Visible",
        L"Height=44",
        L"Width=44",
        L"Margin=2,4,2,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2",
        L"CornerRadius=13"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.BatteryIconContent > Grid#ContainerGrid > StackPanel, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.BatteryIconContent > Grid#ContainerGrid > StackPanel", {
        L"HorizontalAlignment=Center"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock > TextBlock", {
        L"FontSize=30"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock", {
        L"FontSize=32",
        L"Margin=-1,0,0,0"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock", {
        L"VerticalAlignment=Center",
        L"HorizontalAlignment=Center"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.OmniButton#NotificationCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.OmniButton#NotificationCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock#LanguageInnerTextBlock > TextBlock#InnerTextBlock", {
        L"FontSize=16",
        L"FontWeight=Bold"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent", {
        L"FontWeight=bold",
        L"FontSize=32"}},
```

```
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
        L"Height=70",
        L"Grid.Column=2",
        L"Width=Auto",
        L"HorizontalAlignment=Left",
        L"Margin=0,-2,0.5,2"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"Margin=0,0,0.5,0",
        L"Height=60",
        L"VerticalAlignment=Bottom",
        L"Padding=0",
        L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
        L"BorderBrush=#40FFFFFF",
        L"BorderThickness=0,1,1,1",
        L"CornerRadius=0,20,20,0",
        L"Visibility=Visible"}},
    ThemeTargetStyles{L"SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView > Grid#ContainerGrid > Border#BackgroundBorder", {
        L"Background=transparent",
        L"BorderBrush=#40FFFFFF",
        L"BorderThickness=2,0,0,0",
        L"Padding=0",
        L"CornerRadius=0",
        L"Height=40"}},
    ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
        L"FontWeight=Bold",
        L"Margin=-2,9,2,-9",
        L"Foreground=White",
        L"Visibility=visible",
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"-9\"/>",
        L"FontSize=13",
        L"FontFamily=vivo Sans EN VF"}},
    ThemeTargetStyles{L"TextBlock#TimeInnerTextBlock", {
        L"Foreground=white",
        L"Width=Auto",
        L"FontWeight=Bold",
        L"FontSize=15",
        L"Margin=0,-4,5,4"}},
    ThemeTargetStyles{L"SystemTray.DateTimeIconContent > Grid#ContainerGrid", {
        L"CornerRadius=13",
        L"BorderThickness=1.2",
        L"Background:=<LinearGradientBrush StartPoint=\"0.50,-1.50\" EndPoint=\"0.50,2.50\"><GradientStop Offset=\"0.48\" Color=\"#FF3A40\"/><GradientStop Offset=\"0.49\" Color=\"#141414\"/></LinearGradientBrush>",
        L"BorderBrush:=$IconBorder",
        L"Height=45",
        L"Margin=0,3,15,-1",
        L"Width=auto"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock", {
        L"FontSize=30"}},
```

```
   ThemeTargetStyles{L":root > ScrollViewer > ScrollContentPresenter > Border > Grid", {
      L"ColumnDefinitions:=<ColumnDefinitionCollection><ColumnDefinition
Width=\"*\"/><ColumnDefinition Width=\"Auto\"/><ColumnDefinition
Width=\"Auto\"/><ColumnDefinition Width=\"*\"/></ColumnDefinitionCollection>",
      L"ActualWidth=>containerGridWidth"}},
   ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid >
SystemTray.Stack#NonActivatableStack > Grid#Content > SystemTray.StackListView#IconStack >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon >
Grid#ContainerGrid > Border#BackgroundBorder, SystemTray.SystemTrayFrame >
Grid#SystemTrayFrameGrid > SystemTray.Stack#NonActivatableStack > Grid#Content >
SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Border#BackgroundBorder", {
      L"Background:=Transparent",
      L"BorderThickness=0"}},
   ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid >
Border#BackgroundBorder", {
      L"Background:=Transparent",
      L"BorderThickness=0"}},
   ThemeTargetStyles{L"SystemTray.Stack#MainStack", {
      L"Visibility=1"}},
   ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid >
SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon >
Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid >
SystemTray.AdaptiveTextBlock#Underlay > TextBlock#InnerTextBlock,
SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid >
SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon >
Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid >
SystemTray.AdaptiveTextBlock#Underlay > TextBlock#InnerTextBlock", {
      L"Foreground=white"}},
   ThemeTargetStyles{L"Taskbar.TaskListButtonPanel@CommonStates > Grid >
Border#BackgroundElement, Taskbar.TaskListButtonPanel@CommonStates >
Border#BackgroundElement", {
      L"CornerRadius=13",
      L"Margin=0,5.5,0,5.5",
      L"Background:=$IconBackground",
      L"BorderBrush:=$IconBorder",
      L"BorderThickness=1.2"}},
   ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid >
Rectangle#BackgroundStroke", {
      L"Fill:=<WindhawkBlur BlurAmount=\"3.5\" TintColor=\"#2D101010\"/>",
      L"Stroke:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
      L"StrokeThickness=5",
      L"RadiusX=14",
      L"RadiusY=14",
      L"Fill:=<<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>"}},
   ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid >
Rectangle#BackgroundFill", {
      L"Canvas.ZIndex=0",
      L"Fill:=",
      L"Stroke:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
      L"StrokeThickness=5",
      L"RadiusX=14",
      L"RadiusY=14"}},
   ThemeTargetStyles{L"Taskbar.FlyoutFrame >
Windows.UI.Xaml.Controls.Canvas#HoverFlyoutCanvas >
Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid >
Windows.UI.Xaml.Controls.ContentPresenter#HoverFlyoutContent >
Taskbar.TaskItemThumbnailList >
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskItemThumbnailListRepeater >
Taskbar.TaskItemThumbnailView > Windows.UI.Xaml.Controls.Grid >
Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
      L"VerticalAlignment=Bottom",
      L"Canvas.ZIndex=1",
      L"Background:=<WindhawkBlur BlurAmount=\"5\" TintColor=\"#761E1E1E\"/>",
      L"Height=25",
      L"CornerRadius=0,0,15,15",
```

```
    L"Margin=5,0,5,0"}},
  ThemeTargetStyles{L"Border#HoverFlyoutBackground", {
    L"Margin=4,36,4,0",
    L"Canvas.ZIndex=1",
    L"Width=Auto",
    L"Background:=Transparent",
    L"BorderThickness=0",
    L"CornerRadius=13"}},
  ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#ThumbBarRepeater >
Taskbar.ThumbBarButton#ThumbBarButton >
Windows.UI.Xaml.Controls.ContentPresenter#BorderElement", {
    L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#761E1E1E\"/>",
    L"Margin=0,-20,0,20"}},
  ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#IconsRepeater >
Windows.UI.Xaml.Controls.Image", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#CloseButton", {
    L"HorizontalAlignment=left",
    L"Grid.ColumnSpan=1",
    L"Grid.RowSpan=1",
    L"Canvas.ZIndex=1",
    L"CornerRadius=20",
    L"Width=28",
    L"Height=28",
    L"Margin=-18,40,15,-40",
    L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#80ffffff\"/>",
    L"Foreground=black",
    L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#761E1E1E\"/>"}},
  ThemeTargetStyles{L"Taskbar.FlyoutFrame >
Windows.UI.Xaml.Controls.Canvas#HoverFlyoutCanvas >
Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid >
Windows.UI.Xaml.Controls.ContentPresenter#HoverFlyoutContent >
Taskbar.TaskItemThumbnailList >
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskItemThumbnailListRepeater >
Taskbar.TaskItemThumbnailView > Windows.UI.Xaml.Controls.Grid >
Windows.UI.Xaml.Controls.TextBlock#DisplayNameTextBlock", {
    L"Grid.ColumnSpan=2",
    L"Grid.RowSpan=2",
    L"VerticalAlignment=bottom",
    L"HorizontalAlignment=Center",
    L"Margin=0,-5,0,5",
    L"Canvas.ZIndex=1"}},
  ThemeTargetStyles{L"SystemTray.NotifyIconView@CommonStates > Grid#ContainerGrid >
Border#BackgroundBorder", {
    L"CornerRadius=13",
    L"Background:=$IconBackground",
    L"BorderBrush:=$IconBorder",
    L"Margin=2",
    L"BorderThickness=1.2"}},
  ThemeTargetStyles{L"Border#OverflowFlyoutBackgroundBorder", {
    L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
    L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#60ffffff\"/>",
    L"BorderThickness=2",
    L"CornerRadius=32,32,30,30",
    L"Margin=-10"}},
  ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Rectangle#RunningIndicator,
Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Rectangle#RunningIndicator", {
    L"Fill:=#90ffffff",
    L"RadiusX=2",
    L"RadiusY=2",
```

```
        L"Margin=-2",
        L"Height=4",
        L"Width=4",
        L"Margin=8,0,0,1",
        L"Width@ActiveRunningIndicator=8",
        L"Fill@ActiveRunningIndicator=#60CDFF"}},
    ThemeTargetStyles{L"Grid#IconPanel > TextBlock#LabelControl,
Taskbar.TaskListLabeledButtonPanel > TextBlock#LabelControl", {
        L"Margin=4,0,0,0",
        L"Foreground=White"}},
    ThemeTargetStyles{L"Taskbar.SearchBoxButton", {
        L"Background:=<WindhawkBlur BlurAmount=\"60\" TintColor=\"#35ffffff\"/>",
        L"CornerRadius=20",
        L"Margin=2,6,2,6",
        L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,1\" StartPoint=\"0,0\"><GradientStop
Color=\"#E0ffffff\" Offset=\"0.0\"/><GradientStop Color=\"#20ffffff\"
Offset=\"0.5\"/><GradientStop Color=\"#A0ffffff\" Offset=\"1.0\"/></LinearGradientBrush>",
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"TextBlock#SearchBoxTextBlock", {
        L"FontSize=12",
        L"Foreground=White"}},
    ThemeTargetStyles{L"Grid", {
        L"RequestedTheme=2"}},
    ThemeTargetStyles{L"Taskbar.StartButton#StartButton", {
        L"Background:=<WindhawkBlur BlurAmount=\"60\" TintColor=\"#35ffffff\"/>",
        L"CornerRadius=20",
        L"Margin=2,6,2,6",
        L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,1\" StartPoint=\"0,0\"><GradientStop
Color=\"#E0ffffff\" Offset=\"0.0\"/><GradientStop Color=\"#20ffffff\"
Offset=\"0.5\"/><GradientStop Color=\"#A0ffffff\" Offset=\"1.0\"/></LinearGradientBrush>",
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"Border#MultiWindowElement", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"SystemTray.TextIconContent > Grid >
SystemTray.AdaptiveTextBlock#Base > TextBlock", {
        L"Foreground=White"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton", {
        L"Margin=-12,0,0,0"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl > Grid >
SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Grid >
Border#BackgroundElement, SearchUx.SearchUI.SearchButtonControl > Grid >
SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Border#BackgroundElement", {
        L"Margin=0,5.5,0,5.5",
        L"CornerRadius=13",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder"}},

    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.
Name=Task View]", {
        L"Background:=<WindhawkBlur BlurAmount=\"60\" TintColor=\"#35ffffff\"/>",
        L"CornerRadius=20",
        L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,1\" StartPoint=\"0,0\"><GradientStop
Color=\"#E0ffffff\" Offset=\"0.0\"/><GradientStop Color=\"#20ffffff\"
Offset=\"0.5\"/><GradientStop Color=\"#A0ffffff\" Offset=\"1.0\"/></LinearGradientBrush>",
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Border,
Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Border", {
        L"Background@InactiveRunningIndicatorPointerOver:=<WindhawkBlur BlurAmount=\"40\"
TintColor=\"#10ffffff\"/>",
        L"CornerRadius=13",
```

```
        L"BorderBrush@InactiveRunningIndicatorPointerOver:=<LinearGradientBrush EndPoint=\"1,0\" StartPoint=\"0,0\"><GradientStop Color=\"#80ffffff\" Offset=\"0.01\"/><GradientStop Color=\"{ThemeResource SurfaceStrokeColorDefault}\" Offset=\"0.55\"/><GradientStop Color=\"#80ffffff\" Offset=\"1\"/></LinearGradientBrush>",
        L"BorderThickness@InactiveRunningIndicatorPointerOver=1"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border#BackgroundElement, Taskbar.TaskListLabeledButtonPanel@CommonStates > Border#BackgroundElement", {
        L"CornerRadius=13",
        L"Margin=0,5.5,0,5.5",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"TaskbarTaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground > Grid > Rectangle#BackgroundFill", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon", {
        L"Background:=<WindhawkBlur BlurAmount=\"10\" TintColor=\"#40ffffff\"/>",
        L"CornerRadius=13",
        L"Margin=2",
        L"Padding=2",
        L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,0\" StartPoint=\"0,0\"><GradientStop Color=\"#80ffffff\" Offset=\"0.0\"/><GradientStop Color=\"{ThemeResource SurfaceStrokeColorDefault}\" Offset=\"0.55\"/><GradientStop Color=\"#80ffffff\" Offset=\"1\"/></LinearGradientBrush>",
        L"BorderThickness=2"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#ConfirmatorMainGrid", {
        L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
        L"CornerRadius=24",
        L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
        L"BorderThickness=2",
        L"Margin=0,0,0,10"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid.Border#ConfirmatorMainGrid", {
        L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalTrackRect", {
        L"Fill=#20ffffff",
        L"RadiusX=12",
        L"RadiusY=12",
        L"Height=18",
        L"Margin=0"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalDecreaseRect", {
        L"Fill:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColor}\" />",
        L"RadiusX=12",
        L"RadiusY=12",
        L"Height=18"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#VolumeConfirmator", {
        L"Padding=8,0,8,0"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#BrightnessConfirmator", {
        L"Padding=8,0,8,0"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.TextBlock#volumeLevelText", {
        L"Foreground=White"}},
}, {
    L"IconBackground=<LinearGradientBrush StartPoint=\"0.47,-0.29\" EndPoint=\"0.50,1.29\"><GradientStop Offset=\"0.18\" Color=\"#2F2F2F\"/><GradientStop Offset=\"0.3\" Color=\"#292929\"/><GradientStop Offset=\"0.5\" Color=\"#141414\"/><GradientStop Offset=\"0.68\" Color=\"#080808\"/><GradientStop Offset=\"0.81\" Color=\"#000000\"/></LinearGradientBrush>",
    L"IconBorder=<LinearGradientBrush StartPoint=\"0.04,-0.14\" EndPoint=\"1.22,1.10\"><GradientStop Offset=\"0.18\" Color=\"#4FFFFFFF\"/><GradientStop Offset=\"0.34\" Color=\"#661D1D1D\"/><GradientStop Offset=\"0.63\" Color=\"#00000000\"/><GradientStop Offset=\"0.72\" Color=\"#662D2D2D\"/><GradientStop Offset=\"0.84\" Color=\"#4FFFFFFF\"/></LinearGradientBrush>",
}};
```

```
const Theme g_themeOS26_Liquid_Glass_variant_DarkTaskbar = {{
   ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
      L"Height=80",
      L"MaxHeight=80",
      L"HorizontalAlignment=Stretch"}},
   ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
      L"VerticalAlignment=Center"}},
   ThemeTargetStyles{L"Grid#IconPanel > Image#Icon,
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
      L"Height=24",
      L"Width=24",
      L"Margin=5,0,-5,0"}},
   ThemeTargetStyles{L"Taskbar.TaskListButtonPanel", {
      L"Width=55",
      L"Height=70"}},
   ThemeTargetStyles{L"Grid#IconPanel, Taskbar.TaskListLabeledButtonPanel", {
      L"Width=55",
      L"Height=70"}},
   ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl > Grid >
SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel", {
      L"Width=55",
      L"Height=70"}},
   ThemeTargetStyles{L"Grid#RootGrid > Taskbar.TaskbarBackground", {
      L"Margin=-15,0,-15,0"}},
   ThemeTargetStyles{L"Grid#RootGrid > Taskbar.TaskbarBackground > Grid", {
      L"CornerRadius=20",
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"BorderThickness=1",
      L"BorderBrush=#40FFFFFF",
      L"Padding=-1"}},
   ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
      L"Fill=Transparent"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
      L"Visibility=Visible",
      L"Margin=0,8,0,2",
      L"Padding=20,0,20,0"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.FlyoutPresenter", {
      L"RequestedTheme=Dark",
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"BorderThickness=2",
      L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
      L"CornerRadius=33",
      L"Padding=2,3,2,3"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapPickerBorder", {
      L"RequestedTheme=Dark",
      L"Background:=Transparent",
      L"BorderBrush:=Transparent",
      L"BorderThickness=2",
      L"Margin=0"}},
   ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab >
Grid#ModalRootGrid > Border#BackgroundElement", {
      L"Background=Transparent",
      L"BorderThickness=0",
      L"CornerRadius=50"}},
```

```
   ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab > Grid#ModalRootGrid > Border#BackgroundElement > WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemList", {
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"BorderThickness=2",
      L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
      L"CornerRadius=50"}},
   ThemeTargetStyles{L"MenuFlyoutPresenter", {
      L"CornerRadius=20"}},
   ThemeTargetStyles{L"MenuFlyoutPresenter > Border", {
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"BorderThickness=2",
      L"CornerRadius=25",
      L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>"}},
   ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > Grid > SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid, ScrollViewer > ScrollContentPresenter > Border > Grid > SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid", {
      L"Height=35",
      L"CornerRadius=12",
      L"Background:=$IconBackground",
      L"BorderBrush:=$IconBorder",
      L"BorderThickness=1.2"}},
   ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
      L"Width=Auto",
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"CornerRadius=15",
      L"Margin=200,5,-420,-5",
      L"RenderTransform:=<TranslateTransform X=\"-435\" Y=\"-2\"/>",
      L"Padding=10,2",
      L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,1\" StartPoint=\"0,0\"><GradientStop Color=\"#50ffffff\" Offset=\"0.0\"/><GradientStop Color=\"#10ffffff\" Offset=\"0.5\"/><GradientStop Color=\"#30ffffff\" Offset=\"1.0\"/></LinearGradientBrush>",
      L"BorderThickness=2",
      L"Visibility=Visible"}},
   ThemeTargetStyles{L"Taskbar.TaskListButtonPanel@CommonStates > Grid > Border#BackgroundElement, Taskbar.TaskListButtonPanel@CommonStates > Border#BackgroundElement", {
      L"CornerRadius=15",
      L"Margin=0,5.5,0,5.5",
      L"Background:=$IconBackground",
      L"BorderBrush:=$IconBorder",
      L"BorderThickness=1.2"}},
   ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid > Rectangle#BackgroundStroke", {
      L"Fill:=<WindhawkBlur BlurAmount=\"3.5\" TintColor=\"#2D101010\"/>",
      L"Stroke:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
      L"StrokeThickness=5",
      L"RadiusX=14",
      L"RadiusY=14",
      L"Fill:=<<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>"}},
   ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid > Rectangle#BackgroundFill", {
      L"Canvas.ZIndex=0",
      L"Fill:=",
      L"Stroke:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
```

```
        L"StrokeThickness=5",
        L"RadiusX=14",
        L"RadiusY=14"}},
    ThemeTargetStyles{L"Taskbar.FlyoutFrame > Windows.UI.Xaml.Controls.Canvas#HoverFlyoutCanvas > Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid > Windows.UI.Xaml.Controls.ContentPresenter#HoverFlyoutContent > Taskbar.TaskItemThumbnailList > Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskItemThumbnailListRepeater > Taskbar.TaskItemThumbnailView > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"VerticalAlignment=Bottom",
        L"Canvas.ZIndex=1",
        L"Background:=<WindhawkBlur BlurAmount=\"5\" TintColor=\"#761E1E1E\"/>",
        L"Height=25",
        L"CornerRadius=0,0,15,15",
        L"Margin=5,0,5,0"}},
    ThemeTargetStyles{L"Border#HoverFlyoutBackground", {
        L"Margin=4,36,4,0",
        L"Canvas.ZIndex=1",
        L"Width=Auto",
        L"Background:=Transparent",
        L"BorderThickness=0",
        L"CornerRadius=15"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#IconsRepeater > Windows.UI.Xaml.Controls.Image", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#ThumbBarRepeater > Taskbar.ThumbBarButton#ThumbBarButton > Windows.UI.Xaml.Controls.ContentPresenter#BorderElement", {
        L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#761E1E1E\"/>",
        L"Margin=0,-20,0,20"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#CloseButton", {
        L"HorizontalAlignment=left",
        L"Grid.ColumnSpan=1",
        L"Grid.RowSpan=1",
        L"Canvas.ZIndex=1",
        L"CornerRadius=20",
        L"Width=28",
        L"Height=28",
        L"Margin=-18,40,15,-40",
        L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#80ffffff\"/>",
        L"Foreground=black",
        L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#761E1E1E\"/>"}},
    ThemeTargetStyles{L"Taskbar.FlyoutFrame > Windows.UI.Xaml.Controls.Canvas#HoverFlyoutCanvas > Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid > Windows.UI.Xaml.Controls.ContentPresenter#HoverFlyoutContent > Taskbar.TaskItemThumbnailList > Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskItemThumbnailListRepeater > Taskbar.TaskItemThumbnailView > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.TextBlock#DisplayNameTextBlock", {
        L"Grid.ColumnSpan=2",
        L"Grid.RowSpan=2",
        L"VerticalAlignment=bottom",
        L"HorizontalAlignment=Center",
        L"Margin=0,-5,0,5",
        L"Canvas.ZIndex=1"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView@CommonStates > Grid#ContainerGrid > Border#BackgroundBorder", {
        L"CornerRadius=12",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
```

```
        L"Margin=2",
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView@CommonStates > Grid#ContainerGrid > Border#BackgroundBorder", {
        L"CornerRadius=12",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"Margin=2",
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"Border#OverflowFlyoutBackgroundBorder", {
        L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
        L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#60ffffff\"/>",
        L"BorderThickness=2",
        L"CornerRadius=32,32,30,30",
        L"Margin=-10"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid > Grid > SystemTray.TextIconContent", {
        L"CornerRadius=15"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Rectangle#RunningIndicator, Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Rectangle#RunningIndicator", {
        L"Fill:=#90ffffff",
        L"RadiusX=3",
        L"RadiusY=3",
        L"Margin=-2",
        L"Height=6",
        L"Width=6",
        L"Margin=10,0,0,-2",
        L"Width@ActiveRunningIndicator=12",
        L"Fill@ActiveRunningIndicator=#60CDFF"}},
    ThemeTargetStyles{L"Grid#IconPanel > TextBlock#LabelControl, Taskbar.TaskListLabeledButtonPanel > TextBlock#LabelControl", {
        L"Margin=4,0,0,0",
        L"Foreground=White"}},
    ThemeTargetStyles{L"Taskbar.SearchBoxButton", {
        L"Background:=<WindhawkBlur BlurAmount=\"60\" TintColor=\"#35ffffff\"/>",
        L"CornerRadius=20",
        L"Margin=2,6,2,6",
        L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,1\" StartPoint=\"0,0\"><GradientStop Color=\"#E0ffffff\" Offset=\"0.0\"/><GradientStop Color=\"#20ffffff\" Offset=\"0.5\"/><GradientStop Color=\"#A0ffffff\" Offset=\"1.0\"/></LinearGradientBrush>",
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"TextBlock#SearchBoxTextBlock", {
        L"FontSize=12",
        L"Foreground=White"}},
    ThemeTargetStyles{L"Grid", {
        L"RequestedTheme=2"}},

    ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton[AutomationProperties.Name=Copilot] > Grid#IconPanel > Border#BackgroundElement, Taskbar.TaskListButton#TaskListButton[AutomationProperties.Name=Copilot] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border#BackgroundElement", {
        L"Background:=$IconBackground"}},
    ThemeTargetStyles{L"Taskbar.StartButton#StartButton", {
        L"Background:=<WindhawkBlur BlurAmount=\"60\" TintColor=\"#35ffffff\"/>",
        L"CornerRadius=20",
        L"Margin=2,6,2,6",
        L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,1\" StartPoint=\"0,0\"><GradientStop Color=\"#E0ffffff\" Offset=\"0.0\"/><GradientStop Color=\"#20ffffff\" Offset=\"0.5\"/><GradientStop Color=\"#A0ffffff\" Offset=\"1.0\"/></LinearGradientBrush>",
```

```
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"Border#MultiWindowElement", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"TextBlock#TimeInnerTextBlock", {
        L"Foreground=White",
        L"FontSize=18",
        L"FontFamily=Quantico",
        L"Margin=0",
        L"Padding=0",
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"1\"/>"}},
    ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
        L"Foreground=White",
        L"Visibility=Collapsed",
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"-9\"/>",
        L"FontSize=11",
        L"FontFamily=vivo Sans EN VF"}},
    ThemeTargetStyles{L"SystemTray.TextIconContent > Grid >
SystemTray.AdaptiveTextBlock#Base > TextBlock", {
        L"Foreground=White"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton", {
        L"Margin=-12,0,0,0"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl > Grid >
SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Grid >
Border#BackgroundElement, SearchUx.SearchUI.SearchButtonControl > Grid >
SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Border#BackgroundElement", {
        L"Margin=0,5.5,0,5.5",
        L"CornerRadius=15",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.
Name=Task View]", {
        L"Background:=<WindhawkBlur BlurAmount=\"60\" TintColor=\"#35ffffff\"/>",
        L"CornerRadius=20",
        L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,1\" StartPoint=\"0,0\"><GradientStop
Color=\"#F0ffffff\" Offset=\"0.0\"/><GradientStop Color=\"#20ffffff\"
Offset=\"0.5\"/><GradientStop Color=\"#A0ffffff\" Offset=\"1.0\"/></LinearGradientBrush>",
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Border,
Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Border", {
        L"Background@InactiveRunningIndicatorPointerOver:=<WindhawkBlur BlurAmount=\"40\"
TintColor=\"#10ffffff\"/>",
        L"CornerRadius=12",
        L"BorderBrush@InactiveRunningIndicatorPointerOver:=<LinearGradientBrush
EndPoint=\"1,0\" StartPoint=\"0,0\"><GradientStop Color=\"#80ffffff\"
Offset=\"0.0\"/><GradientStop Color=\"{ThemeResource SurfaceStrokeColorDefault}\"
Offset=\"0.55\"/><GradientStop Color=\"#80ffffff\" Offset=\"1\"/></LinearGradientBrush>",
        L"BorderThickness@InactiveRunningIndicatorPointerOver=1"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border#BackgroundElement,
Taskbar.TaskListLabeledButtonPanel@CommonStates > Border#BackgroundElement", {
        L"CornerRadius=15",
        L"Margin=0,5.5,0,5.5",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.2"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground >
Grid > Rectangle#BackgroundStroke", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground >
Grid > Rectangle#BackgroundFill", {
```

```
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon", {
        L"Background:=<WindhawkBlur BlurAmount=\"10\" TintColor=\"#40ffffff\"/>",
        L"CornerRadius=12",
        L"Margin=2",
        L"Padding=2",
        L"BorderBrush:=<LinearGradientBrush EndPoint=\"1,0\" StartPoint=\"0,0\"><GradientStop
Color=\"#80ffffff\" Offset=\"0.0\"/><GradientStop Color=\"{ThemeResource
SurfaceStrokeColorDefault}\" Offset=\"0.55\"/><GradientStop Color=\"#80ffffff\"
Offset=\"1\"/></LinearGradientBrush>",
        L"BorderThickness=2"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#ConfirmatorMainGrid", {
        L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
        L"CornerRadius=24",
        L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
        L"BorderThickness=2",
        L"Margin=0,0,0,10"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid.Border#ConfirmatorMainGrid", {
        L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalTrackRect", {
        L"Fill=#20ffffff",
        L"RadiusX=12",
        L"RadiusY=12",
        L"Height=18",
        L"Margin=0"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalDecreaseRect", {
        L"Fill=#ff7060",
        L"RadiusX=12",
        L"RadiusY=12",
        L"Height=18"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#VolumeConfirmator", {
        L"Padding=8,0,8,0"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#BrightnessConfirmator", {
        L"Padding=8,0,8,0"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.TextBlock#volumeLevelText", {
        L"Foreground=White"}},
}, {
    L"IconBackground=<LinearGradientBrush StartPoint=\"0.47,-0.29\"
EndPoint=\"0.50,1.29\"><GradientStop Offset=\"0.18\" Color=\"#2F2F2F\"/><GradientStop
Offset=\"0.3\" Color=\"#292929\"/><GradientStop Offset=\"0.5\"
Color=\"#141414\"/><GradientStop Offset=\"0.68\" Color=\"#080808\"/><GradientStop
Offset=\"0.81\" Color=\"#000000\"/></LinearGradientBrush>",
    L"IconBorder=<LinearGradientBrush StartPoint=\"0.25,-0.20\"
EndPoint=\"0.99,1.39\"><GradientStop Offset=\"0.11\" Color=\"#50FFFFFF\"/><GradientStop
Offset=\"0.3\" Color=\"#631C1C1C\"/><GradientStop Offset=\"0.62\"
Color=\"#591C1C1C\"/><GradientStop Offset=\"0.77\"
Color=\"#50FFFFFF\"/></LinearGradientBrush>",
}};

const Theme g_themeFrostyGlass = {{
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
        L"Margin=0,0,0,4",
        L"BorderThickness=$BorderThickness",
        L"BorderBrush:=$BorderBrush",
        L"CornerRadius=$CornerRadius",
        L"Background:=$Background",
        L"Padding=2,0,1.5,0"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
        L"Margin=3.5",
```

```
    L"Background:=$Background",
    L"CornerRadius=$CornerRadius"}},
  ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
    L"Margin=12,0,12,4",
    L"Background:=$Background",
    L"BorderThickness=$BorderThickness",
    L"BorderBrush:=$BorderBrush",
    L"CornerRadius=$CornerRadius",
    L"Padding=-1,0,-1,0"}},
  ThemeTargetStyles{L"SystemTray.ChevronIconView", {
    L"Padding=4.5,4,2,4",
    L"CornerRadius=7"}},
  ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon", {
    L"Padding=$TrayPadding",
    L"CornerRadius=7"}},
  ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton", {
    L"Padding=$TrayPadding",
    L"CornerRadius=7"}},
  ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton", {
    L"Padding=2,4,4.5,4",
    L"CornerRadius=7"}},
  ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Grid >
ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.IconView#SystemTrayIcon > Grid", {
    L"Padding=2,2,5,4",
    L"CornerRadius=7",
    L"HorizontalAlignment=Center"}},
  ThemeTargetStyles{L"SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid >
ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent >
Grid#ContainerGrid", {
    L"Padding=$TrayPadding",
    L"CornerRadius=7"}},
  ThemeTargetStyles{L"SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel >
ContentPresenter > SystemTray.IconView#SystemTrayIcon", {
    L"Padding=$TrayPadding",
    L"CornerRadius=7"}},
  ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack", {
    L"Visibility=1"}},
  ThemeTargetStyles{L"Taskbar.Gripper#GripperControl", {
    L"Width=Auto"}},
  ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
    L"HorizontalAlignment=Right",
    L"CornerRadius=$CornerRadius",
    L"Width=Auto"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#AugmentedEntryPointContentGrid", {
    L"Margin=4,0,0,0",
    L"HorizontalAlignment=Center"}},
  ThemeTargetStyles{L"TextBlock#TimeInnerTextBlock", {
    L"FontSize=13",
    L"FontFamily=Segoe UI VF",
    L"Margin=0",
    L"Padding=0",
    L"RenderTransform:=<TranslateTransform X=\"5\" Y=\"2\" />",
    L"Width=Auto"}},
  ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
    L"Visibility=1",
    L"RenderTransform:=<TranslateTransform X=\"4\" Y=\"0\" />",
```

```
        L"FontSize=11",
        L"FontFamily=Segoe UI VF"}},
    ThemeTargetStyles{L"TextBlock#InnerTextBlock[Text=\uE971]", {
        L"Text=\uED14"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#ConfirmatorMainGrid", {
        L"BorderThickness=$BorderThickness",
        L"BorderBrush:=$BorderBrush",
        L"Background:=$Background",
        L"CornerRadius=10"}},
    ThemeTargetStyles{L"TextBlock#SearchBoxTextBlock", {
        L"Text=Search",
        L"FontSize=10",
        L"FontFamily=vivo Sans EN VF"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Grid >
ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.IconView#SystemTrayIcon > Grid > Grid > SystemTray.TextIconContent", {
        L"Visibility=1"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button", {
        L"BorderThickness=1",
        L"CornerRadius=7"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#OverflowFlyoutBackgroundBorder",
{
        L"BorderThickness=$BorderThickness",
        L"BorderBrush:=$BorderBrush",
        L"Background:=$Background",
        L"CornerRadius:=$CornerRadius"}},

    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopB
arElement#VirtualDesktopBar", {
        L"CornerRadius=$CornerRadius",
        L"Background:=$Background"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#BackgroundDimmingLayer", {
        L"Background:=$Background",
        L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapBarBorder", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness=$BorderThickness",
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"10\" />",
        L"Margin=0,0,0,-10",
        L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapPickerBorder", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness=$BorderThickness",
        L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SearchPillBackgroundElement", {
        L"BorderBrush:=$ElementBorderBrush",
        L"CornerRadius=$ElementCornerRadius",
        L"BorderThickness=$ElementBorderThickness",
        L"Margin=0"}},
    ThemeTargetStyles{L"Taskbar.TaskbarExtensionElement", {
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"0\" />"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"0\" />",
        L"Padding=1.5,4,1.5,4"}},
```

```
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.ToolTip > Windows.UI.Xaml.Controls.ContentPresenter#LayoutRoot", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness:=$BorderThickness",
        L"CornerRadius:=$CornerRadius"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl", {
        L"Width=Auto",
        L"Margin=-1,0,-1,0",
        L"Padding=$TrayPadding",
        L"CornerRadius=7"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopBarElement > Windows.UI.Xaml.Controls.Grid#GridElement > Windows.UI.Xaml.Controls.Border#VirtualDesktopSwitcherBackground", {
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness=$BorderThickness",
        L"Background:=$Background",
        L"CornerRadius=$CornerRadius"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemListViewItem > Grid > Border", {
        L"Background:=$Background",
        L"CornerRadius=$CornerRadius",
        L"BorderThickness:=$BorderThickness",
        L"BorderBrush:=$BorderBrush"}},
    ThemeTargetStyles{L"Border#VirtualDesktopBarBackground", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness=$BorderThickness",
        L"CornerRadius=$CornerRadius",
        L"Margin=0,0,0,-3"}},
    ThemeTargetStyles{L"Border#HoverFlyoutBackground", {
        L"BorderBrush:=$BorderBrush",
        L"CornerRadius=$CornerRadius",
        L"BorderThickness=$BorderThickness",
        L"Background:=$Background"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid > Rectangle#BackgroundFill", {
        L"Fill:=Transparent"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputSwitcher > ContentControl > ContentPresenter > Grid", {
        L"BorderBrush:=$BorderBrush",
        L"CornerRadius=$CornerRadius",
        L"BorderThickness=$BorderThickness",
        L"Background:=$Background"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton", {
        L"CornerRadius=7",
        L"Padding=$TrayPadding",
        L"Visibility=0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundElement, Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"CornerRadius=7",
        L"Padding=$TrayPadding",
        L"Visibility=0"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid@DockingStates", {
        L"Tag=horizontal",
```

```
        L"Tag@DockedLeft=vertical",
        L"Tag@DockedRight=vertical",
        L"Tag=>taskbarDock"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
        L"HorizontalAlignment=Center",
        L"Width={{taskbarDock==`vertical`?skip():`Auto`}}",
        L"Padding=1,0,1,0"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"Visibility=0"}},
    ThemeTargetStyles{L"Grid#IconPanel > Border#BackgroundElement,
Taskbar.TaskListLabeledButtonPanel > Border#BackgroundElement", {
        L"Visibility=0"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.
AutomationId=StartButton]", {
        L"Visibility=0"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputS
witcher > ContentControl > ContentPresenter > Grid > Grid", {
        L"Background:=Transparent",
        L"BorderBrush:=Transparent"}},
    ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView > Grid@CommonStates >
Border#BackgroundBorder", {
        L"CornerRadius:=$CornerRadius"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopEl
ementThemed > Windows.UI.Xaml.Controls.Grid#MainGrid >
Windows.UI.Xaml.Controls.Border#MainBorder", {
        L"CornerRadius:=$CornerRadius",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness:=$BorderThickness",
        L"Background:=$Background",
        L"Margin=0,0,0,0"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopEl
ementThemed > Windows.UI.Xaml.Controls.Grid#MainGrid >
Windows.UI.Xaml.Controls.Border#BorderHighlight", {
        L"CornerRadius:=$CornerRadius",
        L"Visibility=1"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopEl
ementThemed > Windows.UI.Xaml.Controls.Grid#MainGrid >
Windows.UI.Xaml.Controls.Border#ActiveDesktopPill", {
        L"CornerRadius:=$CornerRadius"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.NewVirtualDeskt
opElementThemed#NewVirtualDesktopButtonThemed >
Windows.UI.Xaml.Controls.Grid#MainGrid > Windows.UI.Xaml.Controls.Border#MainBorder", {
        L"CornerRadius:=$CornerRadius"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.NewVirtualDeskt
opElementThemed#NewVirtualDesktopButtonThemed >
Windows.UI.Xaml.Controls.Grid#MainGrid >
Windows.UI.Xaml.Controls.Border#BorderHighlight", {
        L"CornerRadius:=$CornerRadius",
        L"Visibility=1"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.MenuFlyoutPresenter >
Windows.UI.Xaml.Controls.Border", {
        L"Background:=$Background",
        L"CornerRadius:=$CornerRadius",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness:=$BorderThickness"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#BackgroundControl >
Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Shapes.Rectangle#BackgroundFill", {
        L"Visibility=Collapsed"}},
```

```
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab > Windows.UI.Xaml.Controls.Grid#ModalRootGrid > Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"Background=Transparent",
        L"BorderThickness=0",
        L"CornerRadius:=$CornerRadius"}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab > Windows.UI.Xaml.Controls.Grid#ModalRootGrid > Windows.UI.Xaml.Controls.Border#BackgroundElement > WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemList", {
        L"Background:=$Background",
        L"BorderThickness:=$BorderThickness",
        L"BorderBrush:=$BorderBrush",
        L"CornerRadius:=$CornerRadius"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness:=$BorderThickness",
        L"CornerRadius:=10"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground > Grid > Rectangle#BackgroundFill", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
        L"Margin=0"}},
    ThemeTargetStyles{L"Taskbar.Gripper#GripperControl", {
        L"Width=Auto",
        L"MinWidth=24"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"Background:=$Background",
        L"CornerRadius:=10",
        L"BorderThickness:=$BorderThickness",
        L"BorderBrush:=$BorderBrush",
        L"BackgroundSizing=InnerBorderEdge"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchIconButton > SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundElement, SearchUx.SearchUI.SearchIconButton > SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Windows.UI.Xaml.Controls.Border#BackgroundElement", {
        L"CornerRadius:=7",
        L"Padding=$TrayPadding"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SearchPillBackgroundElement", {
        L"CornerRadius:=7",
        L"Padding=$TrayPadding"}},
    ThemeTargetStyles{L"Border#TaskbarSearchBackground", {
        L"CornerRadius=$CornerRadius",
        L"Background:=$Background",
        L"BorderThickness=$BorderThickness",
        L"BorderBrush:=$BorderBrush"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView > Grid#ContainerGrid > Border#BackgroundBorder", {
        L"CornerRadius=7"}},
}, {
    L"Background=<WindhawkBlur BlurAmount=\"20\" TintColor=\"{ThemeResource SystemChromeDarkColor}\" TintOpacity=\"0.15\" />",
    L"BorderBrush2=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"{ThemeResource SystemChromeHighColor}\" Offset=\"0.0\" /><GradientStop Color=\"{ThemeResource SystemChromeLowColor}\" Offset=\"0.25\" /><GradientStop Color=\"{ThemeResource SystemChromeHighColor}\" Offset=\"1\" /></LinearGradientBrush>",
```

```
    L"BorderThickness=1",
    L"CornerRadius=10",
    L"BorderBrush=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#50808080\" Offset=\"0.0\" /><GradientStop Color=\"#50404040\" Offset=\"0.25\" /><GradientStop Color=\"#50808080\" Offset=\"1\" /></LinearGradientBrush>",
    L"Background2=<AcrylicBrush TintColor=\"{ThemeResource SystemChromeAltHighColor}\" TintOpacity=\"0.3\" FallbackColor=\"{ThemeResource SystemChromeAltHighColor}\" />",
    L"TrayPadding=2,4,2,4",
    L"ElementBG=<SolidColorBrush Color=\"{ThemeResource SystemChromeAltHighColor}\" Opacity=\"0.3\" />",
    L"ElementBorderThickness=1",
    L"ElementBorderBrush=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#50808080\" Offset=\"1\" /><GradientStop Color=\"#50606060\" Offset=\"0.15\" /></LinearGradientBrush>",
    L"ElementCornerRadius=10",
    L"Background3=Transparent",
    L"BorderBrush3=Transparent",
}};

const Theme g_themeFrostedAcrylic = {{
    ThemeTargetStyles{L":root > ScrollViewer > ScrollContentPresenter > Border > Grid", {
        L"ActualWidth=>containerGridWidth"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid@DockingStates", {
        L"Tag=horizontal",
        L"Tag@DockedLeft=vertical",
        L"Tag@DockedRight=vertical",
        L"Tag=>taskbarDock"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
        L"Width={{taskbarDock==`vertical`?skip():`Auto`}}",
        L"MinWidth:=100",
        L"MaxWidth={{containerGridWidth>0?containerGridWidth:`Infinity`}}",
        L"Margin=0",
        L"Background:=Transparent",
        L"HorizontalAlignment=Stretch",
        L"Padding=0"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
        L"Width=Auto",
        L"MinWidth=100",
        L"HorizontalAlignment=Stretch",
        L"Margin=$SideMargin,2,$SideMargin,3",
        L"BorderThickness=$BorderThickness",
        L"BorderBrush:=$BorderBrush",
        L"CornerRadius=$CornerRadius",
        L"Background:=$Background",
        L"Padding=0"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#BackgroundControl > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Shapes.Rectangle#BackgroundFill", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#BackgroundControl > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Shapes.Rectangle#BackgroundStroke", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
        L"Width=Auto",
        L"HorizontalAlignment=Right",
        L"Height=40",
        L"VerticalAlignment=Center",
        L"Margin=0,0,$TrayRightMargin,0",
        L"Background:=$Background"}},
```

```
  ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
    L"Background:=Transparent",
    L"CornerRadius=6"}},
  ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl", {
    L"Width=48",
    L"MinWidth=48",
    L"MaxWidth=48",
    L"Margin=0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton", {
    L"Width=48",
    L"MinWidth=48",
    L"MaxWidth=48",
    L"Height=60",
    L"MinHeight=60",
    L"MaxHeight=60",
    L"Padding=0",
    L"Margin=0",
    L"HorizontalAlignment=Center",
    L"VerticalAlignment=Center"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton > Grid#IconPanel, Taskbar.TaskListButton >
Taskbar.TaskListLabeledButtonPanel", {
    L"Height=60",
    L"MinHeight=60",
    L"MaxHeight=60",
    L"Padding=2,4,2,4",
    L"Margin=0",
    L"HorizontalAlignment=Center",
    L"VerticalAlignment=Center"}},
  ThemeTargetStyles{L"Grid#IconPanel > TextBlock, Taskbar.TaskListLabeledButtonPanel >
TextBlock, Taskbar.TaskListButtonPanel > Grid > TextBlock, Taskbar.TaskListButtonPanel >
TextBlock, SearchUx.SearchUI.SearchButtonRootGrid > Grid > TextBlock,
SearchUx.SearchUI.SearchButtonRootGrid > TextBlock", {
    L"Visibility=Collapsed",
    L"Margin=0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButtonPanel > Grid > TextBlock", {
    L"Visibility=Collapsed",
    L"Margin=0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButtonPanel > Grid > StackPanel > TextBlock", {
    L"Visibility=Collapsed",
    L"Opacity=0",
    L"Width=0",
    L"MinWidth=0",
    L"MaxWidth=0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButtonPanel > Grid > Grid > TextBlock", {
    L"Visibility=Collapsed",
    L"Opacity=0",
    L"Width=0",
    L"MinWidth=0",
    L"MaxWidth=0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButtonPanel > Grid > ContentPresenter > TextBlock", {
    L"Visibility=Collapsed",
    L"Opacity=0",
    L"Width=0",
    L"MinWidth=0",
    L"MaxWidth=0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButtonPanel > Grid > StackPanel > TextBlock,
Taskbar.TaskListButtonPanel > StackPanel > TextBlock", {
```

```
        L"Visibility=Collapsed",
        L"Opacity=0",
        L"Width=0",
        L"MinWidth=0",
        L"MaxWidth=0"}},
    ThemeTargetStyles{L"Taskbar.OverflowToggleButton#OverflowButton > Taskbar.TaskListButtonPanel#OverflowToggleButtonRootPanel > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.FontIcon#FontIcon > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.TextBlock, Taskbar.OverflowToggleButton#OverflowButton > Taskbar.TaskListButtonPanel#OverflowToggleButtonRootPanel > Windows.UI.Xaml.Controls.FontIcon > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.TextBlock", {
        L"Text=\uE8F9"}},
    ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack", {
        L"Visibility=Visible"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#MostRecentlyUsedDivider", {
        L"Height=32",
        L"Width=2"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#LeftOverflowButtonDivider", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#RightOverflowButtonDivider", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#OverflowFlyoutBackgroundBorder", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness=$BorderThickness",
        L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#ConfirmatorMainGrid", {
        L"CornerRadius=22",
        L"BorderThickness=$BorderThickness",
        L"BorderBrush:=$BorderBrush",
        L"Background:=$Background"}},

    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.VirtualDesktopBarElement#VirtualDesktopBar > Grid > Border", {
        L"BorderThickness=$BorderThickness",
        L"BorderBrush:=$BorderBrush",
        L"Background:=$Background",
        L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapBarBorder", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"CornerRadius=$CornerRadius",
        L"BorderThickness=$BorderThickness"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Border#SnapPickerBorder", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"CornerRadius=$CornerRadius",
        L"BorderThickness=$BorderThickness"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.ToolTip > Windows.UI.Xaml.Controls.ContentPresenter#LayoutRoot", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness=$BorderThickness",
        L"CornerRadius=1"}},
    ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView > Grid#BackgroundGrid > Border#BackgroundElement", {
        L"Background:=$Background",
```

```
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness=$BorderThickness",
        L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView > Border#BackgroundElement", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness=$BorderThickness",
        L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"Taskbar.TaskItemThumbnailView > Grid >
Windows.UI.Xaml.Shapes.Rectangle#BackgroundFill", {
        L"Fill:=$Background",
        L"Visibility=Visible"}},
    ThemeTargetStyles{L"Popup > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness=$BorderThickness",
        L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"Popup > Grid > Border", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness=$BorderThickness",
        L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab >
Grid#ModalRootGrid > Border#BackgroundElement", {
        L"Background=Transparent",
        L"BorderThickness=0",
        L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.Switcher.AltTab >
Grid#ModalRootGrid > Border#BackgroundElement >
WindowsInternal.ComposableShell.Experiences.Switcher.SwitchItemList", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness=$BorderThickness",
        L"CornerRadius=$CornerRadius"}},

    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputS
witcher", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness=$BorderThickness",
        L"CornerRadius=$CornerRadius",
        L"Padding=6"}},

    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputS
witcher > ContentControl > ContentPresenter > Grid", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness=$BorderThickness",
        L"CornerRadius=$CornerRadius"}},

    ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputS
witcher > ContentControl > ContentPresenter > Grid > Border", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
        L"BorderThickness=$BorderThickness",
        L"CornerRadius=$CornerRadius"}},
    ThemeTargetStyles{L"Popup >
Windows.UI.Xaml.Controls.Border#InputSwitcherPopupBorder", {
        L"Background:=$Background",
        L"BorderBrush:=$BorderBrush",
```

```
      L"BorderThickness=$BorderThickness",
      L"CornerRadius=$CornerRadius"}},
   ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid > Rectangle#BackgroundFill", {
      L"Fill:=$Background",
      L"Visibility=Visible"}},
}, {
   L"SideMargin=40",
   L"TrayRightMargin=50",
   L"Translucent=<WindhawkBlur BlurAmount=\"15\" TintColor=\"#10808080\"/>",
   L"Glass=<WindhawkBlur BlurAmount=\"12\" TintColor=\"#20FFFFFF\" TintOpacity=\"0.3\" />",
   L"Frosted=<WindhawkBlur BlurAmount=\"20\" TintColor=\"#30D0D0D0\" TintOpacity=\"0.2\" />",
   L"Acrylic=<WindhawkBlur BlurAmount=\"30\" TintColor=\"#40B0B0B0\" TintOpacity=\"0.25\" />",
   L"Background=$Glass",
   L"BorderBrush=<LinearGradientBrush StartPoint=\"0,0\" EndPoint=\"0,1\"><GradientStop Color=\"#50808080\" Offset=\"0.0\" /><GradientStop Color=\"#30404040\" Offset=\"0.25\" /><GradientStop Color=\"#40808080\" Offset=\"1\" /></LinearGradientBrush>",
   L"BorderThickness=0.5,1,0.5,1",
   L"CornerRadius=4",
   L"ElementBorderThickness=0.3,0.3,0.3,1",
   L"ElementCornerRadius=12",
   L"TrayPadding=2,4,2,4",
}};

const Theme g_themePills = {{
   ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton", {
      L"ActualWidth => BtnW"}},
   ThemeTargetStyles{L"Taskbar.TaskbarBackground#BackgroundControl > Grid > Rectangle#BackgroundFill", {
      L"Fill := $taskbarFill"}},
   ThemeTargetStyles{L"Taskbar.TaskbarBackground#BackgroundControl > Grid > Rectangle#BackgroundStroke", {
      L"Fill := $taskbarStrokeColor"}},
   ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > Grid > Taskbar.TaskbarFrame", {
      L"Height => TaskbarHeight"}},
   ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Border#BackgroundElement, Taskbar.TaskListLabeledButtonPanel#IconPanel@RunningIndicatorStates > Border#BackgroundElement", {
      L"Height := {{TaskbarHeight-($taskbarBottomOffset+$taskbarTopOffset)-2*($highlightOffset)}}",
      L"Height@NoRunningIndicator := {{TaskbarHeight-($taskbarBottomOffset+$taskbarTopOffset)}}",
      L"Margin := {{$highlightOffset}},{{($taskbarTopOffset-$taskbarBottomOffset)/2-2}},{{$highlightOffset+2}},{{($taskbarBottomOffset-$taskbarTopOffset)/2-2}}",
      L"Margin@NoRunningIndicator := 0,{{$taskbarTopOffset-4}},2,{{$taskbarBottomOffset-4}}",
      L"Background@ActiveRunningIndicator :=",
      L"Background@NoRunningIndicator := $buttonFill",
      L"BorderThickness = 0",
      L"BorderThickness@NoRunningIndicator := $borderThickness",
      L"BorderBrush@NoRunningIndicator := $buttonBorderColor",
      L"CornerRadius@NoRunningIndicator := $buttonRadius",
      L"CornerRadius := $highlightRadius",
      L"Canvas.ZIndex = 2",
      L"Canvas.ZIndex@NoRunningIndicator = -10"}},
   ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border#BackgroundElement, Taskbar.TaskListLabeledButtonPanel#IconPanel@CommonStates > Border#BackgroundElement", {
      L"Opacity = 1",
```

```
      L"Opacity@InactiveNormal = 1"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Rectangle#RunningIndicator,
Taskbar.TaskListLabeledButtonPanel#IconPanel@RunningIndicatorStates >
Rectangle#RunningIndicator", {
      L"Opacity = 1",
      L"Opacity@NoRunningIndicator = 0",
      L"MinWidth = {{BtnW-6}}",
      L"MaxWidth = {{BtnW-6}}",
      L"HorizontalAlignment = 0",
      L"Grid.ColumnSpan = 2",
      L"Height := {{TaskbarHeight-($taskbarBottomOffset+$taskbarTopOffset)}}",
      L"Margin := 0,{{$taskbarTopOffset-4}},0,{{$taskbarBottomOffset-4}}",
      L"RadiusX := $buttonRadius",
      L"RadiusY := $buttonRadius",
      L"StrokeThickness := $borderThickness",
      L"Fill := $buttonFill",
      L"Stroke := $buttonBorderColor",
      L"Canvas.ZIndex = -10"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ProgressBar#ProgressIndicator", {
      L"Opacity = 1",
      L"MinWidth = {{BtnW-6}}",
      L"MaxWidth = {{BtnW-6}}",
      L"HorizontalAlignment = 0",
      L"Grid.ColumnSpan = 2",
      L"Height := {{TaskbarHeight-($taskbarBottomOffset+$taskbarTopOffset)}}",
      L"Margin := 0,{{$taskbarTopOffset-4}},0,{{$taskbarBottomOffset-4}}"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ProgressBar#ProgressIndicator >
Grid#LayoutRoot", {
      L"BorderThickness = 0",
      L"CornerRadius := $buttonRadius",
      L"Canvas.ZIndex = 1"}},
    ThemeTargetStyles{L"Border#ProgressBarRoot > Border > Grid", {
      L"Height = Auto"}},
    ThemeTargetStyles{L"Grid#LayoutRoot > Border#ProgressBarRoot > Border > Grid >
Rectangle#ProgressBarTrack", {
      L"Fill = Transparent"}},
    ThemeTargetStyles{L"Grid#LayoutRoot@CommonStates > Border#ProgressBarRoot >
Border > Grid > Rectangle#DeterminateProgressBarIndicator,
Grid#LayoutRoot@CommonStates > Border#ProgressBarRoot > Border > Grid >
Rectangle#IndeterminateProgressBarIndicator, Grid#LayoutRoot@CommonStates >
Border#ProgressBarRoot > Border > Grid > Rectangle#IndeterminateProgressBarIndicator2", {
      L"StrokeThickness = 1",
      L"RadiusX := $buttonRadius",
      L"RadiusY := $buttonRadius",
      L"Fill := $progressColor",
      L"Fill@Paused := <SolidColorBrush Color=\"orange\" Opacity=\"0.2\"/>"}},
    ThemeTargetStyles{L"Border#MultiWindowElement", {
      L"Visibility = 1"}},
    ThemeTargetStyles{L"Grid#IconPanel > TextBlock#LabelControl,
Taskbar.TaskListLabeledButtonPanel > TextBlock#LabelControl", {
      L"ActualWidth => LabelWidth",
      L"Margin := {{$iconLabelSpacing-12}},{{$taskbarTopOffset}},6,{{$taskbarBottomOffset}}",
      L"Padding := {{$leftRightPadding}},0",
      L"HorizontalAlignment = 1",
      L"VerticalAlignment = 1",
      L"RenderTransform := <TranslateTransform X=\"0\" Y=\"-1\" />",
      L"Canvas.ZIndex = 3"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton", {
      L"Margin := {{($buttonSpacing-6)/2}},0,{{($buttonSpacing-6)/2}},0"}},
```

```
    ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton > Grid#IconPanel@CommonStates > Image#Icon, Taskbar.TaskListButton#TaskListButton > Taskbar.TaskListLabeledButtonPanel#IconPanel@CommonStates > Image#Icon", {
        L"ActualWidth => ImageIconWidth",
        L"Margin := 0,{{$taskbarTopOffset}},0,{{$taskbarBottomOffset}}",
        L"HorizontalAlignment = 0",
        L"Canvas.ZIndex = 3",
        L"RenderTransformOrigin = 1,0.5",
        L"RenderTransform := <TranslateTransform X=\"{{max((ImageIconWidth/2-3),10)}}\" Y=\"0\" />",
        L"RenderTransform@InactivePointerOver := <TransformGroup><TranslateTransform X=\"{{max((ImageIconWidth/2-3),10)}}\" Y=\"0\" /><ScaleTransform ScaleX = \"0.9\" ScaleY = \"0.9\" /></TransformGroup>"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton > Grid#IconPanel, Taskbar.TaskListButton#TaskListButton > Taskbar.TaskListLabeledButtonPanel#IconPanel", {
        L"MinWidth := {{max(40,(2*ImageIconWidth+2))}}"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Rectangle#DefaultIcon, Taskbar.TaskListLabeledButtonPanel@CommonStates > Rectangle#DefaultIcon", {
        L"Opacity = 1",
        L"Stretch = 2",
        L"Height = 3",
        L"Width = 3",
        L"Visibility = 1",
        L"Visibility@MultiWindowNormal = 0",
        L"Visibility@MultiWindowActive = 0",
        L"Visibility@MultiWindowPressed = 0",
        L"Visibility@MultiWindowPointerOver = 0",
        L"Visibility@RequestingAttentionMulti = 0",
        L"Visibility@RequestingAttentionMultiPointerOver = 0",
        L"Visibility@RequestingAttentionMultiPressed = 0",
        L"Fill := $multiWinIndicatorColor",
        L"RadiusX = 2",
        L"RadiusY = 2",
        L"StrokeThickness = 0",
        L"HorizontalAlignment = 0",
        L"Margin = 4,0,0,0",
        L"Canvas.ZIndex = 4"}},
    ThemeTargetStyles{L"Taskbar.TaskbarExtensionElement", {
        L"Visibility = 1"}},
    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
        L"Width = 1",
        L"Height = 0"}},
    ThemeTargetStyles{L"Grid#IconPanel > Image#OverlayIcon, Grid#IconPanel > Taskbar.Badge#BadgeControl, Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#OverlayIcon, Taskbar.TaskListLabeledButtonPanel#IconPanel > Taskbar.Badge#BadgeControl", {
        L"Width := $badgeSize",
        L"Height := $badgeSize",
        L"Margin := $badgeNudge",
        L"RenderTransform := <TranslateTransform X=\"{{LabelWidth>0?6:0}}\" Y=\"0\" />",
        L"Canvas.ZIndex = 3"}},
    ThemeTargetStyles{L"Grid#IconPanel > Taskbar.Badge#BadgeControl > Grid > TextBlock#BadgeText, Taskbar.TaskListLabeledButtonPanel#IconPanel > Taskbar.Badge#BadgeControl > Grid > TextBlock#BadgeText", {
        L"FontSize = 8",
        L"HorizontalAlignment = 1"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.OmniButton#NotificationCenterButton, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.OmniButton#NotificationCenterButton", {
        L"Margin := 0,0,{{$taskbarRightOffset-12}},0"}},
```

```
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.OmniButton#NotificationCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.OmniButton#NotificationCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel", {
        L"Padding = 2,0"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.OmniButton#NotificationCenterButton > Grid, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.OmniButton#NotificationCenterButton > Grid", {
        L"Margin := {{$sysTraySpacing}},{{$taskbarTopOffset}},0,{{$taskbarBottomOffset}}",
        L"Padding := {{-$borderThickness}}",
        L"CornerRadius := {{$buttonRadius}},{{$buttonRadius*$taskbarSidesRounded}},{{$buttonRadius*$taskbarSidesRounded}},{{$buttonRadius}}",
        L"BorderThickness := $borderThickness",
        L"Background := $buttonFill",
        L"BorderBrush := $buttonBorderColor"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Grid > Border#BackgroundBorder, SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Border#BackgroundBorder, SystemTray.ChevronIconView > Grid#ContainerGrid > Border#BackgroundBorder, SystemTray.OmniButton#ControlCenterButton > Grid > Border#BackgroundBorder, SystemTray.NotifyIconView#NotifyItemIcon > Grid#ContainerGrid > Border#BackgroundBorder, Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid > Border#BackgroundElement, Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Border#BackgroundElement", {
        L"Margin := {{$highlightOffset}}",
        L"CornerRadius := $highlightRadius",
        L"BorderThickness = 0"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Grid > ContentPresenter#ContentPresenter", {
        L"Margin = 0,0,0,1"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid", {
        L"Margin := 0,{{$taskbarTopOffset}},0,{{$taskbarBottomOffset}}",
        L"Padding := {{-$borderThickness}}",
        L"CornerRadius := 0,$buttonRadius,$buttonRadius,0",
        L"BorderThickness := 0,{{$borderThickness}},{{$borderThickness}},{{$borderThickness}}",
        L"Background := $buttonFill",
        L"BorderBrush := $buttonBorderColor"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.Stack#MainStack > Grid#Content, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.Stack#MainStack > Grid#Content", {
        L"Margin := 0,{{$taskbarTopOffset}},0,{{$taskbarBottomOffset}}",
        L"Padding := {{-$borderThickness}}",
        L"BorderThickness := 0,$borderThickness,0,$borderThickness",
        L"Background := $buttonFill",
        L"BorderBrush := $buttonBorderColor"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.Stack#NonActivatableStack > Grid#Content, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.Stack#NonActivatableStack > Grid#Content", {
        L"Margin := 0,{{$taskbarTopOffset}},0,{{$taskbarBottomOffset}}",
        L"Padding := {{-$borderThickness}}",
        L"BorderThickness := 0,$borderThickness,0,$borderThickness",
        L"Background := $buttonFill",
        L"BorderBrush := $buttonBorderColor"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.NotificationAreaIcons#NotificationAreaIcons > ItemsPresenter > StackPanel, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.NotificationAreaIcons#NotificationAreaIcons > ItemsPresenter > StackPanel", {
        L"Margin := 0,{{$taskbarTopOffset}},0,{{$taskbarBottomOffset}}",
        L"Padding := {{-$borderThickness}}",
        L"BorderThickness := 0,$borderThickness,0,$borderThickness",
```

```
        L"Background := $buttonFill",
        L"BorderBrush := $buttonBorderColor"}},
    ThemeTargetStyles{L"SystemTray.Stack#NotifyIconStack > Grid#Content >
SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter", {
        L"Margin := 0,{{$taskbarTopOffset}},0,{{$taskbarBottomOffset}}",
        L"Padding := {{-$borderThickness}}",
        L"BorderThickness := $borderThickness,$borderThickness,0,$borderThickness",
        L"Background := $buttonFill",
        L"CornerRadius := $buttonRadius,0,0,$buttonRadius",
        L"BorderBrush := $buttonBorderColor"}},
    ThemeTargetStyles{L"SystemTray.TextIconContent > Grid#ContainerGrid >
SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock, SystemTray.TextIconContent >
Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Underlay > TextBlock#InnerTextBlock,
SystemTray.BatteryIconContent > Grid#ContainerGrid > StackPanel > Grid > TextBlock", {
        L"FontSize := $sysTrayIconSize"}},
    ThemeTargetStyles{L"SystemTray.LanguageTextIconContent > Grid#ContainerGrid >
SystemTray.AdaptiveTextBlock#LanguageInnerTextBlock > TextBlock#InnerTextBlock", {
        L"FontSize := {{$sysTrayIconSize-4}}"}},
    ThemeTargetStyles{L"SystemTray.ImageIconContent > Grid#ContainerGrid > Image", {
        L"Width := $sysTrayIconSize",
        L"Height := $sysTrayIconSize"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock#LanguageInnerTextBlock >
TextBlock#InnerTextBlock", {
        L"Margin = 0,0,0,2.5",
        L"MaxLines = 1"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > Grid >
Taskbar.TaskbarFrame > Grid#RootGrid >
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater >
Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid >
Grid#AugmentedEntryPointContentGrid > Grid > Grid >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Border >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Grid > Border#LargeTicker1, ScrollViewer >
ScrollContentPresenter > Border > Grid > Taskbar.TaskbarFrame > Grid#RootGrid >
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater >
Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Grid#AugmentedEntryPointContentGrid > Grid > Grid >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Border >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Grid > Border#LargeTicker1", {
        L"ActualWidth => WeatherIconWidth",
        L"RenderTransform := <TranslateTransform X=\"0\" Y=\"1\" />"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > Grid >
Taskbar.TaskbarFrame > Grid#RootGrid >
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater >
Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid >
Grid#AugmentedEntryPointContentGrid > Grid > Grid >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Border >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Grid > Border#LargeTicker2 >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > TextBlock[1], ScrollViewer >
ScrollContentPresenter > Border > Grid > Taskbar.TaskbarFrame > Grid#RootGrid >
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater >
Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Grid#AugmentedEntryPointContentGrid > Grid > Grid >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Border >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Grid > Border#LargeTicker2 >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > TextBlock[1]", {
        L"ActualWidth => WeatherTempWidth",
        L"RenderTransform := <TranslateTransform X=\"0\" Y=\"8\" />"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > Grid >
Taskbar.TaskbarFrame > Grid#RootGrid >
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater >
Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid >
Grid#AugmentedEntryPointContentGrid > Grid > Grid >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Border >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Grid > Border#LargeTicker2 >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > TextBlock[2], ScrollViewer >
ScrollContentPresenter > Border > Grid > Taskbar.TaskbarFrame > Grid#RootGrid >
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater >
Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Grid#AugmentedEntryPointContentGrid > Grid > Grid >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Border >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Grid > Border#LargeTicker2 >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > TextBlock[2]", {
```

```
        L"ActualWidth => WeatherCondWidth",
        L"RenderTransform := <TranslateTransform X=\"{{(WeatherTempWidth + 8)}}\" Y=\"-8\"
/>"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid >
Grid#AugmentedEntryPointContentGrid,
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Grid#AugmentedEntryPointContentGrid", {
        L"Margin = 0",
        L"Width := {{WeatherCondWidth+WeatherTempWidth + WeatherIconWidth + 44}}",
        L"VerticalAlignment = 1",
        L"HorizontalAlignment = 0",
        L"RenderTransform := <TranslateTransform X=\"0\" Y=\"1\" />"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid >
Grid#AugmentedEntryPointContentGrid > Grid > Grid >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel,
Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Grid#AugmentedEntryPointContentGrid > Grid > Grid >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel", {
        L"Margin = 0"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
        L"Width := {{WeatherCondWidth + WeatherTempWidth + WeatherIconWidth + 36}}",
        L"MinWidth = 52",
        L"Margin := {{$taskbarLeftOffset}},{{$taskbarTopOffset}},56,{{$taskbarBottomOffset}}",
        L"Padding = 0",
        L"CornerRadius :=
{{$buttonRadius*$taskbarSidesRounded}},{{$buttonRadius}},{{$buttonRadius}},{{$buttonRadius
*$taskbarSidesRounded}}",
        L"BorderThickness := $borderThickness",
        L"Background := $buttonFill",
        L"BorderBrush := $buttonBorderColor"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > Grid >
Taskbar.TaskbarFrame > Grid#RootGrid >
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater >
Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid >
Grid#AugmentedEntryPointContentGrid > Grid > Grid >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel, ScrollViewer > ScrollContentPresenter >
Border > Grid > Taskbar.TaskbarFrame > Grid#RootGrid >
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater >
Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Grid#AugmentedEntryPointContentGrid > Grid > Grid >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel", {
        L"VerticalAlignment = 1"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > Grid >
Taskbar.TaskbarFrame > Grid#RootGrid >
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater >
Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid >
Grid#AugmentedEntryPointContentGrid > Grid > Grid, ScrollViewer > ScrollContentPresenter >
Border > Grid > Taskbar.TaskbarFrame > Grid#RootGrid >
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater >
Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Grid#AugmentedEntryPointContentGrid > Grid > Grid ", {
        L"VerticalAlignment = 1"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > Grid >
Taskbar.TaskbarFrame > Grid#RootGrid >
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater >
Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid >
Grid#AugmentedEntryPointContentGrid > Grid > Grid[1], ScrollViewer >
ScrollContentPresenter > Border > Grid > Taskbar.TaskbarFrame > Grid#RootGrid >
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater >
Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Grid#AugmentedEntryPointContentGrid > Grid > Grid[1]", {
        L"HorizontalAlignment = 0",
        L"RenderTransform := <TranslateTransform X=\"8\" Y=\"-2\" />"}},

ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#BadgeAnchorLargeTick
er, Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#BadgeAnchorSmallTicker", {
        L"MaxWidth = 20"}},
```

```
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > Grid >
Taskbar.TaskBarFrame > Grid#RootGrid >
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater >
Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid >
Grid#AugmentedEntryPointContentGrid > Grid > Grid[2], ScrollViewer >
ScrollContentPresenter > Border > Grid > Taskbar.TaskBarFrame > Grid#RootGrid >
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater >
Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Grid#AugmentedEntryPointContentGrid > Grid > Grid[2]", {
        L"HorizontalAlignment = 0",
        L"RenderTransformOrigin = 0.5,0.5",
        L"RenderTransform := <TransformGroup><TranslateTransform X=\"20\" Y=\"-4\"
/><ScaleTransform ScaleX = \"0.75\" ScaleY = \"0.75\" /></TransformGroup>"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > Grid >
Taskbar.TaskBarFrame > Grid#RootGrid >
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater >
Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid >
Grid#AugmentedEntryPointContentGrid > Grid > Grid >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Border >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Grid, ScrollViewer >
ScrollContentPresenter > Border > Grid > Taskbar.TaskBarFrame > Grid#RootGrid >
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater >
Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Grid#AugmentedEntryPointContentGrid > Grid > Grid >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Border >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Grid", {
        L"Background = Transparent"}},
    ThemeTargetStyles{L"Grid#ContainerGrid@ > Rectangle#ShowDesktopPipe", {
        L"Opacity = 1",
        L"Grid.Row=0",
        L"Grid.RowSpan=3",
        L"Grid.Column=0",
        L"Grid.ColumnSpan=3",
        L"Width = 4",
        L"Height = 4",
        L"Height@PointerOver = 10",
        L"Height@Pressed = 6",
        L"RadiusX = 2",
        L"RadiusY = 2",
        L"Margin := 0,{{$taskbarTopOffset}},0,{{$taskbarBottomOffset}}",
        L"Fill := $showDesktopIndicatorColor"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#OverflowToggleButtonRootPanel", {
        L"Margin := 0,{{$taskbarTopOffset}},0,{{$taskbarBottomOffset}}",
        L"Padding = 0",
        L"Background := $buttonFill",
        L"CornerRadius := 0,{{$buttonRadius}},{{$buttonRadius}},0",
        L"BorderThickness := 0,{{$borderThickness}},{{$borderThickness}},{{$borderThickness}}",
        L"BorderBrush := $buttonBorderColor"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#OverflowToggleButtonRootPanel > Grid >
Border#BackgroundElement, Taskbar.TaskListButtonPanel#OverflowToggleButtonRootPanel >
Border#BackgroundElement", {
        L"Margin := {{$highlightOffset}}",
        L"BorderThickness = 0",
        L"CornerRadius := $highlightRadius"}},
    ThemeTargetStyles{L"Grid#VdSwitcherBar", {
        L"Padding = 8,1,6,0",
        L"Height = 24",
        L"BorderThickness := $borderThickness",
        L"CornerRadius := $buttonRadius",
        L"Background := $buttonFill",
        L"BorderBrush := $buttonBorderColor"}},
    ThemeTargetStyles{L"Grid#VdSwitcherBar > Button > ContentPresenter", {
        L"BorderThickness = 0"}},
```

```
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > Taskbar.FlyoutFrame > Canvas#HoverFlyoutCanvas > Grid#HoverFlyoutGrid > Border#HoverFlyoutBackground, WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputSwitcher > ContentControl > ContentPresenter > Grid, Grid#OverflowRootGrid > Border", {
        L"Shadow :="}},

ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#OverflowFlyoutListRepeater > Taskbar.TaskListButton#TaskListButton > Grid#IconPanel, Microsoft.UI.Xaml.Controls.ItemsRepeater#OverflowFlyoutListRepeater > Taskbar.TaskListButton#TaskListButton > Taskbar.TaskListLabeledButtonPanel#IconPanel", {
        L"MinWidth = 28"}},
    ThemeTargetStyles{L"ScrollContentPresenter > Border > Taskbar.FlyoutFrame > Canvas#HoverFlyoutCanvas > Grid#HoverFlyoutGrid > ContentPresenter#HoverFlyoutContent > Taskbar.OverflowFlyoutList > ScrollViewer#OverflowScrollView > Border#Root > Grid > ScrollContentPresenter#ScrollContentPresenter > Microsoft.UI.Xaml.Controls.ItemsRepeater#OverflowFlyoutListRepeater > Taskbar.TaskListButton#TaskListButton > Grid#IconPanel@CommonStates > Image#Icon, ScrollContentPresenter > Border > Taskbar.FlyoutFrame > Canvas#HoverFlyoutCanvas > Grid#HoverFlyoutGrid > ContentPresenter#HoverFlyoutContent > Taskbar.OverflowFlyoutList > ScrollViewer#OverflowScrollView > Border#Root > Grid > ScrollContentPresenter#ScrollContentPresenter > Microsoft.UI.Xaml.Controls.ItemsRepeater#OverflowFlyoutListRepeater > Taskbar.TaskListButton#TaskListButton > Taskbar.TaskListLabeledButtonPanel#IconPanel@CommonStates > Image#Icon", {
        L"RenderTransformOrigin = 0.5,0.5",
        L"RenderTransform := <TranslateTransform X=\"{{max((ImageIconWidth/2-3),10/2)}}\" Y=\"0\" />",
        L"RenderTransform@InactivePointerOver := <TransformGroup><ScaleTransform ScaleX = \"0.9\" ScaleY = \"0.9\" /><TranslateTransform X=\"{{max((ImageIconWidth/2-3),10/2)}}\" Y=\"0\" /></TransformGroup>"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > Taskbar.FlyoutFrame > Canvas#HoverFlyoutCanvas > Grid#HoverFlyoutGrid > ContentPresenter#HoverFlyoutContent > Taskbar.OverflowFlyoutList > ScrollViewer#OverflowScrollView > Border#Root > Grid > ScrollContentPresenter#ScrollContentPresenter > Microsoft.UI.Xaml.Controls.ItemsRepeater#OverflowFlyoutListRepeater > Taskbar.TaskListButton#TaskListButton > Grid#IconPanel > Border#BackgroundElement, ScrollViewer > ScrollContentPresenter > Border > Taskbar.FlyoutFrame > Canvas#HoverFlyoutCanvas > Grid#HoverFlyoutGrid > ContentPresenter#HoverFlyoutContent > Taskbar.OverflowFlyoutList > ScrollViewer#OverflowScrollView > Border#Root > Grid > ScrollContentPresenter#ScrollContentPresenter > Microsoft.UI.Xaml.Controls.ItemsRepeater#OverflowFlyoutListRepeater > Taskbar.TaskListButton#TaskListButton > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border#BackgroundElement", {
        L"Background = Transparent",
        L"BorderBrush = Transparent",
        L"Margin = 0"}},

ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#OverflowFlyoutListRepeater > Taskbar.TaskListButton#TaskListButton > Grid#IconPanel > Rectangle#RunningIndicator, Microsoft.UI.Xaml.Controls.ItemsRepeater#OverflowFlyoutListRepeater > Taskbar.TaskListButton#TaskListButton > Taskbar.TaskListLabeledButtonPanel#IconPanel > Rectangle#RunningIndicator", {
        L"Opacity = 0"}},
}, {
    L"taskbarLeftOffset = 12",
    L"taskbarRightOffset = 12",
    L"taskbarTopOffset = 4",
    L"taskbarBottomOffset = 4",
    L"borderThickness = 2",
    L"buttonRadius = 7",
    L"highlightRadius = 5",
    L"highlightOffset = 4",
    L"buttonSpacing = 6",
    L"sysTraySpacing = 6",
    L"iconLabelSpacing = 10",
    L"leftRightPadding = 8",
    L"badgeSize = 12",
    L"badgeNudge = 0,4,2,0",
    L"sysTrayIconSize = 16",
    L"taskbarSidesRounded = 1",
    L"buttonFill = <WindhawkBlur BlurAmount=\"7\" TintColor=\"{ThemeResource AdaptiveFill}\" TintOpacity=\"0.2\" TintLuminosityOpacity=\"0.2\" />",
```

```
    L"buttonBorderColor = <SolidColorBrush Color=\"{ThemeResource AdaptiveBorder}\"
Opacity=\"1\"/>",
    L"taskbarFill = {{skip()}}",
    L"taskbarStrokeColor = {{skip()}}",
    L"progressColor = <SolidColorBrush Color=\"{ThemeResource SystemAccentColor}\"
Opacity=\"0.2\"/>",
    L"showDesktopIndicatorColor = <SolidColorBrush Color=\"{ThemeResource
SystemAccentColor}\" Opacity=\"0.7\"/>",
    L"multiWinIndicatorColor = <SolidColorBrush Color=\"{ThemeResource AdaptiveIndicator}\"
Opacity=\"0.7\"/>",
}, {
    L"AdaptiveFill@Light =#FFFFFF",
    L"AdaptiveFill@Dark =#000000",
    L"AdaptiveBorder@Light =#90B4B4B4",
    L"AdaptiveBorder@Dark =#90454545",
    L"AdaptiveIndicator@Light =#000000",
    L"AdaptiveIndicator@Dark =#FFFFFF",
}};

const Theme g_themeBlob = {{
    ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
        L"Height => TaskbarHeight"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#BackgroundControl > Grid >
Rectangle#BackgroundFill", {
        L"Fill := $taskbarFill"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#BackgroundControl > Grid >
Rectangle#BackgroundStroke", {
        L"Height := $taskbarStrokeHeight",
        L"Fill := $taskbarStrokeColor"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton > Grid#IconPanel,
Taskbar.TaskListButton#TaskListButton > Taskbar.TaskListLabeledButtonPanel#IconPanel", {
        L"Padding := {{$buttonSpacing-2}},{{$taskbarTopOffset}},{{$buttonSpacing-
2}},{{$taskbarBottomOffset}}",
        L"MinWidth := $buttonMinWidth"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater >
Taskbar.ExperienceToggleButton#LaunchListButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
        L"Padding := {{$buttonSpacing-2}},{{$taskbarTopOffset}},{{$buttonSpacing-
2}},{{$taskbarBottomOffset}}",
        L"MinWidth := $buttonMinWidth"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater >
Taskbar.TaskbarExtensionElement > ContentPresenter >
SearchUx.SearchUI.SearchButtonControl > Grid > SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel", {
        L"Padding := {{$buttonSpacing-2}},{{$taskbarTopOffset}},{{$buttonSpacing-
2}},{{$taskbarBottomOffset}}",
        L"MinWidth := $buttonMinWidth"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater >
Taskbar.ExperienceToggleButton#LaunchListButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel@CommonStates > Grid >
Border#BackgroundElement,
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater >
Taskbar.ExperienceToggleButton#LaunchListButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel@CommonStates >
Border#BackgroundElement", {
        L"Background@ActiveNormal := $blobFill",
        L"Background@ActivePressed := $blobFill",
        L"Background@InactivePressed := $blobFill",
        L"Background@ActivePointerOver := $blobFill",
        L"Background@InactivePointerOver := $blobFill",
        L"CornerRadius := $highlightRadius",
        L"BorderThickness = 0",
        L"BackgroundTransition := <BrushTransition Duration=\"0:0:0\"/>"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater >
Taskbar.TaskbarExtensionElement > ContentPresenter >
```

```
SearchUx.SearchUI.SearchButtonControl > Grid > SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel@CommonStates >
Grid > Border#BackgroundElement,
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater >
Taskbar.TaskbarExtensionElement > ContentPresenter >
SearchUx.SearchUI.SearchButtonControl > Grid > SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel@CommonStates >
Border#BackgroundElement", {
    L"Background@ActiveNormal := $blobFill",
    L"Background@ActivePressed := $blobFill",
    L"Background@InactivePressed := $blobFill",
    L"Background@ActivePointerOver := $blobFill",
    L"Background@InactivePointerOver := $blobFill",
    L"CornerRadius := $highlightRadius",
    L"BorderThickness = 0",
    L"BackgroundTransition := <BrushTransition Duration=\"0:0:0\"/>"}},
  ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid >
Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon,
Taskbar.ExperienceToggleButton#LaunchListButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon", {
    L"Margin := 0,0,0,{{$taskbarContentOffset-2}}"}},

ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Grid > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon,
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon", {
    L"Margin := 0,0,0,{{$taskbarContentOffset-2}}"}},
  ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border#BackgroundElement,
Taskbar.TaskListLabeledButtonPanel#IconPanel@CommonStates > Border#BackgroundElement",
{
    L"Margin = 0",
    L"Background@ActiveNormal := $blobFill",
    L"Background@InactiveNormal = Transparent",
    L"Background@ActivePointerOver := $blobFill",
    L"Background@InactivePointerOver := $blobFill",
    L"Background@ActivePressed := $blobFill",
    L"Background@InactivePressed := $blobFill",
    L"Background@MultiWindowActive := $blobFill",
    L"Background@MultiWindowPointerOver := $blobFill",
    L"Background@MultiWindowPressed := $blobFill",
    L"Background@RequestingAttentionPointerOver := $blobFill",
    L"Background@RequestingAttentionPressed := $blobFill",
    L"Background@RequestingAttentionMultiPointerOver := $blobFill",
    L"Background@RequestingAttentionMultiPressed := $blobFill",
    L"CornerRadius := $highlightRadius",
    L"BorderThickness = 0",
    L"Canvas.ZIndex = 0",
    L"BackgroundTransition := <BrushTransition Duration=\"0:0:0\"/>"}},
  ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Rectangle#RunningIndicator,
Taskbar.TaskListLabeledButtonPanel#IconPanel@CommonStates > Rectangle#RunningIndicator",
{
    L"Opacity@NoRunningIndicator = 0",
    L"Width = 2",
    L"Height = 2",
    L"VerticalAlignment = 1",
    L"Margin = 3,0,0,0",
    L"Opacity = 0",
    L"Opacity@MultiWindowNormal = 1",
    L"Opacity@MultiWindowActive = 1",
    L"Opacity@MultiWindowPressed = 1",
    L"Opacity@MultiWindowPointerOver = 1",
    L"Opacity@RequestingAttentionMulti = 1",
```

```
      L"Opacity@RequestingAttentionMultiPointerOver = 1",
      L"Opacity@RequestingAttentionMultiPressed = 1",
      L"Fill := $multiWinIndicatorColor",
      L"RadiusX = 0",
      L"RadiusY = 0",
      L"StrokeThickness = 0",
      L"Canvas.ZIndex = 4"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ProgressBar#ProgressIndicator", {
      L"Height := {{TaskbarHeight-($taskbarBottomOffset+$taskbarTopOffset)}}",
      L"Margin = 1,0,0,0",
      L"CornerRadius := $highlightRadius"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ProgressBar#ProgressIndicator > Grid#LayoutRoot", {
      L"CornerRadius := $highlightRadius",
      L"Canvas.ZIndex = 1"}},
    ThemeTargetStyles{L"Border#ProgressBarRoot > Border > Grid", {
      L"Height = Auto"}},
    ThemeTargetStyles{L"Grid#LayoutRoot > Border#ProgressBarRoot > Border > Grid > Rectangle#ProgressBarTrack", {
      L"Fill = Transparent"}},
    ThemeTargetStyles{L"Grid#LayoutRoot@CommonStates > Border#ProgressBarRoot > Border > Grid > Rectangle#DeterminateProgressBarIndicator", {
      L"Fill := $progressColor",
      L"Fill@Paused := $progressPausedColor"}},
    ThemeTargetStyles{L"Grid#LayoutRoot@CommonStates > Border#ProgressBarRoot > Border > Grid > Rectangle#IndeterminateProgressBarIndicator", {
      L"Fill := $progressColor",
      L"Fill@Paused := $progressPausedColor"}},
    ThemeTargetStyles{L"Grid#LayoutRoot@CommonStates > Border#ProgressBarRoot > Border > Grid > Rectangle#IndeterminateProgressBarIndicator2", {
      L"Fill := $progressColor",
      L"Fill@Paused := $progressPausedColor"}},
    ThemeTargetStyles{L"Border#MultiWindowElement", {
      L"Visibility = 1"}},
    ThemeTargetStyles{L"Grid#IconPanel > TextBlock#LabelControl, Taskbar.TaskListLabeledButtonPanel > TextBlock#LabelControl", {
      L"Margin := {{$iconLabelSpacing}},0,4,{{$taskbarContentOffset}}",
      L"Padding := {{$leftRightPadding}},0",
      L"Canvas.ZIndex = 3"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton#TaskListButton > Grid#IconPanel > Image#Icon, Taskbar.TaskListButton#TaskListButton > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
      L"Margin := {{$iconSpacing}},0,0,{{$taskbarContentOffset-2}}",
      L"Canvas.ZIndex = 3",
      L"HorizontalAlignment = 0"}},
    ThemeTargetStyles{L"Grid#IconPanel > Rectangle#DefaultIcon, Taskbar.TaskListLabeledButtonPanel > Rectangle#DefaultIcon", {
      L"Stretch = 1",
      L"Height = 8",
      L"Width = 1",
      L"Visibility = 0",
      L"Fill := $separatorColor",
      L"RenderTransform := <TranslateTransform X=\"-14\" Y=\"0\" />"}},
    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
      L"Visibility := {{1-$showStartButton}}"}},
    ThemeTargetStyles{L"Grid#IconPanel > Image#OverlayIcon, Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#OverlayIcon", {
      L"Width := $badgeSize",
      L"Height := $badgeSize",
```

```
        L"Margin := $badgeNudge",
        L"Canvas.ZIndex = 3"}},
    ThemeTargetStyles{L"Grid#IconPanel > Taskbar.Badge#BadgeControl, Taskbar.TaskListLabeledButtonPanel#IconPanel > Taskbar.Badge#BadgeControl", {
        L"MinWidth := $badgeSize",
        L"Width := $badgeSize",
        L"Height := $badgeSize",
        L"Margin := $badgeNudge",
        L"Canvas.ZIndex = 3"}},
    ThemeTargetStyles{L"Grid#IconPanel > Taskbar.Badge#BadgeControl > Grid > TextBlock#BadgeText, Taskbar.TaskListLabeledButtonPanel#IconPanel > Taskbar.Badge#BadgeControl > Grid > TextBlock#BadgeText", {
        L"FontSize = 8"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.OmniButton#NotificationCenterButton, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.OmniButton#NotificationCenterButton", {
        L"Margin := 0,0,{{$taskbarRightOffset-12}},0"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Grid@CommonStates > Border#BackgroundBorder", {
        L"Margin := 2,{{$taskbarTopOffset}},0,{{$taskbarBottomOffset}}",
        L"Background@PointerOver := $blobFill",
        L"Background@Pressed := $blobFill",
        L"Background@Checked := $blobFill",
        L"Background@CheckedPointerOver := $blobFill",
        L"Background@CheckedPressed := $blobFill",
        L"CornerRadius := $highlightRadius",
        L"BorderThickness = 0",
        L"BackgroundTransition := <BrushTransition Duration=\"0:0:0\"/>",
        L"CornerRadius := $highlightRadius"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid@CommonStates > Border#BackgroundBorder", {
        L"Margin := 2,{{$taskbarTopOffset}},2,{{$taskbarBottomOffset}}",
        L"Background@PointerOver := $blobFill",
        L"Background@Pressed := $blobFill",
        L"Background@Checked := $blobFill",
        L"Background@CheckedPointerOver := $blobFill",
        L"Background@CheckedPressed := $blobFill",
        L"CornerRadius := $highlightRadius",
        L"BorderThickness = 0",
        L"BackgroundTransition := <BrushTransition Duration=\"0:0:0\"/>",
        L"CornerRadius := $highlightRadius"}},
    ThemeTargetStyles{L"SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid@ > Border#BackgroundBorder", {
        L"Margin := 2,{{$taskbarTopOffset}},2,{{$taskbarBottomOffset}}",
        L"Background@CheckedNormal := $blobFill",
        L"Background@CheckedPressed := $blobFill",
        L"Background@Pressed := $blobFill",
        L"Background@CheckedPointerOver := $blobFill",
        L"Background@PointerOver := $blobFill",
        L"CornerRadius := $highlightRadius",
        L"BorderThickness = 0",
        L"BackgroundTransition := <BrushTransition Duration=\"0:0:0\"/>"}},
    ThemeTargetStyles{L"SystemTray.NotifyIconView#NotifyItemIcon > Grid#ContainerGrid@ > Border#BackgroundBorder", {
        L"Margin := 2,{{$taskbarTopOffset}},2,{{$taskbarBottomOffset}}",
        L"Background@CheckedNormal := $blobFill",
        L"Background@CheckedPressed := $blobFill",
        L"Background@Pressed := $blobFill",
        L"Background@CheckedPointerOver := $blobFill",
```

```
        L"Background@PointerOver := $blobFill",
        L"CornerRadius := $highlightRadius",
        L"BorderThickness = 0",
        L"BackgroundTransition := <BrushTransition Duration=\"0:0:0\"/>"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView > Grid#ContainerGrid@ > Border#BackgroundBorder", {
        L"Margin := 2,{{$taskbarTopOffset}},2,{{$taskbarBottomOffset}}",
        L"Background@CheckedNormal := $blobFill",
        L"Background@CheckedPressed := $blobFill",
        L"Background@Pressed := $blobFill",
        L"Background@CheckedPointerOver := $blobFill",
        L"Background@PointerOver := $blobFill",
        L"CornerRadius := $highlightRadius",
        L"BorderThickness = 0",
        L"BackgroundTransition := <BrushTransition Duration=\"0:0:0\"/>"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel", {
        L"Margin := 0,0,0,{{$taskbarBottomOffset-$taskbarTopOffset+$taskbarContentOffset}}"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter", {
        L"Margin := 0,0,0,{{$taskbarBottomOffset-$taskbarTopOffset+$taskbarContentOffset-2}}"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock#LanguageInnerTextBlock > TextBlock#InnerTextBlock", {
        L"MaxLines = 1",
        L"Margin := 0,0,0,{{$taskbarBottomOffset-$taskbarTopOffset+$taskbarContentOffset}}"}},
    ThemeTargetStyles{L"SystemTray.Stack#MainStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base, SystemTray.Stack#MainStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base", {
        L"Margin := 0,0,0,{{$taskbarBottomOffset-$taskbarTopOffset+$taskbarContentOffset-2}}"}},
    ThemeTargetStyles{L"SystemTray.NotificationAreaIcons#NotificationAreaIcons > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.NotifyIconView#NotifyItemIcon > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.ImageIconContent > Grid#ContainerGrid > Image", {
        L"Margin := 0,0,0,{{$taskbarBottomOffset-$taskbarTopOffset+$taskbarContentOffset-2}}"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter", {
        L"Margin := 0,0,0,{{$taskbarBottomOffset-$taskbarTopOffset+$taskbarContentOffset-2}}"}},
    ThemeTargetStyles{L"Grid#OverflowRootGrid > Border", {
        L"Shadow :="}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid > Grid#AugmentedEntryPointContentGrid > Grid > Grid > AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Border > AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Grid > Border#LargeTicker2 > AdaptiveCards.Rendering.Uwp.WholeItemsPanel, Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid#AugmentedEntryPointContentGrid > Grid > Grid > AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Border > AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Grid > Border#LargeTicker2 > AdaptiveCards.Rendering.Uwp.WholeItemsPanel", {
        L"RenderTransform := <TranslateTransform X=\"0\" Y=\"-1\" />"}},
    ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid > Grid#AugmentedEntryPointContentGrid > Grid > Grid > AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Border > AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Grid > Border#LargeTicker2 > AdaptiveCards.Rendering.Uwp.WholeItemsPanel > TextBlock[1], Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid#AugmentedEntryPointContentGrid > Grid > Grid > AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Border > AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Grid > Border#LargeTicker2 > AdaptiveCards.Rendering.Uwp.WholeItemsPanel > TextBlock[1]", {
        L"ActualWidth => WeatherTempWidth",
        L"RenderTransform := <TranslateTransform X=\"0\" Y=\"8\" />"}},
```

ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid > Grid#AugmentedEntryPointContentGrid > Grid > Grid > AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Border > AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Grid > Border#LargeTicker2 > AdaptiveCards.Rendering.Uwp.WholeItemsPanel > TextBlock[2], Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid#AugmentedEntryPointContentGrid > Grid > Grid > AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Border > AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Grid > Border#LargeTicker2 > AdaptiveCards.Rendering.Uwp.WholeItemsPanel > TextBlock[2]", {
L"ActualWidth => WeatherCondWidth",
L"RenderTransform := <TranslateTransform X=\"{{WeatherTempWidth+8}}\" Y=\"-8\" />"}},
ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid > Grid#AugmentedEntryPointContentGrid, Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid#AugmentedEntryPointContentGrid", {
L"Margin := 0,0,0,{{$taskbarContentOffset-4}}",
L"Width := {{WeatherCondWidth+WeatherTempWidth+62}}"}},
ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel", {
L"Width := {{WeatherCondWidth+WeatherTempWidth+62}}",
L"Margin := {{$taskbarLeftOffset-12}},0,56,0",
L"Padding := 12,{{$taskbarTopOffset}},0,{{$taskbarBottomOffset}}"}},
ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel@CommonStates > Grid > Border#BackgroundElement, Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel@CommonStates > Border#BackgroundElement", {
L"Background@ActiveNormal := $blobFill",
L"Background@ActivePressed := $blobFill",
L"Background@InactivePressed := $blobFill",
L"Background@ActivePointerOver := $blobFill",
L"Background@InactivePointerOver := $blobFill",
L"CornerRadius := $highlightRadius",
L"BorderThickness = 0",
L"BackgroundTransition := <BrushTransition Duration=\"0:0:0\"/>"}},
ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater > Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid > Grid#AugmentedEntryPointContentGrid > Grid > Grid > AdaptiveCards.Rendering.Uwp.WholeItemsPanel, Taskbar.TaskbarFrame > Grid#RootGrid > Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater > Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid#AugmentedEntryPointContentGrid > Grid > Grid > AdaptiveCards.Rendering.Uwp.WholeItemsPanel", {
L"Margin = 0,3,0,0"}},
ThemeTargetStyles{L"Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid > Grid#AugmentedEntryPointContentGrid > Grid[0] > Grid[2], Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid#AugmentedEntryPointContentGrid > Grid[0] > Grid[2]", {
L"HorizontalAlignment = 1",
L"VerticalAlignment = 1",
L"RenderTransformOrigin = -0.5,0.5",
L"RenderTransform := <TransformGroup><ScaleTransform ScaleX = \"0.9\" ScaleY = \"0.9\" /><TranslateTransform X=\"10\" Y=\"8\" /></TransformGroup>"}},
ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid > Grid#AugmentedEntryPointContentGrid > Grid > Grid > AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Border > AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Grid, Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid#AugmentedEntryPointContentGrid > Grid > Grid > AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Border > AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Grid", {
L"Background = Transparent"}},
ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater > Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid > Grid#AugmentedEntryPointContentGrid > Grid > Grid, Taskbar.TaskbarFrame > Grid#RootGrid >

```
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater >
Taskbar.AugmentedEntryPointButton#AugmentedEntryPointButton >
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Grid#AugmentedEntryPointContentGrid > Grid > Grid", {
        L"Margin = 16,0,0,0",
        L"HorizontalAlignment = 0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid >
Grid#AugmentedEntryPointContentGrid > Grid > Grid >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Border >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Grid > Border#LargeTicker1 >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel >
Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#BadgeAnchorLargeTicker,
Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel >
Grid#AugmentedEntryPointContentGrid > Grid > Grid >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Border >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel > Grid > Border#LargeTicker1 >
AdaptiveCards.Rendering.Uwp.WholeItemsPanel >
Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#BadgeAnchorLargeTicker", {
        L"Margin := 0,0,0,1"}},

ThemeTargetStyles{L"WindowsInternal.ComposableShell.Experiences.TextInput.Common.InputS
witcher > ContentControl > ContentPresenter > Grid", {
        L"Shadow :="}},
    ThemeTargetStyles{L"Grid#ContainerGrid@ > Rectangle#ShowDesktopPipe", {
        L"Grid.Row = 0",
        L"Grid.RowSpan = 3",
        L"Grid.Column = 0",
        L"Grid.ColumnSpan = 3",
        L"Width = 4",
        L"Height = 4",
        L"Height@PointerOver = 10",
        L"Height@Pressed = 6",
        L"RadiusX = 2",
        L"RadiusY = 2",
        L"Margin := 0,{{$taskbarTopOffset}},0,{{$taskbarBottomOffset+$taskbarContentOffset-4}}",
        L"Fill := $showDesktopIndicatorColor"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#OverflowToggleButtonRootPanel", {
        L"Margin := 8,{{$taskbarTopOffset}},0,{{$taskbarBottomOffset}}",
        L"Padding = 0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel#OverflowToggleButtonRootPanel > Grid >
Border#BackgroundElement, Taskbar.TaskListButtonPanel#OverflowToggleButtonRootPanel >
Border#BackgroundElement", {
        L"CornerRadius := $highlightRadius"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > Taskbar.FlyoutFrame >
Canvas#HoverFlyoutCanvas > Grid#HoverFlyoutGrid > Border#HoverFlyoutBackground", {
        L"Shadow :="}},

ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#OverflowFlyoutListRepeater >
Taskbar.TaskListButton#TaskListButton > Grid#IconPanel,
Microsoft.UI.Xaml.Controls.ItemsRepeater#OverflowFlyoutListRepeater >
Taskbar.TaskListButton#TaskListButton > Taskbar.TaskListLabeledButtonPanel#IconPanel", {
        L"MinWidth = 28",
        L"Padding := {{$buttonSpacing-2}},4"}},

ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#OverflowFlyoutListRepeater >
Taskbar.TaskListButton#TaskListButton > Grid#IconPanel > Image#Icon,
Microsoft.UI.Xaml.Controls.ItemsRepeater#OverflowFlyoutListRepeater >
Taskbar.TaskListButton#TaskListButton > Taskbar.TaskListLabeledButtonPanel#IconPanel >
Image#Icon", {
        L"Margin = 0"}},

ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#OverflowFlyoutListRepeater >
Taskbar.TaskListButton#TaskListButton > Grid#IconPanel > Border#BackgroundElement,
Microsoft.UI.Xaml.Controls.ItemsRepeater#OverflowFlyoutListRepeater >
Taskbar.TaskListButton#TaskListButton > Taskbar.TaskListLabeledButtonPanel#IconPanel >
Border#BackgroundElement", {
        L"Margin = 0"}},

ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#OverflowFlyoutListRepeater >
Taskbar.TaskListButton#TaskListButton > Grid#IconPanel > Rectangle#RunningIndicator,
Microsoft.UI.Xaml.Controls.ItemsRepeater#OverflowFlyoutListRepeater >
```

```
Taskbar.TaskListButton#TaskListButton > Taskbar.TaskListLabeledButtonPanel#IconPanel > Rectangle#RunningIndicator", {
      L"Opacity = 0"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Path#BlobShape", {
      L"Fill := $blobFill"}},
}, {
   L"taskbarLeftOffset = 12",
   L"taskbarRightOffset = 12",
   L"taskbarTopOffset = 8",
   L"taskbarBottomOffset = 4",
   L"taskbarStrokeHeight = 4",
   L"taskbarContentOffset = 4",
   L"highlightRadius = 8",
   L"buttonMinWidth = 36",
   L"buttonSpacing = 4",
   L"iconSpacing = 8",
   L"iconLabelSpacing = 4",
   L"leftRightPadding = 4",
   L"badgeSize = 12",
   L"badgeNudge = 4,6,0,0",
   L"showStartButton = 1",
   L"blobFill = <SolidColorBrush Color=\"{ThemeResource AdaptiveBlob}\"/>",
   L"taskbarStrokeColor = <SolidColorBrush Color=\"{ThemeResource AdaptiveBlob}\"/>",
   L"taskbarFill = <SolidColorBrush Color=\"{ThemeResource Default}\"/>",
   L"progressColor = <SolidColorBrush Color=\"{ThemeResource SystemAccentColor}\" Opacity=\"0.2\"/>",
   L"progressPausedColor = <SolidColorBrush Color=\"orange\" Opacity=\"0.2\"/>",
   L"showDesktopIndicatorColor = <SolidColorBrush Color=\"{ThemeResource SystemAccentColor}\" Opacity=\"0.7\"/>",
   L"multiWinIndicatorColor = <SolidColorBrush Color=\"{ThemeResource AdaptiveIndicator}\" Opacity=\"0.7\"/>",
   L"separatorColor = <SolidColorBrush Color=\"{ThemeResource AdaptiveIndicator}\" Opacity=\"0.15\"/>",
}, {
   L"AdaptiveIndicator@Light =#000000",
   L"AdaptiveIndicator@Dark =#FFFFFF",
   L"AdaptiveBlob@Light =#FFFFFF",
   L"AdaptiveBlob@Dark =#09131E",
}};

const Theme g_themeMinecraft_Hotbar = {{
   ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Image#Icon, Taskbar.TaskListLabeledButtonPanel#IconPanel@CommonStates > Image#Icon", {
      L"Margin=5.5,0,-5.5,0",
      L"Margin@ActiveNormal=9,0,-10,0",
      L"Margin@ActivePointerOver=9,0,-10,0",
      L"Margin@InactivePointerOver=7,0,-7,0",
      L"Margin@ActivePressed=9,0,-10,0"}},
   ThemeTargetStyles{L"Grid#RootGrid > Taskbar.TaskbarBackground > Grid", {
      L"Background:=transparent"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
      L"Background:=transparent"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground#BackgroundControl > Grid > Rectangle#BackgroundFill", {
      L"Margin=0,9,0,9",
      L"Fill:=<WindhawkBlur BlurAmount=\"4\" />"}},
   ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
      L"Fill=Transparent"}},
```

```
   ThemeTargetStyles{L"Taskbar.TaskListButtonPanel", {
      L"MinWidth=55"}},
   ThemeTargetStyles{L"Grid#IconPanel, Taskbar.TaskListLabeledButtonPanel", {
      L"MinWidth=55"}},
   ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl > Grid >
SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel", {
      L"MinWidth=55"}},
   ThemeTargetStyles{L"Taskbar.TaskListButtonPanel@CommonStates > Grid >
Border#BackgroundElement, Taskbar.TaskListButtonPanel@CommonStates >
Border#BackgroundElement", {
      L"CornerRadius=0",
      L"Margin=0,5.5,0,5.5"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid@DockingStates", {
      L"Tag=horizontal",
      L"Tag@DockedLeft=vertical",
      L"Tag@DockedRight=vertical",
      L"Tag=>taskbarDock"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
      L"Width={{taskbarDock==`vertical`?skip():`Auto`}}",
      L"MaxWidth={{containerGridWidth>0?containerGridWidth:`Infinity`}}",
      L"Height={{taskbarDock==`vertical`?skip():70}}",
      L"MinWidth:=100",
      L"Grid.Column=1",
      L"HorizontalAlignment=Center"}},
   ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
      L"Transitions:=<TransitionCollection><EntranceThemeTransition HorizontalOffset=\"0\"
VerticalOffset=\"0\" /></TransitionCollection>",
      L"Margin=0,4,0,-4",
      L"Padding=20,0,20,0"}},
   ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
      L"Height=70",
      L"CornerRadius=0",
      L"Grid.Column=2",
      L"Width=Auto",
      L"HorizontalAlignment=Left"}},
   ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
      L"Background:=<WindhawkBlur BlurAmount=\"4\" />",
      L"Height=55",
      L"Margin=0,-7,0,7",
      L"VerticalAlignment=Bottom",
      L"BorderBrush=transparent",
      L"BorderThickness=0",
      L"CornerRadius=0",
      L"Visibility=Visible"}},
   ThemeTargetStyles{L":root > ScrollViewer > ScrollContentPresenter > Border > Grid", {
      L"ColumnDefinitions:=<ColumnDefinitionCollection><ColumnDefinition
Width=\"*\"/><ColumnDefinition Width=\"Auto\"/><ColumnDefinition
Width=\"Auto\"/><ColumnDefinition Width=\"*\"/></ColumnDefinitionCollection>",
      L"ActualWidth=>containerGridWidth"}},
   ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Rectangle#RunningIndicator,
Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Rectangle#RunningIndicator", {
      L"Fill:=#90ffffff",
      L"RadiusX=0",
      L"RadiusY=0",
      L"Margin=-1",
      L"Height=2",
      L"Width=4",
```

```
        L"Width@ActiveRunningIndicator=8",
        L"Fill@ActiveRunningIndicator=#60CDFF",
        L"Margin=6,0,-6,0",
        L"Margin@ActiveRunning=20,0,-20,0"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.
AutomationId=StartButton] > Taskbar.TaskListButtonPanel > Grid >
Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon,
Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.AutomationId=StartB
utton] > Taskbar.TaskListButtonPanel > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon",
{
        L"Visibility=0"}},
    ThemeTargetStyles{L"Border#MultiWindowElement", {
        L"Visibility=Collapsed"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.
Name=Task View] > Taskbar.TaskListButtonPanel > Grid >
Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer,
Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.Name=Task View] >
Taskbar.TaskListButtonPanel > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer", {
        L"Visibility=0"}},

ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.
Name=Task View] > Taskbar.TaskListButtonPanel > Grid > Viewbox,
Taskbar.ExperienceToggleButton#LaunchListButton[AutomationProperties.Name=Task View] >
Taskbar.TaskListButtonPanel > Viewbox", {
        L"Visibility=0"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl > Grid >
SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Grid >
Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer, SearchUx.SearchUI.SearchButtonControl >
Grid > SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel >
Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer", {
        L"Visibility=0"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border#BackgroundElement,
Taskbar.TaskListLabeledButtonPanel@CommonStates > Border#BackgroundElement", {
        L"CornerRadius=0",
        L"Margin=-3,3.5,-3,3.5",
        L"Background:=$IconBackground",
        L"Background@InactivePointerOver:=$ActiveBackground",
        L"Background@ActiveNormal:=$ActiveBackground",
        L"Background@ActivePointerOver:=$ActiveBackground",
        L"Background@MultiWindowPointerOver:=$ActiveBackground",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates,
Taskbar.TaskListLabeledButtonPanel@CommonStates", {
        L"Width@InactivePointerOver=59",
        L"Width@MultiWindowPointerOver=59",
        L"Margin@InactivePointerOver=0,-5,0,-5",
        L"Margin@MultiWindowPointerOver=0,-5,0,-5",
        L"Width@ActiveNormal=63",
        L"Width@ActivePointerOver=63",
        L"Width@MultiWindowPointerOver=63",
        L"Width@ActivePressed=63",
        L"Margin@ActivePointerOver=0,-5,0,-5",
        L"Margin@ActivePressed=0,-5,0,-5",
        L"Margin@ActiveNormal=0,-5,0,-5"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel@CommonStates > Grid >
Border#BackgroundElement, Taskbar.TaskListButtonPanel@CommonStates >
Border#BackgroundElement", {
        L"Canvas.ZIndex=0",
        L"CornerRadius=0",
        L"Margin=-3,3.5,-3,3.5",
        L"Background:=$IconBackground",
        L"Background@InactivePointerOver:=$ActiveBackground",
```

```
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel@CommonStates", {
        L"Canvas.ZIndex=0",
        L"Width@InactivePointerOver=59",
        L"Margin@InactivePointerOver=0,-5,0,-5"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl > Grid >
SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel@CommonStates >
Grid > Border#BackgroundElement, SearchUx.SearchUI.SearchButtonControl > Grid >
SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel@CommonStates >
Border#BackgroundElement", {
        L"CornerRadius=0",
        L"Margin=-3,3.5,-3,3.5",
        L"Background:=$IconBackground",
        L"Background@InactivePointerOver:=$ActiveBackground",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl > Grid >
SearchUx.SearchUI.SearchIconButton >
SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel@CommonStates", {
        L"Canvas.ZIndex=0",
        L"Width@InactivePointerOver=59",
        L"Margin@InactivePointerOver=0,-5,0,-5"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid >
SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView >
Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid,
SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid >
SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView >
Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid", {
        L"CornerRadius=0",
        L"Height=55",
        L"Width=55",
        L"Width@PointerOver=59",
        L"CornerRadius=0",
        L"Background:=$IconBackground",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"SystemTray.LanguageTextIconContent >
Windows.UI.Xaml.Controls.Grid#ContainerGrid", {
        L"CornerRadius=0",
        L"Height=55",
        L"Width=55",
        L"Margin=-1,0,-13,0",
        L"Background:=$IconBackground",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel >
ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid >
Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid,
SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid >
ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent >
Grid#ContainerGrid", {
        L"CornerRadius=0",
        L"Height=55",
        L"Width=55",
        L"Background:=$IconBackground",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack", {
        L"Visibility=1",
        L"CornerRadius=0",
        L"Height=55",
        L"Width=55",
        L"Background:=$IconBackground",
        L"BorderThickness=0"}},
```

```
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[1] > SystemTray.IconView > Grid > Grid", {
        L"Margin=-3,0,-8,0",
        L"Visibility=visible",
        L"CornerRadius=0",
        L"Height=55",
        L"Width=55",
        L"Background:=$IconBackground",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[2] > SystemTray.IconView > Grid > Grid", {
        L"Margin=8,0,-8,0",
        L"Visibility=1",
        L"CornerRadius=0",
        L"Height=55",
        L"Width=55",
        L"Background:=$IconBackground",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[3] > SystemTray.IconView > Grid > Grid", {
        L"Margin=8,0,-8,0",
        L"Visibility=Visible",
        L"CornerRadius=0",
        L"Height=55",
        L"Width=55",
        L"Background:=$IconBackground",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.BatteryIconContent > Grid#ContainerGrid > StackPanel, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.BatteryIconContent > Grid#ContainerGrid > StackPanel", {
        L"HorizontalAlignment=Center"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock > TextBlock", {
        L"FontSize=30"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock", {
        L"FontSize=32",
        L"Foreground=white"}},
    ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
        L"FontWeight=Bold",
        L"Margin=3,9,7,-9",
        L"Foreground=White",
        L"Visibility=visible",
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"-9\"/>",
        L"FontSize=13",
        L"FontFamily=vivo Sans EN VF"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent", {
        L"Foreground=white",
```

```
        L"FontWeight=bold",
        L"FontSize=32"}},
    ThemeTargetStyles{L"TextBlock#TimeInnerTextBlock", {
        L"Foreground=white",
        L"Width=Auto",
        L"FontWeight=Bold",
        L"FontSize=15",
        L"Margin=6,-4,6,4"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock", {
        L"Foreground=white",
        L"FontSize=30"}},
    ThemeTargetStyles{L"TextBlock#SearchBoxTextBlock", {
        L"FontSize=12",
        L"Foreground=White"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock#LanguageInnerTextBlock > TextBlock#InnerTextBlock", {
        L"FontSize=16",
        L"FontWeight=Bold"}},
    ThemeTargetStyles{L"SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.DateTimeIconContent > Grid#ContainerGrid", {
        L"CornerRadius=0",
        L"BorderThickness=0",
        L"Background:=<ImageBrush ImageSource=\"https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Minecraft%20Hotbar/Assets/hotbar.png\" />",
        L"Height=55",
        L"Margin=1,0,-5,0",
        L"Width=auto"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.Stack#NonActivatableStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Border#BackgroundBorder, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.Stack#NonActivatableStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Border#BackgroundBorder", {
        L"Background:=Transparent",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid > Border#BackgroundBorder", {
        L"Background:=Transparent",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid > Rectangle#BackgroundFill", {
        L"Canvas.ZIndex=0",
        L"Fill:=",
        L"Stroke:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
        L"StrokeThickness=5",
        L"RadiusX=5",
        L"RadiusY=5"}},
    ThemeTargetStyles{L"Taskbar.FlyoutFrame > Windows.UI.Xaml.Controls.Canvas#HoverFlyoutCanvas > Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid > Windows.UI.Xaml.Controls.ContentPresenter#HoverFlyoutContent > Taskbar.TaskItemThumbnailList > Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskItemThumbnailListRepeater > Taskbar.TaskItemThumbnailView > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"VerticalAlignment=Bottom",
```

```
    L"Canvas.ZIndex=1",
    L"Background:=<WindhawkBlur BlurAmount=\"5\" TintColor=\"#761E1E1E\"/>",
    L"Height=25",
    L"CornerRadius=0,0,15,15",
    L"Margin=5,0,5,0"}},
  ThemeTargetStyles{L"Border#HoverFlyoutBackground", {
    L"Margin=4,36,4,0",
    L"Canvas.ZIndex=1",
    L"Width=Auto",
    L"Background:=Transparent",
    L"BorderThickness=0",
    L"CornerRadius=5"}},
  ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#ThumbBarRepeater > Taskbar.ThumbBarButton#ThumbBarButton > Windows.UI.Xaml.Controls.ContentPresenter#BorderElement", {
    L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#761E1E1E\"/>",
    L"Margin=0,-20,0,20"}},
  ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#IconsRepeater > Windows.UI.Xaml.Controls.Image", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#CloseButton", {
    L"Grid.ColumnSpan=1",
    L"Grid.RowSpan=1",
    L"Canvas.ZIndex=1",
    L"CornerRadius=0",
    L"Width=28",
    L"Height=28",
    L"Margin=-18,40,15,-40",
    L"Background:=#1D1D1D",
    L"Foreground=white",
    L"BorderThickness=3",
    L"BorderBrush:=<LinearGradientBrush StartPoint=\"-0.07,-0.06\" EndPoint=\"1.02,1.04\"><GradientStop Offset=\"0.49\" Color=\"#000000\"/><GradientStop Offset=\"0.53\" Color=\"#3A3636\"/></LinearGradientBrush>"}},
  ThemeTargetStyles{L"Taskbar.FlyoutFrame > Windows.UI.Xaml.Controls.Canvas#HoverFlyoutCanvas > Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid > Windows.UI.Xaml.Controls.ContentPresenter#HoverFlyoutContent > Taskbar.TaskItemThumbnailList > Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskItemThumbnailListRepeater > Taskbar.TaskItemThumbnailView > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.TextBlock#DisplayNameTextBlock", {
    L"Foreground=white",
    L"Grid.ColumnSpan=2",
    L"Grid.RowSpan=2",
    L"VerticalAlignment=bottom",
    L"HorizontalAlignment=Center",
    L"Margin=0,-5,0,5",
    L"Canvas.ZIndex=1"}},
  ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > SystemTray.NotificationAreaOverflow > Grid#OverflowRootGrid > Border#OverflowFlyoutBackgroundBorder", {
    L"Background:=<WindhawkBlur BlurAmount=\"4\" />"}},
  ThemeTargetStyles{L"SystemTray.OmniButton#NotificationCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid", {
    L"Visibility=1"}},
  ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > SystemTray.NotificationAreaOverflow > Grid#OverflowRootGrid > ItemsControl > ItemsPresenter > WrapGrid > ContentPresenter > SystemTray.NotifyIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.ImageIconContent > Grid#ContainerGrid", {
    L"Background:=$IconBackground"}},
```

```
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > Grid > SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.NotificationAreaIcons#NotificationAreaIcons > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.NotifyIconView#NotifyItemIcon > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.ImageIconContent > Grid#ContainerGrid, ScrollViewer > ScrollContentPresenter > Border > Grid > SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.NotificationAreaIcons#NotificationAreaIcons > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.NotifyIconView#NotifyItemIcon > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.ImageIconContent > Grid#ContainerGrid", {
        L"Width=55",
        L"Height=55",
        L"Background:=$IconBackground"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > Grid > Taskbar.TaskBarFrame > Grid#RootGrid > Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater > Taskbar.TaskListButton#TaskListButton > Grid#IconPanel > TextBlock#LabelControl, ScrollViewer > ScrollContentPresenter > Border > Grid > Taskbar.TaskbarFrame > Grid#RootGrid > Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskbarFrameRepeater > Taskbar.TaskListButton#TaskListButton > Taskbar.TaskListLabeledButtonPanel#IconPanel > TextBlock#LabelControl", {
        L"Visibility=1"}},
}, {
    L"IconBackground= <ImageBrush Stretch=\"UniformtoFill\" ImageSource=\"https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Minecraft%20Hotbar/Assets/hotbar.png\" />",
    L"ActiveBackground= <ImageBrush Stretch=\"UniformtoFill\" ImageSource=\"https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/Minecraft%20Hotbar/Assets/hotbaractive.png\" />",
}};


const Theme g_themeOne_UI_8_5_variant_Dock = {{
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.GamingApp_8wekyb3d8bbwe!Microsoft.Xbox.AppL] > Grid#IconPanel > Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.GamingApp_8wekyb3d8bbwe!Microsoft.Xbox.App] > Grid#IconPanel > Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.GamingApp_8wekyb3d8bbwe!App] > Grid#IconPanel > Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.GamingApp_8wekyb3d8bbwe!Microsoft.Xbox.AppL] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.GamingApp_8wekyb3d8bbwe!Microsoft.Xbox.App] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.GamingApp_8wekyb3d8bbwe!App] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/xbox.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.GamingApp_8wekyb3d8bbwe!Microsoft.Xbox.AppL] > Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.GamingApp_8wekyb3d8bbwe!Microsoft.Xbox.App] > Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.GamingApp_8wekyb3d8bbwe!App] > Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.GamingApp_8wekyb3d8bbwe!Microsoft.Xbox.AppL] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.GamingApp_8wekyb3d8bbwe!Microsoft.Xbox.App] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.GamingApp_8wekyb3d8bbwe!App] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.WindowsCamera_8wekyb3d8bbwe!App] > Grid#IconPanel > Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.WindowsCamera_8wekyb3d8bbwe!App] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/windowscamera.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
```

ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsCamera_8wekyb3d8bbwe!App] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsCamera_8wekyb3d8bbwe!App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {

L"Visibility=Collapsed"}},

ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
{1AC14E77-02E7-4E5D-B744-2EB1AE5198B7}\\cmd.exe] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
C:\\Windows\\System32\\cmd.exe] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {1AC14E77-02E7-4E5D-
B744-2EB1AE5198B7}\\cmd.exe] > Taskbar.TaskListLabeledButtonPanel#IconPanel >
Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
C:\\Windows\\System32\\cmd.exe] > Taskbar.TaskListLabeledButtonPanel#IconPanel >
Image#Icon", {

L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/cmd.png",

L"Height=40",

L"Width=40",

L"Margin=0,0,0,0"}},

ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
{1AC14E77-02E7-4E5D-B744-2EB1AE5198B7}\\cmd.exe] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
C:\\Windows\\System32\\cmd.exe] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {1AC14E77-02E7-4E5D-
B744-2EB1AE5198B7}\\cmd.exe] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
C:\\Windows\\System32\\cmd.exe] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border",
{

L"Visibility=Collapsed"}},

ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
{1AC14E77-02E7-4E5D-B744-2EB1AE5198B7}\\WindowsPowerShell\\v1.0\\powershell.exe] >
Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {A7CC3F66-580E-49E6-
913B-556111443D24}\\Windows PowerShell\\Windows PowerShell.lnk] > Grid#IconPanel >
Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {1AC14E77-
02E7-4E5D-B744-2EB1AE5198B7}\\WindowsPowerShell\\v1.0\\powershell.exe] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {A7CC3F66-580E-49E6-
913B-556111443D24}\\Windows PowerShell\\Windows PowerShell.lnk] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {

L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/powershell.png",

L"Height=40",

L"Width=40",

L"Margin=0,0,0,0"}},

ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
{1AC14E77-02E7-4E5D-B744-2EB1AE5198B7}\\WindowsPowerShell\\v1.0\\powershell.exe] >
Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
{A7CC3F66-580E-49E6-913B-556111443D24}\\Windows PowerShell\\Windows
PowerShell.lnk] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {1AC14E77-02E7-4E5D-
B744-2EB1AE5198B7}\\WindowsPowerShell\\v1.0\\powershell.exe] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {A7CC3F66-580E-49E6-
913B-556111443D24}\\Windows PowerShell\\Windows PowerShell.lnk] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {

L"Visibility=Collapsed"}},

ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
{1AC14E77-02E7-4E5D-B744-2EB1AE5198B7}\\osk.exe] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {1AC14E77-02E7-4E5D-
B744-2EB1AE5198B7}\\osk.exe] > Taskbar.TaskListLabeledButtonPanel#IconPanel >
Image#Icon", {

L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/osk.png",

L"Height=40",

L"Width=40",

L"Margin=0,0,0,0"}},

ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
{1AC14E77-02E7-4E5D-B744-2EB1AE5198B7}\\osk.exe] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {1AC14E77-02E7-4E5D-
B744-2EB1AE5198B7}\\osk.exe] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {

L"Visibility=Collapsed"}},

ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Paint_8wekyb3d8bbwe!App] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Paint_8wekyb3d8bbwe!App] > Taskbar.TaskListLabeledButtonPanel#IconPanel >
Image#Icon", {

L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/paint.png",
L"Height=40",
L"Width=40",
L"Margin=0,0,0,0"}},
ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.Paint_8wekyb3d8bbwe!App] > Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.Paint_8wekyb3d8bbwe!App] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
L"Visibility=Collapsed"}},
ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.ScreenSketch_8wekyb3d8bbwe!App] > Grid#IconPanel > Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.ScreenSketch_8wekyb3d8bbwe!App] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/snippingtool.png",
L"Height=40",
L"Width=40",
L"Margin=0,0,0,0"}},
ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.ScreenSketch_8wekyb3d8bbwe!App] > Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.ScreenSketch_8wekyb3d8bbwe!App] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
L"Visibility=Collapsed"}},
ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:SpotifyAB.SpotifyMusic_zpdnekdrzrea0!Spotify] > Grid#IconPanel > Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:SpotifyAB.SpotifyMusic_zpdnekdrzrea0!Spotify] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/Spotify.png",
L"Height=40",
L"Width=40",
L"Margin=0,0,0,0"}},
ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:SpotifyAB.SpotifyMusic_zpdnekdrzrea0!Spotify] > Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:SpotifyAB.SpotifyMusic_zpdnekdrzrea0!Spotify] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
L"Visibility=Collapsed"}},
ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.Windows.Photos_8wekyb3d8bbwe!App] > Grid#IconPanel > Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.Windows.Photos_8wekyb3d8bbwe!App] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/photos.png",
L"Height=40",
L"Width=40",
L"Margin=0,0,0,0"}},
ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.Windows.Photos_8wekyb3d8bbwe!App] > Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.Windows.Photos_8wekyb3d8bbwe!App] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
L"Visibility=Collapsed"}},
ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.ZuneMusic_8wekyb3d8bbwe!Microsoft.ZuneMusic] > Grid#IconPanel > Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.ZuneMusic_8wekyb3d8bbwe!Microsoft.ZuneMusic] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/mediaplayer.png",
L"Height=40",
L"Width=40",
L"Margin=0,0,0,0"}},
ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.ZuneMusic_8wekyb3d8bbwe!Microsoft.ZuneMusic] > Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:

```
Microsoft.ZuneMusic_8wekyb3d8bbwe!Microsoft.ZuneMusic] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Valve.Steam.Client] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Valve.Steam.Client] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/steam.png",
    L"Height=40",
    L"Width=40",
    L"Margin=0,0,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Valve.Steam.Client] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Valve.Steam.Client] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsCalculator_8wekyb3d8bbwe!App] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsCalculator_8wekyb3d8bbwe!App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/calculator.png",
    L"Height=40",
    L"Width=40",
    L"Margin=0,0,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsCalculator_8wekyb3d8bbwe!App] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsCalculator_8wekyb3d8bbwe!App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsAlarms_8wekyb3d8bbwe!App] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsAlarms_8wekyb3d8bbwe!App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/time.png",
    L"Height=40",
    L"Width=40",
    L"Margin=0,0,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsAlarms_8wekyb3d8bbwe!App] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsAlarms_8wekyb3d8bbwe!App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.SecHealthUI_8wekyb3d8bbwe!SecHealthUI] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.SecHealthUI_8wekyb3d8bbwe!SecHealthUI] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/security.png",
    L"Height=40",
    L"Width=40",
    L"Margin=0,0,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.SecHealthUI_8wekyb3d8bbwe!SecHealthUI] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.SecHealthUI_8wekyb3d8bbwe!SecHealthUI] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsStore_8wekyb3d8bbwe!App] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsStore_8wekyb3d8bbwe!App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/store.png",
    L"Height=40",
```

```
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsStore_8wekyb3d8bbwe!App] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsStore_8wekyb3d8bbwe!App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
5319275A.WhatsAppDesktop_cv1g1gvanyjgm!App] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
5319275A.WhatsAppDesktop_cv1g1gvanyjgm!App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/whatsapp.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
5319275A.WhatsAppDesktop_cv1g1gvanyjgm!App] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
5319275A.WhatsAppDesktop_cv1g1gvanyjgm!App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Office.WINWORD.EXE.15] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Office.WINWORD.EXE.15] > Taskbar.TaskListLabeledButtonPanel#IconPanel >
Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/word.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Office.WINWORD.EXE.15] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Office.WINWORD.EXE.15] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border",
{
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Office.POWERPNT.EXE.15] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Office.POWERPNT.EXE.15] > Taskbar.TaskListLabeledButtonPanel#IconPanel >
Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/powerpoint.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Office.POWERPNT.EXE.15] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Office.POWERPNT.EXE.15] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border",
{
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Office.ONENOTE.EXE.15] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Office.ONENOTE.EXE.15] > Taskbar.TaskListLabeledButtonPanel#IconPanel >
Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/onenote.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Office.ONENOTE.EXE.15] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Office.ONENOTE.EXE.15] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.OneDrive] > Grid#IconPanel > Image#Icon,
```

```
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.OneDrive] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/OneDrive.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.OneDrive] > Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.OneDrive] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: MSTeams_8wekyb3d8bbwe!MSTeams] > Grid#IconPanel > Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: MSTeams_8wekyb3d8bbwe!MSTeams] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/ms-teams.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: MSTeams_8wekyb3d8bbwe!MSTeams] > Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: MSTeams_8wekyb3d8bbwe!MSTeams] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.OutlookForWindows_8wekyb3d8bbwe!Microsoft.OutlookForWindows] > Grid#IconPanel > Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.OutlookForWindows_8wekyb3d8bbwe!Microsoft.OutlookForWindows] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/outlook.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.OutlookForWindows_8wekyb3d8bbwe!Microsoft.OutlookForWindows] > Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.OutlookForWindows_8wekyb3d8bbwe!Microsoft.OutlookForWindows] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.Windows.ControlPanel] > Grid#IconPanel > Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.Windows.ControlPanel] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/ControlPanel.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.Windows.ControlPanel] > Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.Windows.ControlPanel] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.Windows.Explorer] > Grid#IconPanel > Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.Windows.Explorer] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/explorer.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.Windows.Explorer] > Grid#IconPanel > Border,
```

```
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Windows.Explorer] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
com.squirrel.Discord.Discord] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
com.squirrel.Discord.Discord] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/discord.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
com.squirrel.Discord.Discord] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
com.squirrel.Discord.Discord] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
windows.immersivecontrolpanel_cw5n1h2txyewy!microsoft.windows.immersivecontrolpanel] >
Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
windows.immersivecontrolpanel_cw5n1h2txyewy!microsoft.windows.immersivecontrolpanel] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/systemsettings.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
windows.immersivecontrolpanel_cw5n1h2txyewy!microsoft.windows.immersivecontrolpanel] >
Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
windows.immersivecontrolpanel_cw5n1h2txyewy!microsoft.windows.immersivecontrolpanel] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsNotepad_8wekyb3d8bbwe!App] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsNotepad_8wekyb3d8bbwe!App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/notepad.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsNotepad_8wekyb3d8bbwe!App] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsNotepad_8wekyb3d8bbwe!App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
com.github.th-ch.youtube-music] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: com.github.th-ch.youtube-
music] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/YouTubeMusic.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
com.github.th-ch.youtube-music] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: com.github.th-ch.youtube-
music] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
BlueStacks_nxt] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: BlueStacks_nxt] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/Bluestacks.png",
        L"Height=40",
        L"Width=40",
```

```
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: BlueStacks_nxt] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: BlueStacks_nxt] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {7C5A40EF-A0FB-4BFC-874A-C0F2E0B9FA8E}\\NCH Software\\DeskFX\\deskfx.exe] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {7C5A40EF-A0FB-4BFC-874A-C0F2E0B9FA8E}\\NCH Software\\DeskFX\\deskfx.exe] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/DeskFX.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {7C5A40EF-A0FB-4BFC-874A-C0F2E0B9FA8E}\\NCH Software\\DeskFX\\deskfx.exe] > Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {7C5A40EF-A0FB-4BFC-874A-C0F2E0B9FA8E}\\NCH Software\\DeskFX\\deskfx.exe] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: RamenSoftware.Windhawk] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: RamenSoftware.Windhawk] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/windhawk.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: RamenSoftware.Windhawk] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: RamenSoftware.Windhawk] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: 5319275A.51895FA4EA97F_cv1g1gvanyjgm!App] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: 5319275A.51895FA4EA97F_cv1g1gvanyjgm!App] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/whatsapp.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: 5319275A.51895FA4EA97F_cv1g1gvanyjgm!App] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: 5319275A.51895FA4EA97F_cv1g1gvanyjgm!App] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel", {
        L"Width=45",
        L"Height=60"}},
    ThemeTargetStyles{L"Grid#IconPanel, Taskbar.TaskListLabeledButtonPanel", {
        L"Width=45",
        L"Height=60"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl > Grid > SearchUx.SearchUI.SearchIconButton > SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel", {
        L"Width=45",
        L"Height=60"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border#BackgroundElement, Taskbar.TaskListLabeledButtonPanel@CommonStates > Border#BackgroundElement", {
        L"CornerRadius=0",
        L"Margin=0,5.5,0,5.5",
        L"Background:=$IconBackground",
```

```
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.3"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl > Grid > SearchUx.SearchUI.SearchIconButton > SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Grid > Border#BackgroundElement, SearchUx.SearchUI.SearchButtonControl > Grid > SearchUx.SearchUI.SearchIconButton > SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Border#BackgroundElement", {
        L"Margin=0,5.5,0,5.5",
        L"CornerRadius=0",
        L"Background:=<ImageBrush Stretch=\"Uniform\"> <ImageBrush.ImageSource> <BitmapImage UriSource=\"https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/Search.png\" DecodePixelType=\"Logical\" DecodePixelWidth=\"64\" DecodePixelHeight=\"64\" /> </ImageBrush.ImageSource> </ImageBrush>",
        L"BorderBrush:=$IconBorder"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl > Grid > SearchUx.SearchUI.SearchIconButton > SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Grid > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon, SearchUx.SearchUI.SearchButtonControl > Grid > SearchUx.SearchUI.SearchIconButton > SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon", {
        L"Visibility=1"}},
    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[3] > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid > Border#BackgroundElement, Taskbar.ExperienceToggleButton#LaunchListButton[3] > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Border#BackgroundElement", {
        L"Margin=0,5.5,0,5.5",
        L"Background:=<ImageBrush Stretch=\"Uniform\"><ImageBrush.ImageSource><BitmapImage UriSource=\"https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/darkbg.png\" DecodePixelType=\"Logical\" DecodePixelWidth=\"64\" DecodePixelHeight=\"64\"/></ImageBrush.ImageSource></ImageBrush>",
        L"BorderThickness=0",
        L"CornerRadius=0"}},
    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[1] > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid > Border#BackgroundElement, Taskbar.ExperienceToggleButton#LaunchListButton[1] > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Border#BackgroundElement", {
        L"Margin=0,5.5,0,5.5",
        L"Background:=<ImageBrush Stretch=\"Uniform\"><ImageBrush.ImageSource><BitmapImage UriSource=\"https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/Start.png\" DecodePixelType=\"Logical\" DecodePixelWidth=\"64\" DecodePixelHeight=\"64\"/></ImageBrush.ImageSource></ImageBrush>",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"Border#MultiWindowElement", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Rectangle#RunningIndicator, Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Rectangle#RunningIndicator", {
        L"Fill=white"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid@DockingStates", {
        L"Tag=horizontal",
        L"Tag@DockedLeft=vertical",
        L"Tag@DockedRight=vertical",
        L"Tag=>taskbarDock"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
        L"Width={{taskbarDock==`vertical`?skip():`Auto`}}",
        L"MaxWidth={{containerGridWidth>0?containerGridWidth:`Infinity`}}",
        L"Grid.Column=1",
        L"Transitions:=<TransitionCollection><RepositionThemeTransition IsStaggeringEnabled=\"False\"/></TransitionCollection>",
        L"Height={{taskbarDock==`vertical`?skip():70}}",
        L"MaxHeight=70",
        L"HorizontalAlignment=Center"}},
```

```
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
        L"Margin=0,8,0,2",
        L"Padding=20,0,20,0",
        L"BorderBrush=#40FFFFFF"}},
    ThemeTargetStyles{L"Grid#RootGrid > Taskbar.TaskbarBackground", {
        L"Margin=-20,0,-20,0"}},
    ThemeTargetStyles{L"Grid#RootGrid > Taskbar.TaskbarBackground > Grid", {
        L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
        L"CornerRadius=25,0,0,25",
        L"BorderThickness=1,1,0,1",
        L"Width=Auto",
        L"BorderBrush=#40FFFFFF",
        L"Padding=-1"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground >
Grid > Rectangle#BackgroundFill", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
        L"Height=70",
        L"Grid.Column=2",
        L"Width=Auto",
        L"HorizontalAlignment=Left",
        L"Margin=0,-2,0.5,2"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"Margin=0,0,0.5,0",
        L"Height=60",
        L"VerticalAlignment=Bottom",
        L"Padding=0",
        L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
        L"BorderBrush=#40FFFFFF",
        L"BorderThickness=0,1,1,1",
        L"CornerRadius=0,25,25,0",
        L"Visibility=Visible",
        L"BorderBrush=#40FFFFFF"}},
    ThemeTargetStyles{L":root > ScrollViewer > ScrollContentPresenter > Border > Grid", {
        L"ColumnDefinitions:=<ColumnDefinitionCollection><ColumnDefinition
Width=\"*\"/><ColumnDefinition Width=\"Auto\"/><ColumnDefinition
Width=\"Auto\"/><ColumnDefinition Width=\"*\"/></ColumnDefinitionCollection>",
        L"ActualWidth=>containerGridWidth"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid >
SystemTray.Stack#NonActivatableStack > Grid#Content > SystemTray.StackListView#IconStack >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon >
Grid#ContainerGrid > Border#BackgroundBorder, SystemTray.SystemTrayFrame >
Grid#SystemTrayFrameGrid > SystemTray.Stack#NonActivatableStack > Grid#Content >
SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Border#BackgroundBorder", {
        L"Background:=Transparent",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid >
Border#BackgroundBorder", {
        L"Background:=Transparent",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"SystemTray.Stack#MainStack", {
        L"Visibility=1"}},
    ThemeTargetStyles{L"SystemTray.Stack#NotifyIconStack > Grid#Content >
SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.ChevronIconView > Grid#ContainerGrid > Border#BackgroundBorder", {
        L"Background=transparent",
        L"BorderBrush=#40FFFFFF",
```

```
        L"BorderThickness=2,0,0,0",
        L"Padding=0",
        L"CornerRadius=0",
        L"Height=35"}},
    ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
        L"FontWeight=Bold",
        L"Margin=-2,9,2,-9",
        L"Foreground=White",
        L"Visibility=visible",
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"-9\"/>",
        L"FontSize=13",
        L"FontFamily=vivo Sans EN VF"}},
    ThemeTargetStyles{L"TextBlock#TimeInnerTextBlock", {
        L"Foreground=white",
        L"Width=Auto",
        L"FontWeight=Bold",
        L"FontSize=14",
        L"Margin=0,-1,5,1"}},
    ThemeTargetStyles{L"SystemTray.DateTimeIconContent > Grid#ContainerGrid", {
        L"CornerRadius=12",
        L"BorderThickness=1.3",
        L"Background:=<LinearGradientBrush StartPoint=\"0.03,0.03\"
EndPoint=\"0.97,0.97\"><GradientStop Offset=\"0.03\" Color=\"#FC4418\"/><GradientStop
Offset=\"0.49\" Color=\"#F002B0\"/><GradientStop Offset=\"0.99\"
Color=\"#7300FF\"/></LinearGradientBrush>",
        L"BorderBrush:=$IconBorder",
        L"Height=40",
        L"Margin=0,0,15,-6",
        L"Width=auto"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid >
SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon >
Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid >
SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock, SystemTray.SystemTrayFrame >
Grid#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid >
ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter >
SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid >
SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base >
TextBlock#InnerTextBlock", {
        L"FontSize=25"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid >
SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView >
Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid,
SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid >
SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack >
ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView >
Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid", {
        L"Height=40",
        L"Width=40",
        L"Margin=25,2,12,0",
        L"CornerRadius=15",
        L"Background:=$IconBackground",
        L"BorderBrush:=transparent",
        L"BorderThickness=1.3"}},
    ThemeTargetStyles{L"SystemTray.LanguageTextIconContent >
Windows.UI.Xaml.Controls.Grid#ContainerGrid", {
        L"Height=40",
        L"Width=40",
        L"Margin=-6,4,-3,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.3",
        L"CornerRadius=15"}},
```

```
    ThemeTargetStyles{L"SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid, SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid", {
        L"Height=40",
        L"Width=40",
        L"Margin=-3,4,3,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.3",
        L"CornerRadius=15"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > SystemTray.NotificationAreaOverflow > Grid#OverflowRootGrid > ItemsControl > ItemsPresenter > WrapGrid > ContentPresenter > SystemTray.NotifyIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.ImageIconContent > Grid#ContainerGrid", {
        L"Background:=<ImageBrush Stretch=\"Uniform\"> <ImageBrush.ImageSource> <BitmapImage UriSource=\"https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/darkbg.png\" DecodePixelType=\"Logical\" DecodePixelWidth=\"64\" DecodePixelHeight=\"64\" /> </ImageBrush.ImageSource> </ImageBrush>",
        L"Width=35",
        L"Height=35"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > Grid > SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.NotificationAreaIcons#NotificationAreaIcons > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.NotifyIconView#NotifyItemIcon > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.ImageIconContent > Grid#ContainerGrid, ScrollViewer > ScrollContentPresenter > Border > Grid > SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.NotificationAreaIcons#NotificationAreaIcons > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.NotifyIconView#NotifyItemIcon > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.ImageIconContent > Grid#ContainerGrid", {
        L"Background:=$IconBackground",
        L"Height=40",
        L"Width=40",
        L"Margin=3,2,3,-2"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border > SystemTray.NotificationAreaOverflow > Grid#OverflowRootGrid > Border#OverflowFlyoutBackgroundBorder", {
        L"CornerRadius=24",
        L"Background:=<WindhawkBlur BlurAmount=\"8\" />"}},
    ThemeTargetStyles{L"SystemTray.Stack#ShowDesktopStack", {
        L"Visibility=1",
        L"Height=40",
        L"Width=40",
        L"Margin=0,4,0,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.3",
        L"CornerRadius=15"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[1] > SystemTray.IconView > Grid > Grid", {
        L"Visibility=visible",
        L"Height=40",
        L"Width=40",
        L"Margin=2,4,2,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.3",
        L"CornerRadius=15"}},
```

```
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[2] > SystemTray.IconView > Grid > Grid", {
        L"Visibility=visible",
        L"Height=40",
        L"Width=40",
        L"Margin=2,4,2,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.3",
        L"CornerRadius=15"}},
    ThemeTargetStyles{L"SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter[3] > SystemTray.IconView > Grid > Grid", {
        L"Visibility=Visible",
        L"Height=40",
        L"Width=40",
        L"Margin=2,4,2,0",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.3",
        L"CornerRadius=15"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.BatteryIconContent > Grid#ContainerGrid > StackPanel, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.BatteryIconContent > Grid#ContainerGrid > StackPanel", {
        L"HorizontalAlignment=Center"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock > TextBlock", {
        L"Foreground:=black",
        L"FontSize=30"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.Stack#NotifyIconStack > Grid#Content > SystemTray.StackListView#IconStack > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.ChevronIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock", {
        L"Foreground:=black",
        L"FontSize=25"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock", {
        L"VerticalAlignment=Center",
        L"HorizontalAlignment=Center"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.OmniButton#NotificationCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.OmniButton#NotificationCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock#LanguageInnerTextBlock > TextBlock#InnerTextBlock", {
        L"Foreground:=black",
```

```
        L"FontSize=12",
        L"FontWeight=Bold"}},
    ThemeTargetStyles{L"SystemTray.ChevronIconView > Grid#ContainerGrid >
ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent", {
        L"Foreground:=black",
        L"FontWeight=bold",
        L"FontSize=25"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid >
Rectangle#BackgroundStroke", {
        L"Fill:=<WindhawkBlur BlurAmount=\"3.5\" TintColor=\"#2D101010\"/>",
        L"Stroke:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
        L"StrokeThickness=5",
        L"RadiusX=14",
        L"RadiusY=14",
        L"Fill:=<<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid >
Rectangle#BackgroundFill", {
        L"Canvas.ZIndex=0",
        L"Fill:=",
        L"Stroke:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
        L"StrokeThickness=5",
        L"RadiusX=14",
        L"RadiusY=14"}},
    ThemeTargetStyles{L"Taskbar.FlyoutFrame >
Windows.UI.Xaml.Controls.Canvas#HoverFlyoutCanvas >
Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid >
Windows.UI.Xaml.Controls.ContentPresenter#HoverFlyoutContent >
Taskbar.TaskItemThumbnailList >
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskItemThumbnailListRepeater >
Taskbar.TaskItemThumbnailView > Windows.UI.Xaml.Controls.Grid >
Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"VerticalAlignment=Bottom",
        L"Canvas.ZIndex=1",
        L"Background:=<WindhawkBlur BlurAmount=\"5\" TintColor=\"#761E1E1E\"/>",
        L"Height=25",
        L"CornerRadius=0,0,15,15",
        L"Margin=5,0,5,0"}},
    ThemeTargetStyles{L"Border#HoverFlyoutBackground", {
        L"Margin=4,36,4,0",
        L"Canvas.ZIndex=1",
        L"Width=Auto",
        L"Background:=Transparent",
        L"BorderThickness=0",
        L"CornerRadius=15"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#ThumbBarRepeater >
Taskbar.ThumbBarButton#ThumbBarButton >
Windows.UI.Xaml.Controls.ContentPresenter#BorderElement", {
        L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#761E1E1E\"/>",
        L"Margin=0,-20,0,20"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#IconsRepeater >
Windows.UI.Xaml.Controls.Image", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#CloseButton", {
        L"HorizontalAlignment=left",
        L"Grid.ColumnSpan=1",
        L"Grid.RowSpan=1",
        L"Canvas.ZIndex=1",
        L"CornerRadius=20",
        L"Width=28",
        L"Height=28",
```

```
      L"Margin=-18,40,15,-40",
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#80ffffff\"/>",
      L"Foreground=black",
      L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#761E1E1E\"/>"}},
   ThemeTargetStyles{L"Taskbar.FlyoutFrame >
Windows.UI.Xaml.Controls.Canvas#HoverFlyoutCanvas >
Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid >
Windows.UI.Xaml.Controls.ContentPresenter#HoverFlyoutContent >
Taskbar.TaskItemThumbnailList >
Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskItemThumbnailListRepeater >
Taskbar.TaskItemThumbnailView > Windows.UI.Xaml.Controls.Grid >
Windows.UI.Xaml.Controls.TextBlock#DisplayNameTextBlock", {
      L"Grid.ColumnSpan=2",
      L"Grid.RowSpan=2",
      L"VerticalAlignment=bottom",
      L"HorizontalAlignment=Center",
      L"Margin=0,-5,0,5",
      L"Canvas.ZIndex=1"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#ConfirmatorMainGrid", {
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"CornerRadius=24",
      L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
      L"BorderThickness=2",
      L"Margin=0,0,0,10"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid.Border#ConfirmatorMainGrid", {
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalTrackRect", {
      L"Fill=#20ffffff",
      L"RadiusX=12",
      L"RadiusY=12",
      L"Height=18",
      L"Margin=0"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalDecreaseRect", {
      L"Fill:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColor}\" />",
      L"RadiusX=12",
      L"RadiusY=12",
      L"Height=18"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#VolumeConfirmator", {
      L"Padding=8,0,8,0"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#BrightnessConfirmator", {
      L"Padding=8,0,8,0"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.TextBlock#volumeLevelText", {
      L"Foreground=White"}},
}, {
   L"IconBackground=<ImageBrush Stretch=\"Uniform\"> <ImageBrush.ImageSource>
<BitmapImage UriSource=\"https://raw.githubusercontent.com/ramensoftware/windows-11-
taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/template.png\"
DecodePixelType=\"Logical\" DecodePixelWidth=\"64\" DecodePixelHeight=\"64\" />
</ImageBrush.ImageSource> </ImageBrush>",
   L"IconBorder=transparent",
}};

const Theme g_themeOne_UI_8_5_variant_Taskbar = {{
   ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.GamingApp_8wekyb3d8bbwe!Microsoft.Xbox.AppL] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.GamingApp_8wekyb3d8bbwe!Microsoft.Xbox.App] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.GamingApp_8wekyb3d8bbwe!App] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.GamingApp_8wekyb3d8bbwe!Microsoft.Xbox.AppL] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.GamingApp_8wekyb3d8bbwe!Microsoft.Xbox.App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon,
```

```
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.GamingApp_8wekyb3d8bbwe!App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/xbox.png",
    L"Height=40",
    L"Width=40",
    L"Margin=0,0,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.GamingApp_8wekyb3d8bbwe!Microsoft.Xbox.App] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.GamingApp_8wekyb3d8bbwe!Microsoft.Xbox.App] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.GamingApp_8wekyb3d8bbwe!App] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.GamingApp_8wekyb3d8bbwe!Microsoft.Xbox.App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.GamingApp_8wekyb3d8bbwe!Microsoft.Xbox.App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.GamingApp_8wekyb3d8bbwe!App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsCamera_8wekyb3d8bbwe!App] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsCamera_8wekyb3d8bbwe!App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/windowscamera.png",
    L"Height=40",
    L"Width=40",
    L"Margin=0,0,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
{1AC14E77-02E7-4E5D-B744-2EB1AE5198B7}\\cmd.exe] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
C:\\Windows\\System32\\cmd.exe] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {1AC14E77-02E7-4E5D-B744-2EB1AE5198B7}\\cmd.exe] > Taskbar.TaskListLabeledButtonPanel#IconPanel >
Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
C:\\Windows\\System32\\cmd.exe] > Taskbar.TaskListLabeledButtonPanel#IconPanel >
Image#Icon", {
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/cmd.png",
    L"Height=40",
    L"Width=40",
    L"Margin=0,0,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
{1AC14E77-02E7-4E5D-B744-2EB1AE5198B7}\\cmd.exe] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
C:\\Windows\\System32\\cmd.exe] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {1AC14E77-02E7-4E5D-B744-2EB1AE5198B7}\\cmd.exe] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
C:\\Windows\\System32\\cmd.exe] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border",
{
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
{1AC14E77-02E7-4E5D-B744-2EB1AE5198B7}\\WindowsPowerShell\\v1.0\\powershell.exe] >
Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {A7CC3F66-580E-49E6-913B-556111443D24}\\Windows PowerShell\\Windows PowerShell.lnk] > Grid#IconPanel >
Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {1AC14E77-02E7-4E5D-B744-2EB1AE5198B7}\\WindowsPowerShell\\v1.0\\powershell.exe] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {A7CC3F66-580E-49E6-913B-556111443D24}\\Windows PowerShell\\Windows PowerShell.lnk] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/powershell.png",
    L"Height=40",
    L"Width=40",
    L"Margin=0,0,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
{1AC14E77-02E7-4E5D-B744-2EB1AE5198B7}\\WindowsPowerShell\\v1.0\\powershell.exe] >
Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
{A7CC3F66-580E-49E6-913B-556111443D24}\\Windows PowerShell\\Windows
PowerShell.lnk] > Grid#IconPanel > Border,
```

```
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {1AC14E77-02E7-4E5D-B744-2EB1AE5198B7}\\WindowsPowerShell\\v1.0\\powershell.exe] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {A7CC3F66-580E-49E6-913B-556111443D24}\\Windows PowerShell\\Windows PowerShell.lnk] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {1AC14E77-02E7-4E5D-B744-2EB1AE5198B7}\\osk.exe] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {1AC14E77-02E7-4E5D-B744-2EB1AE5198B7}\\osk.exe] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/osk.png",
    L"Height=40",
    L"Width=40",
    L"Margin=0,0,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.Paint_8wekyb3d8bbwe!App] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.Paint_8wekyb3d8bbwe!App] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/paint.png",
    L"Height=40",
    L"Width=40",
    L"Margin=0,0,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.Paint_8wekyb3d8bbwe!App] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.Paint_8wekyb3d8bbwe!App] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.ScreenSketch_8wekyb3d8bbwe!App] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.ScreenSketch_8wekyb3d8bbwe!App] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/snippingtool.png",
    L"Height=40",
    L"Width=40",
    L"Margin=0,0,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.ScreenSketch_8wekyb3d8bbwe!App] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.ScreenSketch_8wekyb3d8bbwe!App] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: SpotifyAB.SpotifyMusic_zpdnekdrzrea0!Spotify] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: SpotifyAB.SpotifyMusic_zpdnekdrzrea0!Spotify] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/Spotify.png",
    L"Height=40",
    L"Width=40",
    L"Margin=0,0,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: SpotifyAB.SpotifyMusic_zpdnekdrzrea0!Spotify] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: SpotifyAB.SpotifyMusic_zpdnekdrzrea0!Spotify] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.Windows.Photos_8wekyb3d8bbwe!App] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.Windows.Photos_8wekyb3d8bbwe!App] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/photos.png",
    L"Height=40",
    L"Width=40",
```

```
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Windows.Photos_8wekyb3d8bbwe!App] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Windows.Photos_8wekyb3d8bbwe!App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.ZuneMusic_8wekyb3d8bbwe!Microsoft.ZuneMusic] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.ZuneMusic_8wekyb3d8bbwe!Microsoft.ZuneMusic] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/mediaplayer.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.ZuneMusic_8wekyb3d8bbwe!Microsoft.ZuneMusic] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.ZuneMusic_8wekyb3d8bbwe!Microsoft.ZuneMusic] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Valve.Steam.Client] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Valve.Steam.Client] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/steam.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Valve.Steam.Client] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Valve.Steam.Client] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsCalculator_8wekyb3d8bbwe!App] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsCalculator_8wekyb3d8bbwe!App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/calculator.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsCalculator_8wekyb3d8bbwe!App] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsCalculator_8wekyb3d8bbwe!App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsAlarms_8wekyb3d8bbwe!App] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsAlarms_8wekyb3d8bbwe!App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/time.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsAlarms_8wekyb3d8bbwe!App] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsAlarms_8wekyb3d8bbwe!App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.SecHealthUI_8wekyb3d8bbwe!SecHealthUI] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.SecHealthUI_8wekyb3d8bbwe!SecHealthUI] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
```

```
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/security.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.SecHealthUI_8wekyb3d8bbwe!SecHealthUI] > Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.SecHealthUI_8wekyb3d8bbwe!SecHealthUI] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.WindowsStore_8wekyb3d8bbwe!App] > Grid#IconPanel > Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.WindowsStore_8wekyb3d8bbwe!App] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/store.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.WindowsStore_8wekyb3d8bbwe!App] > Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.WindowsStore_8wekyb3d8bbwe!App] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:5319275A.WhatsAppDesktop_cv1g1gvanyjgm!App] > Grid#IconPanel > Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:5319275A.WhatsAppDesktop_cv1g1gvanyjgm!App] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/whatsapp.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:5319275A.WhatsAppDesktop_cv1g1gvanyjgm!App] > Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:5319275A.WhatsAppDesktop_cv1g1gvanyjgm!App] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.Office.WINWORD.EXE.15] > Grid#IconPanel > Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.Office.WINWORD.EXE.15] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/word.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.Office.WINWORD.EXE.15] > Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.Office.WINWORD.EXE.15] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.Office.POWERPNT.EXE.15] > Grid#IconPanel > Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.Office.POWERPNT.EXE.15] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/powerpoint.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:Microsoft.Office.POWERPNT.EXE.15] > Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
```

```
Microsoft.Office.POWERPNT.EXE.15] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border",
{
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Office.ONENOTE.EXE.15] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Office.ONENOTE.EXE.15] > Taskbar.TaskListLabeledButtonPanel#IconPanel >
Image#Icon", {
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/onenote.png",
    L"Height=40",
    L"Width=40",
    L"Margin=0,0,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Office.ONENOTE.EXE.15] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Office.ONENOTE.EXE.15] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.OneDrive] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.OneDrive] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/OneDrive.png",
    L"Height=40",
    L"Width=40",
    L"Margin=0,0,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.OneDrive] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: Microsoft.OneDrive] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
MSTeams_8wekyb3d8bbwe!MSTeams] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
MSTeams_8wekyb3d8bbwe!MSTeams] > Taskbar.TaskListLabeledButtonPanel#IconPanel >
Image#Icon", {
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/ms-teams.png",
    L"Height=40",
    L"Width=40",
    L"Margin=0,0,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
MSTeams_8wekyb3d8bbwe!MSTeams] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
MSTeams_8wekyb3d8bbwe!MSTeams] > Taskbar.TaskListLabeledButtonPanel#IconPanel >
Border", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.OutlookForWindows_8wekyb3d8bbwe!Microsoft.OutlookForWindows] >
Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.OutlookForWindows_8wekyb3d8bbwe!Microsoft.OutlookForWindows] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/outlook.png",
    L"Height=40",
    L"Width=40",
    L"Margin=0,0,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.OutlookForWindows_8wekyb3d8bbwe!Microsoft.OutlookForWindows] >
Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.OutlookForWindows_8wekyb3d8bbwe!Microsoft.OutlookForWindows] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Windows.ControlPanel] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Windows.ControlPanel] > Taskbar.TaskListLabeledButtonPanel#IconPanel >
Image#Icon", {
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/ControlPanel.png",
    L"Height=40",
```

```
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Windows.ControlPanel] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Windows.ControlPanel] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Windows.Explorer] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Windows.Explorer] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/explorer.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Windows.Explorer] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.Windows.Explorer] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
com.squirrel.Discord.Discord] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
com.squirrel.Discord.Discord] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/discord.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
com.squirrel.Discord.Discord] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
com.squirrel.Discord.Discord] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
windows.immersivecontrolpanel_cw5n1h2txyewy!microsoft.windows.immersivecontrolpanel] >
Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
windows.immersivecontrolpanel_cw5n1h2txyewy!microsoft.windows.immersivecontrolpanel]
> Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/systemsettings.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
windows.immersivecontrolpanel_cw5n1h2txyewy!microsoft.windows.immersivecontrolpanel] >
Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
windows.immersivecontrolpanel_cw5n1h2txyewy!microsoft.windows.immersivecontrolpanel] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsNotepad_8wekyb3d8bbwe!App] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsNotepad_8wekyb3d8bbwe!App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
        L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-
guide/refs/heads/main/Themes/One%20UI%208.5/Assets/notepad.png",
        L"Height=40",
        L"Width=40",
        L"Margin=0,0,0,0"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsNotepad_8wekyb3d8bbwe!App] > Grid#IconPanel > Border,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
Microsoft.WindowsNotepad_8wekyb3d8bbwe!App] >
Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid:
com.github.th-ch.youtube-music] > Grid#IconPanel > Image#Icon,
Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: com.github.th-ch.youtube-
music] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
```

```
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/YouTubeMusic.png",
    L"Height=40",
    L"Width=40",
    L"Margin=0,0,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: com.github.th-ch.youtube-music] > Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: com.github.th-ch.youtube-music] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: BlueStacks_nxt] > Grid#IconPanel > Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: BlueStacks_nxt] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/Bluestacks.png",
    L"Height=40",
    L"Width=40",
    L"Margin=0,0,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: BlueStacks_nxt] > Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: BlueStacks_nxt] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {7C5A40EF-A0FB-4BFC-874A-C0F2E0B9FA8E}\\NCH Software\\DeskFX\\deskfx.exe] > Grid#IconPanel > Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {7C5A40EF-A0FB-4BFC-874A-C0F2E0B9FA8E}\\NCH Software\\DeskFX\\deskfx.exe] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/DeskFX.png",
    L"Height=40",
    L"Width=40",
    L"Margin=0,0,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {7C5A40EF-A0FB-4BFC-874A-C0F2E0B9FA8E}\\NCH Software\\DeskFX\\deskfx.exe] > Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: {7C5A40EF-A0FB-4BFC-874A-C0F2E0B9FA8E}\\NCH Software\\DeskFX\\deskfx.exe] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: RamenSoftware.Windhawk] > Grid#IconPanel > Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: RamenSoftware.Windhawk] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/windhawk.png",
    L"Height=40",
    L"Width=40",
    L"Margin=0,0,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: RamenSoftware.Windhawk] > Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: RamenSoftware.Windhawk] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
    L"Visibility=Collapsed"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: 5319275A.51895FA4EA97F_cv1g1gvanyjgm!App] > Grid#IconPanel > Image#Icon, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: 5319275A.51895FA4EA97F_cv1g1gvanyjgm!App] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Image#Icon", {
    L"Source=https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/whatsapp.png",
    L"Height=40",
    L"Width=40",
    L"Margin=0,0,0,0"}},
  ThemeTargetStyles{L"Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: 5319275A.51895FA4EA97F_cv1g1gvanyjgm!App] > Grid#IconPanel > Border, Taskbar.TaskListButton[AutomationProperties.AutomationId=Appid: 5319275A.51895FA4EA97F_cv1g1gvanyjgm!App] > Taskbar.TaskListLabeledButtonPanel#IconPanel > Border", {
    L"Visibility=Collapsed"}},
```

```
    ThemeTargetStyles{L"Taskbar.TaskListButtonPanel", {
        L"Width=45",
        L"Height=60"}},
    ThemeTargetStyles{L"Grid#IconPanel, Taskbar.TaskListLabeledButtonPanel", {
        L"Width=45",
        L"Height=60"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl > Grid > SearchUx.SearchUI.SearchIconButton > SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel", {
        L"Width=45",
        L"Height=60"}},
    ThemeTargetStyles{L"Grid#IconPanel@CommonStates > Border#BackgroundElement, Taskbar.TaskListLabeledButtonPanel@CommonStates > Border#BackgroundElement", {
        L"CornerRadius=0",
        L"Margin=0,5.5,0,5.5",
        L"Background:=$IconBackground",
        L"BorderBrush:=$IconBorder",
        L"BorderThickness=1.3"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl > Grid > SearchUx.SearchUI.SearchIconButton > SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Grid > Border#BackgroundElement, SearchUx.SearchUI.SearchButtonControl > Grid > SearchUx.SearchUI.SearchIconButton > SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Border#BackgroundElement", {
        L"Margin=0,5.5,0,5.5",
        L"CornerRadius=0",
        L"Background:=<ImageBrush Stretch=\"Uniform\"> <ImageBrush.ImageSource> <BitmapImage UriSource=\"https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/Search.png\" DecodePixelType=\"Logical\" DecodePixelWidth=\"64\" DecodePixelHeight=\"64\" /> </ImageBrush.ImageSource> </ImageBrush>",
        L"BorderBrush:=$IconBorder"}},
    ThemeTargetStyles{L"SearchUx.SearchUI.SearchButtonControl > Grid > SearchUx.SearchUI.SearchIconButton > SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Grid > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon, SearchUx.SearchUI.SearchButtonControl > Grid > SearchUx.SearchUI.SearchIconButton > SearchUx.SearchUI.SearchButtonRootGrid#SearchBoxButtonRootPanel > Microsoft.UI.Xaml.Controls.AnimatedVisualPlayer#Icon", {
        L"Visibility=1"}},
    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[3] > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid > Border#BackgroundElement, Taskbar.ExperienceToggleButton#LaunchListButton[3] > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Border#BackgroundElement", {
        L"Margin=0,5.5,0,5.5",
        L"Background:=<ImageBrush Stretch=\"Uniform\"><ImageBrush.ImageSource><BitmapImage UriSource=\"https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/darkbg.png\" DecodePixelType=\"Logical\" DecodePixelWidth=\"64\" DecodePixelHeight=\"64\"/></ImageBrush.ImageSource></ImageBrush>",
        L"BorderThickness=0",
        L"CornerRadius=0"}},
    ThemeTargetStyles{L"Taskbar.ExperienceToggleButton#LaunchListButton[1] > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Grid > Border#BackgroundElement, Taskbar.ExperienceToggleButton#LaunchListButton[1] > Taskbar.TaskListButtonPanel#ExperienceToggleButtonRootPanel > Border#BackgroundElement", {
        L"Margin=0,5.5,0,5.5",
        L"Background:=<ImageBrush Stretch=\"Uniform\"><ImageBrush.ImageSource><BitmapImage UriSource=\"https://raw.githubusercontent.com/ramensoftware/windows-11-taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/Start.png\" DecodePixelType=\"Logical\" DecodePixelWidth=\"64\" DecodePixelHeight=\"64\"/></ImageBrush.ImageSource></ImageBrush>",
        L"BorderThickness=0"}},
    ThemeTargetStyles{L"Border#MultiWindowElement", {
        L"Visibility=Collapsed"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame", {
```

```
        L"Transitions:=<TransitionCollection><RepositionThemeTransition
IsStaggeringEnabled=\"False\"/></TransitionCollection>",
        L"Height=70",
        L"MaxHeight=70",
        L"HorizontalAlignment=Stretch"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid", {
        L"Margin=0,8,0,2",
        L"BorderBrush=#40FFFFFF"}},
    ThemeTargetStyles{L"Grid#RootGrid > Taskbar.TaskbarBackground", {
        L"Margin=5,0,5,0"}},
    ThemeTargetStyles{L"Grid#RootGrid > Taskbar.TaskbarBackground > Grid", {
        L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#761E1E1E\"/>",
        L"CornerRadius=10",
        L"BorderThickness=1",
        L"BorderBrush=#40FFFFFF",
        L"Padding=-1"}},
    ThemeTargetStyles{L"Taskbar.TaskbarFrame > Grid#RootGrid > Taskbar.TaskbarBackground >
Grid > Rectangle#BackgroundFill", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"Rectangle#BackgroundStroke", {
        L"Fill=Transparent"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame", {
        L"Height=70",
        L"Margin=0,-2,0.5,2"}},
    ThemeTargetStyles{L"StackPanel#SystemTrayFrameGrid, Grid#SystemTrayFrameGrid", {
        L"VerticalAlignment=bottom",
        L"Height=60",
        L"Padding=0",
        L"BorderBrush=#40FFFFFF"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border >
SystemTray.NotificationAreaOverflow > Grid#OverflowRootGrid > ItemsControl >
ItemsPresenter > WrapGrid > ContentPresenter > SystemTray.NotifyIconView >
Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid >
SystemTray.ImageIconContent > Grid#ContainerGrid", {
        L"Background:=<ImageBrush Stretch=\"Uniform\"> <ImageBrush.ImageSource>
<BitmapImage UriSource=\"https://raw.githubusercontent.com/ramensoftware/windows-11-
taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/darkbg.png\"
DecodePixelType=\"Logical\" DecodePixelWidth=\"64\" DecodePixelHeight=\"64\" />
</ImageBrush.ImageSource> </ImageBrush>",
        L"Width=35",
        L"Height=35"}},
    ThemeTargetStyles{L"ScrollViewer > ScrollContentPresenter > Border >
SystemTray.NotificationAreaOverflow > Grid#OverflowRootGrid >
Border#OverflowFlyoutBackgroundBorder", {
        L"CornerRadius=24",
        L"Background:=<WindhawkBlur BlurAmount=\"8\" />"}},
    ThemeTargetStyles{L"TextBlock#TimeInnerTextBlock", {
        L"Foreground=white",
        L"Width=Auto",
        L"FontWeight=Bold",
        L"FontSize=15",
        L"Margin=0,-4,5,4"}},
    ThemeTargetStyles{L"TextBlock#DateInnerTextBlock", {
        L"FontWeight=Bold",
        L"Margin=-2,9,2,-9",
        L"Foreground=White",
        L"Visibility=visible",
        L"RenderTransform:=<TranslateTransform X=\"0\" Y=\"-9\"/>",
        L"FontSize=13",
        L"FontFamily=vivo Sans EN VF"}},
```

```
    ThemeTargetStyles{L"SystemTray.ChevronIconView > Grid#ContainerGrid > ContentPresenter#ContentPresenter > Grid#ContentGrid > SystemTray.TextIconContent", {
        L"Foreground=white",
        L"FontWeight=bold",
        L"FontSize=25"}},
    ThemeTargetStyles{L"SystemTray.AdaptiveTextBlock#LanguageInnerTextBlock > TextBlock#InnerTextBlock", {
        L"Foreground=white",
        L"FontSize=13",
        L"FontWeight=Bold"}},
    ThemeTargetStyles{L"SystemTray.SystemTrayFrame > StackPanel#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock, SystemTray.SystemTrayFrame > Grid#SystemTrayFrameGrid > SystemTray.OmniButton#ControlCenterButton > Grid > ContentPresenter#ContentPresenter > ItemsPresenter > StackPanel > ContentPresenter > SystemTray.IconView#SystemTrayIcon > Grid#ContainerGrid > Grid#ContentGrid > SystemTray.TextIconContent > Grid#ContainerGrid > SystemTray.AdaptiveTextBlock#Base > TextBlock#InnerTextBlock", {
        L"Foreground=white",
        L"FontSize=25"}},
    ThemeTargetStyles{L"Grid#IconPanel@RunningIndicatorStates > Rectangle#RunningIndicator, Taskbar.TaskListLabeledButtonPanel@RunningIndicatorStates > Rectangle#RunningIndicator", {
        L"Fill=white"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid > Rectangle#BackgroundStroke", {
        L"Fill:=<WindhawkBlur BlurAmount=\"3.5\" TintColor=\"#2D101010\"/>",
        L"Stroke:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
        L"StrokeThickness=5",
        L"RadiusX=14",
        L"RadiusY=14",
        L"Fill:=<<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>"}},
    ThemeTargetStyles{L"Taskbar.TaskbarBackground#HoverFlyoutBackgroundControl > Grid > Rectangle#BackgroundFill", {
        L"Canvas.ZIndex=0",
        L"Fill:=",
        L"Stroke:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
        L"StrokeThickness=5",
        L"RadiusX=14",
        L"RadiusY=14"}},
    ThemeTargetStyles{L"Taskbar.FlyoutFrame > Windows.UI.Xaml.Controls.Canvas#HoverFlyoutCanvas > Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid > Windows.UI.Xaml.Controls.ContentPresenter#HoverFlyoutContent > Taskbar.TaskItemThumbnailList > Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskItemThumbnailListRepeater > Taskbar.TaskItemThumbnailView > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.Border#BackgroundBorder", {
        L"VerticalAlignment=Bottom",
        L"Canvas.ZIndex=1",
        L"Background:=<WindhawkBlur BlurAmount=\"5\" TintColor=\"#761E1E1E\"/>",
        L"Height=25",
        L"CornerRadius=0,0,15,15",
        L"Margin=5,0,5,0"}},
    ThemeTargetStyles{L"Border#HoverFlyoutBackground", {
        L"Margin=4,36,4,0",
        L"Canvas.ZIndex=1",
        L"Width=Auto",
        L"Background:=Transparent",
        L"BorderThickness=0",
        L"CornerRadius=15"}},
    ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#ThumbBarRepeater > Taskbar.ThumbBarButton#ThumbBarButton > Windows.UI.Xaml.Controls.ContentPresenter#BorderElement", {
```

```
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#761E1E1E\"/>",
      L"Margin=0,-20,0,20"}},
   ThemeTargetStyles{L"Microsoft.UI.Xaml.Controls.ItemsRepeater#IconsRepeater > Windows.UI.Xaml.Controls.Image", {
      L"Visibility=Collapsed"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Button#CloseButton", {
      L"HorizontalAlignment=left",
      L"Grid.ColumnSpan=1",
      L"Grid.RowSpan=1",
      L"Canvas.ZIndex=1",
      L"CornerRadius=20",
      L"Width=28",
      L"Height=28",
      L"Margin=-18,40,15,-40",
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#80ffffff\"/>",
      L"Foreground=black",
      L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#761E1E1E\"/>"}},
   ThemeTargetStyles{L"Taskbar.FlyoutFrame > Windows.UI.Xaml.Controls.Canvas#HoverFlyoutCanvas > Windows.UI.Xaml.Controls.Grid#HoverFlyoutGrid > Windows.UI.Xaml.Controls.ContentPresenter#HoverFlyoutContent > Taskbar.TaskItemThumbnailList > Microsoft.UI.Xaml.Controls.ItemsRepeater#TaskItemThumbnailListRepeater > Taskbar.TaskItemThumbnailView > Windows.UI.Xaml.Controls.Grid > Windows.UI.Xaml.Controls.TextBlock#DisplayNameTextBlock", {
      L"Grid.ColumnSpan=2",
      L"Grid.RowSpan=2",
      L"VerticalAlignment=bottom",
      L"HorizontalAlignment=Center",
      L"Margin=0,-5,0,5",
      L"Canvas.ZIndex=1"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#ConfirmatorMainGrid", {
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>",
      L"CornerRadius=24",
      L"BorderBrush:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#30ffffff\"/>",
      L"BorderThickness=2",
      L"Margin=0,0,0,10"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid.Border#ConfirmatorMainGrid", {
      L"Background:=<WindhawkBlur BlurAmount=\"8\" TintColor=\"#2D101010\"/>"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalTrackRect", {
      L"Fill=#20ffffff",
      L"RadiusX=12",
      L"RadiusY=12",
      L"Height=18",
      L"Margin=0"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Shapes.Rectangle#HorizontalDecreaseRect", {
      L"Fill:=<SolidColorBrush Color=\"{ThemeResource SystemAccentColor}\" />",
      L"RadiusX=12",
      L"RadiusY=12",
      L"Height=18"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#VolumeConfirmator", {
      L"Padding=8,0,8,0"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.Grid#BrightnessConfirmator", {
      L"Padding=8,0,8,0"}},
   ThemeTargetStyles{L"Windows.UI.Xaml.Controls.TextBlock#volumeLevelText", {
      L"Foreground=White"}},
}, {
   L"IconBackground=<ImageBrush Stretch=\"Uniform\"> <ImageBrush.ImageSource> <BitmapImage UriSource=\"https://raw.githubusercontent.com/ramensoftware/windows-11-
```

```
taskbar-styling-guide/refs/heads/main/Themes/One%20UI%208.5/Assets/template.png\"
DecodePixelType=\"Logical\" DecodePixelWidth=\"64\" DecodePixelHeight=\"64\" />
</ImageBrush.ImageSource> </ImageBrush>",
    L"IconBorder=transparent",
}};

// clang-format on

std::atomic<bool> g_initialized;
thread_local bool g_initializedForThread;

HANDLE g_restartExplorerPromptThread;
std::atomic<HWND> g_restartExplorerPromptWindow;

constexpr WCHAR kRestartExplorerPromptTitle[] =
    L"Windows 11 Taskbar Styler - Windhawk";
constexpr WCHAR kRestartExplorerPromptTextFormat[] =
    L"Restarting Explorer is required for the mod to activate.\n\nDo you want "
    L"to restart Explorer now?\n\nStatus code: 0x%08X";
constexpr WCHAR kRestartExplorerCommand[] =
    LR"(cmd /c "echo Terminating Explorer...")"
    LR"( & taskkill /f /im explorer.exe)"
    LR"( & timeout /t 1 /nobreak >nul)"
    LR"( & start explorer.exe)"
    LR"( & echo Starting Explorer...)"
    LR"( & timeout /t 3 /nobreak >nul")";

void PromptToRestartExplorer(HRESULT statusCode) {
    if (g_restartExplorerPromptThread) {
        if (WaitForSingleObject(g_restartExplorerPromptThread, 0) !=
            WAIT_OBJECT_0) {
            return;
        }

        CloseHandle(g_restartExplorerPromptThread);
    }

    g_restartExplorerPromptThread = CreateThread(
        nullptr, 0,
        [](LPVOID lpParameter) -> DWORD {
            HRESULT statusCode =
                static_cast<HRESULT>(reinterpret_cast<ULONG_PTR>(lpParameter));

            WCHAR promptText[256];
            _snwprintf_s(promptText, _TRUNCATE,
                         kRestartExplorerPromptTextFormat, statusCode);

            TASKDIALOGCONFIG taskDialogConfig{
                .cbSize = sizeof(taskDialogConfig),
                .dwFlags = TDF_ALLOW_DIALOG_CANCELLATION,
                .dwCommonButtons = TDCBF_YES_BUTTON | TDCBF_NO_BUTTON,
                .pszWindowTitle = kRestartExplorerPromptTitle,
                .pszMainIcon = TD_INFORMATION_ICON,
                .pszContent = promptText,
                .pfCallback = [](HWND hwnd, UINT msg, WPARAM wParam,
                                 LPARAM lParam, LONG_PTR lpRefData) -> HRESULT {
```

```
            switch (msg) {
              case TDN_CREATED:
                g_restartExplorerPromptWindow = hwnd;
                SetWindowPos(hwnd, HWND_TOPMOST, 0, 0, 0, 0,
                             SWP_NOMOVE | SWP_NOSIZE);
                break;

              case TDN_DESTROYED:
                g_restartExplorerPromptWindow = nullptr;
                break;
            }

            return S_OK;
          },
        };

        int button;
        if (SUCCEEDED(TaskDialogIndirect(&taskDialogConfig, &button,
                                         nullptr, nullptr)) &&
          button == IDYES) {
          WCHAR commandLine[ARRAYSIZE(kRestartExplorerCommand)];
          memcpy(commandLine, kRestartExplorerCommand,
               sizeof(kRestartExplorerCommand));
          STARTUPINFO si = {
            .cb = sizeof(si),
          };
          PROCESS_INFORMATION pi{};
          if (CreateProcess(nullptr, commandLine, nullptr, nullptr, FALSE,
                            0, nullptr, nullptr, &si, &pi)) {
            CloseHandle(pi.hThread);
            CloseHandle(pi.hProcess);
          }
        }

        return 0;
      },
      reinterpret_cast<LPVOID>(static_cast<ULONG_PTR>(statusCode)), 0,
      nullptr);
}

// An InstanceHandle is the address of an interface on the element, so it names
// an element only for as long as that element lives: an element allocated over
// a destroyed one is reported under the same handle. Everything the mod records
// is therefore keyed by an id minted per reported element, which is never
// reused, rather than by the handle itself.
enum class ElementId : uint64_t { None = 0 };

ElementId GetOrCreateElementId(
   InstanceHandle handle,
   winrt::Windows::Foundation::IInspectable const& element);
ElementId FindElementId(InstanceHandle handle);
void ForgetElementId(InstanceHandle handle);

void ApplyCustomizations(ElementId elementId,
                  winrt::Windows::UI::Xaml::FrameworkElement element,
```

```
                    PCWSTR fallbackClassName);
void CleanupCustomizations(ElementId elementId);
void QueueDiagnosticsRelease(InstanceHandle handle);
void FlushDiagnosticsReleasesIfQuiet();

void HandleClickThroughIslandRoot(
   winrt::Windows::Foundation::IInspectable const& inspectable);

HMODULE GetCurrentModuleHandle() {
   HMODULE module;
   if (!GetModuleHandleEx(GET_MODULE_HANDLE_EX_FLAG_FROM_ADDRESS |
                          GET_MODULE_HANDLE_EX_FLAG_UNCHANGED_REFCOUNT,
                       L"", &module)) {
      return nullptr;
   }

   return module;
}

// The XAML composition diagnostics rebuild a process-wide visual tree walker
// without any locking whenever a DirectComposition visual is added, so any UI
// thread which adds one corrupts the heap while another thread is in the same
// code. Only element mutations are needed here, and those are reported by an
// unrelated code path, so the composition diagnostics are kept from being
// created at all: XamlDiagnostics::CreateCompVisualDiag skips them when the
// HKLM\Software\Microsoft\XAML\Debug\DisableCompositionDiag value is 1.
// Windows.UI.Xaml.dll reads and caches the value once, from within
// AdviseVisualTreeChange, so answering that single read is enough.
thread_local bool g_reportCompositionDiagAsDisabled;

//////////////////////////////////////////////////////////////////////////////
// clang-format off

#pragma region winrt_hpp

#include <Unknwn.h>
#include <weakreference.h>
#include <winrt/base.h>

// forward declare namespaces we alias
namespace winrt {
   namespace Windows {
      namespace Foundation {}
      namespace UI::Xaml {}
   }
}

// alias some long namespaces for convenience
namespace wf = winrt::Windows::Foundation;
namespace wux = winrt::Windows::UI::Xaml;

// A weak reference for the object, or an empty one when the object is null or
// doesn't support weak references: cppwinrt's make_weak dereferences a null
// pointer for an object without that support instead of reporting it. Throws,
// as make_weak does, when the object supports weak references but one can't be
```

```
// made.
winrt::weak_ref<wf::IInspectable> TryMakeWeak(wf::IInspectable const& object)
{
   if (!object.try_as<::IWeakReferenceSource>())
   {
      return nullptr;
   }

   return winrt::make_weak(object);
}

#pragma endregion  // winrt_hpp

#pragma region visualtreewatcher_hpp

#include <winrt/Windows.UI.Xaml.h>

// XamlDiagnostics implements this interface too, and xamlom.h does not declare
// it. UnregisterInstance closes the runtime object cached for a handle, the
// only reference the diagnostics keep to an element once it was reported.
static constexpr GUID IID_IXamlDiagnosticsTestHooks =
   {0x735941a2, 0x3ee3, 0x495a, {0x8d, 0xa9, 0x97, 0x26, 0x27, 0x00, 0x30, 0x75}};

struct IXamlDiagnosticsTestHooks : IUnknown
{
   virtual HRESULT STDMETHODCALLTYPE UnregisterInstance(InstanceHandle handle) = 0;
   virtual HRESULT STDMETHODCALLTYPE TryGetDispatcherQueueForObject(InstanceHandle
handle, void** dispatcherQueue) = 0;
};

// The handle a mutation callback would report for an element, for elements
// which were reached some other way, e.g. by walking the visual tree. Derived
// the way the diagnostics derive it, by querying IInspectable and taking the
// pointer, and not through GetHandleFromIInspectable: that one creates the
// runtime object when none is cached, so asking it about an element whose
// reference was released would take a new reference and pin it again.
InstanceHandle HandleFromInspectable(wf::IInspectable const& instance)
{
   winrt::com_ptr<::IInspectable> inspectable;

winrt::check_hresult(reinterpret_cast<::IUnknown*>(winrt::get_abi(instance))->QueryInterface(
winrt::guid_of<wf::IInspectable>(), inspectable.put_void()));
   return reinterpret_cast<InstanceHandle>(inspectable.get());
}

class VisualTreeWatcher : public winrt::implements<VisualTreeWatcher,
IVisualTreeServiceCallback2, winrt::non_agile>
{
public:
   VisualTreeWatcher(winrt::com_ptr<IUnknown> site);

   VisualTreeWatcher(const VisualTreeWatcher&) = delete;
   VisualTreeWatcher& operator=(const VisualTreeWatcher&) = delete;

   VisualTreeWatcher(VisualTreeWatcher&&) = delete;
   VisualTreeWatcher& operator=(VisualTreeWatcher&&) = delete;
```

```
    ~VisualTreeWatcher();

    void UnadviseVisualTreeChange();

    bool ReleaseDiagnosticsReference(InstanceHandle handle);

private:
    HRESULT STDMETHODCALLTYPE OnVisualTreeChange(ParentChildRelation relation,
VisualElement element, VisualMutationType mutationType) override;
    HRESULT STDMETHODCALLTYPE OnElementStateChanged(InstanceHandle element,
VisualElementState elementState, LPCWSTR context) noexcept override;

    wf::IInspectable FromHandle(InstanceHandle handle)
    {
        wf::IInspectable obj;
        winrt::check_hresult(m_XamlDiagnostics->GetIInspectableFromHandle(handle,
reinterpret_cast<::IInspectable**>(winrt::put_abi(obj))));
        return obj;
    }

    winrt::com_ptr<IXamlDiagnostics> m_XamlDiagnostics = nullptr;
    winrt::com_ptr<IXamlDiagnosticsTestHooks> m_XamlDiagnosticsTestHooks = nullptr;
};

#pragma endregion  // visualtreewatcher_hpp

#pragma region visualtreewatcher_cpp

VisualTreeWatcher::VisualTreeWatcher(winrt::com_ptr<IUnknown> site) :
    m_XamlDiagnostics(site.as<IXamlDiagnostics>())
{
    Wh_Log(L"Constructing VisualTreeWatcher");

    HRESULT hr = m_XamlDiagnostics->QueryInterface(IID_IXamlDiagnosticsTestHooks,
m_XamlDiagnosticsTestHooks.put_void());
    if (FAILED(hr)) {
        Wh_Log(L"IXamlDiagnosticsTestHooks is unavailable, elements will be leaked: %08X", hr);
    }

    //
winrt::check_hresult(m_XamlDiagnostics.as<IVisualTreeService3>()->AdviseVisualTreeChange(t
his));

    // Calling AdviseVisualTreeChange from the current thread causes the app to
    // hang in Advising::RunOnUIThread sometimes. Creating a new thread and
    // calling it from there fixes it.
    HANDLE thread = CreateThread(
        nullptr, 0,
        [](LPVOID lpParam) -> DWORD {
            auto watcher = reinterpret_cast<VisualTreeWatcher*>(lpParam);
            auto service = watcher->m_XamlDiagnostics.as<IVisualTreeService3>();
            g_reportCompositionDiagAsDisabled = true;
            HRESULT hr = service->AdviseVisualTreeChange(watcher);
            g_reportCompositionDiagAsDisabled = false;
            watcher->Release();
            if (FAILED(hr)) {
```

```
            Wh_Log(L"AdviseVisualTreeChange failed with error %08X", hr);
            PromptToRestartExplorer(hr);
        }
        return 0;
    },
    this, 0, nullptr);
  if (thread) {
    AddRef();
    CloseHandle(thread);
  }
}

VisualTreeWatcher::~VisualTreeWatcher()
{
  Wh_Log(L"Destructing VisualTreeWatcher");
}

void VisualTreeWatcher::UnadviseVisualTreeChange()
{
  Wh_Log(L"UnadviseVisualTreeChange VisualTreeWatcher");
  HRESULT hr =
m_XamlDiagnostics.as<IVisualTreeService3>()->UnadviseVisualTreeChange(this);
  if (FAILED(hr)) {
    Wh_Log(L"UnadviseVisualTreeChange failed with error %08X", hr);
  }
}

// Reports whether dropping the reference destroyed the element, which is what
// tells the caller that the handle is free to name a different element from now
// on and that the id recorded for this one has to go.
bool VisualTreeWatcher::ReleaseDiagnosticsReference(InstanceHandle handle)
{
  if (!m_XamlDiagnosticsTestHooks) {
    return false;
  }

  winrt::weak_ref<wf::IInspectable> weakElement;
  {
    // Not through FromHandle: a handle whose runtime object is already gone
    // fails to resolve routinely, and throwing for it would pay for an
    // originate with a stack capture every time. The strong reference has
    // to be gone again before the release below, hence the scope.
    wf::IInspectable element;
    HRESULT hr = m_XamlDiagnostics->GetIInspectableFromHandle(handle,
reinterpret_cast<::IInspectable**>(winrt::put_abi(element)));
    if (SUCCEEDED(hr) && element) {
      try {
        weakElement = TryMakeWeak(element);
      } catch (...) {
        Wh_Log(L"Error %08X", winrt::to_hresult());
      }
    }
  }

  HRESULT hr = m_XamlDiagnosticsTestHooks->UnregisterInstance(handle);
  if (FAILED(hr)) {
```

```
    Wh_Log(L"UnregisterInstance failed with error %08X", hr);
    return false;
  }

  // Not every reported object supports weak references, and then the release
  // just proceeds unobserved.
  return weakElement && !weakElement.get();
}

HRESULT VisualTreeWatcher::OnVisualTreeChange(ParentChildRelation relation, VisualElement element, VisualMutationType mutationType) try
{
  Wh_Log(L"=========================================");

  switch (mutationType)
  {
  case Add:
    Wh_Log(L"Mutation type: Add %llu", element.Handle);
    break;

  case Remove:
    Wh_Log(L"Mutation type: Remove %llu", element.Handle);
    break;

  default:
    Wh_Log(L"Mutation type: %d %llu", static_cast<int>(mutationType), element.Handle);
    break;
  }

  Wh_Log(L"Element type: %s", element.Type);

  if (!g_initializedForThread)
  {
    Wh_Log(L"Not initialized for thread %u", GetCurrentThreadId());
    return S_OK;
  }

  // Caught here rather than by the handler below, so that the bookkeeping
  // which hands the element's reference back still runs when the styling work
  // throws. Otherwise a single failed element would be held for good.
  try
  {
    if (mutationType == Add)
    {
      const auto inspectable = FromHandle(element.Handle);
      auto elementId = GetOrCreateElementId(element.Handle, inspectable);
      auto frameworkElement = inspectable.try_as<wux::FrameworkElement>();
      if (frameworkElement)
      {
        Wh_Log(L"FrameworkElement name: %s", frameworkElement.Name().c_str());
        if (elementId == ElementId::None)
        {
          Wh_Log(L"Skipping element which can't be given an id");
        }
        else
```

```
            {
               ApplyCustomizations(elementId, frameworkElement, element.Type);
            }
         }
         else
         {
            Wh_Log(L"Skipping non-FrameworkElement");
            HandleClickThroughIslandRoot(inspectable);
         }
      }
      else if (mutationType == Remove)
      {
         CleanupCustomizations(FindElementId(element.Handle));
      }
   }
   catch (...)
   {
      Wh_Log(L"Error %08X", winrt::to_hresult());
   }

   // A tree discarded whole is never dismantled, so it reports no removals to
   // be released by.
   FlushDiagnosticsReleasesIfQuiet();

   if (mutationType == Add)
   {
      QueueDiagnosticsRelease(element.Handle);
      QueueDiagnosticsRelease(relation.Parent);
   }
   else if (mutationType == Remove)
   {
      // Queued rather than released outright: this report arrives from inside
      // the Leave walk which is still visiting the subtree being removed.
      QueueDiagnosticsRelease(element.Handle);
      ForgetElementId(element.Handle);
   }

   return S_OK;
}
catch (...)
{
   HRESULT hr = winrt::to_hresult();
   Wh_Log(L"Error %08X", hr);

   // Returning an error prevents (some?) further messages, always return
   // success.
   // return hr;
   return S_OK;
}

HRESULT VisualTreeWatcher::OnElementStateChanged(InstanceHandle, VisualElementState,
LPCWSTR) noexcept
{
   return S_OK;
}
```

```
#pragma endregion  // visualtreewatcher_cpp

#pragma region tap_hpp

#include <ocidl.h>

winrt::com_ptr<VisualTreeWatcher> g_visualTreeWatcher;

// {C85D8CC7-5463-40E8-A432-F5916B6427E5}
static constexpr CLSID CLSID_WindhawkTAP = { 0xc85d8cc7, 0x5463, 0x40e8, { 0xa4, 0x32, 0xf5,
0x91, 0x6b, 0x64, 0x27, 0xe5 } };

class WindhawkTAP : public winrt::implements<WindhawkTAP, IObjectWithSite,
winrt::non_agile>
{
public:
    HRESULT STDMETHODCALLTYPE SetSite(IUnknown *pUnkSite) override;
    HRESULT STDMETHODCALLTYPE GetSite(REFIID riid, void **ppvSite) noexcept override;

private:
    winrt::com_ptr<IUnknown> site;
};

#pragma endregion  // tap_hpp

#pragma region tap_cpp

HRESULT WindhawkTAP::SetSite(IUnknown *pUnkSite) try
{
    // Only ever 1 VTW at once.
    if (g_visualTreeWatcher)
    {
        g_visualTreeWatcher->UnadviseVisualTreeChange();
        g_visualTreeWatcher = nullptr;
    }

    site.copy_from(pUnkSite);

    if (site)
    {
        // Decrease refcount increased by InitializeXamlDiagnosticsEx.
        FreeLibrary(GetCurrentModuleHandle());

        g_visualTreeWatcher = winrt::make_self<VisualTreeWatcher>(site);
    }

    return S_OK;
}
catch (...)
{
    HRESULT hr = winrt::to_hresult();
    Wh_Log(L"Error %08X", hr);
    return hr;
}
```

```
HRESULT WindhawkTAP::GetSite(REFIID riid, void **ppvSite) noexcept
{
    return site.as(riid, ppvSite);
}

#pragma endregion  // tap_cpp

#pragma region simplefactory_hpp

#include <Unknwn.h>

template<class T>
struct SimpleFactory : winrt::implements<SimpleFactory<T>, IClassFactory, winrt::non_agile>
{
    HRESULT STDMETHODCALLTYPE CreateInstance(IUnknown* pUnkOuter, REFIID riid, void**
ppvObject) override try
    {
        if (!pUnkOuter)
        {
            *ppvObject = nullptr;
            return winrt::make<T>().as(riid, ppvObject);
        }
        else
        {
            return CLASS_E_NOAGGREGATION;
        }
    }
    catch (...)
    {
        HRESULT hr = winrt::to_hresult();
        Wh_Log(L"Error %08X", hr);
        return hr;
    }

    HRESULT STDMETHODCALLTYPE LockServer(BOOL) noexcept override
    {
        return S_OK;
    }
};

#pragma endregion  // simplefactory_hpp

#pragma region module_cpp

#include <combaseapi.h>

#pragma clang diagnostic push
#pragma clang diagnostic ignored "-Wdll-attribute-on-redeclaration"

__declspec(dllexport)
HRESULT __Use_decl_annotations_ STDAPI DllGetClassObject(REFCLSID rclsid, REFIID riid, LPVOID* ppv)
try
{
    if (rclsid == CLSID_WindhawkTAP)
    {
        *ppv = nullptr;
```

```
        return winrt::make<SimpleFactory<WindhawkTAP>>().as(riid, ppv);
    }
    else
    {
        return CLASS_E_CLASSNOTAVAILABLE;
    }
}
catch (...)
{
    HRESULT hr = winrt::to_hresult();
    Wh_Log(L"Error %08X", hr);
    return hr;
}

__declspec(dllexport)
_Use_decl_annotations_ STDAPI DllCanUnloadNow()
{
    if (winrt::get_module_lock())
    {
        return S_FALSE;
    }
    else
    {
        return S_OK;
    }
}

#pragma clang diagnostic pop

#pragma endregion  // module_cpp

#pragma region api_cpp

bool g_inInjectWindhawkTAP = false;

using PFN_INITIALIZE_XAML_DIAGNOSTICS_EX = decltype(&InitializeXamlDiagnosticsEx);

HRESULT InjectWindhawkTAP() noexcept
{
    HMODULE module = GetCurrentModuleHandle();
    if (!module)
    {
        return HRESULT_FROM_WIN32(GetLastError());
    }

    WCHAR location[MAX_PATH];
    switch (GetModuleFileName(module, location, ARRAYSIZE(location)))
    {
    case 0:
    case ARRAYSIZE(location):
        return HRESULT_FROM_WIN32(GetLastError());
    }

    const HMODULE wux(LoadLibraryEx(L"Windows.UI.Xaml.dll", nullptr,
LOAD_LIBRARY_SEARCH_SYSTEM32));
```

```
    if (!wux) [[unlikely]]
    {
        return HRESULT_FROM_WIN32(GetLastError());
    }

    const auto ixde =
reinterpret_cast<PFN_INITIALIZE_XAML_DIAGNOSTICS_EX>(GetProcAddress(wux,
"InitializeXamlDiagnosticsEx"));
    if (!ixde) [[unlikely]]
    {
        return HRESULT_FROM_WIN32(GetLastError());
    }

    // I didn't find a better way than trying many connections until one works.
    // Reference:
    // https://github.com/microsoft/microsoft-ui-
xaml/blob/d74a0332cf0d5e58f12eddce1070fa7a79b4c2db/src/dxaml/xcp/dxaml/lib/DXamlCor
e.cpp#L2782
    g_inInjectWindhawkTAP = true;

    HRESULT hr;
    for (int i = 0; i < 10000; i++)
    {
        WCHAR connectionName[256];
        wsprintf(connectionName, L"VisualDiagConnection%d", i + 1);

        hr = ixde(connectionName, GetCurrentProcessId(), L"", location, CLSID_WindhawkTAP,
nullptr);
        if (hr != HRESULT_FROM_WIN32(ERROR_NOT_FOUND))
        {
            break;
        }
    }

    g_inInjectWindhawkTAP = false;

    return hr;
}

#pragma endregion  // api_cpp

// clang-format on
////////////////////////////////////////////////////////////////////////////////

#include <windhawk_utils.h>

#include <algorithm>
#include <charconv>
#include <chrono>
#include <cmath>
#include <condition_variable>
#include <filesystem>
#include <limits>
#include <list>
#include <memory>
#include <mutex>
#include <optional>
```

```
#include <random>
#include <sstream>
#include <string>
#include <string_view>
#include <type_traits>
#include <unordered_map>
#include <unordered_set>
#include <variant>
#include <vector>

using namespace std::string_view_literals;

#include <initguid.h>

#include <commctrl.h>
#include <d2d1_1.h>
#include <roapi.h>
#include <shlwapi.h>
#include <windows.graphics.effects.h>
#include <windows.ui.xaml.hosting.desktopwindowxamlsource.h>
#include <winstring.h>

#include <winrt/Windows.Foundation.Collections.h>
#include <winrt/Windows.Foundation.h>
#include <winrt/Windows.Graphics.Effects.h>
#include <winrt/Windows.Networking.Connectivity.h>
#include <winrt/Windows.Storage.Streams.h>
#include <winrt/Windows.System.Power.h>
#include <winrt/Windows.System.h>
#include <winrt/Windows.UI.Composition.h>
#include <winrt/Windows.UI.Core.h>
#include <winrt/Windows.UI.Text.h>
#include <winrt/Windows.UI.ViewManagement.h>
#include <winrt/Windows.UI.Xaml.Controls.h>
#include <winrt/Windows.UI.Xaml.Hosting.h>
#include <winrt/Windows.UI.Xaml.Markup.h>
#include <winrt/Windows.UI.Xaml.Media.Imaging.h>
#include <winrt/Windows.UI.Xaml.Media.h>
#include <winrt/Windows.UI.Xaml.Shapes.h>
#include <winrt/Windows.UI.Xaml.h>

#define WH_WINRT_WINUI2
#include <winrt/Microsoft.UI.Xaml.Controls.h>

using namespace winrt::Windows::UI::Xaml;

namespace muxc = winrt::Microsoft::UI::Xaml::Controls;
namespace wge = winrt::Windows::Graphics::Effects;
namespace wuc = winrt::Windows::UI::Composition;
namespace wuxh = wux::Hosting;
namespace awge = ABI::Windows::Graphics::Effects;

enum class XamlDiagnosticsHandling {
    kAlert,
    kBlock,
```

```
    kAllow,
};

struct {
    bool clickThroughTaskbar;
    XamlDiagnosticsHandling xamlDiagnosticsHandling;
} g_settings;

// https://stackoverflow.com/a/51274008
template <auto fn>
struct deleter_from_fn {
    template <typename T>
    constexpr void operator()(T* arg) const {
        fn(arg);
    }
};
using string_setting_unique_ptr =
    std::unique_ptr<const WCHAR[], deleter_from_fn<Wh_FreeStringSetting>>;

using PropertyKeyValue =
    std::pair<DependencyProperty, winrt::Windows::Foundation::IInspectable>;

using PropertyValuesUnresolved =
    std::vector<std::pair<std::wstring, std::wstring>>;
using PropertyValues = std::vector<PropertyKeyValue>;
using PropertyValuesMaybeUnresolved =
    std::variant<PropertyValuesUnresolved, PropertyValues>;

struct ElementMatcher {
    enum class Kind {
        Element,   // Normal element matcher.
        Wildcard,  // '*': matches zero or more intermediate ancestors.
        Root,      // ':root': asserts the next element has no parent.
    };
    Kind kind = Kind::Element;
    std::wstring type;
    std::wstring name;
    std::optional<std::wstring> visualStateGroupName;
    int oneBasedIndex = 0;
    PropertyValuesMaybeUnresolved propertyValues;
};

// A `Property[@VisualState][:]=value` rule that sets a control property.
// `value` may contain `{{...}}` placeholders, in which case `isDynamic()`
// returns true and the rule is re-resolved on every apply.
struct ValueRule {
    std::wstring propertyName;
    std::wstring visualState;
    std::wstring value;
    bool isXamlValue = false;

    bool isDynamic() const { return value.find(L"{{") != std::wstring::npos; }
};

// A `Property=>VarName` rule that observes a control property and writes its
```

```
// current value into the named mod-global style variable.
struct CaptureRule {
   std::wstring propertyName;
   std::wstring varName;
};

// Parsed-but-not-yet-resolved rules for one target. Captures and value-rules
// are intentionally split: they live in different fields of `ResolvedRules`
// post-resolution, and the parser already validates that captures cannot carry
// `:=` or `@VisualState`.
struct UnresolvedRules {
   std::vector<ValueRule> valueRules;
   std::vector<CaptureRule> captureRules;
};

struct XamlBlurBrushParams {
   float blurAmount;
   winrt::Windows::UI::Color tint;
   std::optional<uint8_t> tintOpacity;
   std::wstring tintThemeResourceKey;  // Empty if not from ThemeResource
   std::optional<float> tintLuminosityOpacity;
   std::optional<float> tintSaturation;
   std::optional<float> noiseOpacity;
   std::optional<float> noiseDensity;
   std::optional<winrt::Windows::UI::Color> fallbackColor;
   std::wstring fallbackThemeResourceKey;  // Empty if not from ThemeResource
};

// Holds the raw rule body for a style whose value depends on `{{...}}`
// substitutions. Re-resolved on every apply and on every variable change.
// `propertyName` is kept alongside the value because Windows.UI.Xaml's
// DependencyProperty does not expose its name, and the re-resolution path needs
// to feed the name back to the XAML parser.
struct DynamicStyleTemplate {
   std::wstring propertyName;
   std::wstring rawValue;
   bool isXamlValue = false;
};

// Tagged value for one (property, visualState) cell of PropertyOverrides.
// Possible states:
// - IInspectable       : fully resolved WinRT value (literal or static XAML).
//                        Apply directly via SetValue.
// - XamlBlurBrushParams : parsed `<WindhawkBlur .../>` parameters. The brush
//                        instance is constructed at apply time (needs the live
//                        UIElement).
// - DynamicStyleTemplate: rule body contains `{{...}}` substitutions.
//                        Re-resolved on every apply and on every variable
//                        change. This arm appears only inside
//                        PropertyOverrides cells; it is never stored in
//                        ElementPropertyCustomizationState::customValue (see
//                        notes there).
using PropertyOverrideValue =
   std::variant<winrt::Windows::Foundation::IInspectable,
                XamlBlurBrushParams,
```

```
            DynamicStyleTemplate>;

// Property -> visual state -> value.
using PropertyOverrides =
   std::unordered_map<DependencyProperty,
                      std::unordered_map<std::wstring, PropertyOverrideValue>>;

// Resolved counterpart to CaptureRule: the property name string has been turned
// into an actual DependencyProperty by the XAML parser, so the apply path can
// call RegisterPropertyChangedCallback / GetValue directly without re-resolving
// on every use.
struct CaptureSpec {
   DependencyProperty property{nullptr};
   std::wstring varName;
};

struct ResolvedRules {
   PropertyOverrides propertyOverrides;
   std::vector<CaptureSpec> captures;
   // Whether this target consumes style variables. Lets ApplyCustomizations
   // skip the visual-tree bookkeeping that only variable users need.
   bool hasDynamicValues = false;
};

using PropertyOverridesMaybeUnresolved =
   std::variant<UnresolvedRules, ResolvedRules>;

// A `{{Var}}` reference resolved for one consuming property. The owner lets a
// value change on some other capture of the same name be skipped.
struct StyleVariableDependency {
   std::wstring name;
   ElementId owner = ElementId::None;  // None when the variable was undefined
};

// Interned node of an element's visual-tree spine. Nodes are shared by every
// tracked element under the same ancestor, so the pool holds one node per
// distinct ancestor rather than a full path per element. Once a node exists its
// `parent` and `depth` are final; an element that is later reparented keeps the
// spine it was first seen with, and only the nodes of a spine interned before
// its root object was attached (see GetOrCreateElementTreeNode) are ever
// replaced.
struct ElementTreeNode {
   // A node can outlive the object it describes -- descendant nodes and
   // not-yet-cleaned-up ElementCustomizationState entries keep it alive -- so
   // this is what proves a pool hit isn't a recycled address.
   winrt::weak_ref<DependencyObject> ref;
   std::shared_ptr<ElementTreeNode> parent;
   uint32_t depth = 0;
   // The depth-0 node this spine hangs from, `this` for a root itself. The
   // parent chain keeps it alive, so a raw pointer is enough.
   ElementTreeNode* root = nullptr;
};

// Keyed by the object's IUnknown pointer: COM only guarantees a stable pointer
// for that interface, and the same element is reached both as a
```

```
// FrameworkElement and as a VisualTreeHelper::GetParent result.
thread_local std::unordered_map<void*, std::weak_ptr<ElementTreeNode>>
   g_elementTreeNodes;

// Expired pool entries are reaped once the map grows past this, which is then
// set to twice the surviving size, making the sweep amortized O(1).
thread_local size_t g_elementTreeNodesReapThreshold = 64;

void* ElementIdentityKey(DependencyObject const& object) {
   return winrt::get_abi(object.as<winrt::Windows::Foundation::IUnknown>());
}

// A depth-0 node is a placeholder root until proven otherwise: if its object
// has since gained a parent, the spine was interned before that object was
// attached and stops short of the real root. Asked of any node on the spine,
// not just of the root itself, so that a descendant interned through a
// placeholder root is repaired too.
bool IsStaleSpine(ElementTreeNode const& node) {
   auto object = node.root->ref.get();
   return object && Media::VisualTreeHelper::GetParent(object);
}

// Fetch (or build) the spine node for `object`. Uses
// VisualTreeHelper::GetParent rather than Parent(), same reason as in
// FindElementPropertyOverrides. Returns nullptr if a node can't be built,
// leaving callers with no proximity information rather than a wrong answer.
std::shared_ptr<ElementTreeNode> GetOrCreateElementTreeNode(
   DependencyObject object) {
   if (!object) {
      return nullptr;
   }

   std::shared_ptr<ElementTreeNode> node;

   // Ancestors still lacking a node, innermost first. The walk stops at the
   // first ancestor that is already interned, so a new sibling of an
   // already-seen element costs one GetParent call.
   std::vector<DependencyObject> missing;

   try {
      for (auto iter = object; iter;
           iter = Media::VisualTreeHelper::GetParent(iter)) {
         auto key = ElementIdentityKey(iter);

         if (auto it = g_elementTreeNodes.find(key);
            it != g_elementTreeNodes.end()) {
            auto existing = it->second.lock();
            // A weak_ref never resolves to an object other than its own, so
            // a live ref proves this address hasn't been recycled since.
            if (!existing || !existing->ref.get()) {
               Wh_Log(L"Replacing stale tree node for a reused address");
               g_elementTreeNodes.erase(it);
            } else if (!IsStaleSpine(*existing)) {
               node = std::move(existing);
               break;
```

```
            } else {
                // Drop the node and keep walking: the ancestors above it
                // are stale for the same reason, up to the placeholder
                // root, above which the real spine gets built. A stale
                // shared_ptr already cached elsewhere (see
                // EnsureElementTreeNode) is refreshed the same way on its
                // own next use, so no element is stuck unrankable.
                Wh_Log(L"Rebuilding tree node interned before attachment");
                g_elementTreeNodes.erase(it);
            }
        }

        missing.push_back(iter);
    }

    for (auto it = missing.rbegin(); it != missing.rend(); ++it) {
        auto fresh = std::make_shared<ElementTreeNode>();
        fresh->ref = *it;
        fresh->depth = node ? node->depth + 1 : 0;
        fresh->root = node ? node->root : fresh.get();
        fresh->parent = std::move(node);
        g_elementTreeNodes[ElementIdentityKey(*it)] = fresh;
        node = std::move(fresh);
    }
  } catch (winrt::hresult_error const& ex) {
    Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
    return nullptr;
  }

  return node;
}

void ReapElementTreeNodesIfNeeded() {
  if (g_elementTreeNodes.size() < g_elementTreeNodesReapThreshold) {
    return;
  }

  std::erase_if(g_elementTreeNodes,
                [](const auto& item) { return item.second.expired(); });
  g_elementTreeNodesReapThreshold =
    std::max<size_t>(64, g_elementTreeNodes.size() * 2);
}

// Depth of the lowest common ancestor of two spine nodes, or -1 when they have
// none (separate visual trees, or a node that couldn't be built). A node counts
// as its own ancestor, so an element on the other's parent chain scores its own
// depth -- the deepest score that element can reach.
int ElementTreeLcaDepth(ElementTreeNode const* a, ElementTreeNode const* b) {
  if (!a || !b) {
    return -1;
  }

  while (a->depth > b->depth) {
    a = a->parent.get();
  }
```

```
   while (b->depth > a->depth) {
      b = b->parent.get();
   }

   while (a != b) {
      a = a->parent.get();
      b = b->parent.get();
      if (!a || !b) {
         return -1;
      }
   }

   return static_cast<int>(a->depth);
}

struct ElementCustomizationRules {
   ElementMatcher elementMatcher;
   std::vector<ElementMatcher> parentElementMatchers;
   PropertyOverridesMaybeUnresolved propertyOverrides;
};

thread_local std::vector<ElementCustomizationRules>
   g_elementsCustomizationRules;

struct ElementPropertyCustomizationState {
   std::optional<winrt::Windows::Foundation::IInspectable> originalValue;
   // The most recently applied value, re-pushed by the per-DP property-
   // changed callback when something external (animation, system Setter)
   // overrides it. Although PropertyOverrideValue's variant declares a
   // DynamicStyleTemplate arm, customValue here is always either IInspectable
   // or XamlBlurBrushParams in practice -- dynamic styles get resolved into
   // one of those before being stored, and the source template lives
   // separately in `dynamicTemplate` below.
   std::optional<PropertyOverrideValue> customValue;
   // The value SetOrClearValue wrote for customValue, which is what a write
   // by something else is told apart from.
   winrt::Windows::Foundation::IInspectable lastAppliedValue{nullptr};
   int64_t propertyChangedToken = 0;
   // Source template for dynamic styles whose value contains `{{...}}`
   // substitutions; re-evaluated whenever a referenced variable changes, with
   // the resolved result written back into `customValue`. Empty for static
   // styles.
   std::optional<DynamicStyleTemplate> dynamicTemplate;
   // Style variables this property's value depends on, each with the capture
   // that supplied it. Populated alongside `dynamicTemplate`; empty for static
   // styles.
   std::vector<StyleVariableDependency> variableDependencies;
   // Makes this property re-resolve on any change to any of its variables:
   // expansion aborts at the first failure, so the names past that point have
   // no recorded owner and a targeted propagation would never reach them.
   bool lastResolveFailed = false;
};

struct CapturePropertyCustomizationState {
   std::wstring varName;
```

```
    int64_t propertyChangedToken = 0;
};

struct ElementCustomizationStateForVisualStateGroup {
    std::unordered_map<DependencyProperty, ElementPropertyCustomizationState>
        propertyCustomizationStates;
    winrt::event_token visualStateGroupCurrentStateChangedToken;
};

struct ElementCustomizationState {
    winrt::weak_ref<FrameworkElement> element;

    // Cached weak ref to the element's XamlRoot at register time. Used during
    // cleanup paths to find this element's per-XamlRoot StyleVariableState
    // even after the element above has been GC'd. A weak_ref (not a raw
    // pointer) so that an expired XamlRoot does not silently collide with a
    // freshly-allocated one at the same address.
    winrt::weak_ref<XamlRoot> xamlRoot;

    // Scores how close each capture of a style variable is to this element.
    // Only built for elements that capture or consume a variable.
    std::shared_ptr<ElementTreeNode> treeNode;

    // Capture state lives at the element level: capture rules (`Prop=>Var`) are
    // intentionally not visual-state-aware (the parser rejects `@VisualState`
    // on them), and a single element observed by multiple targets with
    // different VSGs should still only register one
    // RegisterPropertyChangedCallback per DP and one SizeChanged subscription.
    std::unordered_map<DependencyProperty, CapturePropertyCustomizationState>
        captureCustomizationStates;

    // ActualWidth/ActualHeight (and other layout-driven DPs) do not fire
    // RegisterPropertyChangedCallback on UWP, so any element with capture rules
    // also subscribes to `FrameworkElement.SizeChanged` to pick up size
    // changes.
    winrt::event_token captureSizeChangedToken;

    // Use list to avoid reallocations on insertion, as pointers to items are
    // captured in callbacks and stored.
    std::list<std::pair<std::optional<winrt::weak_ref<VisualStateGroup>>,
                        ElementCustomizationStateForVisualStateGroup>>
        perVisualStateGroup;
};

thread_local std::unordered_map<ElementId, ElementCustomizationState>
    g_elementsCustomizationState;

// The weak reference is what keeps an id honest. A handle is an address, so a
// destroyed element can be replaced by one reporting the same handle, and an
// entry whose element is gone, or is no longer the element being asked about,
// belongs to that destroyed predecessor and must not name the new one.
struct ElementIdEntry {
    ElementId id = ElementId::None;
    winrt::weak_ref<wf::IInspectable> element;
};
```

```
thread_local std::unordered_map<InstanceHandle, ElementIdEntry> g_elementIds;
thread_local uint64_t g_lastElementId;

ElementId GetOrCreateElementId(InstanceHandle handle,
                               wf::IInspectable const& element) {
   if (!handle || !element) {
      return ElementId::None;
   }

   auto& entry = g_elementIds[handle];
   if (entry.id != ElementId::None && entry.element.get() == element) {
      return entry.id;
   }

   entry.id = static_cast<ElementId>(++g_lastElementId);

   winrt::weak_ref<wf::IInspectable> weakElement;
   try {
      weakElement = TryMakeWeak(element);
   } catch (winrt::hresult_error const& ex) {
      Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
   }

   if (!weakElement) {
      // Without a weak reference the entry cannot be told apart from one for
      // a successor at the same address, so neither it nor the id it names is
      // kept: an id no lookup can reach again would key state that nothing
      // could ever tear down, on an element nothing would then hold back from
      // being released.
      g_elementIds.erase(handle);
      return ElementId::None;
   }

   entry.element = std::move(weakElement);
   return entry.id;
}

// By handle alone, for the element which is being reported as removed: it is
// the element the entry was made for, and a stale entry names something already
// destroyed, whose state is due for teardown either way.
ElementId FindElementId(InstanceHandle handle) {
   auto it = g_elementIds.find(handle);
   return it != g_elementIds.end() ? it->second.id : ElementId::None;
}

void ForgetElementId(InstanceHandle handle) {
   g_elementIds.erase(handle);
}

// Dead entries are reaped once the map grows past this, which is then set to
// twice the surviving size, making the sweep amortized O(1).
thread_local size_t g_elementIdsReapThreshold = 64;

// An element whose diagnostics reference was handed back is destroyed without a
```

```
// removal being reported for it, so what the mod keys by that element has to be
// found rather than told. An entry whose weak reference no longer resolves
// names such an element, and is torn down the way its removal would have.
void ReapDeadElementIdsIfNeeded() {
   if (g_elementIds.size() < g_elementIdsReapThreshold) {
      return;
   }

   // Collected before anything is torn down: CleanupCustomizations runs XAML
   // work which can re-enter ApplyCustomizations and rehash the map.
   std::vector<std::pair<InstanceHandle, ElementId>> dead;
   for (const auto& [handle, entry] : g_elementIds) {
      if (!entry.element.get()) {
         dead.push_back({handle, entry.id});
      }
   }

   if (!dead.empty()) {
      Wh_Log(L"Reaping %zu of %zu element ids", dead.size(),
          g_elementIds.size());
   }

   for (const auto& [handle, elementId] : dead) {
      CleanupCustomizations(elementId);
      g_elementIds.erase(handle);
   }

   g_elementIdsReapThreshold = std::max<size_t>(64, g_elementIds.size() * 2);
}

// The element's spine node. An element can be matched before its subtree is
// attached, in which case the eager build in ApplyCustomizations interns a
// spine that stops at a placeholder root; re-checked on every use so it's
// rebuilt once the subtree is actually in the tree.
ElementTreeNode* EnsureElementTreeNode(
   ElementCustomizationState& elementCustomizationState) {
   if (!elementCustomizationState.treeNode ||
      IsStaleSpine(*elementCustomizationState.treeNode)) {
      if (auto element = elementCustomizationState.element.get()) {
         elementCustomizationState.treeNode =
            GetOrCreateElementTreeNode(element);
      }
   }

   return elementCustomizationState.treeNode.get();
}

// Mod-global style variable registry. Populated by `Property=>VarName` capture
// rules and consumed by `{{VarName}}` substitutions in other styles. Every
// capturing element gets its own entry, so a name stays defined until its last
// capture goes away, and a consumer reading the name resolves to whichever
// capture is closest to it in the visual tree.
struct StyleVariableValue {
   std::wstring stringForm;       // invariant-formatted text representation
   std::optional<double> numeric;  // only present when source was numeric
```

```
   // True for primitive captures whose `stringForm` is meaningful to insert
   // verbatim into a XAML attribute (numeric, boolean, string). False for
   // opaque types -- their stringForm is the captured class name, kept only
   // for diagnostics; bare-identifier substitution skips such variables.
   bool substitutable = false;
};

// One element's capture of a variable. FindElementPropertyOverrides dedupes
// captures by name, so (name, elementId) identifies an entry.
struct StyleVariableCapture {
   ElementId elementId;
   StyleVariableValue value;
};

struct StyleVariableConsumer {
   ElementId elementId;
   DependencyProperty property{nullptr};
   // Each consumer remembers its own fallbackClassName so that propagation can
   // re-resolve dynamic styles using the consumer's match-site context, not
   // the (potentially different) capturer's.
   std::wstring fallbackClassName;
};

// Per-XamlRoot scope for the style variable registry. Multiple taskbars on one
// UI thread each have their own XamlRoot; keying by XamlRoot prevents
// `Property=>Var` captures on one taskbar from being substituted into
// `{{Var}}` on another. Identity is tracked via weak_ref so that a destroyed
// XamlRoot's slot cannot be confused with a new XamlRoot allocated at the
// same address. std::list is used because pointers to existing entries must
// stay valid as new entries are added or stale ones reaped: lambdas
// registered on per-element captures hold a StyleVariableState* for the
// lifetime of the entry.
struct StyleVariableState {
   winrt::weak_ref<XamlRoot> xamlRoot;
   std::unordered_map<std::wstring, std::vector<StyleVariableCapture>>
      variables;
   std::unordered_map<std::wstring, std::vector<StyleVariableConsumer>>
      consumers;
   // How many entries the two maps above hold for each element. They're keyed
   // by variable name, so without this, asking whether an element appears in
   // either of them means walking every name.
   std::unordered_map<ElementId, size_t> elementRefs;
};

void AddStyleVariableElementRef(StyleVariableState* state,
                                ElementId elementId) {
   state->elementRefs[elementId]++;
}

void ReleaseStyleVariableElementRefs(StyleVariableState* state,
                                     ElementId elementId,
                                     size_t count) {
   if (!count) {
      return;
   }
```

```
    auto it = state->elementRefs.find(elementId);
    if (it == state->elementRefs.end()) {
        return;
    }

    if (it->second > count) {
        it->second -= count;
    } else {
        state->elementRefs.erase(it);
    }
}

thread_local std::list<StyleVariableState> g_styleVariableState;

// Non-zero while a frame is holding a StyleVariableState*. Suppresses reaping,
// which would otherwise free the entry out from under that frame: applying or
// restoring a style runs arbitrary XAML work that can re-enter and reap.
thread_local int g_styleVariableStatePinDepth;

struct StyleVariableStatePin {
    StyleVariableStatePin() { g_styleVariableStatePinDepth++; }
    ~StyleVariableStatePin() { g_styleVariableStatePinDepth--; }
    StyleVariableStatePin(const StyleVariableStatePin&) = delete;
    StyleVariableStatePin& operator=(const StyleVariableStatePin&) = delete;
};

// Non-zero while PropagateStyleVariableChange is running, so nested calls queue
// instead of recursing. Separate from the pin above: a propagation started from
// a pinned but non-propagating frame must run, not queue with nobody to drain
// it.
thread_local int g_styleVariablePropagationDepth;

struct PendingStyleVariablePropagation {
    StyleVariableState* state;
    std::wstring varName;
    std::optional<ElementId> changedOwner;

    bool operator==(const PendingStyleVariablePropagation&) const = default;
};

// Propagations queued while another one is running, drained by the outermost
// PropagateStyleVariableChange frame. Not kept per-StyleVariableState: a nested
// propagation can target a different XamlRoot than the outer frame's, and that
// queue still has to be drained.
thread_local std::vector<PendingStyleVariablePropagation>
    g_pendingStyleVariablePropagations;

// Look up (or create) the entry for a live XamlRoot. Reaps any entries whose
// XamlRoot has been destroyed before searching, so a recycled address cannot
// collide with a stale entry.
StyleVariableState* GetStyleVariableState(XamlRoot const& xamlRoot) {
    if (!xamlRoot) {
        return nullptr;
    }
```

```
   if (g_styleVariableStatePinDepth == 0) {
      g_styleVariableState.remove_if([](StyleVariableState const& entry) {
         return !entry.xamlRoot.get();
      });
   }
   for (auto& entry : g_styleVariableState) {
      if (entry.xamlRoot.get() == xamlRoot) {
         return &entry;
      }
   }
   auto& fresh = g_styleVariableState.emplace_back();
   fresh.xamlRoot = xamlRoot;
   return &fresh;
}

// Look up an existing entry from a cached weak_ref. Returns nullptr if the
// XamlRoot is already gone (cleanup is then a no-op since the entry has been
// or will be reaped).
StyleVariableState* GetStyleVariableState(
   winrt::weak_ref<XamlRoot> const& xamlRootWeak) {
   auto strong = xamlRootWeak.get();
   if (!strong) {
      return nullptr;
   }
   return GetStyleVariableState(strong);
}

// Convenience for entry points that have a FrameworkElement. Returns nullptr
// if the element is detached (no XamlRoot yet).
StyleVariableState* GetStyleVariableState(FrameworkElement const& element) {
   if (!element) {
      return nullptr;
   }
   XamlRoot xamlRoot{nullptr};
   try {
      xamlRoot = element.XamlRoot();
   } catch (...) {
      // Defensive: detached elements may throw on XamlRoot().
   }
   return GetStyleVariableState(xamlRoot);
}

thread_local bool g_elementPropertyModifying;

thread_local std::list<
   std::pair<winrt::weak_ref<DependencyObject>,
         winrt::Windows::Foundation::IAsyncOperation<bool>>>
   g_delayedBackgroundFillSet;

// An image with a remote source fails to load when the process starts before
// the network is up. Such images are tracked so that the load can be retried
// once there's internet access, and are cached in a file in the mod storage
// folder, which is what's loaded when it's there, so that the image shows up at
// once and offline. Only a target which has no image is retried, and only a
// source which isn't showing anything is replaced, so an image that's currently
```

```
// displayed can't be blanked out.
struct TrackedImage {
   // An ImageBrush or an Image element. Both hold an image source which can
   // fail to load and both report the outcome, but through unrelated types, so
   // the source is addressed by DependencyProperty and each type gets its own
   // revoker pair.
   winrt::weak_ref<DependencyObject> target;
   DependencyProperty sourceProperty{nullptr};
   // The remote address: the entry's identity and what's downloaded, even
   // while the cached file is what's loaded.
   winrt::Windows::Foundation::Uri uri{nullptr};
   std::wstring url;
   // The cached copy of the image, empty when there's no cache folder.
   std::filesystem::path cachePath;

   // Decode properties of the BitmapImage the style declared, reapplied to the
   // BitmapImage a retry creates.
   int32_t decodePixelWidth = 0;
   int32_t decodePixelHeight = 0;
   Media::Imaging::DecodePixelType decodePixelType =
      Media::Imaging::DecodePixelType::Physical;
   Media::Imaging::BitmapCreateOptions createOptions =
      Media::Imaging::BitmapCreateOptions::None;
   bool autoPlay = true;

   Media::ImageBrush::ImageFailed_revoker brushImageFailedRevoker;
   Media::ImageBrush::ImageOpened_revoker brushImageOpenedRevoker;
   Controls::Image::ImageFailed_revoker elementImageFailedRevoker;
   Controls::Image::ImageOpened_revoker elementImageOpenedRevoker;

   // Whether the target has an image. Retries target the ones which don't.
   bool loaded = false;

   // Whether the target is loading from the cached file rather than from the
   // remote address, which is what a load failure is judged by.
   bool usingCache = false;

   ULONGLONG lastRetryTick = 0;
   int retryCount = 0;
};

struct TrackedImagesForThread {
   // Entries are held by shared_ptr so that event handlers can reference them
   // via a weak_ptr and do nothing once an entry is gone.
   std::list<std::shared_ptr<TrackedImage>> images;
   winrt::Windows::System::DispatcherQueue dispatcher{nullptr};
   winrt::Windows::System::DispatcherQueueTimer retryTimer{nullptr};
   winrt::Windows::System::DispatcherQueueTimer::Tick_revoker
      retryTimerTickRevoker;
   // Tick the scheduled retry round is due at, zero if none is scheduled.
   ULONGLONG retryDueTick = 0;
};

thread_local TrackedImagesForThread g_trackedImagesForThread;
```

```
// The remote address of each cached file which has been substituted for one, so
// that a target given an already substituted source is tracked as well.
// Outlives the entries, since the style value it describes is shared by targets
// which come and go. Thread local like that value.
thread_local std::unordered_map<std::wstring, winrt::Windows::Foundation::Uri>
    g_imageCacheUriRemotes;

// A single connectivity transition raises several network status events, and
// the state right after the first one isn't final yet.
constexpr DWORD kNetworkChangeDebounceMs = 2000;

// Minimum delay between the retries of an image, doubling with each attempt up
// to about five minutes. Also keeps a retry from being started while the
// previous one is still loading.
constexpr ULONGLONG kImageRetryBaseDelayMs = 5000;
constexpr int kImageRetryMaxBackoffShift = 6;
constexpr ULONGLONG kImageRetryMaxDelayMs = kImageRetryBaseDelayMs
                                << kImageRetryMaxBackoffShift;

// Caps the attempts of an image, bounding the series of retries which a failure
// starts. The count starts over once the image has been idle for the maximum
// delay, so connectivity which returns much later can still recover it.
constexpr int kImageRetryMaxCount = 20;

// Guards the globals below it. The network status handler acquires it, so it
// must never be held while adding or removing that handler: the event source
// can wait for an invocation which is already in flight, and registering from a
// UI thread pumps messages, which can re-enter this code on the same thread.
std::mutex g_imageRetryMutex;
bool g_imageRetryActive;
// The dispatcher of each UI thread which has tracked images, used to run a
// retry on the thread that owns the image.
std::vector<winrt::weak_ref<winrt::Windows::System::DispatcherQueue>>
    g_imageRetryDispatchers;
winrt::event_token g_networkStatusChangedToken;
// Set while a thread is registering the handler outside the mutex, so that a
// concurrent or re-entrant call doesn't register a second one.
bool g_networkStatusChangedRegistering;
// Callbacks which are on their way into mod code, counted so that the module
// isn't freed out from under them.
size_t g_imageRetryPendingCallbacks;
std::condition_variable g_imageRetryPendingCallbacksCv;

// A cached file is fetched again once it's this old, and its write time is
// stamped whether or not the fetch gets through, so that the write time doubles
// as when the file was last known to be in use.
constexpr ULONGLONG kImageCacheRefreshIntervalMs = 7ULL * 24 * 60 * 60 * 1000;
// A file which nothing stamps ages until it's swept. Long enough for a theme
// which is switched away from and back to keep its images.
constexpr ULONGLONG kImageCacheMaxUnusedMs = 30ULL * 24 * 60 * 60 * 1000;

// Guards the globals below it.
std::mutex g_imageDownloadMutex;
// The URL of each image to fetch, or an empty string for a cache sweep. The
// path a URL is cached at follows from the URL, so it isn't carried along.
```

```
std::list<std::wstring> g_imageDownloadQueue;
// The URL of every queued and in flight job, so that one image isn't fetched
// twice at once. A job which failed is dropped: the retries of the image it's
// for are what ask again, and they're already paced and capped.
std::unordered_set<std::wstring> g_imageDownloadUrls;
// The URL of every cached file which failed to load, taking the images it's
// for back to the remote address for the rest of the process. Not per entry,
// since the file is what was rejected and the entries which share the URL
// would otherwise hand it out again. Global for the same reason: the file is
// process wide, not thread wide.
std::unordered_set<std::wstring> g_imageCacheRejectedUrls;
PTP_WORK g_imageDownloadWork;
// Whether a callback is draining the queue; a job added meanwhile joins it.
bool g_imageDownloadRunning;
bool g_imageDownloadStopping;

enum class ResourceVariableTheme {
   None,
   Dark,
   Light,
};

enum class ResourceVariableType {
   String,
   Xaml,
   ThemeResourceReference,
};

struct ResourceVariableEntry {
   std::wstring key;
   std::wstring value;
   ResourceVariableTheme theme;
   ResourceVariableType type;
};

thread_local std::vector<ResourceVariableEntry> g_resourceVariables;

// Track original resource values for restoration (per-thread since
// Application::Current().Resources() is per-thread).
thread_local std::unordered_map<std::wstring,
                        winrt::Windows::Foundation::IInspectable>
   g_originalResourceValues;

// Track our merged theme dictionary for cleanup (per-thread).
thread_local ResourceDictionary g_resourceVariablesThemeDict{nullptr};

// For listening to theme color changes (per-thread).
thread_local winrt::Windows::UI::ViewManagement::UISettings g_uiSettings{
   nullptr};
thread_local winrt::event_token g_colorValuesChangedToken;

// Per-XamlRoot state for the click-through taskbar option. Each taskbar
// (primary and per secondary monitor) has its own XamlRoot on the shared UI
// thread, and is clipped independently via SetWindowRgn. Identity is tracked
// via weak_ref so a destroyed XamlRoot's slot cannot be confused with a new one
```

```
// at the same address. std::list is used so the ClickThroughTaskbarState*
// returned by GetClickThroughState stays valid even if a reentrant XAML call
// adds or reaps entries while the pointer is in use.
struct ClickThroughTaskbarState {
    winrt::weak_ref<XamlRoot> xamlRoot;
    // The XAML island's native window, from IDesktopWindowXamlSourceNative. Its
    // GA_ROOT ancestor is the top-level taskbar window the region is applied
    // to.
    HWND islandHwnd = nullptr;
    winrt::weak_ref<FrameworkElement> taskbarFrame;
    winrt::weak_ref<FrameworkElement> systemTrayFrame;
    FrameworkElement::LayoutUpdated_revoker layoutUpdatedRevoker;
    // Quantized signature of the last applied region, to skip redundant work on
    // the frequent LayoutUpdated event.
    std::vector<long long> lastRegionSignature;
};

thread_local std::list<ClickThroughTaskbarState> g_clickThroughTaskbarState;

// Bounding box of the window region last applied, per top-level taskbar
// window. Tells whether the current region is still ours: Explorer can clear or
// replace it with no XAML layout change, e.g. on an auto-hide toggle, which the
// signature can't see. Keyed by window, which outlives the XamlRoot when the
// island is rebuilt.
thread_local std::unordered_map<HWND, RECT> g_clickThroughAppliedRgnBoxes;

// Look up (or create) the entry for a live XamlRoot. Reaps any entries whose
// XamlRoot has been destroyed before searching, so a recycled address cannot
// collide with a stale entry.
ClickThroughTaskbarState* GetClickThroughState(XamlRoot const& xamlRoot) {
    if (!xamlRoot) {
        return nullptr;
    }
    g_clickThroughTaskbarState.remove_if(
        [](ClickThroughTaskbarState const& entry) {
            return !entry.xamlRoot.get();
        });
    for (auto& entry : g_clickThroughTaskbarState) {
        if (entry.xamlRoot.get() == xamlRoot) {
            return &entry;
        }
    }
    auto& fresh = g_clickThroughTaskbarState.emplace_back();
    fresh.xamlRoot = xamlRoot;
    return &fresh;
}

// Tracks XAML island roots (DesktopWindowXamlSource) seen for click-through, so
// the island's native window can be matched to a taskbar's XamlRoot. Needed
// because a freshly created island (e.g. a secondary taskbar attached after the
// mod loaded) reports its root before the content/XamlRoot is ready, so the
// association has to be resolved lazily once the frames lay out.
struct ClickThroughIslandRoot {
    winrt::weak_ref<wuxh::DesktopWindowXamlSource> source;
    HWND islandHwnd = nullptr;
```

```
};

thread_local std::list<ClickThroughIslandRoot> g_clickThroughIslandRoots;

// Top-level taskbar windows subclassed for click-through, tracked so each is
// subclassed once and released on cleanup. The subclass reapplies the clip
// region as soon as the window moves or is shown (e.g. an auto-hidden taskbar
// sliding into view), since Explorer resets the region across auto-hide state
// changes and the XAML LayoutUpdated event can lag well behind that.
thread_local std::unordered_set<HWND> g_clickThroughSubclassedWindows;

thread_local bool g_applyingClickThroughRegion = false;

// Find the island native window whose root content shares the given XamlRoot.
// Reaps entries whose source has been destroyed. Returns nullptr if not found
// yet (the island's content may not be attached at the time of the call).
HWND ResolveClickThroughIslandHwnd(XamlRoot const& xamlRoot) {
   g_clickThroughIslandRoots.remove_if(
      [](ClickThroughIslandRoot const& entry) {
         return !entry.source.get();
      });
   for (auto& entry : g_clickThroughIslandRoots) {
      auto source = entry.source.get();
      if (!source) {
         continue;
      }
      auto content = source.Content();
      if (!content) {
         continue;
      }
      XamlRoot contentXamlRoot = nullptr;
      try {
         contentXamlRoot = content.XamlRoot();
      } catch (...) {
      }
      if (contentXamlRoot && contentXamlRoot == xamlRoot) {
         return entry.islandHwnd;
      }
   }
   return nullptr;
}

winrt::Windows::Foundation::IInspectable ReadLocalValueWithWorkaround(
   DependencyObject elementDo,
   DependencyProperty property) {
   auto value = elementDo.ReadLocalValue(property);
   if (value) {
      auto className = winrt::get_class_name(value);
      if (className == L"Windows.UI.Xaml.Data.BindingExpressionBase" ||
         className == L"Windows.UI.Xaml.Data.BindingExpression") {
         // BindingExpressionBase was observed to be returned for XAML
         // properties that were declared as following:
         //
         // <Border ... CornerRadius="{TemplateBinding CornerRadius}" />
         //
```

```
            // Calling SetValue with it fails with an error, so we won't be able
            // to use it to restore the value. As a workaround, we use
            // GetAnimationBaseValue to get the value.
            Wh_Log(L"ReadLocalValue returned %s, using GetAnimationBaseValue",
                className.c_str());
            value = elementDo.GetAnimationBaseValue(property);
        }
    }

    Wh_Log(L"Read property value %s",
        value ? (value == DependencyProperty::UnsetValue()
                    ? L"(unset)"
                    : winrt::get_class_name(value).c_str())
              : L"(null)");

    return value;
}

////////////////////////////////////////////////////////////////////////////
// Noise generation
//
// Generates a tileable noise BMP in memory. Density controls the brightness
// distribution curve via a power function (lower density = sparser bright
// pixels). Opacity is handled downstream by the composition effect graph.
winrt::Windows::Storage::Streams::IRandomAccessStream CreateNoiseStream(
    float density) {
    // Cache the last stream to avoid regenerating when density hasn't changed.
    // The cached stream is never read directly; callers get independent clones
    // via CloneStream() so they don't share a seek cursor.
    thread_local float cachedDensity = std::numeric_limits<float>::quiet_NaN();
    thread_local winrt::Windows::Storage::Streams::InMemoryRandomAccessStream
        cachedStream{nullptr};

    if (density == cachedDensity && cachedStream) {
        return cachedStream.CloneStream();
    }

    // Use 256x256 to minimize visible tiling seams.
    constexpr int kSize = 256;
    constexpr DWORD kBpp = 32;
    constexpr DWORD rowSize = kSize * (kBpp / 8);
    constexpr DWORD dataSize = rowSize * kSize;

    BITMAPFILEHEADER fileHeader{
        .bfType = 0x4D42,  // "BM"
        .bfSize =
            sizeof(BITMAPFILEHEADER) + sizeof(BITMAPINFOHEADER) + dataSize,
        .bfOffBits = sizeof(BITMAPFILEHEADER) + sizeof(BITMAPINFOHEADER),
    };

    BITMAPINFOHEADER infoHeader{
        .biSize = sizeof(BITMAPINFOHEADER),
        .biWidth = kSize,
        .biHeight = kSize,
        .biPlanes = 1,
```

```
    .biBitCount = kBpp,
    .biSizeImage = dataSize,
  };

  std::vector<uint8_t> pixels(dataSize);

  // Precompute the density power curve as a lookup table so that
  // std::pow is called 256 times instead of once per pixel (65536).
  float safeDensity = std::clamp(density, 0.001f, 1.0f);
  float exponent = 1.0f / safeDensity;

  uint8_t lut[256];
  for (int i = 0; i < 256; i++) {
    lut[i] = static_cast<uint8_t>(std::pow(i / 255.0f, exponent) * 255.0f);
  }

  std::mt19937 rng(0);
  std::uniform_int_distribution<int> dist(0, 255);

  for (size_t i = 0; i < pixels.size(); i += 4) {
    uint8_t gray = lut[dist(rng)];

    // Fully opaque; opacity is applied downstream by ColorMatrixEffect.
    pixels[i] = gray;
    pixels[i + 1] = gray;
    pixels[i + 2] = gray;
    pixels[i + 3] = 255;
  }

  winrt::Windows::Storage::Streams::InMemoryRandomAccessStream stream;
  winrt::Windows::Storage::Streams::DataWriter writer(stream);
  writer.WriteBytes(winrt::array_view<const uint8_t>(
    reinterpret_cast<const uint8_t*>(&fileHeader), sizeof(fileHeader)));
  writer.WriteBytes(winrt::array_view<const uint8_t>(
    reinterpret_cast<const uint8_t*>(&infoHeader), sizeof(infoHeader)));
  writer.WriteBytes(pixels);
  writer.StoreAsync().get();
  writer.DetachStream();

  cachedStream = std::move(stream);
  cachedDensity = density;

  return cachedStream.CloneStream();
}

// Blur background implementation, copied from TranslucentTB.
//////////////////////////////////////////////////////////////////////////////
// clang-format off
template <> inline constexpr winrt::guid
winrt::impl::guid_v<winrt::impl::abi_t<winrt::Windows::Foundation::IPropertyValue>>{
  winrt::impl::guid_v<winrt::Windows::Foundation::IPropertyValue>
};

typedef enum MY_D2D1_GAUSSIANBLUR_OPTIMIZATION
{
```

```
    MY_D2D1_GAUSSIANBLUR_OPTIMIZATION_SPEED = 0,
    MY_D2D1_GAUSSIANBLUR_OPTIMIZATION_BALANCED = 1,
    MY_D2D1_GAUSSIANBLUR_OPTIMIZATION_QUALITY = 2,
    MY_D2D1_GAUSSIANBLUR_OPTIMIZATION_FORCE_DWORD = 0xffffffff

} MY_D2D1_GAUSSIANBLUR_OPTIMIZATION;

//////////////////////////////////////////////////////////////////////////////
// XamlBlurBrush.h
class XamlBlurBrush : public Media::XamlCompositionBrushBaseT<XamlBlurBrush>
{
public:
    XamlBlurBrush(UIElement element,
                  float blurAmount,
                  winrt::Windows::UI::Color tint,
                  std::optional<uint8_t> tintOpacity,
                  winrt::hstring tintThemeResourceKey,
                  std::optional<float> tintLuminosityOpacity,
                  std::optional<float> tintSaturation,
                  std::optional<float> noiseOpacity,
                  std::optional<float> noiseDensity,
                  std::optional<winrt::Windows::UI::Color> fallbackColor,
                  winrt::hstring fallbackThemeResourceKey);
    ~XamlBlurBrush();

    void OnConnected();
    void OnDisconnected();

private:
    void RefreshThemeTint();
    void RefreshFallbackColor();
    bool ShouldUseFallback() const;
    void RefreshBrush();
    wuc::CompositionBrush CreateEffectBrush();
    wuc::CompositionBrush CreateFallbackBrush();

    wuc::Compositor m_compositor;
    float m_blurAmount;
    winrt::Windows::UI::Color m_tint;
    std::optional<uint8_t> m_tintOpacity;
    winrt::hstring m_tintThemeResourceKey;
    std::optional<float> m_tintLuminosityOpacity;
    std::optional<float> m_tintSaturation;
    std::optional<float> m_noiseOpacity;
    std::optional<float> m_noiseDensity;
    std::optional<winrt::Windows::UI::Color> m_fallbackColor;
    winrt::hstring m_fallbackThemeResourceKey;
    Media::SolidColorBrush m_proxyBrush{nullptr};
    Media::SolidColorBrush m_fallbackProxyBrush{nullptr};
    winrt::weak_ref<FrameworkElement> m_weakProxyElement;
    winrt::hstring m_proxyKey;
    winrt::hstring m_fallbackProxyKey;
    winrt::Windows::UI::ViewManagement::UISettings m_uiSettings{nullptr};
    winrt::event_token m_advancedEffectsEnabledChangedToken{};
    winrt::event_token m_energySaverStatusChangedToken{};
```

```
    winrt::Windows::System::DispatcherQueue m_dispatcher{nullptr};
    HKEY m_powerKey{nullptr};
    HANDLE m_regNotifyEvent{nullptr};
    HANDLE m_regWaitHandle{nullptr};

    static void CALLBACK OnEnergySaverRegistryChanged(PVOID context,
                                      BOOLEAN timerOrWaitFired);
};

//////////////////////////////////////////////////////////////////////////////
// windows.graphics.effects.interop.h
#ifndef BUILD_WINDOWS
namespace ABI {
#endif
namespace Windows {
namespace Graphics {
namespace Effects {

typedef interface IGraphicsEffectSource                 IGraphicsEffectSource;
typedef interface IGraphicsEffectD2D1Interop            IGraphicsEffectD2D1Interop;


typedef enum GRAPHICS_EFFECT_PROPERTY_MAPPING
{
    GRAPHICS_EFFECT_PROPERTY_MAPPING_UNKNOWN,
    GRAPHICS_EFFECT_PROPERTY_MAPPING_DIRECT,
    GRAPHICS_EFFECT_PROPERTY_MAPPING_VECTORX,
    GRAPHICS_EFFECT_PROPERTY_MAPPING_VECTORY,
    GRAPHICS_EFFECT_PROPERTY_MAPPING_VECTORZ,
    GRAPHICS_EFFECT_PROPERTY_MAPPING_VECTORW,
    GRAPHICS_EFFECT_PROPERTY_MAPPING_RECT_TO_VECTOR4,
    GRAPHICS_EFFECT_PROPERTY_MAPPING_RADIANS_TO_DEGREES,
    GRAPHICS_EFFECT_PROPERTY_MAPPING_COLORMATRIX_ALPHA_MODE,
    GRAPHICS_EFFECT_PROPERTY_MAPPING_COLOR_TO_VECTOR3,
    GRAPHICS_EFFECT_PROPERTY_MAPPING_COLOR_TO_VECTOR4
} GRAPHICS_EFFECT_PROPERTY_MAPPING;

//+-----------------------------------------------------------------------------
//
//  Interface:
//      IGraphicsEffectD2D1Interop
//
//  Synopsis:
//      An interface providing a Interop counterpart to IGraphicsEffect
//      and allowing for metadata queries.
//
//------------------------------------------------------------------------------

#undef INTERFACE
#define INTERFACE IGraphicsEffectD2D1Interop
DECLARE_INTERFACE_IID_(IGraphicsEffectD2D1Interop, IUnknown, "2FC57384-A068-44D7-
A331-30982FCF7177")
{
    STDMETHOD(GetEffectId)(
        _Out_ GUID * id
```

```
        ) PURE;

    STDMETHOD(GetNamedPropertyMapping)(
        LPCWSTR name,
        _Out_ UINT * index,
        _Out_ GRAPHICS_EFFECT_PROPERTY_MAPPING * mapping
        ) PURE;

    STDMETHOD(GetPropertyCount)(
        _Out_ UINT * count
        ) PURE;

    STDMETHOD(GetProperty)(
        UINT index,
        _Outptr_ winrt::impl::abi_t<winrt::Windows::Foundation::IPropertyValue> ** value
        ) PURE;

    STDMETHOD(GetSource)(
        UINT index,
        _Outptr_ IGraphicsEffectSource ** source
        ) PURE;

    STDMETHOD(GetSourceCount)(
        _Out_ UINT * count
        ) PURE;
};


} // namespace Effects
} // namespace Graphics
} // namespace Windows
#ifndef BUILD_WINDOWS
} // namespace ABI
#endif

template <> inline constexpr winrt::guid
winrt::impl::guid_v<ABI::Windows::Graphics::Effects::IGraphicsEffectD2D1Interop>{
    0x2FC57384, 0xA068, 0x44D7, { 0xA3, 0x31, 0x30, 0x98, 0x2F, 0xCF, 0x71, 0x77 }
};


//////////////////////////////////////////////////////////////////////////////
// CompositeEffect.h
struct CompositeEffect : winrt::implements<CompositeEffect, wge::IGraphicsEffect,
wge::IGraphicsEffectSource, awge::IGraphicsEffectD2D1Interop>
{
public:
    // IGraphicsEffectD2D1Interop
    HRESULT STDMETHODCALLTYPE GetEffectId(GUID* id) noexcept override;
    HRESULT STDMETHODCALLTYPE GetNamedPropertyMapping(LPCWSTR name, UINT* index,
awge::GRAPHICS_EFFECT_PROPERTY_MAPPING* mapping) noexcept override;
    HRESULT STDMETHODCALLTYPE GetPropertyCount(UINT* count) noexcept override;
    HRESULT STDMETHODCALLTYPE GetProperty(UINT index,
winrt::impl::abi_t<winrt::Windows::Foundation::IPropertyValue>** value) noexcept override;
    HRESULT STDMETHODCALLTYPE GetSource(UINT index, awge::IGraphicsEffectSource**
source) noexcept override;
    HRESULT STDMETHODCALLTYPE GetSourceCount(UINT* count) noexcept override;
```

```
    // IGraphicsEffect
    winrt::hstring Name();
    void Name(winrt::hstring name);

    std::vector<wge::IGraphicsEffectSource> Sources;
    D2D1_COMPOSITE_MODE Mode = D2D1_COMPOSITE_MODE_SOURCE_OVER;
private:
    winrt::hstring m_name = L"CompositeEffect";
};

//////////////////////////////////////////////////////////////////////////////
// CompositeEffect.cpp
HRESULT CompositeEffect::GetEffectId(GUID* id) noexcept
{
    if (id == nullptr) [[unlikely]]
    {
        return E_INVALIDARG;
    }

    *id = CLSID_D2D1Composite;
    return S_OK;
}

HRESULT CompositeEffect::GetNamedPropertyMapping(LPCWSTR name, UINT* index,
awge::GRAPHICS_EFFECT_PROPERTY_MAPPING* mapping) noexcept
{
    if (index == nullptr || mapping == nullptr) [[unlikely]]
    {
        return E_INVALIDARG;
    }

    const std::wstring_view nameView(name);
    if (nameView == L"Mode")
    {
        *index = D2D1_COMPOSITE_PROP_MODE;
        *mapping = awge::GRAPHICS_EFFECT_PROPERTY_MAPPING_DIRECT;

        return S_OK;
    }

    return E_INVALIDARG;
}

HRESULT CompositeEffect::GetPropertyCount(UINT* count) noexcept
{
    if (count == nullptr) [[unlikely]]
    {
        return E_INVALIDARG;
    }

    *count = 1;
    return S_OK;
}
```

```
HRESULT CompositeEffect::GetProperty(UINT index,
winrt::impl::abi_t<winrt::Windows::Foundation::IPropertyValue>** value) noexcept try
{
    if (value == nullptr) [[unlikely]]
    {
        return E_INVALIDARG;
    }

    switch (index)
    {
        case D2D1_COMPOSITE_PROP_MODE:
            *value =
wf::PropertyValue::CreateUInt32((UINT32)Mode).as<winrt::impl::abi_t<winrt::Windows::Found
ation::IPropertyValue>>().detach();
            break;

        default:
            return E_BOUNDS;
    }

    return S_OK;
}
catch (...)
{
    return winrt::to_hresult();
}

HRESULT CompositeEffect::GetSource(UINT index, awge::IGraphicsEffectSource** source)
noexcept try
{
    if (source == nullptr) [[unlikely]]
    {
        return E_INVALIDARG;
    }

    winrt::copy_to_abi(Sources.at(index), *reinterpret_cast<void**>(source));
    return S_OK;
}
catch (...)
{
    return winrt::to_hresult();
}

HRESULT CompositeEffect::GetSourceCount(UINT* count) noexcept
{
    if (count == nullptr) [[unlikely]]
    {
        return E_INVALIDARG;
    }

    *count = static_cast<UINT>(Sources.size());
    return S_OK;
}

winrt::hstring CompositeEffect::Name()
{
```

```
    return m_name;
}

void CompositeEffect::Name(winrt::hstring name)
{
    m_name = name;
}

//////////////////////////////////////////////////////////////////////////////
// FloodEffect.h
struct FloodEffect : winrt::implements<FloodEffect, wge::IGraphicsEffect,
wge::IGraphicsEffectSource, awge::IGraphicsEffectD2D1Interop>
{
public:
    // IGraphicsEffectD2D1Interop
    HRESULT STDMETHODCALLTYPE GetEffectId(GUID* id) noexcept override;
    HRESULT STDMETHODCALLTYPE GetNamedPropertyMapping(LPCWSTR name, UINT* index,
awge::GRAPHICS_EFFECT_PROPERTY_MAPPING* mapping) noexcept override;
    HRESULT STDMETHODCALLTYPE GetPropertyCount(UINT* count) noexcept override;
    HRESULT STDMETHODCALLTYPE GetProperty(UINT index,
winrt::impl::abi_t<winrt::Windows::Foundation::IPropertyValue>** value) noexcept override;
    HRESULT STDMETHODCALLTYPE GetSource(UINT index, awge::IGraphicsEffectSource**
source) noexcept override;
    HRESULT STDMETHODCALLTYPE GetSourceCount(UINT* count) noexcept override;

    // IGraphicsEffect
    winrt::hstring Name();
    void Name(winrt::hstring name);

    winrt::Windows::UI::Color Color{};
private:
    winrt::hstring m_name = L"FloodEffect";
};

//////////////////////////////////////////////////////////////////////////////
// FloodEffect.cpp
HRESULT FloodEffect::GetEffectId(GUID* id) noexcept
{
    if (id == nullptr) [[unlikely]]
    {
        return E_INVALIDARG;
    }

    *id = CLSID_D2D1Flood;
    return S_OK;
}

HRESULT FloodEffect::GetNamedPropertyMapping(LPCWSTR name, UINT* index,
awge::GRAPHICS_EFFECT_PROPERTY_MAPPING* mapping) noexcept
{
    if (index == nullptr || mapping == nullptr) [[unlikely]]
    {
        return E_INVALIDARG;
    }

    const std::wstring_view nameView(name);
    if (nameView == L"Color")
```

```
    {
        *index = D2D1_FLOOD_PROP_COLOR;
        *mapping = awge::GRAPHICS_EFFECT_PROPERTY_MAPPING_DIRECT;

        return S_OK;
    }

    return E_INVALIDARG;
}

HRESULT FloodEffect::GetPropertyCount(UINT* count) noexcept
{
    if (count == nullptr) [[unlikely]]
    {
        return E_INVALIDARG;
    }

    *count = 1;
    return S_OK;
}

HRESULT FloodEffect::GetProperty(UINT index,
winrt::impl::abi_t<winrt::Windows::Foundation::IPropertyValue>** value) noexcept try
{
    if (value == nullptr) [[unlikely]]
    {
        return E_INVALIDARG;
    }

    switch (index)
    {
        case D2D1_FLOOD_PROP_COLOR:
            *value = wf::PropertyValue::CreateSingleArray({
                Color.R / 255.0f,
                Color.G / 255.0f,
                Color.B / 255.0f,
                Color.A / 255.0f,
            }).as<winrt::impl::abi_t<winrt::Windows::Foundation::IPropertyValue>>().detach();
            break;

        default:
            return E_BOUNDS;
    }

    return S_OK;
}
catch (...)
{
    return winrt::to_hresult();
}

HRESULT FloodEffect::GetSource(UINT, awge::IGraphicsEffectSource** source) noexcept
{
    if (source == nullptr) [[unlikely]]
    {
```

```
        return E_INVALIDARG;
    }

    return E_BOUNDS;
}

HRESULT FloodEffect::GetSourceCount(UINT* count) noexcept
{
    if (count == nullptr) [[unlikely]]
    {
        return E_INVALIDARG;
    }

    *count = 0;
    return S_OK;
}

winrt::hstring FloodEffect::Name()
{
    return m_name;
}

void FloodEffect::Name(winrt::hstring name)
{
    m_name = name;
}

//////////////////////////////////////////////////////////////////////////////
// BorderEffect.h
struct BorderEffect : winrt::implements<BorderEffect, wge::IGraphicsEffect,
wge::IGraphicsEffectSource, awge::IGraphicsEffectD2D1Interop>
{
public:
    // IGraphicsEffectD2D1Interop
    HRESULT STDMETHODCALLTYPE GetEffectId(GUID* id) noexcept override;
    HRESULT STDMETHODCALLTYPE GetNamedPropertyMapping(LPCWSTR name, UINT* index,
awge::GRAPHICS_EFFECT_PROPERTY_MAPPING* mapping) noexcept override;
    HRESULT STDMETHODCALLTYPE GetPropertyCount(UINT* count) noexcept override;
    HRESULT STDMETHODCALLTYPE GetProperty(UINT index,
winrt::impl::abi_t<winrt::Windows::Foundation::IPropertyValue>** value) noexcept override;
    HRESULT STDMETHODCALLTYPE GetSource(UINT index, awge::IGraphicsEffectSource**
source) noexcept override;
    HRESULT STDMETHODCALLTYPE GetSourceCount(UINT* count) noexcept override;

    // IGraphicsEffect
    winrt::hstring Name();
    void Name(winrt::hstring name);

    wge::IGraphicsEffectSource Source{nullptr};
    D2D1_BORDER_EDGE_MODE ExtendX = D2D1_BORDER_EDGE_MODE_WRAP;
    D2D1_BORDER_EDGE_MODE ExtendY = D2D1_BORDER_EDGE_MODE_WRAP;
private:
    winrt::hstring m_name = L"BorderEffect";
};

//////////////////////////////////////////////////////////////////////////////
```

```
// BorderEffect.cpp
HRESULT BorderEffect::GetEffectId(GUID* id) noexcept
{
   if (!id)
   {
      return E_INVALIDARG;
   }

   *id = CLSID_D2D1Border;
   return S_OK;
}

HRESULT BorderEffect::GetNamedPropertyMapping(LPCWSTR name, UINT* index,
awge::GRAPHICS_EFFECT_PROPERTY_MAPPING* mapping) noexcept
{
   if (!index || !mapping)
   {
      return E_INVALIDARG;
   }

   const std::wstring_view nameView(name);
   if (nameView == L"ExtendX")
   {
      *index = D2D1_BORDER_PROP_EDGE_MODE_X;
      *mapping = awge::GRAPHICS_EFFECT_PROPERTY_MAPPING_DIRECT;

      return S_OK;
   }

   if (nameView == L"ExtendY")
   {
      *index = D2D1_BORDER_PROP_EDGE_MODE_Y;
      *mapping = awge::GRAPHICS_EFFECT_PROPERTY_MAPPING_DIRECT;

      return S_OK;
   }

   return E_INVALIDARG;
}

HRESULT BorderEffect::GetPropertyCount(UINT* count) noexcept
{
   if (!count)
   {
      return E_INVALIDARG;
   }

   *count = 2;
   return S_OK;
}

HRESULT BorderEffect::GetProperty(UINT index,
winrt::impl::abi_t<winrt::Windows::Foundation::IPropertyValue>** value) noexcept try
{
   if (!value)
   {
```

```
        return E_INVALIDARG;
    }

    switch (index)
    {
        case D2D1_BORDER_PROP_EDGE_MODE_X:
            *value =
wf::PropertyValue::CreateUInt32((UINT32)ExtendX).as<winrt::impl::abi_t<winrt::Windows::Foundation::IPropertyValue>>().detach();
            break;

        case D2D1_BORDER_PROP_EDGE_MODE_Y:
            *value =
wf::PropertyValue::CreateUInt32((UINT32)ExtendY).as<winrt::impl::abi_t<winrt::Windows::Foundation::IPropertyValue>>().detach();
            break;

        default:
            return E_BOUNDS;
    }

    return S_OK;
}
catch (...)
{
    return winrt::to_hresult();
}

HRESULT BorderEffect::GetSource(UINT index, awge::IGraphicsEffectSource** source) noexcept
{
    if (!source)
    {
        return E_INVALIDARG;
    }

    if (index == 0 && Source)
    {
        winrt::copy_to_abi(Source, *reinterpret_cast<void**>(source));
        return S_OK;
    }

    return E_BOUNDS;
}

HRESULT BorderEffect::GetSourceCount(UINT* count) noexcept
{
    if (!count)
    {
        return E_INVALIDARG;
    }

    *count = 1;
    return S_OK;
}

winrt::hstring BorderEffect::Name()
```

```
{
    return m_name;
}

void BorderEffect::Name(winrt::hstring name)
{
    m_name = name;
}

//////////////////////////////////////////////////////////////////////////////
// GaussianBlurEffect.h
struct GaussianBlurEffect : winrt::implements<GaussianBlurEffect, wge::IGraphicsEffect,
wge::IGraphicsEffectSource, awge::IGraphicsEffectD2D1Interop>
{
public:
    // IGraphicsEffectD2D1Interop
    HRESULT STDMETHODCALLTYPE GetEffectId(GUID* id) noexcept override;
    HRESULT STDMETHODCALLTYPE GetNamedPropertyMapping(LPCWSTR name, UINT* index,
awge::GRAPHICS_EFFECT_PROPERTY_MAPPING* mapping) noexcept override;
    HRESULT STDMETHODCALLTYPE GetPropertyCount(UINT* count) noexcept override;
    HRESULT STDMETHODCALLTYPE GetProperty(UINT index,
winrt::impl::abi_t<winrt::Windows::Foundation::IPropertyValue>** value) noexcept override;
    HRESULT STDMETHODCALLTYPE GetSource(UINT index, awge::IGraphicsEffectSource**
source) noexcept override;
    HRESULT STDMETHODCALLTYPE GetSourceCount(UINT* count) noexcept override;

    // IGraphicsEffect
    winrt::hstring Name();
    void Name(winrt::hstring name);

    wge::IGraphicsEffectSource Source;

    float BlurAmount = 3.0f;
    MY_D2D1_GAUSSIANBLUR_OPTIMIZATION Optimization =
MY_D2D1_GAUSSIANBLUR_OPTIMIZATION_BALANCED;
    D2D1_BORDER_MODE BorderMode = D2D1_BORDER_MODE_SOFT;
private:
    winrt::hstring m_name = L"GaussianBlurEffect";
};

//////////////////////////////////////////////////////////////////////////////
// GaussianBlurEffect.cpp
HRESULT GaussianBlurEffect::GetEffectId(GUID* id) noexcept
{
    if (id == nullptr) [[unlikely]]
    {
        return E_INVALIDARG;
    }

    *id = CLSID_D2D1GaussianBlur;
    return S_OK;
}

HRESULT GaussianBlurEffect::GetNamedPropertyMapping(LPCWSTR name, UINT* index,
awge::GRAPHICS_EFFECT_PROPERTY_MAPPING* mapping) noexcept
{
    if (index == nullptr || mapping == nullptr) [[unlikely]]
```

```
    {
        return E_INVALIDARG;
    }

    const std::wstring_view nameView(name);
    if (nameView == L"BlurAmount")
    {
        *index = D2D1_GAUSSIANBLUR_PROP_STANDARD_DEVIATION;
        *mapping = awge::GRAPHICS_EFFECT_PROPERTY_MAPPING_DIRECT;

        return S_OK;
    }
    else if (nameView == L"Optimization")
    {
        *index = D2D1_GAUSSIANBLUR_PROP_OPTIMIZATION;
        *mapping = awge::GRAPHICS_EFFECT_PROPERTY_MAPPING_DIRECT;

        return S_OK;
    }
    else if (nameView == L"BorderMode")
    {
        *index = D2D1_GAUSSIANBLUR_PROP_BORDER_MODE;
        *mapping = awge::GRAPHICS_EFFECT_PROPERTY_MAPPING_DIRECT;

        return S_OK;
    }

    return E_INVALIDARG;
}

HRESULT GaussianBlurEffect::GetPropertyCount(UINT* count) noexcept
{
    if (count == nullptr) [[unlikely]]
    {
        return E_INVALIDARG;
    }

    *count = 3;
    return S_OK;
}

HRESULT GaussianBlurEffect::GetProperty(UINT index,
winrt::impl::abi_t<winrt::Windows::Foundation::IPropertyValue>** value) noexcept try
{
    if (value == nullptr) [[unlikely]]
    {
        return E_INVALIDARG;
    }

    switch (index)
    {
        case D2D1_GAUSSIANBLUR_PROP_STANDARD_DEVIATION:
            *value =
wf::PropertyValue::CreateSingle(BlurAmount).as<winrt::impl::abi_t<winrt::Windows::Foundatio
n::IPropertyValue>>().detach();
            break;
```

```
        case D2D1_GAUSSIANBLUR_PROP_OPTIMIZATION:
            *value = wf::PropertyValue::CreateUInt32((UINT32)Optimization).as<winrt::impl::abi_t<winrt::Windows::Foundation::IPropertyValue>>().detach();
            break;

        case D2D1_GAUSSIANBLUR_PROP_BORDER_MODE:
            *value = wf::PropertyValue::CreateUInt32((UINT32)BorderMode).as<winrt::impl::abi_t<winrt::Windows::Foundation::IPropertyValue>>().detach();
            break;

        default:
            return E_BOUNDS;
    }

    return S_OK;
}
catch (...)
{
    return winrt::to_hresult();
}

HRESULT GaussianBlurEffect::GetSource(UINT index, awge::IGraphicsEffectSource** source) noexcept
{
    if (source == nullptr) [[unlikely]]
    {
        return E_INVALIDARG;
    }

    if (index == 0)
    {
        winrt::copy_to_abi(Source, *reinterpret_cast<void**>(source));
        return S_OK;
    }
    else
    {
        return E_BOUNDS;
    }
}

HRESULT GaussianBlurEffect::GetSourceCount(UINT* count) noexcept
{
    if (count == nullptr) [[unlikely]]
    {
        return E_INVALIDARG;
    }

    *count = 1;
    return S_OK;
}

winrt::hstring GaussianBlurEffect::Name()
{
    return m_name;
```

```
}

void GaussianBlurEffect::Name(winrt::hstring name)
{
    m_name = name;
}

//////////////////////////////////////////////////////////////////////////////
// ColorMatrixEffect.h
struct ColorMatrixEffect : winrt::implements<ColorMatrixEffect, wge::IGraphicsEffect,
wge::IGraphicsEffectSource, awge::IGraphicsEffectD2D1Interop>
{
public:
    // IGraphicsEffectD2D1Interop
    HRESULT STDMETHODCALLTYPE GetEffectId(GUID* id) noexcept override;
    HRESULT STDMETHODCALLTYPE GetNamedPropertyMapping(LPCWSTR name, UINT* index,
awge::GRAPHICS_EFFECT_PROPERTY_MAPPING* mapping) noexcept override;
    HRESULT STDMETHODCALLTYPE GetPropertyCount(UINT* count) noexcept override;
    HRESULT STDMETHODCALLTYPE GetProperty(UINT index,
winrt::impl::abi_t<winrt::Windows::Foundation::IPropertyValue>** value) noexcept override;
    HRESULT STDMETHODCALLTYPE GetSource(UINT index, awge::IGraphicsEffectSource**
source) noexcept override;
    HRESULT STDMETHODCALLTYPE GetSourceCount(UINT* count) noexcept override;

    // IGraphicsEffect
    winrt::hstring Name();
    void Name(winrt::hstring name);

    wge::IGraphicsEffectSource Source{nullptr};

    // D2D1_MATRIX_5X4_F: 5 rows x 4 columns (20 floats), initialized to identity.
    float Matrix[20] = {
        1, 0, 0, 0,
        0, 1, 0, 0,
        0, 0, 1, 0,
        0, 0, 0, 1,
        0, 0, 0, 0,
    };

    uint32_t AlphaMode = D2D1_COLORMATRIX_ALPHA_MODE_PREMULTIPLIED;
    bool ClampOutput = false;
private:
    winrt::hstring m_name = L"ColorMatrixEffect";
};

//////////////////////////////////////////////////////////////////////////////
// ColorMatrixEffect.cpp
HRESULT ColorMatrixEffect::GetEffectId(GUID* id) noexcept
{
    if (!id)
    {
        return E_INVALIDARG;
    }

    *id = CLSID_D2D1ColorMatrix;
    return S_OK;
```

```
}

HRESULT ColorMatrixEffect::GetNamedPropertyMapping(LPCWSTR name, UINT* index,
awge::GRAPHICS_EFFECT_PROPERTY_MAPPING* mapping) noexcept
{
   if (!index || !mapping)
   {
      return E_INVALIDARG;
   }

   const std::wstring_view nameView(name);
   if (nameView == L"ColorMatrix")
   {
      *index = D2D1_COLORMATRIX_PROP_COLOR_MATRIX;
      *mapping = awge::GRAPHICS_EFFECT_PROPERTY_MAPPING_DIRECT;

      return S_OK;
   }

   if (nameView == L"AlphaMode")
   {
      *index = D2D1_COLORMATRIX_PROP_ALPHA_MODE;
      *mapping = awge::GRAPHICS_EFFECT_PROPERTY_MAPPING_DIRECT;

      return S_OK;
   }

   if (nameView == L"ClampOutput")
   {
      *index = D2D1_COLORMATRIX_PROP_CLAMP_OUTPUT;
      *mapping = awge::GRAPHICS_EFFECT_PROPERTY_MAPPING_DIRECT;

      return S_OK;
   }

   return E_INVALIDARG;
}

HRESULT ColorMatrixEffect::GetPropertyCount(UINT* count) noexcept
{
   if (!count)
   {
      return E_INVALIDARG;
   }

   *count = 3;
   return S_OK;
}

HRESULT ColorMatrixEffect::GetProperty(UINT index,
winrt::impl::abi_t<winrt::Windows::Foundation::IPropertyValue>** value) noexcept try
{
   if (!value)
   {
      return E_INVALIDARG;
   }
```

```
    switch (index)
    {
        case D2D1_COLORMATRIX_PROP_COLOR_MATRIX:
            *value = wf::PropertyValue::CreateSingleArray(
                winrt::array_view<const float>(Matrix, Matrix + 20)
            ).as<winrt::impl::abi_t<winrt::Windows::Foundation::IPropertyValue>>().detach();
            break;

        case D2D1_COLORMATRIX_PROP_ALPHA_MODE:
            *value =
wf::PropertyValue::CreateUInt32(AlphaMode).as<winrt::impl::abi_t<winrt::Windows::Foundatio
n::IPropertyValue>>().detach();
            break;

        case D2D1_COLORMATRIX_PROP_CLAMP_OUTPUT:
            *value =
wf::PropertyValue::CreateBoolean(ClampOutput).as<winrt::impl::abi_t<winrt::Windows::Found
ation::IPropertyValue>>().detach();
            break;

        default:
            return E_BOUNDS;
    }

    return S_OK;
}
catch (...)
{
    return winrt::to_hresult();
}

HRESULT ColorMatrixEffect::GetSource(UINT index, awge::IGraphicsEffectSource** source)
noexcept
{
    if (!source)
    {
        return E_INVALIDARG;
    }

    if (index == 0 && Source)
    {
        winrt::copy_to_abi(Source, *reinterpret_cast<void**>(source));
        return S_OK;
    }

    return E_BOUNDS;
}

HRESULT ColorMatrixEffect::GetSourceCount(UINT* count) noexcept
{
    if (!count)
    {
        return E_INVALIDARG;
    }

    *count = 1;
```

```
    return S_OK;
}

winrt::hstring ColorMatrixEffect::Name()
{
    return m_name;
}

void ColorMatrixEffect::Name(winrt::hstring name)
{
    m_name = name;
}

//////////////////////////////////////////////////////////////////////////////
// XamlBlurBrush.cpp
XamlBlurBrush::XamlBlurBrush(UIElement element,
                             float blurAmount,
                             winrt::Windows::UI::Color tint,
                             std::optional<uint8_t> tintOpacity,
                             winrt::hstring tintThemeResourceKey,
                             std::optional<float> tintLuminosityOpacity,
                             std::optional<float> tintSaturation,
                             std::optional<float> noiseOpacity,
                             std::optional<float> noiseDensity,
                             std::optional<winrt::Windows::UI::Color> fallbackColor,
                             winrt::hstring fallbackThemeResourceKey) :
    m_compositor(wuxh::ElementCompositionPreview::GetElementVisual(element)
                 .Compositor()),
    m_blurAmount(blurAmount),
    m_tint(tint),
    m_tintOpacity(tintOpacity),
    m_tintThemeResourceKey(std::move(tintThemeResourceKey)),
    m_tintLuminosityOpacity(tintLuminosityOpacity),
    m_tintSaturation(tintSaturation),
    m_noiseOpacity(noiseOpacity),
    m_noiseDensity(noiseDensity),
    m_fallbackColor(fallbackColor),
    m_fallbackThemeResourceKey(std::move(fallbackThemeResourceKey))
{
    auto fe = element.try_as<FrameworkElement>();

    auto createProxy = [&](winrt::hstring const& themeResourceKey)
        -> Media::SolidColorBrush
    {
        if (!fe)
        {
            return nullptr;
        }
        std::wstring xaml =
            L"<SolidColorBrush"
            L" xmlns=\"http://schemas.microsoft.com/winfx/2006/xaml/"
            L"presentation\""
            L" Color=\"{ThemeResource " +
            std::wstring(themeResourceKey) + L"}\"/>";
        try
```

```
    {
        return Markup::XamlReader::Load(winrt::hstring(xaml))
            .try_as<Media::SolidColorBrush>();
    }
    catch (winrt::hresult_error const& ex)
    {
        Wh_Log(L"Failed to create proxy brush: %08X", ex.code());
        return nullptr;
    }
};

static std::atomic<uint64_t> s_proxyCounter{0};

if (!m_tintThemeResourceKey.empty())
{
    if (auto proxyBrush = createProxy(m_tintThemeResourceKey))
    {
        auto proxyKey = winrt::hstring(
            L"__WhBlurProxy_" +
            std::to_wstring(++s_proxyCounter));
        fe.Resources().Insert(
            winrt::box_value(proxyKey), proxyBrush);
        m_proxyBrush = proxyBrush;
        m_weakProxyElement = winrt::make_weak(fe);
        m_proxyKey = proxyKey;
        Wh_Log(L"Tint proxy brush for %s inserted with key %s",
            m_tintThemeResourceKey.c_str(),
            proxyKey.c_str());
    }

    if (m_proxyBrush)
    {
        m_proxyBrush.RegisterPropertyChangedCallback(
            Media::SolidColorBrush::ColorProperty(),
            [weakThis = get_weak()](auto&&, auto&&)
            {
                if (auto self = weakThis.get())
                {
                    Wh_Log(L"Tint theme color changed");
                    self->RefreshBrush();
                }
            });
    }
}

if (!m_fallbackThemeResourceKey.empty())
{
    if (auto proxyBrush = createProxy(m_fallbackThemeResourceKey))
    {
        auto proxyKey = winrt::hstring(
            L"__WhBlurFallbackProxy_" +
            std::to_wstring(++s_proxyCounter));
        fe.Resources().Insert(
            winrt::box_value(proxyKey), proxyBrush);
        m_fallbackProxyBrush = proxyBrush;
```

```
        if (!m_weakProxyElement.get())
        {
            m_weakProxyElement = winrt::make_weak(fe);
        }
        m_fallbackProxyKey = proxyKey;
        Wh_Log(L"Fallback proxy brush for %s inserted with key %s",
            m_fallbackThemeResourceKey.c_str(),
            proxyKey.c_str());
    }

    if (m_fallbackProxyBrush)
    {
        m_fallbackProxyBrush.RegisterPropertyChangedCallback(
            Media::SolidColorBrush::ColorProperty(),
            [weakThis = get_weak()](auto&&, auto&&)
            {
                if (auto self = weakThis.get())
                {
                    Wh_Log(L"Fallback theme color changed");
                    self->RefreshBrush();
                }
            });
    }
}

if (m_fallbackColor || !m_fallbackThemeResourceKey.empty())
{
    m_dispatcher =
        winrt::Windows::System::DispatcherQueue::GetForCurrentThread();

    try
    {
        m_uiSettings = winrt::Windows::UI::ViewManagement::UISettings();
        auto dispatcher = m_dispatcher;
        m_advancedEffectsEnabledChangedToken =
            m_uiSettings.AdvancedEffectsEnabledChanged(
                [weakThis = get_weak(), dispatcher](auto&&, auto&&)
                {
                    dispatcher.TryEnqueue([weakThis]
                    {
                        if (auto self = weakThis.get())
                        {
                            Wh_Log(L"AdvancedEffectsEnabled changed");
                            self->RefreshBrush();
                        }
                    });
                });
        m_energySaverStatusChangedToken =
            winrt::Windows::System::Power::PowerManager::
                EnergySaverStatusChanged(
                    [weakThis = get_weak(), dispatcher](auto&&, auto&&)
                    {
                        dispatcher.TryEnqueue([weakThis]
                        {
                            if (auto self = weakThis.get())
```

```
                {
                    Wh_Log(L"EnergySaverStatus changed");
                    self->RefreshBrush();
                }
            });
        });
}
catch (winrt::hresult_error const& ex)
{
    Wh_Log(L"Failed to register fallback state listeners: %08X",
        ex.code());
}

// Watch HKLM\SYSTEM\CurrentControlSet\Control\Power for changes to
// EnergySaverState. On Windows 11 24H2+ neither the WinRT
// PowerManager.EnergySaverStatus property nor the Win32
// GetSystemPowerStatus.SystemStatusFlag flag reliably reflects the
// "Always use energy saver" setting; the registry value is the only
// signal that updates in that case. The wait callback re-arms the
// notification and posts a brush refresh on the UI thread.
LONG regStatus = RegOpenKeyExW(
    HKEY_LOCAL_MACHINE,
    L"SYSTEM\\CurrentControlSet\\Control\\Power", 0, KEY_NOTIFY,
    &m_powerKey);
if (regStatus == ERROR_SUCCESS)
{
    m_regNotifyEvent = CreateEventW(nullptr, FALSE, FALSE, nullptr);
    if (m_regNotifyEvent)
    {
        regStatus = RegNotifyChangeKeyValue(m_powerKey, FALSE,
                                REG_NOTIFY_CHANGE_LAST_SET,
                                m_regNotifyEvent, TRUE);
        if (regStatus == ERROR_SUCCESS)
        {
            if (!RegisterWaitForSingleObject(
                    &m_regWaitHandle, m_regNotifyEvent,
                    OnEnergySaverRegistryChanged, this, INFINITE,
                    WT_EXECUTEINWAITTHREAD))
            {
                Wh_Log(L"RegisterWaitForSingleObject failed: %lu",
                    GetLastError());
                m_regWaitHandle = nullptr;
            }
        }
        else
        {
            Wh_Log(L"RegNotifyChangeKeyValue failed: %ld", regStatus);
            CloseHandle(m_regNotifyEvent);
            m_regNotifyEvent = nullptr;
            RegCloseKey(m_powerKey);
            m_powerKey = nullptr;
        }
    }
    else
    {
```

```
            Wh_Log(L"CreateEvent failed: %lu", GetLastError());
            RegCloseKey(m_powerKey);
            m_powerKey = nullptr;
        }
      }
      else
      {
        Wh_Log(L"RegOpenKeyEx for Power key failed: %ld", regStatus);
      }
    }
}

void CALLBACK XamlBlurBrush::OnEnergySaverRegistryChanged(PVOID context,
                                          BOOLEAN)
{
    auto* self = static_cast<XamlBlurBrush*>(context);

    // Re-arm before dispatching so a rapid second change isn't dropped.
    if (self->m_powerKey && self->m_regNotifyEvent)
    {
        RegNotifyChangeKeyValue(self->m_powerKey, FALSE,
                                REG_NOTIFY_CHANGE_LAST_SET,
                                self->m_regNotifyEvent, TRUE);
    }

    if (self->m_dispatcher)
    {
        auto weakThis = self->get_weak();
        self->m_dispatcher.TryEnqueue([weakThis]
        {
            if (auto strongThis = weakThis.get())
            {
                Wh_Log(L"Power registry key changed, refreshing brush");
                strongThis->RefreshBrush();
            }
        });
    }
}

XamlBlurBrush::~XamlBlurBrush()
{
    // Tear down the registry watch first so no more callbacks can fire while
    // we close the underlying handles.
    if (m_regWaitHandle)
    {
        UnregisterWaitEx(m_regWaitHandle, INVALID_HANDLE_VALUE);
        m_regWaitHandle = nullptr;
    }
    if (m_regNotifyEvent)
    {
        CloseHandle(m_regNotifyEvent);
        m_regNotifyEvent = nullptr;
    }
    if (m_powerKey)
    {
```

```
        RegCloseKey(m_powerKey);
        m_powerKey = nullptr;
    }

    if (m_uiSettings && m_advancedEffectsEnabledChangedToken.value)
    {
        try
        {
            m_uiSettings.AdvancedEffectsEnabledChanged(
                m_advancedEffectsEnabledChangedToken);
        }
        catch (...)
        {
            Wh_Log(L"Error %08X", winrt::to_hresult());
        }
    }

    if (m_energySaverStatusChangedToken.value)
    {
        try
        {
            winrt::Windows::System::Power::PowerManager::
                EnergySaverStatusChanged(m_energySaverStatusChangedToken);
        }
        catch (...)
        {
            Wh_Log(L"Error %08X", winrt::to_hresult());
        }
    }

    if (auto element = m_weakProxyElement.get())
    {
        try
        {
            if (!m_proxyKey.empty())
            {
                element.Resources().Remove(winrt::box_value(m_proxyKey));
            }
            if (!m_fallbackProxyKey.empty())
            {
                element.Resources().Remove(
                    winrt::box_value(m_fallbackProxyKey));
            }
        }
        catch (...)
        {
            HRESULT hr = winrt::to_hresult();
            Wh_Log(L"Error %08X", hr);
        }
    }
}

void XamlBlurBrush::OnConnected()
{
    if (!CompositionBrush())
```

```
    {
        RefreshThemeTint();
        RefreshFallbackColor();

        CompositionBrush(ShouldUseFallback() ? CreateFallbackBrush()
                                             : CreateEffectBrush());
    }
}

wuc::CompositionBrush XamlBlurBrush::CreateFallbackBrush()
{
    return m_compositor.CreateColorBrush(m_fallbackColor.value_or(m_tint));
}

wuc::CompositionBrush XamlBlurBrush::CreateEffectBrush()
{
    auto backdropBrush = m_compositor.CreateBackdropBrush();

    // Rec. 709 luma coefficients, used for saturation and luminosity.
    constexpr float kLumaR = 0.2126f;
    constexpr float kLumaG = 0.7152f;
    constexpr float kLumaB = 0.0722f;

    // 1. Blur
    auto blurEffect = winrt::make_self<GaussianBlurEffect>();
    blurEffect->Source = wuc::CompositionEffectSourceParameter(L"backdrop");
    blurEffect->BlurAmount = m_blurAmount;
    blurEffect->Name(L"BlurEffect");

    wge::IGraphicsEffectSource topOfStack = *blurEffect;

    // 2. Saturation (optional)
    if (m_tintSaturation && *m_tintSaturation != 1.0f)
    {
        float s = std::max(*m_tintSaturation, 0.0f);
        float invS = 1.0f - s;

        auto satMatrix = winrt::make_self<ColorMatrixEffect>();
        satMatrix->Source = topOfStack;

        // Standard saturation matrix: lerp between luminance and identity.
        auto& m = satMatrix->Matrix;
        m[0]  = invS * kLumaR + s; m[1]  = invS * kLumaR;     m[2]  = invS * kLumaR;     m[3]  = 0.0f;
        m[4]  = invS * kLumaG;     m[5]  = invS * kLumaG + s; m[6]  = invS * kLumaG;     m[7]  = 0.0f;
        m[8]  = invS * kLumaB;     m[9]  = invS * kLumaB;     m[10] = invS * kLumaB + s; m[11] = 0.0f;
        m[12] = 0.0f;              m[13] = 0.0f;              m[14] = 0.0f;              m[15] = 1.0f;

        satMatrix->Name(L"SaturationEffect");
        topOfStack = *satMatrix;
    }

    // 3. Luminosity (optional) - shifts pixel luminance towards the tint's
    // luminance, blended by the opacity factor.
    if (m_tintLuminosityOpacity && *m_tintLuminosityOpacity > 0.0f)
    {
```

```
        float op = std::clamp(*m_tintLuminosityOpacity, 0.0f, 1.0f);

        float tintLum = (m_tint.R / 255.0f) * kLumaR +
                        (m_tint.G / 255.0f) * kLumaG +
                        (m_tint.B / 255.0f) * kLumaB;

        auto lumMatrix = winrt::make_self<ColorMatrixEffect>();
        lumMatrix->Source = topOfStack;

        auto& m = lumMatrix->Matrix;
        m[0]  = 1.0f - (kLumaR * op); m[1]  = -(kLumaR * op);      m[2]  = -(kLumaR * op);      m[3]  =
0.0f;
        m[4]  = -(kLumaG * op);      m[5]  = 1.0f - (kLumaG * op); m[6]  = -(kLumaG * op);      m[7]  =
0.0f;
        m[8]  = -(kLumaB * op);      m[9]  = -(kLumaB * op);      m[10] = 1.0f - (kLumaB * op); m[11]
= 0.0f;
        m[12] = 0.0f;                m[13] = 0.0f;                m[14] = 0.0f;                m[15] = 1.0f;
        m[16] = tintLum * op;        m[17] = tintLum * op;        m[18] = tintLum * op;        m[19] =
0.0f;

        lumMatrix->Name(L"LuminosityBlend");
        topOfStack = *lumMatrix;
    }

    // 4. Noise overlay (optional) - procedural tiled noise with adjustable
    // density and opacity.
    wuc::CompositionSurfaceBrush noiseBrush{nullptr};
    if (m_noiseOpacity && *m_noiseOpacity > 0.0f)
    {
        float density = m_noiseDensity.value_or(1.0f);

        auto stream = CreateNoiseStream(density);
        auto surface =
            Media::LoadedImageSurface::StartLoadFromStream(stream);
        noiseBrush = m_compositor.CreateSurfaceBrush(surface);
        noiseBrush.Stretch(wuc::CompositionStretch::None);

        // Tile via border effect (wrap mode).
        auto borderEffect = winrt::make_self<BorderEffect>();
        borderEffect->Source =
            wuc::CompositionEffectSourceParameter(L"NoiseSource");

        // Scale all channels by opacity for premultiplied blending.
        float nOp = std::clamp(*m_noiseOpacity, 0.0f, 1.0f);

        auto opacityEffect = winrt::make_self<ColorMatrixEffect>();
        opacityEffect->Source = *borderEffect;
        // Matrix: Scale all channels by opacity (for premultiplied blending).
        opacityEffect->Matrix[0] = nOp;
        opacityEffect->Matrix[5] = nOp;
        opacityEffect->Matrix[10] = nOp;
        opacityEffect->Matrix[15] = nOp;
        opacityEffect->Name(L"NoiseOpacityEffect");

        // Composite noise over the current stack.
        auto noiseComposite = winrt::make_self<CompositeEffect>();
```

```
        noiseComposite->Mode = D2D1_COMPOSITE_MODE_SOURCE_OVER;
        noiseComposite->Sources.push_back(topOfStack);
        noiseComposite->Sources.push_back(*opacityEffect);
        noiseComposite->Name(L"NoiseComposite");
        topOfStack = *noiseComposite;
    }

    // 5. Tint (flood color composited over the stack).
    auto floodEffect = winrt::make_self<FloodEffect>();
    floodEffect->Color = m_tint;
    floodEffect->Name(L"FloodEffect");

    auto compositeEffect = winrt::make_self<CompositeEffect>();
    compositeEffect->Mode = D2D1_COMPOSITE_MODE_SOURCE_OVER;
    compositeEffect->Sources.push_back(topOfStack);
    compositeEffect->Sources.push_back(*floodEffect);

    auto factory = m_compositor.CreateEffectFactory(*compositeEffect);
    auto brush = factory.CreateBrush();

    brush.SetSourceParameter(L"backdrop", backdropBrush);

    // Bind the noise brush if we created one.
    if (noiseBrush)
    {
        brush.SetSourceParameter(L"NoiseSource", noiseBrush);
    }

    return brush;
}

void XamlBlurBrush::OnDisconnected()
{
    if (const auto brush = CompositionBrush())
    {
        brush.Close();
        CompositionBrush(nullptr);
    }
}

void XamlBlurBrush::RefreshThemeTint()
{
    if (!m_proxyBrush)
    {
        return;
    }

    m_tint = m_proxyBrush.Color();
    if (m_tintOpacity)
    {
        m_tint.A = *m_tintOpacity;
    }
}

void XamlBlurBrush::RefreshFallbackColor()
```

```
{
   if (!m_fallbackProxyBrush)
   {
      return;
   }

   m_fallbackColor = m_fallbackProxyBrush.Color();
}

bool XamlBlurBrush::ShouldUseFallback() const
{
   if (!m_fallbackColor && m_fallbackThemeResourceKey.empty())
   {
      return false;
   }

   // The HKLM\SYSTEM\CurrentControlSet\Control\Power\EnergySaverState value
   // is the only signal that consistently reflects "Always use energy saver"
   // on Windows 11 24H2+; the WinRT and Win32 power-status APIs can stay
   // stuck in the off state on those builds. 1 = enabled, 2 = disabled.
   bool energySaverActive = false;
   HKEY key{};
   if (RegOpenKeyExW(HKEY_LOCAL_MACHINE,
                L"SYSTEM\\CurrentControlSet\\Control\\Power", 0,
                KEY_QUERY_VALUE, &key) == ERROR_SUCCESS)
   {
      DWORD value = 0;
      DWORD type = 0;
      DWORD size = sizeof(value);
      if (RegQueryValueExW(key, L"EnergySaverState", nullptr, &type,
                     reinterpret_cast<LPBYTE>(&value),
                     &size) == ERROR_SUCCESS &&
         type == REG_DWORD)
      {
         energySaverActive = (value == 1);
      }
      RegCloseKey(key);
   }

   // Backup for older Windows where the registry value above isn't populated.
   if (!energySaverActive)
   {
      SYSTEM_POWER_STATUS powerStatus{};
      if (GetSystemPowerStatus(&powerStatus) &&
         powerStatus.SystemStatusFlag != 0)
      {
         energySaverActive = true;
      }
   }

   bool advancedEffectsOff = false;
   if (m_uiSettings)
   {
      try
      {
```

```
            advancedEffectsOff = !m_uiSettings.AdvancedEffectsEnabled();
        }
        catch (...)
        {
            Wh_Log(L"AdvancedEffectsEnabled query failed: %08X",
                winrt::to_hresult());
        }
    }

    return energySaverActive || advancedEffectsOff;
}

void XamlBlurBrush::RefreshBrush()
{
    if (const auto brush = CompositionBrush())
    {
        brush.Close();
        CompositionBrush(nullptr);
        OnConnected();
    }
}

// clang-format on
////////////////////////////////////////////////////////////////////////////////

// Helper functions for tracking, caching and retrying remote image loads.

// Reports true if the query itself fails, as a retry which turns out to be
// pointless is harmless, while skipping a necessary one leaves images missing.
bool HasInternetAccess() {
    try {
        auto profile = winrt::Windows::Networking::Connectivity::
            NetworkInformation::GetInternetConnectionProfile();
        return profile && profile.GetNetworkConnectivityLevel() ==
                          winrt::Windows::Networking::Connectivity::
                              NetworkConnectivityLevel::InternetAccess;
    } catch (winrt::hresult_error const& ex) {
        Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
        return true;
    }
}

// The folder the remote images are cached in, empty if it's not available, in
// which case images are only ever loaded from their remote source.
const std::filesystem::path& GetImageCacheDir() {
    static const std::filesystem::path dir = []() -> std::filesystem::path {
        WCHAR storagePathBuffer[MAX_PATH];
        if (!Wh_GetModStoragePath(storagePathBuffer,
                                  ARRAYSIZE(storagePathBuffer))) {
            Wh_Log(L"Wh_GetModStoragePath failed");
            return std::filesystem::path();
        }

        auto path = std::filesystem::path{storagePathBuffer} / L"images";
```

```
        std::error_code ec;
        std::filesystem::create_directories(path, ec);
        if (!std::filesystem::is_directory(path, ec)) {
            Wh_Log(L"Failed to create %s", path.c_str());
            return std::filesystem::path();
        }

        return path;
    }();

    return dir;
}

// The cached copy of a remote image, named uniquely after its URL. Empty when
// there's no cache folder. The extension of the URL is kept so that the folder
// can be looked through.
std::filesystem::path ImageCachePath(std::wstring_view url) {
    const auto& cacheDir = GetImageCacheDir();
    if (cacheDir.empty()) {
        return std::filesystem::path();
    }

    // FNV-1a; one mod's folder, so an unlikely collision is good enough.
    uint64_t hash = 14695981039346656037ULL;
    for (wchar_t c : url) {
        hash ^= (uint16_t)c;
        hash *= 1099511628211ULL;
    }

    WCHAR hashString[17];
    swprintf_s(hashString, L"%016llx", hash);

    std::wstring name = hashString;

    auto urlPath = url.substr(0, url.find_first_of(L"?#"));
    auto extension = std::filesystem::path(urlPath).extension().native();
    if (extension.length() >= 2 && extension.length() <= 5 &&
        std::all_of(extension.begin() + 1, extension.end(), [](wchar_t c) {
            return (c >= L'a' && c <= L'z') || (c >= L'A' && c <= L'Z') ||
                   (c >= L'0' && c <= L'9');
        })) {
        name += extension;
    }

    return cacheDir / name;
}

// XAML loads a local image through a file URI.
winrt::Windows::Foundation::Uri ImageCacheFileUri(
    const std::filesystem::path& path) {
    // Room for every character to be escaped, plus the scheme.
    std::wstring uri(path.native().size() * 3 + 16, L'\0');

    DWORD uriLength = (DWORD)uri.size();
    HRESULT hr = UrlCreateFromPath(path.c_str(), uri.data(), &uriLength, 0);
```

```
    if (FAILED(hr)) {
        Wh_Log(L"UrlCreateFromPath returned 0x%08X", hr);
        return nullptr;
    }

    uri.resize(uriLength);

    try {
        return winrt::Windows::Foundation::Uri(uri);
    } catch (winrt::hresult_error const& ex) {
        Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
        return nullptr;
    }
}

// The time since a file was written.
ULONGLONG FileAgeMs(std::filesystem::file_time_type writeTime) {
    auto age = std::filesystem::file_time_type::clock::now() - writeTime;

    // A file stamped in the future, e.g. after a clock change, is brand new.
    if (age.count() <= 0) {
        return 0;
    }

    return std::chrono::duration_cast<std::chrono::milliseconds>(age).count();
}

// The time since a file was written, nullopt if there's no such file.
std::optional<ULONGLONG> FileAgeMs(const std::filesystem::path& path) {
    std::error_code ec;
    auto writeTime = std::filesystem::last_write_time(path, ec);
    if (ec) {
        return std::nullopt;
    }

    return FileAgeMs(writeTime);
}

void TouchFile(const std::filesystem::path& path) {
    std::error_code ec;
    std::filesystem::last_write_time(
        path, std::filesystem::file_time_type::clock::now(), ec);
}

// Removes the files which haven't been stamped for a long time, which is what
// becomes of a theme's images once it's out of use, and of what an interrupted
// download leaves behind.
void SweepImageCache() {
    const auto& cacheDir = GetImageCacheDir();
    if (cacheDir.empty()) {
        return;
    }

    Wh_Log(L"Sweeping the image cache");
```

```
    try {
        std::error_code ec;
        for (const auto& entry :
             std::filesystem::directory_iterator(cacheDir, ec)) {
            if (!entry.is_regular_file(ec)) {
                continue;
            }

            auto writeTime = entry.last_write_time(ec);
            if (ec || FileAgeMs(writeTime) < kImageCacheMaxUnusedMs) {
                continue;
            }

            Wh_Log(L"Removing unused cached image: %s",
                   entry.path().filename().c_str());
            std::filesystem::remove(entry.path(), ec);
        }
    } catch (const std::exception& ex) {
        Wh_Log(L"Error sweeping the image cache: %S", ex.what());
    }
}

// Via a temporary file, so that a partial or failed response, which the engine
// writes before the status code is known, never becomes the cached image.
void DownloadImage(const std::wstring& url) {
    auto cachePath = ImageCachePath(url);
    if (cachePath.empty()) {
        return;
    }

    auto tempPath = cachePath;
    tempPath += L".tmp" + std::to_wstring(GetCurrentProcessId());

    bool succeeded = false;

    WH_GET_URL_CONTENT_OPTIONS options{
        .optionsSize = sizeof(options),
        .targetFilePath = tempPath.c_str(),
    };

    if (const WH_URL_CONTENT* urlContent =
            Wh_GetUrlContent(url.c_str(), &options)) {
        if (urlContent->statusCode == 200) {
            succeeded = true;
        } else {
            Wh_Log(L"Wh_GetUrlContent returned %d", urlContent->statusCode);
        }

        Wh_FreeUrlContent(urlContent);
    } else {
        Wh_Log(L"Wh_GetUrlContent failed");
    }

    std::error_code ec;
```

```
    if (succeeded) {
        auto size = std::filesystem::file_size(tempPath, ec);
        if (ec || size == 0) {
            Wh_Log(L"No downloaded file to move into place");
            succeeded = false;
        }
    }

    if (succeeded) {
        std::filesystem::rename(tempPath, cachePath, ec);
        if (ec) {
            // Another process can be reading it.
            Wh_Log(L"Failed to move %s into place", tempPath.c_str());
        }
    }

    std::filesystem::remove(tempPath, ec);
}

void ProcessImageDownloads() {
    for (;;) {
        std::wstring url;

        {
            std::lock_guard<std::mutex> lock(g_imageDownloadMutex);

            if (g_imageDownloadStopping || g_imageDownloadQueue.empty()) {
                g_imageDownloadRunning = false;
                return;
            }

            url = std::move(g_imageDownloadQueue.front());
            g_imageDownloadQueue.pop_front();
        }

        if (url.empty()) {
            SweepImageCache();
            continue;
        }

        Wh_Log(L"Downloading image: %s", url.c_str());
        DownloadImage(url);

        std::lock_guard<std::mutex> lock(g_imageDownloadMutex);
        g_imageDownloadUrls.erase(url);
    }
}

// Must be called with g_imageDownloadMutex held, with the job it's for already
// queued.
void SubmitImageDownloadWork() {
    if (g_imageDownloadRunning) {
        return;
    }
```

```
   if (!g_imageDownloadWork) {
      g_imageDownloadWork =
         CreateThreadpoolWork([](PTP_CALLBACK_INSTANCE, PVOID,
                                 PTP_WORK) { ProcessImageDownloads(); },
                              nullptr, nullptr);
      if (!g_imageDownloadWork) {
         Wh_Log(L"Failed to create the image download work item");
         g_imageDownloadQueue.clear();
         g_imageDownloadUrls.clear();
         return;
      }
   }

   g_imageDownloadRunning = true;
   SubmitThreadpoolWork(g_imageDownloadWork);
}

void QueueImageDownload(const std::wstring& url) {
   std::lock_guard<std::mutex> lock(g_imageDownloadMutex);

   if (g_imageDownloadStopping) {
      return;
   }

   if (!g_imageDownloadUrls.insert(url).second) {
      return;
   }

   g_imageDownloadQueue.push_back(url);

   SubmitImageDownloadWork();
}

// Asks the download thread for a sweep, once per process. Not tied to there
// being anything to download, so that a cache which is fully up to date, and
// whose unused files nothing else removes, is swept too.
void QueueImageCacheSweep() {
   static std::once_flag once;
   std::call_once(once, []() {
      std::lock_guard<std::mutex> lock(g_imageDownloadMutex);

      if (g_imageDownloadStopping) {
         return;
      }

      g_imageDownloadQueue.emplace_back();

      SubmitImageDownloadWork();
   });
}

// Whether the cached file of a URL failed to load, which takes the images it's
// for back to the remote address.
bool IsImageCacheRejected(const std::wstring& url) {
   std::lock_guard<std::mutex> lock(g_imageDownloadMutex);
```

```
   return g_imageCacheRejectedUrls.contains(url);
}

void RejectImageCache(const std::wstring& url) {
   std::lock_guard<std::mutex> lock(g_imageDownloadMutex);

   g_imageCacheRejectedUrls.insert(url);
}

void StopImageDownloads() {
   PTP_WORK work;

   {
      std::lock_guard<std::mutex> lock(g_imageDownloadMutex);

      g_imageDownloadStopping = true;
      g_imageDownloadQueue.clear();
      g_imageDownloadUrls.clear();
      g_imageCacheRejectedUrls.clear();

      work = g_imageDownloadWork;
      g_imageDownloadWork = nullptr;
   }

   // A request which is in flight can't be cancelled, so a server which is
   // slow to answer holds up the unload for as long as it takes. Accepted as
   // it is: the alternative is letting the callback run on into a module which
   // is going away.
   if (work) {
      WaitForThreadpoolWorkCallbacks(work, TRUE);
      CloseThreadpoolWork(work);
   }
}

// The address an entry should load from: the cached file when there is one, the
// remote address otherwise. Asks for the download the answer implies.
winrt::Windows::Foundation::Uri ImageSourceUri(
   const std::shared_ptr<TrackedImage>& tracked) {
   if (!tracked->cachePath.empty() && !IsImageCacheRejected(tracked->url)) {
      if (auto age = FileAgeMs(tracked->cachePath)) {
         if (*age >= kImageCacheRefreshIntervalMs) {
            // Stamped whether or not the download gets through, so that an
            // offline machine doesn't lose the images it's using.
            TouchFile(tracked->cachePath);
            QueueImageDownload(tracked->url);
         }

         if (auto uri = ImageCacheFileUri(tracked->cachePath)) {
            return uri;
         }
      } else {
         QueueImageDownload(tracked->url);
      }
   }
```

```
    return tracked->uri;
}

// Takes the BitmapImage the style declared off a cached file which has been
// rejected, so that reapplying the value doesn't put the file which failed back
// on the target. Nothing is showing from that file, so this is the one case
// where a source is replaced without regard for what the target holds.
void RestoreRejectedImageSource(
    const std::shared_ptr<TrackedImage>& tracked,
    Media::Imaging::BitmapImage const& bitmapImage) {
    try {
        if (bitmapImage.UriSource().Equals(tracked->uri) ||
            !IsImageCacheRejected(tracked->url)) {
            return;
        }

        Wh_Log(L"Loading remote image for: %s", tracked->url.c_str());

        bitmapImage.UriSource(tracked->uri);
    } catch (winrt::hresult_error const& ex) {
        Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
    }
}

void StartImageRetry(const std::shared_ptr<TrackedImage>& tracked) {
    auto target = tracked->target.get();
    if (!target) {
        return;
    }

    Wh_Log(L"Retrying image load for: %s", tracked->url.c_str());

    tracked->lastRetryTick = GetTickCount64();
    tracked->retryCount++;

    // An Image element's source is a property the mod customizes, and writing
    // to it would otherwise be seen as an external change and reverted to the
    // failed source the style declared.
    bool wasModifying = g_elementPropertyModifying;
    g_elementPropertyModifying = true;

    try {
        // The cached file when a download has landed since the last attempt.
        auto uri = ImageSourceUri(tracked);
        tracked->usingCache = !uri.Equals(tracked->uri);

        Media::Imaging::BitmapImage retryImage;
        // Bypass the XAML image cache: a retry is only needed when what the
        // cache holds for the URI is a failed or missing image.
        retryImage.CreateOptions(
            tracked->createOptions |
            Media::Imaging::BitmapCreateOptions::IgnoreImageCache);
        retryImage.DecodePixelType(tracked->decodePixelType);
        retryImage.DecodePixelWidth(tracked->decodePixelWidth);
```

```
      retryImage.DecodePixelHeight(tracked->decodePixelHeight);
      retryImage.AutoPlay(tracked->autoPlay);

      // A BitmapImage is loaded by the framework as part of the tree it's
      // used in, so it has to be assigned to the target for anything to
      // happen. A new object rather than the failed one, since reassigning
      // the same URI to a BitmapImage doesn't reload it. The target's own
      // ImageOpened and ImageFailed report how this attempt went.
      target.SetValue(tracked->sourceProperty, retryImage);

      // The URI goes last: XAML decodes an image to the size it's displayed
      // at only when the BitmapImage is already connected to the live tree by
      // the time its source is set. Setting the URI first decodes at the
      // image's natural size, which is then scaled at render time and looks
      // poor.
      retryImage.UriSource(uri);
    } catch (winrt::hresult_error const& ex) {
      Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
    }

    g_elementPropertyModifying = wasModifying;
}

// The wait before the next attempt of an image which has been retried
// `retryCount` times.
ULONGLONG ImageRetryDelayMs(int retryCount) {
    return kImageRetryBaseDelayMs
        << std::clamp(retryCount - 1, 0, kImageRetryMaxBackoffShift);
}

void ScheduleImageLoadRetryOnCurrentThread(ULONGLONG delayMs, bool reschedule);

// Retries every image which is due, and arms the next round for the earliest
// image which isn't, so that a failed image recovers on its own instead of
// waiting for something external to start a round.
void RetryFailedImageLoadsOnCurrentThread() {
    if (!g_initializedForThread) {
      return;
    }

    Wh_Log(L"Retrying failed image loads on current thread");

    auto& images = g_trackedImagesForThread.images;

    std::erase_if(images,
                  [](const auto& tracked) { return !tracked->target.get(); });

    // Copy the entries before iterating: a retry can raise image events, and
    // their handlers modify the entries.
    std::vector<std::shared_ptr<TrackedImage>> snapshot(images.begin(),
                                                        images.end());

    ULONGLONG tick = GetTickCount64();

    ULONGLONG nextRoundDelay = 0;
```

```
    auto armNextRoundIn = [&nextRoundDelay](ULONGLONG delay) {
      if (!nextRoundDelay || delay < nextRoundDelay) {
        nextRoundDelay = delay;
      }
    };

    for (const auto& tracked : snapshot) {
      if (tracked->loaded) {
        continue;
      }

      ULONGLONG remaining = 0;

      if (tracked->lastRetryTick) {
        ULONGLONG sinceLastRetry = tick - tracked->lastRetryTick;
        if (sinceLastRetry >= kImageRetryMaxDelayMs) {
          tracked->retryCount = 0;
        } else {
          ULONGLONG delay = ImageRetryDelayMs(tracked->retryCount);
          if (sinceLastRetry < delay) {
            remaining = delay - sinceLastRetry;
          }
        }
      }

      // An image which ran out of attempts is left to a network status
      // change, which is what the count starting over is for.
      if (tracked->retryCount >= kImageRetryMaxCount) {
        continue;
      }

      if (remaining) {
        armNextRoundIn(remaining);
        continue;
      }

      StartImageRetry(tracked);

      if (tracked->retryCount < kImageRetryMaxCount) {
        armNextRoundIn(ImageRetryDelayMs(tracked->retryCount));
      }
    }

    if (nextRoundDelay) {
      ScheduleImageLoadRetryOnCurrentThread(nextRoundDelay,
                                            /*reschedule=*/false);
    }
}

// Runs a retry round in `delayMs`. A round which is already scheduled is kept
// if it's due sooner, unless `reschedule` moves it to the new time.
void ScheduleImageLoadRetryOnCurrentThread(ULONGLONG delayMs, bool reschedule) {
    if (!g_initializedForThread) {
      return;
    }
```

```
    auto& state = g_trackedImagesForThread;

    ULONGLONG dueTick = GetTickCount64() + delayMs;

    if (!reschedule && state.retryDueTick && state.retryDueTick <= dueTick) {
        return;
    }

    try {
        if (!state.retryTimer) {
            if (!state.dispatcher) {
                return;
            }

            state.retryTimer = state.dispatcher.CreateTimer();
            state.retryTimer.IsRepeating(false);
            state.retryTimerTickRevoker = state.retryTimer.Tick(
                winrt::auto_revoke,
                [](winrt::Windows::System::DispatcherQueueTimer const&,
                   winrt::Windows::Foundation::IInspectable const&) {
                    g_trackedImagesForThread.retryDueTick = 0;
                    RetryFailedImageLoadsOnCurrentThread();
                });
        }

        state.retryTimer.Stop();
        state.retryTimer.Interval(std::chrono::milliseconds{delayMs});
        state.retryTimer.Start();
        state.retryDueTick = dueTick;
    } catch (winrt::hresult_error const& ex) {
        Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
    }
}

// Counts a callback which is about to be handed to code outside the mod for as
// long as the returned reference is alive, so that StopImageLoadRetries waits
// for it whether it ends up running or being dropped. Null once the retries
// have been stopped, which is the caller's cue not to hand it over at all.
std::shared_ptr<void> TrackImageRetryCallback() {
    {
        std::lock_guard<std::mutex> lock(g_imageRetryMutex);

        if (!g_imageRetryActive) {
            return nullptr;
        }

        g_imageRetryPendingCallbacks++;
    }

    // The pointer is only a non-null tag; the deleter is what the reference is
    // for, and it runs whether the shared_ptr is destroyed or its construction
    // throws.
    return std::shared_ptr<void>(&g_imageRetryPendingCallbacks, [](void*) {
        {
```

```
            std::lock_guard<std::mutex> lock(g_imageRetryMutex);

            g_imageRetryPendingCallbacks--;
        }

        g_imageRetryPendingCallbacksCv.notify_all();
    });
}

void ScheduleImageLoadRetryOnAllUiThreads() {
    // Losing connectivity raises a network status event just like gaining it
    // does, and there's nothing to retry with no internet access.
    if (!HasInternetAccess()) {
        Wh_Log(L"No internet access, not retrying image loads");
        return;
    }

    std::vector<winrt::Windows::System::DispatcherQueue> dispatchers;
    {
        std::lock_guard<std::mutex> lock(g_imageRetryMutex);

        if (!g_imageRetryActive) {
            return;
        }

        for (auto& weakDispatcher : g_imageRetryDispatchers) {
            if (auto dispatcher = weakDispatcher.get()) {
                dispatchers.push_back(dispatcher);
            }
        }

        std::erase_if(g_imageRetryDispatchers, [](const auto& weakDispatcher) {
            return !weakDispatcher.get();
        });
    }

    for (auto& dispatcher : dispatchers) {
        auto callbackRef = TrackImageRetryCallback();
        if (!callbackRef) {
            return;
        }

        try {
            dispatcher.TryEnqueue([callbackRef]() {
                ScheduleImageLoadRetryOnCurrentThread(kNetworkChangeDebounceMs,
                                                      /*reschedule=*/true);
            });
        } catch (winrt::hresult_error const& ex) {
            Wh_Log(L"Error dispatching retry to UI thread %08X: %s", ex.code(),
                   ex.message().c_str());
        }
    }
}

void OnNetworkStatusChanged(
```

```
    winrt::Windows::Foundation::IInspectable const& sender) {
    Wh_Log(L">");

    // Removing the handler doesn't wait for an invocation which is already in
    // flight, so this one counts itself instead.
    auto callbackRef = TrackImageRetryCallback();
    if (!callbackRef) {
        return;
    }

    // Runs on a Windows Runtime thread pool thread, where the connectivity
    // query is allowed and doesn't hold up a UI thread.
    ScheduleImageLoadRetryOnAllUiThreads();
}

// Must not be called with g_imageRetryMutex held.
winrt::event_token RegisterNetworkStatusChangedHandler() {
    try {
        auto token = winrt::Windows::Networking::Connectivity::
            NetworkInformation::NetworkStatusChanged(OnNetworkStatusChanged);
        Wh_Log(L"Registered global network status change handler");
        return token;
    } catch (winrt::hresult_error const& ex) {
        Wh_Log(L"Error registering network status handler %08X: %s", ex.code(),
            ex.message().c_str());
        return {};
    }
}

// Must not be called with g_imageRetryMutex held.
void UnregisterNetworkStatusChangedHandler(winrt::event_token token) {
    try {
        winrt::Windows::Networking::Connectivity::NetworkInformation::
            NetworkStatusChanged(token);
        Wh_Log(L"Unregistered global network status change handler");
    } catch (winrt::hresult_error const& ex) {
        Wh_Log(L"Error unregistering network status handler %08X: %s",
            ex.code(), ex.message().c_str());
    }
}

void StopImageLoadRetries() {
    winrt::event_token token;

    {
        std::lock_guard<std::mutex> lock(g_imageRetryMutex);

        // Makes any handler which acquires the mutex from here on return
        // early, which is what stops the retries. Removing the handler only
        // stops further invocations.
        g_imageRetryActive = false;

        token = g_networkStatusChangedToken;
        g_networkStatusChangedToken = {};
```

```
    g_imageRetryDispatchers.clear();
  }

  if (token) {
    UnregisterNetworkStatusChangedHandler(token);
  }

  // The module is freed once the mod is uninitialized, so the callbacks which
  // are already on their way into it are let through first. What they wait on
  // is a connectivity query and a dispatcher pass of a UI thread, and the
  // uninitialization which follows depends on those threads running anyway.
  std::unique_lock<std::mutex> lock(g_imageRetryMutex);
  g_imageRetryPendingCallbacksCv.wait(
    lock, [] { return g_imageRetryPendingCallbacks == 0; });
}

// Drops the calling thread from the dispatcher registry, and stops the retries
// altogether once the last thread is out of it.
void StopImageLoadRetriesForCurrentThread() {
  auto dispatcher = g_trackedImagesForThread.dispatcher;
  if (!dispatcher) {
    return;
  }

  g_trackedImagesForThread.dispatcher = nullptr;

  winrt::event_token token;

  {
    std::lock_guard<std::mutex> lock(g_imageRetryMutex);

    std::erase_if(g_imageRetryDispatchers, [&dispatcher](
                                  const auto& weakDispatcher) {
      auto registeredDispatcher = weakDispatcher.get();
      return !registeredDispatcher || registeredDispatcher == dispatcher;
    });

    if (!g_imageRetryDispatchers.empty()) {
      return;
    }

    // What StopImageLoadRetries does, kept under the lock which found the
    // registry empty so that a thread which registers in between isn't
    // stopped as well.
    g_imageRetryActive = false;

    token = g_networkStatusChangedToken;
    g_networkStatusChangedToken = {};
  }

  if (token) {
    UnregisterNetworkStatusChangedHandler(token);
  }
}
```

```
void SetupImageTracking(DependencyObject const& target,
                        DependencyProperty const& sourceProperty,
                        Media::Imaging::BitmapImage const& bitmapImage,
                        winrt::Windows::Foundation::Uri const& uri) {
    auto& images = g_trackedImagesForThread.images;

    std::erase_if(images,
                  [](const auto& tracked) { return !tracked->target.get(); });

    auto it = std::find_if(images.begin(), images.end(),
                           [&target](const auto& tracked) {
                              if (auto trackedTarget = tracked->target.get()) {
                                 return trackedTarget == target;
                              }
                              return false;
                           });

    if (it != images.end()) {
       // Resolved style values are cached, so the same source object is
       // applied to many targets and reapplied on every visual state change.
       // Keep the load state which was collected so far unless the source
       // changed.
       if ((*it)->uri.Equals(uri)) {
          RestoreRejectedImageSource(*it, bitmapImage);

          // The value being applied takes over from whatever a retry has put
          // there, so it's what a load failure is judged by.
          (*it)->usingCache = !bitmapImage.UriSource().Equals(uri);
          return;
       }

       images.erase(it);
    }

    Wh_Log(L"Tracking %s with remote image source: %s",
           winrt::get_class_name(target).c_str(), uri.RawUri().c_str());

    auto tracked = std::make_shared<TrackedImage>();
    tracked->target = winrt::make_weak(target);
    tracked->sourceProperty = sourceProperty;
    tracked->uri = uri;
    tracked->url = std::wstring(uri.RawUri());
    tracked->cachePath = ImageCachePath(tracked->url);

    if (!tracked->cachePath.empty()) {
       QueueImageCacheSweep();
    }

    try {
       tracked->decodePixelWidth = bitmapImage.DecodePixelWidth();
       tracked->decodePixelHeight = bitmapImage.DecodePixelHeight();
       tracked->decodePixelType = bitmapImage.DecodePixelType();
       tracked->createOptions = bitmapImage.CreateOptions();
       tracked->autoPlay = bitmapImage.AutoPlay();
       // A load which completed before tracking started raises no further
```

```
   // event, so the decoded size is what tells an image that's there from
   // one that isn't. An image which is still loading counts as missing,
   // which at worst costs a redundant download.
   tracked->loaded = bitmapImage.PixelWidth() != 0;

   // The cached file when there is one, and a download asked for when
   // there isn't.
   auto sourceUri = ImageSourceUri(tracked);

   // Assigned to the BitmapImage the style declared rather than to a new
   // object, so that the value the element customization state knows stays
   // the one which is applied and nothing looks like an external change.
   // Only while the image isn't showing anything: swapping a source which
   // is would blank its targets for the length of a load, and the file is
   // there for the next process either way.
   if (!tracked->loaded && !sourceUri.Equals(bitmapImage.UriSource())) {
      bool fromCache = !sourceUri.Equals(uri);
      Wh_Log(L"Loading %s image for: %s",
          fromCache ? L"cached" : L"remote", tracked->url.c_str());

      // Recorded before the substitution, so that a source which was
      // pointed at a cached file is never one no entry can be recovered
      // from.
      if (fromCache) {
         g_imageCacheUriRemotes.insert_or_assign(
            std::wstring(sourceUri.RawUri()), uri);
      }

      bitmapImage.UriSource(sourceUri);
   }

   tracked->usingCache = !bitmapImage.UriSource().Equals(uri);
} catch (winrt::hresult_error const& ex) {
   Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
}

std::weak_ptr<TrackedImage> trackedWeak = tracked;

auto onImageFailed = [trackedWeak](
                  winrt::Windows::Foundation::IInspectable const&,
                  ExceptionRoutedEventArgs const& e) {
   auto tracked = trackedWeak.lock();
   if (!tracked) {
      return;
   }

   Wh_Log(L"Image load failed for: %s, error: %s", tracked->url.c_str(),
       e.ErrorMessage().c_str());

   tracked->loaded = false;

   // A cached file which doesn't decode is dropped, taking every image the
   // URL is for back to the remote address and fetching the file once
   // more, in case it was left incomplete, for the next process. The
   // declared source is pointed back at the remote address by the next
```

```
    // apply, rather than from here, so that a source isn't swapped from
    // within the event which reports how loading it went.
    if (tracked->usingCache) {
        Wh_Log(L"Dropping the cached image which failed to load");

        tracked->usingCache = false;
        RejectImageCache(tracked->url);

        std::error_code ec;
        std::filesystem::remove(tracked->cachePath, ec);

        QueueImageDownload(tracked->url);
    }

    // Waiting for the base delay coalesces the failures of a batch of
    // images into a single round, and keeps the retries off the burst of
    // requests the failures came from.
    ScheduleImageLoadRetryOnCurrentThread(kImageRetryBaseDelayMs,
                                          /*reschedule=*/false);
};

auto onImageOpened = [trackedWeak](
                         winrt::Windows::Foundation::IInspectable const&,
                         RoutedEventArgs const&) {
    auto tracked = trackedWeak.lock();
    if (!tracked) {
        return;
    }

    Wh_Log(L"Image loaded for: %s", tracked->url.c_str());

    tracked->loaded = true;
    tracked->retryCount = 0;
    tracked->lastRetryTick = 0;
};

if (auto brush = target.try_as<Media::ImageBrush>()) {
    tracked->brushImageFailedRevoker =
        brush.ImageFailed(winrt::auto_revoke, onImageFailed);
    tracked->brushImageOpenedRevoker =
        brush.ImageOpened(winrt::auto_revoke, onImageOpened);
} else if (auto image = target.try_as<Controls::Image>()) {
    tracked->elementImageFailedRevoker =
        image.ImageFailed(winrt::auto_revoke, onImageFailed);
    tracked->elementImageOpenedRevoker =
        image.ImageOpened(winrt::auto_revoke, onImageOpened);
}

images.push_back(std::move(tracked));

bool registerHandler = false;

{
    std::lock_guard<std::mutex> lock(g_imageRetryMutex);
```

```
      g_imageRetryActive = true;

      if (!g_trackedImagesForThread.dispatcher) {
        try {
          auto dispatcher = winrt::Windows::System::DispatcherQueue::
            GetForCurrentThread();
          if (dispatcher) {
            g_trackedImagesForThread.dispatcher = dispatcher;
            g_imageRetryDispatchers.push_back(
              winrt::make_weak(dispatcher));
            Wh_Log(
              L"Registered UI thread dispatcher for network retry");
          }
        } catch (winrt::hresult_error const& ex) {
          Wh_Log(L"Error getting dispatcher for current thread %08X: %s",
              ex.code(), ex.message().c_str());
        }
      }

      if (!g_networkStatusChangedToken &&
        !g_networkStatusChangedRegistering) {
        g_networkStatusChangedRegistering = true;
        registerHandler = true;
      }
    }

    if (!registerHandler) {
      return;
    }

    winrt::event_token token = RegisterNetworkStatusChangedHandler();

    bool stopped;

    {
      std::lock_guard<std::mutex> lock(g_imageRetryMutex);

      g_networkStatusChangedRegistering = false;

      stopped = !g_imageRetryActive;
      if (!stopped) {
        g_networkStatusChangedToken = token;
      }
    }

    // StopImageLoadRetries ran while the handler was being registered, so it
    // found no token to remove.
    if (stopped && token) {
      UnregisterNetworkStatusChangedHandler(token);
    }
}

// Tracks the target if the image source is a remote URL, which can fail to
// load and is worth caching.
void TrackIfRemoteImageSource(
```

```
    DependencyObject const& target,
    DependencyProperty const& sourceProperty,
    winrt::Windows::Foundation::IInspectable const& imageSource) {
    auto bitmapImage = imageSource.try_as<Media::Imaging::BitmapImage>();
    if (!bitmapImage) {
        return;
    }

    auto uri = bitmapImage.UriSource();
    if (!uri) {
        return;
    }

    auto scheme = uri.SchemeName();
    if (scheme == L"file") {
        // A cached file which was substituted for its remote address, by which
        // the entry is identified. A style value is shared by many targets, so
        // the substitution a previous target's entry made is what the rest of
        // them are given.
        auto it = g_imageCacheUriRemotes.find(std::wstring(uri.RawUri()));
        if (it == g_imageCacheUriRemotes.end()) {
            return;
        }

        uri = it->second;
    } else if (scheme != L"http" && scheme != L"https") {
        return;
    }

    SetupImageTracking(target, sourceProperty, bitmapImage, uri);
}

// Returns the value written, UnsetValue for a clear, so that callers can
// record what they applied. Reading it back is not an option: while a visual
// state has a setter on the property, ReadLocalValue keeps reporting the value
// from before that state rather than the one written.
winrt::Windows::Foundation::IInspectable SetOrClearValue(
    DependencyObject elementDo,
    DependencyProperty property,
    const PropertyOverrideValue& overrideValue,
    bool initialApply = false) {
    winrt::Windows::Foundation::IInspectable value;
    if (auto* inspectable =
            std::get_if<winrt::Windows::Foundation::IInspectable>(
                &overrideValue)) {
        value = *inspectable;
    } else if (auto* blurBrushParams =
                   std::get_if<XamlBlurBrushParams>(&overrideValue)) {
        if (auto uiElement = elementDo.try_as<UIElement>()) {
            value = winrt::make<XamlBlurBrush>(
                uiElement, blurBrushParams->blurAmount, blurBrushParams->tint,
                blurBrushParams->tintOpacity,
                winrt::hstring(blurBrushParams->tintThemeResourceKey),
                blurBrushParams->tintLuminosityOpacity,
                blurBrushParams->tintSaturation, blurBrushParams->noiseOpacity,
```

```
            blurBrushParams->noiseDensity, blurBrushParams->fallbackColor,
            winrt::hstring(blurBrushParams->fallbackThemeResourceKey));
    } else {
        Wh_Log(L"Can't get UIElement for blur brush");
        return nullptr;
    }
} else {
    Wh_Log(L"Unsupported override value");
    return nullptr;
}

// If customized before
// `winrt::Taskbar::implementation::TaskbarBackground::OnApplyTemplate` is
// executed, it can lead to a crash, or the customization may be overridden.
// See:
// https://github.com/ramensoftware/windows-11-taskbar-styling-guide/issues/4
if (winrt::get_class_name(elementDo) ==
        L"Windows.UI.Xaml.Shapes.Rectangle" &&
    elementDo.as<FrameworkElement>().Name() == L"BackgroundFill" &&
    property == Shapes::Shape::FillProperty()) {
    auto it = std::find_if(g_delayedBackgroundFillSet.begin(),
                           g_delayedBackgroundFillSet.end(),
                           [&elementDo](const auto& it) {
                               if (auto elementDoIter = it.first.get()) {
                                   return elementDoIter == elementDo;
                               }
                               return false;
                           });

    if (value != DependencyProperty::UnsetValue() && initialApply &&
        it == g_delayedBackgroundFillSet.end()) {
        Wh_Log(L"Delaying SetValue for BackgroundFill");
        auto asyncOp = elementDo.Dispatcher().TryRunAsync(
            winrt::Windows::UI::Core::CoreDispatcherPriority::High,
            [elementDo, property, value]() {
                Wh_Log(L"Running delayed SetValue for BackgroundFill");
                g_elementPropertyModifying = true;
                try {
                    elementDo.SetValue(property, value);
                } catch (winrt::hresult_error const& ex) {
                    Wh_Log(L"Error %08X: %s", ex.code(),
                           ex.message().c_str());
                }
                g_elementPropertyModifying = false;
                std::erase_if(g_delayedBackgroundFillSet,
                              [&elementDo](const auto& it) {
                                  if (auto elementDoIter = it.first.get()) {
                                      return elementDoIter == elementDo;
                                  }
                                  return false;
                              });
            });
        g_delayedBackgroundFillSet.emplace_back(elementDo,
                                                std::move(asyncOp));
        return value;
```

```
    } else if (it != g_delayedBackgroundFillSet.end()) {
        Wh_Log(L"Canceling delayed SetValue for BackgroundFill");
        it->second.Cancel();
        g_delayedBackgroundFillSet.erase(it);
    }
}

if (value == DependencyProperty::UnsetValue()) {
    Wh_Log(L"Clearing property value");
    try {
        elementDo.ClearValue(property);
    } catch (winrt::hresult_error const& ex) {
        Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
    }
    return value;
}

Wh_Log(L"Setting property value %s",
       value ? winrt::get_class_name(value).c_str() : L"(null)");

// Track a remote image source for retry on network reconnection. A style
// can declare one as the ImageBrush a property is set to (e.g. Background),
// as the ImageSource of an ImageBrush it targets, or as the Source of an
// Image element.
if (auto imageBrush = value.try_as<Media::ImageBrush>()) {
    TrackIfRemoteImageSource(imageBrush,
                             Media::ImageBrush::ImageSourceProperty(),
                             imageBrush.ImageSource());
} else if (auto imageBrush = elementDo.try_as<Media::ImageBrush>()) {
    if (property == Media::ImageBrush::ImageSourceProperty()) {
        TrackIfRemoteImageSource(imageBrush, property, value);
    }
} else if (auto image = elementDo.try_as<Controls::Image>()) {
    if (property == Controls::Image::SourceProperty()) {
        TrackIfRemoteImageSource(image, property, value);
    }
}

// This might fail. See `ReadLocalValueWithWorkaround` for an example (which
// we now handle but there might be other cases).
try {
    // `setter.Value()` returns font weight as an int. Using it with
    // `SetValue` results in the following error: 0x80004002 (No such
    // interface supported). Box it as `Windows.UI.Text.FontWeight` as a
    // workaround.
    if (property == Controls::TextBlock::FontWeightProperty() ||
        property == Controls::Control::FontWeightProperty() ||
        property == Controls::RichTextBlock::FontWeightProperty() ||
        property == Controls::FontIcon::FontWeightProperty() ||
        property == Controls::FontIconSource::FontWeightProperty() ||
        property == Controls::ContentPresenter::FontWeightProperty()) {
        auto valueInt = value.try_as<int>();
        if (valueInt && *valueInt >= std::numeric_limits<uint16_t>::min() &&
            *valueInt <= std::numeric_limits<uint16_t>::max()) {
            value = winrt::box_value(winrt::Windows::UI::Text::FontWeight{
```

```
                static_cast<uint16_t>(*valueInt)});
            }
        }

        // Grid ColumnDefinitions/RowDefinitions hold DependencyObjects
        // (ColumnDefinition/RowDefinition) that the layout engine writes
        // ActualWidth/ActualHeight back into. The resolved value is parsed once
        // and cached, so applying it to more than one grid - e.g. a taskbar per
        // monitor, all sharing one UI thread - would set the same collection on
        // each, and one monitor's column sizes would then leak onto another's.
        // Give each element a private copy. The scratch Grid owns the fresh
        // collection until SetValue reassigns ownership to the target, so it's
        // kept alive through the SetValue call below.
        Controls::Grid definitionsCloneOwner{nullptr};
        if (auto sourceColumns =
                value.try_as<Controls::ColumnDefinitionCollection>()) {
            definitionsCloneOwner = Controls::Grid{};
            auto clonedColumns = definitionsCloneOwner.ColumnDefinitions();
            for (auto const& column : sourceColumns) {
                Controls::ColumnDefinition clonedColumn;
                clonedColumn.Width(column.Width());
                clonedColumn.MinWidth(column.MinWidth());
                clonedColumn.MaxWidth(column.MaxWidth());
                clonedColumns.Append(clonedColumn);
            }
            value = clonedColumns;
        } else if (auto sourceRows =
                    value.try_as<Controls::RowDefinitionCollection>()) {
            definitionsCloneOwner = Controls::Grid{};
            auto clonedRows = definitionsCloneOwner.RowDefinitions();
            for (auto const& row : sourceRows) {
                Controls::RowDefinition clonedRow;
                clonedRow.Height(row.Height());
                clonedRow.MinHeight(row.MinHeight());
                clonedRow.MaxHeight(row.MaxHeight());
                clonedRows.Append(clonedRow);
            }
            value = clonedRows;
        }

        elementDo.SetValue(property, value);
    } catch (winrt::hresult_error const& ex) {
        Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
    }

    return value;
}

// https://stackoverflow.com/a/5665377
std::wstring EscapeXmlAttribute(std::wstring_view data) {
    std::wstring buffer;
    buffer.reserve(data.size());
    for (const auto c : data) {
        switch (c) {
            case '&':
```

```
            buffer.append(L"&amp;");
            break;
          case '\"':
            buffer.append(L"&quot;");
            break;
          // case '\'':
          //     buffer.append(L"&apos;");
          //     break;
          case '<':
            buffer.append(L"&lt;");
            break;
          case '>':
            buffer.append(L"&gt;");
            break;
          default:
            buffer.push_back(c);
            break;
        }
    }

    return buffer;
}

// https://stackoverflow.com/a/54364173
std::wstring_view TrimStringView(std::wstring_view s) {
    s.remove_prefix(std::min(s.find_first_not_of(L" \t\r\v\n"), s.size()));
    s.remove_suffix(
        std::min(s.size() - s.find_last_not_of(L" \t\r\v\n") - 1, s.size()));
    return s;
}

// https://stackoverflow.com/a/46931770
std::vector<std::wstring_view> SplitStringView(std::wstring_view s,
                                               std::wstring_view delimiter) {
    size_t pos_start = 0, pos_end, delim_len = delimiter.length();
    std::wstring_view token;
    std::vector<std::wstring_view> res;

    while ((pos_end = s.find(delimiter, pos_start)) !=
           std::wstring_view::npos) {
        token = s.substr(pos_start, pos_end - pos_start);
        pos_start = pos_end + delim_len;
        res.push_back(token);
    }

    res.push_back(s.substr(pos_start));
    return res;
}

std::optional<PropertyOverrideValue> ParseNonXamlPropertyOverrideValue(
    std::wstring_view stringValue) {
    // Example:
    // <WindhawkBlur BlurAmount="10" TintColor="#FFFF0000"/>

    auto substr = TrimStringView(stringValue);
```

```
constexpr auto kWindhawkBlurPrefix = L"<WindhawkBlur "sv;
if (!substr.starts_with(kWindhawkBlurPrefix)) {
    return std::nullopt;
}
Wh_Log(L"%.*s", static_cast<int>(substr.length()), substr.data());
substr = substr.substr(std::size(kWindhawkBlurPrefix));

constexpr auto kWindhawkBlurSuffix = L"/>"sv;
if (!substr.ends_with(kWindhawkBlurSuffix)) {
    throw std::runtime_error("WindhawkBlur: Bad suffix");
}
substr = substr.substr(0, substr.size() - std::size(kWindhawkBlurSuffix));

bool pendingTintColorThemeResource = false;
bool pendingFallbackColorThemeResource = false;
std::wstring tintThemeResourceKey;
std::wstring fallbackThemeResourceKey;
winrt::Windows::UI::Color tint{};
std::optional<winrt::Windows::UI::Color> fallbackColor;
float tintOpacity = std::numeric_limits<float>::quiet_NaN();
float tintLuminosityOpacity = std::numeric_limits<float>::quiet_NaN();
float tintSaturation = std::numeric_limits<float>::quiet_NaN();
float noiseOpacity = std::numeric_limits<float>::quiet_NaN();
float noiseDensity = std::numeric_limits<float>::quiet_NaN();
float blurAmount = 0;

constexpr auto kTintColorThemeResourcePrefix =
    L"TintColor=\"{ThemeResource"sv;
constexpr auto kTintColorThemeResourceSuffix = L"}\""sv;
constexpr auto kTintColorPrefix = L"TintColor=\"#"sv;
constexpr auto kTintOpacityPrefix = L"TintOpacity=\""sv;
constexpr auto kTintLuminosityOpacityPrefix = L"TintLuminosityOpacity=\""sv;
constexpr auto kTintSaturationPrefix = L"TintSaturation=\""sv;
constexpr auto kNoiseOpacityPrefix = L"NoiseOpacity=\""sv;
constexpr auto kNoiseDensityPrefix = L"NoiseDensity=\""sv;
constexpr auto kBlurAmountPrefix = L"BlurAmount=\""sv;
constexpr auto kFallbackColorThemeResourcePrefix =
    L"FallbackColor=\"{ThemeResource"sv;
constexpr auto kFallbackColorThemeResourceSuffix = L"}\""sv;
constexpr auto kFallbackColorPrefix = L"FallbackColor=\"#"sv;
for (const auto prop : SplitStringView(substr, L" ")) {
    const auto propSubstr = TrimStringView(prop);
    if (propSubstr.empty()) {
        continue;
    }

    Wh_Log(L"  %.*s", static_cast<int>(propSubstr.length()),
           propSubstr.data());

    if (pendingTintColorThemeResource) {
        if (!propSubstr.ends_with(kTintColorThemeResourceSuffix)) {
            throw std::runtime_error(
                "WindhawkBlur: Invalid TintColor theme resource syntax");
        }
```

```
    pendingTintColorThemeResource = false;

    tintThemeResourceKey = propSubstr.substr(
        0,
        propSubstr.size() - std::size(kTintColorThemeResourceSuffix));

    continue;
}

if (pendingFallbackColorThemeResource) {
    if (!propSubstr.ends_with(kFallbackColorThemeResourceSuffix)) {
        throw std::runtime_error(
            "WindhawkBlur: Invalid FallbackColor theme resource "
            "syntax");
    }

    pendingFallbackColorThemeResource = false;

    fallbackThemeResourceKey = propSubstr.substr(
        0, propSubstr.size() -
               std::size(kFallbackColorThemeResourceSuffix));

    continue;
}

if (propSubstr == kTintColorThemeResourcePrefix) {
    pendingTintColorThemeResource = true;
    continue;
}

if (propSubstr == kFallbackColorThemeResourcePrefix) {
    pendingFallbackColorThemeResource = true;
    continue;
}

if (propSubstr.starts_with(kTintColorPrefix) &&
    propSubstr.back() == L'\"') {
    auto valStr = propSubstr.substr(
        std::size(kTintColorPrefix),
        propSubstr.size() - std::size(kTintColorPrefix) - 1);

    bool hasAlpha;
    switch (valStr.size()) {
        case 6:
            hasAlpha = false;
            break;
        case 8:
            hasAlpha = true;
            break;
        default:
            throw std::runtime_error(
                "WindhawkBlur: Unsupported TintColor value");
    }
```

```
    auto valNum = std::stoul(std::wstring(valStr), nullptr, 16);
    uint8_t a = hasAlpha ? HIBYTE(HIWORD(valNum)) : 255;
    uint8_t r = LOBYTE(HIWORD(valNum));
    uint8_t g = HIBYTE(LOWORD(valNum));
    uint8_t b = LOBYTE(LOWORD(valNum));
    tint = {a, r, g, b};
    continue;
}

if (propSubstr.starts_with(kFallbackColorPrefix) &&
    propSubstr.back() == L'\"') {
    auto valStr = propSubstr.substr(
        std::size(kFallbackColorPrefix),
        propSubstr.size() - std::size(kFallbackColorPrefix) - 1);

    bool hasAlpha;
    switch (valStr.size()) {
        case 6:
            hasAlpha = false;
            break;
        case 8:
            hasAlpha = true;
            break;
        default:
            throw std::runtime_error(
                "WindhawkBlur: Unsupported FallbackColor value");
    }

    auto valNum = std::stoul(std::wstring(valStr), nullptr, 16);
    uint8_t a = hasAlpha ? HIBYTE(HIWORD(valNum)) : 255;
    uint8_t r = LOBYTE(HIWORD(valNum));
    uint8_t g = HIBYTE(LOWORD(valNum));
    uint8_t b = LOBYTE(LOWORD(valNum));
    fallbackColor = winrt::Windows::UI::Color{a, r, g, b};
    continue;
}

if (propSubstr.starts_with(kTintOpacityPrefix) &&
    propSubstr.back() == L'\"') {
    auto valStr = propSubstr.substr(
        std::size(kTintOpacityPrefix),
        propSubstr.size() - std::size(kTintOpacityPrefix) - 1);
    tintOpacity = std::stof(std::wstring(valStr));
    continue;
}

if (propSubstr.starts_with(kTintLuminosityOpacityPrefix) &&
    propSubstr.back() == L'\"') {
    auto valStr = propSubstr.substr(
        std::size(kTintLuminosityOpacityPrefix),
        propSubstr.size() - std::size(kTintLuminosityOpacityPrefix) -
            1);
    tintLuminosityOpacity = std::stof(std::wstring(valStr));
    continue;
}
```

```
    if (propSubstr.starts_with(kTintSaturationPrefix) &&
        propSubstr.back() == L'\"') {
        auto valStr = propSubstr.substr(
            std::size(kTintSaturationPrefix),
            propSubstr.size() - std::size(kTintSaturationPrefix) - 1);
        tintSaturation = std::stof(std::wstring(valStr));
        continue;
    }

    if (propSubstr.starts_with(kNoiseOpacityPrefix) &&
        propSubstr.back() == L'\"') {
        auto valStr = propSubstr.substr(
            std::size(kNoiseOpacityPrefix),
            propSubstr.size() - std::size(kNoiseOpacityPrefix) - 1);
        noiseOpacity = std::stof(std::wstring(valStr));
        continue;
    }

    if (propSubstr.starts_with(kNoiseDensityPrefix) &&
        propSubstr.back() == L'\"') {
        auto valStr = propSubstr.substr(
            std::size(kNoiseDensityPrefix),
            propSubstr.size() - std::size(kNoiseDensityPrefix) - 1);
        noiseDensity = std::stof(std::wstring(valStr));
        continue;
    }

    if (propSubstr.starts_with(kBlurAmountPrefix) &&
        propSubstr.back() == L'\"') {
        auto valStr = propSubstr.substr(
            std::size(kBlurAmountPrefix),
            propSubstr.size() - std::size(kBlurAmountPrefix) - 1);
        blurAmount = std::stof(std::wstring(valStr));
        continue;
    }

    throw std::runtime_error("WindhawkBlur: Bad property");
}

if (pendingTintColorThemeResource) {
    throw std::runtime_error(
        "WindhawkBlur: Unterminated TintColor theme resource");
}

if (pendingFallbackColorThemeResource) {
    throw std::runtime_error(
        "WindhawkBlur: Unterminated FallbackColor theme resource");
}

if (!std::isnan(tintOpacity)) {
    if (tintOpacity < 0.0f) {
        tintOpacity = 0.0f;
    } else if (tintOpacity > 1.0f) {
        tintOpacity = 1.0f;
```

```
        }

        tint.A = static_cast<uint8_t>(tintOpacity * 255.0f);
    }

    return XamlBlurBrushParams{
        .blurAmount = blurAmount,
        .tint = tint,
        .tintOpacity =
            !std::isnan(tintOpacity) ? std::optional(tint.A) : std::nullopt,
        .tintThemeResourceKey = std::move(tintThemeResourceKey),
        .tintLuminosityOpacity = !std::isnan(tintLuminosityOpacity)
                                     ? std::optional(tintLuminosityOpacity)
                                     : std::nullopt,
        .tintSaturation = !std::isnan(tintSaturation)
                              ? std::optional(tintSaturation)
                              : std::nullopt,
        .noiseOpacity = !std::isnan(noiseOpacity) ? std::optional(noiseOpacity)
                                                  : std::nullopt,
        .noiseDensity = !std::isnan(noiseDensity) ? std::optional(noiseDensity)
                                                  : std::nullopt,
        .fallbackColor = fallbackColor,
        .fallbackThemeResourceKey = std::move(fallbackThemeResourceKey),
    };
}

Style GetStyleFromXamlSetters(const std::wstring_view type,
                              const std::wstring_view xamlStyleSetters) {
    std::wstring xaml =
        LR"(<ResourceDictionary
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    xmlns:muxc="using:Microsoft.UI.Xaml.Controls")";

    if (auto pos = type.rfind('.'); pos != type.npos) {
        auto typeNamespace = std::wstring_view(type).substr(0, pos);
        auto typeName = std::wstring_view(type).substr(pos + 1);

        xaml += L"\n    xmlns:windhawkstyler=\"using:";
        xaml += EscapeXmlAttribute(typeNamespace);
        xaml +=
            L"\">\n"
            L"    <Style TargetType=\"windhawkstyler:";
        xaml += EscapeXmlAttribute(typeName);
        xaml += L"\">\n";
    } else {
        xaml +=
            L">\n"
            L"    <Style TargetType=\"";
        xaml += EscapeXmlAttribute(type);
        xaml += L"\">\n";
    }
```

```
    xaml += xamlStyleSetters;

    xaml +=
        L"    </Style>\n"
        L"</ResourceDictionary>";

    Wh_Log(L"======================================== XAML:");
    std::wstringstream ss(xaml);
    std::wstring line;
    while (std::getline(ss, line, L'\n')) {
        Wh_Log(L"%s", line.c_str());
    }
    Wh_Log(L"========================================");

    auto resourceDictionary =
        Markup::XamlReader::Load(xaml).as<ResourceDictionary>();

    auto [styleKey, styleInspectable] = resourceDictionary.First().Current();
    return styleInspectable.as<Style>();
}

Style GetStyleFromXamlSettersWithFallbackType(
    const std::wstring_view type,
    const std::wstring_view fallbackType,
    const std::wstring_view xamlStyleSetters) {
    try {
        return GetStyleFromXamlSetters(type, xamlStyleSetters);
    } catch (winrt::hresult_error const& ex) {
        constexpr HRESULT kStowedException = 0x802B000A;
        if (ex.code() != kStowedException || fallbackType.empty() ||
            fallbackType == type) {
            throw;
        }

        // For some types such as JumpViewUI.JumpListListViewItem, the following
        // error is returned:
        //
        // Error 802B000A: Failed to create a 'System.Type' from the text
        // 'windhawkstyler:JumpListListViewItem'. [Line: 8 Position: 12]
        //
        // Retry with a fallback type, which will allow to at least use the
        // basic properties.
        Wh_Log(L"Retrying with fallback type type due to error %08X: %s",
            ex.code(), ex.message().c_str());
        return GetStyleFromXamlSetters(fallbackType, xamlStyleSetters);
    }
}

const ResolvedRules& GetResolvedPropertyOverrides(
    const std::wstring_view type,
    const std::wstring_view fallbackType,
    PropertyOverridesMaybeUnresolved* propertyOverridesMaybeUnresolved) {
    if (const auto* resolved =
            std::get_if<ResolvedRules>(propertyOverridesMaybeUnresolved)) {
        return *resolved;
```

```
}

ResolvedRules resolved;

try {
   const auto& unresolved =
      std::get<UnresolvedRules>(*propertyOverridesMaybeUnresolved);
   const auto& valueRules = unresolved.valueRules;
   const auto& captureRules = unresolved.captureRules;

   if (!valueRules.empty() || !captureRules.empty()) {
      // Build a single XAML <Style> with one <Setter> per rule. Setters
      // for value rules come first, followed by one per capture rule.
      // Dynamic / capture rules emit a placeholder `{x:Null}` value -- we
      // only need the resolved DependencyProperty from those setters; the
      // value is computed elsewhere (per apply for dynamic, never for
      // captures).
      std::wstring xaml;

      std::vector<std::optional<PropertyOverrideValue>>
         propertyOverrideValues;
      propertyOverrideValues.reserve(valueRules.size());

      for (const auto& rule : valueRules) {
         const bool isDynamic = rule.isDynamic();

         propertyOverrideValues.push_back(
            !isDynamic && rule.isXamlValue
               ? ParseNonXamlPropertyOverrideValue(rule.value)
               : std::nullopt);

         xaml += L"       <Setter Property=\"";
         xaml += EscapeXmlAttribute(rule.propertyName);
         xaml += L"\"";
         if (isDynamic || propertyOverrideValues.back() ||
            (rule.isXamlValue && rule.value.empty())) {
            xaml += L" Value=\"{x:Null}\" />\n";
         } else if (!rule.isXamlValue) {
            xaml += L" Value=\"";
            xaml += EscapeXmlAttribute(rule.value);
            xaml += L"\" />\n";
         } else {
            xaml +=
               L">\n"
               L"          <Setter.Value>\n";
            xaml += rule.value;
            xaml +=
               L"\n"
               L"          </Setter.Value>\n"
               L"       </Setter>\n";
         }
      }

      for (const auto& rule : captureRules) {
         xaml += L"       <Setter Property=\"";
```

```
            xaml += EscapeXmlAttribute(rule.propertyName);
            xaml += L"\" Value=\"{x:Null}\" />\n";
          }

          auto style = GetStyleFromXamlSettersWithFallbackType(
            type, fallbackType, xaml);

          uint32_t setterIndex = 0;
          for (size_t i = 0; i < valueRules.size(); i++, setterIndex++) {
            const auto& rule = valueRules[i];
            const auto setter =
              style.Setters().GetAt(setterIndex).as<Setter>();
            auto property = setter.Property();
            if (rule.isDynamic()) {
              resolved.propertyOverrides[property][rule.visualState] =
                DynamicStyleTemplate{rule.propertyName, rule.value,
                                     rule.isXamlValue};
              resolved.hasDynamicValues = true;
            } else {
              resolved.propertyOverrides[property][rule.visualState] =
                propertyOverrideValues[i].value_or(
                  rule.isXamlValue && rule.value.empty()
                    ? DependencyProperty::UnsetValue()
                    : setter.Value());
            }
          }

          for (const auto& rule : captureRules) {
            const auto setter =
              style.Setters().GetAt(setterIndex++).as<Setter>();
            resolved.captures.push_back({setter.Property(), rule.varName});
          }
        }

        Wh_Log(L"%.*s: %zu override styles, %zu captures",
            static_cast<int>(type.length()), type.data(),
            resolved.propertyOverrides.size(), resolved.captures.size());
      } catch (winrt::hresult_error const& ex) {
        Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
      } catch (std::exception const& ex) {
        Wh_Log(L"Error: %S", ex.what());
      }

      *propertyOverridesMaybeUnresolved = std::move(resolved);
      return std::get<ResolvedRules>(*propertyOverridesMaybeUnresolved);
    }

    // Resolve a single style rule's expanded textual value into a usable
    // PropertyOverrideValue. Built for re-resolving dynamic `{{...}}` styles on
    // every variable change; falls back to the same XAML-Setter parse trick used by
    // the bulk resolver above. propertyName is the property whose XAML name should
    // appear on the synthetic Setter (already known at apply time).
    std::optional<PropertyOverrideValue> ResolveExpandedSinglePropertyValue(
      std::wstring_view type,
      std::wstring_view fallbackType,
```

```
    std::wstring_view propertyName,
    std::wstring_view expandedValue,
    bool isXamlValue) {
    if (isXamlValue) {
        if (auto blur = ParseNonXamlPropertyOverrideValue(expandedValue)) {
            return *blur;
        }

        if (TrimStringView(expandedValue).empty()) {
            return PropertyOverrideValue{DependencyProperty::UnsetValue()};
        }
    }

    std::wstring xaml = L"       <Setter Property=\"";
    xaml += EscapeXmlAttribute(propertyName);
    xaml += L"\"";
    if (!isXamlValue) {
        xaml += L" Value=\"";
        xaml += EscapeXmlAttribute(expandedValue);
        xaml += L"\" />\n";
    } else {
        xaml +=
            L">\n"
            L"           <Setter.Value>\n";
        xaml += expandedValue;
        xaml +=
            L"\n"
            L"           </Setter.Value>\n"
            L"       </Setter>\n";
    }

    try {
        auto style =
            GetStyleFromXamlSettersWithFallbackType(type, fallbackType, xaml);
        const auto setter = style.Setters().GetAt(0).as<Setter>();
        return PropertyOverrideValue{setter.Value()};
    } catch (winrt::hresult_error const& ex) {
        Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
    } catch (std::exception const& ex) {
        Wh_Log(L"Error: %S", ex.what());
    }

    return std::nullopt;
}

const PropertyValues& GetResolvedPropertyValues(
    const std::wstring_view type,
    const std::wstring_view fallbackType,
    PropertyValuesMaybeUnresolved* propertyValuesMaybeUnresolved) {
    if (const auto* resolved =
            std::get_if<PropertyValues>(propertyValuesMaybeUnresolved)) {
        return *resolved;
    }

    PropertyValues propertyValues;
```

```
    try {
        const auto& propertyValuesStr =
            std::get<PropertyValuesUnresolved>(*propertyValuesMaybeUnresolved);
        if (!propertyValuesStr.empty()) {
            std::wstring xaml;

            for (const auto& [property, value] : propertyValuesStr) {
                xaml += L"       <Setter Property=\"";
                xaml += EscapeXmlAttribute(property);
                xaml += L"\" Value=\"";
                xaml += EscapeXmlAttribute(value);
                xaml += L"\" />\n";
            }

            auto style = GetStyleFromXamlSettersWithFallbackType(
                type, fallbackType, xaml);

            for (size_t i = 0; i < propertyValuesStr.size(); i++) {
                const auto setter = style.Setters().GetAt(i).as<Setter>();
                propertyValues.push_back({
                    setter.Property(),
                    setter.Value(),
                });
            }
        }

        Wh_Log(L"%.*s: %zu matcher styles", static_cast<int>(type.length()),
            type.data(), propertyValues.size());
    } catch (winrt::hresult_error const& ex) {
        Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
    } catch (std::exception const& ex) {
        Wh_Log(L"Error: %S", ex.what());
    }

    *propertyValuesMaybeUnresolved = std::move(propertyValues);
    return std::get<PropertyValues>(*propertyValuesMaybeUnresolved);
}

// https://stackoverflow.com/a/12835139
VisualStateGroup GetVisualStateGroup(FrameworkElement element,
                                     std::wstring_view visualStateGroupName) {
    // The TaskListButtonPanel child element of the search box (with "Icon and
    // label" configuration) returns a list of size 1, but accessing the first
    // item leads to a null dereference crash. Skip this element.
    if (winrt::get_class_name(element) == L"Taskbar.TaskListButtonPanel") {
        auto parent = Media::VisualTreeHelper::GetParent(element)
                          .try_as<FrameworkElement>();
        if (parent && winrt::get_class_name(parent) ==
                          L"Taskbar.SearchBoxLaunchListButton") {
            return nullptr;
        }
    }

    // Same as above for an updated element layout (around Jun 2025).
```

```
    if (winrt::get_class_name(element) ==
        L"SearchUx.SearchUI.SearchButtonRootGrid") {
        auto parent = Media::VisualTreeHelper::GetParent(element)
                          .try_as<FrameworkElement>();
        if (parent && winrt::get_class_name(parent) ==
                          L"SearchUx.SearchUI.SearchPillButton") {
            return nullptr;
        }
    }

    auto list = VisualStateManager::GetVisualStateGroups(element);

    for (const auto& v : list) {
        if (v.Name() == visualStateGroupName) {
            return v;
        }
    }

    return nullptr;
}

// Locale-independent double formatter. Uses `std::to_chars` shortest round-trip
// representation so XAML always sees `.` as the decimal separator. Non-finite
// values use the spellings the XAML double converter accepts (case-sensitive;
// it rejects "-NaN").
std::wstring FormatDoubleInvariant(double d) {
    if (std::isnan(d)) {
        return L"NaN";
    }
    if (std::isinf(d)) {
        return d < 0 ? L"-Infinity" : L"Infinity";
    }
    char buf[64];
    auto [end, ec] = std::to_chars(buf, buf + std::size(buf), d);
    if (ec != std::errc{}) {
        return L"0";
    }
    return std::wstring(buf, end);
}

// Locale-independent double parser. Accepts an optional leading sign followed
// by a decimal fraction or exponent. Returns std::nullopt on partial / bad
// input.
std::optional<double> ParseDoubleInvariant(std::wstring_view sv) {
    std::string narrow;
    narrow.reserve(sv.size());
    for (auto c : sv) {
        if (c > 127) {
            return std::nullopt;
        }
        narrow.push_back(static_cast<char>(c));
    }
    double result = 0;
    auto* first = narrow.data();
    auto* last = first + narrow.size();
```

```
    auto [ptr, ec] = std::from_chars(first, last, result);
    if (ec != std::errc{} || ptr != last) {
        return std::nullopt;
    }
    return result;
}

using UnboxedPropertyValue = std::variant<std::wstring,
                                          bool,
                                          char16_t,
                                          uint8_t,
                                          int16_t,
                                          uint16_t,
                                          int32_t,
                                          uint32_t,
                                          int64_t,
                                          uint64_t,
                                          float,
                                          double>;

// Unwraps a boxed primitive into a typed primitive variant. Dispatches on
// IPropertyValue::Type(). Returns std::nullopt for non-primitive (opaque)
// values such as brushes or thicknesses.
std::optional<UnboxedPropertyValue> TryUnboxPropertyValue(
    winrt::Windows::Foundation::IInspectable const& value) {
    using winrt::Windows::Foundation::IPropertyValue;
    using winrt::Windows::Foundation::PropertyType;

    auto pv = value.try_as<IPropertyValue>();
    if (!pv) {
        return std::nullopt;
    }

    switch (pv.Type()) {
        case PropertyType::String:
            return UnboxedPropertyValue{std::wstring(pv.GetString())};
        case PropertyType::Boolean:
            return UnboxedPropertyValue{pv.GetBoolean()};
        case PropertyType::Char16:
            return UnboxedPropertyValue{pv.GetChar16()};
        case PropertyType::Double:
            return UnboxedPropertyValue{pv.GetDouble()};
        case PropertyType::Single:
            return UnboxedPropertyValue{pv.GetSingle()};
        case PropertyType::UInt8:
            return UnboxedPropertyValue{pv.GetUInt8()};
        case PropertyType::Int16:
            return UnboxedPropertyValue{pv.GetInt16()};
        case PropertyType::UInt16:
            return UnboxedPropertyValue{pv.GetUInt16()};
        case PropertyType::Int32:
            return UnboxedPropertyValue{pv.GetInt32()};
        case PropertyType::UInt32:
            return UnboxedPropertyValue{pv.GetUInt32()};
        case PropertyType::Int64:
```

```
            return UnboxedPropertyValue{pv.GetInt64()};
        case PropertyType::UInt64:
            return UnboxedPropertyValue{pv.GetUInt64()};
        case PropertyType::OtherType: {
            // Common for enums.
            if (auto intVal = value.try_as<int32_t>()) {
                return UnboxedPropertyValue{*intVal};
            }
            return std::nullopt;
        }
        default: {
            return std::nullopt;
        }
    }
}

// Invariant-formatted text form, suitable for XAML attribute use or diagnostic
// logs.
std::wstring FormatUnboxedPropertyValue(UnboxedPropertyValue const& v) {
    return std::visit(
        [](auto const& x) -> std::wstring {
            using T = std::decay_t<decltype(x)>;
            if constexpr (std::is_same_v<T, std::wstring>) {
                return x;
            } else if constexpr (std::is_same_v<T, bool>) {
                return x ? L"True" : L"False";
            } else if constexpr (std::is_same_v<T, char16_t>) {
                // Single-character text form so substitution emits the
                // character itself.
                return std::wstring(1, static_cast<wchar_t>(x));
            } else if constexpr (std::is_floating_point_v<T>) {
                return FormatDoubleInvariant(static_cast<double>(x));
            } else {
                return std::to_wstring(x);
            }
        },
        v);
}

// Numeric-as-double form, or std::nullopt if the value isn't numeric (i.e.
// holds a string).
std::optional<double> UnboxedPropertyValueAsNumeric(
    UnboxedPropertyValue const& v) {
    return std::visit(
        [](auto const& x) -> std::optional<double> {
            using T = std::decay_t<decltype(x)>;
            if constexpr (std::is_same_v<T, std::wstring>) {
                return std::nullopt;
            } else {
                return static_cast<double>(x);
            }
        },
        v);
}
```

```
bool TestElementMatcher(FrameworkElement element,
                        ElementMatcher& matcher,
                        VisualStateGroup* visualStateGroup,
                        PCWSTR fallbackClassName) {
  if (!matcher.type.empty() &&
      matcher.type != winrt::get_class_name(element) &&
      (!fallbackClassName || matcher.type != fallbackClassName)) {
      return false;
  }

  if (!matcher.name.empty() && matcher.name != element.Name()) {
      return false;
  }

  if (matcher.oneBasedIndex) {
      auto parent = Media::VisualTreeHelper::GetParent(element);
      if (!parent) {
          return false;
      }

      int index = matcher.oneBasedIndex - 1;
      if (index < 0 ||
          index >= Media::VisualTreeHelper::GetChildrenCount(parent) ||
          Media::VisualTreeHelper::GetChild(parent, index) != element) {
          return false;
      }
  }

  auto elementDo = element.as<DependencyObject>();

  for (const auto& propertyValue : GetResolvedPropertyValues(
          matcher.type,
          fallbackClassName ? fallbackClassName
                            : winrt::name_of<FrameworkElement>(),
          &matcher.propertyValues)) {
      const auto value =
          ReadLocalValueWithWorkaround(elementDo, propertyValue.first);
      if (!value) {
          Wh_Log(L"Null property value");
          return false;
      } else if (value == DependencyProperty::UnsetValue()) {
          return false;
      }

      auto expectedUnboxed = TryUnboxPropertyValue(propertyValue.second);
      auto valueUnboxed = TryUnboxPropertyValue(value);
      if (!expectedUnboxed || !valueUnboxed) {
          Wh_Log(L"Unsupported property class: %s",
              winrt::get_class_name(value).c_str());
          return false;
      }

      if (*expectedUnboxed != *valueUnboxed) {
          return false;
      }
```

```
    }

    if (matcher.visualStateGroupName && visualStateGroup) {
        *visualStateGroup =
            GetVisualStateGroup(element, *matcher.visualStateGroupName);
    }

    return true;
}

// Aggregated resolved rules for an element. Value-rules are still bucketed by
// visual-state-group (each target's rules live under that target's @VSGName);
// captures are intentionally NOT per-VSG -- they are wired up once at element
// level (see SetUpCapturesForElement).
struct ElementResolvedRules {
    std::unordered_map<VisualStateGroup, PropertyOverrides> overridesPerVSG;
    std::vector<CaptureSpec> captures;
    bool hasDynamicValues = false;
};

ElementResolvedRules FindElementPropertyOverrides(FrameworkElement element,
                                        PCWSTR fallbackClassName) {
    ElementResolvedRules result;
    std::unordered_set<DependencyProperty> propertiesAdded;
    std::unordered_set<std::wstring> capturesAdded;

    for (auto it = g_elementsCustomizationRules.rbegin();
         it != g_elementsCustomizationRules.rend(); ++it) {
        auto& override = *it;

        VisualStateGroup visualStateGroup = nullptr;

        if (!TestElementMatcher(element, override.elementMatcher,
                            &visualStateGroup, fallbackClassName)) {
            continue;
        }

        // Using iter.Parent() was sometimes returning null, so use
        // VisualTreeHelper::GetParent below instead.
        //
        // Recursive lambda so that '*' can backtrack: when a candidate match
        // for the wildcard's next matcher leads to a failure further up the
        // chain, retry with a farther ancestor.
        auto& parentMatchers = override.parentElementMatchers;
        auto matchParents = [&](auto& self, FrameworkElement iter,
                            size_t mi) -> bool {
            if (mi >= parentMatchers.size()) {
                return true;
            }

            auto& matcher = parentMatchers[mi];

            if (matcher.kind == ElementMatcher::Kind::Root) {
                if (Media::VisualTreeHelper::GetParent(iter)) {
                    return false;
```

```
      }

      return self(self, iter, mi + 1);
    }

    if (matcher.kind == ElementMatcher::Kind::Wildcard) {
      // '*' is always followed by an Element matcher (validated at
      // parse time). Walk up parents and try recursing for each
      // ancestor that matches the next matcher.
      auto& nextMatcher = parentMatchers[mi + 1];
      auto cur = iter;
      while (true) {
        auto parent = Media::VisualTreeHelper::GetParent(cur)
                        .try_as<FrameworkElement>();
        if (!parent) {
          return false;
        }

        cur = parent;
        if (TestElementMatcher(cur, nextMatcher, &visualStateGroup,
                               nullptr) &&
          self(self, cur, mi + 2)) {
          return true;
        }
      }
    }

    auto parent = Media::VisualTreeHelper::GetParent(iter)
                    .try_as<FrameworkElement>();
    if (!parent) {
      return false;
    }

    if (!TestElementMatcher(parent, matcher, &visualStateGroup,
                            nullptr)) {
      return false;
    }

    return self(self, parent, mi + 1);
};

if (!matchParents(matchParents, element, 0)) {
    continue;
}

const auto& resolvedRules = GetResolvedPropertyOverrides(
    override.elementMatcher.type,
    fallbackClassName ? fallbackClassName
                      : winrt::name_of<FrameworkElement>(),
    &override.propertyOverrides);

result.hasDynamicValues |= resolvedRules.hasDynamicValues;

auto& propertyOverridesForVSG =
    result.overridesPerVSG[visualStateGroup];
```

```
      for (const auto& [property, valuesPerVisualState] :
          resolvedRules.propertyOverrides) {
        bool propertyInserted = propertiesAdded.insert(property).second;
        if (!propertyInserted) {
          continue;
        }

        auto& propertyOverrides = propertyOverridesForVSG[property];
        for (const auto& [visualState, value] : valuesPerVisualState) {
          propertyOverrides.insert({visualState, value});
        }
      }

      for (const auto& capture : resolvedRules.captures) {
        if (!capturesAdded.insert(capture.varName).second) {
          continue;
        }

        result.captures.push_back(capture);
      }
    }

    std::erase_if(result.overridesPerVSG,
                  [](const auto& item) { return item.second.empty(); });

    return result;
}

struct StyleVariableResolution {
    // Points into state->variables; only valid until that map is next touched,
    // so read it out before doing anything that could apply a style.
    const StyleVariableValue* value = nullptr;
    ElementId owner = ElementId::None;
};

// How well a capture serves a consumer, as a sort key -- smaller is better.
// Captures are ranked by, in order:
//
//  1. Deepest common ancestor with the consumer.
//  2. Shallowest capture element. On a tie the capture that lies on the
//     consumer's own parent chain *is* the common ancestor, so this is what
//     makes a capture on an ancestor beat one on a cousin below it.
//  3. Registration order, applied by the callers below keeping the last of
//     equal keys. Only a last resort, but latest-wins is the useful direction:
//     a host that rebuilds a subtree often leaves the previous copy in the tree
//     beside the new one, and a consumer above both ties on the keys above.
//     The newer capture is the live one.
//
// The closest capture wins even when its value is opaque, in which case the
// consuming style is skipped rather than falling through to a farther capture
// that happens to be usable.
std::pair<int, int> StyleVariableCaptureRank(
    ElementTreeNode const* consumerNode,
    ElementTreeNode const* captureNode) {
    int lcaDepth = ElementTreeLcaDepth(consumerNode, captureNode);
```

```
   int captureDepth = captureNode ? static_cast<int>(captureNode->depth)
                                  : std::numeric_limits<int>::max();
   return {-lcaDepth, captureDepth};
}

// Pick the capture of `varName` that `consumerNode` should read.
StyleVariableResolution FindWinningCapture(
   StyleVariableState* state,
   const std::wstring& varName,
   ElementTreeNode const* consumerNode) {
   StyleVariableResolution result;

   auto it = state->variables.find(varName);
   if (it == state->variables.end() || it->second.empty()) {
      return result;
   }

   const auto& captures = it->second;
   if (captures.size() == 1) {
      // The common case by far: nothing to rank, and the owner's spine node
      // never has to be resolved.
      return {&captures.front().value, captures.front().elementId};
   }

   std::pair<int, int> bestRank;
   for (const auto& capture : captures) {
      ElementTreeNode const* captureNode = nullptr;
      if (auto elementIt =
            g_elementsCustomizationState.find(capture.elementId);
         elementIt != g_elementsCustomizationState.end()) {
         captureNode = EnsureElementTreeNode(elementIt->second);
      }

      auto rank = StyleVariableCaptureRank(consumerNode, captureNode);
      if (!result.value || rank <= bestRank) {
         bestRank = rank;
         result = {&capture.value, capture.elementId};
      }
   }

   return result;
}

// A capture reduced to what ranking needs. The node is held by strong ref so a
// snapshot stays usable even after re-entrant work tears the owning element
// down.
struct StyleVariableCandidate {
   ElementId owner = ElementId::None;
   std::shared_ptr<ElementTreeNode> node;
};

// Resolve every capture's spine node once. A pass that ranks one variable
// against many consumers would otherwise repeat the same lookups per consumer,
// and only the ranking actually varies between them.
std::vector<StyleVariableCandidate> SnapshotStyleVariableCaptures(
```

```
   const std::vector<StyleVariableCapture>& captures) {
   std::vector<StyleVariableCandidate> candidates;
   candidates.reserve(captures.size());

   for (const auto& capture : captures) {
      StyleVariableCandidate candidate;
      candidate.owner = capture.elementId;
      if (auto elementIt =
            g_elementsCustomizationState.find(capture.elementId);
         elementIt != g_elementsCustomizationState.end()) {
         auto& elementCustomizationState = elementIt->second;
         EnsureElementTreeNode(elementCustomizationState);
         candidate.node = elementCustomizationState.treeNode;
      }

      candidates.push_back(std::move(candidate));
   }

   return candidates;
}

// The owner FindWinningCapture would pick, ranked from a snapshot. A snapshot
// taken before a re-entrant capture change can go stale, which at worst skips a
// consumer that needed redoing -- the change that invalidated it queues its own
// propagation, and that pass re-snapshots and picks the consumer up.
ElementId PickWinningCaptureOwner(
   const std::vector<StyleVariableCandidate>& candidates,
   ElementTreeNode const* consumerNode) {
   ElementId owner = ElementId::None;
   bool haveBest = false;
   std::pair<int, int> bestRank;

   for (const auto& candidate : candidates) {
      auto rank =
         StyleVariableCaptureRank(consumerNode, candidate.node.get());
      if (!haveBest || rank <= bestRank) {
         haveBest = true;
         bestRank = rank;
         owner = candidate.owner;
      }
   }

   return owner;
}

// What a `{{...}}` expansion needs. `consumerNode` is the consuming element's
// position in the tree, used to pick the closest capture of each name.
struct StyleVariableLookupContext {
   StyleVariableState* state;
   ElementTreeNode const* consumerNode;
   std::vector<StyleVariableDependency>* outDeps;
};

bool IsValidStyleVariableIdentifier(std::wstring_view sv) {
   if (sv.empty()) {
```

```
        return false;
    }
    auto isStart = [](wchar_t c) {
        return (c >= L'A' && c <= L'Z') || (c >= L'a' && c <= L'z') ||
               c == L'_';
    };
    auto isCont = [&](wchar_t c) {
        return isStart(c) || (c >= L'0' && c <= L'9');
    };
    if (!isStart(sv[0])) {
        return false;
    }
    for (size_t i = 1; i < sv.size(); i++) {
        if (!isCont(sv[i])) {
            return false;
        }
    }
    return true;
}

// Value produced while evaluating a `{{ ... }}` expression: either a number or
// a string. Number literals and numeric variables produce numbers; backtick-
// delimited string literals and string-typed variables produce strings.
struct StyleExpressionValue {
    // Engaged => numeric value; otherwise `text` holds the string value.
    std::optional<double> number;
    std::wstring text;

    static StyleExpressionValue Number(double d) { return {d, std::wstring()}; }
    static StyleExpressionValue String(std::wstring s) {
        return {std::nullopt, std::move(s)};
    }

    bool IsNumber() const { return number.has_value(); }
};

// Thrown by a live skip() call to unwind the evaluator. Not a std::exception,
// so a generic failure handler on the way up does not mistake it for an error.
struct StyleVariableSkipRequested {};

// Recursive-descent evaluator for `{{ ... }}` expressions. Operands: number
// literals, backtick-delimited string literals, style variable references, and
// parenthesized subexpressions. Operators: binary + - * /, unary - / +, the
// comparisons < <= == >= > !=, the conditional operator cond ? a : b, the
// two-arg functions min(a, b) and max(a, b), and skip(), which throws
// StyleVariableSkipRequested so the consuming style is left unapplied.
// Standard math precedence.
// Arithmetic, relational, unary-sign, and min/max operators require numeric
// operands; == and != compare two numbers or two strings; the conditional
// selects one of its (possibly string) branches. Evaluate() formats the result
// to text.
//
// Variable references pushed into outDeps so the dependent style can be
// re-evaluated when those variables change.
class StyleVariableExpressionEvaluator {
```

```
public:
 StyleVariableExpressionEvaluator(std::wstring_view text,
                                  const StyleVariableLookupContext* context)
    : m_text(text), m_context(context) {}

 // Returns the text form of the result: numeric results are formatted with
 // FormatDoubleInvariant, string results are returned verbatim. Throws
 // std::runtime_error on parse / evaluation failure (including when a value
 // is used where the grammar requires a number).
 std::wstring Evaluate() {
    m_pos = 0;
    SkipWhitespace();
    StyleExpressionValue v = ParseExpression();
    SkipWhitespace();
    if (m_pos != m_text.size()) {
       throw std::runtime_error(
          "Unexpected trailing characters in style variable expression");
    }
    if (v.IsNumber()) {
       return FormatDoubleInvariant(*v.number);
    }
    return v.text;
 }

private:
 void SkipWhitespace() {
    while (m_pos < m_text.size() &&
          (m_text[m_pos] == L' ' || m_text[m_pos] == L'\t' ||
           m_text[m_pos] == L'\r' || m_text[m_pos] == L'\n')) {
       m_pos++;
    }
 }

 bool ConsumeChar(wchar_t c) {
    SkipWhitespace();
    if (m_pos < m_text.size() && m_text[m_pos] == c) {
       m_pos++;
       return true;
    }
    return false;
 }

 // Tries to consume the multi-char operator `op` at the current position
 // (after skipping leading whitespace). The operator must match exactly with
 // no embedded whitespace; advances past it and returns true on success.
 bool ConsumeOperator(std::wstring_view op) {
    SkipWhitespace();
    if (m_text.size() - m_pos >= op.size() &&
       m_text.compare(m_pos, op.size(), op) == 0) {
       m_pos += op.size();
       return true;
    }
    return false;
 }
```

```
// Unwraps a numeric operand. In a dead ternary branch (m_live == false) the
// value is discarded, so a string operand is tolerated (reported as 0)
// rather than aborting the whole expression.
double RequireNumber(const StyleExpressionValue& v) {
   if (v.IsNumber()) {
      return *v.number;
   }
   if (m_live) {
      throw std::runtime_error(
         "Non-numeric value used where a number is required in style "
         "variable expression");
   }
   return 0.0;
}

// Equality test for == / !=. Two numbers compare numerically, two strings
// compare by content. A number/string mismatch is always unequal rather
// than an error, so `{{var == `` ? default : var}}` can supply a fallback
// for an undefined variable (which reads as the empty string) without
// failing when the variable is instead a captured number.
bool ValuesEqual(const StyleExpressionValue& a,
                 const StyleExpressionValue& b) {
   if (a.IsNumber() && b.IsNumber()) {
      return *a.number == *b.number;
   }
   if (!a.IsNumber() && !b.IsNumber()) {
      return a.text == b.text;
   }
   return false;
}

StyleExpressionValue ParseExpression() { return ParseTernary(); }

// Conditional operator `cond ? thenVal : elseVal`, right-associative.
// Short-circuit: only the taken branch is evaluated. The untaken branch is
// still parsed (to advance the position and enforce syntax) with m_live
// cleared, which suppresses value-level errors (division by zero, a
// non-numeric / undefined variable, an unknown function), skip(), and
// dependency capture for that branch.
StyleExpressionValue ParseTernary() {
   StyleExpressionValue cond = ParseEquality();
   if (!ConsumeChar(L'?')) {
      return cond;
   }
   bool condTrue = RequireNumber(cond) != 0.0;
   bool prevLive = m_live;

   m_live = prevLive && condTrue;
   StyleExpressionValue thenVal = ParseExpression();
   m_live = prevLive;

   if (!ConsumeChar(L':')) {
      throw std::runtime_error(
         "Missing ':' for '?' in style variable expression");
   }
```

```
    m_live = prevLive && !condTrue;
    StyleExpressionValue elseVal = ParseTernary();
    m_live = prevLive;

    return condTrue ? thenVal : elseVal;
}

StyleExpressionValue ParseEquality() {
    StyleExpressionValue v = ParseRelational();
    while (true) {
        if (ConsumeOperator(L"==")) {
            v = StyleExpressionValue::Number(
                ValuesEqual(v, ParseRelational()) ? 1.0 : 0.0);
        } else if (ConsumeOperator(L"!=")) {
            v = StyleExpressionValue::Number(
                ValuesEqual(v, ParseRelational()) ? 0.0 : 1.0);
        } else {
            break;
        }
    }
    return v;
}

StyleExpressionValue ParseRelational() {
    StyleExpressionValue v = ParseAdditive();
    while (true) {
        // Match the two-char operators before their single-char prefixes.
        if (ConsumeOperator(L"<=")) {
            double lhs = RequireNumber(v);
            v = StyleExpressionValue::Number(
                lhs <= RequireNumber(ParseAdditive()) ? 1.0 : 0.0);
        } else if (ConsumeOperator(L">=")) {
            double lhs = RequireNumber(v);
            v = StyleExpressionValue::Number(
                lhs >= RequireNumber(ParseAdditive()) ? 1.0 : 0.0);
        } else if (ConsumeOperator(L"<")) {
            double lhs = RequireNumber(v);
            v = StyleExpressionValue::Number(
                lhs < RequireNumber(ParseAdditive()) ? 1.0 : 0.0);
        } else if (ConsumeOperator(L">")) {
            double lhs = RequireNumber(v);
            v = StyleExpressionValue::Number(
                lhs > RequireNumber(ParseAdditive()) ? 1.0 : 0.0);
        } else {
            break;
        }
    }
    return v;
}

StyleExpressionValue ParseAdditive() {
    StyleExpressionValue v = ParseTerm();
    while (true) {
        SkipWhitespace();
```

```
      if (ConsumeChar(L'+')) {
         double lhs = RequireNumber(v);
         v = StyleExpressionValue::Number(lhs +
                                  RequireNumber(ParseTerm()));
      } else if (ConsumeChar(L'-')) {
         double lhs = RequireNumber(v);
         v = StyleExpressionValue::Number(lhs -
                                  RequireNumber(ParseTerm()));
      } else {
         break;
      }
   }
   return v;
}

StyleExpressionValue ParseTerm() {
   StyleExpressionValue v = ParseFactor();
   while (true) {
      SkipWhitespace();
      if (ConsumeChar(L'*')) {
         double lhs = RequireNumber(v);
         v = StyleExpressionValue::Number(lhs *
                                  RequireNumber(ParseFactor()));
      } else if (ConsumeChar(L'/')) {
         double lhs = RequireNumber(v);
         double rhs = RequireNumber(ParseFactor());
         if (rhs == 0.0) {
            if (m_live) {
               throw std::runtime_error(
                  "Division by zero in style variable expression");
            }
            // Dead ternary branch: the result is discarded, so skip the
            // divide instead of throwing or producing inf/nan.
            v = StyleExpressionValue::Number(lhs);
         } else {
            v = StyleExpressionValue::Number(lhs / rhs);
         }
      } else {
         break;
      }
   }
   return v;
}

StyleExpressionValue ParseFactor() {
   SkipWhitespace();
   if (ConsumeChar(L'+')) {
      return StyleExpressionValue::Number(RequireNumber(ParseFactor()));
   }
   if (ConsumeChar(L'-')) {
      return StyleExpressionValue::Number(-RequireNumber(ParseFactor()));
   }
   return ParsePrimary();
}
```

```
StyleExpressionValue ParsePrimary() {
   SkipWhitespace();
   if (m_pos >= m_text.size()) {
      throw std::runtime_error(
         "Unexpected end of style variable expression");
   }

   wchar_t c = m_text[m_pos];
   if (c == L'(') {
      m_pos++;
      StyleExpressionValue v = ParseExpression();
      SkipWhitespace();
      if (!ConsumeChar(L')')) {
         throw std::runtime_error(
            "Missing ')' in style variable expression");
      }
      return v;
   }

   if (c == L'`') {
      return ParseStringLiteral();
   }

   if ((c >= L'0' && c <= L'9') || c == L'.') {
      return StyleExpressionValue::Number(ParseNumberLiteral());
   }

   if ((c >= L'A' && c <= L'Z') || (c >= L'a' && c <= L'z') || c == L'_') {
      return ParseIdentifierOrCall();
   }

   throw std::runtime_error(
      "Unexpected character in style variable expression");
}

// Backtick-delimited string literal. A doubled backtick encodes one literal
// backtick character; every other character is taken verbatim. Backtick is
// used (rather than a quote) so that literals don't clash with the string
// quoting of YAML settings or with the double quotes of XAML attributes,
// inside which these expressions often appear. The literal must be closed
// before the end of the expression.
StyleExpressionValue ParseStringLiteral() {
   m_pos++;  // Skip the opening backtick.
   std::wstring out;
   while (m_pos < m_text.size()) {
      wchar_t c = m_text[m_pos];
      if (c == L'`') {
         if (m_pos + 1 < m_text.size() && m_text[m_pos + 1] == L'`') {
            out.push_back(L'`');
            m_pos += 2;
            continue;
         }
         m_pos++;
         return StyleExpressionValue::String(std::move(out));
      }
```

```
      out.push_back(c);
      m_pos++;
    }
    throw std::runtime_error(
      "Unterminated string literal in style variable expression");
  }

  double ParseNumberLiteral() {
    size_t start = m_pos;
    bool sawDigit = false;
    bool sawDot = false;
    while (m_pos < m_text.size()) {
      wchar_t c = m_text[m_pos];
      if (c >= L'0' && c <= L'9') {
        sawDigit = true;
        m_pos++;
      } else if (c == L'.' && !sawDot) {
        sawDot = true;
        m_pos++;
      } else {
        break;
      }
    }
    if (m_pos < m_text.size() &&
      (m_text[m_pos] == L'e' || m_text[m_pos] == L'E')) {
      m_pos++;
      if (m_pos < m_text.size() &&
        (m_text[m_pos] == L'+' || m_text[m_pos] == L'-')) {
        m_pos++;
      }
      while (m_pos < m_text.size() && m_text[m_pos] >= L'0' &&
          m_text[m_pos] <= L'9') {
        m_pos++;
      }
    }
    if (!sawDigit) {
      throw std::runtime_error(
        "Bad number literal in style variable expression");
    }
    auto parsed = ParseDoubleInvariant(m_text.substr(start, m_pos - start));
    if (!parsed) {
      throw std::runtime_error(
        "Bad number literal in style variable expression");
    }
    return *parsed;
  }

  StyleExpressionValue ParseIdentifierOrCall() {
    size_t start = m_pos;
    while (m_pos < m_text.size()) {
      wchar_t c = m_text[m_pos];
      if ((c >= L'A' && c <= L'Z') || (c >= L'a' && c <= L'z') ||
        (c >= L'0' && c <= L'9') || c == L'_') {
        m_pos++;
      } else {
```

```
            break;
        }
    }
    std::wstring_view ident = m_text.substr(start, m_pos - start);
    SkipWhitespace();
    if (m_pos < m_text.size() && m_text[m_pos] == L'(') {
        m_pos++;
        if (ident == L"skip") {
            if (!ConsumeChar(L')')) {
                throw std::runtime_error(
                    "skip() takes no arguments in style variable "
                    "expression");
            }
            if (m_live) {
                throw StyleVariableSkipRequested{};
            }
            // Dead ternary branch: value discarded.
            return StyleExpressionValue::Number(0.0);
        }
        double a = RequireNumber(ParseExpression());
        if (!ConsumeChar(L',')) {
            throw std::runtime_error(
                "Expected ',' in min/max style variable call");
        }
        double b = RequireNumber(ParseExpression());
        if (!ConsumeChar(L')')) {
            throw std::runtime_error(
                "Missing ')' after min/max style variable call");
        }
        if (ident == L"min") {
            return StyleExpressionValue::Number((a < b) ? a : b);
        }
        if (ident == L"max") {
            return StyleExpressionValue::Number((a > b) ? a : b);
        }
        if (m_live) {
            throw std::runtime_error(
                "Unknown function in style variable expression");
        }
        // Dead ternary branch: value discarded, don't fail on the name.
        return StyleExpressionValue::Number(0.0);
    }
    return LookupVariable(std::wstring(ident));
}

StyleExpressionValue LookupVariable(const std::wstring& name) {
    // In a dead ternary branch (m_live == false) the value is discarded, so
    // skip the lookup along with dependency capture and the value-level
    // errors below; the branch must not abort the whole expression, and
    // every operator tolerates a string operand while not live.
    if (!m_live) {
        return StyleExpressionValue::String(L"");
    }

    auto resolution =
```

```
        FindWinningCapture(m_context->state, name, m_context->consumerNode);

    if (m_context->outDeps) {
      m_context->outDeps->push_back({name, resolution.owner});
    }
    if (!resolution.value) {
      Wh_Log(L"Style variable '%s' not defined; treating as empty string",
          name.c_str());
      // Undefined reads as the empty string sentinel, so `{{var == `` ?
      // default : var}}` can detect the undefined state and substitute a
      // fallback. Arithmetic on an undefined variable then fails
      // RequireNumber and skips the style, rather than silently using 0.
      return StyleExpressionValue::String(L"");
    }
    if (resolution.value->numeric) {
      return StyleExpressionValue::Number(*resolution.value->numeric);
    }
    // Non-numeric primitive (e.g. a captured string property): usable as a
    // string operand.
    if (resolution.value->substitutable) {
      return StyleExpressionValue::String(resolution.value->stringForm);
    }
    // Opaque capture (brush, thickness, etc.): no value form usable in an
    // expression.
    throw std::runtime_error(
      "Style variable used in expression is not a primitive value");
  }

  std::wstring_view m_text;
  const StyleVariableLookupContext* m_context;
  size_t m_pos = 0;
  // When false, we're parsing (but discarding) the untaken branch of a
  // ternary; value-level errors and dependency capture are suppressed.
  bool m_live = true;
};

// Evaluate a single expression body (the text between `{{` and `}}`). If the
// body is a bare identifier, returns the variable's `stringForm` directly --
// but only when the captured value is a primitive type flagged `substitutable`
// (numeric, boolean, or string). Missing variables and opaque-type captures
// both cause this function to return std::nullopt, at which point
// ExpandStyleVariables aborts the whole expansion and the consuming style is
// skipped. This matches the arithmetic path's behaviour of failing closed
// rather than substituting a value that won't parse. A live skip() call unwinds
// through here as StyleVariableSkipRequested.
std::optional<std::wstring> EvaluateStyleVariableExpression(
  std::wstring_view exprText,
  const StyleVariableLookupContext* context) {
  auto trimmed = TrimStringView(exprText);
  if (trimmed.empty()) {
    Wh_Log(L"Empty style variable expression");
    return std::nullopt;
  }

  if (IsValidStyleVariableIdentifier(trimmed)) {
```

```
      std::wstring name(trimmed);
      auto resolution =
          FindWinningCapture(context->state, name, context->consumerNode);
      if (context->outDeps) {
        context->outDeps->push_back({name, resolution.owner});
      }
      if (!resolution.value) {
        Wh_Log(L"Style variable '%s' not yet defined; skipping style",
               name.c_str());
        return std::nullopt;
      }
      if (!resolution.value->substitutable) {
        Wh_Log(
            L"Style variable '%s' is not substitutable (captured type "
            L"'%s'); skipping style",
            name.c_str(), resolution.value->stringForm.c_str());
        return std::nullopt;
      }
      return resolution.value->stringForm;
    }

    try {
      StyleVariableExpressionEvaluator eval(trimmed, context);
      return eval.Evaluate();
    } catch (StyleVariableSkipRequested const&) {
      Wh_Log(L"skip() reached in '%.*s'; leaving style unapplied",
             static_cast<int>(trimmed.size()), trimmed.data());
      throw;
    } catch (std::exception const& ex) {
      Wh_Log(L"Style variable expression failed: %S (in '%.*s')", ex.what(),
             static_cast<int>(trimmed.size()), trimmed.data());
      return std::nullopt;
    }
  }

  // Walks the input text, repeatedly expanding the innermost `{{ ... }}`
  // substitution. Returns std::nullopt on parse failure (and logs a warning); a
  // StyleVariableSkipRequested from an expression propagates out.
  //
  // Inner-matching rule: the first `}}` is paired with the *rightmost* `{{` that
  // precedes it. So `{{{x}}}` -> `{` + value-of-x + `}` (literal outer braces).
  //
  // Substituted text is treated as literal (no further `{{...}}` expansion of the
  // substituted output) to keep behavior predictable.
  std::optional<std::wstring> ExpandStyleVariables(
      std::wstring_view input,
      const StyleVariableLookupContext* context) {
    std::wstring result(input);
    size_t scanFrom = 0;

    while (true) {
      size_t closePos = std::wstring::npos;
      for (size_t i = scanFrom; i + 1 < result.size(); i++) {
        if (result[i] == L'}' && result[i + 1] == L'}') {
          closePos = i;
```

```
            break;
         }
      }
      if (closePos == std::wstring::npos) {
         break;
      }

      // Find rightmost `{{` strictly before closePos. Search from closePos -
      // 1 downward; the pair occupies indices (j-1, j).
      size_t openPos = std::wstring::npos;
      if (closePos >= 2) {
         for (size_t j = closePos - 1; j >= 1; j--) {
            if (result[j - 1] == L'{' && result[j] == L'{') {
               openPos = j - 1;
               break;
            }
            if (j == 1) {
               break;
            }
         }
      }

      if (openPos == std::wstring::npos) {
         Wh_Log(L"Unmatched '}}' in style value at offset %zu", closePos);
         return std::nullopt;
      }

      std::wstring_view exprText(result.data() + openPos + 2,
                                 closePos - openPos - 2);
      auto expanded = EvaluateStyleVariableExpression(exprText, context);
      if (!expanded) {
         return std::nullopt;
      }

      size_t spanLen = closePos + 2 - openPos;
      result.replace(openPos, spanLen, *expanded);
      scanFrom = openPos + expanded->size();
   }

   return result;
}

// Read a property's current effective value and convert it to a
// StyleVariableValue suitable for `{{Var}}` substitution. Numeric primitives
// produce both string + numeric forms and are flagged substitutable; boolean
// and string primitives are flagged substitutable but have no numeric form.
// Opaque types (brushes, thicknesses, etc.) record only the captured class name
// as a diagnostic and are NOT flagged substitutable -- the bare- identifier
// substitution path skips them rather than emitting a class name into the XAML
// output.
StyleVariableValue ReadCapturedStyleVariableValue(FrameworkElement element,
                                                  DependencyProperty property) {
   StyleVariableValue out;

   auto elementDo = element.as<DependencyObject>();
```

```
    winrt::Windows::Foundation::IInspectable value{nullptr};
    // Get effective value so layout-driven properties like ActualWidth (which
    // never have a local value) still capture.
    try {
        value = elementDo.GetValue(property);
    } catch (winrt::hresult_error const& ex) {
        Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
    }
    if (!value || value == DependencyProperty::UnsetValue()) {
        out.stringForm = L"";
        return out;
    }

    try {
        if (auto unboxed = TryUnboxPropertyValue(value)) {
            out.stringForm = FormatUnboxedPropertyValue(*unboxed);
            out.numeric = UnboxedPropertyValueAsNumeric(*unboxed);
            out.substitutable = true;
            return out;
        }

        // Opaque value (brush, thickness, etc.). Stored as a diagnostic only;
        // not flagged substitutable, so bare `{{Var}}` skips the consuming
        // style with a clear log message rather than emitting `className` into
        // the XAML.
        out.stringForm = std::wstring(winrt::get_class_name(value));
    } catch (winrt::hresult_error const& ex) {
        Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
        out.stringForm = L"";
    }
    return out;
}

// Remove this (elementId, property) entry from the consumer lists of every
// variable named in oldDeps, then add it for every variable named in newDeps.
// `fallbackClassName` is stored on each newly-added consumer entry so the
// per-consumer context is preserved across propagations; it is irrelevant when
// newDeps is empty (pure-removal calls from the cleanup paths).
void UpdateStyleVariableConsumers(
    StyleVariableState* state,
    ElementId elementId,
    DependencyProperty property,
    PCWSTR fallbackClassName,
    const std::vector<StyleVariableDependency>& oldDeps,
    const std::vector<StyleVariableDependency>& newDeps) {
    if (!state) {
        // The element's XamlRoot has already been destroyed (or was never
        // available); the StyleVariableState entry has been or will be reaped,
        // and there is nothing to clean up. New registrations (newDeps) are
        // also dropped on the floor: without a state we cannot route
        // propagations anyway.
        return;
    }

    for (const auto& dep : oldDeps) {
```

```
      auto it = state->consumers.find(dep.name);
      if (it == state->consumers.end()) {
        continue;
      }
      auto& consumers = it->second;
      ReleaseStyleVariableElementRefs(
        state, elementId,
        std::erase_if(consumers, [&](const StyleVariableConsumer& c) {
          return c.elementId == elementId && c.property == property;
        }));
      if (consumers.empty()) {
        state->consumers.erase(it);
      }
    }

    std::wstring fallbackClassNameStr =
      fallbackClassName ? fallbackClassName : L"";
    for (const auto& dep : newDeps) {
      auto& consumers = state->consumers[dep.name];
      bool already = std::any_of(consumers.begin(), consumers.end(),
                                 [&](const StyleVariableConsumer& c) {
                                   return c.elementId == elementId &&
                                          c.property == property;
                                 });
      if (!already) {
        consumers.push_back({elementId, property, fallbackClassNameStr});
        AddStyleVariableElementRef(state, elementId);
      }
    }
}

// Value comparison of `a` and `b` as boxes of the first struct type in
// T, Ts... that `a` is a box of; nullopt when it is none of them.
template <typename T, typename... Ts>
std::optional<bool> SameBoxedStruct(
    winrt::Windows::Foundation::IInspectable const& a,
    winrt::Windows::Foundation::IInspectable const& b) {
    if (auto ra = a.try_as<winrt::Windows::Foundation::IReference<T>>()) {
      auto rb = b.try_as<winrt::Windows::Foundation::IReference<T>>();
      return rb && ra.Value() == rb.Value();
    }
    if constexpr (sizeof...(Ts) > 0) {
      return SameBoxedStruct<Ts...>(a, b);
    } else {
      return std::nullopt;
    }
}

// Whether two values read from a property are the same local value. XAML boxes
// value types anew on every read, so those compare by value: primitives and
// enums through TryUnboxPropertyValue, then the struct types styles commonly
// set. Any other boxed value type is taken as unchanged: adopting a value that
// may be the mod's own would leave it in place on cleanup, the worse mistake.
// Reference types, UnsetValue included, compare by identity.
bool SameLocalValue(winrt::Windows::Foundation::IInspectable const& a,
```

```
                winrt::Windows::Foundation::IInspectable const& b) {
    if (a == b) {
        return true;
    }
    if (!a || !b) {
        return false;
    }

    auto ua = TryUnboxPropertyValue(a);
    auto ub = TryUnboxPropertyValue(b);
    if (ua || ub) {
        return ua && ub &&
            std::visit(
                [](auto const& x, auto const& y) -> bool {
                    using X = std::decay_t<decltype(x)>;
                    if constexpr (!std::is_same_v<
                                    X, std::decay_t<decltype(y)>>) {
                        return false;
                    } else if constexpr (std::is_floating_point_v<X>) {
                        return x == y || (std::isnan(x) && std::isnan(y));
                    } else {
                        return x == y;
                    }
                },
                *ua, *ub);
    }

    if (auto same = SameBoxedStruct<
            Thickness, CornerRadius, GridLength,
            winrt::Windows::Foundation::Point, winrt::Windows::Foundation::Size,
            winrt::Windows::Foundation::Rect, winrt::Windows::UI::Color,
            winrt::Windows::UI::Text::FontWeight>(a, b)) {
        return *same;
    }

    auto isBoxedValue = [](winrt::Windows::Foundation::IInspectable const& v) {
        return v.try_as<winrt::Windows::Foundation::IPropertyValue>() !=
                nullptr ||
            std::wstring_view(winrt::get_class_name(v))
                .starts_with(L"Windows.Foundation.IReference`1<");
    };
    return isBoxedValue(a) && isBoxedValue(b);
}

// Record a write by something other than the mod as the property's pre-style
// value. A local value still equal to the one the mod applied means nothing
// external happened (an animation changes only the effective value), and
// adopting the mod's own brush would make it survive cleanup and crash when
// the DLL is unloaded.
void AdoptExternalValueAsOriginal(
    FrameworkElement element,
    DependencyProperty property,
    ElementPropertyCustomizationState* propertyCustomizationState) {
    if (!propertyCustomizationState->customValue) {
        return;
```

```
    }
    auto localValue = ReadLocalValueWithWorkaround(element, property);
    if (!SameLocalValue(localValue,
                        propertyCustomizationState->lastAppliedValue)) {
        propertyCustomizationState->originalValue = localValue;
    }
}

// Put the property back to its pre-style value and forget what was applied.
// Leaves the dynamic template alone, so a later variable change can apply the
// style again.
void UnapplyStyleValue(
    FrameworkElement element,
    DependencyProperty property,
    ElementPropertyCustomizationState* propertyCustomizationState) {
    AdoptExternalValueAsOriginal(element, property, propertyCustomizationState);
    if (propertyCustomizationState->originalValue) {
        bool wasModifying = g_elementPropertyModifying;
        g_elementPropertyModifying = true;
        SetOrClearValue(element, property,
                        *propertyCustomizationState->originalValue);
        g_elementPropertyModifying = wasModifying;
        propertyCustomizationState->originalValue.reset();
    }
    propertyCustomizationState->lastAppliedValue = nullptr;
    propertyCustomizationState->customValue.reset();
}

// Re-evaluate the dynamic template stored on `propertyCustomizationState` and
// return the resolved IInspectable / XamlBlurBrushParams ready to be applied.
// Updates the (elementId, property) -> state->consumers registry to match the
// freshly computed dependency set so future variable changes route to this
// property. The dependency registry is committed *before* the final XAML
// resolution attempt: ExpandStyleVariables records every variable name it scans
// into newDeps even on partial parse failure, which lets a future change to any
// of those variables re-enter this function and retry. The trade-off is that on
// resolution failure the caller's last-good `customValue` is preserved (we
// return std::nullopt and the caller leaves the property as-is); this
// self-heals on the next variable change.
//
// `fallbackClassName` is the consumer-element's own fallback class name (the
// one that was used when matching the consumer's target rule), which is
// generally NOT the same as the capturer's. It is what
// ResolveExpandedSinglePropertyValue feeds to the synthetic <Style> used to
// re-parse the rule body, and it is also stored on each new
// StyleVariableConsumer entry so subsequent propagations route through this
// same context.
//
// `elementCustomizationState` is the consumer's own entry when the caller
// already has it, saving the lookup needed to rank captures by proximity; pass
// nullptr to have it looked up from `elementId`.
//
// Returns std::nullopt if the state has no template, expansion failed, or XAML
// resolution failed. A skip() in the rule body also yields std::nullopt, after
// putting the property back to its original value.
```

```
std::optional<PropertyOverrideValue> ResolveDynamicStyleValue(
    StyleVariableState* state,
    ElementId elementId,
    FrameworkElement element,
    DependencyProperty property,
    PCWSTR fallbackClassName,
    ElementPropertyCustomizationState* propertyCustomizationState,
    ElementCustomizationState* elementCustomizationState) {
    if (!propertyCustomizationState->dynamicTemplate) {
        return std::nullopt;
    }

    const auto& tmpl = *propertyCustomizationState->dynamicTemplate;

    if (!elementCustomizationState) {
        if (auto it = g_elementsCustomizationState.find(elementId);
            it != g_elementsCustomizationState.end()) {
            elementCustomizationState = &it->second;
        }
    }

    ElementTreeNode const* consumerNode =
        elementCustomizationState
            ? EnsureElementTreeNode(*elementCustomizationState)
            : nullptr;

    std::vector<StyleVariableDependency> newDeps;
    StyleVariableLookupContext context{state, consumerNode, &newDeps};
    std::optional<std::wstring> expanded;
    bool skipped = false;
    try {
        expanded = ExpandStyleVariables(tmpl.rawValue, &context);
    } catch (StyleVariableSkipRequested const&) {
        skipped = true;
    }

    UpdateStyleVariableConsumers(
        state, elementId, property, fallbackClassName,
        propertyCustomizationState->variableDependencies, newDeps);
    propertyCustomizationState->variableDependencies = std::move(newDeps);

    if (skipped) {
        // Every variable that decides whether skip() is reached was read
        // before it, so targeted propagation still reaches this property.
        propertyCustomizationState->lastResolveFailed = false;
        UnapplyStyleValue(element, property, propertyCustomizationState);
        return std::nullopt;
    }

    if (!expanded) {
        propertyCustomizationState->lastResolveFailed = true;
        return std::nullopt;
    }

    auto typeName = winrt::get_class_name(element);
```

```
   auto resolved = ResolveExpandedSinglePropertyValue(
      std::wstring_view(typeName),
      fallbackClassName ? std::wstring_view(fallbackClassName)
                        : winrt::name_of<FrameworkElement>(),
      tmpl.propertyName, *expanded, tmpl.isXamlValue);
   if (!resolved) {
      Wh_Log(
         L"Dynamic style resolution failed for '%s' on %s; keeping "
         L"previously applied value",
         tmpl.propertyName.c_str(), typeName.c_str());
   }
   propertyCustomizationState->lastResolveFailed = !resolved;
   return resolved;
}

// Whether a change to `varName` can alter this property's resolved value.
// `changedOwner` is set when one capture's value changed: only consumers that
// read from that capture are affected. It is empty when the set of captures
// changed instead, in which case `winningOwner` is the capture the consumer
// would read now, and only a consumer whose recorded owner differs needs
// redoing.
bool StyleVariableChangeAffectsConsumer(
   const ElementPropertyCustomizationState& propertyCustomizationState,
   const std::wstring& varName,
   std::optional<ElementId> changedOwner,
   ElementId winningOwner) {
   if (propertyCustomizationState.lastResolveFailed) {
      return true;
   }

   for (const auto& dep : propertyCustomizationState.variableDependencies) {
      if (dep.name != varName) {
         continue;
      }

      return changedOwner ? dep.owner == *changedOwner
                          : dep.owner != winningOwner;
   }

   return false;
}

// Re-evaluate the dependent styles a change to `varName` can actually reach.
// Each consumer carries its own fallbackClassName (recorded when the consumer
// was registered), so propagation uses the consumer's own match-site context to
// re-parse the rule body, even when the capturer was matched against a
// different type/fallback class.
void PropagateStyleVariableChangeCore(StyleVariableState* state,
                                      const std::wstring& varName,
                                      std::optional<ElementId> changedOwner) {
   auto consumersIt = state->consumers.find(varName);
   if (consumersIt == state->consumers.end()) {
      return;
   }
```

```
// Only the ranking varies per consumer, so the captures' spine nodes are
// resolved once for the whole pass. Needed only when the set of captures
// changed; a value change routes by the recorded owner instead.
std::vector<StyleVariableCandidate> candidates;
if (!changedOwner) {
   if (auto varIt = state->variables.find(varName);
      varIt != state->variables.end()) {
      candidates = SnapshotStyleVariableCaptures(varIt->second);
   }
}

auto consumersCopy = consumersIt->second;
for (const auto& consumer : consumersCopy) {
   auto stateIt = g_elementsCustomizationState.find(consumer.elementId);
   if (stateIt == g_elementsCustomizationState.end()) {
      continue;
   }
   // A reference rather than the iterator: applying a style below can
   // realize children, which re-enters ApplyCustomizations and may rehash
   // g_elementsCustomizationState. Rehashing invalidates iterators but not
   // references to the mapped values. A re-entrant cleanup or re-apply of
   // this same elementId would invalidate both the reference and the loop
   // below, but the re-entrancy is for the newly realized children.
   auto& elementState = stateIt->second;

   auto element = elementState.element.get();
   if (!element) {
      continue;
   }

   // A handful of pointer comparisons against the snapshot above, far
   // cheaper than the re-parse it avoids.
   ElementId winningOwner =
      changedOwner ? ElementId::None
                   : PickWinningCaptureOwner(
                        candidates, EnsureElementTreeNode(elementState));

   PCWSTR consumerFallbackClassName =
      consumer.fallbackClassName.empty()
         ? nullptr
         : consumer.fallbackClassName.c_str();

   for (auto& [vsgWeak, vsgState] : elementState.perVisualStateGroup) {
      auto propIt =
         vsgState.propertyCustomizationStates.find(consumer.property);
      if (propIt == vsgState.propertyCustomizationStates.end()) {
         continue;
      }
      auto& propState = propIt->second;
      if (!propState.dynamicTemplate) {
         continue;
      }

      if (!StyleVariableChangeAffectsConsumer(
            propState, varName, changedOwner, winningOwner)) {
```

```
            continue;
          }

          auto resolved = ResolveDynamicStyleValue(
            state, consumer.elementId, element, consumer.property,
            consumerFallbackClassName, &propState, &elementState);
          if (!resolved) {
            continue;
          }
          AdoptExternalValueAsOriginal(element, consumer.property,
                                       &propState);
          if (!propState.originalValue) {
            propState.originalValue =
              ReadLocalValueWithWorkaround(element, consumer.property);
          }
          propState.customValue = *resolved;

          bool wasModifying = g_elementPropertyModifying;
          g_elementPropertyModifying = true;
          propState.lastAppliedValue =
            SetOrClearValue(element, consumer.property, *resolved);
          g_elementPropertyModifying = wasModifying;
        }
      }
    }

    // Notify the styles that depend on `varName`. `changedOwner` names the capture
    // whose value changed, or is empty when captures were added or removed.
    //
    // Applying a style can realize children (running ApplyCustomizations, which
    // adds captures) or write a captured property (running a capture callback,
    // which g_elementPropertyModifying deliberately does not suppress), so this
    // re-enters. Nested calls queue instead of running, and the outermost frame
    // drains the queue, which also coalesces a burst into one pass.
    void PropagateStyleVariableChange(StyleVariableState* state,
                                      const std::wstring& varName,
                                      std::optional<ElementId> changedOwner) {
      PendingStyleVariablePropagation propagation{state, varName, changedOwner};

      if (g_styleVariablePropagationDepth > 0) {
        auto& pending = g_pendingStyleVariablePropagations;
        if (std::find(pending.begin(), pending.end(), propagation) ==
            pending.end()) {
          pending.push_back(std::move(propagation));
        }
        return;
      }

      StyleVariableStatePin statePin;

      struct DepthScope {
        DepthScope() { g_styleVariablePropagationDepth++; }
        ~DepthScope() { g_styleVariablePropagationDepth--; }
      } depthScope;
```

```
    PropagateStyleVariableChangeCore(state, varName, changedOwner);

    // A style that writes a property some rule captures keeps refilling the
    // queue. The unchanged-value fast path settles most such loops within a
    // round or two; a value that oscillates never settles, so give up loudly
    // instead of hanging the UI thread.
    constexpr int kMaxDrainRounds = 32;

    for (int round = 0; !g_pendingStyleVariablePropagations.empty(); round++) {
        if (round >= kMaxDrainRounds) {
            Wh_Log(
                L"Style variables did not settle after %d rounds; dropping %zu "
                L"queued update(s)",
                kMaxDrainRounds, g_pendingStyleVariablePropagations.size());
            g_pendingStyleVariablePropagations.clear();
            break;
        }

        auto pending = std::move(g_pendingStyleVariablePropagations);
        g_pendingStyleVariablePropagations.clear();
        for (const auto& pendingPropagation : pending) {
            PropagateStyleVariableChangeCore(pendingPropagation.state,
                                             pendingPropagation.varName,
                                             pendingPropagation.changedOwner);
        }
    }
}

// std::optional<double>'s operator== follows IEEE (NaN != NaN), which would
// report a NaN capture such as Height=Auto as changed on every re-read.
bool SameNumericValue(const std::optional<double>& a,
                      const std::optional<double>& b) {
    if (a.has_value() != b.has_value()) {
        return false;
    }
    return !a || *a == *b || (std::isnan(*a) && std::isnan(*b));
}

// Store a capture's freshly read value and notify dependents if it changed.
// The comparison is against this capture's own previous value: comparing
// against whichever capture currently wins would silently drop a second
// capturer's change whenever it happened to match. Used by every path that
// publishes a captured value -- the per-property capture callback and the
// SizeChanged catch-all -- so the no-op fast path applies uniformly.
void SetStyleVariableIfChangedAndPropagate(StyleVariableState* state,
                                           const std::wstring& varName,
                                           ElementId owner,
                                           StyleVariableValue value) {
    auto varIt = state->variables.find(varName);
    if (varIt == state->variables.end()) {
        return;
    }

    auto& captures = varIt->second;
    auto it = std::find_if(captures.begin(), captures.end(),
```

```
                    [owner](const StyleVariableCapture& capture) {
                      return capture.elementId == owner;
                    });
  if (it == captures.end()) {
    // The capture was torn down between the notification and here.
    return;
  }

  if (it->value.stringForm == value.stringForm &&
    SameNumericValue(it->value.numeric, value.numeric) &&
    it->value.substitutable == value.substitutable) {
    Wh_Log(L"Style variable '%s' unchanged at '%s'", varName.c_str(),
        value.stringForm.c_str());
    return;
  }

  Wh_Log(L"Style variable '%s' changed: '%s' -> '%s'", varName.c_str(),
      it->value.stringForm.c_str(), value.stringForm.c_str());
  it->value = std::move(value);
  PropagateStyleVariableChange(state, varName, owner);
}

// True for layout-driven DPs whose updates do not fire
// RegisterPropertyChangedCallback on UWP, so capture rules on those DPs need
// `FrameworkElement.SizeChanged` as their notification source instead.
bool IsLayoutDrivenSizeProperty(DependencyProperty property) {
  return property == FrameworkElement::ActualWidthProperty() ||
      property == FrameworkElement::ActualHeightProperty();
}

// Wire up `Property=>VarName` capture rules for an element. Called once per
// matched element (captures are not visual-state-aware). Seeds the variables
// from the current property values, registers per-DP property-changed
// callbacks, and -- because UWP's ActualWidth/ActualHeight don't fire those
// callbacks on layout -- subscribes to FrameworkElement.SizeChanged as a
// catch-all that re-reads every active capture on resize.
//
// Seeding writes the captured values into state->variables in a single batch
// (to avoid intermediate inconsistent states for consumers that depend on
// multiple variables from this element) and only then propagates. Every seeded
// name propagates, even one whose value matches an existing capture's: adding a
// capture changes which captures a consumer chooses between, so the consumers
// have to be re-scored regardless of the value. The function does not need the
// capturer's fallbackClassName: each StyleVariableConsumer entry already
// carries its own consumer-side fallback, so propagation routes through the
// right context per consumer.
void SetUpCapturesForElement(StyleVariableState* state,
                    ElementId elementId,
                    FrameworkElement element,
                    const std::vector<CaptureSpec>& captures,
                    ElementCustomizationState* elementState) {
  if (captures.empty()) {
    return;
  }
```

```
auto elementDo = element.as<DependencyObject>();
winrt::weak_ref<FrameworkElement> elementWeakRef = element;

// Names seeded below, propagated once the whole batch is in place.
std::vector<std::wstring> seededVarNames;
seededVarNames.reserve(captures.size());

// Captures whose source DP is layout-driven (ActualWidth/ActualHeight) need
// a SizeChanged subscription as their notification source. Collect them so
// we only subscribe once and only when needed.
std::vector<std::pair<DependencyProperty, std::wstring>>
    sizeChangedCaptures;

for (const auto& capture : captures) {
    const auto [it, inserted] =
        elementState->captureCustomizationStates.insert(
            {capture.property, {}});
    if (!inserted) {
        // Same DP captured twice on this element (different rules with the
        // same property); keep the first and warn so the dropped second is
        // not a silent footgun for users who later try to reference the
        // dropped variable in a `{{...}}` substitution.
        Wh_Log(
            L"Capture for property already registered on %s; "
            L"dropping duplicate variable '%s' (kept: '%s')",
            winrt::get_class_name(element).c_str(), capture.varName.c_str(),
            it->second.varName.c_str());
        continue;
    }
    auto& captureState = it->second;
    captureState.varName = capture.varName;

    auto value = ReadCapturedStyleVariableValue(element, capture.property);

    // No entry for this element can exist yet: the insert above rejects a
    // second capture of the same DP, and FindElementPropertyOverrides
    // rejects a second capture of the same name.
    auto& capturesForVar = state->variables[capture.varName];
    Wh_Log(
        L"Seeding capture variable '%s' from %s with value '%s' "
        L"(%zu other capture(s))",
        capture.varName.c_str(), winrt::get_class_name(element).c_str(),
        value.stringForm.c_str(), capturesForVar.size());
    capturesForVar.push_back({elementId, std::move(value)});
    AddStyleVariableElementRef(state, elementId);

    seededVarNames.push_back(capture.varName);

    if (IsLayoutDrivenSizeProperty(capture.property)) {
        sizeChangedCaptures.push_back({capture.property, capture.varName});
        // No property-changed callback: the DP doesn't fire one for layout
        // updates anyway, and SizeChanged below covers it.
        continue;
    }
```

```
      std::wstring varName = capture.varName;
      captureState.propertyChangedToken =
        elementDo.RegisterPropertyChangedCallback(
          capture.property,
          [state, varName, elementId, elementWeakRef](
            DependencyObject sender, DependencyProperty property) {
            auto element = elementWeakRef.get();
            if (!element) {
              return;
            }
            auto value =
              ReadCapturedStyleVariableValue(element, property);
            SetStyleVariableIfChangedAndPropagate(
              state, varName, elementId, std::move(value));
          });
    }

    if (!sizeChangedCaptures.empty()) {
      elementState->captureSizeChangedToken = element.SizeChanged(
        [state, elementId, elementWeakRef,
         sizeChangedCaptures = std::move(sizeChangedCaptures)](
          winrt::Windows::Foundation::IInspectable const& sender,
          SizeChangedEventArgs const& e) {
          auto element = elementWeakRef.get();
          if (!element) {
            return;
          }
          Wh_Log(L"SizeChanged on %s: %.3fx%.3f",
              winrt::get_class_name(element).c_str(),
              e.NewSize().Width, e.NewSize().Height);
          for (const auto& [property, varName] : sizeChangedCaptures) {
            auto value =
              ReadCapturedStyleVariableValue(element, property);
            SetStyleVariableIfChangedAndPropagate(
              state, varName, elementId, std::move(value));
          }
        });
    }

    // The new captures may be closer to consumers registered before this
    // element was matched than whatever they were reading.
    for (const auto& varName : seededVarNames) {
      PropagateStyleVariableChange(state, varName, std::nullopt);
    }
}

// Tear down capture subscriptions for an element. Called from
// CleanupCustomizations and UninitializeSettingsAndTap before the
// ElementCustomizationState entry is erased.
void RestoreCapturesForElement(FrameworkElement element,
                               const ElementCustomizationState& elementState) {
    if (!element) {
      return;
    }
```

```
    for (const auto& [property, captureState] :
         elementState.captureCustomizationStates) {
      if (!captureState.propertyChangedToken) {
        continue;
      }
      try {
        element.UnregisterPropertyChangedCallback(
          property, captureState.propertyChangedToken);
      } catch (winrt::hresult_error const& ex) {
        Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
      }
    }

    if (elementState.captureSizeChangedToken) {
      try {
        element.SizeChanged(elementState.captureSizeChangedToken);
      } catch (winrt::hresult_error const& ex) {
        Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
      }
    }
}

void ApplyCustomizationsForVisualStateGroup(
    StyleVariableState* state,
    ElementId elementId,
    FrameworkElement element,
    VisualStateGroup visualStateGroup,
    PCWSTR fallbackClassName,
    PropertyOverrides propertyOverrides,
    ElementCustomizationStateForVisualStateGroup*
      elementCustomizationStateForVisualStateGroup) {
    auto elementDo = element.as<DependencyObject>();

    VisualState currentVisualState(
      visualStateGroup ? visualStateGroup.CurrentState() : nullptr);

    std::wstring currentVisualStateName(
      currentVisualState ? currentVisualState.Name() : L"");

    for (const auto& [property, valuesPerVisualState] : propertyOverrides) {
      const auto [propertyCustomizationStatesIt, inserted] =
        elementCustomizationStateForVisualStateGroup
          ->propertyCustomizationStates.insert({property, {}});
      if (!inserted) {
        continue;
      }

      auto& propertyCustomizationState =
        propertyCustomizationStatesIt->second;

      auto it = valuesPerVisualState.find(currentVisualStateName);
      if (it == valuesPerVisualState.end() &&
        !currentVisualStateName.empty()) {
        it = valuesPerVisualState.find(L"");
      }
```

```
    if (it != valuesPerVisualState.end()) {
        std::optional<PropertyOverrideValue> resolved;
        if (auto* tmpl = std::get_if<DynamicStyleTemplate>(&it->second)) {
            propertyCustomizationState.dynamicTemplate = *tmpl;
            resolved = ResolveDynamicStyleValue(
                state, elementId, element, property, fallbackClassName,
                &propertyCustomizationState,
                /*elementCustomizationState=*/nullptr);
        } else {
            resolved = it->second;
        }

        if (resolved) {
            propertyCustomizationState.originalValue =
                ReadLocalValueWithWorkaround(element, property);
            propertyCustomizationState.customValue = *resolved;
            propertyCustomizationState.lastAppliedValue = SetOrClearValue(
                element, property, *resolved, /*initialApply=*/true);
        }
    }

    propertyCustomizationState.propertyChangedToken =
        elementDo.RegisterPropertyChangedCallback(
            property,
            [&propertyCustomizationState](DependencyObject sender,
                                          DependencyProperty property) {
                if (g_elementPropertyModifying) {
                    return;
                }

                auto element = sender.try_as<FrameworkElement>();
                if (!element) {
                    return;
                }

                if (!propertyCustomizationState.customValue) {
                    return;
                }

                AdoptExternalValueAsOriginal(element, property,
                                             &propertyCustomizationState);

                Wh_Log(L"Re-applying style for %s",
                       winrt::get_class_name(element).c_str());

                g_elementPropertyModifying = true;
                propertyCustomizationState.lastAppliedValue =
                    SetOrClearValue(
                        element, property,
                        *propertyCustomizationState.customValue);
                g_elementPropertyModifying = false;
            });
}
```

```
if (visualStateGroup) {
   winrt::weak_ref<FrameworkElement> elementWeakRef = element;
   std::wstring fallbackClassNameStr =
      fallbackClassName ? fallbackClassName : L"";
   elementCustomizationStateForVisualStateGroup
      ->visualStateGroupCurrentStateChangedToken =
      visualStateGroup.CurrentStateChanged(
         [state, elementWeakRef, propertyOverrides, elementId,
          fallbackClassNameStr,
          elementCustomizationStateForVisualStateGroup](
            winrt::Windows::Foundation::IInspectable const& sender,
            VisualStateChangedEventArgs const& e) {
            auto element = elementWeakRef.get();
            if (!element) {
               return;
            }

            Wh_Log(L"Re-applying all styles for %s",
                  winrt::get_class_name(element).c_str());

            g_elementPropertyModifying = true;

            auto& propertyCustomizationStates =
               elementCustomizationStateForVisualStateGroup
                  ->propertyCustomizationStates;

            PCWSTR fallbackClassNamePtr =
               fallbackClassNameStr.empty()
                  ? nullptr
                  : fallbackClassNameStr.c_str();

            for (const auto& [property, valuesPerVisualState] :
                 propertyOverrides) {
               auto& propertyCustomizationState =
                  propertyCustomizationStates.at(property);

               auto newState = e.NewState();
               auto newStateName =
                  std::wstring{newState ? newState.Name() : L""};
               auto it = valuesPerVisualState.find(newStateName);
               if (it == valuesPerVisualState.end()) {
                  it = valuesPerVisualState.find(L"");
                  if (it != valuesPerVisualState.end()) {
                     auto oldState = e.OldState();
                     auto oldStateName = std::wstring{
                        oldState ? oldState.Name() : L""};
                     if (!valuesPerVisualState.contains(
                           oldStateName)) {
                        continue;
                     }
                  }
               }

               if (it != valuesPerVisualState.end()) {
                  std::optional<PropertyOverrideValue> resolved;
```

```
    if (auto* tmpl = std::get_if<DynamicStyleTemplate>(
            &it->second)) {
        propertyCustomizationState.dynamicTemplate =
            *tmpl;
        resolved = ResolveDynamicStyleValue(
            state, elementId, element, property,
            fallbackClassNamePtr,
            &propertyCustomizationState,
            /*elementCustomizationState=*/nullptr);
    } else {
        // Transitioning from dynamic to static for this
        // visual state: clear template metadata and
        // unregister consumer entries.
        if (propertyCustomizationState
                .dynamicTemplate) {
            UpdateStyleVariableConsumers(
                state, elementId, property,
                /*fallbackClassName=*/nullptr,
                propertyCustomizationState
                    .variableDependencies,
                {});
            propertyCustomizationState
                .variableDependencies.clear();
            propertyCustomizationState.dynamicTemplate
                .reset();
        }

        resolved = it->second;
    }

    if (resolved) {
        AdoptExternalValueAsOriginal(
            element, property,
            &propertyCustomizationState);
        if (!propertyCustomizationState.originalValue) {
            propertyCustomizationState.originalValue =
                ReadLocalValueWithWorkaround(element,
                                             property);
        }

        propertyCustomizationState.customValue =
            *resolved;
        propertyCustomizationState.lastAppliedValue =
            SetOrClearValue(element, property,
                            *resolved);
    }
} else {
    if (propertyCustomizationState.dynamicTemplate) {
        UpdateStyleVariableConsumers(
            state, elementId, property,
            /*fallbackClassName=*/nullptr,
            propertyCustomizationState
                .variableDependencies,
            {});
        propertyCustomizationState.variableDependencies
```

```
                        .clear();
                    propertyCustomizationState.dynamicTemplate
                        .reset();
                }
                UnapplyStyleValue(element, property,
                                  &propertyCustomizationState);
              }
            }

            g_elementPropertyModifying = false;
          });
  }
}

void RestoreCustomizationsForVisualStateGroup(
  StyleVariableState* state,
  ElementId elementId,
  FrameworkElement element,
  std::optional<winrt::weak_ref<VisualStateGroup>>
    visualStateGroupOptionalWeakPtr,
  const ElementCustomizationStateForVisualStateGroup&
    elementCustomizationStateForVisualStateGroup) {
  if (element) {
    for (const auto& [property, propState] :
         elementCustomizationStateForVisualStateGroup
           .propertyCustomizationStates) {
      try {
        element.UnregisterPropertyChangedCallback(
          property, propState.propertyChangedToken);
      } catch (winrt::hresult_error const& ex) {
        Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
      }

      if (!propState.variableDependencies.empty()) {
        UpdateStyleVariableConsumers(state, elementId, property,
                                     /*fallbackClassName=*/nullptr,
                                     propState.variableDependencies,
                                     {});
      }

      if (propState.originalValue) {
        SetOrClearValue(element, property, *propState.originalValue);
      }
    }
  } else {
    // Element is gone; still clear consumer entries so a stale (elementId,
    // property) pair isn't visited during PropagateStyleVariableChange.
    for (const auto& [property, propState] :
         elementCustomizationStateForVisualStateGroup
           .propertyCustomizationStates) {
      if (!propState.variableDependencies.empty()) {
        UpdateStyleVariableConsumers(state, elementId, property,
                                     /*fallbackClassName=*/nullptr,
                                     propState.variableDependencies,
                                     {});
```

```
        }
      }
    }

    auto visualStateGroupIter = visualStateGroupOptionalWeakPtr
                              ? visualStateGroupOptionalWeakPtr->get()
                              : nullptr;
    if (visualStateGroupIter && elementCustomizationStateForVisualStateGroup
                              .visualStateGroupCurrentStateChangedToken) {
      try {
        visualStateGroupIter.CurrentStateChanged(
          elementCustomizationStateForVisualStateGroup
            .visualStateGroupCurrentStateChangedToken);
      } catch (winrt::hresult_error const& ex) {
        Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
      }
    }
}

// Quantize a size to the nearest 0.5px so signatures and comparisons aren't
// thrown off by sub-pixel jitter.
long long QuantizeLayoutSize(double value) {
    if (value < 0 || std::isnan(value)) {
        return 0;
    }
    return static_cast<long long>(value * 2 + 0.5);
}

bool IsTaskbarTopLevelWindow(HWND hWnd) {
    WCHAR className[32];
    if (!GetClassName(hWnd, className, ARRAYSIZE(className))) {
        return false;
    }

    return _wcsicmp(className, L"Shell_TrayWnd") == 0 ||
           _wcsicmp(className, L"Shell_SecondaryTrayWnd") == 0;
}

// Click-through taskbar: clip the top-level taskbar window to the union of the
// TaskbarFrame and SystemTrayFrame rects so the empty areas become both
// invisible and click-through.
void UpdateClickThroughRegion(ClickThroughTaskbarState& state) {
    auto taskbarFrame = state.taskbarFrame.get();
    if (!taskbarFrame) {
        Wh_Log(L"No live TaskbarFrame");
        return;
    }

    XamlRoot xamlRoot = nullptr;
    try {
        xamlRoot = taskbarFrame.XamlRoot();
    } catch (...) {
        Wh_Log(L"Error %08X: %s", winrt::to_hresult(),
               winrt::to_message().c_str());
    }
```

```
if (!xamlRoot) {
    Wh_Log(L"No XamlRoot for TaskbarFrame");
    return;
}

// Resolve the island native window lazily. A freshly created island reports
// its root before its content/XamlRoot is ready, so the association can't
// be made at island-add time; by the time the frames lay out it can.
if (!state.islandHwnd) {
    state.islandHwnd = ResolveClickThroughIslandHwnd(xamlRoot);
}
if (!state.islandHwnd) {
    Wh_Log(L"Island window not resolved yet");
    return;
}

// XamlRoot.RasterizationScale reports a scale factor snapped to one of the
// standard values, so with a custom display scaling level (e.g. 110%) it
// doesn't match the DIP-to-pixel ratio the island is laid out with, and the
// region comes out too small. The island window's DPI gives the actual
// ratio; RasterizationScale is only a fallback.
double rasterizationScale = xamlRoot.RasterizationScale();
UINT islandDpi = GetDpiForWindow(state.islandHwnd);
double scale = islandDpi ? islandDpi / 96.0 : rasterizationScale;
if (scale <= 0) {
    Wh_Log(L"Invalid scale %f", scale);
    return;
}

auto content = xamlRoot.Content();
if (!content) {
    Wh_Log(L"No XamlRoot content");
    return;
}

HWND topLevelWnd = GetAncestor(state.islandHwnd, GA_ROOT);
if (!topLevelWnd || !IsTaskbarTopLevelWindow(topLevelWnd)) {
    Wh_Log(L"Island %08X is not under a taskbar window",
        (DWORD)(ULONG_PTR)state.islandHwnd);
    return;
}

// Bounds of a frame relative to the island root, in DIPs.
auto frameRect = [&content](FrameworkElement const& frame)
    -> std::optional<winrt::Windows::Foundation::Rect> {
    if (!frame) {
        return std::nullopt;
    }
    auto rect = frame.TransformToVisual(content).TransformBounds(
        {0, 0, static_cast<float>(frame.ActualWidth()),
         static_cast<float>(frame.ActualHeight())});
    if (rect.Width <= 0 || rect.Height <= 0) {
        return std::nullopt;
    }
    return rect;
```

```
};

auto taskbarFrameRect = frameRect(taskbarFrame);
if (!taskbarFrameRect) {
   // Without the taskbar frame there's nothing meaningful to keep, and an
   // empty region would hide the whole taskbar. Retry on the next pass.
   Wh_Log(L"TaskbarFrame not laid out yet");
   return;
}

auto systemTrayFrameRect = frameRect(state.systemTrayFrame.get());

// LayoutUpdated is frequent; only re-apply when the result actually
// changes.
auto appendRect =
   [](std::vector<long long>& sig,
     std::optional<winrt::Windows::Foundation::Rect> const& rect) {
      if (rect) {
         sig.push_back(QuantizeLayoutSize(rect->X));
         sig.push_back(QuantizeLayoutSize(rect->Y));
         sig.push_back(QuantizeLayoutSize(rect->Width));
         sig.push_back(QuantizeLayoutSize(rect->Height));
      } else {
         sig.push_back(-1);
      }
   };

std::vector<long long> signature;
signature.push_back(reinterpret_cast<long long>(topLevelWnd));
signature.push_back(QuantizeLayoutSize(scale * 100));
appendRect(signature, taskbarFrameRect);
appendRect(signature, systemTrayFrameRect);

RECT tlRect;
if (!GetWindowRect(topLevelWnd, &tlRect)) {
   Wh_Log(L"GetWindowRect failed for %08X", (DWORD)(ULONG_PTR)topLevelWnd);
   return;
}

int tlWidth = tlRect.right - tlRect.left;
int tlHeight = tlRect.bottom - tlRect.top;

RECT currentRgnBox;
bool hasRgn = GetWindowRgnBox(topLevelWnd, &currentRgnBox) != ERROR;
auto appliedRgnBoxIt = g_clickThroughAppliedRgnBoxes.find(topLevelWnd);
bool rgnIsOurs = hasRgn &&
            appliedRgnBoxIt != g_clickThroughAppliedRgnBoxes.end() &&
            EqualRect(&currentRgnBox, &appliedRgnBoxIt->second);

// Explorer clips the taskbar window through the very same window region:
// TaskbarController::UpdateHostWindowClip narrows it to the collapsed
// extent while the tablet taskbar is folded, and that narrowed window is
// what keeps the fold from being expanded by a pointer anywhere along the
// taskbar. Our region can't stand in for it, since the frames stay laid out
// at the expanded size (the fold is a composition transform), so replacing
```

```
// it would widen the hover area rather than narrow it. Only take the window
// over when nothing else is clipping it: no region, or the full-window one
// Explorer sets when it isn't narrowing anything.
if (hasRgn && !rgnIsOurs) {
   RECT fullRgnBox = {0, 0, tlWidth, tlHeight};
   if (!EqualRect(&currentRgnBox, &fullRgnBox)) {
      Wh_Log(L"Leaving the region of %08X to its owner",
          (DWORD)(ULONG_PTR)topLevelWnd);
      return;
   }
}

// Skip the redundant SetWindowRgn (and the redraw it forces) only when the
// desired region is unchanged AND the window still carries the region we
// applied. Toggling auto-hide makes Explorer reset the taskbar window
// region without any XAML layout change, so a matching signature alone
// doesn't prove our region is still in effect.
if (signature == state.lastRegionSignature && rgnIsOurs) {
   return;
}

// Convert DIP rects to physical pixels in top-level window coordinates. The
// island client origin and the window rect are both physical/screen pixels,
// so multiplying DIPs by the rasterization scale keeps everything
// DPI-consistent.
POINT islandOrigin = {0, 0};
ClientToScreen(state.islandHwnd, &islandOrigin);

auto toWindowRect =
   [&](winrt::Windows::Foundation::Rect const& dip) -> RECT {
   int left = (islandOrigin.x - tlRect.left) + std::lround(dip.X * scale);
   int top = (islandOrigin.y - tlRect.top) + std::lround(dip.Y * scale);
   int right = left + std::lround(dip.Width * scale);
   int bottom = top + std::lround(dip.Height * scale);
   return RECT{std::clamp(left, 0, tlWidth), std::clamp(top, 0, tlHeight),
            std::clamp(right, 0, tlWidth),
            std::clamp(bottom, 0, tlHeight)};
};

RECT tf = toWindowRect(*taskbarFrameRect);
HRGN rgn = CreateRectRgn(tf.left, tf.top, tf.right, tf.bottom);
if (!rgn) {
   Wh_Log(L"CreateRectRgn failed");
   return;
}

RECT rgnBox = tf;

if (systemTrayFrameRect) {
   RECT st = toWindowRect(*systemTrayFrameRect);
   if (HRGN trayRgn =
         CreateRectRgn(st.left, st.top, st.right, st.bottom)) {
      CombineRgn(rgn, rgn, trayRgn, RGN_OR);
      DeleteObject(trayRgn);
      UnionRect(&rgnBox, &rgnBox, &st);
```

```
        }
    }

    Wh_Log(L"Applying region to %08X, dpi=%u, rasterization scale=%f",
        (DWORD)(ULONG_PTR)topLevelWnd, islandDpi, rasterizationScale);

    // SetWindowRgn takes ownership of the region on success. Record what was
    // applied only then, so a failed apply is retried on the next pass. The
    // call sends WM_WINDOWPOSCHANGED synchronously; the guard stops the
    // subclass from reentering and reapplying mid-call.
    g_applyingClickThroughRegion = true;
    BOOL applied = SetWindowRgn(topLevelWnd, rgn, TRUE);
    g_applyingClickThroughRegion = false;
    if (applied) {
        state.lastRegionSignature = std::move(signature);
        g_clickThroughAppliedRgnBoxes[topLevelWnd] = rgnBox;
    } else {
        Wh_Log(L"SetWindowRgn failed for %08X", (DWORD)(ULONG_PTR)topLevelWnd);
        DeleteObject(rgn);
    }
}

// Reapply the click-through clip for the taskbar window rooted at topLevelWnd.
// UpdateClickThroughRegion is a no-op when the clip is already correct, so this
// is cheap to call on every window move.
void ReapplyClickThroughForTopLevel(HWND topLevelWnd) {
    for (auto& entry : g_clickThroughTaskbarState) {
        if (entry.islandHwnd &&
            GetAncestor(entry.islandHwnd, GA_ROOT) == topLevelWnd) {
            UpdateClickThroughRegion(entry);
        }
    }
}

LRESULT CALLBACK ClickThroughTaskbarSubclassProc(HWND hWnd,
                                                 UINT uMsg,
                                                 WPARAM wParam,
                                                 LPARAM lParam,
                                                 DWORD_PTR) {
    switch (uMsg) {
        case WM_WINDOWPOSCHANGED: {
            // The taskbar moved or was shown - e.g. an auto-hidden taskbar
            // sliding into view, or auto-hide being toggled. Explorer resets
            // the window region across these transitions, so reapply the clip
            // now rather than waiting for the next XAML layout pass. Ignore the
            // notification our own SetWindowRgn raises, to avoid reentering the
            // region update.
            LRESULT result = DefSubclassProc(hWnd, uMsg, wParam, lParam);
            if (!g_applyingClickThroughRegion) {
                ReapplyClickThroughForTopLevel(hWnd);
            }
            return result;
        }

        case WM_NCDESTROY:
```

```
            g_clickThroughSubclassedWindows.erase(hWnd);
            g_clickThroughAppliedRgnBoxes.erase(hWnd);
            break;
        }

        return DefSubclassProc(hWnd, uMsg, wParam, lParam);
    }

    // Subclass the taskbar's top-level window so the clip is reapplied as soon as
    // the window moves or is shown (e.g. an auto-hidden taskbar sliding into view),
    // which Explorer accompanies by resetting the window region. Idempotent; the
    // top-level window is resolved from the island once it is known.
    void EnsureClickThroughSubclass(ClickThroughTaskbarState& state) {
        if (!state.islandHwnd) {
            return;
        }

        HWND topLevelWnd = GetAncestor(state.islandHwnd, GA_ROOT);
        if (!topLevelWnd || !IsTaskbarTopLevelWindow(topLevelWnd)) {
            return;
        }

        if (g_clickThroughSubclassedWindows.contains(topLevelWnd)) {
            return;
        }

        if (WindhawkUtils::SetWindowSubclassFromAnyThread(
                topLevelWnd, ClickThroughTaskbarSubclassProc, 0)) {
            g_clickThroughSubclassedWindows.insert(topLevelWnd);
        }
    }

    void HandleClickThroughElement(FrameworkElement element) {
        auto className = winrt::get_class_name(element);

        bool isTaskbarFrame = className == L"Taskbar.TaskbarFrame";
        bool isSystemTrayFrame = className == L"SystemTray.SystemTrayFrame";
        if (!isTaskbarFrame && !isSystemTrayFrame) {
            return;
        }

        XamlRoot xamlRoot = nullptr;
        try {
            xamlRoot = element.XamlRoot();
        } catch (...) {
            Wh_Log(L"Error %08X: %s", winrt::to_hresult(),
                winrt::to_message().c_str());
        }
        if (!xamlRoot) {
            Wh_Log(L"No XamlRoot for %s", className.c_str());
            return;
        }

        auto* state = GetClickThroughState(xamlRoot);
        if (!state) {
```

```
        Wh_Log(L"No state for %s", className.c_str());
        return;
    }

    if (isTaskbarFrame) {
        state->taskbarFrame = element;
    } else {
        state->systemTrayFrame = element;
    }

    // Hook LayoutUpdated once per XamlRoot, on the taskbar frame (always
    // present; its layout passes also cover system tray changes in the same
    // tree).
    if (isTaskbarFrame && !state->layoutUpdatedRevoker) {
        auto weakXamlRoot = winrt::make_weak(xamlRoot);
        state->layoutUpdatedRevoker = element.LayoutUpdated(
            winrt::auto_revoke,
            [weakXamlRoot](winrt::Windows::Foundation::IInspectable const&,
                           winrt::Windows::Foundation::IInspectable const&) {
                auto strongXamlRoot = weakXamlRoot.get();
                if (!strongXamlRoot) {
                    return;
                }
                if (auto* state = GetClickThroughState(strongXamlRoot)) {
                    UpdateClickThroughRegion(*state);
                    EnsureClickThroughSubclass(*state);
                }
            });
    }

    UpdateClickThroughRegion(*state);
    EnsureClickThroughSubclass(*state);
}

// The XAML island root reports as a DesktopWindowXamlSource (not a
// FrameworkElement), so it bypasses ApplyCustomizations. Track it here so its
// native window can be matched to a taskbar's XamlRoot once the frames lay out
// (see ResolveClickThroughIslandHwnd). Resolving the HWND from the root is how
// UWPSpy handles system XAML islands.
void HandleClickThroughIslandRoot(
    winrt::Windows::Foundation::IInspectable const& inspectable) {
    if (!g_settings.clickThroughTaskbar) {
        return;
    }

    auto source = inspectable.try_as<wuxh::DesktopWindowXamlSource>();
    if (!source) {
        return;
    }

    HWND islandHwnd = nullptr;
    if (auto native = inspectable.try_as<IDesktopWindowXamlSourceNative>()) {
        native->get_WindowHandle(&islandHwnd);
    }
    if (!islandHwnd) {
```

```
        Wh_Log(L"Failed to get window handle for island root");
        return;
    }

    // Reap dead entries and replace any with this same window, then track the
    // island.
    g_clickThroughIslandRoots.remove_if(
        [&](ClickThroughIslandRoot const& entry) {
            return !entry.source.get() || entry.islandHwnd == islandHwnd;
        });
    try {
        auto weakSource = winrt::make_weak(source);
        auto& entry = g_clickThroughIslandRoots.emplace_back();
        entry.source = std::move(weakSource);
        entry.islandHwnd = islandHwnd;
    } catch (...) {
        Wh_Log(L"Error %08X: %s", winrt::to_hresult(),
            winrt::to_message().c_str());
    }
}

// Remove the click-through regions applied by the mod from the taskbar windows
// on the current thread, restoring the normal rectangular windows. A region
// other than the one recorded as applied is Explorer's own clip (see
// UpdateClickThroughRegion) and is left in place.
void ClearClickThroughRegions() {
    for (const auto& [topLevelWnd, appliedRgnBox] :
         g_clickThroughAppliedRgnBoxes) {
        // Stale if the window was destroyed with no subclass attached to drop
        // the record; the handle may have been reused since.
        if (!IsTaskbarTopLevelWindow(topLevelWnd)) {
            continue;
        }

        RECT currentRgnBox;
        if (GetWindowRgnBox(topLevelWnd, &currentRgnBox) == ERROR ||
            !EqualRect(&currentRgnBox, &appliedRgnBox)) {
            continue;
        }

        SetWindowRgn(topLevelWnd, nullptr, TRUE);
    }
    g_clickThroughAppliedRgnBoxes.clear();
}

// Explorer never erases the taskbar window: Shell_TrayWnd,
// Shell_SecondaryTrayWnd and their legacy children all use a NULL class brush,
// and nothing paints into them, so whatever DWM leaves in the window's
// redirection surface stays there for the window's lifetime. The surface is
// normally all zeros, which composites as fully transparent, but a
// reallocation - a mode or display change, a fullscreen transition - can leave
// opaque white in it. An opaque taskbar background hides that, a see-through
// style doesn't, and it shows as white boxes behind the taskbar.
//
// Filling the window black puts the surface back to zeros: GDI leaves the
```

```
// alpha channel at zero, so the fill composites as transparent rather than as
// an opaque black bar. Invalidating instead does nothing, since the NULL brush
// means the erase draws nothing over the leftovers. The DC must come from
// GetDCEx without DCX_CLIPCHILDREN, or the fill is clipped away entirely - the
// XAML island covers the whole window, and the leftovers sit underneath it.
void ResetTaskbarWindowSurface(HWND hWnd) {
    RECT rect;
    if (!GetWindowRect(hWnd, &rect)) {
        return;
    }

    HDC hdc = GetDCEx(hWnd, nullptr, DCX_WINDOW | DCX_CACHE);
    if (!hdc) {
        Wh_Log(L"GetDCEx failed for %08X", (DWORD)(ULONG_PTR)hWnd);
        return;
    }

    RECT fillRect = {0, 0, rect.right - rect.left, rect.bottom - rect.top};
    FillRect(hdc, &fillRect, (HBRUSH)GetStockObject(BLACK_BRUSH));

    ReleaseDC(hWnd, hdc);
}

thread_local std::unordered_set<HWND> g_taskbarSurfaceSubclassedWindows;

// A reallocation invalidates the window, so the fill rides the paint which
// follows it. The fill can't be left to that paint: every DC it hands out is
// clipped to what the child windows leave uncovered, and the XAML island
// leaves nothing.
LRESULT CALLBACK TaskbarSurfaceSubclassProc(HWND hWnd,
                                            UINT uMsg,
                                            WPARAM wParam,
                                            LPARAM lParam,
                                            DWORD_PTR) {
    switch (uMsg) {
        case WM_PAINT:
            ResetTaskbarWindowSurface(hWnd);
            break;

        case WM_NCDESTROY:
            g_taskbarSurfaceSubclassedWindows.erase(hWnd);
            break;
    }

    return DefSubclassProc(hWnd, uMsg, wParam, lParam);
}

// Idempotent, so a window reached both by creation and by the sweep below is
// only subclassed once.
void EnsureTaskbarSurfaceSubclass(HWND hWnd) {
    if (g_taskbarSurfaceSubclassedWindows.contains(hWnd)) {
        return;
    }

    if (!WindhawkUtils::SetWindowSubclassFromAnyThread(
```

```
        hWnd, TaskbarSurfaceSubclassProc, 0)) {
      return;
    }

    g_taskbarSurfaceSubclassedWindows.insert(hWnd);

    // Leftovers already in the surface don't invalidate the window, so no
    // paint would come to fill on.
    ResetTaskbarWindowSurface(hWnd);
}

// Covers the taskbars which already exist when the mod is enabled. Later ones
// are subclassed as they are created.
void EnsureTaskbarSurfaceSubclasses() {
    EnumThreadWindows(
        GetCurrentThreadId(),
        [](HWND hWnd, LPARAM) -> BOOL {
            if (IsTaskbarTopLevelWindow(hWnd)) {
                EnsureTaskbarSurfaceSubclass(hWnd);
            }

            return TRUE;
        },
        0);
}

// Item elements the current virtualization pass recycled, consumed by the
// matching ElementPrepared. A freshly created element is prepared without ever
// having been cleared, and its Add mutation already applied the styles, so
// re-matching it would restore and re-push every value for nothing.
thread_local std::unordered_set<ElementId> g_recycledElements;

// The item each element was last matched against, which lets a reuse for that
// same item be left alone. A layout can realize elements during measure only to
// ask for their size and recycle them again during arrange, so an element is
// cleared and handed straight back for the same item on every layout pass.
// Re-matching there sets dependency properties from inside the pass, which
// dirties layout and schedules another one, and layout never settles: XAML
// gives up after enough passes and fails the process with a layout cycle.
thread_local std::unordered_map<ElementId, winrt::weak_ref<wf::IInspectable>>
    g_elementMatchedItems;

struct VirtualizingRepeaterState {
    muxc::ItemsRepeater::ElementClearing_revoker elementClearingRevoker;
    muxc::ItemsRepeater::ElementPrepared_revoker elementPreparedRevoker;
};

thread_local std::unordered_map<ElementId, VirtualizingRepeaterState>
    g_virtualizingRepeaters;

// The id of an element which was reached some other way, e.g. by walking the
// visual tree. None for an element the mutation callbacks never reported, which
// leaves callers to skip it rather than key it by something made up.
ElementId ElementIdFromElement(FrameworkElement const& element) {
    if (!element) {
```

```
        return ElementId::None;
    }

    try {
        auto it = g_elementIds.find(HandleFromInspectable(element));
        if (it == g_elementIds.end() || it->second.element.get() != element) {
            return ElementId::None;
        }

        return it->second.id;
    } catch (winrt::hresult_error const& ex) {
        Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
        return ElementId::None;
    }
}

// Tear down and re-match every element of a subtree. The whole subtree is
// revisited rather than only its root, since a rule can match a descendant
// through a condition on an ancestor, and descendants of a reused element get
// no mutation of their own.
void ReapplyCustomizationsForSubtree(FrameworkElement element) {
    // Caught per element, both because these run from a layout pass the caller
    // can't fail, and so that one element's failure doesn't skip the rest of
    // the subtree.
    try {
        if (auto elementId = ElementIdFromElement(element);
            elementId != ElementId::None) {
            CleanupCustomizations(elementId);
            auto className = winrt::get_class_name(element);
            ApplyCustomizations(elementId, element, className.c_str());
        }
    } catch (winrt::hresult_error const& ex) {
        Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
    }

    // Snapshotted because applying a style runs arbitrary XAML work which can
    // change the children collection mid-walk.
    std::vector<FrameworkElement> children;
    try {
        int count = Media::VisualTreeHelper::GetChildrenCount(element);
        for (int i = 0; i < count; i++) {
            if (auto child = Media::VisualTreeHelper::GetChild(element, i)
                                 .try_as<FrameworkElement>()) {
                children.push_back(std::move(child));
            }
        }
    } catch (winrt::hresult_error const& ex) {
        Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
        return;
    }

    for (const auto& child : children) {
        ReapplyCustomizationsForSubtree(child);
    }
}
```

```
// A weak reference to the item a repeater realized an element for, empty when
// there is no such item or it supports no weak reference. Weak so that a
// destroyed item can't be mistaken for a successor at the same address, which
// would leave an element wearing the styles matched for its predecessor.
winrt::weak_ref<wf::IInspectable> RepeaterItemAt(
    muxc::ItemsRepeater const& repeater,
    int index) {
    try {
        auto itemsSourceView = repeater.ItemsSourceView();
        if (!itemsSourceView || index < 0 || index >= itemsSourceView.Count()) {
            return nullptr;
        }

        return TryMakeWeak(itemsSourceView.GetAt(index));
    } catch (winrt::hresult_error const& ex) {
        Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
        return nullptr;
    }
}

// A virtualizing container recycles its item elements instead of destroying
// them: the recycle pool collapses the element and leaves it parented, so XAML
// diagnostics reports no Remove/Add mutation and the styles matched for the
// previous item would stay on the element once it's reused for another one.
// Treat a cleared element as removed and a prepared one as newly added.
void HandleVirtualizingRepeater(ElementId elementId, FrameworkElement element) {
    auto repeater = element.try_as<muxc::ItemsRepeater>();
    if (!repeater || g_virtualizingRepeaters.contains(elementId)) {
        return;
    }

    Wh_Log(L"Tracking recycling of %s", winrt::get_class_name(element).c_str());

    auto& state = g_virtualizingRepeaters[elementId];

    state.elementClearingRevoker = repeater.ElementClearing(
        winrt::auto_revoke,
        [](muxc::ItemsRepeater const&,
           muxc::ItemsRepeaterElementClearingEventArgs const& args) {
            auto element = args.Element().try_as<FrameworkElement>();
            if (!element) {
                return;
            }

            auto elementId = ElementIdFromElement(element);
            if (elementId == ElementId::None) {
                return;
            }

            Wh_Log(L"Element cleared: %llu", static_cast<uint64_t>(elementId));

            // Nothing is restored here. Whether the styles still apply depends
            // on the item the element is handed back for, which only the
            // matching ElementPrepared knows, and until then they sit on an
```

```
        // element the recycle pool keeps out of sight.
        g_recycledElements.insert(elementId);
      });

    state.elementPreparedRevoker = repeater.ElementPrepared(
      winrt::auto_revoke,
      [](muxc::ItemsRepeater const& sender,
        muxc::ItemsRepeaterElementPreparedEventArgs const& args) {
        auto element = args.Element().try_as<FrameworkElement>();
        if (!element) {
          return;
        }

        auto elementId = ElementIdFromElement(element);
        if (elementId == ElementId::None) {
          return;
        }

        auto item = RepeaterItemAt(sender, args.Index());

        // Held across the walk below, so that an item which no weak
        // reference can get back, such as one a source boxes anew on every
        // read, is never recorded: an entry which could match nothing would
        // keep the element held for good.
        auto strongItem = item.get();

        if (!g_recycledElements.erase(elementId)) {
          // Freshly created, so the styles its Add mutation applied are
          // the ones for this item, and only the item is recorded.
          if (strongItem) {
            g_elementMatchedItems[elementId] = std::move(item);
          }
          return;
        }

        if (strongItem) {
          auto it = g_elementMatchedItems.find(elementId);
          if (it != g_elementMatchedItems.end() &&
            it->second.get() == strongItem) {
            Wh_Log(L"Element reused for the same item: %llu",
                static_cast<uint64_t>(elementId));
            return;
          }
        }

        Wh_Log(L"Element reused: %llu", static_cast<uint64_t>(elementId));

        ReapplyCustomizationsForSubtree(element);

        // After the walk, which erases the entry as part of the teardown.
        if (strongItem) {
          g_elementMatchedItems[elementId] = std::move(item);
        }
      });
}
```

```
void MergeResourceVariables();

void ApplyCustomizations(ElementId elementId,
                         FrameworkElement element,
                         PCWSTR fallbackClassName) {
   // Merge resource dictionary on first element add. Merging it earlier on
   // window creation doesn't work, perhaps merged dictionaries are reset
   // during initialization.
   if (!g_resourceVariablesThemeDict) {
      MergeResourceVariables();
   }

   // Handle click-through before the no-customizations early return below,
   // since it must run for the taskbar elements even with no styles
   // configured.
   if (g_settings.clickThroughTaskbar) {
      HandleClickThroughElement(element);
   }

   // Before the early return below: a repeater rarely has styles of its own,
   // but its item elements do.
   HandleVirtualizingRepeater(elementId, element);

   // Everything below holds `state` across calls that run arbitrary XAML work
   // and can re-enter, so keep the entry from being reaped underneath it.
   StyleVariableStatePin statePin;

   auto* state = GetStyleVariableState(element);
   if (!state) {
      Wh_Log(L"No XamlRoot for %s, skipping",
           winrt::get_class_name(element).c_str());
      return;
   }

   auto resolved = FindElementPropertyOverrides(element, fallbackClassName);
   if (resolved.overridesPerVSG.empty() && resolved.captures.empty()) {
      return;
   }

   Wh_Log(L"Applying styles to %s", winrt::get_class_name(element).c_str());

   auto& elementCustomizationState = g_elementsCustomizationState[elementId];

   for (const auto& [visualStateGroupOptionalWeakPtrIter, stateIter] :
       elementCustomizationState.perVisualStateGroup) {
      RestoreCustomizationsForVisualStateGroup(
         state, elementId, element, visualStateGroupOptionalWeakPtrIter,
         stateIter);
   }

   elementCustomizationState.element = element;
   elementCustomizationState.xamlRoot = state->xamlRoot;
   elementCustomizationState.perVisualStateGroup.clear();
```

```
   // Elements that neither capture nor consume a variable pay nothing. The
   // rest get their spine now that the element has been matched; if it isn't
   // attached yet the spine stops at a placeholder root, which
   // EnsureElementTreeNode rebuilds on first use once the element is actually
   // in the tree. Cleared unconditionally so a re-apply that drops all
   // variable use cannot leave a stale node behind.
   elementCustomizationState.treeNode = nullptr;
   if (!resolved.captures.empty() || resolved.hasDynamicValues) {
      elementCustomizationState.treeNode =
         GetOrCreateElementTreeNode(element);
   }

   // Wire up captures first so any variables they define are visible to
   // dynamic value-rules applied below. Note: SetUpCapturesForElement does not
   // need this element's fallbackClassName -- propagation routes through each
   // consumer's own stored fallback.
   SetUpCapturesForElement(state, elementId, element, resolved.captures,
                           &elementCustomizationState);

   for (auto& [visualStateGroup, overridesForVisualStateGroup] :
        resolved.overridesPerVSG) {
      std::optional<winrt::weak_ref<VisualStateGroup>>
         visualStateGroupOptionalWeakPtr;
      if (visualStateGroup) {
         visualStateGroupOptionalWeakPtr = visualStateGroup;
      }

      elementCustomizationState.perVisualStateGroup.push_back(
         {visualStateGroupOptionalWeakPtr, {}});
      auto* elementCustomizationStateForVisualStateGroup =
         &elementCustomizationState.perVisualStateGroup.back().second;

      ApplyCustomizationsForVisualStateGroup(
         state, elementId, element, visualStateGroup, fallbackClassName,
         std::move(overridesForVisualStateGroup),
         elementCustomizationStateForVisualStateGroup);
   }
}

// The diagnostics create a runtime object which holds every element they
// report, and drop it only once the element is reported as removed. Removals
// are reported for elements taken out of their parent, which is how a taskbar
// button or a recycled list item is released, but a tree which is discarded
// whole, Task View for example, is never taken apart that way: dropping its
// root would destroy it. Holding every element is what stops that destruction,
// so nothing is ever removed and nothing is ever reported. Elements nothing is
// keyed by are handed back from here so the teardown can happen.
//
// Reporting an element recreates the runtime object of its parent, which is why
// each report queues the parent as well, and why the queue is drained only once
// the burst of reports has stopped: releasing mid-burst would just be undone by
// the next child of whatever was released.
thread_local std::vector<InstanceHandle> g_pendingDiagnosticsRelease;
thread_local ULONGLONG g_lastDiagnosticsReleaseQueueTick;
thread_local bool g_diagnosticsReleaseDrainQueued;
```

```
thread_local winrt::Windows::System::DispatcherQueueTimer
   g_diagnosticsReleaseDrainTimer{nullptr};
thread_local winrt::Windows::System::DispatcherQueueTimer::Tick_revoker
   g_diagnosticsReleaseDrainTimerTickRevoker;

// Long enough to sit out a tree being built.
constexpr ULONGLONG kDiagnosticsReleaseDelay = 200;

// The drain waits on a one-shot timer, which the thread teardown can stop,
// rather than on a dispatcher item, which it cannot: the module is freed once
// the mod is uninitialized, and an item still on the dispatcher would call into
// it. The interval only has to carry the drain out of the report which arms it.
constexpr ULONGLONG kDiagnosticsReleaseDrainDelay = 1;

// Releasing an element the mod still records something for would strand that
// recording, since no removal is reported for a handle whose runtime object is
// gone. Such an element stays held, and its own removal releases it.
//
// Which leaves a styled element of a tree discarded whole held for good, and
// with it everything below it, since a parent holds its children. Rules that
// match the taskbar match almost nothing of such a tree, so what this retains
// is small, but a rule written against a bare type would retain much more.
bool ElementHasState(ElementId elementId) {
   if (elementId == ElementId::None) {
      return false;
   }

   if (g_elementsCustomizationState.contains(elementId) ||
      g_virtualizingRepeaters.contains(elementId) ||
      g_recycledElements.contains(elementId) ||
      g_elementMatchedItems.contains(elementId)) {
      return true;
   }

   for (const auto& state : g_styleVariableState) {
      if (state.elementRefs.contains(elementId)) {
         return true;
      }
   }

   for (const auto& propagation : g_pendingStyleVariablePropagations) {
      if (propagation.changedOwner == elementId) {
         return true;
      }
   }

   return false;
}

void FlushDiagnosticsReleases() {
   auto pending = std::move(g_pendingDiagnosticsRelease);
   g_pendingDiagnosticsRelease.clear();

   if (!g_visualTreeWatcher) {
      return;
```

```
    }

    // A handle is queued once per report naming it, so a parent appears once
    // per child, and each repeat would pay for another ElementHasState scan.
    std::sort(pending.begin(), pending.end());
    pending.erase(std::unique(pending.begin(), pending.end()), pending.end());

    for (InstanceHandle handle : pending) {
        if (ElementHasState(FindElementId(handle))) {
            continue;
        }

        if (g_visualTreeWatcher->ReleaseDiagnosticsReference(handle)) {
            ForgetElementId(handle);
        }
    }

    // Here rather than anywhere else on the report path: the releases above are
    // what let elements be destroyed unreported, and this runs from the
    // dispatcher, so the teardown of what they were keyed by is outside the
    // walk the reports came from.
    ReapDeadElementIdsIfNeeded();
}

void QueueDiagnosticsRelease(InstanceHandle handle) {
    if (!handle) {
        return;
    }

    g_pendingDiagnosticsRelease.push_back(handle);
    g_lastDiagnosticsReleaseQueueTick = GetTickCount64();
}

// Whether the burst has stopped is decided when this is scheduled: the report
// which schedules it queues its own handles right afterwards, so the time since
// the last queue is short again by the time this runs.
void DrainDiagnosticsReleases() {
    g_diagnosticsReleaseDrainQueued = false;
    FlushDiagnosticsReleases();
}

// Reports arrive from inside XAML's own Enter and Leave walks, and a release
// there re-enters the diagnostics while the tree is being mutated: dropping the
// last reference to an element the walk is still visiting destroys it mid-walk.
// The drain is therefore armed on the thread's dispatcher, which runs it once
// the walk has finished.
//
// Whether to arm it is decided by the next report rather than by a recurring
// timer, so that nothing of the mod is left waiting on a thread it does not
// tear down. A thread which goes quiet therefore holds its last burst until it
// is used again.
void FlushDiagnosticsReleasesIfQuiet() {
    if (g_pendingDiagnosticsRelease.empty() ||
        g_diagnosticsReleaseDrainQueued ||
        GetTickCount64() - g_lastDiagnosticsReleaseQueueTick <
```

```
        kDiagnosticsReleaseDelay) {
      return;
    }

    try {
      if (!g_diagnosticsReleaseDrainTimer) {
        auto dispatcherQueue =
          winrt::Windows::System::DispatcherQueue::GetForCurrentThread();
        if (!dispatcherQueue) {
          // Releasing from here is the one thing that isn't safe, so the
          // elements stay held instead.
          Wh_Log(L"No dispatcher queue, elements will be held");
          return;
        }

        g_diagnosticsReleaseDrainTimer = dispatcherQueue.CreateTimer();
        g_diagnosticsReleaseDrainTimer.IsRepeating(false);
        g_diagnosticsReleaseDrainTimer.Interval(
          std::chrono::milliseconds{kDiagnosticsReleaseDrainDelay});
        g_diagnosticsReleaseDrainTimerTickRevoker =
          g_diagnosticsReleaseDrainTimer.Tick(
            winrt::auto_revoke,
            [](winrt::Windows::System::DispatcherQueueTimer const&,
              winrt::Windows::Foundation::IInspectable const&) {
               DrainDiagnosticsReleases();
            });
      }

      g_diagnosticsReleaseDrainTimer.Start();
      g_diagnosticsReleaseDrainQueued = true;
    } catch (winrt::hresult_error const& ex) {
      Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
    }
}

void StopDiagnosticsReleases() {
    g_pendingDiagnosticsRelease.clear();

    if (g_diagnosticsReleaseDrainTimer) {
      try {
        g_diagnosticsReleaseDrainTimer.Stop();
      } catch (winrt::hresult_error const& ex) {
        Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
      }
    }

    g_diagnosticsReleaseDrainTimerTickRevoker.revoke();
    g_diagnosticsReleaseDrainTimer = nullptr;
    g_diagnosticsReleaseDrainQueued = false;
}

void CleanupCustomizations(ElementId elementId) {
    // Unconditional: a repeater, or an item element which matched no rule, has
    // no customization state but can still have virtualization bookkeeping.
    g_virtualizingRepeaters.erase(elementId);
```

```
g_recycledElements.erase(elementId);
g_elementMatchedItems.erase(elementId);

auto it = g_elementsCustomizationState.find(elementId);
if (it == g_elementsCustomizationState.end()) {
   return;
}

// A reference rather than the iterator: restoring a style below runs
// arbitrary XAML work that can re-enter ApplyCustomizations and rehash
// g_elementsCustomizationState, which invalidates iterators but not
// references to the mapped values. A re-entrant cleanup or re-apply of this
// same elementId would invalidate both the reference and the loop below,
// but the re-entrancy is for other elements, not the one being torn down
// here.
auto& elementCustomizationState = it->second;

// Likewise keeps `state` from being reaped by a re-entrant lookup.
StyleVariableStatePin statePin;

auto element = elementCustomizationState.element.get();
auto* state = GetStyleVariableState(elementCustomizationState.xamlRoot);

RestoreCapturesForElement(element, elementCustomizationState);

// Drop this element's captures from the registry. Other elements may still
// capture the same names, so a name only becomes undefined once its last
// capture is gone. Runs after RestoreCapturesForElement so the
// just-unregistered capture callbacks can't re-seed a variable
// mid-teardown.
std::vector<std::wstring> removedVarNames;
if (state) {
   for (const auto& [property, captureState] :
        elementCustomizationState.captureCustomizationStates) {
      if (captureState.varName.empty()) {
         continue;
      }

      auto varIt = state->variables.find(captureState.varName);
      if (varIt == state->variables.end()) {
         continue;
      }

      size_t removed =
         std::erase_if(varIt->second,
                  [elementId](const StyleVariableCapture& capture) {
                     return capture.elementId == elementId;
                  });
      if (!removed) {
         continue;
      }

      ReleaseStyleVariableElementRefs(state, elementId, removed);

      removedVarNames.push_back(captureState.varName);
```

```
          if (varIt->second.empty()) {
            state->variables.erase(varIt);
          }
        }
      }

      for (const auto& [visualStateGroupOptionalWeakPtrIter, stateIter] :
          elementCustomizationState.perVisualStateGroup) {
        RestoreCustomizationsForVisualStateGroup(
          state, elementId, element, visualStateGroupOptionalWeakPtrIter,
          stateIter);
      }

      // By elementId, not by `it`: a re-entrant apply above may have rehashed the
      // map since the lookup.
      g_elementsCustomizationState.erase(elementId);

      ReapElementTreeNodesIfNeeded();

      // Deferred until this element is out of g_elementsCustomizationState, both
      // so it can't be scored as a winning capture while being torn down and so
      // the loops above don't walk state that re-entrant style applies could
      // invalidate. Every removal propagates, not just the one that left a name
      // undefined: dropping one of several captures still changes which one wins
      // for the consumers that were closest to it. Nothing is collected without a
      // `state`, and the pin above keeps that entry alive until here.
      for (const auto& varName : removedVarNames) {
        PropagateStyleVariableChange(state, varName, std::nullopt);
      }
    }

    using StyleConstant = std::pair<std::wstring, std::wstring>;
    using StyleConstants = std::vector<StyleConstant>;

    std::wstring ApplyStyleConstants(std::wstring_view style,
                                     const StyleConstants& styleConstants) {
      std::wstring result;

      size_t lastPos = 0;
      size_t findPos;

      while ((findPos = style.find('$', lastPos)) != style.npos) {
        result.append(style, lastPos, findPos - lastPos);

        const StyleConstant* constant = nullptr;
        for (const auto& s : styleConstants) {
          if (s.first == style.substr(findPos + 1, s.first.size())) {
            constant = &s;
            break;
          }
        }

        if (constant) {
          result += constant->second;
          lastPos = findPos + 1 + constant->first.size();
```

```
      } else {
        result += '$';
        lastPos = findPos + 1;
      }
    }

    // Care for the rest after last occurrence.
    result += style.substr(lastPos);

    return result;
}

std::optional<StyleConstant> ParseStyleConstant(
    std::wstring_view constant,
    const StyleConstants& styleConstants) {
    // Skip if commented.
    if (constant.starts_with(L"//")) {
      return std::nullopt;
    }

    auto eqPos = constant.find(L'=');
    if (eqPos == constant.npos) {
      Wh_Log(L"Skipping entry with no '=': %.*s",
          static_cast<int>(constant.length()), constant.data());
      return std::nullopt;
    }

    auto key = TrimStringView(constant.substr(0, eqPos));
    auto valueRaw = TrimStringView(constant.substr(eqPos + 1));
    auto value = ApplyStyleConstants(valueRaw, styleConstants);

    return StyleConstant{std::wstring(key), std::move(value)};
}

StyleConstants LoadStyleConstants(
    const std::vector<PCWSTR>& themeStyleConstants) {
    StyleConstants result;

    auto addToResult = [&result](StyleConstant sc) {
      // Keep sorted by name length to replace long names first. Reverse the
      // order to allow overriding definitions with the same name.
      auto insertIndex = std::lower_bound(
        result.begin(), result.end(), sc,
        [](const StyleConstant& a, const StyleConstant& b) {
          return a.first.size() > b.first.size();
        });

      result.insert(insertIndex, std::move(sc));
    };

    for (const auto themeStyleConstant : themeStyleConstants) {
      if (auto parsed = ParseStyleConstant(themeStyleConstant, result)) {
        addToResult(std::move(*parsed));
      }
    }
```

```
  for (int i = 0;; i++) {
    string_setting_unique_ptr constantSetting(
      Wh_GetStringSetting(L"styleConstants[%d]", i));
    if (!*constantSetting.get()) {
      break;
    }

    if (auto parsed = ParseStyleConstant(constantSetting.get(), result)) {
      addToResult(std::move(*parsed));
    }
  }

  return result;
}

ElementMatcher ElementMatcherFromString(std::wstring_view str) {
  ElementMatcher result;
  PropertyValuesUnresolved propertyValuesUnresolved;

  auto trimmed = TrimStringView(str);
  if (trimmed == L"*") {
    result.kind = ElementMatcher::Kind::Wildcard;
    return result;
  }
  if (trimmed == L":root") {
    result.kind = ElementMatcher::Kind::Root;
    return result;
  }

  auto i = str.find_first_of(L"#@[");
  result.type = TrimStringView(str.substr(0, i));
  if (result.type.empty()) {
    throw std::runtime_error("Bad target syntax, empty type");
  }

  while (i != str.npos) {
    auto iNext = str.find_first_of(L"#@[", i + 1);
    auto nextPart =
      str.substr(i + 1, iNext == str.npos ? str.npos : iNext - (i + 1));

    switch (str[i]) {
      case L'#':
        if (!result.name.empty()) {
          throw std::runtime_error(
            "Bad target syntax, more than one name");
        }

        result.name = TrimStringView(nextPart);
        if (result.name.empty()) {
          throw std::runtime_error("Bad target syntax, empty name");
        }
        break;

      case L'@':
```

```
            if (result.visualStateGroupName) {
               throw std::runtime_error(
                  "Bad target syntax, more than one visual state group");
            }

            result.visualStateGroupName = TrimStringView(nextPart);
            break;

         case L'[': {
            auto rule = TrimStringView(nextPart);
            if (rule.length() == 0 || rule.back() != L']') {
               throw std::runtime_error("Bad target syntax, missing ']'");
            }

            rule = TrimStringView(rule.substr(0, rule.length() - 1));
            if (rule.length() == 0) {
               throw std::runtime_error(
                  "Bad target syntax, empty property");
            }

            if (rule.find_first_not_of(L"0123456789") == rule.npos) {
               result.oneBasedIndex = std::stoi(std::wstring(rule));
               break;
            }

            auto ruleEqPos = rule.find(L'=');
            if (ruleEqPos == rule.npos) {
               throw std::runtime_error(
                  "Bad target syntax, missing '=' in property");
            }

            auto ruleKey = TrimStringView(rule.substr(0, ruleEqPos));
            auto ruleVal = TrimStringView(rule.substr(ruleEqPos + 1));

            if (ruleKey.length() == 0) {
               throw std::runtime_error(
                  "Bad target syntax, empty property name");
            }

            propertyValuesUnresolved.push_back(
               {std::wstring(ruleKey), std::wstring(ruleVal)});
            break;
         }

         default:
            throw std::runtime_error("Bad target syntax");
      }

      i = iNext;
   }

   result.propertyValues = std::move(propertyValuesUnresolved);

   return result;
}
```

```
// Parses a single `controlStyles[*].styles[*]` entry into either a ValueRule
// (`Property[@VisualState][:]=value`) or a CaptureRule (`Property=>VarName`).
// Throws std::runtime_error on malformed input or disallowed combinations such
// as `:=>` or `@VisualState=>`.
std::variant<ValueRule, CaptureRule> ParseRule(std::wstring_view str) {
   auto eqPos = str.find(L'=');
   if (eqPos == str.npos) {
      throw std::runtime_error("Bad style syntax, '=' is missing");
   }

   auto name = str.substr(0, eqPos);
   auto value = str.substr(eqPos + 1);

   if (!value.empty() && value.front() == L'>') {
      // Capture rule: `Property=>VarName`. The right-hand side (after the
      // leading `>` marker) is the name of a mod-global style variable into
      // which the property's current value is captured.
      value = value.substr(1);

      if (!name.empty() && name.back() == L':') {
         throw std::runtime_error(
            "Bad style syntax, ':=>' is not valid (':=' XAML value "
            "cannot be combined with '=>' capture)");
      }

      if (name.find(L'@') != name.npos) {
         throw std::runtime_error(
            "Bad style syntax, '@VisualState' not allowed on a capture "
            "rule");
      }

      auto trimmedPropertyName = TrimStringView(name);
      if (trimmedPropertyName.empty()) {
         throw std::runtime_error("Bad style syntax, empty name");
      }

      auto trimmedVarName = TrimStringView(value);
      if (trimmedVarName.empty()) {
         throw std::runtime_error(
            "Bad style syntax, empty capture variable name");
      }
      if (!IsValidStyleVariableIdentifier(trimmedVarName)) {
         throw std::runtime_error(
            "Bad style syntax, invalid capture variable name");
      }

      return CaptureRule{std::wstring(trimmedPropertyName),
                         std::wstring(trimmedVarName)};
   }

   ValueRule result;
   result.value = TrimStringView(value);

   if (!name.empty() && name.back() == L':') {
```

```
      result.isXamlValue = true;
      name = name.substr(0, name.size() - 1);
    }

    auto atPos = name.find(L'@');
    if (atPos != name.npos) {
      result.visualState = TrimStringView(name.substr(atPos + 1));
      name = name.substr(0, atPos);
    }

    result.propertyName = TrimStringView(name);
    if (result.propertyName.empty()) {
      throw std::runtime_error("Bad style syntax, empty name");
    }

    return result;
}

std::wstring AdjustTypeName(std::wstring_view type) {
    if (type.find_first_of(L".:") == type.npos) {
      if (type == L"Rectangle") {
        return L"Windows.UI.Xaml.Shapes.Rectangle";
      }

      return L"Windows.UI.Xaml.Controls." + std::wstring{type};
    }

    static const std::vector<std::pair<std::wstring_view, std::wstring_view>>
      adjustments = {
        {L"taskbar:", L"Taskbar."},
        {L"systemtray:", L"SystemTray."},
        {L"udk:", L"WindowsUdk.UI.Shell."},
        {L"muxc:", L"Microsoft.UI.Xaml.Controls."},
      };

    for (const auto& adjustment : adjustments) {
      if (type.starts_with(adjustment.first)) {
        auto result = std::wstring{adjustment.second};
        result += type.substr(adjustment.first.size());
        return result;
      }
    }

    return std::wstring{type};
}

// Splits a target string on the commas which separate targets, ignoring commas
// which are part of a `[Property=Value]` clause.
std::vector<std::wstring_view> SplitTargetString(std::wstring_view target) {
    std::vector<std::wstring_view> result;

    size_t partBegin = 0;
    bool inProperty = false;
    for (size_t i = 0; i < target.size(); i++) {
      switch (target[i]) {
```

```
        case L'[':
          inProperty = true;
          break;

        case L']':
          inProperty = false;
          break;

        case L',':
          if (!inProperty) {
            result.push_back(target.substr(partBegin, i - partBegin));
            partBegin = i + 1;
          }
          break;
      }
    }

    result.push_back(target.substr(partBegin));

    return result;
}

void AddElementCustomizationRulesForSingleTarget(
    std::wstring_view target,
    const std::vector<std::wstring>& styles) {
    ElementCustomizationRules elementCustomizationRules;

    auto targetParts = SplitStringView(target, L" > ");

    bool first = true;
    bool hasVisualStateGroup = false;
    for (auto i = targetParts.rbegin(); i != targetParts.rend(); ++i) {
      const auto& targetPart = *i;
      const bool isLeftmost = (i + 1 == targetParts.rend());

      auto matcher = ElementMatcherFromString(targetPart);

      const auto& prevParents =
          elementCustomizationRules.parentElementMatchers;
      const bool prevIsWildcard =
          !prevParents.empty() &&
          prevParents.back().kind == ElementMatcher::Kind::Wildcard;

      switch (matcher.kind) {
        case ElementMatcher::Kind::Element:
          matcher.type = AdjustTypeName(matcher.type);
          break;

        case ElementMatcher::Kind::Wildcard:
          if (first) {
            throw std::runtime_error(
                "Bad target syntax, '*' can't be the matched element");
          }
          if (isLeftmost) {
            throw std::runtime_error(
```

```
            "Bad target syntax, '*' can't be the leftmost target "
            "part");
      }
      if (prevIsWildcard) {
        throw std::runtime_error(
            "Bad target syntax, '*' can't be adjacent to another "
            "'*'");
      }
      break;

    case ElementMatcher::Kind::Root:
      if (first) {
        throw std::runtime_error(
            "Bad target syntax, ':root' can't be the matched "
            "element");
      }
      if (!isLeftmost) {
        throw std::runtime_error(
            "Bad target syntax, ':root' must be the leftmost "
            "target part");
      }
      if (prevIsWildcard) {
        throw std::runtime_error(
            "Bad target syntax, ':root' must be followed by a "
            "non-wildcard target part");
      }
      break;
  }

  if (matcher.visualStateGroupName) {
    if (hasVisualStateGroup) {
      throw std::runtime_error(
          "Element type can't have more than one visual state group");
    }

    hasVisualStateGroup = true;
  }

  if (first) {
    UnresolvedRules unresolvedRules;
    for (const auto& style : styles) {
      auto parsed = ParseRule(style);
      if (auto* valueRule = std::get_if<ValueRule>(&parsed)) {
        unresolvedRules.valueRules.push_back(std::move(*valueRule));
      } else {
        unresolvedRules.captureRules.push_back(
            std::move(std::get<CaptureRule>(parsed)));
      }
    }

    elementCustomizationRules.elementMatcher = std::move(matcher);
    elementCustomizationRules.propertyOverrides =
        std::move(unresolvedRules);
  } else {
    elementCustomizationRules.parentElementMatchers.push_back(
```

```
            std::move(matcher));
        }

        first = false;
    }

    g_elementsCustomizationRules.push_back(
        std::move(elementCustomizationRules));
}

void AddElementCustomizationRules(std::wstring_view target,
                                  const std::vector<std::wstring>& styles) {
    auto targets = SplitTargetString(target);

    for (const auto& singleTarget : targets) {
        try {
            AddElementCustomizationRulesForSingleTarget(singleTarget, styles);
        } catch (winrt::hresult_error const& ex) {
            Wh_Log(L"Error %08X for target %.*s", ex.code(),
                static_cast<int>(singleTarget.length()),
                singleTarget.data());
        } catch (std::exception const& ex) {
            Wh_Log(L"Error for target %.*s: %S",
                static_cast<int>(singleTarget.length()), singleTarget.data(),
                ex.what());
        }
    }
}

bool ProcessSingleTargetStylesFromSettings(
    int index,
    const StyleConstants& styleConstants) {
    string_setting_unique_ptr targetStringSetting(
        Wh_GetStringSetting(L"controlStyles[%d].target", index));
    if (!*targetStringSetting.get()) {
        return false;
    }

    // Skip if commented.
    if (targetStringSetting[0] == L'/' && targetStringSetting[1] == L'/') {
        return true;
    }

    Wh_Log(L"Processing styles for %s", targetStringSetting.get());

    std::vector<std::wstring> styles;

    for (int styleIndex = 0;; styleIndex++) {
        string_setting_unique_ptr styleSetting(Wh_GetStringSetting(
            L"controlStyles[%d].styles[%d]", index, styleIndex));
        if (!*styleSetting.get()) {
            break;
        }

        // Skip if commented.
```

```
      if (styleSetting[0] == L'/' && styleSetting[1] == L'/') {
        continue;
      }

      styles.push_back(
        ApplyStyleConstants(styleSetting.get(), styleConstants));
    }

    if (styles.size() > 0) {
      AddElementCustomizationRules(targetStringSetting.get(), styles);
    }

    return true;
}

std::optional<ResourceVariableEntry> ParseResourceVariable(
    std::wstring_view entry,
    const StyleConstants& styleConstants) {
    // Skip if commented.
    if (entry.starts_with(L"//")) {
      return std::nullopt;
    }

    // Find the first '=' to split key and value.
    auto eqPos = entry.find(L'=');
    if (eqPos == entry.npos) {
      Wh_Log(L"Skipping entry with no '=': %.*s",
          static_cast<int>(entry.length()), entry.data());
      return std::nullopt;
    }

    auto keyPart = TrimStringView(entry.substr(0, eqPos));
    auto valueRaw = TrimStringView(entry.substr(eqPos + 1));
    auto value = ApplyStyleConstants(valueRaw, styleConstants);

    constexpr std::wstring_view kThemeResourcePrefix = L"{ThemeResource ";

    ResourceVariableType type = ResourceVariableType::String;
    if (keyPart.size() > 0 && keyPart.back() == L':') {
      type = ResourceVariableType::Xaml;
      keyPart = keyPart.substr(0, keyPart.size() - 1);
      keyPart = TrimStringView(keyPart);
    } else if (value.starts_with(kThemeResourcePrefix) &&
          value.ends_with(L"}")) {
      type = ResourceVariableType::ThemeResourceReference;
      value = TrimStringView(
        value.substr(kThemeResourcePrefix.size(),
                value.size() - kThemeResourcePrefix.size() - 1));
    }

    ResourceVariableTheme theme = ResourceVariableTheme::None;
    std::wstring key;

    // Check for @theme suffix in key part.
    auto atPos = keyPart.find(L'@');
```

```
    if (atPos != keyPart.npos) {
        key = TrimStringView(keyPart.substr(0, atPos));
        auto themePart = TrimStringView(keyPart.substr(atPos + 1));
        if (themePart == L"Dark") {
            theme = ResourceVariableTheme::Dark;
        } else if (themePart == L"Light") {
            theme = ResourceVariableTheme::Light;
        } else {
            Wh_Log(L"Unknown theme '%.*s', expected 'Dark' or 'Light'",
                   static_cast<int>(themePart.size()), themePart.data());
            return std::nullopt;
        }
    } else {
        key = std::wstring(keyPart);
    }

    return ResourceVariableEntry{std::move(key), std::move(value), theme, type};
}

winrt::Windows::Foundation::IInspectable ParseXamlValue(
    std::wstring_view xamlValue) {
    std::wstring xaml;
    xaml += L"       <Setter Property=\"Tag\">\n";
    xaml += L"           <Setter.Value>\n";
    xaml += xamlValue;
    xaml += L"\n";
    xaml += L"           </Setter.Value>\n";
    xaml += L"       </Setter>\n";

    auto style = GetStyleFromXamlSetters(L"FrameworkElement", xaml);
    return style.Setters().GetAt(0).as<Setter>().Value();
}

bool ProcessResourceVariable(ResourceDictionary resources,
                             ResourceDictionary darkDict,
                             ResourceDictionary lightDict,
                             const ResourceVariableEntry& entry) {
    auto boxedKey = winrt::box_value(entry.key);

    if (entry.theme != ResourceVariableTheme::None) {
        ResourceDictionary& targetDict =
            entry.theme == ResourceVariableTheme::Dark ? darkDict : lightDict;

        if (targetDict.HasKey(boxedKey)) {
            Wh_Log(
                L"Resource variable key '%s' already exists in theme '%s', "
                L"skipping",
                entry.key.c_str(),
                entry.theme == ResourceVariableTheme::Dark ? L"Dark"
                                                           : L"Light");
            return false;
        }

        winrt::Windows::Foundation::IInspectable value;
        switch (entry.type) {
```

```
      case ResourceVariableType::String:
        value = winrt::box_value(entry.value);
        break;
      case ResourceVariableType::Xaml:
        value =
          entry.value.empty() ? nullptr : ParseXamlValue(entry.value);
        break;
      case ResourceVariableType::ThemeResourceReference:
        value = resources.Lookup(winrt::box_value(entry.value));
        break;
    }

    targetDict.Insert(boxedKey, value);

    return true;
}

// key= - convert using existing resource type.
auto existingResource = resources.TryLookup(boxedKey);
if (!existingResource) {
    Wh_Log(L"Resource variable key '%s' not found, skipping",
        entry.key.c_str());
    return false;
}

auto [it, inserted] =
    g_originalResourceValues.try_emplace(entry.key, existingResource);
if (!inserted) {
    Wh_Log(L"Resource variable key '%s' already modified, skipping",
        entry.key.c_str());
    return false;
}

winrt::Windows::Foundation::IInspectable value;
switch (entry.type) {
    case ResourceVariableType::String: {
        auto resourceClassName = winrt::get_class_name(existingResource);

        // Unwrap IReference<T> to get inner type name.
        if (resourceClassName.starts_with(
                L"Windows.Foundation.IReference`1<") &&
            resourceClassName.ends_with(L'>')) {
            size_t prefixSize =
                sizeof("Windows.Foundation.IReference`1<") - 1;
            resourceClassName =
                winrt::hstring(resourceClassName.data() + prefixSize,
                               resourceClassName.size() - prefixSize - 1);
        }

        value = Markup::XamlBindingHelper::ConvertValue(
            Interop::TypeName{resourceClassName},
            winrt::box_value(entry.value));
        break;
    }
```

```
        case ResourceVariableType::Xaml:
          value = entry.value.empty() ? nullptr : ParseXamlValue(entry.value);
          break;

        case ResourceVariableType::ThemeResourceReference:
          value = resources.Lookup(winrt::box_value(entry.value));
          break;
      }

      resources.Insert(boxedKey, value);

      return true;
    }

    void RefreshThemeResourceEntries() {
      if (g_resourceVariables.empty()) {
        return;
      }

      Wh_Log(L"Refreshing theme resource entries");

      auto resources = Application::Current().Resources();

      auto darkDict = g_resourceVariablesThemeDict.ThemeDictionaries()
                          .TryLookup(winrt::box_value(L"Dark"))
                          .try_as<ResourceDictionary>();
      auto lightDict = g_resourceVariablesThemeDict.ThemeDictionaries()
                           .TryLookup(winrt::box_value(L"Light"))
                           .try_as<ResourceDictionary>();

      for (const auto& entry : g_resourceVariables) {
        if (entry.type != ResourceVariableType::ThemeResourceReference) {
          continue;
        }

        try {
          auto boxedKey = winrt::box_value(entry.key);
          auto value = resources.Lookup(winrt::box_value(entry.value));

          if (entry.theme == ResourceVariableTheme::Dark && darkDict) {
            darkDict.Insert(boxedKey, value);
          } else if (entry.theme == ResourceVariableTheme::Light &&
                     lightDict) {
            lightDict.Insert(boxedKey, value);
          } else {
            resources.Insert(boxedKey, value);
          }
        } catch (winrt::hresult_error const& ex) {
          Wh_Log(L"Error refreshing '%s': %08X", entry.key.c_str(),
                 ex.code());
        }
      }
    }

    std::vector<ResourceVariableEntry> ProcessResourceVariablesFromSettings(
```

```
    const StyleConstants& styleConstants,
    const std::vector<PCWSTR>& themeResourceVariables) {
    std::vector<ResourceVariableEntry> resourceVariables;

    for (const auto& themeResourceVariable : themeResourceVariables) {
      Wh_Log(L"Processing theme resource variable %s", themeResourceVariable);

      auto parsed =
        ParseResourceVariable(themeResourceVariable, styleConstants);
      if (parsed) {
        resourceVariables.push_back(std::move(*parsed));
      }
    }

    for (int i = 0;; i++) {
      string_setting_unique_ptr setting(
        Wh_GetStringSetting(L"themeResourceVariables[%d]", i));
      if (!*setting.get()) {
        break;
      }

      Wh_Log(L"Processing resource variable %s", setting.get());

      auto parsed = ParseResourceVariable(setting.get(), styleConstants);
      if (parsed) {
        resourceVariables.push_back(std::move(*parsed));
      }
    }

    return resourceVariables;
}

void MergeResourceVariables() {
    auto resources = Application::Current().Resources();

    // Create theme dictionaries for @Dark/@Light resources.
    g_resourceVariablesThemeDict = ResourceDictionary();
    ResourceDictionary darkDict;
    ResourceDictionary lightDict;
    bool hasThemeResources = false;
    bool hasThemeResourceReferences = false;

    for (auto it = g_resourceVariables.rbegin();
         it != g_resourceVariables.rend(); ++it) {
      Wh_Log(L"Processing resource variable %s", it->key.c_str());

      try {
        if (ProcessResourceVariable(resources, darkDict, lightDict, *it)) {
          if (it->theme != ResourceVariableTheme::None) {
            hasThemeResources = true;
          }

          if (it->type == ResourceVariableType::ThemeResourceReference) {
            hasThemeResourceReferences = true;
          }
```

```
            }
        } catch (winrt::hresult_error const& ex) {
            Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
        } catch (std::exception const& ex) {
            Wh_Log(L"Error: %S", ex.what());
        }
    }

    if (hasThemeResources) {
        g_resourceVariablesThemeDict.ThemeDictionaries().Insert(
            winrt::box_value(L"Dark"), darkDict);
        g_resourceVariablesThemeDict.ThemeDictionaries().Insert(
            winrt::box_value(L"Light"), lightDict);

        resources.MergedDictionaries().Append(g_resourceVariablesThemeDict);
    }

    // Register for color changes to refresh theme resource references.
    if (hasThemeResourceReferences) {
        g_uiSettings = winrt::Windows::UI::ViewManagement::UISettings();
        auto dispatcherQueue =
            winrt::Windows::System::DispatcherQueue::GetForCurrentThread();
        g_colorValuesChangedToken =
            g_uiSettings.ColorValuesChanged([dispatcherQueue](auto&&, auto&&) {
                dispatcherQueue.TryEnqueue(RefreshThemeResourceEntries);
            });
    }
}

std::optional<bool> IsOsFeatureEnabled(UINT32 featureId) {
    enum FEATURE_ENABLED_STATE {
        FEATURE_ENABLED_STATE_DEFAULT = 0,
        FEATURE_ENABLED_STATE_DISABLED = 1,
        FEATURE_ENABLED_STATE_ENABLED = 2,
    };

#pragma pack(push, 1)
    struct RTL_FEATURE_CONFIGURATION {
        unsigned int featureId;
        unsigned __int32 group : 4;
        FEATURE_ENABLED_STATE enabledState : 2;
        unsigned __int32 enabledStateOptions : 1;
        unsigned __int32 unused1 : 1;
        unsigned __int32 variant : 6;
        unsigned __int32 variantPayloadKind : 2;
        unsigned __int32 unused2 : 16;
        unsigned int payload;
    };
#pragma pack(pop)

    using RtlQueryFeatureConfiguration_t =
        int(NTAPI*)(UINT32, int, INT64*, RTL_FEATURE_CONFIGURATION*);
    static RtlQueryFeatureConfiguration_t pRtlQueryFeatureConfiguration = []() {
        HMODULE hNtDll = GetModuleHandle(L"ntdll.dll");
        return hNtDll ? (RtlQueryFeatureConfiguration_t)GetProcAddress(
```

```
                    hNtDll, "RtlQueryFeatureConfiguration")
                : nullptr;
    }();

    if (!pRtlQueryFeatureConfiguration) {
        Wh_Log(L"RtlQueryFeatureConfiguration not found");
        return std::nullopt;
    }

    RTL_FEATURE_CONFIGURATION feature = {0};
    INT64 changeStamp = 0;
    HRESULT hr =
        pRtlQueryFeatureConfiguration(featureId, 1, &changeStamp, &feature);
    if (SUCCEEDED(hr)) {
        Wh_Log(L"RtlQueryFeatureConfiguration result for %u: %d", featureId,
            feature.enabledState);

        switch (feature.enabledState) {
            case FEATURE_ENABLED_STATE_DISABLED:
                return false;
            case FEATURE_ENABLED_STATE_ENABLED:
                return true;
            case FEATURE_ENABLED_STATE_DEFAULT:
                return std::nullopt;
        }
    } else {
        Wh_Log(L"RtlQueryFeatureConfiguration error for %u: %08X", featureId,
            hr);
    }

    return std::nullopt;
}

void ProcessAllStylesFromSettings() {
    PCWSTR themeName = Wh_GetStringSetting(L"theme");
    const Theme* theme = nullptr;
    if (wcscmp(themeName, L"TranslucentTaskbar") == 0) {
        theme = &g_themeTranslucentTaskbar;
    } else if (wcscmp(themeName, L"DockLike") == 0) {
        theme = &g_themeDockLike;
    } else if (wcscmp(themeName, L"SimplyTransparent") == 0) {
        theme = &g_themeSimplyTransparent;
    } else if (wcscmp(themeName, L"Squircle") == 0) {
        // Weather widget on the right.
        //
https://www.reddit.com/r/Windows11/comments/1dnew8x/my_weather_widget_is_on_the_ri
ght_side/
        constexpr UINT32 kExtendedModeAEPForTaskbar = 48660958;
        theme = IsOsFeatureEnabled(kExtendedModeAEPForTaskbar).value_or(false)
                ? &g_themeSquircle_variant_WeatherOnTheRight
                : &g_themeSquircle;
    } else if (wcscmp(themeName, L"Matter") == 0) {
        theme = &g_themeMatter;
    } else if (wcscmp(themeName, L"WinXP") == 0) {
        theme = &g_themeWinXP;
    } else if (wcscmp(themeName, L"WinXP_variant_Zune") == 0) {
```

```
    theme = &g_themeWinXP_variant_Zune;
} else if (wcscmp(themeName, L"Bubbles") == 0) {
    theme = &g_themeBubbles;
} else if (wcscmp(themeName, L"RosePine") == 0) {
    theme = &g_themeRosePine;
} else if (wcscmp(themeName, L"WinVista") == 0) {
    theme = &g_themeWinVista;
} else if (wcscmp(themeName, L"CleanSlate") == 0) {
    theme = &g_themeCleanSlate;
} else if (wcscmp(themeName, L"Lucent") == 0) {
    theme = &g_themeLucent;
} else if (wcscmp(themeName, L"Lucent_variant_Light") == 0) {
    theme = &g_themeLucent_variant_Light;
} else if (wcscmp(themeName, L"SunValley") == 0) {
    theme = &g_themeSunValley;
} else if (wcscmp(themeName, L"21996Taskbar") == 0) {
    theme = &g_theme21996Taskbar;
} else if (wcscmp(themeName, L"BottomDensy") == 0) {
    theme = &g_themeBottomDensy;
} else if (wcscmp(themeName, L"BottomDensy_variant_NoInd") == 0) {
    theme = &g_themeBottomDensy_variant_NoInd;
} else if (wcscmp(themeName, L"TaskbarXII") == 0) {
    theme = &g_themeTaskbarXII;
} else if (wcscmp(themeName, L"xdark") == 0) {
    theme = &g_themexdark;
} else if (wcscmp(themeName, L"Windows7") == 0) {
    theme = &g_themeWindows7;
} else if (wcscmp(themeName, L"Aeris") == 0) {
    theme = &g_themeAeris;
} else if (wcscmp(themeName, L"Plasma") == 0) {
    theme = &g_themePlasma;
} else if (wcscmp(themeName, L"WindowGlass") == 0) {
    theme = &g_themeWindowGlass;
} else if (wcscmp(themeName, L"WindowGlass_variant_Split") == 0) {
    theme = &g_themeWindowGlass_variant_Split;
} else if (wcscmp(themeName, L"WindowGlass_variant_FullLength") == 0) {
    theme = &g_themeWindowGlass_variant_FullLength;
} else if (wcscmp(themeName, L"Surface") == 0) {
    theme = &g_themeSurface;
} else if (wcscmp(themeName, L"Oversimplified&Accentuated") == 0) {
    theme = &g_themeOversimplified_Accentuated;
} else if (wcscmp(themeName, L"Luminosity_variant_Dock") == 0) {
    theme = &g_themeLuminosity_variant_Dock;
} else if (wcscmp(themeName, L"Luminosity_variant_Classic") == 0) {
    theme = &g_themeLuminosity_variant_Classic;
} else if (wcscmp(themeName, L"Luminosity_variant_Compact") == 0) {
    theme = &g_themeLuminosity_variant_Compact;
} else if (wcscmp(themeName, L"LayerMicaUI") == 0) {
    theme = &g_themeLayerMicaUI;
} else if (wcscmp(themeName, L"Fluid") == 0) {
    theme = &g_themeFluid;
} else if (wcscmp(themeName, L"TintedGlass") == 0) {
    theme = &g_themeTintedGlass;
} else if (wcscmp(themeName, L"TaskbarToStatusbar") == 0) {
    theme = &g_themeTaskbarToStatusbar;
```

```
} else if (wcscmp(themeName, L"UltraWideFriendly") == 0) {
    theme = &g_themeUltraWideFriendly;
} else if (wcscmp(themeName, L"LiquidGlass2") == 0) {
    theme = &g_themeLiquidGlass2;
} else if (wcscmp(themeName, L"LiquidGlass") == 0) {
    theme = &g_themeLiquidGlass;
} else if (wcscmp(themeName, L"LiquidGlass_variant_Alternate") == 0) {
    theme = &g_themeLiquidGlass_variant_Alternate;
} else if (wcscmp(themeName, L"Borderless") == 0) {
    theme = &g_themeBorderless;
} else if (wcscmp(themeName, L"Command_Center") == 0) {
    theme = &g_themeCommand_Center;
} else if (wcscmp(themeName, L"FluentGlass") == 0) {
    theme = &g_themeFluentGlass;
} else if (wcscmp(themeName, L"OS26_Liquid_Glass_variant_ClearMacDock") ==
        0) {
    theme = &g_themeOS26_Liquid_Glass_variant_ClearMacDock;
} else if (wcscmp(themeName,
            L"OS26_Liquid_Glass_variant_ClearMacDockCompact") == 0) {
    theme = &g_themeOS26_Liquid_Glass_variant_ClearMacDockCompact;
} else if (wcscmp(themeName, L"OS26_Liquid_Glass_variant_ClearTaskbar") ==
        0) {
    theme = &g_themeOS26_Liquid_Glass_variant_ClearTaskbar;
} else if (wcscmp(themeName, L"OS26_Liquid_Glass_variant_DarkMacDock") ==
        0) {
    theme = &g_themeOS26_Liquid_Glass_variant_DarkMacDock;
} else if (wcscmp(themeName,
            L"OS26_Liquid_Glass_variant_DarkMacDockCompact") == 0) {
    theme = &g_themeOS26_Liquid_Glass_variant_DarkMacDockCompact;
} else if (wcscmp(themeName, L"OS26_Liquid_Glass_variant_DarkTaskbar") ==
        0) {
    theme = &g_themeOS26_Liquid_Glass_variant_DarkTaskbar;
} else if (wcscmp(themeName, L"FrostyGlass") == 0) {
    theme = &g_themeFrostyGlass;
} else if (wcscmp(themeName, L"FrostedAcrylic") == 0) {
    theme = &g_themeFrostedAcrylic;
} else if (wcscmp(themeName, L"Pills") == 0) {
    theme = &g_themePills;
} else if (wcscmp(themeName, L"Blob") == 0) {
    theme = &g_themeBlob;
} else if (wcscmp(themeName, L"Minecraft_Hotbar") == 0) {
    theme = &g_themeMinecraft_Hotbar;
} else if (wcscmp(themeName, L"One_UI_8_5_variant_Dock") == 0) {
    theme = &g_themeOne_UI_8_5_variant_Dock;
} else if (wcscmp(themeName, L"One_UI_8_5_variant_Taskbar") == 0) {
    theme = &g_themeOne_UI_8_5_variant_Taskbar;
}
Wh_FreeStringSetting(themeName);

StyleConstants styleConstants = LoadStyleConstants(
    theme ? theme->styleConstants : std::vector<PCWSTR>{});

if (theme) {
    for (const auto& themeTargetStyle : theme->targetStyles) {
        try {
```

```
            std::vector<std::wstring> styles;
            styles.reserve(themeTargetStyle.styles.size());
            for (const auto& s : themeTargetStyle.styles) {
              styles.push_back(ApplyStyleConstants(s, styleConstants));
            }

            AddElementCustomizationRules(themeTargetStyle.target, styles);
          } catch (winrt::hresult_error const& ex) {
            Wh_Log(L"Error %08X", ex.code());
          } catch (std::exception const& ex) {
            Wh_Log(L"Error: %S", ex.what());
          }
        }
      }

      for (int i = 0;; i++) {
        try {
          if (!ProcessSingleTargetStylesFromSettings(i, styleConstants)) {
            break;
          }
        } catch (winrt::hresult_error const& ex) {
          Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
        } catch (std::exception const& ex) {
          Wh_Log(L"Error: %S", ex.what());
        }
      }

      g_resourceVariables = ProcessResourceVariablesFromSettings(
        styleConstants,
        theme ? theme->themeResourceVariables : std::vector<PCWSTR>{});
    }

    void UninitializeResourceVariables() {
      // Unregister color change handler.
      if (g_colorValuesChangedToken) {
        g_uiSettings.ColorValuesChanged(g_colorValuesChangedToken);
        g_colorValuesChangedToken = {};
      }
      g_uiSettings = nullptr;
      g_resourceVariables.clear();

      // Restore original resource values.
      auto resources = Application::Current().Resources();
      for (const auto& [key, originalValue] : g_originalResourceValues) {
        try {
          resources.Insert(winrt::box_value(key), originalValue);
        } catch (...) {
          HRESULT hr = winrt::to_hresult();
          Wh_Log(L"Error %08X", hr);
        }
      }
      g_originalResourceValues.clear();

      // Remove our merged theme dictionary.
      if (g_resourceVariablesThemeDict) {
```

```
        auto merged = resources.MergedDictionaries();
        uint32_t index;
        if (merged.IndexOf(g_resourceVariablesThemeDict, index)) {
            merged.RemoveAt(index);
        }
        g_resourceVariablesThemeDict = nullptr;
    }
}

void UninitializeForCurrentThread() {
    // Drop the click-through subclasses before restoring the windows below.
    // ClearClickThroughRegions calls SetWindowRgn, which sends
    // WM_WINDOWPOSCHANGED synchronously, and a still-attached subclass would
    // reapply the very region being cleared.
    for (HWND hWnd : g_clickThroughSubclassedWindows) {
        WindhawkUtils::RemoveWindowSubclassFromAnyThread(
            hWnd, ClickThroughTaskbarSubclassProc);
    }
    g_clickThroughSubclassedWindows.clear();

    for (HWND hWnd : g_taskbarSurfaceSubclassedWindows) {
        WindhawkUtils::RemoveWindowSubclassFromAnyThread(
            hWnd, TaskbarSurfaceSubclassProc);
    }
    g_taskbarSurfaceSubclassedWindows.clear();

    // Restore taskbars clipped for click-through, then drop tracking (revokers
    // auto-unhook LayoutUpdated).
    ClearClickThroughRegions();
    g_clickThroughTaskbarState.clear();
    g_clickThroughIslandRoots.clear();

    // Clear tracked images for this thread (revokers will automatically
    // unregister).
    if (auto& timer = g_trackedImagesForThread.retryTimer) {
        try {
            timer.Stop();
        } catch (winrt::hresult_error const& ex) {
            Wh_Log(L"Error %08X: %s", ex.code(), ex.message().c_str());
        }
    }
    g_trackedImagesForThread.retryTimerTickRevoker.revoke();
    g_trackedImagesForThread.retryTimer = nullptr;
    g_trackedImagesForThread.retryDueTick = 0;
    g_trackedImagesForThread.images.clear();
    g_imageCacheUriRemotes.clear();
    StopImageLoadRetriesForCurrentThread();

    for (const auto& [elementDo, asyncOp] : g_delayedBackgroundFillSet) {
        asyncOp.Cancel();
    }

    g_delayedBackgroundFillSet.clear();

    // Before the teardown below, so that no recycling callback can fire into
```

```
   // half-cleared state, and no pending release can consult state which is
   // being cleared.
   StopDiagnosticsReleases();
   g_virtualizingRepeaters.clear();
   g_recycledElements.clear();
   g_elementMatchedItems.clear();

   // Detached from the global before being walked: restoring a value runs
   // arbitrary XAML work, and whatever it re-enters looks its elements up in
   // g_elementsCustomizationState. Walking a map nothing else can reach keeps
   // a re-entrant insert or erase from invalidating this loop, and leaving the
   // global empty makes those lookups miss, which is what teardown wants.
   auto elementsCustomizationState = std::move(g_elementsCustomizationState);
   g_elementsCustomizationState.clear();

   for (const auto& [elementId, elementCustomizationState] :
       elementsCustomizationState) {
      auto element = elementCustomizationState.element.get();
      auto* state = GetStyleVariableState(elementCustomizationState.xamlRoot);

      RestoreCapturesForElement(element, elementCustomizationState);

      for (const auto& [visualStateGroupOptionalWeakPtrIter, stateIter] :
          elementCustomizationState.perVisualStateGroup) {
         RestoreCustomizationsForVisualStateGroup(
            state, elementId, element, visualStateGroupOptionalWeakPtrIter,
            stateIter);
      }
   }

   // Before g_elementTreeNodes, since the states hold the last strong refs to
   // the spine nodes.
   elementsCustomizationState.clear();
   g_elementTreeNodes.clear();
   g_elementTreeNodesReapThreshold = 64;
   g_pendingStyleVariablePropagations.clear();
   g_styleVariableState.clear();

   // After everything keyed by an id. g_lastElementId keeps counting, since an
   // id must never name two elements.
   g_elementIds.clear();
   g_elementIdsReapThreshold = 64;

   g_elementsCustomizationRules.clear();

   UninitializeResourceVariables();

   g_initializedForThread = false;
}

void UninitializeSettingsAndTap() {
   if (g_visualTreeWatcher) {
      g_visualTreeWatcher->UnadviseVisualTreeChange();
      g_visualTreeWatcher = nullptr;
   }
```

```
    g_initialized = false;
}

void InitializeForCurrentThread() {
    if (g_initializedForThread) {
        return;
    }

    EnsureTaskbarSurfaceSubclasses();

    ProcessAllStylesFromSettings();

    g_initializedForThread = true;
}

void InitializeSettingsAndTap() {
    if (g_initialized.exchange(true)) {
        return;
    }

    HRESULT hr = InjectWindhawkTAP();
    if (FAILED(hr)) {
        Wh_Log(L"Error %08X", hr);
    }
}

using RunFromWindowThreadProc_t = void(WINAPI*)(PVOID parameter);

bool RunFromWindowThread(HWND hWnd,
                         RunFromWindowThreadProc_t proc,
                         PVOID procParam) {
    static const UINT runFromWindowThreadRegisteredMsg =
        RegisterWindowMessage(L"Windhawk_RunFromWindowThread_" WH_MOD_ID);

    struct RUN_FROM_WINDOW_THREAD_PARAM {
        RunFromWindowThreadProc_t proc;
        PVOID procParam;
    };

    DWORD dwThreadId = GetWindowThreadProcessId(hWnd, nullptr);
    if (dwThreadId == 0) {
        return false;
    }

    if (dwThreadId == GetCurrentThreadId()) {
        proc(procParam);
        return true;
    }

    HHOOK hook = SetWindowsHookEx(
        WH_CALLWNDPROC,
        [](int nCode, WPARAM wParam, LPARAM lParam) -> LRESULT {
            if (nCode == HC_ACTION) {
                const CWPSTRUCT* cwp = (const CWPSTRUCT*)lParam;
```

```
            if (cwp->message == runFromWindowThreadRegisteredMsg) {
                RUN_FROM_WINDOW_THREAD_PARAM* param =
                    (RUN_FROM_WINDOW_THREAD_PARAM*)cwp->lParam;
                param->proc(param->procParam);
            }
        }

        return CallNextHookEx(nullptr, nCode, wParam, lParam);
    },
    nullptr, dwThreadId);
  if (!hook) {
    return false;
  }

  RUN_FROM_WINDOW_THREAD_PARAM param;
  param.proc = proc;
  param.procParam = procParam;
  SendMessage(hWnd, runFromWindowThreadRegisteredMsg, 0, (LPARAM)&param);

  UnhookWindowsHookEx(hook);

  return true;
}

void OnWindowCreated(HWND hWnd,
                     HWND hWndParent,
                     LPCWSTR lpClassName,
                     PCSTR funcName) {
  BOOL bTextualClassName = ((ULONG_PTR)lpClassName & ~(ULONG_PTR)0xffff) != 0;

  // Only on an initialized thread, since teardown only visits those and a
  // subclass left behind would outlive the mod. A taskbar created before its
  // thread is initialized is picked up by the sweep there.
  if (g_initializedForThread && IsTaskbarTopLevelWindow(hWnd)) {
    EnsureTaskbarSurfaceSubclass(hWnd);
  }

  WCHAR className[64];
  if (hWndParent && GetClassName(hWnd, className, ARRAYSIZE(className)) &&
    _wcsicmp(className,
             L"Windows.UI.Composition.DesktopWindowContentBridge") == 0 &&
    GetClassName(hWndParent, className, ARRAYSIZE(className)) &&
    _wcsicmp(className, L"Shell_TrayWnd") == 0) {
    Wh_Log(L"Initializing - Created DesktopWindowContentBridge window");
    InitializeForCurrentThread();
    InitializeSettingsAndTap();
    return;
  }

  if (bTextualClassName &&
    (_wcsicmp(lpClassName, L"XamlExplorerHostIslandWindow") == 0 ||
     _wcsicmp(lpClassName, L"Shell_InputSwitchTopLevelWindow") == 0)) {
    Wh_Log(L"Initializing - Created XAML host window: %08X via %S",
           (DWORD)(ULONG_PTR)hWnd, funcName);
    InitializeForCurrentThread();
```

```
      InitializeSettingsAndTap();
      return;
   }
}

using CreateWindowExW_t = decltype(&CreateWindowExW);
CreateWindowExW_t CreateWindowExW_Original;
HWND WINAPI CreateWindowExW_Hook(DWORD dwExStyle,
                        LPCWSTR lpClassName,
                        LPCWSTR lpWindowName,
                        DWORD dwStyle,
                        int X,
                        int Y,
                        int nWidth,
                        int nHeight,
                        HWND hWndParent,
                        HMENU hMenu,
                        HINSTANCE hInstance,
                        PVOID lpParam) {
   HWND hWnd = CreateWindowExW_Original(dwExStyle, lpClassName, lpWindowName,
                           dwStyle, X, Y, nWidth, nHeight,
                           hWndParent, hMenu, hInstance, lpParam);
   if (!hWnd) {
      return hWnd;
   }

   OnWindowCreated(hWnd, hWndParent, lpClassName, __FUNCTION__);

   return hWnd;
}

using CreateWindowInBand_t = HWND(WINAPI*)(DWORD dwExStyle,
                              LPCWSTR lpClassName,
                              LPCWSTR lpWindowName,
                              DWORD dwStyle,
                              int X,
                              int Y,
                              int nWidth,
                              int nHeight,
                              HWND hWndParent,
                              HMENU hMenu,
                              HINSTANCE hInstance,
                              PVOID lpParam,
                              DWORD dwBand);
CreateWindowInBand_t CreateWindowInBand_Original;
HWND WINAPI CreateWindowInBand_Hook(DWORD dwExStyle,
                           LPCWSTR lpClassName,
                           LPCWSTR lpWindowName,
                           DWORD dwStyle,
                           int X,
                           int Y,
                           int nWidth,
                           int nHeight,
                           HWND hWndParent,
                           HMENU hMenu,
```

```
                        HINSTANCE hInstance,
                        PVOID lpParam,
                        DWORD dwBand) {
    HWND hWnd = CreateWindowInBand_Original(
        dwExStyle, lpClassName, lpWindowName, dwStyle, X, Y, nWidth, nHeight,
        hWndParent, hMenu, hInstance, lpParam, dwBand);
    if (!hWnd) {
        return hWnd;
    }

    OnWindowCreated(hWnd, hWndParent, lpClassName, __FUNCTION__);

    return hWnd;
}

using CreateWindowInBandEx_t = HWND(WINAPI*)(DWORD dwExStyle,
                                LPCWSTR lpClassName,
                                LPCWSTR lpWindowName,
                                DWORD dwStyle,
                                int X,
                                int Y,
                                int nWidth,
                                int nHeight,
                                HWND hWndParent,
                                HMENU hMenu,
                                HINSTANCE hInstance,
                                PVOID lpParam,
                                DWORD dwBand,
                                DWORD dwTypeFlags);
CreateWindowInBandEx_t CreateWindowInBandEx_Original;
HWND WINAPI CreateWindowInBandEx_Hook(DWORD dwExStyle,
                        LPCWSTR lpClassName,
                        LPCWSTR lpWindowName,
                        DWORD dwStyle,
                        int X,
                        int Y,
                        int nWidth,
                        int nHeight,
                        HWND hWndParent,
                        HMENU hMenu,
                        HINSTANCE hInstance,
                        PVOID lpParam,
                        DWORD dwBand,
                        DWORD dwTypeFlags) {
    HWND hWnd = CreateWindowInBandEx_Original(
        dwExStyle, lpClassName, lpWindowName, dwStyle, X, Y, nWidth, nHeight,
        hWndParent, hMenu, hInstance, lpParam, dwBand, dwTypeFlags);
    if (!hWnd) {
        return hWnd;
    }

    OnWindowCreated(hWnd, hWndParent, lpClassName, __FUNCTION__);

    return hWnd;
}
```

```
PFN_INITIALIZE_XAML_DIAGNOSTICS_EX InitializeXamlDiagnosticsEx_Original;
HRESULT WINAPI
InitializeXamlDiagnosticsEx_Hook(_In_ PCWSTR endPointName,
                                 _In_ DWORD pid,
                                 _In_ PCWSTR wszDllXamlDiagnostics,
                                 _In_ PCWSTR wszTAPDllName,
                                 _In_ CLSID tapClsid,
                                 _In_opt_ PCWSTR wszInitializationData) {
  if (g_inInjectWindhawkTAP) {
    return InitializeXamlDiagnosticsEx_Original(
      endPointName, pid, wszDllXamlDiagnostics, wszTAPDllName, tapClsid,
      wszInitializationData);
  }

  bool blockCall = false;

  switch (g_settings.xamlDiagnosticsHandling) {
    case XamlDiagnosticsHandling::kAlert: {
      void* retAddress = __builtin_return_address(0);

      WCHAR modulePath[MAX_PATH];
      PCWSTR modulePathStr = L"<unknown>";
      HMODULE module;
      if (GetModuleHandleEx(
            GET_MODULE_HANDLE_EX_FLAG_FROM_ADDRESS |
              GET_MODULE_HANDLE_EX_FLAG_UNCHANGED_REFCOUNT,
            reinterpret_cast<LPCWSTR>(retAddress), &module)) {
        switch (GetModuleFileName(module, modulePath,
                                  ARRAYSIZE(modulePath))) {
          case 0:
          case ARRAYSIZE(modulePath):
            break;

          default:
            modulePathStr = modulePath;
            break;
        }
      }

      WCHAR message[1024];
      _snwprintf_s(
        message, _TRUNCATE,
        L"The following module is trying to use XAML diagnostics:\n\n"
        L"%s\n\n"
        L"There can only be one consumer at a time. Blocking it might "
        L"break that module, but allowing it might break this mod.\n\n"
        L"Do you want to block it?\n\n"
        L"Note: You can change this behavior in the mod settings.",
        modulePathStr);
      int result = MessageBox(nullptr, message,
                              L"Windows 11 Taskbar Styler - Windhawk",
                              MB_YESNO | MB_ICONQUESTION | MB_TOPMOST);
      blockCall = (result == IDYES);
      break;
```

```
        }

        case XamlDiagnosticsHandling::kBlock:
          blockCall = true;
          break;

        case XamlDiagnosticsHandling::kAllow:
          blockCall = false;
          break;
      }

      if (blockCall) {
        Wh_Log(L"Blocking InitializeXamlDiagnosticsEx call");
        // Return success to avoid exception in the caller.
        return S_OK;
      }

      Wh_Log(L"Allowing InitializeXamlDiagnosticsEx call");
      return InitializeXamlDiagnosticsEx_Original(
        endPointName, pid, wszDllXamlDiagnostics, wszTAPDllName, tapClsid,
        wszInitializationData);
}

bool HookInitializeXamlDiagnosticsExIfNeeded() {
    if (InitializeXamlDiagnosticsEx_Original) {
        return false;  // Already hooked
    }

    const HMODULE wux = GetModuleHandle(L"Windows.UI.Xaml.dll");
    if (!wux) {
        return false;  // DLL not loaded yet
    }

    const auto ixde = reinterpret_cast<PFN_INITIALIZE_XAML_DIAGNOSTICS_EX>(
        GetProcAddress(wux, "InitializeXamlDiagnosticsEx"));
    if (!ixde) {
        return false;
    }

    Wh_Log(L"Hooking InitializeXamlDiagnosticsEx to handle other consumers");
    return WindhawkUtils::SetFunctionHook(
        ixde, InitializeXamlDiagnosticsEx_Hook,
        &InitializeXamlDiagnosticsEx_Original);
}

using LoadLibraryExW_t = decltype(&LoadLibraryExW);
LoadLibraryExW_t LoadLibraryExW_Original;
HMODULE WINAPI LoadLibraryExW_Hook(LPCWSTR lpLibFileName,
                                   HANDLE hFile,
                                   DWORD dwFlags) {
    HMODULE module = LoadLibraryExW_Original(lpLibFileName, hFile, dwFlags);

    if (module && !InitializeXamlDiagnosticsEx_Original && lpLibFileName) {
        PCWSTR fileName = wcsrchr(lpLibFileName, L'\\');
        fileName = fileName ? fileName + 1 : lpLibFileName;
```

```
    if (_wcsicmp(fileName, L"Windows.UI.Xaml.dll") == 0 &&
        HookInitializeXamlDiagnosticsExIfNeeded()) {
        Wh_ApplyHookOperations();
    }
  }

  return module;
}

using RegOpenKeyExW_t = decltype(&RegOpenKeyExW);
RegOpenKeyExW_t RegOpenKeyExW_Original;
LSTATUS WINAPI RegOpenKeyExW_Hook(HKEY hKey,
                                  LPCWSTR lpSubKey,
                                  DWORD ulOptions,
                                  REGSAM samDesired,
                                  PHKEY phkResult) {
  LSTATUS result = RegOpenKeyExW_Original(hKey, lpSubKey, ulOptions,
                                          samDesired, phkResult);
  if (result == ERROR_SUCCESS || !g_reportCompositionDiagAsDisabled ||
      hKey != HKEY_LOCAL_MACHINE || !lpSubKey ||
      _wcsicmp(lpSubKey, L"Software\\Microsoft\\XAML\\Debug") != 0) {
      return result;
  }

  // The key usually doesn't exist, and the value isn't queried unless the key
  // could be opened, so hand out a key which does exist.
  Wh_Log(L"Substituting the XAML debug key");
  return RegOpenKeyExW_Original(HKEY_LOCAL_MACHINE, L"Software\\Microsoft",
                                ulOptions, samDesired, phkResult);
}

using RegQueryValueExW_t = decltype(&RegQueryValueExW);
RegQueryValueExW_t RegQueryValueExW_Original;
LSTATUS WINAPI RegQueryValueExW_Hook(HKEY hKey,
                                     LPCWSTR lpValueName,
                                     LPDWORD lpReserved,
                                     LPDWORD lpType,
                                     LPBYTE lpData,
                                     LPDWORD lpcbData) {
  if (!g_reportCompositionDiagAsDisabled || !lpValueName ||
      _wcsicmp(lpValueName, L"DisableCompositionDiag") != 0) {
      return RegQueryValueExW_Original(hKey, lpValueName, lpReserved, lpType,
                                       lpData, lpcbData);
  }

  Wh_Log(L"Reporting DisableCompositionDiag as set");

  if (lpType) {
      *lpType = REG_DWORD;
  }

  if (lpData && (!lpcbData || *lpcbData < sizeof(DWORD))) {
      if (lpcbData) {
          *lpcbData = sizeof(DWORD);
      }
```

```
        return ERROR_MORE_DATA;
    }

    if (lpData) {
        *reinterpret_cast<DWORD*>(lpData) = 1;
    }

    if (lpcbData) {
        *lpcbData = sizeof(DWORD);
    }

    return ERROR_SUCCESS;
}

std::vector<HWND> GetXamlHostWnds() {
    struct ENUM_WINDOWS_PARAM {
        std::vector<HWND>* hWnds;
    };

    std::vector<HWND> hWnds;
    ENUM_WINDOWS_PARAM param = {&hWnds};
    EnumWindows(
        [](HWND hWnd, LPARAM lParam) -> BOOL {
            ENUM_WINDOWS_PARAM& param = *(ENUM_WINDOWS_PARAM*)lParam;

            DWORD dwProcessId = 0;
            if (!GetWindowThreadProcessId(hWnd, &dwProcessId) ||
                dwProcessId != GetCurrentProcessId()) {
                return TRUE;
            }

            WCHAR szClassName[32];
            if (GetClassName(hWnd, szClassName, ARRAYSIZE(szClassName)) == 0) {
                return TRUE;
            }

            if (_wcsicmp(szClassName, L"XamlExplorerHostIslandWindow") == 0 ||
                _wcsicmp(szClassName, L"Shell_InputSwitchTopLevelWindow") ==
                    0) {
                param.hWnds->push_back(hWnd);
            }

            return TRUE;
        },
        (LPARAM)&param);

    return hWnds;
}

HWND FindCurrentProcessTaskbarWnd() {
    HWND hTaskbarWnd = nullptr;

    EnumWindows(
        [](HWND hWnd, LPARAM lParam) -> BOOL {
            DWORD dwProcessId;
```

```
            WCHAR className[32];
            if (GetWindowThreadProcessId(hWnd, &dwProcessId) &&
                dwProcessId == GetCurrentProcessId() &&
                GetClassName(hWnd, className, ARRAYSIZE(className)) &&
                _wcsicmp(className, L"Shell_TrayWnd") == 0) {
                *reinterpret_cast<HWND*>(lParam) = hWnd;
                return FALSE;
            }
            return TRUE;
        },
        reinterpret_cast<LPARAM>(&hTaskbarWnd));

    return hTaskbarWnd;
}

HWND GetTaskbarUiWnd() {
    HWND hTaskbarWnd = FindCurrentProcessTaskbarWnd();
    if (!hTaskbarWnd) {
        return nullptr;
    }

    return FindWindowEx(hTaskbarWnd, nullptr,
                        L"Windows.UI.Composition.DesktopWindowContentBridge",
                        nullptr);
}

PTP_TIMER g_statsTimer;

bool StartStatsTimer() {
    static constexpr WCHAR kStatsBaseUrl[] =
        L"https://github.com/ramensoftware/"
        L"windows-11-taskbar-styling-guide/"
        L"releases/download/stats-v6/";

    ULONGLONG lastStatsTime = 0;
    Wh_GetBinaryValue(L"statsTimerLastTime", &lastStatsTime,
                      sizeof(lastStatsTime));

    // -1 can be set for disabling the stats timer.
    if (lastStatsTime == 0xFFFFFFFF'FFFFFFFF) {
        return false;
    }

    FILETIME currentTimeFt;
    GetSystemTimeAsFileTime(&currentTimeFt);

    ULONGLONG currentTime = ((ULONGLONG)currentTimeFt.dwHighDateTime << 32) |
                            currentTimeFt.dwLowDateTime;

    constexpr ULONGLONG k10Minutes = 10 * 60 * 10000000LL;
    constexpr ULONGLONG k24Hours = 24 * 60 * 60 * 10000000LL;

    ULONGLONG minDueTime = currentTime + k10Minutes;
    ULONGLONG maxDueTime = currentTime + k24Hours;
```

```
ULONGLONG dueTime = lastStatsTime + k24Hours;
if (dueTime < minDueTime) {
   dueTime = minDueTime;
} else if (dueTime > maxDueTime) {
   dueTime = maxDueTime;
}

g_statsTimer = CreateThreadpoolTimer(
   [](PTP_CALLBACK_INSTANCE, PVOID, PTP_TIMER) {
      Wh_Log(L">");

      string_setting_unique_ptr themeName(Wh_GetStringSetting(L"theme"));
      if (!*themeName.get()) {
         return;
      }

      HANDLE mutex =
         CreateMutex(nullptr, FALSE, L"WindhawkStats_" WH_MOD_ID);
      if (mutex) {
         WaitForSingleObject(mutex, INFINITE);
      }

      ULONGLONG lastStatsTime = 0;
      Wh_GetBinaryValue(L"statsTimerLastTime", &lastStatsTime,
                        sizeof(lastStatsTime));

      FILETIME currentTimeFt;
      GetSystemTimeAsFileTime(&currentTimeFt);
      ULONGLONG currentTime =
         ((ULONGLONG)currentTimeFt.dwHighDateTime << 32) |
         currentTimeFt.dwLowDateTime;

      const WH_URL_CONTENT* content = nullptr;
      if (currentTime - lastStatsTime >= k10Minutes) {
         Wh_SetBinaryValue(L"statsTimerLastTime", &currentTime,
                           sizeof(currentTime));

         std::wstring themeNameEscaped = themeName.get();
         std::replace(themeNameEscaped.begin(), themeNameEscaped.end(),
                      L' ', L'_');
         std::replace(themeNameEscaped.begin(), themeNameEscaped.end(),
                      L'&', L'_');
         std::replace(themeNameEscaped.begin(), themeNameEscaped.end(),
                      L'.', L'_');

         std::wstring statsUrl = kStatsBaseUrl;
         statsUrl += themeNameEscaped;
         statsUrl += L".txt";

         Wh_Log(L"Submitting stats to %s", statsUrl.c_str());

         content = Wh_GetUrlContent(statsUrl.c_str(), nullptr);
      } else {
         Wh_Log(L"Skipping, last submission %llu seconds ago",
                (currentTime - lastStatsTime) / 10000000LL);
```

```
        }

        if (mutex) {
            ReleaseMutex(mutex);
            CloseHandle(mutex);
        }

        if (!content) {
            Wh_Log(L"Failed to get stats content");
            return;
        }

        if (content->statusCode != 200) {
            Wh_Log(L"Stats content status code: %d", content->statusCode);
        }

        Wh_FreeUrlContent(content);
        Wh_Log(L"Stats content submitted");
      },
      nullptr, nullptr);
  if (!g_statsTimer) {
      Wh_Log(L"Failed to create stats timer");
      return false;
  }

  constexpr DWORD k24HoursInMs = 24 * 60 * 60 * 1000;
  constexpr ULONGLONG k10MinutesInMs = 10 * 60 * 1000;

  FILETIME dueTimeFt;
  dueTimeFt.dwLowDateTime = (DWORD)(dueTime & 0xFFFFFFFF);
  dueTimeFt.dwHighDateTime = (DWORD)(dueTime >> 32);
  SetThreadpoolTimer(g_statsTimer, &dueTimeFt, k24HoursInMs, k10MinutesInMs);
  return true;
}

void StopStatsTimer() {
  if (g_statsTimer) {
      SetThreadpoolTimer(g_statsTimer, nullptr, 0, 0);
      WaitForThreadpoolTimerCallbacks(g_statsTimer, TRUE);
      CloseThreadpoolTimer(g_statsTimer);
      g_statsTimer = nullptr;
  }
}

void LoadSettings() {
  g_settings.clickThroughTaskbar = Wh_GetIntSetting(L"clickThroughTaskbar");

  PCWSTR xamlDiagnosticsHandling =
      Wh_GetStringSetting(L"xamlDiagnosticsHandling");
  g_settings.xamlDiagnosticsHandling = XamlDiagnosticsHandling::kAlert;
  if (wcscmp(xamlDiagnosticsHandling, L"block") == 0) {
      g_settings.xamlDiagnosticsHandling = XamlDiagnosticsHandling::kBlock;
  } else if (wcscmp(xamlDiagnosticsHandling, L"allow") == 0) {
      g_settings.xamlDiagnosticsHandling = XamlDiagnosticsHandling::kAllow;
  }
```

```
    Wh_FreeStringSetting(xamlDiagnosticsHandling);
}

BOOL Wh_ModInit() {
    Wh_Log(L">");

    LoadSettings();

    WindhawkUtils::SetFunctionHook(CreateWindowExW, CreateWindowExW_Hook,
                                   &CreateWindowExW_Original);

    HMODULE user32Module =
        LoadLibraryEx(L"user32.dll", nullptr, LOAD_LIBRARY_SEARCH_SYSTEM32);
    if (user32Module) {
        auto pCreateWindowInBand = (CreateWindowInBand_t)GetProcAddress(
            user32Module, "CreateWindowInBand");
        if (pCreateWindowInBand) {
            WindhawkUtils::SetFunctionHook(pCreateWindowInBand,
                                           CreateWindowInBand_Hook,
                                           &CreateWindowInBand_Original);
        }

        auto pCreateWindowInBandEx = (CreateWindowInBandEx_t)GetProcAddress(
            user32Module, "CreateWindowInBandEx");
        if (pCreateWindowInBandEx) {
            WindhawkUtils::SetFunctionHook(pCreateWindowInBandEx,
                                           CreateWindowInBandEx_Hook,
                                           &CreateWindowInBandEx_Original);
        }
    }

    HMODULE kernelBaseModule = GetModuleHandle(L"kernelbase.dll");
    auto pKernelBaseLoadLibraryExW = (decltype(&LoadLibraryExW))GetProcAddress(
        kernelBaseModule, "LoadLibraryExW");
    WindhawkUtils::SetFunctionHook(pKernelBaseLoadLibraryExW,
                                   LoadLibraryExW_Hook,
                                   &LoadLibraryExW_Original);

    auto pKernelBaseRegOpenKeyExW =
        (RegOpenKeyExW_t)GetProcAddress(kernelBaseModule, "RegOpenKeyExW");
    WindhawkUtils::SetFunctionHook(pKernelBaseRegOpenKeyExW, RegOpenKeyExW_Hook,
                                   &RegOpenKeyExW_Original);

    auto pKernelBaseRegQueryValueExW = (RegQueryValueExW_t)GetProcAddress(
        kernelBaseModule, "RegQueryValueExW");
    WindhawkUtils::SetFunctionHook(pKernelBaseRegQueryValueExW,
                                   RegQueryValueExW_Hook,
                                   &RegQueryValueExW_Original);

    // Hook immediately if DLL is already loaded.
    HookInitializeXamlDiagnosticsExIfNeeded();

    StartStatsTimer();

    return TRUE;
```

```
}

void Wh_ModAfterInit() {
    Wh_Log(L">");

    bool initialize = false;

    HWND hTaskbarUiWnd = GetTaskbarUiWnd();
    if (hTaskbarUiWnd) {
        Wh_Log(L"Initializing - Found DesktopWindowContentBridge window");
        RunFromWindowThread(
            hTaskbarUiWnd, [](PVOID) { InitializeForCurrentThread(); },
            nullptr);
        initialize = true;
    }

    for (auto hXamlHostWnd : GetXamlHostWnds()) {
        Wh_Log(L"Initializing for %08X", (DWORD)(ULONG_PTR)hXamlHostWnd);
        RunFromWindowThread(
            hXamlHostWnd, [](PVOID) { InitializeForCurrentThread(); }, nullptr);
        initialize = true;
    }

    if (initialize) {
        InitializeSettingsAndTap();
    }
}

void Wh_ModUninit() {
    Wh_Log(L">");

    HWND restartExplorerPromptWindow = g_restartExplorerPromptWindow;
    if (restartExplorerPromptWindow) {
        PostMessage(restartExplorerPromptWindow, WM_CLOSE, 0, 0);
    }

    if (g_restartExplorerPromptThread) {
        WaitForSingleObject(g_restartExplorerPromptThread, INFINITE);
        CloseHandle(g_restartExplorerPromptThread);
        g_restartExplorerPromptThread = nullptr;
    }

    StopStatsTimer();

    StopImageDownloads();

    // Before the UI threads are uninitialized, so that a retry can't be
    // scheduled on a thread which is being uninitialized.
    StopImageLoadRetries();

    UninitializeSettingsAndTap();

    HWND hTaskbarUiWnd = GetTaskbarUiWnd();
    if (hTaskbarUiWnd) {
        Wh_Log(L"Uninitializing - Found DesktopWindowContentBridge window");
```

```
        RunFromWindowThread(
            hTaskbarUiWnd, [](PVOID) { UninitializeForCurrentThread(); },
            nullptr);
    }

    for (auto hXamlHostWnd : GetXamlHostWnds()) {
        Wh_Log(L"Uninitializing for %08X", (DWORD)(ULONG_PTR)hXamlHostWnd);
        RunFromWindowThread(
            hXamlHostWnd, [](PVOID) { UninitializeForCurrentThread(); },
            nullptr);
    }
}

void Wh_ModSettingsChanged() {
    Wh_Log(L">");

    UninitializeSettingsAndTap();

    LoadSettings();

    bool initialize = false;

    HWND hTaskbarUiWnd = GetTaskbarUiWnd();
    if (hTaskbarUiWnd) {
        Wh_Log(L"Reinitializing - Found DesktopWindowContentBridge window");
        RunFromWindowThread(
            hTaskbarUiWnd,
            [](PVOID) {
                UninitializeForCurrentThread();
                InitializeForCurrentThread();
            },
            nullptr);
        initialize = true;
    }

    for (auto hXamlHostWnd : GetXamlHostWnds()) {
        Wh_Log(L"Reinitializing for %08X", (DWORD)(ULONG_PTR)hXamlHostWnd);
        RunFromWindowThread(
            hXamlHostWnd,
            [](PVOID) {
                UninitializeForCurrentThread();
                InitializeForCurrentThread();
            },
            nullptr);
        initialize = true;
    }

    if (initialize) {
        InitializeSettingsAndTap();
    }
}
```